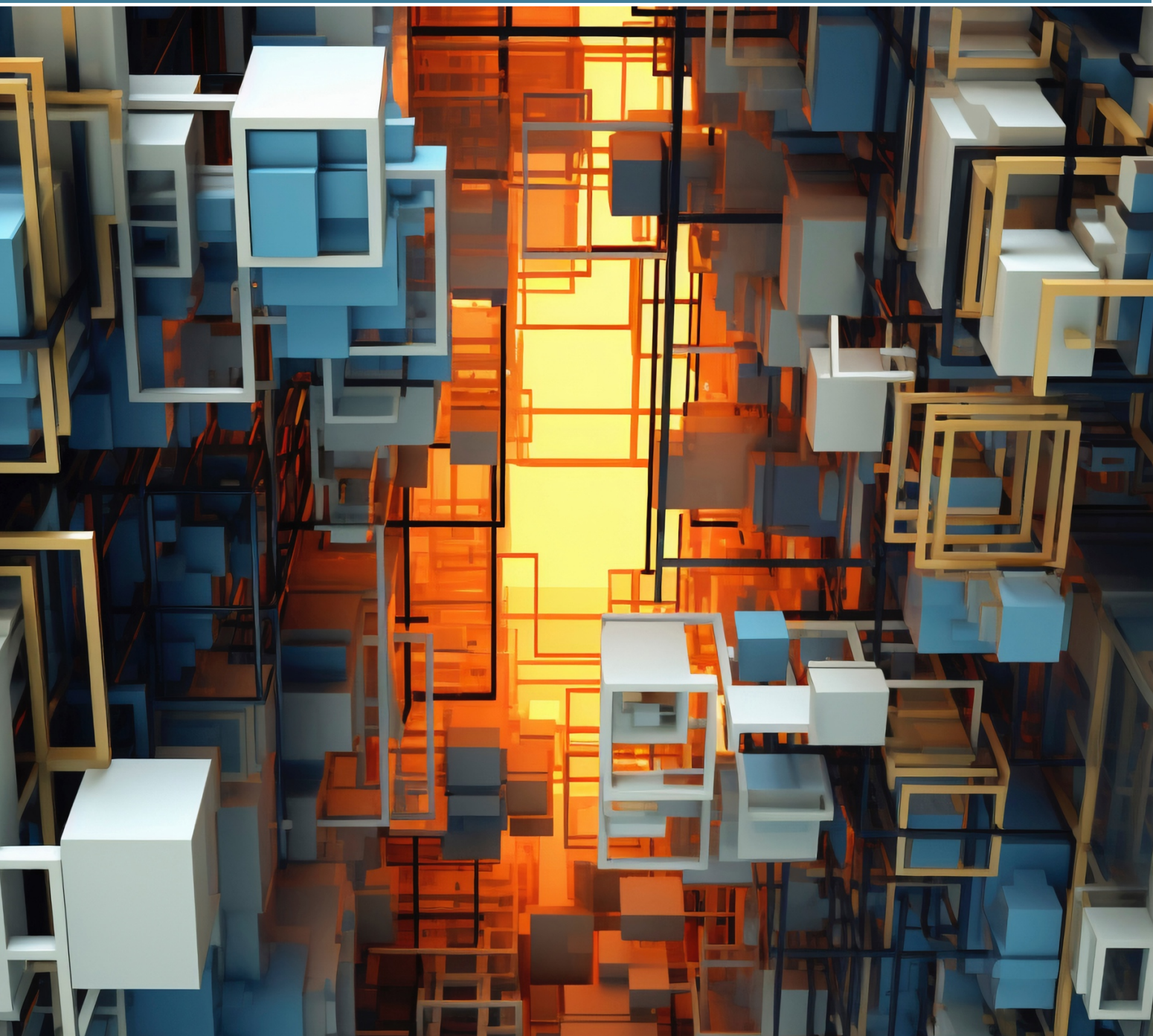


Καρακασίδης Α. Κολωνιάρη Γ. Γούναρης Α. Παπαδόπουλος Α. Ν.

Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων



Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων

Αλέξανδρος Καρακασίδης
ΕΔΙΠ, Πανεπιστήμιο Μακεδονίας

Γεωργία Κολωνιάρη
Αναπληρώτρια Καθηγήτρια, Πανεπιστήμιο Μακεδονίας

Αναστάσιος Γούναρης
Καθηγητής, Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης

Απόστολος Ν. Παπαδόπουλος
Καθηγητής, Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης



ΚΑΛΛΙΠΟΣ
ΑΝΟΙΚΤΕΣ ΑΚΑΔΗΜΑΪΚΕΣ ΕΚΔΟΣΕΙΣ
www.kallipos.gr

Τίτλος πρωτοτύπου: «Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων»
Copyright © 2025, ΚΑΛΛΙΠΟΣ, ΑΝΟΙΚΤΕΣ ΑΚΑΔΗΜΑΪΚΕΣ ΕΚΔΟΣΕΙΣ
(ΣΕΑΒ + ΕΛΚΕ-ΕΜΠ)



Το παρόν έργο διατίθεται με τους όρους της άδειας Creative Commons Αναφορά Δημιουργού – Μη Εμπορική Χρήση – Παρόμοια Διανομή 4.0. Για να δείτε τους όρους της άδειας αυτής επισκεφτείτε τον ιστότοπο <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.el>

Αν τυχόν κάποιο τμήμα του έργου διατίθεται με διαφορετικό καθεστώς αδειοδότησης, αυτό αναφέρεται ρητά και ειδικώς στην οικεία θέση.

Συντελεστές έκδοσης

Γλωσσική επιμέλεια: Κατερίνα Αλεξοπούλου

Τεχνική επεξεργασία: Γ. Κολωνιάρη, Α. Καρακασίδης, Α. Γούναρης, Α. Παπαδόπουλος

Εξώφυλλο: Έλενα-Νατάσα Καΐτσα

Κεντρική Ομάδα Υποστήριξης

Γλωσσικός Έλεγχος: Γεωργία Τριανταφυλλίδου

Γραφιστικός Έλεγχος: Έλενα-Νατάσα Καΐτσα

Βιβλιοθηκονομική Επεξεργασία: Μενέλαος Γουρνιζάκης

ΚΑΛΛΙΠΟΣ

Εθνικό Μετσόβιο Πολυτεχνείο

Ηρώων Πολυτεχνείου 9

15780 Ζωγράφου

www.kallipos.gr

Βιβλιογραφική αναφορά:

Καρακασίδης, Α., Κολωνιάρη, Γ., Γούναρης, Α., & Παπαδόπουλος, Α. (2025).

Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων [Μεταπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις.

Διαθέσιμο στο: <http://dx.doi.org/10.57713/kallipos-1058>

ISBN: 978-618-228-309-7

Στους γονείς μας

*The reasonable man adapts himself to the world.
The unreasonable one persists in trying to adapt the world to himself.
Therefore, all progress depends on the unreasonable man.*

George Bernard Shaw

ΠΡΟΛΟΓΟΣ

Τα τελευταία χρόνια παρατηρείται μια έκρηξη στην παραγωγή δεδομένων από διαφορετικές πηγές με αποτέλεσμα να εμφανίζονται σημαντικές προκλήσεις σχετικές με την αποτελεσματικότητα και την απόδοση των εφαρμογών. Τα μεγάλα δεδομένα (big data) είναι παντού και τα χαρακτηριστικά τους επιβάλλουν διαφορετικές τεχνικές επεξεργασίας και ανάλυσης. Το βιβλίο αυτό αποτελεί μία προσπάθεια μελέτης των τεχνολογικών εξελίξεων στον χώρο. Το βιβλίο μπορεί να χρησιμοποιηθεί ως βασικό υλικό για μεταπτυχιακά μαθήματα, αλλά τμήματά του θα μπορούσαν να χρησιμοποιηθούν και σε μαθήματα προπτυχιακού επιπέδου.

Η Δομή του Βιβλίου

Το βιβλίο αποτελείται από έντεκα (11) κεφάλαια και ένα (1) παράρτημα. Αναλυτικότερα, το Κεφάλαιο 1 εισάγει τον αναγνώστη στον χώρο των μεγάλων δεδομένων περιγράφοντας τα βασικά χαρακτηριστικά και τις προκλήσεις. Το Κεφάλαιο 2 εστιάζεται σε κατακευματισμένα συστήματα αρχείων που αποτελούν τη βάση για την αποθήκευση των δεδομένων σε συστάδες υπολογιστών. Το Κεφάλαιο 3 μελετά το προγραμματιστικό μοντέλο MapReduce και πώς αυτό υποστηρίζεται από το σύστημα Hadoop. Το Κεφάλαιο 4 εστιάζεται στα συστήματα βάσεων δεδομένων NOSQL τα οποία διαφοροποιούνται σημαντικά σε σχέση με τα παραδοσιακά σχεσιακά συστήματα. Το Κεφάλαιο 5 αποτελεί μία εισαγωγή στο σύστημα Apache Spark που στοχεύει στην αποδοτική ανάλυση μεγάλων δεδομένων, ενώ το Κεφάλαιο 6 εισέρχεται σε μεγαλύτερο βάθος και μελετά εξειδικευμένες λειτουργίες του συστήματος. Το Κεφάλαιο 7 περιγράφει συστήματα για την ανάλυση ροών δεδομένων ενώ το Κεφάλαιο 8 εστιάζεται στην ανάλυση δεδομένων δικτύων (γραφημάτων) τα οποία χρησιμοποιούνται από πολλές εφαρμογές και έχουν σημαντικές απαιτήσεις. Το Κεφάλαιο 9 περιγράφει τεχνικές διαχείρισης πόρων και εκτέλεσης εφαρμογών. Το Κεφάλαιο 10 στοχεύει στην επεξεργασία μεγάλων δεδομένων με χρήση GPUs, ενώ το Κεφάλαιο 11 μελετά αλγοριθμικά θέματα ανάλυσης μεγάλων δεδομένων. Στο βιβλίο συμπεριλαμβάνεται και ένα παράρτημα, το οποίο περιέχει χρήσιμες οδηγίες για την εγκατάσταση και τη χρήση του συστήματος Apache Hadoop.

Οι βιβλιογραφικές αναφορές βρίσκονται στο τέλος κάθε κεφαλαίου αλλά είναι επίσης συ-

γκεντρωμένες και στο τέλος του βιβλίου για μεγαλύτερη ευκολία κατά την ανάγνωση. Αναφέρουμε επίσης ότι αν δεν σημειώνεται διαφορετικά, όλα τα σχήματα προέρχονται από το προσωπικό αρχείο των συγγραφέων.

Διορθώσεις και Υποδείξεις

Κάθε βιβλίο αναπόφευκτα θα περιέχει κάποια λάθη. Παροτρύνουμε τους αναγνώστες να επικοινωνήσουν μαζί μας για οποιοδήποτε λάθος που τυχόν υπάρχει στο σύνολο του κειμένου, καθώς θεωρούμε ότι οι υποδείξεις των αναγνωστών είναι πολύ σημαντικές για τη βελτίωση του βιβλίου σε επόμενες εκδόσεις.

Ευχαριστίες

Το βιβλίο αυτό δεν θα γινόταν πραγματικότητα χωρίς τη συμπαράσταση μερικών ανθρώπων. Αρχικά θα θέλαμε να ευχαριστήσουμε όλους τους ανθρώπους του Κάλλιπου για τη συμπαράσταση και την καθοδήγησή τους σε όλη τη διάρκεια της συγγραφής του παρόντος.

Ευχαριστούμε επίσης θερμά τους Απόστολο Γιαννουλίδη, Γεωργία Κούγκα, Δημήτριο Μάρκου, Ιωάννη Μαυρουδόπουλο και Χρήστο Μπέλλα για την πολύτιμη βοήθειά τους.

Θεσσαλονίκη,
Οκτώβριος 2024

Αλέξανδρος Καρακασίδης
Γεωργία Κολωνiάρη
Αναστάσιος Γούναρης
Απόστολος Ν. Παπαδόπουλος

ΠΕΡΙΕΧΟΜΕΝΑ

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

xvii

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

xix

1	Εισαγωγή στα Μεγάλα Δεδομένα	1
1.1	Τι είναι τα μεγάλα δεδομένα	2
1.2	Ιστορική αναδρομή	6
1.2.1	Η στοίβα λογισμικού της Google	7
1.2.2	Το οικοσύστημα του Hadoop	8
1.3	Μεγάλα δεδομένα και υπολογιστικό νέφος	10
1.4	Αναλυτική μεγάλων δεδομένων	11
1.5	Εφαρμογές ανάλυσης μεγάλων δεδομένων	13
1.5.1	Μεγάλα δεδομένα στην υγεία	13
1.5.2	Μεγάλα δεδομένα στο ηλεκτρονικό εμπόριο και στα συστήματα συστάσεων	14
1.5.3	Μεγάλα δεδομένα και το Διαδίκτυο των Πραγμάτων (IoT)	15
1.5.4	Μεγάλα δεδομένα στην εκπαίδευση	17
1.6	Περαιτέρω μελέτη	18
	BIBΛΙΟΓΡΑΦΙΑ	18
2	Καταναμημένα Συστήματα Αρχείων	23
2.1	Εισαγωγή	24
2.2	Λύσεις ανοικτού κώδικα	24
2.2.1	GlusterFS	25
2.2.1.1	Αρχιτεκτονική GlusterFS	25
	Καταναμημένος τόμος GlusterFS.	26
	Αναπαραχθείς τόμος GlusterFS.	26
	Διάχυτος τόμος GlusterFS.	26
	Καταναμημένος αναπαραχθείς τόμος GlusterFS.	26
	Καταναμημένος διάχυτος τόμος GlusterFS.	26
	Μεταφραστές.	26
2.2.1.2	Λειτουργία του GlusterFS	27
2.2.2	Ceph	27
2.2.2.1	Αρχιτεκτονική	27
2.2.2.2	Λειτουργία	28
	Αποθήκευση δεδομένων.	28
2.2.3	Alluxio	29
2.2.3.1	Αρχιτεκτονική	29
	Masters.	29
	Workers.	29
	Client.	30
2.2.3.2	Λειτουργία	30
	Ανάγνωση.	30
	Εγγραφή.	30
2.3	Λύσεις αποθήκευσης νέφους	31
2.3.1	OpenStack Swift	31
2.3.2	Συνδρομητικές υπηρεσίες	32
2.3.2.1	Amazon S3	32
2.3.2.2	Google Cloud	33
2.3.2.3	Windows Azure Storage	33
2.4	Apache Hadoop Distributed File System	33
2.4.1	Αρχιτεκτονική του Apache Hadoop	34

2.4.2	HDFS: το κατανεμημένο σύστημα αρχείων του Hadoop	34
2.4.2.1	Χαρακτηριστικά του HDFS	35
	Αστοχία υλικού.	35
	Πρόσβαση σε δεδομένα μεγάλης κλίμακας με μορφή ροής.	35
2.4.3	Έννοιες και οντότητες του HDFS	36
2.4.3.1	NameNode	36
2.4.3.2	Secondary NameNode	38
2.4.3.3	DataNode	38
2.4.3.4	Πελάτης HDFS	39
2.4.3.5	Τα blocks στο HDFS	39
2.4.4	Λειτουργίες αρχείων και διαχείριση blocks	42
2.4.5	Εντολές στο HDFS	43
2.5	Συζήτηση	44
2.6	Περαιτέρω μελέτη	45
	BIBΛΙΟΓΡΑΦΙΑ	46
3	Εισαγωγή στο MapReduce	49
3.1	Εισαγωγή	50
	Οριζόντια έναντι κατακόρυφης κλιμάκωσης	50
	Οι αστοχίες είναι σύνηθες φαινόμενο.	51
	Μετακινούμε τον υπολογισμό, όχι τα δεδομένα.	51
	Σειριακή επεξεργασία με τη μορφή ροής.	51
	Απόκρυψη του συστήματος.	51
3.2	Κατανοώντας το MapReduce	51
3.3	Μείξη και ταξινόμηση	57
3.4	Ροή δεδομένων	58
3.5	Ειδικά θέματα του MapReduce	59
3.5.1	Συνδυαστές και διαμεριστές	59
3.5.2	Συνδυαστές	59
3.5.3	Διαμεριστές	61
3.5.4	Δευτερεύουσα ταξινόμηση	61
3.5.5	Αλυσίδες εργασιών MapReduce	62
3.5.6	Μη ανάγκη χρήσης reducer	62
3.6	Σχεδιασμός αλγορίθμων MapReduce	63
3.6.1	Περιγραφή του προβλήματος	63
3.6.2	Σχεδιασμός αλγορίθμου	63
3.6.3	Υλοποίηση σε Java	65
3.6.4	Εκτέλεση υλοποίησης Java	67
3.6.5	Τροποποίηση για χρήση συνδυαστή	68
3.6.6	Υλοποίηση σε Python	69
3.6.7	Εκτέλεση υλοποίησης σε Python	71
3.6.8	Πλήθος διεργασιών map και reduce	72
3.7	Συζήτηση	73
3.8	Περαιτέρω μελέτη	73
	BIBΛΙΟΓΡΑΦΙΑ	73
4	Συστήματα NoSQL	75
4.1	NoSQL βάσεις δεδομένων	77
4.1.1	Αρχές σχεδίασης	78
4.1.1.1	Συστήματα διαχείρισης δεδομένων χωρίς σχήμα	79
4.1.1.2	Ενσωμάτωση δεδομένων	80
4.1.2	Διαμέριση και αναπαραγωγή δεδομένων	84
4.1.2.1	Sharding	84
4.1.2.2	Αναπαραγωγή δεδομένων	86
4.1.3	Συνέπεια δεδομένων	86
4.2	Key-value stores	89
4.2.1	Παραδείγματα key-value stores	90
4.3	Column-family stores	92
4.3.1	Παραδείγματα column-family Stores	93
4.4	Βάσεις δεδομένων εγγράφων	96
4.4.1	Παραδείγματα βάσεων δεδομένων εγγράφων	96

4.5	Βάσεις δεδομένων γραφημάτων	103
4.5.1	Παραδείγματα βάσεων δεδομένων γραφημάτων	105
4.6	Συστήματα διαχείρισης ετερογενών δεδομένων	114
4.6.1	Multi-model database	115
4.6.2	Polystores	115
4.6.3	Σύγκριση	117
4.7	Περαιτέρω μελέτη	117
	BIBΛΙΟΓΡΑΦΙΑ	118
5	Εισαγωγή στο Σύστημα Apache Spark	121
5.1	Εισαγωγή	122
5.2	Η Μεγάλη εικόνα	123
5.3	RDDs, DataFrames και DataSets	124
5.4	Διεργασία οδήγησης, εκτελεστές και διαμερίσεις	125
5.5	Εγκατάσταση και χρήση	128
5.5.1	Εφαρμογές Spark με τη γλώσσα Scala	131
5.5.2	Εφαρμογές Spark με τη γλώσσα Python	137
5.6	Μετασχηματισμοί και ενέργειες	139
5.6.1	Μετασχηματισμοί	139
5.6.2	Ενέργειες	144
5.7	Επίβλεψη εκτέλεσης εφαρμογών Spark	145
5.8	Περαιτέρω μελέτη	147
	BIBΛΙΟΓΡΑΦΙΑ	147
6	Προχωρημένα Θέματα Apache Spark	149
6.1	Εισαγωγή	150
6.2	Βελτίωση επίδοσης	150
6.2.1	Διατήρηση δεδομένων	150
6.2.2	Μετάδοση δεδομένων	154
6.2.3	Συσσώρευση δεδομένων	156
6.3	Επεξεργασία δομημένων δεδομένων	158
6.4	Εξόρυξη δεδομένων και ανακάλυψη γνώσης	164
6.5	Περαιτέρω μελέτη	168
	BIBΛΙΟΓΡΑΦΙΑ	169
7	Επεξεργασία Ροών Δεδομένων	171
7.1	Εισαγωγή	172
7.2	Συστήματα επεξεργασίας ροών δεδομένων	174
7.2.1	Apache Kafka	174
7.2.2	Apache Spark	178
7.2.3	Apache Flink	179
7.2.4	Apache Storm	181
7.2.5	Amazon Kinesis	185
7.3	Περαιτέρω μελέτη	188
	BIBΛΙΟΓΡΑΦΙΑ	188
8	Ανάλυση Δικτύων	191
8.1	Εισαγωγή	192
8.2	Συστήματα	194
8.2.1	Συστήματα NoSQL	194
8.2.1.1	ArangoDB	194
8.2.1.2	JanusGraph	196
8.2.2	Συστήματα μεγάλης κλίμακας	198
8.2.2.1	Apache Giraph	198
8.2.2.2	Apache Spark	200
8.3	Περαιτέρω μελέτη	206
	BIBΛΙΟΓΡΑΦΙΑ	207
9	Διαχείριση Πόρων και Εκτέλεση Εφαρμογών	209
9.1	Εισαγωγή	210
9.2	Γενικά για τη διαχείριση πόρων	210
9.3	Hadoop YARN	211

9.3.1	Οντότητες του YARN	211
9.3.1.1	ResourceManager	212
	Χρονοπρογραμματιστής.	212
	Διαχειριστής εφαρμογών.	212
9.3.1.2	NodeManager	212
9.3.1.3	ApplicationMaster	213
9.3.2	Λειτουργία του YARN	213
9.3.2.1	Διαχείριση πόρων στο YARN	213
9.3.3	Χρονοπρογραμματισμός στο YARN	215
9.3.3.1	Ο μηχανισμός χρονοπρογραμματισμού FIFO	215
9.3.3.2	Ο μηχανισμός χρονοπρογραμματισμού χωρητικότητας	215
9.3.3.3	Ο δίκαιος μηχανισμός χρονοπρογραμματισμού	215
9.3.4	Βασική διαχείριση του YARN	215
9.3.5	Βασική διαχείριση κόμβων	216
	Πληροφορίες κόμβων του YARN	216
	Εκκίνηση και τερματισμός μεμονωμένων κόμβων.	217
9.3.6	Βασική διαχείριση υπηρεσιών	218
9.3.7	Βασική διαχείριση εφαρμογών	218
9.4	Apache Mesos	220
9.4.1	Οντότητες του Mesos	220
9.4.1.1	Οι masters στο Mesos	220
9.4.1.2	Οι slaves	221
9.4.1.3	Τα frameworks	221
9.4.1.4	Οι executors	222
9.4.2	Λειτουργία του Mesos	222
9.4.2.1	Ανοχή σε σφάλματα	224
9.4.2.2	Υψηλή διαθεσιμότητα	224
9.5	Kubernetes	224
9.5.1	Έννοιες του Kubernetes	225
9.5.2	Workloads	225
9.5.3	Άκατοι	225
	9.5.3.1 Εκτέλεση	226
	9.5.3.2 Συνδεσιμότητα	226
	9.5.3.3 Μόνιμη αποθήκευση	226
9.5.4	Κόμβοι	227
9.5.5	Συστατικά στοιχεία του Kubernetes	227
	9.5.5.1 Στοιχεία του επιπέδου ελέγχου	227
	9.5.5.2 Στοιχεία των κόμβων	228
9.5.6	Εκτέλεση εφαρμογής στο Kubernetes	229
9.6	Spark standalone cluster manager	230
9.6.1	Οντότητες του Apache Spark	230
	9.6.1.1 Ο driver	230
	9.6.1.2 Ο master	230
	Διαμόρφωση συστάδας.	230
	Διαμόρφωση πελάτη.	230
	Τοπική διαμόρφωση.	231
	9.6.1.3 Ο worker	231
	9.6.1.4 Executors	231
9.6.2	Λειτουργία του διαχειριστή συστάδας του Spark	231
9.7	Περαιτέρω μελέτη	232
	BIBΛΙΟΓΡΑΦΙΑ	233
10	Παραλληλισμός με MapReduce σε GPUs	235
10.1	Αποδοτικοί αλγόριθμοι στο MapReduce	237
10.1.1	Πολυπλοκότητα αλγορίθμων MapReduce	241
10.1.2	Κατώτερα όρια στον βαθμό αντιγραφής	243
10.1.3	Παραδείγματα εφαρμογής της μεθοδολογίας ανάλυσης	244
10.2	Θέματα αρχιτεκτονικής και προγραμματιστικού μοντέλου CUDA	247
10.2.1	Οργάνωση νημάτων	248
10.2.2	Ιεραρχίες μνήμης	250
10.2.3	Περισσότερα για το SIMT μοντέλο	251
10.3	Παραδείγματα διαχείρισης δεδομένων σε GPUs	252
10.3.1	Πολλαπλασιασμός πινάκων	253

10.3.2	Επεξεργασία πινάκων και εξαγωγή στατιστικών	255
10.3.3	Συχνές λανθασμένες αντιλήψεις	258
10.4	Περαιτέρω μελέτη	260
BIBΛΙΟΓΡΑΦΙΑ		260
11	Αλγοριθμικά Θέματα Ανάλυσης Μεγάλων Δεδομένων	263
11.1	Εισαγωγή	265
11.2	Υπολογισμός ομοιότητας	265
11.2.1	Ομοιότητα εγγράφων - minhashing	266
11.2.2	Ομοιότητα εγγράφων - υπολογισμός	268
11.2.3	Μετρικές απόστασης	271
11.3	Επεξεργασία ροών	272
11.3.1	Αντιπροσωπευτικά δείγματα και φίλτρα Bloom	272
11.3.2	Μέτρηση διακριτών στοιχείων	274
11.4	Μηχανές αναζήτησης	275
11.4.1	Ο αλγόριθμος PageRank	276
11.4.2	Επεκτάσεις και θέματα υλοποίησης	279
11.5	Κανόνες συσχέτισης	281
11.5.1	Αλγόριθμος A-Priori	282
11.5.2	Αντιμετωπίζοντας το πρόβλημα της καταμέτρησης ζευγών και η επίλυσή του	285
11.5.3	Περιορισμός περασμάτων	286
11.6	Ομαδοποίηση	287
11.6.1	Ο αλγόριθμος των Bradley, Fayyad και Reina	289
11.6.2	Ο αλγόριθμος CURE	291
11.6.3	Ο αλγόριθμος DBSCAN	292
11.7	Μηχανική μάθηση για μεγάλα δεδομένα	293
11.7.1	Αντίληπτρα (perceptrons)	294
11.7.2	Μηχανές διανυσμάτων υποστήριξης	295
11.7.3	Εκμάθηση από πλησιέστερους γείτονες	296
11.7.4	Δένδρα αποφάσεων	297
11.8	Ανάλυση μεγάλων γραφών κοινωνικών δικτύων	298
11.8.1	Ενδιαμεσότητα (betweenness)	299
11.8.2	SimRank	299
11.8.3	Καταμέτρηση τριγώνων	301
11.9	Συζήτηση και περαιτέρω μελέτη	302
BIBΛΙΟΓΡΑΦΙΑ		303
ΠΑΡΑΡΤΗΜΑ Α	Εγκατάσταση και Χρήση του Apache Hadoop	307
A.1	Εγκατάσταση Hadoop αυτόνομα	308
A.2	Εκτέλεση σε αυτόνομη εγκατάσταση Hadoop	309
A.3	Κατανεμημένη εγκατάσταση Hadoop	309
A.3.0.1	Επεξεργασία αρχείων ρυθμίσεων στον master και στον worker01	312
A.3.0.2	Προετοιμασία συστήματος αρχείων HDFS	314
A.3.0.3	Προετοιμασία για εκτέλεση	315
A.4	Εκτέλεση σε κατανεμημένη εγκατάσταση Hadoop	316
A.5	Περαιτέρω μελέτη	316
Βιβλιογραφία		316
ΕΥΡΕΤΗΡΙΟ		319
BIBΛΙΟΓΡΑΦΙΑ		325

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

1.1	Τα 3 βασικά χαρακτηριστικά των μεγάλων δεδομένων.	3
1.2	Τα 5 βασικά χαρακτηριστικά των μεγάλων δεδομένων.	5
2.1	Αρχιτεκτονική και πλίνθοι στο GlusterFS.	25
2.2	Επισκόπηση λειτουργίας του Ceph.	28
2.3	Αρχιτεκτονική του Alluxio.	30
2.4	Αρχιτεκτονική του OpenStack Swift. ΔΛ: Διακομιστής Λογαριασμών, ΔΠ: Διακομιστής Περιεχτών, ΔΑ: Διακομιστής Αντικειμένων.	32
2.5	Αρχιτεκτονική και blocks στο HDFS.	41
3.1	Μέτρηση γραμμών αρχείων από έναν υπολογιστή.	52
3.2	Μέτρηση γραμμών αρχείων από K υπολογιστές.	53
3.3	Μέτρηση εμφανίσεων διευθύνσεων IP σε κάθε αρχείο ξεχωριστά με χρήση πίνακα κατακερματισμού.	55
3.4	Διαδικασία MapReduce με χρήση πίνακα κατακερματισμού κατά το map.	56
3.5	Διαδικασία MapReduce με απευθείας εκπομπή ζευγών κλειδιού-τιμής κατά το map, χωρίς χρήση πίνακα κατακερματισμού.	57
3.6	Διαδικασία MapReduce με χρήση Συνδυαστών.	60
4.1	Παράδειγμα διάσπασης πίνακα.	81
4.2	Παράδειγμα απλής ενσωμάτωσης τιμών.	81
4.3	Πολλά-προς-πολλά συσχέτιση (αριστερά) στο μοντέλο οντοτήτων συσχετίσεων και (δεξιά) μετατροπή στο σχεσιακό μοντέλο.	82
4.4	Παράδειγμα ενσωμάτωσης των έργων στον εργαζόμενο.	83
4.5	Παράδειγμα ενσωμάτωσης των εργαζομένων στο έργο	84
4.6	Διάγραμμα του θεωρήματος CAP.	88
4.7	Column families για εργαζομένους.	94
4.8	Γράφημα ιδιοτήτων.	107
4.9	Γράφημα αποτελέσματος για το ερώτημα εργαζόμενοι της Θεσσαλονίκης.	110
4.10	Γράφημα αποτελέσματος για το ερώτημα τμήματα με εργαζομένους από την Αθήνα.	111
5.1	Τα βασικά τμήματα του Spark.	123
5.2	Εκτέλεση της εφαρμογής SimpleApp.	129
5.3	Τα περιεχόμενα του συνδέσμου http://localhost:18080	146
5.4	Λεπτομέρειες για την εκτέλεση της εφαρμογής WordCount.	146
5.5	Λεπτομέρειες για τα στάδια για μία συγκεκριμένη εργασία.	147
8.1	Παραδείγματα δικτύων από διαφορετικές εφαρμογές: (α) δίκτυο πρωτεϊνών ([Ata20]), (β) δίκτυο emails ([CM09]), (γ) οδικό δίκτυο (https://commons.wikimedia.org/wiki/File:India_major_road_network_map.png).	193
8.2	Παράδειγμα γραφήματος.	193
8.3	Τμήμα του γραφήματος της μυθολογίας (πηγή https://docs.janusgraph.org/getting-started/basic-usage/).	197
9.1	Παράδειγμα YARN: υποβολή εφαρμογών στον ResourceManager.	214
9.2	Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager, ο οποίος όμως δεν είναι σε κατάσταση RUNNING.	217
9.3	Παράδειγμα YARN: Αποτέλεσμα list -all σε συστάδα με έναν NodeManager, ο οποίος είναι σε κατάσταση UNHEALTHY.	217

9.4	Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager, ο οποίος είναι σε κατάσταση UNHEALTHY.	217
9.5	Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager και επιλογή -showDetails.	218
9.6	Λίστα εκτέλεσης εφαρμογών στο YARN.	219
9.7	Λεπτομέρειες εκτελούμενης εφαρμογής στο YARN.	219
9.8	Αρχιτεκτονική Mesos.	221
9.9	Παράδειγμα εκτέλεσης Mesos - Μέρος 1.	223
9.10	Παράδειγμα εκτέλεσης Mesos - Μέρος 2.	224
9.11	Αρχιτεκτονική Kubernetes.	227
9.12	Αρχιτεκτονική Spark χρησιμοποιώντας τον αυτόνομο διαχειριστή συστάδας του.	231
10.1	Παράδειγμα πολλαπλασιασμού πινάκων σε δύο κύκλους εργασιών MapReduce.	238
10.2	Αντίστοιχο παράδειγμα πολλαπλασιασμού πινάκων σε έναν κύκλο MapReduce.	239
10.3	Είκοσι εργασίες reduce μαζί εκτελούν μία σύνδεση τριών δρόμων.	240
10.4	Χωρίζοντας τους πίνακες σε ζώνες σε έναν γενικό αλγόριθμο πολλαπλασιασμού πινάκων ενός περάσματος.	242
10.5	Παράδειγμα πίνακα σύνδεσης και ανάθεσης εργασίας γενικής σύνδεσης σε reducers.	246
10.6	Διαφορές στην αρχιτεκτονική μιας CPU (αριστερά) και GPU (δεξιά) [NVI].	248
10.7	Παράδειγμα ενός δισδιάστατου πλέγματος πυρήνα με $5 \times 3 = 15$ blocks, όπου το κάθε ένα αποτελείται από 16 νήματα σε επίσης 2 διαστάσεις.	248
10.8	Παράδειγμα αντιστοίχισης warp σε block νημάτων (από [Bel22]).	249
10.9	Ιεραρχία μνήμης για GPUs που υποστηρίζουν CUDA (εικόνα από [Bel22]).	250
10.10	Αποδοτικότερος υπολογισμός πολλαπλασιασμού πινάκων με τη μέθοδο tiling.	254
10.11	Απλή (αριστερά) και πιο αποδοτική (δεξιά) υλοποίηση υπολογισμού αθροίσματος στοιχείων πίνακα.	255
11.1	Δύο σύνολα με ομοιότητα Jaccard 3/8.	265
11.2	Ένας πίνακας που αναπαριστά τέσσερα σύνολα.	267
11.3	Παράδειγμα μετατροπής του χαρακτηριστικού πίνακα σε πίνακα υπογραφών.	268
11.4	Διαίρεση ενός πίνακα υπογραφών σε λωρίδες με 3 γραμμές ανά λωρίδα.	269
11.5	Παράδειγμα ενός Bloom φίλτρου με $k = 3$	274
11.6	Screenshot από μηχανή αναζήτησης πριν από την εισαγωγή των τεχνικών της Google.	276
11.7	Υποθετική αναπαράσταση του Παγκόσμιου Ιστού με 4 μόνο ιστοσελίδες.	277
11.8	Διαχωρισμός πίνακα μετάβασης και διανύσματος για αποδοτικό υπολογισμό PageRank.	280
11.9	Παράδειγμα μοντέλου δεδομένων καλαθιού αγορών με 5 συναλλαγές.	281
11.10	Σχηματική αναπαράσταση της χρήσης της κύριας μνήμης κατά τη διάρκεια του πρώτου (αριστερά) και του δεύτερου (δεξιά) περάσματος του αλγορίθμου A-Priori.	283
11.11	Παράδειγμα FP-tree στα δεδομένα του Σχήματος 11.9.	287
11.12	Παράδειγμα επιλογής αντιπροσωπευτικών σημείων από κάθε μία από τις δύο συστάδες, που είναι όσο γίνεται μακρύτερα μεταξύ τους (αριστερά) και μετακίνησή τους προς το κεντροειδές (δεξιά).	292
11.13	Παράδειγμα διαμερισμού μερικώς επικαλυπτόμενων συνεχών περιοχών (αριστερά) και τυχαίου διαμερισμού (δεξιά).	293
11.14	Ένα αντίληπτρο διαιρεί τον χώρο με ένα υπερεπίπεδο σε δύο ημι-χώρους, χωρίς αναγκαστικά η ακρίβεια να (μπορεί να) είναι 100%.	294
11.15	Επιλογή υπερεπιπέδου μεγιστοποιώντας το περιθώριο.	295
11.16	Παράδειγμα εκμάθησης από πλησιέστερους γείτονες.	296
11.17	Παράδειγμα γράφου με 2 κοινότητες $\{A, B, C, D\}$ και $\{E, F, G\}$	299

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

1.1	Μονάδες Αποθήκευσης	4
4.1	Ορολογία Riak και σχεσιακών βάσεων δεδομένων.	91
4.2	Ορολογία MongoDB και σχεσιακών βάσεων δεδομένων.	97
4.3	Συγκριτικοί, λογικοί και άλλοι τελεστές στη MongoDB.	100

AKPΩNYMIA

3V Volume, Velocity, Variety.

4V Volume, Velocity, Variety, Veracity.

5V Volume, Velocity, Variety, Veracity, Value.

ACID Atomicity, Consistency, Isolation, Durability.

API Application Programming Interface.

ASCII American Standard Code for Information Interchange.

BASE Basically Available, Soft state and Eventual consistency.

BSON Binary Javascript Object Notation.

CAP Consistency, Availability and Partition tolerance.

CQL Cassandra Query Language.

CRDT Convergent Replicated Data Type.

CRUD Create, Read, Update and Delete.

CUDA Compute Unified Device Architecture.

DHT Distributed Hash Table.

ETL Extract-Transform-Load.

FUSE Filesystem in Userspace.

GDS Graph Data Science.

GFS Google File System.

GPU Graphics processing unit.

GQL Graph Query Language.

HDFS Hadoop Distributed File System.

I/O Input/Output.

IaaS Infrastructure as a Service.

IDE Integrated Development Environment.

IoT Internet of Things.

JSON JavaScript Object Notation.

JVM Java Virtual Machine.

LSH Locality Sensitive Hashing.

LTS Long-Term Support.

NF Normal Form.

NFS Network File System.

PaaS Platform as a Service.

RDD Resilient Distributed Dataset.

RDF Resource Description Framework.

SaaS Software as a Service.

SIMD Single Instruction Multiple Data.

SIMT Single Instruction Multiple Threads.

SM Streaming Multiprocessor.

SQL Structured Query Language.

SSD Solid-State Drive.

SVM Support-vector machine.

TCP Transmission Control Protocol.

XML Extensible Markup Language.

YARN Yet Another Resource Negotiator.

ΚΕΦΑΛΑΙΟ 1

ΕΙΣΑΓΩΓΗ ΣΤΑ ΜΕΓΑΛΑ ΔΕΔΟΜΕΝΑ

Περιεχόμενα

1.1	Τι είναι τα μεγάλα δεδομένα	2
1.2	Ιστορική αναδρομή	6
1.2.1	Η στοίβα λογισμικού της Google	7
1.2.2	Το οικοσύστημα του Hadoop	8
1.3	Μεγάλα δεδομένα και υπολογιστικό νέφος	10
1.4	Αναλυτική μεγάλων δεδομένων	11
1.5	Εφαρμογές ανάλυσης μεγάλων δεδομένων	13
1.5.1	Μεγάλα δεδομένα στην υγεία	13
1.5.2	Μεγάλα δεδομένα στο ηλεκτρονικό εμπόριο και στα συστήματα συστάσεων	14
1.5.3	Μεγάλα δεδομένα και το Διαδίκτυο των Πραγμάτων (IoT)	15
1.5.4	Μεγάλα δεδομένα στην εκπαίδευση	17
1.6	Περαιτέρω μελέτη	18
	BIBΛΙΟΓΡΑΦΙΑ	18



Σύνοψη

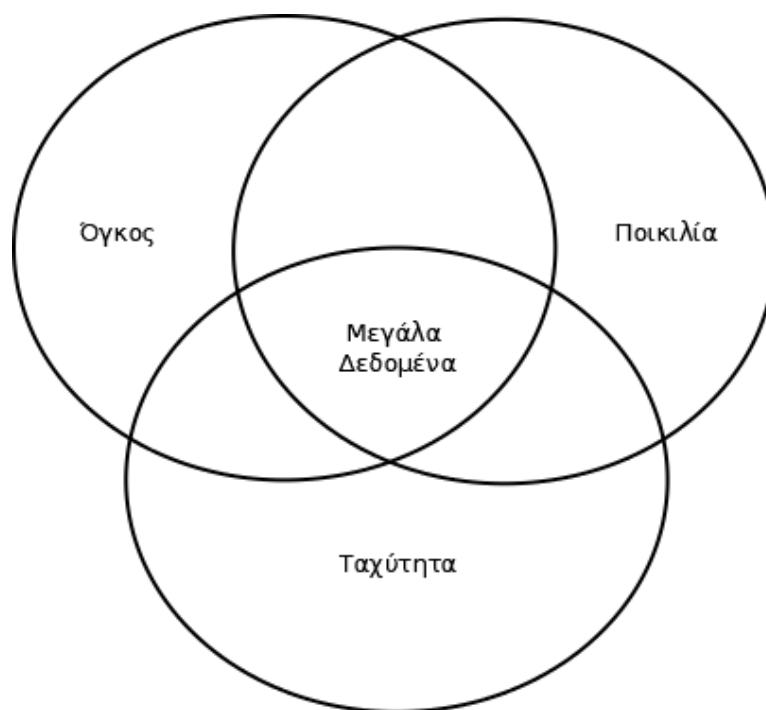
Σε αυτό το κεφάλαιο πραγματοποιείται μια εισαγωγή στην έννοια των Μεγάλων Δεδομένων (*Big Data*), με στόχο να γίνουν κατανοητές οι διαφορετικές προκλήσεις που εμφανίζει η διαχείριση και ανάλυσή τους, αλλά και παράλληλα οι ευκαιρίες που προσφέρει η αποδοτική αξιοποίησή τους. Αρχικά, προσδιορίζονται τα μεγάλα δεδομένα μέσα από τα ιδιαίτερα χαρακτηριστικά τους, καθώς και οι απαιτήσεις που αυτά εισάγουν στις εφαρμογές για την αποδοτική τους ανάλυση και επεξεργασία, έτσι ώστε να κατανοηθούν οι ανάγκες που οδήγησαν στην ανάπτυξη νέων τεχνολογιών, συστημάτων, μοντέλων και αλγορίθμων κατάλληλων για τη διαχείρισή τους. Παράλληλα, αναδεικνύεται η σχέση των μεγάλων δεδομένων με το υπολογιστικό νέφος. Στη συνέχεια, παρουσιάζεται μια σύντομη ιστορική αναδρομή στην εξέλιξη των μεγάλων δεδομένων και αναφέρονται αλγόριθμοι, πλατφόρμες και τεχνολογίες που αποτέλεσαν σταθμούς σε αυτήν την εξέλιξη, και ειδικά η στοίβα λογισμικού της Google και το οικοσύστημα του Hadoop. Τέλος, παρουσιάζεται η έννοια της ανάλυσης μεγάλων δεδομένων και περιγράφονται κάποιες ενδεικτικές μελέτες περίπτωσης της αξιοποίησής τους σε διαφορετικούς τομείς όπως είναι η υγεία, η εκπαίδευση, το ηλεκτρονικό εμπόριο και το Διαδίκτυο των Πραγμάτων.

Προαπαιτούμενη γνώση

Το κεφάλαιο αυτό αποτελεί μία εισαγωγή στα μεγάλα δεδομένα και δεν προϋποθέτει άλλες γνώσεις του αναγνώστη για την κατανόησή του. Αντιθέτως, στόχος του είναι μια πρώτη γνωριμία με ιδιαιτερότητες και προβλήματα που ανακύπτουν κατά τη διαχείριση των μεγάλων δεδομένων και για τα οποία, στα επόμενα κεφάλαια, θα παρουσιαστούν αναλυτικά οι τεχνολογίες και μεθοδολογίες που αναπτύχθηκαν για να τα αντιμετωπίσουν.

1.1 Τι είναι τα μεγάλα δεδομένα

Ο όρος *μεγάλα δεδομένα* (*big data*) είναι πλέον πολύ δημοφιλής και απαντάται σε διάφορα προβλήματα και εφαρμογές, ωστόσο πολλές φορές δεν είναι ξεκάθαρο σε τι αναφερόμαστε ακριβώς όταν μιλάμε για μεγάλα δεδομένα. Συνήθως, ορίζονται ως [Com]: *μεγάλες ποσότητες δεδομένων που παράγονται πολύ γρήγορα από ένα πλήθος από διαφορετικές πηγές*, και προσδιορίζονται μέσα από τα βασικά τους χαρακτηριστικά: τον όγκο (*volume*), την ταχύτητα (*velocity*) και την ποικιλία (*variety*), τα 3V όπως έχουν επικρατήσει (Σχήμα 1.1). Ωστόσο, δεν είναι σαφές τι εννοούμε όταν χρησιμοποιούμε τον χαρακτηρισμό «μεγάλα», και ποιος θα πρέπει να είναι ο όγκος των δεδομένων για να συγκαταλέγονται αυτά στα μεγάλα δεδομένα. Ακόμα, πρόσθετα σημαντικά χαρακτηριστικά έχουν πλέον αναγνωριστεί για τα μεγάλα δεδομένα, και συγκεκριμένα η *εγκυρότητα* (*veracity*) [GK17] που οδήγησε στον ορισμό τους μέσα από 4 πλέον διαστάσεις (4V), και η *αξία* (*value*) ανεβάζοντας τον αριθμό των διαστάσεων στις 5 (5V) [Cou13] (Σχήμα 1.2). Γενικά, η τάση αυτή συνεχίζεται φτάνοντας τα χαρακτηριστικά V σε πάνω από 10, χωρίς όμως αυτά να έχουν επικρατήσει καθολικά και δείχνοντας με αυτόν τον τρόπο τα πολλά και διαφορετικά χαρακτηριστικά των μεγάλων δεδομένων που καλούνται να αντιμετωπίσουν οι διάφοροι επαγγελματίες και ερευνητές κατά την επεξεργασία και



Σχήμα 1.1: Τα 3 βασικά χαρακτηριστικά των μεγάλων δεδομένων.

την ανάλυσή τους σε διαφορετικούς τομείς και εφαρμογές.

Ας δούμε, αρχικά, πιο αναλυτικά τα βασικά αυτά χαρακτηριστικά που έχουν γίνει καθολικά αποδεκτά.

- **Όγκος (Volume).** Η πιο χαρακτηριστική ίσως από τις ιδιότητες των μεγάλων δεδομένων είναι ο μεγάλος όγκος τους όπως λέει και το όνομά τους. Είναι σημαντικό να σημειώσουμε ότι το τι θεωρείται «μεγάλο» αλλάζει σε κάθε εποχή, και αυτό που θεωρούνταν χτες μεγάλου όγκου, μέσα από τεχνολογικές εξελίξεις, κυρίως στο υλικό των υπολογιστών αλλά και στον ρυθμό παραγωγής και τη διαθεσιμότητα δεδομένων που υπάρχει, μπορεί σήμερα να μην χαρακτηρίζεται πλέον έτσι.

Αυτήν τη στιγμή, η ποσότητα των δεδομένων μετράται σε petabytes και exabytes, αλλά και συνολικά έχουμε πλέον φτάσει στα zettabytes (Πίνακας 1.1) και υπάρχει πρόβλεψη ότι ο συνολικός όγκος των διαθέσιμων δεδομένων προς αποθήκευση θα ξεπεράσει τα 185 ZB μέχρι το 2025 [Sta21]. Ο όγκος αυτός προέρχεται από μια πληθώρα πηγών και κυρίως μέσα από τη χρήση των κινητών συσκευών, των μέσων κοινωνικής δικτύωσης αλλά και του *Διαδικτύου των Πραγμάτων (Internet of Things, IoT)*.

- **Ταχύτητα (Velocity).** Η ταχύτητα αναφέρεται στον ρυθμό με τον οποίο παράγονται τα δεδομένα, αλλά και στον ρυθμό μετάδοσής τους, καθώς και στην ταχύτητα με την οποία υποβάλλονται σε επεξεργασία. Το μεγάλο πλήθος των χρηστών και των νέων εφαρμογών που παράγουν νέα δεδομένα μέσω αλληλεπιδράσεων με τους χρήστες ορίζουν ένα νέο μοντέλο δεδομένων που μπορούν να θεωρηθούν ως δεδομένα που βρίσκονται συνεχώς σε ροή. Η απαίτηση για επεξεργασία και ανάλυση αυτών των δεδομένων σε πραγματικό χρόνο, σε συνδυασμό με τον μεγάλο τους όγκο κάνει εμφανή την ανάγκη για τη σχεδίαση νέων αρχιτεκτονικών, μοντέλων και τεχνικών που θα μπορούν να

Πίνακας 1.1: Μονάδες Αποθήκευσης

Συντόμευση	Μονάδα	Αξία
b	bit	0 ή 1
B	byte	8 bits
KB	kilobyte	10^3 bytes
MB	megabyte	10^6 bytes
GB	gigabyte	10^9 bytes
TB	terabyte	10^{12} bytes
PB	petabyte	10^{15} bytes
EB	exabyte	10^{18} bytes
ZB	zettabyte	10^{21} bytes
YB	yottabyte	10^{24} bytes

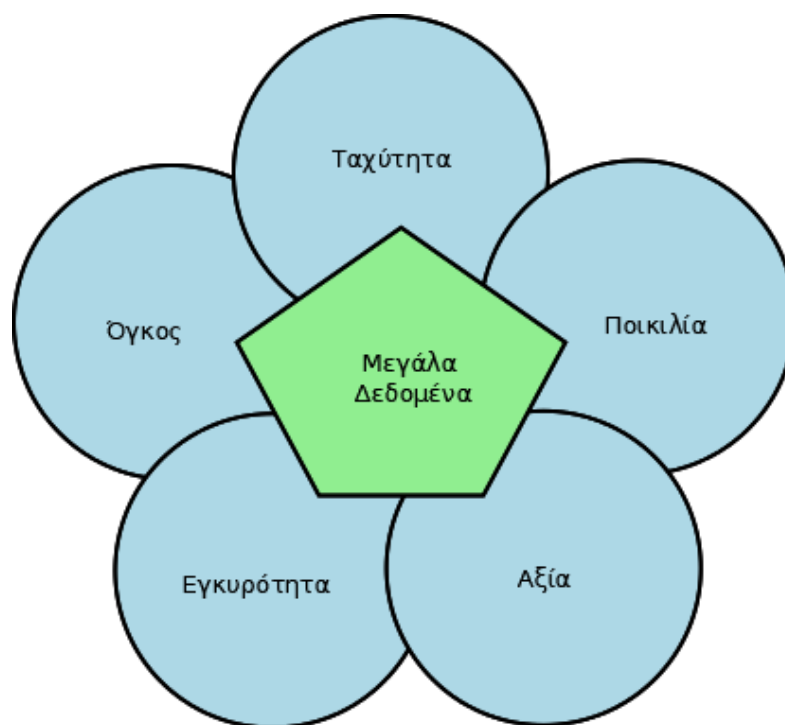
αντεπεξέλθουν στις νέες συνθήκες.

- **Ποικιλία (Variety).** Η ποικιλία αναφέρεται στην ετερογένεια που χαρακτηρίζει τα μεγάλα δεδομένα. Τα δεδομένα αυτά παράγονται από πληθώρα πηγών, αφορούν διαφορετικούς τομείς εφαρμογής και αναπαριστώνται σε διαφορετικές μορφές. Μπορεί να προέρχονται από τον παγκόσμιο ιστό, να είναι σε μορφή κειμένου, πολυμέσων, να αποτελούν μετρήσεις αισθητήρων, κ.ο.κ. Συνεπώς, πρόκειται για *δομημένα (structured)* δεδομένα: σχεσιακά δεδομένα που προέρχονται από παραδοσιακές βάσεις δεδομένων με συγκεκριμένη δομή και αυστηρή τυποποίηση, *ημιδομημένα (semi-structured)* δεδομένα: δεδομένα που δεν ακολουθούν αυστηρή τυποποίηση, αλλά συνδυάζουν κάποια στοιχεία δομής, όπως XML και JSON έγγραφα, καθώς και *αδόμητα (unstructured)* δεδομένα: δεδομένα που δεν έχουν καμία δομή, όπως το ελεύθερο κείμενο. Είναι φανερό ότι όλοι αυτοί οι διαφορετικοί τύποι δεδομένων με τα ποικίλα χαρακτηριστικά, αποθηκεύονται με διαφορετικούς τρόπους και απαιτούν διαφορετικές μεθόδους επεξεργασίας από τις παραδοσιακές βάσεις δεδομένων, οι οποίες σχεδιάστηκαν για την αποδοτική αποθήκευση και επεξεργασία μόνο κανονικών δομημένων δεδομένων.

Πέρα από τα παραπάνω κοινά αποδεκτά χαρακτηριστικά, θα περιγράψουμε σύντομα και τα δύο άλλα χαρακτηριστικά που συνθέτουν τα 5V: την εγκυρότητα και την αξία.

- **Εγκυρότητα (Veracity).** Όπως γίνεται φανερό από την περιγραφή των τρόπων παραγωγής των δεδομένων που προέρχονται από πολλαπλές ετερογενείς πηγές και την ταχύτητα με την οποία αυτά παράγονται, γρήγορα οι χρήστες ήρθαν αντιμέτωποι με προβλήματα στην ποιότητά τους. Τα μεγάλα δεδομένα έχουν συχνά ασυνέπειες, ελλείψεις τιμές, αλλά μπορεί ακόμα να είναι παρωχημένα ή και πλαστά, οπότε χαρακτηρίζονται από αβέβαιο βαθμό εγκυρότητας και θα πρέπει να προσδιορίζονται ως προς την ακρίβεια και την ποιότητά τους.

Ως προς την ακρίβεια, το επιθυμητό είναι τα δεδομένα να μην περιέχουν λάθη, ανακρίβειες ή ασυνέπειες, ώστε να μπορούν να χρησιμοποιηθούν με αξιοπιστία στη λήψη αποφάσεων, χωρίς να υπάρχουν φόβοι ως προς την εγκυρότητά τους ή την παραποίησή τους από τρίτους. Ως προς την ποιότητά τους, προσδιορίζονται επιπλέον η πληρότητα, η ακεραιότητα, η σαφήνεια αλλά και η προέλευση των δεδομένων.



Σχήμα 1.2: Τα 5 βασικά χαρακτηριστικά των μεγάλων δεδομένων.

- **Αξία (Value).** Η αξία αναφέρεται στη δυνατότητα μετατροπής αυτών των δεδομένων σε όφελος. Αναφέρεται, λοιπόν, στα κέρδη και τα συμπεράσματα που μπορεί να εξαγάγει κανείς από την επεξεργασία και την ανάλυση αυτών των δεδομένων. Με άλλα λόγια, αναφέρεται στη χρησιμότητά τους και τα πλεονεκτήματα που μπορούμε να αποκομίσουμε από αυτά.

Προφανώς, η αξία δεν αποτελεί μια ιδιότητα που χαρακτηρίζει μόνο τα μεγάλα δεδομένα, αλλά όλα τα δεδομένα. Ωστόσο, ο μεγάλος όγκος και η διαθεσιμότητα των μεγάλων δεδομένων καθιστούν την απαίτηση για την ορθή εκμετάλλευσή τους, ώστε να παραχθούν μεγαλύτερα οφέλη, πιο αναγκαία από ποτέ.

Είναι σημαντικό εδώ να σημειώσουμε ότι όταν αναφερόμαστε σε κέρδη και οφέλη δεν μιλάμε μόνο για χρηματικό κέρδος, αν και αυτό αποτελεί έναν καθοριστικό παράγοντα της αξίας των δεδομένων. Συνεπώς, αναφερόμαστε σε επιχειρηματική διορατικότητα που μπορεί να προσφέρουν, ανταγωνιστικά πλεονεκτήματα σε επιχειρήσεις, ευκαιρίες για καινοτομία, βελτίωση των επιχειρηματικών διαδικασιών, αποφυγή κινδύνων, κ.ά. Παράλληλα, αναφερόμαστε και σε οφέλη στην ιατρική, τη διαχείριση φυσικών καταστροφών και κινδύνων και γενικότερα σε τομείς και εφαρμογές που μπορούν να βελτιώσουν τη ζωή των πολιτών μιας κοινωνίας.

Βασισμένοι στα παραπάνω χαρακτηριστικά, βλέπουμε ότι τα παραδοσιακά συστήματα και τεχνικές δεν ήταν αρκετά για να ανταποκριθούν στις απαιτήσεις που δημιουργεί η διαθεσιμότητα των μεγάλων δεδομένων, αφού έχουν σχεδιαστεί για δεδομένα με πολύ διαφορετικά χαρακτηριστικά. Αυτές τις απαιτήσεις μπορούμε να τις συνοψίσουμε στα παρακάτω 5 ερωτήματα [BCD16]:

- Πώς αποθηκεύουμε δεδομένα τόσο μεγάλης κλίμακας στους διαθέσιμους πόρους;

- Πώς θα προσπελάσουμε αποδοτικά αυτά τα μεγάλα δεδομένα;
- Πώς διαχειριζόμαστε σύνολα δεδομένων που βρίσκονται σε διαφορετικές μορφές και είναι δομημένα, ημιδομημένα αλλά και αδόμητα;
- Πώς επεξεργαζόμαστε αυτά τα δεδομένα με έναν πλήρως κλιμακούμενο, ευέλικτο και ανθεκτικό σε λάθη τρόπο;
- Πώς αναλύουμε αυτά τα δεδομένα και εξάγουμε επιχειρηματική ευφυΐα από αυτά αποδοτικά και διαδραστικά;

Οι προκλήσεις, λοιπόν, που επιβάλλει η ύπαρξη των μεγάλων δεδομένων συμπεριλαμβάνουν θέματα όπως τη λήψη τους, την αποθήκευση, τη διατήρηση και ενημέρωσή τους, την αναζήτηση και ανάκτησή τους, τον διαμοιρασμό, τη μεταφορά, την ανάλυση και την οπτικοποίησή τους, θέματα των οποίων η αντιμετώπιση ξεπερνά τις δυνατότητες των υφιστάμενων αρχιτεκτονικών και μεθοδολογιών.

Για να απαντηθούν οι παραπάνω προκλήσεις, νέα συστήματα και τεχνικές επεξεργασίας και ανάλυσης σχεδιάστηκαν από την αρχή, με στόχο να ανταποκριθούν στα ιδιαίτερα χαρακτηριστικά των μεγάλων δεδομένων, όπως το παράλληλο προγραμματιστικό πρότυπο MapReduce και το πλαίσιο του Apache Hadoop. Στα επόμενα κεφάλαια θα περιγραφούν πολλές από τις σχετικές τεχνικές και τα εργαλεία που έχουν αναπτυχθεί για τον σκοπό αυτό.

1.2 Ιστορική αναδρομή

Ο όρος «μεγάλα δεδομένα» άρχισε να χρησιμοποιείται το 2005 από τον Roger Mougals, ωστόσο τα θέματα σχετικά με την αποδοτική επεξεργασία και ανάλυση δεδομένων, ώστε να αξιοποιηθούν στη λήψη αποφάσεων και σε άλλες επιχειρηματικές δραστηριότητες, απασχόλησαν τόσο τους ερευνητές όσο και τους επαγγελματίες για πολύ περισσότερα χρόνια.

Αν και ο όρος, σύμφωνα με πολλούς, άρχισε να επικρατεί ακόμα και μετά το 2010, θα πρέπει να γίνει ξεκάθαρο ότι δεν είναι κάτι εντελώς νέο. Ακόμα και πριν από την ύπαρξη των υπολογιστών, δεδομένα αναλύονταν με τη χρήση μαθηματικών και στατιστικών μοντέλων. Ωστόσο, η ανάπτυξη των πρώτων υπολογιστικών συστημάτων ήταν αυτή που ανέδειξε πολύ περισσότερο και έφερε στο προσκήνιο την απαίτηση για επεξεργασία και ανάλυση δεδομένων. Τα πρώτα αυτά συστήματα ήρθαν αντιμέτωπα με αντίστοιχες προκλήσεις που ορίζουν τη σημερινή εποχή τα μεγάλα δεδομένα, αλλά σε ένα διαφορετικό πλαίσιο. Έτσι, πάντα υπήρχε η ανάγκη για αποδοτική επεξεργασία μεγάλης κλίμακας δεδομένων, αλλά το τι θεωρούνταν μεγάλης κλίμακας δεδομένα, όπως ήδη έχουμε αναφέρει, έχει αλλάξει δραστικά. Η ανάπτυξη του *σχεσιακού μοντέλου* το 1970 από την IBM και τον Edgar Codd και η υποστήριξη του ταυτοχρονισμού με την ανάπτυξη του *μοντέλου συναλλαγών*, γενικότερα, αποτέλεσαν σταθμούς στην επίλυση των προκλήσεων της διαχείρισης των δεδομένων. Για πολλούς το πρόβλημα της διαχείρισης δεδομένων θεωρήθηκε πλέον λίγο-πολύ λυμένο, αν και διαρκώς η αύξηση των διαθέσιμων δεδομένων, αλλά και οι αλλαγές στο υλικό και φυσικά η ανάπτυξη του διαδικτύου δημιουργούσαν ευκαιρίες για νέα έρευνα και εξέλιξη. Συνεπώς, συνεχίζοντας αυτήν την τάση, τα μεγάλα δεδομένα και τα άλλα τεχνολογικά επιτεύγματα που έχουν

γίνει μέχρι σήμερα, όπως η έκρηξη του διαδικτύου και των μέσων κοινωνικής δικτύωσης και η ανάπτυξη του υπολογιστικού νέφους, έχουν επαναφέρει στο προσκήνιο τη διαχείριση δεδομένων σε ένα νέο πλαίσιο που παρουσιάζει ιδιαίτερες προκλήσεις αλλά και ευκαιρίες και έχει αναγκάσει επαγγελματίες και ερευνητές του χώρου να αναπροσαρμόσουν παλαιότερες και να σχεδιάσουν νέες λύσεις, τεχνολογίες, πλατφόρμες, τεχνικές και αλγόριθμους, για να αντιμετωπίσουν αποτελεσματικά τις νέες αυτές προκλήσεις.

Στη συνέχεια της σύντομης ιστορικής αναδρομής που θα καλύψουμε σε αυτό το εισαγωγικό κεφάλαιο, θα εστιάσουμε στους πιο σύγχρονους σταθμούς στην ιστορία της εξέλιξης των μεγάλων δεδομένων, που κινητοποιήθηκαν κυρίως από μεγάλες επιχειρήσεις του χώρου που πρωτοστάτησαν στην ανάπτυξη των σχετικών λύσεων, τεχνολογιών και μεθόδων. Google, Yahoo, Amazon αλλά και μέσα κοινωνικής δικτύωσης όπως Facebook, Twitter (μετονομάστηκε σε X) ήρθαν πρώτα αντιμέτωπα με τις προκλήσεις της διαχείρισης μεγάλου όγκου δεδομένων, αφού ήταν τα πρώτα που άρχισαν να συγκεντρώνουν τεράστιο όγκο δεδομένων και, για να τα εκμεταλλευτούν και να τα αξιοποιήσουν ώστε να μεγιστοποιήσουν τα κέρδη τους, θα έπρεπε να αναπτυχθούν νέες υποδομές, νέοι αλγόριθμοι και νέες τεχνολογίες, αφού οι καθιερωμένες λύσεις δεν ήταν ικανοποιητικές.

1.2.1 Η στοίβα λογισμικού της Google

Η Google ήταν ένας από τους πρωτοπόρους, όπως αναφέραμε, σε σχέση με τα μεγάλα δεδομένα. Το πρώτο και επαναστατικό επίτευγμα της Google ήταν η ανάπτυξη του αλγόριθμου PageRank [BP98] για τη διάταξη των αποτελεσμάτων μιας μηχανής αναζήτησης με βάση το κύρος των ιστοσελίδων. Πριν από τον PageRank, η διάταξη των αποτελεσμάτων μιας μηχανής αναζήτησης γινόταν χρησιμοποιώντας ως κριτήριο τη συνάφεια με το ερώτημα του χρήστη με βάση το κειμενικό περιεχόμενο μιας ιστοσελίδας. Ωστόσο, αυτός ο τρόπος βαθμολόγησης είχε προβλήματα τόσο ως προς το πλήθος των αποτελεσμάτων που μπορούσαν να ανακτηθούν όσο, κυρίως, και ως προς την ποιότητα των αποτελεσμάτων, για παράδειγμα την ύπαρξη spam. Η καινοτομία που εισήγαγε ο PageRank είναι να βαθμολογούνται οι σελίδες με βάση το πλήθος και την ποιότητα των υπερσυνδέσμων που δείχνουν προς μια ιστοσελίδα, με τη θεώρηση ότι κάθε υπερσύνδεσμος μπορεί να θεωρηθεί ως μια θετική ψήφος προς το περιεχόμενο και την ποιότητα της συγκεκριμένης ιστοσελίδας. Ο αλγόριθμος αυτός αποτελεί ένα παράδειγμα αλγορίθμου ανάλυσης μεγάλων δεδομένων, αφού βασίζεται στην ανάλυση της δομής του παγκόσμιου ιστού που απαρτίζεται από ένα πολύ μεγάλο πλήθος ιστοσελίδων.

Η Google είχε πολύ μεγάλο ρυθμό ανάπτυξης και, ως αποτέλεσμα, ανέπτυξε γρήγορα κέντρα δεδομένων (*data centers*) και, μάλιστα, γρήγορα ανέπτυξε και εξειδικευμένες αρχιτεκτονικές για τα δικά της κέντρα δεδομένων, επειδή οι ταχέως αυξανόμενες απαιτήσεις της δεν μπορούσαν να αντιμετωπιστούν αποδοτικά από τις υπάρχουσες αρχιτεκτονικές. Πέρα όμως από τις λύσεις στο υλικό, η Google προχώρησε και σε πολλές καινοτόμες λύσεις στο λογισμικό.

Η πλατφόρμα υπολογιστικού νέφους που παρέχει πλέον η Google προσφέρει ένα πλήθος εργαλείων και υποσυστημάτων. Ωστόσο, εδώ θα μείνουμε σε τρεις από τις μεγάλες καινοτόμες λύσεις που αποτέλεσαν θεμέλιους λίθους στην προσέγγιση της Google στο πρόβλημα της ανάλυσης μεγάλων δεδομένων. Οι τρεις αυτές τεχνολογίες είναι:

- **Google File System (GFS)** [GGL03]. Το GFS αποτελεί ένα σύστημα διαχείρισης αρχείων για κατανεμημένες συστάδες (clusters). Το σύστημα αυτό παρέχει διαφάνεια στους προγραμματιστές, οι οποίοι μπορούν να προσπελάσουν δεδομένα που βρίσκονται κατανεμημένα σε πολλές συστάδες υπολογιστών σαν να ήταν αποθηκευμένα σε ένα μοναδικό μηχάνημα. Επιπλέον, παρέχει μηχανισμούς για την ανοχή σε σφάλματα και αυξημένη διαθεσιμότητα χωρίς να επιβαρύνει τον προγραμματιστή με αντίστοιχους προβληματισμούς.
- **BigTable** [CDG⁺06]. Το BigTable αποτελεί ένα από τα πρώτα NoSQL, μη-σχεσιακά συστήματα διαχείρισης δεδομένων που εκμεταλλεύεται το GFS για την αποθήκευση των δεδομένων. Το BigTable εκτελείται πάνω από το GFS και, όπως και εκείνο, μπορεί να χειριστεί αποδοτικά κατανεμημένα δεδομένα σε συστάδες υπολογιστών κρύβοντας από τους προγραμματιστές τις λεπτομέρειες της κατανεμημένης διαχείρισής τους.
- **MapReduce** [DG04]. Το MapReduce αποτελεί ένα ισχυρό παράλληλο προγραμματιστικό μοντέλο που επιτρέπει την εύκολη παραλληλοποίηση πολλών διαδικασιών επεξεργασίας και ανάλυσης δεδομένων και πάλι πάνω από κατανεμημένες συστάδες υπολογιστών.

Η Google, εκτός από τις καινοτομίες που παρουσίασε, αποτέλεσε και τη βάση για πολλαπλές νέες τεχνικές, τεχνολογίες αλλά και λύσεις ανοικτού κώδικα που αναπτύχθηκαν στη συνέχεια, αφού ενέπνευσε ερευνητές και επαγγελματίες του χώρου. Στα επόμενα κεφάλαια θα παρουσιαστούν οι λύσεις ανοικτού λογισμικού που υπάρχουν ως εναλλακτικές για το GFS, το MapReduce και το BigTable.

1.2.2 Το οικοσύστημα του Hadoop

Το έργο Hadoop [Had] αποτέλεσε την ανοικτού κώδικα εναλλακτική λύση στη στοίβα λογισμικού της Google και βασίστηκε σε πολλές τεχνολογίες και τεχνικές που είχαν χρησιμοποιηθεί από την Google.

Όπως η στοίβα λογισμικού της Google, έτσι και το Hadoop ξεκίνησε από ερευνητές που προσπαθούσαν να σχεδιάσουν μια ανοικτού κώδικα μηχανή αναζήτησης που θα μπορούσε να συναγωνιστεί την Google. Συγκεκριμένα, οι Doug Cutting και Mike Cafarella δούλευαν στην ανάπτυξη της μηχανής αναζήτησης Nutch [Nut] που θα βασιζόταν στο Apache Lucene [luc, MHGG10], μια μηχανή ευρετηριοποίησης και αναζήτησης σε κείμενο. Σήμερα πλέον, το Nutch παρέχει έναν αποδοτικό ιχνηλάτη ιστού που βασίζεται στο Hadoop και υποστηρίζει προσθήκες από το Apache Solr [sol, GP14], άλλο ένα Apache έργο για την ανάπτυξη μιας πλατφόρμας αναζήτησης ικανής να χειριστεί μεγάλο όγκο δεδομένων που ακολούθησε την ανάπτυξη του Lucene, αλλά και το Elasticsearch [ela, GT15] το οποίο επίσης αποτελεί μια κατανεμημένη μηχανή αναζήτησης κειμένων που βασίστηκε για την ανάπτυξή του στο Lucene.

Παρότι σήμερα το έργο είναι επιτυχημένο, στην αρχή της ανάπτυξής του το Nutch δεν μπορούσε να ξεπεράσει τα προβλήματα κλιμάκωσης που έπρεπε να αντιμετωπίσει για να ευρετηριοποιήσει το σύνολο των ιστοσελίδων ενός ραγδαία αναπτυσσόμενου παγκόσμιου ιστού. Όταν η Google δημοσίευσε το σύστημα αρχείων GFS καθώς και το προγραμματιστικό μοντέλο MapReduce, οι ερευνητές που εργάζονταν στο Nutch αποφάσισαν ότι θα έπρεπε να

ακολουθήσουν αντίστοιχες προσεγγίσεις, αφού οι καινοτόμες προσεγγίσεις της Google έμοιαζαν να αντιμετωπίζουν με επιτυχία τα προβλήματα που οι ίδιοι ακόμα δεν είχαν καταφέρει να λύσουν. Έτσι, ξεκίνησαν να εργάζονται σε αντίστοιχες ανοικτού κώδικα εναλλακτικές τόσο του GFS όσο και του MapReduce. Μέσα από την έρευνά τους προς την ανάπτυξη μιας αποδοτικής μηχανής αναζήτησης, αλλά και άλλων θεμάτων σχετικά με τη διαχείριση και την ανάλυση μεγάλων δεδομένων, το 2005, προέκυψε το Hadoop.

Το Hadoop έγινε διαθέσιμο για πρώτη φορά το 2007, και από το 2008 αποτέλεσε έργο της Apache κορυφαίου επιπέδου. Ωστόσο, το Hadoop βρήκε μεγάλη απήχηση και υποστήριξη και από μεγάλες εταιρείες. Υποστηρίχτηκε αρχικά από τη Yahoo στην οποία εργάστηκε ο Cutting, αλλά γρήγορα ξεκίνησε να το αξιοποιεί και η Facebook (μετονομάστηκε σε Meta Platforms), καθώς και πολλοί άλλοι μεγάλοι επιχειρηματικοί κολοσσοί.

Το οικοσύστημα του Hadoop πλέον έχει υιοθετηθεί από τις περισσότερες μεγάλες εταιρείες λογισμικού και αποτελεί μέρος των λύσεων που προσφέρουν. Παράλληλα, πολλά προϊόντα έχουν προστεθεί στο πλαίσιο του Hadoop προσφέροντας ολόένα και περισσότερη λειτουργικότητα. Το βασικό έργο αποτελείται από τις ακόλουθες μονάδες:

- **Hadoop Distributed File System (HDFS)** [SKRC10]. Το HDFS αποτελεί ένα κατακευματισμένο σύστημα διαχείρισης αρχείων και την εναλλακτική για το GFS της Google στο έργο του Hadoop. Το HDFS προσφέρει αντίστοιχες δυνατότητες κλιμάκωσης και διαφάνειας με του GFS πάνω από μεγάλες υπολογιστικές συστάδες.
- **Hadoop YARN** [VMD⁺13]. Το Hadoop YARN αποτελεί ένα πλαίσιο για τον χρονοπρογραμματισμό των εργασιών και τη διαχείριση των πόρων των συστάδων στις οποίες εκτελείται το Hadoop.
- **Hadoop MapReduce** [DQR12]. Το Hadoop MapReduce, όπως και το MapReduce της Google, αποτελεί ένα παράλληλο προγραμματιστικό μοντέλο για την επεξεργασία και την ανάλυση μεγάλου όγκου δεδομένων, το οποίο όμως βασίζεται και στο YARN.

Στη στοίβα λογισμικού της Google, ωστόσο, αναφέραμε και μια πρώτη NoSQL βάση δεδομένων, το BigTable. Αντίστοιχη λύση αναπτύχθηκε και για το Hadoop. Έτσι, εμπνευσμένοι και πάλι από το BigTable της Google, οι ερευνητές στο Hadoop ανέπτυξαν την HBase [Geo11]. Η HBase υλοποιείται πάνω από το HDFS, το οποίο χρησιμοποιεί ως το σύστημα οργάνωσης των αρχείων, με τρόπο ανάλογο αυτού που θα χρησιμοποιούσε ένα σχεσιακό σύστημα διαχείρισης βάσεων δεδομένων το σύστημα οργάνωσης αρχείων του λειτουργικού συστήματος. Η HBase εκμεταλλεύεται τις δυνατότητες που της δίνει το HDFS, και συγκεκριμένα τη μεγάλη διαθεσιμότητα που επιτυγχάνεται με την αναπαραγωγή των δεδομένων και την ανοχή σε σφάλματα, και με αυτόν τον τρόπο πετυχαίνει μεγάλη κλιμάκωση στη διαχείριση των μεγάλων δεδομένων.

Πέρα από τα παραπάνω, όπως αναφέραμε, το οικοσύστημα του Hadoop έχει εμπλουτιστεί με πολλά συνεργαζόμενα επιπλέον εργαλεία και συστήματα που δίνουν πολύ περισσότερες δυνατότητες επεξεργασίας και ανάλυσης δεδομένων. Επιγραμματικά αναφέρουμε τα εξής:

- **Apache Spark** [ZXW⁺16]. Το Spark αποτελεί μια μηχανή επεξεργασίας δεδομένων στο Hadoop. Είναι γενικού σκοπού και υποστηρίζει εργασίες που σχετίζονται με μηχανική

δεδομένων, επιστήμη δεδομένων και μηχανική μάθηση είτε σε μεμονωμένα υπολογιστικά συστήματα είτε σε κατανεμημένες συστάδες υπολογιστών.

- **Apache Hive** [hiv]. Το Hive αποτελεί μια κατανεμημένη, ανεκτική σε σφάλματα αποθήκη δεδομένων που υποστηρίζει ανάλυση και επεξεργασία μεγάλου όγκου κατανεμημένων δεδομένων χρησιμοποιώντας την SQL.
- **Pig / Pig Latin** [ORS⁺08]. Το Pig αποτελεί και αυτό μια κατανεμημένη αποθήκη δεδομένων που αναπτύχθηκε για να υποστηρίξει αποδοτική ανάλυση και επεξεργασία δεδομένων, μόνο που αντί για SQL βασίζεται στη γλώσσα Pig Latin, μια διαδικαστική γλώσσα υψηλού επιπέδου για τον ορισμό ροών δεδομένων.
- **Mahout** [OFAD11]. Το Mahout αποτελεί μια βιβλιοθήκη για εξόρυξη δεδομένων και μηχανική μάθηση.
- **Apache Submarine** [sub]. Το Apache Submarine αποτελεί μια πλατφόρμα για την εκτέλεση εργασιών μηχανικής μάθησης και βαθιάς μάθησης πάνω από κατανεμημένες συστάδες υπολογιστών.
- **ZooKeeper** [HKJR10]. Το ZooKeeper αποτελεί μια υψηλής απόδοσης υπηρεσία συντονισμού για κατανεμημένες εφαρμογές.
- **Oozie** [IHB⁺12]. Το Oozie είναι ένας χρονοπρογραμματιστής ροών εργασίας που χρησιμοποιείται για τη διαχείριση των εργασιών του Hadoop.

Στα επόμενα κεφάλαια θα δούμε πολλές από τις μονάδες που αποτελούν το οικοσύστημα του Hadoop, και ιδιαίτερα θα σταθούμε στο HDFS, το MapReduce, το Spark και το YARN.

1.3 Μεγάλα δεδομένα και υπολογιστικό νέφος

Το *υπολογιστικό νέφος* (*cloud computing*) έχει συμβάλει σημαντικά στην ανάπτυξη αποδοτικών λύσεων για τη διαχείριση και ανάλυση μεγάλων δεδομένων. Συχνά, οι δύο όροι, «μεγάλα δεδομένα» και «υπολογιστικό νέφος» απαντώνται μαζί και υπάρχει πληθώρα διαθέσιμων υπηρεσιών νέφους που εξειδικεύονται στην παροχή αποθήκευσης, αλλά κυρίως ανάλυσης μεγάλων δεδομένων.

Με τον όρο υπολογιστικό νέφος αναφερόμαστε στην κατά παραγγελία παροχή υπολογιστικών πόρων, πόρων αποθήκευσης, επεξεργαστικής ισχύος, δικτυακών πόρων αλλά και λογισμικού ή και οποιουδήποτε είδους υπηρεσίας. Αρχικός στόχος της ανάπτυξης του υπολογιστικού νέφους ήταν η εκμετάλλευση απλών υπολογιστικών μονάδων, ώστε να σχηματίζουν υπολογιστικές συστάδες με αυξημένες υπολογιστικές δυνατότητες που θα μπορούσαν να αντεπεξέλθουν στις απαιτήσεις εφαρμογών αναζήτησης και ευρετηριοποίησης του παγκόσμιου ιστού.

Στη συνέχεια, αναπτύχθηκαν τα πρότυπα *υποδομή ως υπηρεσία* (*Infrastructure as a Service, IaaS*), όπου υπολογιστικοί πόροι γίνονται διαθέσιμοι κατά παραγγελία ως κύκλοι επεξεργασίας, *πλατφόρμα ως υπηρεσία* (*Platform as a Service, PaaS*) που παρέχει και την υλική υποδομή

αλλά και το λογισμικό που τρέχει στην πλατφόρμα, και *λογισμικό ως υπηρεσία* (*Software as a Service, SaaS*) όπου οι χρήστες μπορούν να χρησιμοποιούν ολοκληρωμένες εφαρμογές που παρέχονται από το νέφος. Μέσω αυτών των προτύπων, το υπολογιστικό νέφος οδήγησε σε μια στροφή από το παραδοσιακό μοντέλο πελάτη-εξυπηρετητή σε εφαρμογές των οποίων τα δεδομένα αλλά και οι εξυπηρετητές βρίσκονται κάπου γεωγραφικά διάσπαρτα στο υπολογιστικό νέφος και είναι προσπελάσιμα μέσω του διαδικτύου.

Ως αποτέλεσμα, το υπολογιστικό νέφος εμφανίστηκε ως μια πολύ ελκυστική λύση για μικρότερες και μεσαίες επιχειρήσεις που δεν διέθεταν τους πόρους να επενδύσουν σε υλικό μεγάλης υπολογιστικής ισχύος. Η ελαστικότητα που προσφέρει το υπολογιστικό νέφος αποτελεί άλλο ένα ελκυστικό χαρακτηριστικό για αυτές τις επιχειρήσεις, αφού επιτρέπει την επένδυση των πόρων τους ανάλογα με τις ανάγκες τους και τη σταδιακή κλιμάκωση των πόρων που δεσμεύουν ανάλογα με τις απαιτήσεις τους.

Ουσιαστικά, το υπολογιστικό νέφος το συνθέτει ένα σύνολο από διακομιστές, συνήθως υψηλών προδιαγραφών από διαφορετικούς παρόχους, με στόχο τη γρήγορη επεξεργασία δεδομένων. Το μοντέλο αυτό καθιστά τις λύσεις υπολογιστικού νέφους πολύ πιο κατάλληλες στο να χειριστούν τα μεγάλης κλίμακας δεδομένα που έχουμε όταν αναφερόμαστε πλέον σε μεγάλα δεδομένα από ό,τι ένας κεντρικός μεμονωμένος εξυπηρετητής ή, πολύ περισσότερο, ένας προσωπικός υπολογιστής.

Πέρα, λοιπόν, από την αποθήκευση που μπορεί να παρέχει το νέφος σε μεγάλους όγκους δεδομένων, με τη χρήση του «λογισμικού ως υπηρεσία», είναι σε θέση να επεξεργαστεί και να αναλύσει μεγάλα δεδομένα σε πραγματικό χρόνο, παρέχοντας ολοκληρωμένες λύσεις που έχουν συστατικά βάσεων δεδομένων για την αποθήκευση και διαχείριση των μεγάλων δεδομένων, αλλά και λογισμικά συστήματα ανάλυσης, εικονικές μηχανές κ.ά. Λόγω της μεγάλης δημοφιλίας των λύσεων υπολογιστικού νέφους και της απαίτησης διαχείρισης μεγάλων δεδομένων που έχουν πλέον οι περισσότερες επιχειρήσεις, πάροχοι όπως η Amazon, η Google και η Microsoft προσφέρουν τις δικές τους λύσεις μεγάλων δεδομένων στο υπολογιστικό νέφος. Μάλιστα, με την επικράτηση συστημάτων που λαμβάνουν, ενοποιούν, μετασχηματίζουν και οπτικοποιούν τα δεδομένα σε πραγματικό χρόνο, έχουμε πλέον την επικράτηση του όρου της *αναλυτικής μεγάλων δεδομένων* (*big data analytics*).

1.4 Αναλυτική μεγάλων δεδομένων

Η αναλυτική ή ανάλυση μεγάλων δεδομένων έχει ως στόχο την ταχεία επεξεργασία ετερογενών δεδομένων μεγάλου όγκου που προέρχονται από πληθώρα διαφορετικών πηγών, έτσι ώστε να συμβάλλουν στη λήψη αποφάσεων και να παρέχουν επιχειρηματική διορατικότητα. Με άλλα λόγια, μέσω της ανάλυσης των μεγάλων δεδομένων, ο στόχος είναι να αυξήσουμε την αξία αυτών των δεδομένων και να οδηγήσουμε σε καλύτερες επιχειρηματικές λύσεις.

Στην καρδιά της αναλυτικής των μεγάλων δεδομένων έχουμε τεχνικές που προέρχονται από τους τομείς της μηχανικής μάθησης, της εξόρυξης γνώσης από δεδομένα, της αναγνώρισης προτύπων, της επιστήμης δεδομένων και της επιχειρηματικής ευφυΐας. Όλοι αυτοί οι όροι που άπτονται της μηχανικής μάθησης, ουσιαστικά, θεωρούν το ίδιο πρόβλημα από διαφορετικές οπτικές γωνίες. Ωστόσο, στην ουσία τους, όλες έχουν ως στόχο την εκπαίδευση της

μηχανής ώστε να μπορεί να βελτιώνεται καθώς λαμβάνει περισσότερα δεδομένα, με στόχο να ανακαλύπτει πρότυπα και νέα γνώση από εισερχόμενα δεδομένα ή και να προβλέπει τα αποτελέσματα πάνω σε άγνωστα δεδομένα.

Ο στόχος, λοιπόν, της ανάλυσης των δεδομένων είναι να ανακαλυφθούν μέσα από τη μελέτη των δεδομένων και την εκπαίδευση της μηχανής πρότυπα και μοντέλα που θα είναι:

- **Έγκυρα.** Θα πρέπει τα πρότυπα και τα μοντέλα να ταιριάζουν ή να ισχύουν και για νέα άγνωστα δεδομένα με έναν βαθμό βεβαιότητας, και όχι να βρίσκουν εφαρμογή μόνο στα δεδομένα τα οποία χρησιμοποιήθηκαν για την ανάλυση.
- **Χρήσιμα.** Θα πρέπει τα πρότυπα και τα μοντέλα αυτά να μπορούν να αξιοποιηθούν έτσι ώστε να οδηγήσουν σε αποφάσεις που θα αποδώσουν οφέλη ανάλογα με την εφαρμογή που έχουμε κάθε φορά.
- **Μη αναμενόμενα.** Τα πρότυπα και τα μοντέλα δεν θα πρέπει να είναι προφανή, δηλαδή δεν θα πρέπει να μας οδηγούν σε συμπεράσματα που μας ήταν ήδη γνωστά, αλλά θα πρέπει να ανακαλύπτουν άγνωστες και μη αναμενόμενες συσχετίσεις και ιδιότητες των δεδομένων.
- **Κατανοητά.** Ο άνθρωπος θα πρέπει να μπορεί να κατανοήσει και να ερμηνεύσει τα πρότυπα που προκύπτουν.

Κλασικές τεχνικές που χρησιμοποιούνται για την ανάλυση μεγάλων δεδομένων αποτελούν τεχνικές συσταδοποίησης, κατηγοριοποίησης, γενετικοί αλγόριθμοι, νευρωνικά δίκτυα, παλινδρόμηση, ανάλυση κοινωνικών δικτύων, επεξεργασία φυσικής γλώσσας, ανάλυση συναισθήματος, επεξεργασία σήματος, οπτικοποίηση δεδομένων και πολλές άλλες τεχνικές μηχανικής μάθησης. Γενικά, οι μέθοδοι που χρησιμοποιούνται διακρίνονται σε: *περιγραφικές μεθόδους* που έχουν στόχο την ανακάλυψη προτύπων που περιγράφουν τα δεδομένα, όπως η συσταδοποίηση, και *μεθόδους πρόβλεψης* που έχουν ως στόχο την πρόβλεψη ενός αποτελέσματος για άγνωστα δεδομένα με την αξιοποίηση μοντέλων και των γνωστών δεδομένων, όπως η κατηγοριοποίηση.

Στο πλαίσιο της αξιοποίησης της αναλυτικής δεδομένων από επιχειρηματικούς φορείς, μπορούμε να εντοπίσουμε μεταξύ άλλων τα παρακάτω πλεονεκτήματα [BBS22]:

- Συγκέντρωση και αξιοποίηση δεδομένων από διαφορετικές πηγές, όπως το διαδίκτυο, μέσα κοινωνικής δικτύωσης, επιχειρησιακές βάσεις δεδομένων, εξωτερικές πηγές, κ.ά.
- Αναγνώριση κρίσιμων σημείων που βρίσκονται κρυμμένα μέσα σε μεγάλα σύνολα δεδομένων και τα οποία μπορούν να αξιοποιηθούν για τη λήψη αποφάσεων.
- Εντοπισμός προβλημάτων που αφορούν συστήματα και επιχειρηματικές διαδικασίες σε πραγματικό χρόνο, ώστε να μπορέσουν να επιλυθούν πιο αποδοτικά.
- Διευκόλυνση παροχής υπηρεσιών και προϊόντων, ώστε να ανταποκριθούν και να ξεπεράσουν τις προσδοκίες των πελατών.
- Ανταπόκριση σε αιτήματα, ερωτήματα και παράπονα των πελατών σε πραγματικό χρόνο.

1.5 Εφαρμογές ανάλυσης μεγάλων δεδομένων

Προτού ολοκληρώσουμε αυτήν την εισαγωγή στα μεγάλα δεδομένα, θα περιγράψουμε πιο αναλυτικά κάποιες ενδεικτικές εφαρμογές ανάλυσής τους, ώστε να γίνουν πιο κατανοητές οι προκλήσεις που δημιουργούν, οι ιδιαιτερότητές τους, καθώς και οι ευκαιρίες που προσφέρουν.

1.5.1 Μεγάλα δεδομένα στην υγεία

Ο τομέας της υγείας διαθέτει μεγάλα σύνολα δεδομένων που διακρίνονται για την ποικιλομορφία τους. Έτσι, μεγάλο μέρος των δεδομένων αντιστοιχεί σε διαγνωστικούς ελέγχους ασθενών, όπως ακτινογραφίες, υπερηχογραφήματα, αποτελέσματα αιματολογικών αναλύσεων, κ.ά. Παράλληλα, υπάρχουν διαθέσιμα προσωπικά δεδομένα των ασθενών, δημογραφικά στοιχεία, η οικονομική τους κατάσταση, η απασχόλησή τους, οι γενικότερες συνθήκες διαβίωσής τους, καθώς και περιβαλλοντικοί παράγοντες που θα ήταν καλό να συσχετιστούν με τα δεδομένα των ασθενών. Υπάρχουν επίσης δεδομένα για τη φαρμακευτική αγωγή που ακολουθεί ο κάθε ασθενής και όποιες παρενέργειες και αποτελέσματα έχει αυτή η φαρμακευτική αγωγή. Αυτά τα δεδομένα μπορούν να συνδυαστούν με πληροφορίες φαρμακολογίας, αλληλεπιδράσεις φαρμάκων, κ.ο.κ. Τέλος, άλλα σχετικά δεδομένα αφορούν ασφαλιστικές πληροφορίες ασθενών, λογαριασμούς νοσηλείας, εξετάσεων και φαρμάκων, αλλά ακόμα και δεδομένα από μέσα κοινωνικής δικτύωσης ή μέσα μαζικής ενημέρωσης.

Γενικά, υπάρχουν διαθέσιμα δεδομένα που στο σύνολό τους μπορούν να καταγράψουν ένα αναλυτικό ιστορικό του ασθενούς σχετικά με την παθολογία του, τις αγωγές που του έχουν χορηγηθεί και την ανταπόκρισή του, αλλά και να συνδυαστούν με άλλα σχετικά δεδομένα και να βοηθήσουν στη δημιουργία μιας προσωποποιημένης, πιο αποτελεσματικής θεραπευτικής αγωγής για τον ασθενή.

Από την περιγραφή των δεδομένων, είναι φανερό ότι τα διακρίνει μεγάλη ποικιλία. Από εικόνες όπως οι ακτινογραφίες, σε σχεσιακά δεδομένα όπως αποτελέσματα αιματολογικών εξετάσεων, και συνδεδεμένα δεδομένα όπως οι αλληλεπιδράσεις φαρμάκων. Ακόμη, τα δεδομένα αυτά είναι συνήθως κατανεμημένα, για παράδειγμα αποτελέσματα διαγνωστικών εξετάσεων μπορεί να προέρχονται από διάφορα νοσοκομεία, κλινικές ή διαγνωστικά ιατρεία. Τέλος, ένα ακόμα σημαντικό χαρακτηριστικό των δεδομένων αυτών είναι ότι μπορούν και θα πρέπει να παρακολουθούνται και να καταγράφονται σε πραγματικό χρόνο.

Ένα τέτοιο παράδειγμα είναι η καταγραφή σε πραγματικό χρόνο των παραμέτρων υγείας ενός ασθενούς, όπως η αρτηριακή πίεση, ο καρδιακός παλμός και τα επίπεδα σακχάρου μέσω μιας *έξυπνης φορητής συσκευής (smart wearable device)*. Τα δεδομένα αυτά γίνονται αυτομάτως γνωστά στον θεράποντα ιατρό, για παράδειγμα μέσω της μεταφόρτωσής τους σε κάποιο εξυπηρετητή στο υπολογιστικό νέφος, ώστε να είναι σε θέση να παρέμβει εγκαίρως και να προσαρμόσει για παράδειγμα τη φαρμακευτική αγωγή του ασθενούς χωρίς να χρειαστεί ο τελευταίος να πραγματοποιήσει κάποια επίσκεψη στον ιατρό του. Επιπλέον, η παραπάνω προσέγγιση καταγραφής και παρακολούθησης σε πραγματικό χρόνο μέσω ενός αυτόματου συστήματος μπορεί να προκαλέσει την έγκαιρη ιατρική παρέμβαση σε περίπτωση έκτακτης ανάγκης του ασθενούς.

Πέρα από τα οφέλη ως προς τη φροντίδα μεμονωμένων ασθενών, είναι προφανή και πολλαπλά και τα οφέλη που αποκομίζονται από τον συνδυασμό αυτών των δεδομένων για μεγάλο αριθμό ασθενών. Έτσι, ο συνδυασμός των συμπτωμάτων που εμφανίζουν ασθενείς σε συσχέτιση με περιβαλλοντικούς παράγοντες και κλιματικές συνθήκες βοηθά στην εξαγωγή συμπερασμάτων για το ποιες παθολογικές καταστάσεις επηρεάζονται περισσότερο από τέτοιους περιβαλλοντικούς και κλιματολογικούς παράγοντες, αλλά και για το ποιων παθήσεων η έξαρση πιθανώς αναμένεται σε κάθε περίπτωση. Ακόμα, ο συνδυασμός των δεδομένων των αποτελεσμάτων μιας φαρμακευτικής αγωγής σε πολλαπλούς ασθενείς μπορεί να προσφέρει συμπεράσματα για την αποτελεσματικότητα της αγωγής και να οδηγήσει στον σχεδιασμό πιο αποδοτικών ολοκληρωμένων στρατηγικών για την αντιμετώπιση μιας ασθένειας.

Πρόσφατο παράδειγμα της χρήσης των μεγάλων δεδομένων στον τομέα της υγείας αποτελεί και η αντιμετώπιση της πανδημίας COVID-19 τα τελευταία χρόνια [AAKO22, AHV20]. Η ταχύτατη εξάπλωση της ασθένειας παγκοσμίως δημιούργησε μια πολύ σημαντική κατανεμημένη πηγή πληροφορίας και γνώσης που ήταν αναγκαίο να εκμεταλλευτούμε άμεσα για να καταπολεμηθεί αποτελεσματικά αυτή η απειλή στην παγκόσμια υγεία. Τα δεδομένα αυτά χρησιμοποιήθηκαν και χρησιμοποιούνται ακόμη για την έρευνα σχετικά με τον ιό και την ανάπτυξη μεθόδων και μέτρων για την αντιμετώπιση του ίδιου του ιού αλλά και των συνεπειών του.

Συνεπώς, ανέκυψε η άμεση και επείγουσα απαίτηση για την αποθήκευση και ανάλυση αυτού του μεγάλου όγκου δεδομένων, χρησιμοποιώντας διάφορες τεχνολογίες αποθήκευσης και ανάλυσης μεγάλων δεδομένων. Τα δεδομένα που καταγράφονται είναι πολύ μεγάλα σε όγκο, ετερογενή, παράγονται συνεχώς με καταιγιστικούς ρυθμούς από κατανεμημένες πηγές σε παγκόσμια κλίμακα και χαρακτηρίζονται από μεγάλη ποικιλία. Διαθέτουν, δηλαδή, όλα τα χαρακτηριστικά που περιγράφουν τα μεγάλα δεδομένα. Έτσι, αρχικά καταγράφηκαν πληροφορίες για την τοποθεσία και τις μετακινήσεις των ασθενών, συμπτώματα και φυσιολογία των ασθενών, καθώς και άλλες παθήσεις τους, πλήθη κρουσμάτων, νοσηλείων και θανάτων, αποτελέσματα πειραματικών φαρμακευτικών αγωγών και παρενέργειες, αλλά και ερευνητικά αποτελέσματα και σχετική βιβλιογραφία από ερευνητές και ιατρούς σε όλον τον κόσμο που συνετέλεσαν συνολικά στην άμεση διαμόρφωση στρατηγικών για την αντιμετώπιση της πανδημίας σε πολλά επίπεδα, όπως μέτρα περιορισμού μετακινήσεων και επαφών, ανάπτυξη στρατηγικών εκτεταμένων ελέγχων, αλλά και την ταχύτατη ανάπτυξη αποτελεσματικών εμβολίων που θεωρήθηκε ένα από τα πολύ σημαντικά επιτεύγματα του επιστημονικού κόσμου.

1.5.2 Μεγάλα δεδομένα στο ηλεκτρονικό εμπόριο και στα συστήματα συστάσεων

Στον τομέα του ηλεκτρονικού εμπορίου (e-commerce) μεγάλος όγκος δεδομένων δημιουργείται μέσα από τις δραστηριότητες των πελατών στον παγκόσμιο ιστό, τη χρήση μέσων κοινωνικής δικτύωσης, καθώς και τη χρήση κινητών εφαρμογών. Η ανάλυση όλων αυτών των δεδομένων μπορεί να δώσει στις εταιρείες ηλεκτρονικού εμπορίου τη δυνατότητα να κατανοήσουν την αγοραστική συμπεριφορά του πελάτη και, κατά συνέπεια, να τους δώσει τη δυνατότητα να σχεδιάσουν στρατηγικές μάρκετινγκ, προώθησης προϊόντων και διαφήμισης προσαρμοσμένες κατάλληλα στις προτιμήσεις του κάθε πελάτη. Ειδικότερα, κολοσσοί όπως Amazon, Google και Netflix συλλέγουν τεράστιο όγκο δεδομένων που μπορούν να αξιοποι-

ήσουν για να παραγάγουν προσωποποιημένες συστάσεις για τους χρήστες σε πραγματικό χρόνο, που αποτελεί ένα πολύπλοκο και απαιτητικό πρόβλημα το οποίο έχει συγκεντρώσει πολύ μεγάλο ερευνητικό και εμπορικό ενδιαφέρον εξαιτίας των κερδών που μπορεί να αποφέρει.

Τα δεδομένα εκτός από μεγάλα σε όγκο έχουν και πάλι το χαρακτηριστικό της ποικιλομορφίας και συλλέγονται από ετερογενείς κατανομημένες πηγές και σε πραγματικό χρόνο, καταγράφοντας τις ενέργειες των πελατών. Πιο αναλυτικά, τα δεδομένα που συλλέγονται συμπεριλαμβάνουν:

- αναλυτικά δεδομένα ιστοσελίδων, όπως δεδομένα περιήγησης των πελατών σε ιστότοπους ηλεκτρονικού εμπορίου, κίνηση καλαθιού αγορών, χρόνους παραμονής στις διάφορες ιστοσελίδες του ιστότοπου, καθώς και χρόνους μεταξύ της επίσκεψης στην αρχική σελίδα και μιας αγοράς, αγορές, βαθμολογίες και κριτικές σε προϊόντα,
- στοιχεία πελατών, όπως δημογραφικά στοιχεία, φύλο, ηλικία, κ.ά., αλλά και γεωγραφικά δεδομένα της τοποθεσίας ή και της πορείας των πελατών που παρέχονται μέσα από σχετικές υπηρεσίες,
- δεδομένα μέσων κοινωνικής δικτύωσης, σχόλια και κριτικές σε φόρουμ και κοινωνικά δίκτυα, καθώς και φιλίες και ενέργειες των πελατών στα μέσα κοινωνικής δικτύωσης, και
- τάσεις στην αγορά και άλλα δεδομένα επικαιρότητας, όπως εορτασμούς, αργίες, έκτακτα γεγονότα, κ.ά.

Ο συνδυασμός όλων αυτών των δεδομένων μπορεί να προσφέρει πολλά οφέλη στις επιχειρήσεις, όπως:

- βελτίωση παροχής προσωποποιημένων υπηρεσιών και συστάσεων,
- ανάπτυξη στρατηγικών στοχευμένης διαφήμισης και μάρκετινγκ,
- βελτιστοποίηση τιμών και αύξηση πωλήσεων μέσα από προγράμματα ανταμοιβών, βελτίωση της εμπειρίας των πελατών και κατάλληλα προσαρμοσμένες προσφορές,
- πρόβλεψη τάσεων και ζήτησης των προϊόντων που μπορεί να ωφελήσει και στο μάρκετινγκ των εταιρειών αλλά και στη διαχείριση παραγωγής και αποθεμάτων, και
- βελτίωση της ασφάλειας των αγορών μέσα από την αναγνώριση ύποπτων συμπεριφορών που μπορεί να οφείλονται σε απάτες.

1.5.3 Μεγάλα δεδομένα και το Διαδίκτυο των Πραγμάτων (IoT)

Αναφέραμε ήδη ότι μια πηγή μεγάλων δεδομένων αποτελεί και το Διαδίκτυο των Πραγμάτων, IoT, όπου αισθητήρες παράγουν διαρκώς τεράστιους όγκους δεδομένων.

Ας επικεντρωθούμε σε μία εφαρμογή του IoT σε βιομηχανικές μονάδες παραγωγής, όπου οι αισθητήρες καταγράφουν μετρήσεις σχετικά με θερμοκρασίες, πίεση, ταλάντωση και άλλες μετρικές μηχανημάτων και εξαρτημάτων. Πέρα από τα δεδομένα των αισθητήρων, δεδομένα γίνονται διαθέσιμα μέσα από αναφορές και επισημάνσεις των εργαζομένων στις βιομηχανίες, καθώς και άλλα δεδομένα σχετικά με χρόνους και κόστη παραγγελίας, μεταφοράς πρώτων υλών και άλλα δεδομένα σχετικά με τη λειτουργία της παραγωγικής μονάδας.

Ένας από τους στόχους σε μια βιομηχανική μονάδα παραγωγής είναι η ελαχιστοποίηση του κόστους που ανακύπτει για την επιδιόρθωση ή και αντικατάσταση εξοπλισμού που παρουσιάζει βλάβες, καθώς και η ελαχιστοποίηση του χρόνου εκτός λειτουργίας που θα απαιτήσει μια τέτοια επιδιόρθωση ή αντικατάσταση.

Η ανάλυση μεγάλων δεδομένων μπορεί να επιτελέσει σημαντικό ρόλο στην επίτευξη του παραπάνω στόχου. Τα συλλεγόμενα δεδομένα χρησιμοποιούνται για την ανάπτυξη προβλεπτικών μοντέλων συντήρησης του εξοπλισμού. Συγκεκριμένα, τα προβλεπτικά μοντέλα αξιοποιούν ιστορικά δεδομένα και δεδομένα πραγματικού χρόνου από τους αισθητήρες και προβλέπουν το πότε είναι πιθανό ο εξοπλισμός να παρουσιάσει βλάβη ή να χρειαστεί κάποια συντήρηση. Γενικότερα, τα συστήματα αυτά αναγνωρίζουν πρότυπα στα δεδομένα και ανωμαλίες και μπορούν να προβλέψουν την πιθανή βλάβη, ώστε να προγραμματιστούν οι προληπτικές ενέργειες που θα αποτρέψουν μεγάλους χρόνους και κόστη επισκευής. Ακόμη, μπορούν να παρέχουν προβλέψεις σχετικά με τον υπολειπόμενο χρόνο ζωής του εξοπλισμού, ώστε να επιτευχθεί με τον καλύτερο τρόπο και η αντικατάστασή του.

Σε τέτοια συστήματα η έγκαιρη πρόβλεψη, δηλαδή ο εντοπισμός ανωμαλιών ή αποκλίσεων από την τυπική συμπεριφορά των μηχανημάτων, είναι πολύ σημαντική για να επιτευχθεί ο στόχος. Άρα, είναι σημαντική η ταχεία ανάλυση των δεδομένων που παράγονται από τους αισθητήρες σε πραγματικό χρόνο, γεγονός που εισάγει προκλήσεις στον τρόπο αποθήκευσης και ανάλυσης αυτών των ποικιλόμορφων δεδομένων.

Πέρα από τις βιομηχανίες, το IoT παρέχει μεγάλα δεδομένα και στο πλαίσιο των *έξυπνων πόλεων* (*smart cities*). Δεδομένα συλλέγονται από αισθητήρες, όσον αφορά για παράδειγμα περιβαλλοντολογικές μετρήσεις, καιρικές συνθήκες και μετρήσεις ρύπων, από κάμερες που χρησιμοποιούνται στους δρόμους για τη ρύθμιση της κυκλοφορίας αλλά και κάμερες ασφαλείας, από τα συστήματα μεταφορών, συστήματα μαζικών μέσων μεταφοράς αλλά και ιδιωτικά οχήματα, καθώς και από τις ίδιες τις αλληλεπιδράσεις των χρηστών. Τα δεδομένα αυτά είναι προφανώς και πάλι μεγάλου όγκου, ποικιλόμορφα, παράγονται με ταχύτατους ρυθμούς και χρήζουν επεξεργασίας και ανάλυσης σε πραγματικό χρόνο. Η αξία τους επίσης είναι εμφανής μέσα από τις πολλές εφαρμογές που βρίσκει η ανάλυσή τους σε μια έξυπνη πόλη, όπως οι έξυπνες μεταφορές, η έξυπνη ενέργεια, το έξυπνο περιβάλλον, και η έξυπνη ασφάλεια [Ces23].

Στις έξυπνες μεταφορές ο συνδυασμός των δεδομένων κυκλοφορίας στο οδικό δίκτυο και των μετακινήσεων των πολιτών μπορεί από κοινού να προσφέρει πολύτιμες πληροφορίες για την ανάπτυξη περιγραφικών και προβλεπτικών μοντέλων της κίνησης στο οδικό δίκτυο της πόλης. Αυτά τα μοντέλα μπορούν με τη σειρά τους να χρησιμοποιηθούν για να προβλεφθούν και να αποφευχθούν κυκλοφοριακά προβλήματα, αλλά και για να σχεδιαστούν και να προγραμματιστούν καλύτερα τα δρομολόγια των μέσων μαζικής μεταφοράς για να εξυπηρετούν καλύτερα τις ανάγκες των πολιτών, αποσυμφορώντας την κυκλοφορία στο οδικό

δίκτυο. Ακόμη, εφαρμογές διαμοιρασμού διαδρομών και σύστασης της καλύτερης διαδρομής σε πραγματικό χρόνο αναπτύσσονται πάνω στην εκμετάλλευση τέτοιων δεδομένων.

Όσον αφορά την έξυπνη ενέργεια, ο στόχος είναι μέσω της ανάλυσης των μεγάλων δεδομένων να σχεδιαστούν ενεργειακά αποδοτικές υποδομές που θα έχουν μειωμένο ενεργειακό κόστος και θα ελαττώνουν την κατανάλωση ενέργειας σε ένα αστικό περιβάλλον. Χαρακτηριστικό παράδειγμα αποτελεί ο αποδοτικός διαμοιρασμός πόρων σε ένα σύστημα υπολογιστικού νέφους, βασισμένου στη ζήτηση και τις προβλέψεις που μπορούν να γίνουν από μοντέλα μηχανικής μάθησης, έτσι ώστε να πετύχουμε τη χαμηλότερη κατανάλωση ενέργειας για την κάλυψη των απαιτήσεων των καταναλωτών.

Παράλληλα, στο έξυπνο περιβάλλον η συλλογή και ανάλυση δεδομένων σχετικά με το περιβάλλον και καιρικά φαινόμενα αποτελούν πολύ σημαντικά εφόδια για την κατανόηση των φυσικών φαινομένων, που με την κλιματική αλλαγή γίνονται ολοένα και εντονότερα, και το πώς αυτά επηρεάζονται από άλλους παράγοντες, όπως είναι η ποιότητα του αέρα σε ένα αστικό περιβάλλον, η ρύπανση, η χρήση γης, κ.λ.π. Αντίστοιχες τεχνικές μπορούν να χρησιμοποιηθούν και στη βελτίωση των καλλιεργειών και τη σύσταση στους αγρότες αποδοτικών καλλιεργειών προσαρμοσμένων στις νέες περιβαλλοντικές συνθήκες.

Τέλος, η ασφάλεια σε μια έξυπνη πόλη μπορεί να ωφεληθεί πολύπλευρα από την εκμετάλλευση μεγάλων δεδομένων. Δεδομένα σχετικά με εγκλήματα συλλέγονται σε βάσεις δεδομένων από αστυνομικά τμήματα, και μοντέλα μπορούν να εξαχθούν ώστε να προβλέψουν την εγκληματικότητα σε διαφορετικές περιοχές ενός αστικού κέντρου. Τα μοντέλα αυτά μπορούν να αξιοποιηθούν για την καλύτερη διανομή των αστυνομικών δυνάμεων, τον σχεδιασμό περιπολιών, κ.λπ. Παράλληλα, αντίστοιχα δεδομένα μπορούν να αξιοποιηθούν για την αντιμετώπιση της τρομοκρατίας αλλά και κυβερνοεπιθέσεων. Επιπρόσθετα, σε μια άλλη διάσταση, μεγάλα δεδομένα μπορούν να αξιοποιηθούν για την αντιμετώπιση φυσικών καταστροφών, τόσο όσον αφορά την πρόβλεψη και την έγκαιρη αντιμετώπιση, όπως στις περιπτώσεις πυρκαγιάς ή πλημμύρας, αλλά και όσον αφορά τις σωστικές επεμβάσεις μετά από μια καταστροφή, ώστε να σωθούν περισσότερες ζωές.

1.5.4 Μεγάλα δεδομένα στην εκπαίδευση

Η εκπαίδευση είναι άλλος ένας τομέας στον οποίο η ανάλυση των μεγάλων δεδομένων βρίσκει πολλές και σημαντικές εφαρμογές. Οι πιο βασικές εφαρμογές επικεντρώνονται στη συμπεριφορά του εκπαιδευόμενου καθώς και στην απόδοσή του, αλλά εντοπίζονται και εφαρμογές που έχουν ως στόχο γενικότερες βελτιώσεις στο εκπαιδευτικό σύστημα [MIBY20].

Τα δεδομένα στον τομέα της εκπαίδευσης παράγονται μέσα από διδακτικές και μαθησιακές δραστηριότητες, διαδικτυακά μαθήματα, διαδικτυακές πλατφόρμες μάθησης (online learning management systems), αλλά και εκπαιδευτικά ιδρύματα, όπως πανεπιστήμια και σχολεία. Πλέον, οι εκπαιδευτές έχουν τη δυνατότητα να παρακολουθούν την απόδοση των εκπαιδευομένων, τα πρότυπα μάθησης που ακολουθούν και να παρέχουν έγκαιρη ανάδραση.

Ακόμη, σε πολλά διαδικτυακά συστήματα μάθησης οι προτιμήσεις των εκπαιδευομένων καθώς και το υπόβαθρό τους λαμβάνονται υπόψη, ώστε να παραχθούν ειδικά προγράμματα μάθησης που ταιριάζουν καλύτερα στις ανάγκες τους. Με την εισαγωγή τεχνικών ανάλυσης για την αναγνώριση προτύπων αλλά και μηχανικής μάθησης, τα συστήματα μπορούν να

«μαθαίνουν» και να προσαρμόζονται στις προσωποποιημένες ανάγκες κάθε εκπαιδευόμενου ακόμα και κατά τη διάρκεια της εκπαιδευτικής διαδικασίας.

1.6 Περαιτέρω μελέτη

Τα μεγάλα δεδομένα βρίσκονται παντού πλέον και πολλές τεχνολογίες και προσεγγίσεις έχουν αναπτυχθεί για την αποδοτική επεξεργασία και ανάλυσή τους. Σε αυτό το κεφάλαιο κάναμε μια μικρή εισαγωγή στα μεγάλα δεδομένα, αλλά στα επόμενα κεφάλαια θα μελετήσουμε σε βάθος πολλά εργαλεία, τεχνολογίες και αλγορίθμους για την ανάλυση και την επεξεργασία τους. Επίσης, για να γίνει κατανοητή η σημασία των μεγάλων δεδομένων και ποια κέρδη μπορεί να αποφέρει η αξιοποίησή τους, παρουσιάσαμε και κάποιες ενδεικτικές εφαρμογές που αξιοποιούν μεγάλα δεδομένα σε πολλαπλά και διαφορετικά πεδία εφαρμογής, από την υγεία μέχρι το ηλεκτρονικό εμπόριο, και από την εκπαίδευση μέχρι το Διαδίκτυο των Πραγμάτων. Ο ενδιαφερόμενος αναγνώστης μπορεί να βρει εφαρμογές σε περισσότερα πεδία, όπως είναι η ασφάλεια, η βιοπληροφορική, οι οικονομικές και τραπεζικές υπηρεσίες, κ.ά. στο βιβλίο [CAM⁺19]. Εφαρμογές που επικεντρώνονται περισσότερο σε κειμενικά δεδομένα και δεδομένα κοινωνικών δικτύων αναπτύσσονται στο [THA⁺15]. Επίσης, στο [MD13] μπορεί κανείς να βρει μια αναλυτική έρευνα για εφαρμογές των μεγάλων δεδομένων στην υγεία, στο [KTC14] περιγράφεται η αξιοποίηση των μεγάλων δεδομένων στην ηλεκτρονική διακυβέρνηση, ενώ στο [JHSS21] ο αναγνώστης μπορεί να δει μια ωραία σύνοψη των εφαρμογών των μεγάλων δεδομένων στο Industry 4.0.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [AAKO22] Joseph Bamidele Awotunde, Emmanuel Abidemi Adeniyi, Paul Oluwatoba Kolawole, and Roseline Oluwaseun Ogundokun. 8 - application of big data in covid-19 epidemic. In Utku Kose, Deepak Gupta, Victor Hugo C. de Albuquerque, and Ashish Khanna, editors, *Data Science for COVID-19*, pages 141–165. Academic Press, 2022.
- [AHV20] Ibrahim Haleem Khan Abid Haleem, Mohd. Javaid and Raju Vaishya. Significant applications of big data in covid-19 pandemic. *Indian J Orthop*, 54(4):526–528, 2020.
- [BBS22] Endrit Mëziu Blend Berisha and Isak Shabani. Big data analytics in cloud computing: an overview. *Journal of Cloud Computing*, 11(24), 2022.
- [BCD16] Rajkumar Buyya, R.N. Calheiros, and A.V. Dastjerdi. *Big Data: Principles and Paradigms*. 06 2016.
- [BP98] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Comput. Networks*, 30(1-7):107–117, 1998.
- [CAM⁺19] Prabhu Csr, Sreevallabh Aneesh, Aditya Mogadala, Rohit Ghosh, and Jenila Livingston. *Big Data Analytics: Systems, Algorithms, Applications*. 01 2019.
- [CDG⁺06] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach,

- Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 205–218, 2006.
- [Ces23] Eugenio Cesario. Big data analytics and smart cities: applications, challenges, and opportunities. *Frontiers in Big Data*, 6, 2023.
- [Com] European Parliament Committes. Big data. europarl.europa.eu/committees/en/big-data/product-details/20161026CDT00981.
- [Cou13] National Research Council. *Frontiers in Massive Data Analysis*. The National Academies Press, Washington, DC, 2013.
- [DG04] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *OSDI'04: Sixth Symposium on Operating System Design and Implementation*, pages 137–150, San Francisco, CA, 2004.
- [DQR12] Jens Dittrich and Jorge-Arnulfo Quiané-Ruiz. Efficient big data processing in hadoop mapreduce. *Proceedings of the VLDB Endowment*, 5(12):2014–2015, 2012.
- [ela] Elasticsearch. <https://www.elastic.co/>.
- [Geo11] Lars George. *HBase: the definitive guide: random access to your planet-size data*. ” O'Reilly Media, Inc.”, 2011.
- [GGL03] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, pages 20–43, Bolton Landing, NY, 2003.
- [GK17] Purva Grover and Arpan Kumar Kar. Big Data Analytics: A Review on Theoretical Contributions and Tools Used in Literature. *Global Journal of Flexible Systems Management*, 18(3):203–229, September 2017.
- [GP14] Trey Grainger and Timothy Potter. *Solr in action*. Manning Publications Co., 2014.
- [GT15] Clinton Gormley and Zachary Tong. *Elasticsearch: the definitive guide: a distributed real-time search and analytics engine*. ” O'Reilly Media, Inc.”, 2015.
- [Had] Apache Hadoop. <http://hadoop.apache.org/>. Accessed: 2021-09-30.
- [hiv] Apache hive. <http://hive.apache.org/>.
- [HKJR10] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. {ZooKeeper}: Wait-free coordination for internet-scale systems. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, 2010.
- [IHB⁺12] Mohammad Islam, Angelo K. Huang, Mohamed Battisha, Michelle Chiang, Santhosh Srinivasan, Craig Peters, Andreas Neumann, and Alejandro Abdelnur. Oozie: towards a scalable workflow management system for hadoop. In *Proceedings of the 1st ACM SIGMOD Workshop on Scalable Workflow Execution Engines and Technologies*, pages 1–10, 2012.

- [JHSS21] Mohd Javaid, Abid Haleem, Ravi Pratap Singh, and Rajiv Suman. Significant applications of big data in industry 4.0. *Journal of Industrial Integration and Management*, 6(04):429–447, 2021.
- [KTC14] Gang-Hoon Kim, Silvana Trimi, and Ji-Hyong Chung. Big-data applications in the government sector. *Communications of the ACM*, 57(3):78–85, 2014.
- [luc] Apache <https://lucene.apache.org/>.
- [MD13] Travis B Murdoch and Allan S Detsky. The inevitable application of big data to health care. *Jama*, 309(13):1351–1352, 2013.
- [MHGG10] Michael McCandless, Erik Hatcher, Otis Gospodnetić, and O Gospodnetić. *Lucene in action*, volume 2. Manning Greenwich, 2010.
- [MIBY20] Liyana Shuib Maria Ijaz Baig and Elaheh Yadegaridehkordi. Big data in education: a state of the art, limitations, and future research directions. *Int J Educ Technol High Educ*, 17, 2020.
- [Nut] Apache nutch. <https://nutch.apache.org/>.
- [OFAD11] Sean Owen, B Ellen Friedman, Robin Anil, and Ted Dunning. *Mahout in action*. Simon and Schuster, 2011.
- [ORS⁺08] Christopher Olston, Benjamin Reed, Utkarsh Srivastava, Ravi Kumar, and Andrew Tomkins. Pig latin: A not-so-foreign language for data processing. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, SIGMOD '08, page 1099–1110, New York, NY, USA, 2008. Association for Computing Machinery.
- [SKRC10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [sol] Apache solr. <https://solr.apache.org/>.
- [Sta21] Statista. <https://www.statista.com/statistics/871513/worldwide-data-created>, 2021.
- [sub] Apache submarine. <https://submarine.apache.org/>.
- [THA⁺15] Marcello Trovati, Richard Hill, Ashiq Anjum, Shao Ying Zhu, and Lu Liu, editors. *Big-Data Analytics and Cloud Computing: Theory, Algorithms and Applications*. Springer Cham, 2015.
- [VMD⁺13] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Balde-schieler. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th Annual Symposium on Cloud Computing*, SOCC '13, New York, NY, USA, 2013. Association for Computing Machinery.

- [ZXW⁺16] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, and et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

ΚΕΦΑΛΑΙΟ 2

ΚΑΤΑΝΕΜΗΜΕΝΑ ΣΥΣΤΗΜΑΤΑ ΑΡΧΕΙΩΝ

Περιεχόμενα

2.1	Εισαγωγή	24
2.2	Λύσεις ανοικτού κώδικα	24
2.2.1	GlusterFS	25
2.2.2	Ceph	27
2.2.3	Alluxio	29
2.3	Λύσεις αποθήκευσης νέφους	31
2.3.1	OpenStack Swift	31
2.3.2	Συνδρομητικές υπηρεσίες	32
2.4	Apache Hadoop Distributed File System	33
2.4.1	Αρχιτεκτονική του Apache Hadoop	34
2.4.2	HDFS: το κατανεμημένο σύστημα αρχείων του Hadoop	34
2.4.3	Έννοιες και οντότητες του HDFS	36
2.4.4	Λειτουργίες αρχείων και διαχείριση blocks	42
2.4.5	Εντολές στο HDFS	43
2.5	Συζήτηση	44
2.6	Περαιτέρω μελέτη	45
	BIBΛΙΟΓΡΑΦΙΑ	46



Σύνοψη

Το κεφάλαιο αυτό ασχολείται με τα εξής θέματα:

- Παρουσίαση κατανεμημένων συστημάτων αρχείων ανοικτού κώδικα.
- Παρουσίαση κατανεμημένων συστημάτων αρχείων στο Νέφος.
- Μελέτη περίπτωσης του HDFS (Hadoop Distributed File System).

Προαπαιτούμενη γνώση

Για την απρόσκοπτη μελέτη αυτού του κεφαλαίου, ο αναγνώστης θα πρέπει να κατέχει βασικές γνώσεις λειτουργικών συστημάτων, αρχιτεκτονικής υπολογιστών, καθώς και δικτύων υπολογιστών.

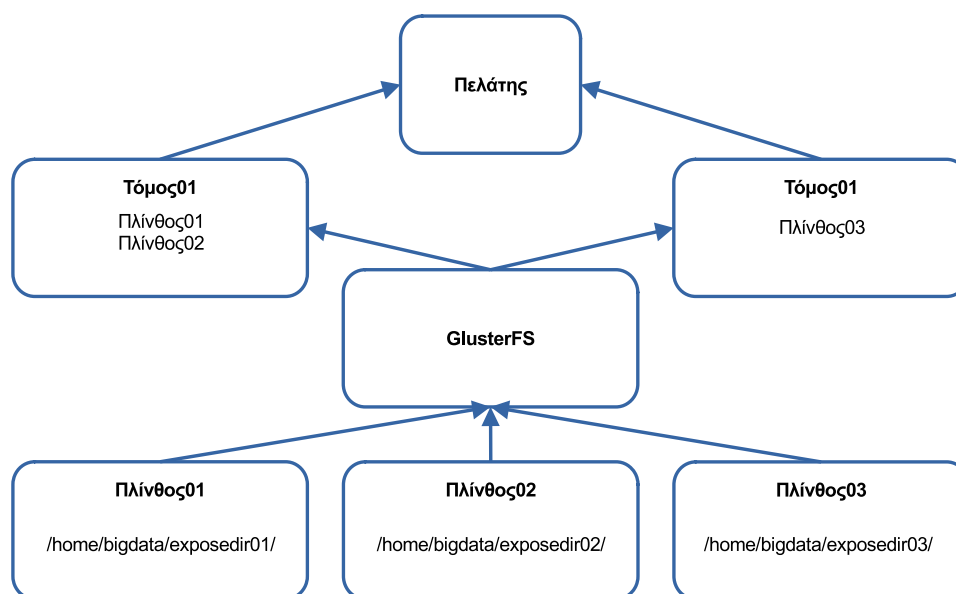
2.1 Εισαγωγή

Ένα κατανεμημένο σύστημα αρχείων επιτρέπει την αποθήκευση και την ανάκτηση αρχείων σε περισσότερες από μία υπολογιστικές μηχανές με τρόπο κατά τον οποίο ο χρήστης να μην χρειάζεται να γνωρίζει τον φυσικό χώρο ή τον τρόπο αποθήκευσης των αρχείων αυτών. Στην εποχή των μεγάλων δεδομένων, έχουν αναπτυχθεί διάφορες λύσεις αυτής της μορφής. Θα περιγράψουμε in-house λύσεις ανοικτού κώδικα οι οποίες επιτρέπουν τη δημιουργία ενός κατανεμημένου συστήματος αρχείων σε μία ιδιόκτητη συστάδα, αλλά και λύσεις αποθήκευσης στο Νέφος. Τέλος, θα εμβαθύνουμε με μία μελέτη περίπτωσης στο HDFS, το κατανεμημένο σύστημα αρχείων του Hadoop, παρουσιάζοντας ενδελεχώς την αρχιτεκτονική, τη λειτουργία, αλλά και πρακτικά θέματα.

Η αποθήκευση σε ένα κατανεμημένο σύστημα αρχείων μπορεί να αφορά αποθήκευση αρχείων (file storage), αποθήκευση αντικειμένων (object storage) και αποθήκευση block (block storage). Στην πρώτη περίπτωση, πρόκειται για αρχεία οργανωμένα ιεραρχικά, όπως στο τοπικό μας σύστημα αρχείων. Στη δεύτερη περίπτωση, αναφερόμαστε σε αρχεία οποιουδήποτε τύπου, είτε κειμενικού είτε δυαδικού, τα οποία είναι προσπελάσιμα μέσω ενός μοναδικού αναγνωριστικού, καθώς δεν υπάρχει ιεραρχία αποθήκευσης όπως στην περίπτωση της αποθήκευσης αρχείων, όπου ένα αντικείμενο αποτελεί έναν συνδυασμό δεδομένων, μεταδεδομένων και του μοναδικού αναγνωριστικού. Τέλος, με την αποθήκευση block αναφερόμαστε σε κάποια συσκευή προσβάσιμη μέσω δικτύου η οποία μπορεί να αποθηκεύσει δεδομένα σε μορφή block, όπως για παράδειγμα ένας δίσκος.

2.2 Λύσεις ανοικτού κώδικα

Στο τμήμα αυτό θα παρουσιάσουμε κατανεμημένα συστήματα αρχείων ανοικτού κώδικα. Συγκεκριμένα, θα δούμε τα GlusterFS, Ceph, και Alluxio.



Σχήμα 2.1: Αρχιτεκτονική και πλίνθοι στο GlusterFS.

2.2.1 GlusterFS

Το GlusterFS είναι ένα επεκτάσιμο, καταναμημένο σύστημα αρχείων υποστηριζόμενο από το RedHat Linux. Συγκεντρώνει πόρους αποθήκευσης δίσκου από πολλούς διαφορετικούς διακομιστές σε έναν ενιαίο χώρο ονομάτων [Glu]. Το GlusterFS χρησιμοποιεί το σύστημα αρχείων XFS για την αποθήκευση δεδομένων σε δίσκους και είναι προσβάσιμο μέσω NFS και SMB.

Έχει τη δυνατότητα κλιμάκωσης της τάξης των petabytes, καθώς και της διαχείρισης χιλιάδων πελατών. Είναι συμβατό με POSIX και δεν απαιτεί εξειδικευμένο υλικό. Επιτρέπει τη βελτιστοποίηση για διαφορετικούς φόρτους εργασίας και είναι λογισμικό ανοικτού κώδικα.

2.2.1.1 Αρχιτεκτονική GlusterFS

Σε αντιδιαστολή με άλλα καταναμημένα συστήματα αρχείων, το GlusterFS δεν βασίζεται σε διακομιστή μεταδεδομένων. Εντοπίζει τα αρχεία αλγοριθμικά χρησιμοποιώντας αλγόριθμο ελαστικού κατακερματισμού [MMBSS20]. Η απουσία διακομιστή μεταδεδομένων επιλύει μία σειρά προβλημάτων. Έτσι, επιτρέπει την ευκολότερη κλιμάκωση, καθώς δεν υπάρχει ένα μοναδικό σημείο επικοινωνίας [DO13]. Αυτό, φυσικά, σημαίνει ότι δεν υπάρχει και ένα μοναδικό σημείο αστοχίας, η μη λειτουργία του οποίου θα καταστήσει το σύστημα μη λειτουργικό.

Η βασική μονάδα αποθήκευσης ονομάζεται πλίνθος (*brick*) και είναι ένας κατάλογος σε έναν διακομιστή. Ένας τόμος (*volume*) είναι μία λογική συλλογή από πλίνθους. Αυτό σημαίνει ότι η συλλογή αυτή δεν υλοποιείται φυσικά, αλλά εικονικά [BBP12]. Οι περισσότερες λειτουργίες διαχείρισης πραγματοποιούνται σε επίπεδο τόμου. Ένας μεταφραστής (*translator*) επεξεργάζεται τους τόμους. Μία δεξαμενή αποθήκευσης (*storage pool*) είναι ένα δίκτυο από διακομιστές αποθήκευσης, οι οποίοι εμπεριέχουν τους πλίνθους.

Στο Σχήμα 2.1 παρατηρούμε την αρχιτεκτονική του GlusterFS. Εδώ έχουμε τρεις πλίνθους (Πλίνθος01-03) οι οποίοι περιέχουν τα αρχεία τα οποία θα γίνουν διαθέσιμα στο σύστημα αρχείων. Οι τόμοι (Τόμος01-02) αποτελούν συλλογές από πλίνθους. Έτσι, στον τόμο «Τόμος01»

συλλέγονται οι πλίνθοι «Πλίνθος01» και «Πλίνθος02», ενώ στον τόμο «Τόμος02» ο πλίνθος «Πλίνθος03».

Το GlusterFS υποστηρίζει διαφορετικούς τύπους τόμων [Glu], καθένας από τους οποίους έχει διαφορετικές δυνατότητες.

Κατανεμημένος τόμος GlusterFS. Στους *κατανεμημένους τόμους (Distributed Volumes)*, τα αρχεία βρίσκονται σε διαφορετικούς πλίνθους, κάτι που σημαίνει ότι δεν μπορούν να υπάρχουν αντίγραφα των δεδομένων. Ένας τέτοιος τόμος, όπως είναι λογικό, κλιμακώνεται πολύ εύκολα, αλλά η αστοχία ενός πλίνθου θα οδηγήσει σε απώλεια δεδομένων. Για να αποφευχθεί κάτι τέτοιο, θα πρέπει να χρησιμοποιηθούν μηχανισμοί εκτός του GlusterFS (π.χ. υλικό).

Αναπαραχθείς τόμος GlusterFS. Στους *αναπαραχθέντες τόμους (Replicated Volumes)* αντίγραφα από όλα τα δεδομένα διατηρούνται σε όλους τους τόμους με πλήθος αντιγράφων οριζόμενο από τον πελάτη, κάτι που προϋποθέτει ελάχιστο αριθμό πλίνθων στο οποίο θα βρίσκεται το κάθε αρχείο, άρα και ελάχιστο αριθμό αντιγράφων του αρχείου. Έτσι, αν οριστούν τρία αντίτυπα για ένα αρχείο, θα πρέπει να υπάρχουν τουλάχιστον τρεις πλίνθοι στον τόμο. Ένας αναπαραχθείς τόμος προσφέρει αξιοπιστία και ανοχή σε σφάλματα.

Διάχυτος τόμος GlusterFS. Ένας *διάχυτος τόμος (Dispersed Volume)* είναι αντίστοιχος με έναν αναπαραχθέντα τόμο, με τη διαφορά ότι καταλαμβάνει λιγότερο χώρο αποθήκευσης, λόγω της χρήσης *κωδίκων διαγραφής (Erasure Codes)*. Το πλεονέκτημα μίας τέτοιας διάταξης είναι ότι μπορεί να προσφέρει αξιοπιστία με μικρότερο κόστος αποθηκευτικού χώρου. Το μειονέκτημα αυτής της διάταξης είναι ότι οι κώδικες διαγραφής λειτουργούν με συγκεκριμένα μεγέθη block, οπότε και κατά την τροποποίηση των αρχείων ή την προσάρτηση δεδομένων σε αυτά απαιτούνται υπολογιστικοί πόροι για τη διαδικασία ενημέρωσης [Glu].

Κατανεμημένος αναπαραχθείς τόμος GlusterFS. Αυτός ο τύπος, ο *κατανεμημένος αναπαραχθείς τόμος (Distributed Replicated Volume)* χρησιμοποιείται όταν ζητούμενο αποτελούν τόσο η υψηλή διαθεσιμότητα των δεδομένων όσο και η δυνατότητα κλιμάκωσης. Τα αρχεία κατανέμονται σε αναπαραχθέντα σύνολα τόμων. Σε αυτήν τη διάταξη υπάρχουν συγκεκριμένοι κανόνες. Το πλήθος των πλίνθων πρέπει να είναι πολλαπλάσιο του αριθμού των αντιγράφων και οι πλίνθοι πρέπει να ορίζονται με συγκεκριμένη σειρά, καθώς γειτονικοί πλίνθοι αποτελούν αντίγραφα ο ένας του άλλου.

Κατανεμημένος διάχυτος τόμος GlusterFS. Τέλος, ένας *κατανεμημένος διάχυτος τόμος (Distributed Dispersed Volume)* προσφέρει δυνατότητα κλιμάκωσης και ανοχή σε σφάλματα αντίστοιχα με την περίπτωση του κατανεμημένου αναπαραχθέντος τόμου, με τα δεδομένα, όμως, να καταλαμβάνουν λιγότερο χώρο μέσω της χρήσης κωδίκων διαγραφής.

Μεταφραστές. Οι μεταφραστές μετατρέπουν αιτήματα των χρηστών σε αιτήματα αποθήκευσης [Glu]. Οι κυριότεροι μεταφραστές πελατών είναι οι παρακάτω:

- Μεταφραστής FUSE, που προσφέρει διεπαφή με τον χώρο χρήστη (userspace).
- Μεταφραστής DHT, που βρίσκει τον σωστό πλίνθο που περιλαμβάνει το αντίστοιχο

αρχείο ή κατάλογο.

- Μεταφραστές πρωτοκόλλου πελάτη, που επικοινωνεί απευθείας με τη διεργασία του κάθε πλίνθου.

2.2.1.2 Λειτουργία του GlusterFS

Κατά τη λειτουργία του GlusterFS, σε κάθε κόμβο της συστάδας εκτελείται μία διεργασία διαχείρισης (glusterd). Μέσω του glusterd μπορεί να δημιουργηθεί μία έμπιστη δεξαμενή διακομιστών που θα αποτελείται από όλους τους κόμβους. Οι πλίνθοι μπορούν να δημιουργηθούν ως κατάλογοι σε αυτούς τους διακομιστές.

Όπως περιγράψαμε και πιο πάνω, ομάδες πλίνθων αποτελούν έναν τόμο. Με άλλα λόγια, ένας τόμος αποτελεί απλώς μία ομαδοποίηση ενός συνόλου πλίνθων, ο οποίος όμως μπορεί να προσαρτηθεί και να χρησιμοποιηθεί όπως και οποιοσδήποτε άλλος τόμος σε ένα λειτουργικό σύστημα.

2.2.2 Ceph

Το Ceph [Cep, WBM⁺06] είναι λογισμικό ανοικτού κώδικα το οποίο σχεδιάστηκε λαμβάνοντας υπόψη ότι συστήματα αποθήκευσης πολύ μεγάλης κλίμακας είναι δυναμικά, με κόμβους να συνδέονται, να αποσυνδέονται και να καταρρέουν κατά τη λειτουργία του συστήματος, υποστηρίζοντας μεταβλητούς φόρτους [WBM⁺06].

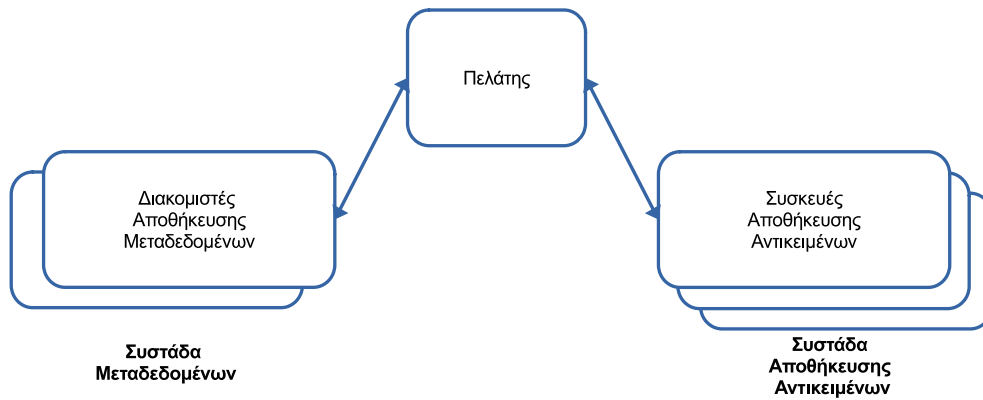
Προσφέρει τη δυνατότητα ενιαίας αποθήκευσης διαφορετικών τύπων δεδομένων. Συγκεκριμένα, προσφέρει δυνατότητα αποθήκευσης σε αντικείμενα, blocks, καθώς και σε αρχεία. Αυτό επιτυγχάνεται μέσω εξειδικευμένων APIs ανά περίπτωση, αλλά και της βιβλιοθήκης LibRados η οποία προσφέρει άμεση πρόσβαση στο RADOS, το σύστημα πάνω στο οποίο βασίζεται το Ceph.

2.2.2.1 Αρχιτεκτονική

Το Ceph, γενικά, αποτελείται από τρία κύρια συστατικά στοιχεία. Έτσι, υπάρχουν οι πελάτες (*clients*), οι οποίοι διαβάζουν και γράφουν δεδομένα, υπάρχει η συστάδα αποθήκευσης αντικειμένων, που αποτελείται από συσκευές αποθήκευσης αντικειμένων (*Object Storage Devices, OSDs*), και η συστάδα μεταδεδομένων για αποθήκευση των μεταδεδομένων των αρχείων, δηλαδή των ονομάτων αρχείων και των διαδρομών καταλόγων [WBM⁺06]. Παρακάτω, παρατίθενται οι βασικές έννοιες του Ceph.

Στο Ceph υπάρχουν πολλαπλές διεργασίες εποπτών (*Ceph Monitors*) για λόγους διαθεσιμότητας. Ο ρόλος τους είναι να διατηρούν αντίγραφα του *χάρτη συστάδας* και να τον παρέχουν στους πελάτες. Περισσότερες λεπτομέρειες αναφέρουμε παρακάτω. Ο δαίμονας *OSD* (*Ceph OSD Daemon*) ελέγχει την κατάσταση της *συσκευής αποθήκευσης αντικειμένων* (*Object Storage Device, OSD*) στην οποία εκτελείται και ενημερώνει τους επόπτες. Θα μπορούσαμε να πούμε αφαιρετικά, ότι οι επόπτες έχουν τον ρόλο Master ενώ οι δαίμονες τον ρόλο Slave.

Πέρα από αυτές τις οντότητες, υπάρχει και ο διαχειριστής *Ceph* (*Ceph Manager*), ο οποίος



Σχήμα 2.2: Επισκόπηση λειτουργίας του Ceph.

εποπτεύει όλες τις οντότητες της συστάδας και έχει τη δυνατότητα να συνδεθεί και να ενημερώσει εξωτερικά συστήματα παρακολούθησης και διαχείρισης. Τέλος, ο διακομιστής μεταδεδομένων Ceph (Ceph Metadata Server, MDS) χρησιμοποιείται στην περίπτωση αποθήκευσης αρχείων, οπότε και διατηρεί τα μεταδεδομένα τους [Ceph].

2.2.2.2 Λειτουργία

Ας δούμε τώρα τη λειτουργία του Ceph, επισκόπηση της οποίας απεικονίζεται στο Σχήμα 2.2. Όταν ένας πελάτης ζητά πρόσβαση σε αρχείο, επικοινωνεί με τη *συστάδα μεταδεδομένων* (Metadata Cluster). Οι διακομιστές μεταδεδομένων είναι διατεταγμένοι ιεραρχικά, ώστε να επιτυγχάνεται αποτελεσματική διαχείριση φόρτου. Για αναγνώσεις και εγγραφές δεδομένων, οι πελάτες επικοινωνούν απευθείας και με τη *συστάδα συσκευών αποθήκευσης αντικειμένων* (Object Storage Cluster). Το Ceph για τον εντοπισμό των αντικειμένων δεδομένων χρησιμοποιεί έναν αλγόριθμο κατακερματισμού που ονομάζεται CRUSH και υπολογίζει τους κόμβους στους οποίους βρίσκονται αποθηκευμένα τα δεδομένα.

Αποθήκευση δεδομένων. Όλα τα δεδομένα στο Ceph αποθηκεύονται ως αντικείμενα του RADOS [WLB07] (Reliable Autonomous Distributed Object Store - Αξιόπιστη Αυτόνομη Κατανεμημένη Αποθήκη Αντικειμένων), είτε πρόκειται για αποθήκευση block, είτε για αποθήκευση αρχείων, είτε για αποθήκευση αντικειμένων. Η αποθήκευση δεδομένων πραγματοποιείται στους δαίμονες OSD και είναι μη ιεραρχική, μην έχοντας καταλόγους και ιεραρχίες καταλόγων. Οι δαίμονες δημιουργούν αντίγραφα των δεδομένων μεταξύ τους με στόχο την ακεραιότητα και τη διαθεσιμότητα των δεδομένων.

Κάθε αντικείμενο που αποθηκεύεται αποτελείται από ένα αναγνωριστικό, το οποίο είναι μοναδικό για όλη τη συστάδα, τα δεδομένα αυτά καθαυτά του αρχείου και μεταδεδομένα με τη μορφή κλειδιού-τιμής. Τα μεταδεδομένα επιτρέπουν στον πελάτη να ορίσει και να ερμηνεύσει τις ιδιότητες του αντικειμένου, όπως π.χ. την ημερομηνία δημιουργίας,

Για τον εντοπισμό των δεδομένων, υπάρχει ο χάρτης συστάδας, που αναφέραμε παραπάνω, ο οποίος αποτελείται από τα παρακάτω συστατικά:

1. Τον χάρτη επιμελητών, που περιλαμβάνει πληροφορίες για τους επιμελητές, όπως τις διευθύνσεις τους κ.λπ.

2. Τον χάρτη OSD, με αντίστοιχες πληροφορίες για τους κόμβους OSD.
3. Τον χάρτη των ομάδων τοποθέτησης με δεδομένα όπως έκδοση, την έκδοση του χάρτη OSD και λεπτομέρειες σχετικά με την κάθε ομάδα τοποθέτησης.
4. Τον χάρτη CRUSH, που περιέχει μία λίστα με τους κόμβους OSD καθώς και δεδομένα φυσικής τοποθεσίας (ικρίωμα, διακομιστής κ.λπ.).
5. Τον χάρτη MDS, που περιέχει δεδομένα και πληροφορίες για τους διακομιστές αποθήκευσης μεταδεδομένων MDS.

2.2.3 Alluxio

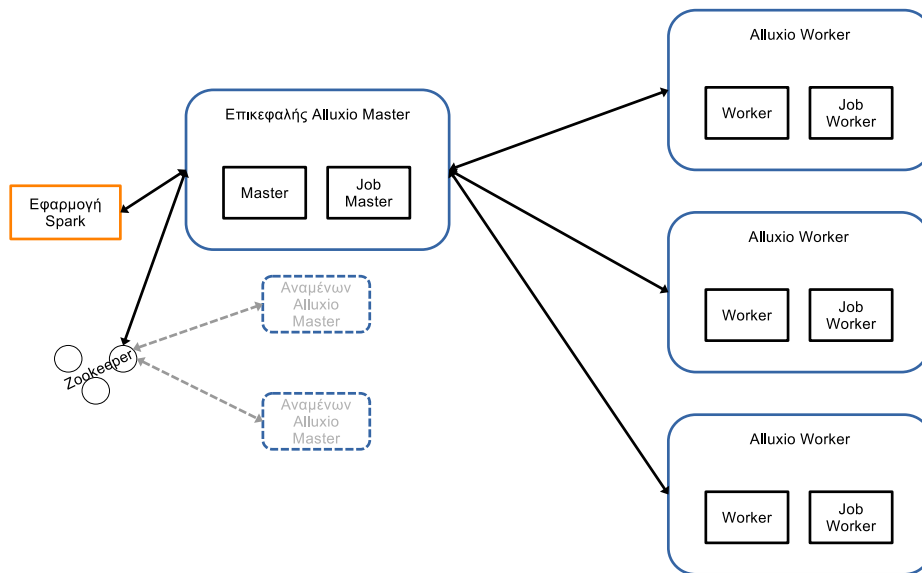
Το Alluxio, το οποίο αρχικά αναπτύχθηκε ως Tachyon, παρουσιάζει μία διαφορά σε σύγκριση με τα υπόλοιπα συστήματα αρχείων που συζητούμε σε αυτό το κεφάλαιο. Το Alluxio είναι ένα εντός μνήμης εικονικό κατανεμημένο σύστημα αρχείων (in-memory virtual distributed file system). Στόχος του Alluxio είναι να επιταχύνει την πρόσβαση στα αποθηκευμένα δεδομένα σε υφιστάμενα κατανεμημένα συστήματα αρχείων. Αυτό το επιτυγχάνει λειτουργώντας ως ενδιάμεσο επίπεδο ανάμεσα στα τυπικά κατανεμημένα συστήματα αρχείων και στις μηχανές επεξεργασίας μεγάλων δεδομένων. Πρακτικά, λειτουργεί ως cache, επιτρέποντας την πρόσβαση στα δεδομένα του από διαφορετικούς κόμβους, ενώ είναι ανθεκτικό στα σφάλματα. Έτσι, για δεδομένα που επαναχρησιμοποιούνται αποφεύγεται η ανάγνωσή τους από τον δίσκο. Αν τα δεδομένα αυτά φιλοξενούνται από το Alluxio στον ίδιο κόμβο, τότε η ταχύτητα πρόσβασής τους είναι αντίστοιχη με την ταχύτητα της μνήμης, ενώ, αν φιλοξενούνται σε άλλον κόμβο του Alluxio, περιορίζεται από την ταχύτητα δικτυακής ζεύξης των δύο κόμβων.

2.2.3.1 Αρχιτεκτονική

Στο Alluxio υπάρχουν τρεις τύποι οντοτήτων [all]: Οι Masters, οι Workers και οι Clients. Ας ξεκινήσουμε από τους Masters του Alluxio και στη συνέχεια θα παρουσιάσουμε τη λειτουργία των Workers και έπειτα των Clients. Αυτές απεικονίζονται στο Σχήμα 2.3.

Masters. Στο Alluxio υπάρχουν δύο είδη διεργασιών που έχουν ρόλο Master: ο Alluxio Master και ο Alluxio Job Master. Ο Alluxio Master εξυπηρετεί τα αιτήματα των χρηστών και καταγράφει τις αλλαγές στα μεταδεδομένα, και μόνο, του συστήματος αρχείων. Μπορεί να υπάρξει ένας μοναδικός *επικεφαλής* (*leading*) Master, ο οποίος είναι ενεργός και πολλοί *αναμείνοντες* (*standby*) Masters με σκοπό την ανοχή σε σφάλματα. Οι αναμείνοντες ενημερώνονται από τις καταγραφές αυτές ώστε να αναλάβουν, αν ο επικεφαλής αποτύχει, οπότε και εκλέγουν νέο επικεφαλής μέσω του Zookeeper. Ο Job Master, από την άλλη, λειτουργεί ως χρονοδρομολογητής εργασιών στο σύστημα αρχείων. Έτσι, ο Alluxio Master ασχολείται με τα αιτήματα πελατών, ενώ ο Job Master με τις λειτουργίες του συστήματος αρχείων.

Workers. Αντιστοίχως με τους δύο κύριους τύπους διεργασιών Master, υπάρχουν και δύο τύποι διεργασιών Worker, οι Workers και οι Job Workers. Ρόλος των πρώτων είναι η διαχείριση των τοπικών πόρων, όπως μνήμης, δίσκου κ.λπ. και η αποθήκευση δεδομένων στη



Σχήμα 2.3: Αρχιτεκτονική του Alluxio.

μνήμη με τη μορφή block. Οι Job Workers έχουν ως ρόλο την πραγματοποίηση ενεργειών (π.χ. ανάγνωση, εγγραφή) σε οποιεσδήποτε τοποθεσίες του συστήματος αρχείων, σύμφωνα με τις οδηγίες του Job Master.

Client. Το πρόγραμμα-πελάτης του Alluxio λειτουργεί ως διεπαφή μεταξύ του συστήματος και του πελάτη επικοινωνώντας με τον επικεφαλής Master. Το εγγενές API του Alluxio είναι σε Java ενώ υποστηρίζει και άλλες γλώσσες και είναι συμβατό τόσο με το HDFS όσο και με το Amazon S3, τα οποία θα μελετήσουμε στη συνέχεια.

2.2.3.2 Λειτουργία

Ας μελετήσουμε τώρα τον τρόπο λειτουργίας του Alluxio όσον αφορά τις εναλλακτικές που προσφέρει για την ανάγνωση και την εγγραφή δεδομένων.

Ανάγνωση. Όσον αφορά την ανάγνωση δεδομένων, υπάρχουν τέσσερις εναλλακτικές προσεγγίσεις. Η πρώτη αφορά την περίπτωση που τα δεδομένα που ζητά ο πελάτης από το σύστημα αρχείων βρίσκονται ήδη στην τοπική μνήμη του Worker του τρέχοντος κόμβου, οπότε και διαβάζονται απευθείας από τη μνήμη. Αν δεν βρίσκονται στην τοπική μνήμη, αλλά στη μνήμη Worker σε άλλον κόμβο, τα δεδομένα διαβάζονται από εκεί και δημιουργείται και ένα αντίγραφο τοπικά, πάντα στη μνήμη. Προτιμάται η ανάγνωση μέσω δικτύου από αυτήν στο κατανεμημένο σύστημα αρχείων του δίσκου, καθώς θεωρείται ταχύτερη. Αν τα δεδομένα δεν υπάρχουν σε κανέναν Worker, θα διαβαστούν από τον δίσκο και στη συνέχεια από τη μνήμη, όπου και θα δημιουργηθεί ένα τοπικό αντίγραφό τους. Τέλος, υπάρχει η δυνατότητα απευθείας ανάγνωσης των δεδομένων από τον δίσκο.

Εγγραφή. Για την εγγραφή των δεδομένων, υπάρχουν και πάλι τέσσερις εναλλακτικές. Μπορούν τα δεδομένα να εγγραφούν μόνο στο Alluxio, δηλαδή οι αλλαγές να πραγματοποιηθούν μόνο στη μνήμη (κυρίως για προσωρινά δεδομένα), να εγγραφούν ταυτόχρονα στη μνήμη

και στο σύστημα αρχείων του δίσκου, να εγγραφούν πρώτα στη μνήμη και στη συνέχεια να μεταφερθούν στον δίσκο, ή μόνο στον δίσκο. Το Alluxio υποστηρίζει ισχυρή συνέπεια για τα δεδομένα που εγγράφει, καθώς όλες οι ενέργειες επικυρώνονται πρώτα από τον Master, πριν εμφανιστούν στον πελάτη ως ολοκληρωμένες [all].

2.3 Λύσεις αποθήκευσης νέφους

Στο τμήμα αυτό θα δούμε αποθηκευτικές λύσεις στο νέφος. Συγκεκριμένα, θα μελετήσουμε τον μηχανισμό αποθήκευσης Swift του OpenStack και θα παρουσιάσουμε συνοπτικά κάποιες εμπορικές υπηρεσίες αποθήκευσης, τις Amazon S3, Google Cloud και Microsoft Azure.

2.3.1 OpenStack Swift

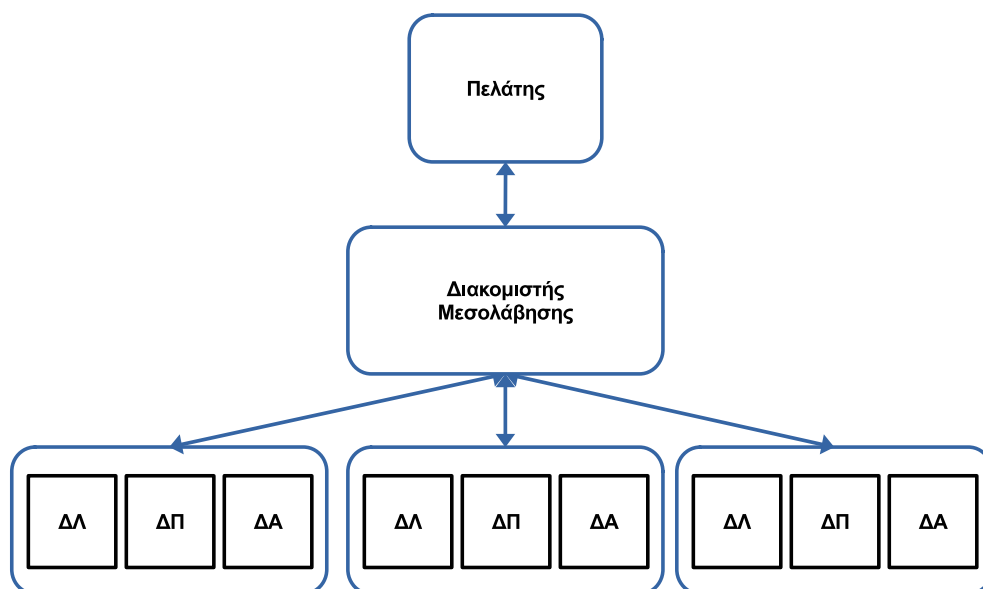
Το OpenStack [ope] είναι ένα λειτουργικό σύστημα νέφους ανοικτού κώδικα που διαχειρίζεται πόρους υπολογιστικούς, δικτυακούς και αποθήκευσης. Ο κύριος μηχανισμός αποθήκευσης είναι το Swift, που αποτελεί τον μηχανισμό του OpenStack για αποθήκευση αντικειμένων. Το Swift είναι καταναμημένο, χωρίς κεντρικούς διακομιστές. Για να διασφαλιστεί η ακεραιότητα των δεδομένων, τα αντικείμενα αντιγράφονται σε πολλαπλούς κόμβους.

Το Swift αποτελεί ένα σύστημα για την αποθήκευση αντικειμένων. Τα αντικείμενα οργάνωνται σε *περιέκτες* (*containers*) και ο κάθε *λογαριασμός χρήστη* (*account*) αποτελείται από μία συλλογή από περιέκτες. Χαρακτηριστικό του Swift, ως αποθήκης αντικειμένων, είναι ότι στον χώρο ονομάτων των αρχείων, πέρα από την οργάνωση που μόλις περιγράφηκε, δεν υπάρχει ιεραρχία. Αυτό σημαίνει ότι δεν υπάρχουν εμφωλευμένοι κατάλογοι.

Σε έναν κόμβο του Swift συνυπάρχει ένα σύνολο διακομιστών, καθένας από τους οποίους εξυπηρετεί τις παραπάνω οντότητες. Έτσι, υπάρχει ο *διακομιστής αντικειμένων* (*object server*), ο *διακομιστής λογαριασμών* (*account server*) και ο *διακομιστής περιεκτών* (*container server*), όπως απεικονίζονται στο Σχήμα 2.4. Ένας χρήστης έχει πρόσβαση στα δεδομένα του διακομιστή επικοινωνώντας με τον *διακομιστή μεσολάβησης* (*proxy server*) του Swift, που είναι και η μόνη διεργασία που επικοινωνεί με εξωτερικούς πελάτες. Οι διακομιστές λογαριασμού και περιεκτών χρησιμοποιούνται για τα μεταδεδομένα των αντίστοιχων οντοτήτων, ενώ ο διακομιστής αντικειμένων εξυπηρετεί τα δεδομένα που είναι αποθηκευμένα ως αντικείμενα στον δίσκο.

Το Swift υποστηρίζει δύο μηχανισμούς αποθήκευσης. Ο πρώτος είναι η δημιουργία αντιγράφων, ενώ ο δεύτερος η χρήση *κωδίκων διαγραφής* (*erasure codes*). Αυτοί οι μηχανισμοί αποθήκευσης υποστηρίζονται από μία σειρά διεργασιών. Έτσι, υπάρχουν οι *ελεγκτές* (*auditors*) οι οποίοι ελέγχουν την ακεραιότητα των αντιγράφων και, σε περίπτωση που εντοπίσουν σφάλματα, απομονώνουν τα αντίγραφα αυτά. Έπειτα, υπάρχουν οι *αντιγραφείς* (*replicators*), ο ρόλος των οποίων είναι να διασφαλίζουν την αποθήκευση πολλαπλών αντιγράφων σε διαφορετικούς κόμβους. Τέλος, οι *ανακατασκευαστές* (*reconstructors*) πραγματοποιούν υπολογισμούς σε τμήματα κωδίκων διαγραφής για την αποκατάσταση των αντικειμένων.

Στο Swift υπάρχει επίσης η έννοια του *δακτυλίου* (*ring*). Ο δακτύλιος αποτελεί τον μηχανισμό αντιγραφής και κατανομής τμημάτων του κατακερματισμένου χώρου ονομάτων αρχείων



Σχήμα 2.4: Αρχιτεκτονική του OpenStack Swift. ΔΛ: Διακομιστής Λογαριασμών, ΔΠ: Διακομιστής Περιεκτών, ΔΑ: Διακομιστής Αντικειμένων.

στο Swift και αποτελεί μία απεικόνιση μεταξύ του ονόματος αντικειμένου και της φυσικής του αποθήκευσης. Έτσι, ο χώρος κατακερματισμού ονομάτων αρχείων χωρίζεται σε διαμερίσματα. Κάθε διαμέρισμα έχει πολλαπλά αντίγραφα. Κάθε δίσκος έχει πολλά διαμερίσματα, ενώ σε κάθε δίσκο μπορεί να υπάρξει μόνον ένα αντίγραφο. Κάθε κόμβος διατηρεί αντίγραφο των δακτυλίων. Ο ίδιος μηχανισμός χρησιμοποιείται για λογαριασμούς και περιέκτες.

Τα αντικείμενα είναι αποθηκευμένα ως blobs σε διαμερίσεις (*partitions*) και υπάρχουν χρονοσφραγίδες, επιτρέποντας στο Swift να αποθηκεύει και παλαιότερες εκδόσεις του ίδιου αντικειμένου. Τα δεδομένα και τα μεταδεδομένα κάθε αρχείου αποθηκεύονται μαζί, αποτελώντας ενιαία οντότητα. Κάθε αντικείμενο αποθηκεύεται ως ένα αρχείο στον δίσκο, εκτός αν περάσει το κατώφλι μέγιστου αρχείου, οπότε και χωρίζεται σε τμήματα.

Το Cinder [cin] είναι ο μηχανισμός αποθήκευσης blocks του OpenStack. Εικονικοί δίσκοι μπορούν να δημιουργούνται και να χρησιμοποιούνται μέσω του Cinder. Η υπηρεσία διαμοιρασμού αρχείων Manila [man] αποτελεί μία από τις πιο πρόσφατες προσθήκες στους μηχανισμούς αποθήκευσης του OpenStack και προσφέρει κοινή χρήση αρχείων.

2.3.2 Συνδρομητικές υπηρεσίες

Το OpenStack Swift αποτελεί χαρακτηριστικό παράδειγμα συστήματος αποθήκευσης στο Νέφος. Πέραν των λύσεων ανοικτού κώδικα, όμως, υπάρχουν και εμπορικές λύσεις που προσφέρονται από παρόχους. Στη συνέχεια, παρουσιάζουμε επιγραμματικά κάποιες από αυτές.

2.3.2.1 Amazon S3

Το Amazon S3 (Amazon Simple Storage Services) αποτελεί την αποθηκευτική λύση της Amazon στο νέφος για την αποθήκευση δεδομένων ως αντικειμένων σε κάδους [s3]. Με τον όρο αντικείμενο περιγράφεται ο συνδυασμός των δεδομένων ενός αρχείου με τα μεταδεδομένα

του. Ένας κάδος (*bucket*) αποτελεί έναν περιέκτη αντικειμένων. Το S3 είναι χωρισμένο σε γεωγραφικές περιοχές (*regions*) ανά τον κόσμο. Ένας κάδος τοποθετείται σε μία περιοχή και περιέχει τα δεδομένα ως αντικείμενα. Κάθε αντικείμενο έχει ένα κλειδί, το οποίο είναι ένα μοναδικό αναγνωριστικό για το αντικείμενο σε έναν κάδο. Επιπλέον, υποστηρίζεται η δυνατότητα ύπαρξης πολλαπλών εκδόσεων (*versioning*) του κάθε αντικειμένου.

2.3.2.2 Google Cloud

Οι υπηρεσίες νέφους της Google προσφέρουν επίσης ένα εύρος επιλογών αποθήκευσης [goo]. Η υπηρεσία Object Storage είναι κατάλληλη για την αποθήκευση αντικειμένων. Και σε αυτήν την περίπτωση, υπάρχουν αντικείμενα που ταυτοποιούνται από ένα μοναδικό προσδιοριστικό. Μία άλλη υπηρεσία είναι η Block Storage για blocks, που μπορεί να είναι δίσκοι εικονικών μηχανών ή βάσεις δεδομένων. Υπάρχει επίσης η αποθήκευση αρχείων (File Storage), η οποία προσφέρεται για διαμοιρασμό αρχείων.

2.3.2.3 Windows Azure Storage

Το Windows Azure Storage (WAS) είναι ένα σύστημα αποθήκευσης Νέφους που παρέχει τη δυνατότητα αποθήκευσης πρακτικά απεριόριστου όγκου δεδομένων. Τα δεδομένα αποθηκεύονται με ανθεκτικότητα, με χρήση τόσο τοπικών όσο και γεωγραφικά διάσπαρτων αντιγράφων. Υποστηρίζονται διάφορες μορφές αποθήκευσης: Blob για αποθήκευση κειμενικών και δυαδικών δεδομένων, διαμοιραζόμενα αρχεία και πίνακες, για αποθήκευση δομημένων δεδομένων, ουρές για αποθήκευση μηνυμάτων και δίσκοι για χρήση με εικονικές μηχανές.

Το WAS αποτελείται από μονάδες αποθήκευσης (*storage stamps*) και την υπηρεσία εντοπισμού (*location service*) [CWO⁺11]. Κάθε μονάδα αποθήκευσης αποτελείται από racks. Η υπηρεσία εντοπισμού αντιστοιχίζει λογαριασμούς χρηστών στις μονάδες αποθήκευσης. Οι λογαριασμοί κατανομούνται μεταξύ των μονάδων αποθήκευσης για εξισορρόπηση φορτίου και ανάκαμψη από σφάλματα. Μία μονάδα αποθήκευσης αποτελείται από τρία επίπεδα: το *επίπεδο ροής* (*stream layer*), το οποίο αποθηκεύει και διαχειρίζεται δεδομένα σε επίπεδο bits, το *επίπεδο διαμέρισης* (*partition layer*), το οποίο διαχειρίζεται δεδομένα σε υψηλότερο επίπεδο, παρέχει τον χώρο ονομάτων και μηχανισμούς συνέπειας των δεδομένων, και το *επίπεδο παρουσίασης* (*front-end layer*), που αναλαμβάνει τη διαχείριση των αιτημάτων από τους πελάτες. Επιπλέον, υπάρχουν δύο μηχανισμοί αντιγραφής, εντός μονάδων, που είναι σύγχρονος, και μεταξύ μονάδων, που είναι ασύγχρονος.

2.4 Apache Hadoop Distributed File System

Αφού μελετήσαμε διάφορες λύσεις κατανεμημένης αποθήκευσης ανοικτού κώδικα για ιδιόκτητες συστάδες όσο και στο Νέφος, θα εστιάσουμε στο HDFS, το κατανεμημένο σύστημα αρχείων του Hadoop ανοικτού κώδικα το οποίο είναι μέρος του Apache Hadoop. Θα μελετήσουμε αναλυτικά τους μηχανισμούς του, ενώ θα παρουσιάσουμε και μία σειρά από εντολές διαχείρισης.

2.4.1 Αρχιτεκτονική του Apache Hadoop

Το Hadoop αποτελεί λογισμικό ανοικτού κώδικα που περιλαμβάνει μία υλοποίηση του προγραμματιστικού προτύπου MapReduce. Το Hadoop έχει σχεδιαστεί για να λειτουργεί σε μία συστάδα υπολογιστών. Αυτό συνήθως αποτελείται από πολλαπλές μηχανές, που συνήθως χρησιμοποιούν Linux και διασυνδέονται σε τοπικά δίκτυα υψηλής ταχύτητας (π.χ. 10-gigabit Ethernet). Μία σημαντική καινοτομία του είναι ότι δεν απαιτεί ακριβό και εξειδικευμένο υλικό, δηλαδή κάποιον υπερυπολογιστή, για να πραγματοποιήσει παράλληλη επεξεργασία μεγάλου όγκου δεδομένων.

Βασίζεται σε υπολογιστές που μπορούν να αγοραστούν στο εμπόριο, οι οποίοι όμως είναι υψηλών προδιαγραφών. Έτσι, το υλικό συνήθως είναι υψηλής ποιότητας όσον αφορά τη μνήμη, τον επεξεργαστή και τον δίσκο. Το Hadoop σχεδιάστηκε σε μια εποχή που η μνήμη RAM δεν ήταν τόσο φτηνή, σε σύγκριση με τους σκληρούς δίσκους. Έτσι, οι υπολογισμοί εμπλέκουν σε σημαντικό βαθμό την αποθήκευση στον δίσκο. Ουσιαστικά, το Hadoop αποτελείται από τρία κύρια υποσυστήματα:

- το κατανεμημένο σύστημα αρχείων HDFS, που είναι υπεύθυνο για την κατανεμημένη αποθήκευση δεδομένων
- τον διαχειριστή πόρων YARN, που κατανέμει την εκτέλεση εργασιών σε μία συστάδα και παρακολουθεί και κατανέμει τους διαθέσιμους πόρους
- τον μηχανισμό MapReduce, που πραγματοποιεί επεξεργασία των δεδομένων ακολουθώντας το υπολογιστικό μοντέλο MapReduce

Αξίζει να αναφέρουμε ότι, βασιζόμενο στο υπολογιστικό μοντέλο MapReduce, μία ακόμη καινοτομία του Hadoop αποτελεί η μεταφορά του υπολογισμού στα δεδομένα τα οποία βρίσκονται αποθηκευμένα σε διάφορους κόμβους της συστάδας και όχι το αντίστροφο. Δεν υπάρχει, δηλαδή, κεντρικός διακομιστής στον οποίο να διατηρούνται τα δεδομένα. Καθώς το σύστημα προορίζεται για την επεξεργασία μεγάλου όγκων δεδομένων, η μετακίνηση αυτών από έναν κόμβο σε άλλον θα απαιτούσε πολύ χρόνο και πόρους ώστε να βρεθούν σε συγκεκριμένες μηχανές που πραγματοποιούν συγκεκριμένους υπολογισμούς. Ο κώδικας, λοιπόν, ο οποίος οπωσδήποτε έχει μικρότερο μέγεθος από τα δεδομένα που υφίστανται επεξεργασία, είναι πιο αποδοτικό να μετακινηθεί και να εκτελεστεί αντί να μεταφερθούν τα δεδομένα.

Τέλος, η διαμόρφωση λειτουργίας είναι τύπου αφέντη/σκλάβου (master/slave). Αυτό σημαίνει ότι ο master αναθέτει μια εργασία, ενώ ο κάθε slave φέρνει εις πέρας την εργασία που έχει αναλάβει. Με την ολοκλήρωση, ενημερώνει τον master ο οποίος του αναθέτει μια νέα εργασία, κ.ο.κ.

2.4.2 HDFS: το κατανεμημένο σύστημα αρχείων του Hadoop

Ένα χαρακτηριστικό των τελευταίων ετών είναι ότι, ενώ οι δυνατότητες αποθήκευσης των σκληρών δίσκων έχουν αυξηθεί, οι ταχύτητες πρόσβασης έχουν διατηρηθεί σε χαμηλά επίπεδα. Η τεχνολογία των δίσκων SSD έχει βελτιώσει αυτήν την κατάσταση, αλλά με αντιστάθμισμα το αυξημένο κόστος. Σε κάθε περίπτωση, η ανάγνωση πολλών TB δεδομένων απαιτεί

σημαντικό χρόνο, αν πραγματοποιηθεί σειριακά. Η λύση σε αυτό το πρόβλημα είναι η παράλληλη ανάγνωση από πολλούς δίσκους ταυτόχρονα.

Εδώ εμφανίζεται το πρόβλημα της αντιμετώπισης των σφαλμάτων. Καθώς αναφερόμαστε σε δεδομένα μεγάλης κλίμακας σε συστάδες, είναι δεδομένο ότι αυτά θα βρίσκονται αποθηκευμένα σε πολλούς υπολογιστές. Αν και μικρή η πιθανότητα να εμφανιστεί κάποιο σφάλμα υλικού, όσο περισσότεροι είναι οι υπολογιστές τόσο αυξάνεται η πιθανότητα ένας τουλάχιστον υπολογιστής να αντιμετωπίσει κάποια αστοχία υλικού.

Μία από τις λύσεις στο πρόβλημα αυτό αποτελεί το *κατανεμημένο σύστημα αρχείων του Hadoop (Hadoop Distributed File System, HDFS)*. Το HDFS παρουσιάζει υψηλή ανοχή σε σφάλματα, με έναν μηχανισμό που θα δούμε στη συνέχεια. Ο σχεδιασμός του το καθιστά κατάλληλο για πολύ μεγάλα αρχεία και για περιπτώσεις δεδομένων τα οποία εγγράφονται μία φορά αλλά διαβάζονται πολλές, όπως είναι τα αρχεία καταγραφής ιστοτόπων των οποίων το μέγεθος αυξάνεται διαρκώς.

Το HDFS, αντιθέτως, δεν είναι κατάλληλο για λειτουργίες που εκτελούν τα συνήθη συστήματα αρχείων που προορίζονται για χρήση σε έναν τοπικό υπολογιστή, καθώς δεν έχει σχεδιαστεί για αυτόν τον σκοπό. Έτσι, δεν είναι κατάλληλο για τυχαία αναζήτηση δεδομένων. Τα πολλά μικρά αρχεία μπορούν να δημιουργήσουν πρόβλημα στον διακομιστή που διατηρεί το ευρετήριο με τα ονόματα αρχείων, λόγω της αρχιτεκτονικής του.

Το HDFS αποθηκεύει χωριστά τα δεδομένα των εφαρμογών και τα μεταδεδομένα του συστήματος αρχείων, όπως ονόματα αρχείων. Συγκεκριμένα, τα δεδομένα της εφαρμογής αποθηκεύονται σε διακομιστές που ονομάζονται *DataNodes*, ενώ τα μεταδεδομένα αποθηκεύονται σε έναν διακομιστή που ονομάζεται *NameNode*. Όλοι αυτοί οι κόμβοι συνδέονται και επικοινωνούν μεταξύ τους με συνδέσεις που χρησιμοποιούν το πρωτόκολλο TCP.

2.4.2.1 Χαρακτηριστικά του HDFS

Ας δούμε τώρα κάποια από τα κυριότερα χαρακτηριστικά του HDFS. Αρχικά, έχει σχεδιαστεί για εγκατάσταση και χρήση σε συμβατικό υλικό, χωρίς να απαιτεί συστήματα εξειδικευμένων αρχιτεκτονικών. Είναι κατάλληλο για μεγάλα σύνολα δεδομένων και πρόσβαση υψηλών επιδόσεων σε αυτά. Τέλος, είναι μέρος του Apache Hadoop [Had].

Αστοχία υλικού. Στα συστήματα που εμπλέκουν μεγάλες συστάδες υπολογιστών η αστοχία αποτελεί συχνό φαινόμενο. Μια εγκατάσταση HDFS μπορεί να αποτελείται από εκατοντάδες κόμβους. Ακόμη και αν η πιθανότητα αστοχίας ενός κόμβου είναι πολύ μικρή, λόγω του μεγάλου πλήθους των διακομιστών, μπορούμε να θεωρήσουμε δεδομένο ότι διαρκώς κόμβοι του συστήματος θα είναι εκτός λειτουργίας. Λαμβάνοντας αυτήν την κατάσταση υπόψη, αφενός δεν θα επιθυμούσαμε να υπάρξει απώλεια δεδομένων, αφετέρου, ακόμη και στην περίπτωση αστοχιών, θα επιθυμούσαμε το σύστημα να παραμένει λειτουργικό. Έτσι, το HDFS επιτυγχάνει γρήγορη αυτόματη ανάκαμψη από αστοχίες, χρησιμοποιώντας μια σειρά μηχανισμών που θα αναφέρουμε παρακάτω.

Πρόσβαση σε δεδομένα μεγάλης κλίμακας με μορφή ροής. Το HDFS έχει σχεδιαστεί για την υποστήριξη εφαρμογών που απαιτούν μαζική επεξεργασία πολύ μεγάλων συνόλων δεδομένων,

όπου κάθε αρχείο μπορεί να φτάνει σε μέγεθος τα πολλά TB, τα οποία αρχεία θα βρίσκονται αποθηκευμένα σε ίσως και χιλιάδες διαφορετικών κόμβων. Επιπλέον, το HDFS θα πρέπει να μπορεί να υποστηρίξει ένα πολύ μεγάλο πλήθος τέτοιων αρχείων. Καθώς δεν προορίζεται για χρήση από εφαρμογές γενικού σκοπού, οι οποίες αντιστοίχως χρησιμοποιούν συστήματα αρχείων γενικού σκοπού, δεν έχει δοθεί έμφαση σε λειτουργίες όπως στη χαμηλή καθυστέρηση στην πρόσβαση στα δεδομένα. Αντιθέτως, στόχος είναι, κατά την πρόσβαση σε ένα σύνολο δεδομένων από το HDFS, αυτή να πραγματοποιείται απρόσκοπτα με τη μορφή ροής, παρέχοντας υψηλό ρυθμό διαμεταγωγής. Για να πετύχει το HDFS υψηλές επιδόσεις, θα πρέπει το μοντέλο δεδομένων να είναι απλό. Έτσι, τα δεδομένα που έχουν εγγραφεί δεν θα πρέπει να τροποποιούνται συχνά, εκτός και αν πρόκειται για προσαρτήσεις στο τέλος των αρχείων ή διαγραφές αυτών. Επιπλέον, καθώς είναι επιθυμητή η δυνατότητα οριζόντιας κλιμάκωσης, θα πρέπει να υπάρχει η δυνατότητα λειτουργίας σε ετερογενείς κόμβους. Είθισται, πάντως, το Hadoop να εγκαθίσταται σε διακομιστές με λειτουργικό σύστημα Linux.

2.4.3 Έννοιες και οντότητες του HDFS

Σε αυτήν την ενότητα εξετάζουμε με λεπτομέρεια τις βασικές έννοιες του HDFS. Επιγραμματικά, το HDFS διαχωρίζει τα αρχεία σε τμήματα που ονομάζονται block και λειτουργεί με διαμόρφωση master/slave. Ο master ονομάζεται NameNode και περιέχει τον κατάλογο με τους slaves, που ονομάζονται DataNodes, στους οποίους βρίσκονται αποθηκευμένα τα blocks. Παρακάτω, παρουσιάζουμε αναλυτικά τον ρόλο των blocks, του NameNode και των DataNodes.

2.4.3.1 NameNode

Ο NameNode διαχειρίζεται τον χώρο ονομάτων του HDFS που είναι μια ιεραρχία αρχείων και καταλόγων, αντίστοιχα με ένα συμβατικό σύστημα αρχείων. Με άλλα λόγια, αποτελεί ένα ευρετήριο το οποίο περιέχει την ιεραρχία των καταλόγων και των αρχείων στο HDFS, καθώς και πληροφορίες που περιγράφουν σε ποιους κόμβους όπου εκτελούνται DataNodes βρίσκεται το καθένα block κάθε ενός αρχείου. Ο NameNode γνωρίζει ποιοι από τους DataNodes είναι ενεργοί, καθώς αυτοί του στέλνουν περιοδικά *παλμούς* (*heartbeats*).

Τόσο κατά την ανάγνωση όσο και κατά την εγγραφή δεδομένων, ένας πελάτης επικοινωνεί αρχικά με τον NameNode ώστε να του γνωστοποιήσει την τοποθεσία των δεδομένων, αν πρόκειται για ανάγνωση, ή την τοποθεσία όπου θα εγγραφούν δεδομένα, εάν πρόκειται για εγγραφή. Και οι δύο αυτές λειτουργίες (ανάγνωση ή εγγραφή) πραγματοποιούνται αρχικά από τον κοντινότερο στη διεργασία που θέλει να πραγματοποιήσει την ενέργεια κόμβο.

Ο NameNode διατηρεί όλες τις πληροφορίες του στην κύρια μνήμη. Το σύνολο των μετα-δεδομένων του συστήματος ονομάτων αρχείων (π.χ. τοποθεσίες blocks) ονομάζονται *εικόνα* (*image*) [SKRC10]. Η εικόνα αποθηκεύεται και στον τοπικό δίσκο του NameNode, οπότε και ονομάζεται *σημείο ελέγχου* (*checkpoint*). Οι αλλαγές που γίνονται στην εικόνα αποθηκεύονται στο τοπικό σύστημα αρχείων σε ένα *ημερολόγιο καταγραφής* (*journal*). Όταν εκκινεί ο NameNode, δημιουργεί την εικόνα στη μνήμη από το σημείο ελέγχου στον δίσκο και στη συνέχεια την ενημερώνει ανατρέχοντας στις αλλαγές που υπάρχουν στο ημερολόγιο καταγραφής. Εγγράφει τη νέα εικόνα και αδειάζει το ημερολόγιο καταγραφής.

Είναι εμφανές ότι, εάν καταρρεύσει ο NameNode, διακόπτεται η λειτουργία της συστάδας, καθώς σε αυτόν βρίσκονται αποθηκευμένες όλες οι πληροφορίες σχετικά με τα αποθηκευμένα αρχεία και τις τοποθεσίες αποθήκευσής τους. Για την επίλυση αυτού του προβλήματος, από την έκδοση Hadoop 2.0.0 και έπειτα εμφανίστηκε η λειτουργία HDFS υψηλής διαθεσιμότητας (High Availability, HA), η οποία επιλύει τα παραπάνω προβλήματα παρέχοντας τη δυνατότητα λειτουργίας πλεονάζοντος κόμβου NameNode, και από την 3η έκδοση του Hadoop περισσότερων κόμβων NameNode στην ίδια συστάδα. Οι NameNodes αυτοί βρίσκονται σε μία ενεργή/παθητική διαμόρφωση [HDFb]. Για να γίνει αυτή η λειτουργία, συνήθως δύο ή περισσότεροι κόμβοι διαμορφώνονται ως NameNodes. Σε οποιαδήποτε χρονική στιγμή, ο ένας κόμβος NameNode βρίσκεται σε *ενεργή* κατάσταση, ενώ οι υπόλοιποι κόμβοι NameNode βρίσκονται σε κατάσταση *αναμονής* (*standby*). Ο ενεργός NameNode βρίσκεται σε πλήρη λειτουργία και διαχειρίζεται τη λειτουργία των DataNodes της συστάδας. Οι NameNodes που βρίσκονται σε αναμονή παρακολουθούν τις λειτουργίες του ενεργού NameNode, ώστε κάποιος από αυτούς να αναλάβει ως master, εάν παραστεί ανάγκη.

Για να συμβεί αυτό, οι NameNodes που συμμετέχουν σε διάταξη HA έχουν πρόσβαση σε έναν κοινό κατάλογο του συστήματος αρχείων. Για αυτόν τον σκοπό, μπορεί να χρησιμοποιηθεί το NFS. Σε κάθε περίπτωση, αυτός ο χώρος αποθήκευσης δεν θα πρέπει να αποτελεί μοναδικό σημείο αποτυχίας. Για αυτόν τον λόγο, συστήνεται η χρήση διάταξης NAS. Για να μπορεί καθένας από τους NameNodes σε αναμονή να αναλάβει άμεσα, όταν ο ενεργός NameNode λάβει μία τροποποίηση για τον χώρο ονομάτων, προσθέτει μία εγγραφή στο ημερολόγιο καταγραφής που είναι αποθηκευμένο στον κοινόχρηστο κατάλογο. Οι αναμένοντες NameNodes βλέπουν την καταγραφή και ενημερώνουν τη δική τους κατάσταση. Καθώς η εγγραφή πραγματοποιείται σε σταθερή αποθήκευση, σε περίπτωση αστοχίας, πραγματοποιείται ανάγνωση του ημερολογίου καταγραφής και στη συνέχεια ένας αναμένων κόμβος έρχεται σε ενεργή κατάσταση. Για τη γρήγορη αντικατάσταση ενός NameNode, ο μηχανισμός παλμών έχει τροποποιηθεί και κάθε DataNode στέλνει παλμούς σε όλους τους NameNodes που συμμετέχουν στη διάταξη HA.

Καθώς ο NameNode διατηρεί στη μνήμη του την εικόνα του χώρου ονομάτων των αρχείων, το πρόβλημα που εμφανίζεται αφορά την περίπτωση που ο NameNode θα πρέπει να διαχειριστεί ένα πολύ μεγάλο πλήθος μικρών αρχείων. Ένα άλλο πρόβλημα που προκύπτει, κυρίως σε μεγάλες συστάδες, αφορά το γεγονός ότι ο NameNode είναι ο μοναδικός κόμβος που εξυπηρετεί όλα τα αιτήματα των πελατών, οι οποίοι μπορεί να είναι και δεκάδες χιλιάδες.

Για να αντιμετωπιστούν τα παραπάνω ζητήματα, αναπτύχθηκε ο μηχανισμός της *ομοσπονδίας HDFS* (*HDFS federation*). Με αυτόν τον μηχανισμό, υπάρχουν πολλαπλοί ανεξάρτητοι NameNodes, καθένας από τους οποίους είναι αποκλειστικά υπεύθυνος μόνο για ένα τμήμα του χώρου ονομάτων αρχείων και δεν επικοινωνεί με τους υπόλοιπους. Οι DataNodes αποθηκεύουν blocks από όλους τους χώρους ονομάτων αρχείων και καθένας από αυτούς εγγράφεται σε όλους τους DataNodes και, αντιστοίχως, στέλνει περιοδικούς παλμούς και αναφορές blocks και δέχεται εντολές από τον κάθε NameNode ξεχωριστά.

2.4.3.2 Secondary NameNode

Ο ρόλος του *δευτερεύοντος NameNode (secondary NameNode)*¹, ο οποίος συνήθως εκτελείται σε διαφορετικό κόμβο από τον NameNode, είναι η συγχώνευση του ημερολογίου καταγραφής και της εικόνας ανά τακτά χρονικά διαστήματα. Αυτό είναι χρήσιμο, καθώς σε μία μεγάλη συστάδα το ημερολόγιο καταγραφής μπορεί να αυξηθεί πολύ σε μέγεθος. Κάτι τέτοιο έχει ως συνέπεια, αφενός μεν να καταλαμβάνει πολύ αποθηκευτικό χώρο, αφετέρου δε να καθυστερεί την επανεκκίνηση του NameNode, καθώς ο τελευταίος θα πρέπει να εφαρμόσει όλες τις αλλαγές στην εικόνα, ιδίως εάν έχει περάσει αρκετός χρόνος από την τελευταία του επανεκκίνηση. Ο δευτερεύων NameNode δεν μπορεί να χρησιμοποιηθεί για να αντικαταστήσει τη λειτουργία του NameNode, καθώς υπολείπεται χρονικά του δεύτερου.

2.4.3.3 DataNode

Με τον όρο DataNode αναφερόμαστε σε διεργασία ή υπολογιστή, ο οποίος διατηρεί blocks από διαφορετικά αρχεία. Αν και δεν συστήνεται, υπάρχει η δυνατότητα ο ίδιος κόμβος να λειτουργεί και ως NameNode και ως DataNode. Στο Σχήμα 2.5 βλέπουμε ότι υπάρχουν έξι DataNodes.

Κατά την εκκίνησή του, κάθε DataNode σε μια συστάδα συνδέεται με τον NameNode και πραγματοποιεί *χειραψία (handshake)* [SKRC10], ώστε να επαληθευτεί το *αναγνωριστικό χώρου ονομάτων (NameSpace id)* και η έκδοση του DataNode. Σε περίπτωση ασυμφωνίας σε τουλάχιστον ένα από αυτά, ο DataNode κλείνει αυτόματα. Μετά τη χειραψία, ο DataNode *εγγραφεται (registers)* στον NameNode. Μετά την εγγραφή τους, οι DataNodes αποκτούν και διατηρούν σε μόνιμη αποθήκευση ένα *αναγνωριστικό αποθήκευσης (storage id)*. Αυτό το αναγνωριστικό αποθήκευσης παραμένει σταθερό και χρησιμεύει για να προσδιορίζει μοναδικά έναν κόμβο, ακόμη και αν αλλάξουν τα χαρακτηριστικά σύνδεσής του με το δίκτυο (διεύθυνση IP, θύρα). Εκχωρείται στον DataNode κατά την εγγραφή στον NameNode για πρώτη φορά και δεν αλλάζει.

Το αναγνωριστικό χώρου ονομάτων δημιουργείται κατά την αρχική διαμόρφωση (format) του HDFS και αποθηκεύεται σε όλους τους κόμβους της συστάδας. Για να διατηρηθεί η ακεραιότητα του συστήματος αρχείων, κόμβοι με διαφορετικό αναγνωριστικό χώρου ονομάτων δεν μπορούν να συνδεθούν στη συστάδα. Σε νέο κόμβο που προστίθεται στη συστάδα δίνεται το τρέχον αναγνωριστικό του χώρου ονομάτων.

Όλοι οι κόμβοι θα πρέπει να έχουν την ίδια έκδοση λογισμικού για την προστασία της ακεραιότητας των δεδομένων που βρίσκονται αποθηκευμένα. Λόγω της διαρκούς ανάπτυξης του HDFS, ασυμφωνία στις εκδόσεις λογισμικού μπορεί να οδηγήσει σε απώλεια δεδομένων.

Κάθε DataNode, κατά την εγγραφή του, ενημερώνει τον NameNode για τα αντίγραφα blocks που φιλοξενεί, στέλνοντάς του μία αναφορά block. Αυτή περιλαμβάνει το αναγνωριστικό block, τη χρονοσφραγίδα δημιουργίας του, καθώς και το μήκος του. Αυτή η αναφορά επαναλαμβάνεται κάθε ώρα, ώστε να ενημερώνεται ο NameNode για τις αλλαγές που έχουν επέλθει.

¹https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html#Secondary_NameNode

Έπειτα από την εγγραφή τους, οι DataNodes επέρχονται σε κατάσταση κανονικής λειτουργίας. Όσο βρίσκονται σε κανονική λειτουργία, κάθε DataNode στέλνει τακτικά στον NameNode σύντομα μηνύματα κατάστασης τα οποία ονομάζονται *παλμοί* (*heartbeats*), όπως είπαμε και πιο πριν. Ο ρόλος των παλμών είναι, αφενός μεν η επιβεβαίωση της λειτουργίας του DataNode από τον NameNode, αφετέρου δε για τη μεταφορά συνοπτικών στατιστικών για τα blocks που διατηρεί.

Εάν ο NameNode δεν λάβει παλμό από έναν DataNode εντός ενός χρονικού ορίου, ο NameNode θεωρεί ότι ο συγκεκριμένος DataNode είναι εκτός λειτουργίας και τα αντίγραφα blocks αυτού του DataNode δεν είναι διαθέσιμα. Αυτό έχει ως αποτέλεσμα ο NameNode να εκκινήσει τη διαδικασία δημιουργίας νέων αντιγράφων των blocks που φιλοξενούνταν στον εν λόγω DataNode σε άλλους DataNodes, οι οποίοι βρίσκονται επιβεβαιωμένα σε κατάσταση κανονικής λειτουργίας, ώστε να ικανοποιηθεί ο ορισμένος παράγοντας αντιγραφής.

Μέσω αυτού του μηχανισμού παλμών, πραγματοποιείται και η επικοινωνία από τον NameNode προς τους DataNodes με τη μορφή απαντήσεων. Στις απαντήσεις του στους παλμούς που λαμβάνει ο NameNode μπορεί να μεταφέρει οδηγίες προς τους DataNodes [SKRC10], για παράδειγμα να του στείλουν άμεσα αναφορά, να αντιγράψουν κάποιο block σε κάποιον άλλον κόμβο, ή ακόμη και να τερματίσουν τη λειτουργία τους. Καθίσταται εμφανές ότι ο μηχανισμός των παλμών είναι πολύ σημαντικός για την ορθή λειτουργία της συστάδας, χωρίς να την επιβαρύνει από άποψη απόδοσης. Να υπενθυμίσουμε σε αυτό το σημείο, ότι στην περίπτωση ύπαρξης διατάξεων υψηλής διαθεσιμότητας ή ομοσπονδίας HDFS παλμοί αποστέλλονται προς όλους τους εμπλεκόμενους NameNodes.

2.4.3.4 Πελάτης HDFS

Οι εφαρμογές χρηστών μπορούν μέσω της βιβλιοθήκης που υλοποιεί τη διεπαφή με το HDFS να το προσπελάζουν. Αντιστοίχως με τα κοινά συστήματα αρχείων που προορίζονται για χρήση σε έναν υπολογιστή, το HDFS υποστηρίζει λειτουργίες ανάγνωσης, εγγραφής και διαγραφής αρχείων και καταλόγων. Ο χρήστης αναφέρεται σε αρχεία και διαδρομές στον χώρο ονομάτων, χωρίς να γνωρίζει απαραίτητα ότι χρησιμοποιούνται διαφορετικοί κόμβοι στο σύστημα για την αποθήκευση των μεταδεδομένων και του συστήματος αρχείων ή ότι τα αρχεία βρίσκονται διασπασμένα σε blocks, καθένα από τα οποία έχει πολλαπλά αντίγραφα.

2.4.3.5 Τα blocks στο HDFS

Στο HDFS ένα block δεν έχει την ίδια έννοια που έχει σε ένα συμβατικό σύστημα αρχείων. Εκεί, ως block θεωρείται ο ελάχιστος χώρος που μπορεί να καταλάβει ένα αρχείο. Έτσι, αντιστοιχεί σε πολύ μικρό χώρο. Για παράδειγμα, αν το μέγεθος block ενός συστήματος αρχείων είναι 512 bytes και ενός αρχείου είναι 128 bytes, το αρχείο αυτό θα καταλαμβάνει στο εν λόγω σύστημα 512 bytes, χώρο ίσο με το μέγεθος block.

Αντιθέτως, στο HDFS το μέγεθος block αντιστοιχεί στο μέγεθος τμήματος στο οποίο θα διαχωριστεί ένα πολύ μεγάλο αρχείο και θα αναγνωστεί ενιαία από μια λειτουργία ανάγνωσης. Στο HDFS το προκαθορισμένο μέγεθος block είναι 128 MB². Αυτό σημαίνει ότι, αν ένα

²Παλαιότερα, η προκαθορισμένη τιμή ήταν 64 MB.

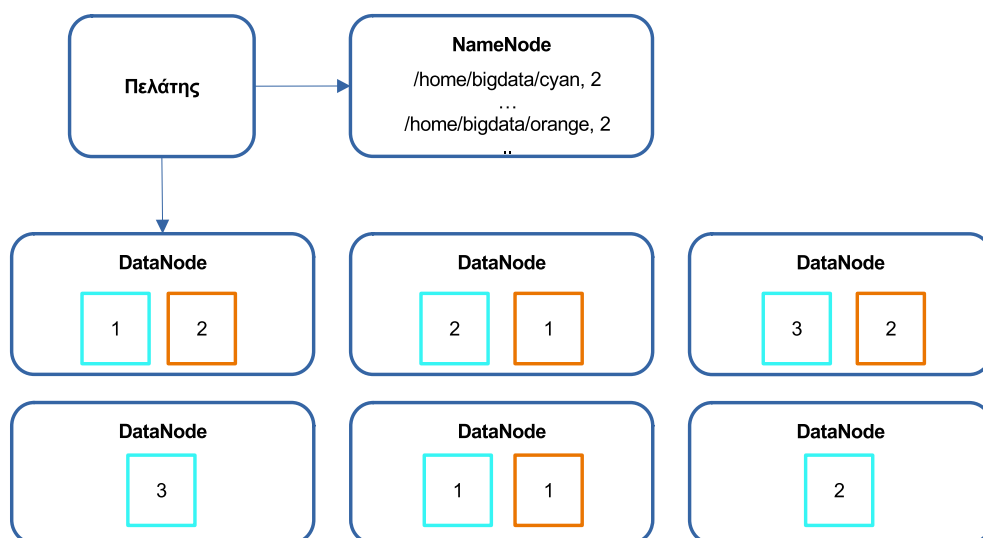
αρχείο έχει μέγεθος 130 MB, θα χωριστεί σε 2 blocks. Το ένα θα καταλαμβάνει 128 MB και το άλλο 2 MB. Επιπλέον, θα καταβληθεί προσπάθεια από το HDFS ώστε καθένα από αυτά τα blocks να τοποθετηθούν σε διαφορετικό κόμβο, ώστε να καταστεί δυνατή η ανάγνωσή τους παράλληλα. Τώρα, στην αντίθετη περίπτωση που ένα αρχείο έχει μέγεθος μικρότερο από το μέγεθος block, τότε αυτό καταλαμβάνει χώρο μικρότερο από το μέγεθος του block. Για παράδειγμα, αν το μέγεθος του αρχείου είναι 59 MB, αυτός θα είναι και ο χώρος που θα καταλάβει στον δίσκο.

Βλέπουμε, λοιπόν, ότι πρόκειται για διαφορετική προσέγγιση στον ορισμό του block από αυτόν που υπάρχει στα συμβατικά λειτουργικά συστήματα. Επί της ουσίας, δεν πρόκειται για το μικρότερο μέγεθος χώρου που μπορεί να δεσμευθεί από ένα αρχείο, αλλά από το μέγεθος στο οποίο το αρχείο αυτό θα διασπαστεί. Δεν πρέπει να ξεχνάμε, εξάλλου, ότι το HDFS σχεδιάστηκε για μεγάλα αρχεία, όπως τα αρχεία καταγραφής ιστοτόπων τα οποία αυξάνονται διαρκώς σε μέγεθος και αποκτούν μέγεθος της τάξης των πολλών GB.

Διασπώντας, λοιπόν, ένα αρχείο σε τμήματα, τα οποία μάλιστα μπορεί να βρίσκονται και σε διαφορετικούς δίσκους ή υπολογιστές, μπορούμε να επιτύχουμε τα εξής. Πρώτα απ' όλα, μπορούμε να διαβάσουμε/επεξεργαστούμε το αρχείο παράλληλα και όχι σειριακά. Έτσι, έχοντας το κατάλληλο σύστημα επεξεργασίας το οποίο επεξεργάζεται blocks μπορούμε να επιταχύνουμε την επεξεργασία ενός αρχείου με τρόπο ανάλογο με τον αριθμό των δίσκων στον οποίο βρίσκονται τα blocks του. Επιπλέον, λαμβάνοντας υπόψη ότι τα αρχεία που θα αποθηκευτούν είναι εξ ορισμού μεγάλα και θα διασπαστούν, δίνεται η δυνατότητα να αποθηκεύσουμε ένα αρχείο το οποίο θα ξεπερνά σε μέγεθος αυτό του μεγαλύτερου δίσκου στο σύστημά μας. Αυτό μπορεί να γίνει επειδή, καθώς ένα αρχείο θα διασπαστεί σε τμήματα ούτως ή άλλως, το άθροισμα του μεγέθους των τμημάτων τα οποία θα βρίσκονται σε διαφορετικούς δίσκους μπορεί να ξεπερνά το μέγεθος ενός δίσκου.

Όσον αφορά την ανοχή σε σφάλματα, κάθε block αντιγράφεται σε περισσότερους από έναν υπολογιστές. Το πλήθος των αντιγράφων ενός αρχείου ονομάζεται *παράγοντας αντιγραφής* (*replication factor*). Μία συνηθισμένη τιμή για τον παράγοντα αντιγραφής είναι το 3. Δηλαδή, υπάρχει σε τρεις διαφορετικές μηχανές αντίγραφο του κάθε block. Φυσικά, και μικρότερες αλλά και μεγαλύτερες τιμές μπορεί να οριστούν, ανάλογα με τις απαιτήσεις μας. Μειώνοντας, όμως, την τιμή κάτω από 3 αυξάνεται η πιθανότητα απώλειας δεδομένων, ενώ αυξάνοντας την τιμή υπερβολικά οδηγούμαστε σε επιπλέον δέσμευση χώρου. Έτσι, η τιμή 3 αποτελεί μία γενικά αποδεκτή επιλογή. Με αυτόν τον τρόπο, σε περίπτωση που αντιμετωπιστεί αστοχία κόμβου που φιλοξενεί κάποιο συγκεκριμένο block, δίνεται η δυνατότητα να συνεχιστεί η προσπάθεια στα δεδομένα από άλλο αντίγραφο. Αντιστοίχως, κατά τον εντοπισμό αστοχίας το HDFS φροντίζει να αποκαταστήσει τον παράγοντα αντιγραφής για το εν λόγω block. Εκτός, όμως, από τους λόγους ανοχής σε σφάλματα, ο παράγοντας αντιγραφής μπορεί να παίζει ρόλο και στις επιδόσεις των εφαρμογών που διαβάζουν blocks. Έτσι, η αύξηση του παράγοντα αντιγραφής μπορεί να χρησιμοποιηθεί και για να κατανεμηθεί ο φόρτος ανάγνωσης σε περιπτώσεις αρχείων που προσπελούνται πολύ συχνά από διαφορετικές εφαρμογές.

Το HDFS χρησιμοποιεί το τοπικό σύστημα αρχείων για να αποθηκεύσει δεδομένα. Αποτελεί, με άλλα λόγια, ένα επιπλέον επίπεδο πάνω από το τοπικό σύστημα αρχείων. Κάθε αντίγραφο block αναπαρίσταται στον DataNode από δύο αρχεία. Το ένα αρχείο περιέχει δεδομένα, ενώ το άλλο αρχείο περιέχει τα μεταδεδομένα του συγκεκριμένου block, συμπεριλαμ-



Σχήμα 2.5: Αρχιτεκτονική και blocks στο HDFS.

βανομένων αθροισμάτων ελέγχου και της χρονοσφραγίδας δημιουργίας του block. Το HDFS προσφέρει τη δυνατότητα μικρορυθμίσεων ως προς τις παραμέτρους αποθήκευσης των αρχείων. Έτσι, υπάρχει η δυνατότητα επιμέρους ρύθμισης του μεγέθους block ανά αρχείο, όπως και του παράγοντα αντιγραφής ανά αρχείο.

Παράδειγμα 2.1. Στο Σχήμα 2.5 μπορούμε να δούμε ένα παράδειγμα αποθήκευσης blocks στο HDFS. Τα blocks αποθηκεύονται σε διακομιστές που ονομάζονται DataNodes, για τους οποίους θα πούμε περισσότερες λεπτομέρειες στη συνέχεια. Εδώ, έχουμε δύο αρχεία, το κυανό που αποτελείται από 3 blocks, με αρίθμηση από 1 έως και 3, και το πορτοκαλί που αποτελείται από 2 blocks, με αρίθμηση από 1 έως και 2. Ας πάρουμε την περίπτωση του κυανού αρχείου. Τα δύο πρώτα blocks έχουν ίδιο μέγεθος, λόγω χάρη 128 MB. Το τρίτο block, το οποίο είναι και το τελευταίο, μπορεί να έχει διαφορετικό μέγεθος. Ο παράγοντας αντιγραφής και για τα δύο αυτά αρχεία είναι ίσος με 2. Βλέπουμε ότι το block του κάθε αρχείου βρίσκεται σε δύο διαφορετικούς διακομιστές. Επίσης, παρατηρούμε ότι block από δύο διαφορετικά αρχεία μπορεί να βρίσκονται στον ίδιο διακομιστή. □

Στο πλαίσιο της Ομοσπονδίας HDFS, έχουν οριστεί οι έννοιες της *δεξαμενής block (block pool)* και του *τόμου χώρου ονομάτων (Namespace volume)*.

Η δεξαμενή block είναι ένα σύνολο από blocks που ανήκουν σε έναν μοναδικό χώρο ονομάτων (Namespace). Όπως είπαμε και πρωτύτερα, οι DataNodes αποθηκεύουν blocks για όλες τις δεξαμενές της συστάδας. Αυτή η διάταξη έχει πλεονεκτήματα. Καθώς κάθε δεξαμενή είναι ανεξάρτητη από την άλλη, τα αναγνωριστικά blocks μπορούν να χορηγούνται ανεξάρτητα από κάθε NameNode για κάθε δεξαμενή. Επιπλέον, σε περίπτωση αστοχίας ενός ομόσπονδου NameNode, οι υπόλοιποι δεν επηρεάζονται και συνεχίζουν κανονικά τη λειτουργία τους. Ένας χώρος ονομάτων και η αντίστοιχη δεξαμενή blocks ονομάζονται μαζί *τόμος χώρου ονομάτων* και αποτελούν μία αυτόνομη οντότητα διαχείρισης.

Όπως είπαμε, ο τρόπος με τον οποίο το HDFS κατορθώνει να επιβιώνει αστοχιών είναι η διάσπαση των αρχείων σε blocks και η εν συνεχεία δημιουργία αντιγράφων του κάθε block με αριθμό αντιγράφων ίσο με τον παράγοντα αντιγραφής. Αυτή η προσέγγιση, όμως, δημιουρ-

γεί προβλήματα. Πρώτα απ' όλα, αυτό της ανάγκης για πολλαπλάσιο αποθηκευτικό χώρο, καθώς κάθε block καταλαμβάνει χώρο ίσες φορές με την τιμή του παράγοντα αντιγραφής. Επιπλέον, με την ύπαρξη πολλαπλών αντιγράφων, καταναλώνεται πολλαπλάσιο εύρος ζώνης για τη μεταφορά των δεδομένων.

Για την αντιμετώπιση των παραπάνω ζητημάτων, από την έκδοση Hadoop 3.0 και έπειτα, το HDFS υποστηρίζει μηχανισμούς *κωδικοποίησης διαγραφής (Erasure Coding, EC)* αντί της ύπαρξης αντιγράφων, επιτυγχάνοντας την ίδια ανοχή σε σφάλματα και ταυτόχρονα σημαντική εξοικονόμηση χώρου³. Έτσι, αντί για την απαίτηση χώρου ίση με τον παράγοντα αντιγραφής, με τη χρήση EC απαιτείται επιπλέον χώρος αυξημένος μόνο κατά 50%. Αξίζει να σημειωθεί ότι στις διαμορφώσεις με EC ο παράγοντας αντιγραφής θα είναι ίσος με 1 καθώς δεν δημιουργούνται αντίγραφα των blocks.

Για να επιτευχθεί η ανοχή σε σφάλματα με τη χρήση EC, υπήρξαν ορισμένες τροποποιήσεις στον τρόπο λειτουργίας του HDFS. Έτσι, δημιουργήθηκαν επεκτάσεις στους NameNodes, στους DataNodes, αλλά και στα προγράμματα-πελάτη που να επιτρέπουν τη χρήση αυτής της τεχνικής. Επιπλέον, υλοποιήθηκαν μηχανισμοί οι οποίοι να επιτρέπουν την εφαρμογή διαφορετικών πολιτικών αντιγραφής και EC σε επίπεδο αρχείου.

Κλείνοντας, να σημειώσουμε ότι παρά όλα τα πλεονεκτήματα της χρήσης EC, υπάρχουν και μειονεκτήματα. Έτσι, απαιτείται υψηλότερη χρήση επεξεργαστικής ισχύος, συγκεκριμένες λειτουργίες του HDFS δεν υποστηρίζονται, σε περίπτωση αστοχίας υπάρχει επιβάρυνση για ανακατασκευή του block που χάθηκε και δεν υπάρχει η δυνατότητα εξισορρόπησης φόρτου, καθώς δεν υπάρχουν πολλαπλά αντίγραφα ενός block στα οποία να μπορούν να διανεμηθούν πολλαπλές αναγνώσεις.

2.4.4 Λειτουργίες αρχείων και διαχείριση blocks

Το μοντέλο που υποστηρίζει το HDFS είναι μονού εγγραφέα - πολλαπλών αναγνωστών (single writer - multiple reader). Όταν μία εφαρμογή-πελάτης έχει ανοίξει ένα αρχείο για εγγραφή, καμία άλλη αντίστοιχη εφαρμογή δεν έχει δικαίωμα εγγραφής. Αντιθέτως, αυτό δεν απαγορεύει σε άλλες εφαρμογές-πελάτες να πραγματοποιούν ταυτόχρονες αναγνώσεις από το αρχείο. Κάποιες από τις συνήθεις λειτουργίες που μπορούν να πραγματοποιηθούν σε ένα αρχείο είναι η εγγραφή, η ανάγνωση και η τροποποίηση ενός αρχείου. Το HDFS, ενώ επιτρέπει τις δύο πρώτες, δεν επιτρέπει την τρίτη. Έτσι, τα δεδομένα αρχείου εφόσον εγγραφούν δεν μπορούν να τροποποιηθούν. Η μόνη τροποποίηση που επιτρέπεται είναι αυτή της προσάρτησης δεδομένων σε ένα αρχείο [SKRC10]. Όταν μια εφαρμογή ξεκινά να εγγράφει σε ένα αρχείο, το κλειδώνει έτσι ώστε να αποκτήσει αποκλειστικό δικαίωμα εγγραφής.

Το HDFS ακολουθεί μια πολιτική τοποθέτησης των block η οποία στοχεύει τόσο στη μείωση του κόστους εγγραφής όσο και στη μεγιστοποίηση της διαθεσιμότητας και ανοχής σε σφάλματα. Καθώς το HDFS προορίζεται για χρήση σε μεγάλα datacenters, είναι λογικό ότι οι κόμβοι που θα το φιλοξενούν θα βρίσκονται τοπολογικά σε διαφορετικά ικρίωματα. Το HDFS ορίζει αποστάσεις μεταξύ κόμβων επιτρέποντας να οριστεί η φυσική τους τοπολογία σε μια συστάδα και υπολογίζοντας τις αποστάσεις βάσει του πλήθους των διαφορετικών δια-

³<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSErasureCoding.html>

μεταγωγέων (switches) που θα πρέπει να παρεμβληθούν μεταξύ δύο κόμβων, θεωρώντας ότι κόμβοι στο ίδιο ικρίωμα μοιράζονται έναν διαμεταγωγέα και ότι τα ικρίωματα συνδέονται με διαμεταγωγείς.

Η προκαθορισμένη πολιτική του HDFS έχει ως εξής. Όταν ένα νέο block δημιουργείται, το HDFS τοποθετεί το πρώτο αντίγραφο στον κόμβο στον οποίο εκτελείται η εφαρμογή που πραγματοποιεί την εγγραφή. Το δεύτερο και το τρίτο αντίγραφο τοποθετούνται σε κόμβους σε διαφορετικό ικρίωμα. Η στρατηγική αυτή ακολουθείται για λόγους ανοχής σε σφάλματα. Όταν ολοκληρωθεί η επιλογή αριθμού κόμβων ίσου με τον παράγοντα αντιγραφής, αυτά θεωρούνται σε αύξουσα διάταξη απόστασης από τον πρώτο κόμβο στον οποίο θα πραγματοποιηθεί η εγγραφή η οποία ξεκινά με μορφή διασωλήνωσης, γράφοντας πακέτα με αυτήν τη σειρά στους κόμβους, χωρίς επιβεβαίωση εγγραφής μεταξύ διαδοχικών πακέτων.

Κατά την ανάγνωση, ακολουθείται αντίστοιχη διαδικασία. Ο πελάτης που πρόκειται να διαβάσει ένα αρχείο λαμβάνει από τον NameNode τα block και τις τοποθεσίες τους σε αύξουσα απόσταση από την τοποθεσία της διεργασίας που θα πραγματοποιήσει την ανάγνωση. Έτσι, όταν μια διεργασία επιχειρήσει την ανάγνωση ενός block, θα επιλέξει αρχικά τον κοντινότερο κόμβο σε αυτήν, σύμφωνα με τη σειρά που της έχει δοθεί. Εάν για οποιονδήποτε λόγο η ανάγνωση αποτύχει, δοκιμάζει τον δεύτερο κοντινότερο κόμβο, κ.ο.κ.

2.4.5 Εντολές στο HDFS

Οι εντολές στο HDFS μπορούν να χωριστούν σε τρεις κατηγορίες. Είναι οι εντολές χρήστη, οι εντολές διαχείρισης και οι εντολές αποσφαλμάτωσης. Καθώς το HDFS εξελίσσεται διαρκώς, η παρουσίαση των εντολών αυτών δεν μπορεί να εξαντληθεί, αλλά θα περιοριστεί σε βασικό επίπεδο. Ο ενδιαφερόμενος αναγνώστης μπορεί να επισκεφθεί την τεκμηρίωση του HDFS [HDFa] για έναν πλήρη κατάλογο υποστηριζόμενων εντολών. Οι εντολές αυτές εισάγονται μέσω του script `bin/hdfs`.

- Κατάλογος αρχείων:

```
hdfs dfs -ls
```

Αντιστοίχως, μπορούν να χρησιμοποιηθούν οι σχετικές εντολές UNIX/Linux συστημάτων για διαχείριση αρχείων βάζοντας όπου `ls` την εντολή: `cp`, `mv`, `rm`, κ.ο.κ.

- Ελεύθερος χώρος στο HDFS:

```
hdfs dfs -df -h
```

- Χώρος που καταλαμβάνουν επιμέρους τα αρχεία και οι φάκελοι:

```
hdfs dfs -du -h
```

- Αντιγραφή αρχείων ή και καταλόγων από το HDFS στο τοπικό σύστημα αρχείων:

```
hdfs dfs -get προέλευση προορισμός
```

Αν ο προορισμός απαλειφθεί, η λήψη γίνεται στον τρέχοντα κατάλογο.

- Αντιγραφή αρχείων ή και καταλόγων από το τοπικό σύστημα αρχείων στο HDFS:


```
hdfs dfs -put προέλευση προορισμός
```

- Αντιγραφή αρχείων εντός του HDFS:

```
hdfs dfs -cp προέλευση προορισμός
```

- Μετακίνηση αρχείων/καταλόγων εντός του HDFS:

```
hdfs dfs -mv προέλευση προορισμός
```

- Βασικές πληροφορίες και στατιστικά του HDFS:

```
hadoop/bin/hdfs dfsadmin -report
```

- Ρύθμιση της κατάστασης ασφαλούς λειτουργίας του NameNode:

```
hadoop/bin/hdfs dfsadmin -safemode εντολή
```

Ενδεικτικά, η εντολή μπορεί να είναι enter, leave, forceExit, κ.λπ.

- Αποδέσμευση DataNode από το HDFS: Στο αρχείο hdfs-site.xml στον διακομιστή του NameNode προσθέτουμε τα παρακάτω:

```
1 <property>
2   <name>dfs.hosts.exclude</name>
3   <value>/home/bigdata/hadoop/etc/hadoop/mydfs.exclude</value>
4 </property>
```

Στο αρχείο mydfs.exclude που ορίσαμε παραπάνω παραθέτουμε ένα προς ένα, ανά γραμμή, τα ονόματα των διακομιστών που επιθυμούμε να αποδεσμεύσουμε:

```
1 hostname1
2 hostname2
```

Στη συνέχεια, εκτελούμε την παρακάτω εντολή για να ενημερωθεί η διαμόρφωση του HDFS από τον NameNode:

```
hdfs dfsadmin -refreshNodes
```

Μπορούμε να ελέγξουμε την ολοκλήρωση της διαδικασίας εκτελώντας όπως και πριν:

```
hdfs dfsadmin -report
```

2.5 Συζήτηση

Σε αυτό το κεφάλαιο εξοικειωθήκαμε με μία σειρά συστημάτων αρχείων αποθήκευσης για μεγάλα δεδομένα. Περιγράψαμε το GlusterFS, το Ceph και το Alluxio, που αποτελούν ανοικτού λογισμικού προσεγγίσεις αποθήκευσης για συστάδες, μελετήσαμε το Swift του OpenStack ως λύση αποθήκευσης στο Νέφος, αλλά αναφέραμε και λύσεις από μεγάλους παρόχους όπως η Google και η Amazon. Τέλος, μελετήσαμε εις βάθος το HDFS, το κατακευματισμένο σύστημα αρχείων του Hadoop.

Διαπιστώνουμε ότι υπάρχει πληθώρα διαφορετικών προσεγγίσεων για την κατακευματισμένη αποθήκευση δεδομένων. Καθεμία προσέγγιση μπορεί να χρησιμοποιηθεί για διαφορετικό

σκοπό, ενώ κάποιες μπορούν να χρησιμοποιηθούν και συνδυαστικά, όπως το Alluxio και το HDFS. Παρ' όλα αυτά, βλέποντας τη γενική εικόνα μπορούμε να εξαγάγουμε κάποια γενικά συμπεράσματα.

Καταρχάς, μπορούμε να διαπιστώσουμε ότι υπάρχουν δύο προσεγγίσεις για τη διάταξη των κόμβων και τον εντοπισμό υπηρεσιών.

Η πρώτη προσέγγιση είναι η διάταξη master/slave. Αυτή τη διάταξη ακολουθούν το GlusterFS, το Alluxio και το HDFS. Σε αυτήν τη διάταξη, ο master χρησιμοποιείται για να διατηρεί πληροφορίες για τη συστάδα και αποτελεί σημείο επαφής με τους πελάτες. Ο master δεν αναλαμβάνει άλλες εργασίες, όπως αναγνώσεις ή εγγραφές αρχείων. Αυτή είναι η δουλειά των workers. Επειδή οι masters αποτελούν κομβικό αλλά και πιθανό σημείο κατάρρευσης, ακολουθούνται προσεγγίσεις με επιπλέον διακομιστές σε κατάσταση αναμονής. Από την άλλη πλευρά, οι workers αναλαμβάνουν τον φόρτο της διακίνησης των δεδομένων και την ενημέρωση του master για την κατάστασή τους.

Η δεύτερη προσέγγιση εμπλέκει μεθόδους κατακερματισμού, όπως συμβαίνει με την περίπτωση του Ceph και του OpenStack Swift. Εδώ, δεν υπάρχει κεντροποιημένος διακομιστής που να διατηρεί πληροφορίες υπό τη μορφή ευρετηρίου για τη θέση των δεδομένων, αλλά χρησιμοποιούνται αλγόριθμοι κατακερματισμού οι οποίοι μέσω υπολογισμών εντοπίζουν τον κόμβο αποθήκευσης του κάθε αντικειμένου.

Η δεύτερη διαπίστωση που μπορούμε να κάνουμε είναι ότι όλες οι προσεγγίσεις δεν αφορούν συστήματα αρχείων μόνιμης αποθήκευσης. Έτσι, το Alluxio διαφοροποιείται από τα υπόλοιπα συστήματα που παρουσιάσαμε, καθώς αποτελεί ένα εικονικό σύστημα αρχείων στη μνήμη, λειτουργώντας ως ενδιάμεσο επίπεδο μεταξύ των πελατών και του συστήματος αρχείων μόνιμης αποθήκευσης.

Τέλος, η τρίτη διαπίστωση που κάνουμε είναι ότι τα συστήματα αρχείων δεν αποθηκεύουν μόνο αρχεία, με την παραδοσιακή έννοια, αλλά blocks και αντικείμενα, όπως το Ceph ή το Swift ή οι λύσεις που προσφέρουν οι πάροχοι Νέφους.

2.6 Περαιτέρω μελέτη

Σε κάθε περίπτωση, το θέμα της κατανεμημένης αποθήκευσης αρχείων δεν εξαντλείται εδώ. Η περιγραφή που έγινε δεν είναι εξαντλητική και πρέπει να θεωρηθεί μόνο ως έναυσμα για περαιτέρω μελέτη. Έτσι, κανείς μπορεί να μελετήσει τις ακόλουθες πηγές για το GlusterFS: [Glu, MMBSS20, DO13, BBP12]. Για το Ceph, εκτός από τον ιστότοπό του [Cep], πολύ περισσότερες λεπτομέρειες σχετικά με τον σχεδιασμό του υπάρχουν στην αντίστοιχη δημοσιευμένη εργασία [WBM⁺06], αλλά και στις εργασίες που περιγράφουν τον αλγόριθμο CRUSH [WBM06] και το σύστημα αποθήκευσης αντικειμένων RADOS [WLB07] πάνω στα οποία στηρίζεται το Ceph. Ο ιστότοπος του Alluxio [all] περιλαμβάνει πολλές λεπτομέρειες σχετικές με την εγκατάσταση και λειτουργία του, ενώ οι παρακάτω εργασίες αναφέρουν περισσότερα στοιχεία σχετικά με την αρχιτεκτονική και τον σχεδιασμό του: [LGZ⁺14, Li18]. Το OpenStack [ope] αποτελεί ένα λειτουργικό σύστημα Νέφους με άλλα δύο υποσυστήματα αποθήκευσης, το Manilla [man] και το Cinder [cin] εκτός του Swift, για το οποίο κανείς μπορεί να ανατρέξει και σε σχετικά βιβλία [Arn14, Mar16]. Για τους παρόχους αποθήκευσης νέφους, υπάρχουν

οι σχετικοί ιστότοποι οι οποίοι ενημερώνονται τακτικά [azu, s3, goo]. Για το HDFS, η αντίστοιχη εργασία [SKRC10] αποτελεί μια καλή αρχή, ενώ ο δικτυακός τόπος του έργου του Apache Hadoop [Had] είναι ενημερωμένος, περιλαμβάνοντας όλες τις σχετικές εξελίξεις στο σύστημα, όπως οι διατάξεις υψηλής διαθεσιμότητας, το ομόσπονδο HDFS κ.λπ. Τέλος, μια σειρά από βιβλία [Whi12, WSV14] μπορούν να βοηθήσουν στην περαιτέρω κατανόηση.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [all] Alluxio Arxhitecture. <https://docs.alluxio.io/os/user/stable/en/overview/Architecture.html>. Accessed: 2021-12-24.
- [Arn14] Joe Arnold. *Openstack swift: Using, administering, and developing for swift object storage*. "O'Reilly Media, Inc.", 2014.
- [azu] Azure Storage. <https://docs.microsoft.com/en-us/azure/storage/common/storage-introduction>. Accessed: 2021-12-18.
- [BBP12] Eric B Boyer, Matthew C Broomfield, and Terrell A Perrotti. Glusterfs one storage server to rule them all. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2012.
- [Cep] Ceph Architecture. <https://docs.ceph.com/en/pacific/architecture/>. Accessed: 2021-12-15.
- [cin] OpenStack Cinder. <https://docs.openstack.org/cinder/latest/>. Accessed: 2021-12-18.
- [CWO⁺11] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, et al. Windows azure storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 143–157, 2011.
- [DO13] Alex Davies and Alessandro Orsaria. Scale out with glusterfs. *Linux Journal*, 2013(235):1, 2013.
- [Glu] GlusterFS. <https://docs.gluster.org/en/latest/Administrator-Guide/>. Accessed: 2021-11-30.
- [goo] Google Cloud Storage. <https://cloud.google.com/products/storage>. Accessed: 2021-12-18.
- [Had] Apache Hadoop. <http://hadoop.apache.org/>. Accessed: 2021-09-30.
- [HDFa] HDFS Commands. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html>. Accessed: 2021-11-30.

- [HDFb] HDFS High Availability. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSHighAvailabilityWithNFS.html>. Accessed: 2021-11-30.
- [LGZ⁺14] Haoyuan Li, Ali Ghodsi, Matei Zaharia, Scott Shenker, and Ion Stoica. Tachyon: Reliable, memory speed storage for cluster computing frameworks. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 1–15, 2014.
- [Li18] Haoyuan Li. *Alluxio: A virtual distributed file system*. University of California, Berkeley, 2018.
- [man] OpenStack Manilla. <https://wiki.openstack.org/wiki/Manila>. Accessed: 2021-12-18.
- [Mar16] Andrey Markelov. *Certified OpenStack Administrator Study Guide*. Springer, 2016.
- [MMBSS20] Divya Muntimadugu, Bhavana Mohanraj, Laura Bailey, and Anjana Sriram Suparna. *Red Hat Gluster Storage 3.3 Administration Guide*. "Red Hat", 2020.
- [ope] OpenStack. <https://www.openstack.org/software/>. Accessed: 2021-12-18.
- [s3] Amazon S3. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html#CoreConcepts>. Accessed: 2021-12-18.
- [SKRC10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [WBM⁺06] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell DE Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Proceedings of the 7th symposium on Operating systems design and implementation*, pages 307–320, 2006.
- [WBMM06] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, and Carlos Maltzahn. Crush: Controlled, scalable, decentralized placement of replicated data. In *SC'06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, pages 31–31. IEEE, 2006.
- [Whi12] Tom White. *Hadoop: The definitive guide*. "O'Reilly Media, Inc.", 2012.
- [WLBM07] Sage A. Weil, Andrew W. Leung, Scott A. Brandt, and Carlos Maltzahn. Rados: a scalable, reliable storage service for petabyte-scale storage clusters. In *Proceedings of the 2nd international workshop on Petascale data storage: held in conjunction with Supercomputing'07*, pages 35–44, 2007.
- [WSV14] Sameer Wadkar, Madhu Siddalingaiah, and Jason Venner. *Pro Apache Hadoop*. Springer, 2014.

ΕΙΣΑΓΩΓΗ ΣΤΟ MAPREDUCE

Περιεχόμενα

3.1	Εισαγωγή	50
3.2	Κατανοώντας το MapReduce	51
3.3	Μείξη και ταξινόμηση	57
3.4	Ροή δεδομένων	58
3.5	Ειδικά θέματα του MapReduce	59
3.5.1	Συνδυαστές και διαμεριστές	59
3.5.2	Συνδυαστές	59
3.5.3	Διαμεριστές	61
3.5.4	Δευτερεύουσα ταξινόμηση	61
3.5.5	Αλυσίδες εργασιών MapReduce	62
3.5.6	Μη ανάγκη χρήσης reducer	62
3.6	Σχεδιασμός αλγορίθμων MapReduce	63
3.6.1	Περιγραφή του προβλήματος	63
3.6.2	Σχεδιασμός αλγορίθμου	63
3.6.3	Υλοποίηση σε Java	65
3.6.4	Εκτέλεση υλοποίησης Java	67
3.6.5	Τροποποίηση για χρήση συνδυαστή	68
3.6.6	Υλοποίηση σε Python	69
3.6.7	Εκτέλεση υλοποίησης σε Python	71
3.6.8	Πλήθος διεργασιών map και reduce	72
3.7	Συζήτηση	73
3.8	Περαιτέρω μελέτη	73
	BIBΛΙΟΓΡΑΦΙΑ	73

Σύνοψη

Στο κεφάλαιο αυτό θα ασχοληθούμε με τα εξής θέματα:

- Περιγραφή και παρουσίαση του μοντέλου υπολογισμού *MapReduce*.
- Μεθοδολογία σχεδίασης αλγορίθμων χρησιμοποιώντας το μοντέλο *MapReduce*.
- Ανάπτυξη προγραμμάτων για αλγορίθμους *MapReduce* για το *Apache Hadoop* χρησιμοποιώντας τις γλώσσες *Java* και *Python*.
- Εκτέλεση προγραμμάτων *MapReduce* στο *Apache Hadoop*.

Προαπαιτούμενη γνώση

Για την κατανόηση αυτού του κεφαλαίου είναι χρήσιμη η εξοικείωση με θέματα σχεδίασης αλγορίθμων. Επίσης, είναι απαραίτητη η γνώση των γλωσσών προγραμματισμού *Java* ή/και *Python*. Τέλος, καθώς τα παραδείγματα είναι σχεδιασμένα για εκτέλεση σε περιβάλλον *Linux*, η εξοικείωση με το εν λόγω λειτουργικό σύστημα είναι ενδεδειγμένη.

3.1 Εισαγωγή

Ο όρος *MapReduce* μπορεί να αφορά δύο έννοιες. Η πρώτη είναι το προγραμματιστικό μοντέλο το οποίο εισήχθη από την Google [DG04] για μαζική επεξεργασία δεδομένων μεγάλης κλίμακας. Η δεύτερη αφορά τον μηχανισμό που υλοποιεί το συγκεκριμένο προγραμματιστικό μοντέλο στη μηχανή ανοικτού κώδικα *Apache Hadoop*. Σε αυτό το κεφάλαιο θα εστιάσουμε κυρίως στην πρώτη έννοια, ενώ θα παρουσιάσουμε παραδείγματα τα οποία υλοποιούνται και στο *MapReduce* του *Hadoop*.

Αυτό που έχει αλλάξει στην επεξεργασία δεδομένων τα τελευταία χρόνια αφορά κυρίως τον όγκο των δεδομένων που υφίστανται επεξεργασία. Για να γίνει αυτό εφικτό, σύμφωνα με τους Lin και Dyer [LD10], προέκυψε μία σειρά από ιδέες και προσεγγίσεις, οι οποίες υλοποιούνται και στον μηχανισμό του *MapReduce* και οι κυριότερες είναι οι παρακάτω:

Οριζόντια έναντι κατακόρυφης κλιμάκωσης Με τον όρο *κατακόρυφη κλιμάκωση* αναφερόμαστε στην αναβάθμιση των δυνατοτήτων ενός συστήματος με τη χρήση περισσότερης μνήμης RAM, μεγαλύτερου σκληρού δίσκου, κ.ο.κ. Με αυτόν τον τρόπο τα περιθώρια αναβάθμισης είναι συγκεκριμένα, ενώ για να υπάρξει η δυνατότητα χρήσης πολλών επεξεργαστών απαιτείται εξειδικευμένο υλικό, το οποίο αυξάνει κατακόρυφα το κόστος. Επιπλέον, τα στοιχεία της αναβάθμισης θα πρέπει να είναι συμβατά με το υφιστάμενο υλικό. Με τον όρο *οριζόντια κλιμάκωση* αναφερόμαστε στη δυνατότητα ενός συστήματος να αυξήσει τις δυνατότητές του προσθέτοντας και άλλες υπολογιστικές μηχανές. Οι μηχανές αυτές μπορεί να μην είναι τελευταίας τεχνολογίας. Έτσι, υπάρχει η δυνατότητα βελτίωσης των υπολογιστικών δυνατοτήτων με χαμηλό κόστος και με μηχανές που προέρχονται από διαφορετικούς προμηθευτές.

Οι αστοχίες είναι σύνηθες φαινόμενο. Καταλήγουμε, λοιπόν, σε μια διάταξη στην οποία έχουμε πολλούς υπολογιστές που σχηματίζουν ένα κατανεμημένο σύστημα. Αυτοί οι υπολογιστές μπορεί να είναι εκατοντάδες ή και χιλιάδες. Αυξάνοντας το πλήθος των υπολογιστών, αυξάνεται και η πιθανότητα αστοχίας σε οποιοδήποτε στοιχείο υλικού, από έναν σκληρό δίσκο έως και την τροφοδοσία. Πώς θα μπορούσε κάτι τέτοιο να επηρεάσει έναν υπολογισμό; Αρκεί να σκεφτούμε ότι πραγματοποιούμε υπολογισμούς σε δεδομένα μεγάλης κλίμακας. Οι υπολογισμοί αυτοί μπορεί να διαρκέσουν πολλές ώρες, ίσως και μέρες. Μια οποιαδήποτε αστοχία υλικού (ή και λογισμικού ακόμη) δεν θα πρέπει να οδηγεί σε ματαίωση του υπολογισμού, καθώς σε μια τέτοια περίπτωση ο υπολογισμός θα έπρεπε να ξεκινήσει από την αρχή. Αντιθέτως, θα πρέπει να υπάρχει μηχανισμός εντοπισμού σφαλμάτων και ανάκαμψης από αυτά, ώστε παρά τη δεδομένη ύπαρξη αστοχιών οι υπολογισμοί μας να ολοκληρώνονται χωρίς να πρέπει να επανεκκινήσουμε την εκτέλεση του προγράμματός μας.

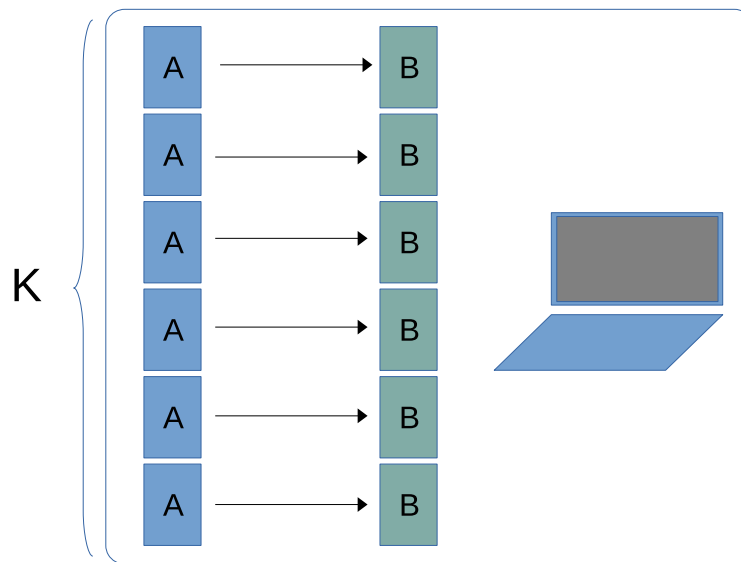
Μετακινούμε τον υπολογισμό, όχι τα δεδομένα. Τα δεδομένα έχουν μεγάλο όγκο, γι' αυτό και ο όρος «Big Data». Αν και μπορούμε να τα μετακινήσουμε για να τα επεξεργαστούμε, κάτι τέτοιο θα κόστιζε πολύ και από άποψη χρόνου και από άποψη πόρων. Αντί αυτού, θα ήταν προτιμότερο να μετακινήσουμε τα προγράμματα που θα επεξεργαστούν τα δεδομένα αυτά, τα οποία έχουν πολύ μικρό μέγεθος σε σύγκριση με τα δεδομένα που επιθυμούμε να επεξεργαστούμε. Χρειαζόμαστε, λοιπόν, έναν μηχανισμό ο οποίος θα επιτρέπει τη μετακίνηση των προγραμμάτων μας με τέτοιον τρόπο ώστε, κατά την επεξεργασία των δεδομένων, να πραγματοποιούνται τοπικά οι υπολογισμοί με όσο το δυνατό μικρότερη μετακίνηση δεδομένων.

Σειριακή επεξεργασία με τη μορφή ροής. Καθώς τα δεδομένα μας εκ των πραγμάτων είναι μεγάλα, αυτά δεν χωρούν στην κύρια μνήμη. Αυτό σημαίνει ότι θα πρέπει να πραγματοποιούνται διαρκείς αναγνώσεις από τη μόνιμη αποθήκευση. Οι αναγνώσεις αυτές θα πρέπει να παράγουν μια συνεχή ροή δεδομένων προς επεξεργασία, ώστε να μην υπάρχουν καθυστερήσεις στον υπολογισμό. Κάτι τέτοιο δεν θα πρέπει να επιβραδύνεται από τους χρόνους αναζήτησης στους δίσκους. Έτσι, θα πρέπει να υπάρξει μηχανισμός που θα προσφέρει σειριακή επεξεργασία στα δεδομένα.

Απόκρυψη του συστήματος. Όπως είπαμε και παραπάνω, είναι επιθυμητό να πραγματοποιήσουμε τους υπολογισμούς μας σε ένα σύστημα το οποίο είναι πολύ πιθανό να αποτελείται και από ετερογενές υλικό. Κάτι τέτοιο δεν θα πρέπει να επηρεάζει τον τρόπο προγραμματισμού. Επιπλέον, θα επιθυμούσαμε ο προγραμματιστής να επικεντρωθεί στην υλοποίηση του υπολογισμού παρέχοντας ένα υψηλού επιπέδου προγραμματιστικό μοντέλο, χωρίς να χρειάζεται να ανησυχεί για προβλήματα όπως κλειδώματα και συνθήκες ανταγωνισμού.

3.2 Κατανοώντας το MapReduce

Ένα από τα χαρακτηριστικά του MapReduce ως προγραμματιστικού μοντέλου είναι η δυνατότητά του να αποκρύψει πολλά στοιχεία του μηχανισμού που πραγματοποιεί την παράλληλη επεξεργασία. Έτσι, ο προγραμματιστής δεν χρειάζεται να ανησυχεί για θέματα όπως κλειδώματα, συνθήκες ανταγωνισμού, αδιέξοδα, κ.λπ. Το κόστος για αυτά τα οφέλη είναι η ανάγκη



Σχήμα 3.1: Μέτρηση γραμμών αρχείων από έναν υπολογιστή.

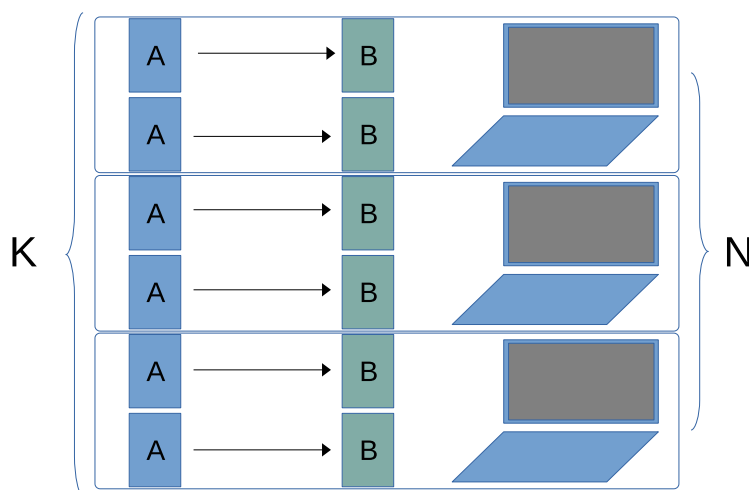
εκμάθησης του προγραμματιστικού μοντέλου MapReduce, του τρόπου προγραμματισμού σε αυτό, καθώς και των μηχανισμών οι οποίοι παρασκηνιακά επιτυγχάνουν την εκτέλεση των καταναμημένων εφαρμογών.

Το πρώτο βήμα για την κατανόηση αυτού του προγραμματιστικού μοντέλου θα είναι μια γενική περιγραφή του ρόλου των βημάτων map και reduce. Έτσι, κατά το βήμα map πραγματοποιείται παράλληλη επεξεργασία χωρίς να υπάρχει δυνατότητα ανταλλαγής δεδομένων, ενώ κατά το βήμα reduce πραγματοποιείται η συγχώνευση και ανταλλαγή δεδομένων. Ο συνδυασμός ενός βήματος map με ένα βήμα reduce αποτελεί έναν κύκλο MapReduce. Για την επίλυση ενός προβλήματος μπορεί να απαιτούνται περισσότεροι από ένας κύκλοι MapReduce.

Θα διαλευκάνουμε την κατάσταση μέσα από μια σειρά από παραδείγματα.

Παράδειγμα 3.1. Έστω ότι έχουμε K αρχεία και θέλουμε να μετρήσουμε το πλήθος των γραμμών σε κάθε ένα από αυτά. Αν έχουμε έναν υπολογιστή στη διάθεσή μας, αυτό που μπορούμε να κάνουμε είναι να επεξεργαστούμε σειριακά τα αρχεία αυτά και να πραγματοποιήσουμε τη μέτρηση του συνολικού πλήθους των γραμμών για το κάθε ένα αρχείο, όπως φαίνεται στο Σχήμα 3.1. Είναι εμφανές ότι ο χρόνος που θα απαιτηθεί για την εργασία αυτή είναι ανάλογος τόσο του πλήθους γραμμών σε κάθε αρχείο όσο και του πλήθους των αρχείων. Θα κάνουμε μια υπόθεση και θα θεωρήσουμε ότι το πλήθος των γραμμών είναι της ίδιας τάξης μεγέθους, επομένως ο κύριος παράγοντας που επηρεάζει τον χρόνο εκτέλεσης είναι το πλήθος των αρχείων. Έτσι, για K αρχεία, ο χρόνος εκτέλεσης θα είναι ανάλογος του K . Αν τα αρχεία αυτά είναι σχετικά μεγάλα (π.χ. αρχεία καταγραφής ιστορικού ιστοτόπων) και πολλά σε αριθμό, είναι εμφανές ότι η ολοκλήρωση της μέτρησης θα είναι μια χρονοβόρα διαδικασία. Κάτι τέτοιο απεικονίζεται στο Σχήμα 3.1.

Το ερώτημα που προκύπτει τώρα είναι αν θα μπορούσαμε να κάνουμε κάτι πιο γρήγορα. Μια απλή ιδέα είναι, καθώς το πλήθος των αρχείων είναι πολύ μεγάλο, να χρησιμοποιήσουμε περισσότερους από έναν υπολογιστές για τη δουλειά αυτή. Έτσι, αν έχουμε N υπολογιστές, μπορούμε να αναθέσουμε K/N αρχεία σε κάθε υπολογιστή και να περιμένουμε ότι ο χρόνος



Σχήμα 3.2: Μέτρηση γραμμών αρχείων από K υπολογιστές.

ολοκλήρωσης αυτής της εργασίας θα είναι ανάλογος του K/N , όπως απεικονίζεται στο Σχήμα 3.2. $\square$

Παράδειγμα 3.2. Έστω ότι έχουμε ένα μεγάλο πλήθος από αρχεία ήχου τα οποία θέλουμε να μετατρέψουμε σε μορφότυπο mp3. Και πάλι, είναι εμφανές ότι, αν χρησιμοποιήσουμε έναν υπολογιστή, η επεξεργασία θα πρέπει να γίνει σειριακά. Έχουμε, λοιπόν, μια κατάσταση αντίστοιχη με το προηγούμενο παράδειγμα. Και πάλι, αν χρησιμοποιήσουμε N υπολογιστές, η εργασία αυτή θα πραγματοποιηθεί N φορές γρηγορότερα, με παρόμοιο τρόπο με πριν. $\square$

Παράδειγμα 3.3. Έστω ότι έχουμε ένα σύστημα MOOC (Massive Open Online Course) στο οποίο εκπαιδευόμενοι μαθαίνουν να προγραμματίζουν στη γλώσσα προγραμματισμού C. Για να ελέγξουν την πρόοδό τους, υποβάλλουν εργασίες οι οποίες ελέγχονται με αυτοματοποιημένο τρόπο μέσω λογισμικού. Κάθε προγραμματιστική εργασία μεταγλωττίζεται, εκτελείται και η έξοδός της συγκρίνεται με ένα πρότυπο αποτέλεσμα. Καθώς οι εργασίες αυτές είναι πολλές σε αριθμό, η χρήση ενός μόνο υπολογιστή θα επέφερε σημαντικές καθυστερήσεις. Και πάλι, αυτό που μπορούμε να κάνουμε είναι να χρησιμοποιήσουμε περισσότερους υπολογιστές ώστε να επιταχυνθεί η διαδικασία. Έτσι, και πάλι, για K εργασίες και N υπολογιστές, μπορούμε να αναθέσουμε K/N εργασίες σε κάθε υπολογιστή και να αναμένουμε ολοκλήρωση και εξαγωγή των αποτελεσμάτων σε K/N χρόνο. $\square$

Εάν δούμε προσεκτικά τα παραπάνω παραδείγματα, θα παρατηρήσουμε ότι υπάρχει μία σειρά από επαναλαμβανόμενες ενέργειες. Υπάρχει ένα πρότυπο. Έχουμε ένα μεγάλο πλήθος από δεδομένα που επιθυμούμε να επεξεργαστούμε. Κάθε ένα από τα στοιχεία δεδομένων στη συνέχεια υφίσταται κάποια επεξεργασία και παράγει κάποιο αποτέλεσμα. Η επεξεργασία μπορεί να είναι μέτρηση, όπως στην περίπτωση της μέτρησης του αριθμού γραμμών κάθε αρχείου, μετασχηματισμός και συμπίεση κάποιου αρχείου, όπως στην περίπτωση των αρχείων ήχου που προαναφέραμε, ή και μεταγλώττιση και εκτέλεση προγραμμάτων, όπως στην περίπτωση των προγραμματιστικών εργασιών.

Μπορούμε να πούμε, γενικά, ότι αυτή η επεξεργασία αποτελεί μια *απεικόνιση* (mapping) από έναν τύπο δεδομένων A σε κάποιον άλλο τύπο δεδομένων B , όπως απεικονίζεται στο Σχήμα 3.1 Έτσι, στην περίπτωση της μέτρησης των γραμμών του αρχείου, πραγματοποιούμε

απεικόνιση από ένα αρχείο σε έναν αριθμό, ο οποίος αντιστοιχεί στο πλήθος των γραμμών του. Στην περίπτωση των αρχείων ήχου, πραγματοποιούμε απεικόνιση από τον έναν τύπο αρχείων ήχου στον άλλο, ενώ στην περίπτωση των προγραμματιστικών εργασιών πραγματοποιούμε απεικόνιση από ένα αρχείο κώδικα σε μία αριθμητική τιμή και πάλι, η οποία θα είναι η αξιολόγηση (βαθμολογία) της συγκεκριμένης εργασίας.

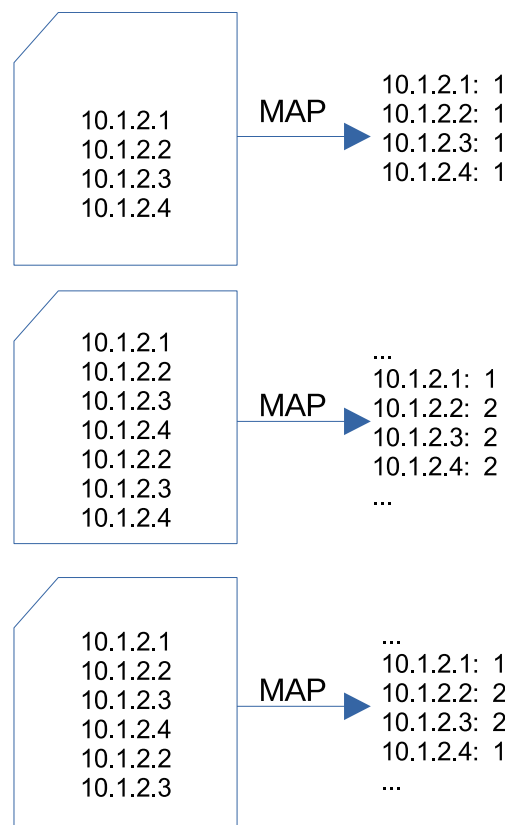
Επιπλέον, παρατηρούμε και κάτι ακόμη. Η επεξεργασία κάθε ενός στοιχείου δεδομένων είναι ανεξάρτητη από την επεξεργασία των υπολοίπων. Αυτό μας δίνει τη δυνατότητα να διασπάσουμε την επεξεργασία των δεδομένων αυτών σε περισσότερους από έναν υπολογιστές, χωρίς να επηρεάζεται το αποτέλεσμα. Αυτό συμβαίνει και κατά τη λειτουργία του βήματος *map* στο μοντέλο MapReduce. Στο σημείο αυτό, ας δούμε ένα παράδειγμα ακόμη, εμπνευσμένο από το [LD10].

Παράδειγμα 3.4. Έστω ότι έχουμε ένα μεγάλο πλήθος αρχείων καταγραφής και θέλουμε να μετρήσουμε σε κάθε ένα από αυτά τις συχνότητες εμφάνισης κάθε μίας διεύθυνσης IP, δηλαδή πόσες φορές εμφανίζεται κάθε διεύθυνση στο συγκεκριμένο αρχείο. Σύμφωνα με όσα είπαμε παραπάνω, και πάλι, αν έχουμε K αρχεία και N διαθέσιμους υπολογιστές, μπορούμε να αναθέσουμε σε κάθε έναν υπολογιστή K/N αρχεία και να πραγματοποιήσουμε την αντίστοιχη επεξεργασία. Σε αυτήν την περίπτωση, μπορούμε να θεωρήσουμε ότι χρησιμοποιούμε έναν πίνακα κατακερματισμού που σε κάθε θέση του υπάρχει και ένας μετρητής. Κάθε φορά που διαβάζουμε μια διεύθυνση IP από το αρχείο, πραγματοποιούμε κατακερματισμό με την ίδια συνάρτηση κατακερματισμού και στη συνέχεια αυξάνουμε τον αντίστοιχο μετρητή.

Όταν ολοκληρωθεί η ανάγνωση του αρχείου, παράγεται ως έξοδος το αποτέλεσμα το οποίο βρίσκεται αποθηκευμένο στον πίνακα κατακερματισμού. Για κάθε αρχείο η έξοδος αυτή θα έχει τη μορφή ζευγών κλειδιού-τιμής. Το κλειδί θα είναι η κάθε μία διεύθυνση IP που εντοπίστηκε, ενώ η τιμή θα είναι η συχνότητα της διεύθυνσης IP στο εν λόγω κείμενο. Πόσες φορές, δηλαδή, εντοπίστηκε αυτή η διεύθυνση. Εδώ, θα πρέπει να επισημάνουμε ότι έχουμε πραγματοποιήσει σιωπηρά μια υπόθεση: ότι ο πίνακας κατακερματισμού που χρησιμοποιήσαμε χωράει στην κύρια μνήμη. Θα αναφερθούμε στη συνέχεια στις επιπλοκές της θεώρησης αυτής.

Έχουμε καταλήξει, λοιπόν, με μετρήσεις για διαφορετικές διευθύνσεις σε διαφορετικά αρχεία, όπως βλέπουμε και στο Σχήμα 3.3. Στο σημείο αυτό, όμως, δημιουργείται το εύλογο ερώτημα: Και αν θέλαμε να μετρήσουμε πόσες φορές εμφανίζεται κάθε διεύθυνση σε *ΟΛΑ* τα αρχεία; Είναι εμφανές ότι, με τον μηχανισμό της απεικόνισης που περιγράψαμε παραπάνω, αυτό δεν είναι ευθέως εφικτό, καθώς ο υπολογισμός για κάθε αρχείο πραγματοποιείται ανεξάρτητα. Έτσι, π.χ. για τη διεύθυνση 10.1.2.3 θα έχουμε 1 εμφάνιση στο αρχείο 1, 2 εμφανίσεις στο αρχείο 2, κ.ο.κ. Εάν η διεύθυνση αυτή δεν απαντάται σε κάποιο αρχείο, δεν θα υπάρχει καν στην έξοδο. Δεν θα υπάρχει, δηλαδή, ζευγάρι κλειδιού-τιμής $\langle 10.1.2.3 \ 0 \rangle$.

Τι θα θέλαμε σε αυτήν την περίπτωση; Θα θέλαμε να μπορούμε να εμφανίσουμε τη συνολική μέτρηση για κάθε μία διεύθυνση. Έτσι, για τη διεύθυνση 10.1.2.3 θα θέλαμε να έχουμε ένα ζευγάρι κλειδιού-τιμής $\langle 10.1.2.3 \ 5 \rangle$. Αντιστοίχως και για τις υπόλοιπες διευθύνσεις. Αυτό σημαίνει ότι, με κάποιον τρόπο, σε έναν υπολογιστή θα πρέπει να καταλήγουν όλες οι όμοιες διευθύνσεις και να υπολογίζεται το εν λόγω άθροισμα. Κάτι τέτοιο θα έχει το εξής αποτέλεσμα: Αντί να έχουμε σε κάθε έναν υπολογιστή μέτρηση των εμφανίσεων κάθε μίας διεύθυνσης για το κάθε αρχείο, θα έχουμε σε κάθε υπολογιστή τον συνολικό αριθμό μετρή-

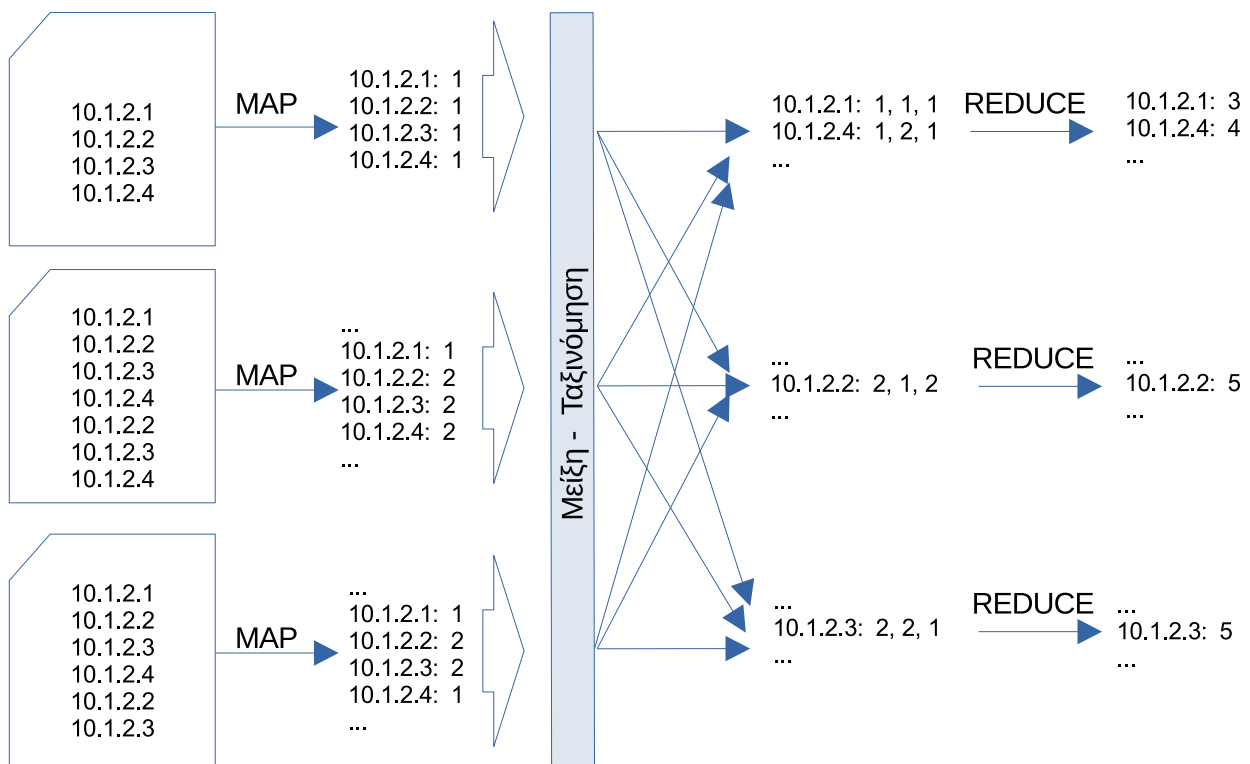


Σχήμα 3.3: Μέτρηση εμφανίσεων διευθύνσεων IP σε κάθε αρχείο ξεχωριστά με χρήση πίνακα κατακερματισμού.

σεων για κάποιες διευθύνσεις σε όλα τα αρχεία. Σε όλους τους υπολογιστές, συνολικά, θα βρίσκονται ζευγάρια κλειδιών-τιμών όπου κάθε ένα κλειδί (διεύθυνση) θα εμφανίζεται μία φορά, ενώ η τιμή θα αφορά το συνολικό πλήθος εμφανίσεων της συγκεκριμένης διεύθυνσης σε όλα τα αρχεία.

Για να γίνει αυτό, θα πρέπει να υπάρξει ένας μηχανισμός ο οποίος θα αποστέλλει τις ίδιες διευθύνσεις στον ίδιο υπολογιστή πάντα, όπου θα γίνεται και η μέτρηση του πλήθους των εμφανίσεων της διεύθυνσης αυτής. Αν ανατρέξουμε πιο πάνω, στη διαδικασία παραγωγής εξόδου του βήματος map, θα διαπιστώσουμε ότι παράγουμε ζευγάρια κλειδιών-τιμών, με κλειδί τη διεύθυνση που εντοπίζουμε και τιμή το πλήθος των εμφανίσεών της. Αυτό που θα μπορούσε να συμβεί λοιπόν είναι, με κάποιον τρόπο, να συγκεντρώνουμε τα ίδια κλειδιά και στη συνέχεια να υπολογίζουμε το άθροισμα των τιμών. Το άθροισμα των τιμών θα αποτελεί και τον συνολικό αριθμό εμφανίσεων της κάθε διεύθυνσης-κλειδιού.

Αυτό που μόλις περιγράψαμε αποτελεί και το βήμα reduce στο μοντέλο MapReduce. Ενώ στο βήμα map δεν υπάρχει επικοινωνία μεταξύ των υπολογιστών και η επεξεργασία πραγματοποιείται ανεξάρτητα, στο βήμα του reduce πραγματοποιείται επικοινωνία ώστε να συγκεντρωθούν σε έναν υπολογιστή οι τιμές οι οποίες θα πρέπει να υποστούν επεξεργασία μαζί. Ο τρόπος με τον οποίο αυτές οι τιμές καταλήγουν μαζί είναι μέσω των κοινών κλειδιών. Έτσι, όταν θέλουμε να πραγματοποιήσουμε επεξεργασία δεδομένων στον ίδιο υπολογιστή μετά το βήμα map, τα δεδομένα αυτά θα πρέπει να έχουν το ίδιο κλειδί, ώστε να μεταφερθούν και να υποστούν επεξεργασία στον ίδιο υπολογιστή κατά το βήμα reduce. Η διαδικασία αυτή απεικονίζεται στο Σχήμα 3.4.

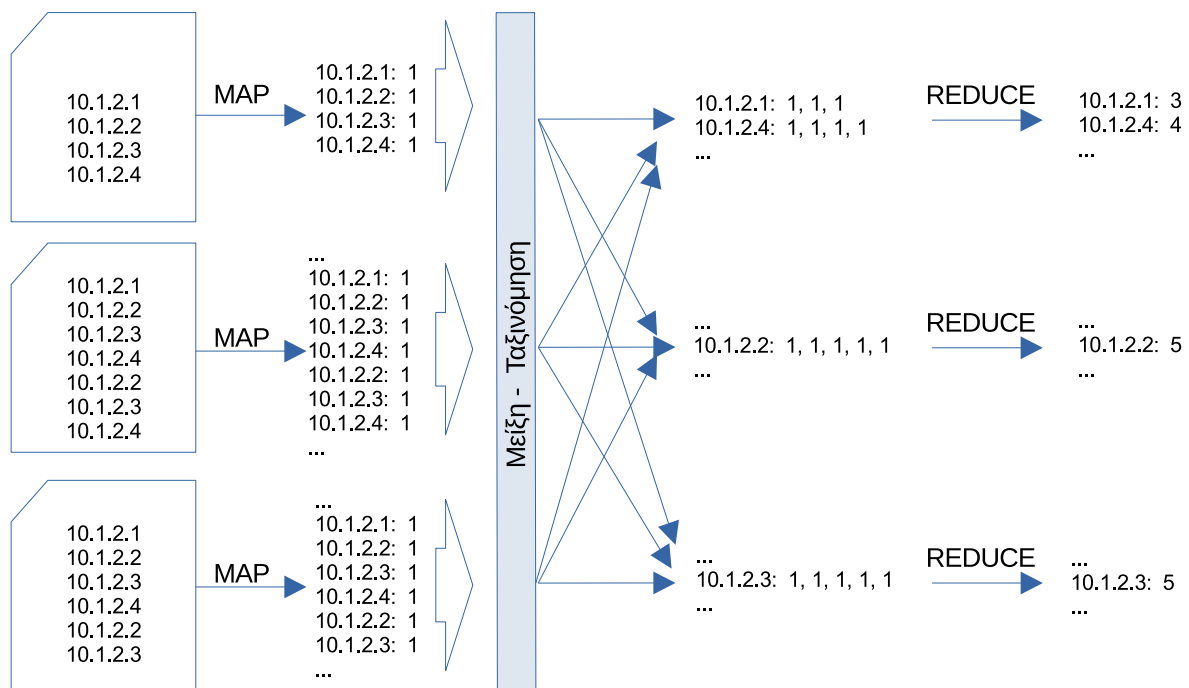


Σχήμα 3.4: Διαδικασία MapReduce με χρήση πίνακα κατακερματισμού κατά το map.

Στο βήμα reduce ο συγκεκριμένος υπολογιστής για κάθε ένα κλειδί λαμβάνει μία λίστα από τιμές οι οποίες αντιστοιχούν στο συγκεκριμένο κλειδί. Έτσι, για τη διεύθυνση 10.1.2.3 που αναφέραμε πιο πριν, ο υπολογιστής ο οποίος θα λάβει τη 10.1.2.3 ως κλειδί στο βήμα reduce, θα λάβει και τη λίστα από τιμές [2, 2, 1] από τους υπολογιστές οι οποίες κατά το βήμα map εντόπισαν και μέτρησαν το συγκεκριμένο κλειδί. Εντός του βήματος reduce μπορεί να πραγματοποιηθεί άθροιση των τιμών αυτών. □

Το βήμα reduce δεν είναι υποχρεωτικό να πραγματοποιηθεί. Μπορεί να παραλειφθεί εάν δεν υπάρχει η ανάγκη να υπάρξει κοινή επεξεργασία κάποιων δεδομένων. Έτσι, για παράδειγμα, στην περίπτωση επεξεργασίας αρχείων ήχου, καθώς ζητούμενο είναι η μετατροπή τους σε μορφότυπο mp3, δεν πραγματοποιούμε το βήμα reduce. Εάν, όμως, θέλουμε να συνενώσουμε ομάδες αυτών των αρχείων, ώστε να πραγματοποιήσουμε π.χ. λίστες εκτέλεσης ανά είδος, τότε θα πρέπει να σχεδιάσουμε το βήμα reduce με τέτοιο τρόπο ώστε τα αρχεία MP3 τα οποία θα τοποθετηθούν στο ίδιο αρχείο ήχου να βρεθούν στον ίδιο υπολογιστή.

Τώρα, ας επισημάνουμε κάποια λεπτά σημεία. Αρχικά, κάθε υπολογιστής επεξεργάζεται περισσότερα από ένα αρχεία. Αυτό σημαίνει ότι εκτελεί το βήμα map για όλα τα αρχεία αυτά. Αντιστοίχως, κάθε υπολογιστής επεξεργάζεται περισσότερα από ένα κλειδιά, άρα μπορεί να εκτελέσει το βήμα reduce για κάθε ένα κλειδί. Επιπλέον, στον ίδιο υπολογιστή μπορεί να εκτελούνται και διεργασίες που πραγματοποιούν βήματα map και βήματα reduce. Ένα σημείο που χρήζει προσοχής και αξίζει να τονίσουμε είναι ότι, για να ξεκινήσει το βήμα reduce, θα πρέπει να έχει ολοκληρωθεί το βήμα map σε όλους τους υπολογιστές. Τότε και μόνο τότε ξεκινά η επεξεργασία κατά το reduce. Ο λόγος είναι ότι θα πρέπει να είναι διαθέσιμη ως είσοδος όλη η έξοδος από το βήμα map, καθώς δεν ξέρουμε σε ποιον υπολογιστή μπορεί να βρίσκονται δεδομένα τα οποία πρέπει να υποστούν επεξεργασία μαζί.



Σχήμα 3.5: Διαδικασία MapReduce με απευθείας εκπομπή ζευγών κλειδιού-τιμής κατά το map, χωρίς χρήση πίνακα κατακερματισμού.

Στο σημείο αυτό, όμως, έχουμε μια εκκρεμότητα αναφορικά με το βήμα map. Είχαμε θεωρήσει ότι ο πίνακας κατακερματισμού χωράει στην κύρια μνήμη στον κάθε υπολογιστή. Κάτι τέτοιο, όμως, μπορεί να μη συμβαίνει στην πραγματικότητα. Το ερώτημα που δημιουργείται είναι τι μπορούμε να κάνουμε για αυτό. Στο σημείο, αυτό μας δίνεται η δυνατότητα να προσφέρουμε μια ελαφρώς διαφορετική προσέγγιση στο πρόβλημα, λαμβάνοντας υπόψη ότι έχουμε τη δυνατότητα του βήματος reduce. Αυτή απεικονίζεται στο Σχήμα 3.5.

Σύμφωνα με αυτά που περιγράψαμε πιο πάνω, ο κάθε υπολογιστής που εκτελεί το βήμα map υπολογίζει τοπικά τον αριθμό των εμφανίσεων της κάθε μιας διεύθυνσης και αποστέλλει αυτήν τη μέτρηση στον υπολογιστή που εκτελεί το βήμα reduce ως ζευγάρι κλειδιού-τιμής της μορφής <διεύθυνση, πλήθος>. Μπορούμε να αποφύγουμε να κρατάμε στη μνήμη τον πίνακα κατακερματισμού. Αυτό μπορεί να γίνει αν κάθε φορά που συναντούμε μια διεύθυνση την αποστέλλουμε στον υπολογιστή που θα εκτελέσει το βήμα reduce ως <διεύθυνση, 1>. Έτσι, δεν υπάρχει η απαίτηση δέσμευσης της μνήμης και στο τέλος αυτής της διαδικασίας, κατά την εκτέλεση του βήματος reduce, η άθροιση θα γίνει σε λίστες τιμών της μορφής [1, 1, ..., 1]. Κάπου εδώ, όμως, μπορεί να δημιουργηθεί ένα επιπλέον ερώτημα: Δεν δημιουργούμε πλεονάζουσα κίνηση στο δίκτυο μεταξύ των υπολογιστών με αυτόν τον τρόπο; Θα συζητήσουμε για το θέμα αυτό στη συνέχεια, όταν θα παρουσιάσουμε τη λειτουργία των συνδυαστών (combiners).

3.3 Μείξη και ταξινόμηση

Στο σημείο αυτό, θα περιγράψουμε εν συντομία τι συμβαίνει στα ενδότερα ενός συστήματος που υλοποιεί το μοντέλο MapReduce, όπως για παράδειγμα στο Hadoop. Για περισσότερες λεπτομέρειες, ο αναγνώστης μπορεί να απευθυνθεί σε πηγές που αναλύουν το ζήτημα σε

μεγαλύτερο βάθος [LD10, DG04].

Σε ένα πρόγραμμα MapReduce, ο προγραμματιστής ορίζει στον αλγόριθμό του δύο κύριες λειτουργίες. Τη λειτουργία map και τη λειτουργία reduce. Τόσο η λειτουργία map, όσο και η λειτουργία reduce εκτελούνται ταυτόχρονα από περισσότερες από μία διεργασίες. Έτσι, πολλές διεργασίες map εκτελούν την ίδια λειτουργία map και αντιστοίχως για τις λειτουργίες reduce. Στις διεργασίες αυτές θα αναφερόμαστε ως mappers και reducers, αντίστοιχα. Οι mappers δεν επικοινωνούν μεταξύ τους, αλλά ούτε και οι reducers επικοινωνούν μεταξύ τους. Υπάρχει, όμως, επικοινωνία των mappers με τους reducers εμμέσως, διά του ορισμού ζευγών κλειδιού-τιμής.

Αυτό που συμβαίνει, λοιπόν, είναι ότι μεταξύ της εκτέλεσης των mappers και των reducers παρεμβάλλεται ένας μηχανισμός που ονομάζεται *μείξη και ταξινόμηση* (*Shuffle and Sort*). Οι έξοδοι των mappers, που είναι ζεύγη κλειδιών-τιμών, προταξινομούνται και εφαρμόζεται ενιαία συνάρτηση κατακερματισμού ώστε τα όμοια κλειδιά να αποσταλούν στην ίδια διεργασία reduce.

Για να εκκινήσουν την επεξεργασία τους οι reducers, θα πρέπει υποχρεωτικά να έχει ολοκληρωθεί η επεξεργασία από όλους τους mappers και αυτοί να έχουν αποστείλει τα δεδομένα τους στους reducers. Τότε και μόνο τότε θα εκκινήσει η λειτουργία των τελευταίων¹. Αυτό συμβαίνει επειδή, όταν ολοκληρωθεί η άφιξη των δεδομένων στους reducers, τα ζεύγη κλειδιών-τιμών ταξινομούνται ως προς το κλειδί σε κάθε reducer. Έτσι, κάθε reducer μπορεί να διαχωρίσει αποτελεσματικά τις τιμές οι οποίες αντιστοιχούν σε κάθε κλειδί, ώστε αυτές να υποστούν επεξεργασία μαζί.

Ουσιαστικά, πραγματοποιείται μία ομαδοποίηση των τιμών ανά κλειδί, αντίστοιχη με τον τελεστή GROUP BY της SQL. Η ομαδοποίηση αυτή και η ταξινόμηση, όμως, πραγματοποιείται μόνο ανά reducer. Τα κλειδιά, δηλαδή, είναι ταξινομημένα μόνο σε κάθε reducer τοπικά. Δεν υπάρχει κάποια εγγύηση για καθολική ταξινόμηση των κλειδιών σε όλους τους reducers.

3.4 Ροή δεδομένων

Ας δούμε τώρα τι συμβαίνει με την αποθήκευση των δεδομένων κατά τη ροή εργασίας. Κατά την εκτέλεσή τους, οι mappers λαμβάνουν ως είσοδο δεδομένα τα οποία βρίσκονται αποθηκευμένα στο σύστημα αρχείων. Αυτό μπορεί να είναι το HDFS, το οποίο αποτελεί και μέρος του Hadoop. Η ανάγνωση πραγματοποιείται από αρχεία και αν τα αρχεία αυτά είναι μεγάλα, σε blocks. Κάθε Mapper διαβάζει ένα αρχείο. Αν το αρχείο ξεπερνά το μέγεθος block του συστήματος, διαχωρίζεται σε τμήματα ανάλογα με το μέγεθος block.

Η έξοδος των mappers είναι ζεύγη κλειδιών-τιμών. Αυτά, καθώς δεν επαρκεί συνήθως η μνήμη, αποθηκεύονται στον τοπικό δίσκο. Δεν αποθηκεύονται στο HDFS², καθώς ο ρόλος τους είναι προσωρινός και δεν αποτελούν τελικά αποτελέσματα³. Τα δεδομένα αυτά απο-

¹Το σημείο αυτό όπου διαχωρίζεται η εκκίνηση των reducers από τους mappers ονομάζεται φράγμα (barrier).

²Λεπτομέρειες για το HDFS μπορείτε να βρείτε στο 2ο κεφάλαιο.

³Η αποθήκευση των ενδιάμεσων δεδομένων στο HDFS θα είχε ως αποτέλεσμα να εκκινήσει η διαδικασία αντιγραφής τους, κάτι που θα επιβάρυνε άσκοπα τη συστάδα, καθώς τα δεδομένα αυτά δεν μας ενδιαφέρει να διατηρηθούν.

στέλλονται στους αντίστοιχους reducers για να υποστούν επεξεργασία. Και πάλι, αποθηκεύονται στον δίσκο του κάθε reducer, όχι όμως και στο HDFS, για τον ίδιο λόγο που περιγράψαμε μόλις πριν. Όταν ο reducer επεξεργαστεί την είσοδό του, τα τελικά ζεύγη κλειδιού-τιμής που προκύπτουν αποθηκεύονται στο κατανομημένο σύστημα αρχείων. Αυτό συμβαίνει επειδή αποτελούν τελικά αποτελέσματα και επιθυμούμε να τα διαφυλάξουμε από ενδεχόμενη απώλεια.

Κάτι που αξίζει να επισημάνουμε στο σημείο αυτό είναι το γεγονός ότι κάθε reducer παράγει την έξοδό του ανεξάρτητα από τους υπόλοιπους. Έτσι, σε περιπτώσεις όπου χρησιμοποιούνται πλέον του ενός reducers, παράγονται πάνω από ένα αρχεία εξόδου, ένα για κάθε reducer. Καθώς η είσοδος σε κάθε reducer είναι ταξινομημένη ως προς το κλειδί, αντιστοίχως δύναται να είναι και η έξοδος σε κάθε αρχείο.

3.5 Ειδικά θέματα του MapReduce

Αφού μελετήσαμε τον βασικό μηχανισμό του MapReduce, ας δούμε και κάποια πιο ειδικά θέματα. Στη συνέχεια, θα αναφερθούμε στους *συνδυαστές* (*Combiners*) και στους *διαμεριστές* (*Partitioners*).

3.5.1 Συνδυαστές και διαμεριστές

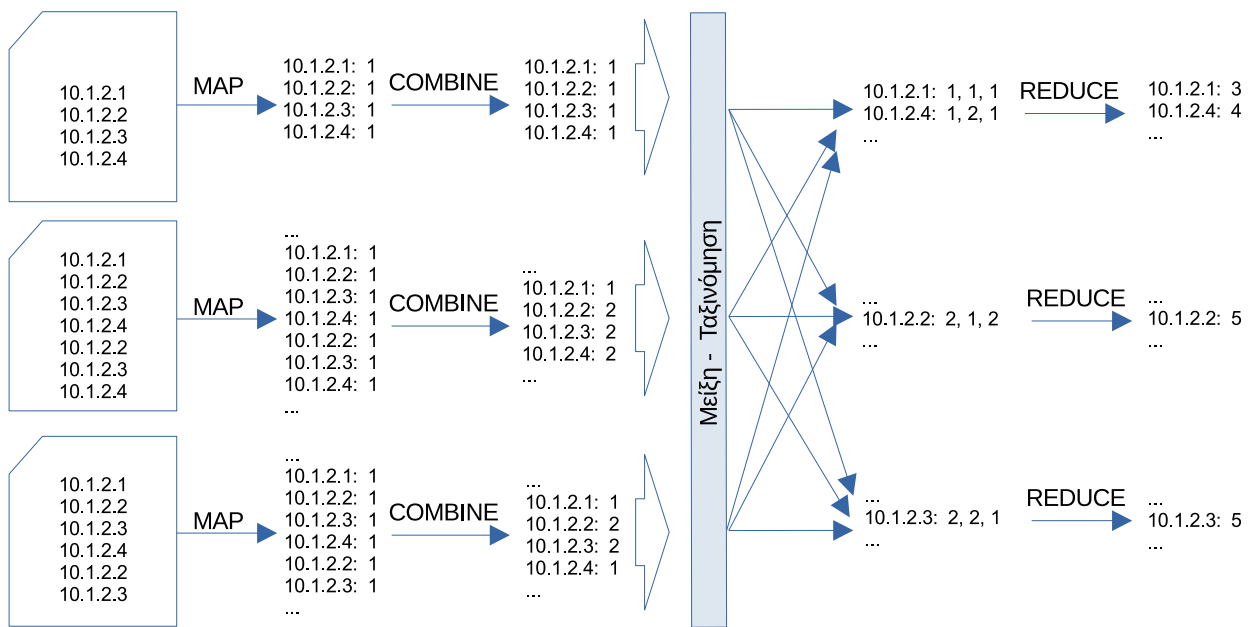
Στην έως τώρα περιγραφή μας είδαμε τα δύο βασικά συστατικά του μοντέλου MapReduce: τους mappers και τους reducers. Αυτά, όμως, δεν είναι τα μόνα που χρησιμοποιούνται στην ανάπτυξη εφαρμογών MapReduce. Υπάρχουν επιπλέον οι *διαμεριστές* (*Partitioners*) και οι *συνδυαστές* (*Combiners*). Παρακάτω, θα συζητήσουμε τον ρόλο και τη λειτουργία των συγκεκριμένων μηχανισμών.

3.5.2 Συνδυαστές

Ας ξεκινήσουμε με τους combiners. Όπως είδαμε, στο αρχικό παράδειγμα MapReduce που παρουσιάσαμε, περιγράψαμε μία προσέγγιση μέτρησης διευθύνσεων στην οποία καταλήγαμε στην παραγωγή εξόδου από το βήμα map της μορφής <διεύθυνση, 1> και θέσαμε τα ερωτήματα της δημιουργίας αλλά και την αντιμετώπιση του πλεονάζοντος όγκου δεδομένων που θα δημιουργηθεί. Απάντηση στο ερώτημα αυτό αποτελούν οι combiners.

Ένας combiner έχει ως στόχο να μειώσει τον όγκο των δεδομένων που μεταδίδονται από τους mappers στους reducers. Ο ρόλος του είναι αποκλειστικά και μόνο να λειτουργήσει ως βελτιστοποίηση. Αυτό σημαίνει ότι η ύπαρξή του ή μη δεν θα πρέπει να επηρεάζει επ' ουδενί την ορθότητα του αλγορίθμου που υλοποιείται. Ο λόγος είναι ότι δεν μπορούμε να ελέγξουμε την εκτέλεση ή μη ενός combiner.

Συγκεκριμένα, το Hadoop μπορεί να εκτελέσει έναν combiner κατά την εκτέλεση μίας εφαρμογής καμία, μία, ή και περισσότερες φορές [Whi12]. Επιπλέον, ο combiner μπορεί να επικληθεί σε διαφορετικά σημεία κατά την εκτέλεση μίας εφαρμογής. Έτσι, μπορεί να εκτελεστεί μετά το βήμα map, αλλά και πριν την εκτέλεση του βήματος reduce, αφού έχει ολοκληρωθεί



Σχήμα 3.6: Διαδικασία MapReduce με χρήση Συνδυαστών.

η διαδικασία μείξης και ταξινόμησης. Ως εκ τούτου, δεν θα πρέπει να επαφίομαστε στην εκτέλεσή του.

Από τα παραπάνω καταλαβαίνουμε ότι, καθώς ένας combiner μπορεί να εκτελεστεί μεταξύ ενός mapper και ενός reducer, η είσοδός του θα πρέπει να οριστεί με τέτοιο τρόπο ώστε να αντιστοιχεί στην έξοδο του mapper και η έξοδός του να οριστεί ώστε να αντιστοιχεί στην είσοδο του reducer. Έτσι, διατηρείται και η ορθότητα του αλγορίθμου ανεξάρτητα με την επίκληση του reducer ή μη.

Στην περίπτωση της μέτρησης των διευθύνσεων που παρουσιάσαμε πρωτύτερα, ένας combiner μπορεί να χρησιμοποιηθεί ώστε να μειώσει τον όγκο των δεδομένων που παράγει ένας mapper. Πώς θα γίνει αυτό; Παίρνοντας ως είσοδο την έξοδο του mapper. Εκεί θα πραγματοποιηθεί μια λειτουργία ομαδοποίησης GroupBy, με αποτέλεσμα για κάθε κλειδί να είναι διαθέσιμη στον Combiner μία λίστα από τιμές, με κάθε τιμή να είναι ίση με 1, καθώς εκπέμψαμε από κάθε mapper ζεύγη τύπου <διεύθυνση,1>. Ο Combiner για κάθε λίστα κάθε κλειδιού θα προσθέσει τις αντίστοιχες τιμές και θα εκπέμψει ζεύγος <διεύθυνση,Σ>, όπου Σ το άθροισμα των "1" στη λίστα. Η διαδικασία αυτή απεικονίζεται στο Σχήμα 3.6.

Εδώ μπορούμε να προχωρήσουμε σε μία σειρά από παρατηρήσεις. Πρώτα απ' όλα, διαπιστώνουμε ότι η έξοδος είναι συμβατή με την είσοδο του reducer, με τέτοιο τρόπο ώστε, είτε παρεμβληθεί ο combiner είτε όχι, η λειτουργία του reducer παραμένει ανεπηρέαστη, καθώς αυτός αναμένει ως είσοδο ζεύγος <διεύθυνση, εμφανίσεις>. Επιπλέον, παρατηρούμε ότι η λειτουργία του combiner είναι πανομοιότυπη με τη λειτουργία του reducer: Αθροίζει τις τιμές ενός κλειδιού που λαμβάνει. Άρα, αφού και οι δύο οντότητες επιτελούν την ίδια εργασία, είναι κάποια από αυτές περιττή;

Ας προσπαθήσουμε να ξεκαθαρίσουμε τα πράγματα. Αρχικά, το γεγονός ότι επιτελείται η ίδια πράξη σε δύο λειτουργίες αφορά την επίλυση του συγκεκριμένου προβλήματος. Επίσης, η λειτουργία του Combiner εκτελείται τοπικά σε κάθε mapper αρχικά, και στη συνέχεια ίσως πριν από κάθε reducer, ώστε να μειώσει τον όγκο των δεδομένων. Μπορούμε να πούμε δηλαδή

ότι ένας combiner είναι ένας τοπικός reducer.

Επιπλέον, δεν μπορεί να χρησιμοποιηθεί πάντα ένας combiner. Καθώς οι combiners λειτουργούν τοπικά και παράγουν δεδομένα σε ένα κατανεμημένο περιβάλλον, τα οποία είναι και αυθαίρετου μεγέθους, μπορεί να καταλήξουν σε οποιονδήποτε reducer, αλλά και καθώς δεν έχουμε εικόνα του τι συμβαίνει στους υπόλοιπους mappers, δεν ξέρουμε τη σειρά με την οποία αυτά θα υποστούν επεξεργασία.

Υπάρχουν δύο προϋποθέσεις οι οποίες θα πρέπει να τηρούνται για τη χρήση combiner σε κάθε πρόβλημα. Οι προϋποθέσεις αυτές σχετίζονται με τη λειτουργία που εκτελεί ο combiner και η οποία θα πρέπει να έχει ταυτόχρονα και την *προσεταιριστική* και την *αντιμεταθετική* ιδιότητα. Η προσεταιριστική ιδιότητα υποδηλώνει ότι μπορεί να ομαδοποιηθεί οποιοδήποτε πλήθος τελεστών και το αποτέλεσμα να είναι όμοιο. Παράδειγμα λειτουργίας που δεν διατηρεί την προσεταιριστική ιδιότητα είναι ο υπολογισμός του μέσου όρου αριθμών: π.χ. $AVERAGE([1, 2, 3,]) = 3.5$, ενώ $AVERAGE([1, 2]) = 1.5$, $AVERAGE([1.5, 3]) = 2.25$. Αντιθέτως, αυτό δεν συμβαίνει με το άθροισμα.

Η αντιμεταθετική ιδιότητα υποδηλώνει τη σειρά με την οποία θα πραγματοποιηθούν υπολογισμοί. Παράδειγμα λειτουργίας που διατηρεί την προσεταιριστική αλλά όχι την αντιμεταθετική ιδιότητα είναι η παράθεση συμβολοσειρών (Concatenation): Έτσι, π.χ. αν θέλουμε να παραθέσουμε τις συμβολοσειρές "I", "am", "John", ανάλογα με τη σειρά της παράθεσης⁴ μπορεί να προκύψει "I am John" ή και "am I John", που είναι ένα διαφορετικό αποτέλεσμα. Αντιθέτως, η συγκεκριμένη λειτουργία διατηρεί την προσεταιριστικότητα, καθώς, ανεξάρτητα με το πώς θα ομαδοποιήσουμε τις παραθέσεις, το αποτέλεσμα θα είναι το ίδιο, αρκεί αυτές να γίνονται με την ίδια σειρά.

3.5.3 Διαμεριστές

Σε ένα σύστημα που υποστηρίζει το μοντέλο MapReduce, εκτός από το γεγονός ότι μπορεί να υπάρχουν πολλές διεργασίες που επιτελούν τη λειτουργία map, αντιστοίχως, μπορούν να υπάρχουν και πολλές διεργασίες που επιτελούν τη λειτουργία reduce. Τι διαφορετικό συμβαίνει σε αυτήν την περίπτωση; Ότι, με κάποιον τρόπο, τα κλειδιά με τις αντίστοιχες λίστες τιμών θα πρέπει να κατανεμηθούν στις διεργασίες αυτές με ενιαίο τρόπο. Ο *διαμεριστής* (*Partitioner*) ορίζει τον τρόπο αυτόν.

Ο χρήστης μπορεί να ορίσει τη δική του συνάρτηση διαμερισμού ώστε να κατορθώσει την εξισορρόπηση φόρτου μεταξύ των διεργασιών reduce. Εάν δεν το κάνει, ακολουθείται η προκαθορισμένη συμπεριφορά, σύμφωνα με την οποία κατακερματίζεται το κλειδί του ζεύγους <κλειδί, τιμή> που προέρχεται από κάποιον mapper.

3.5.4 Δευτερεύουσα ταξινόμηση

Όπως έχουμε δει, κατά τη φάση της μείξης-ταξινόμησης, το Hadoop εγγυάται ότι τα κλειδιά θα είναι ταξινομημένα. Τι γίνεται, όμως, στην περίπτωση κατά την οποία επιθυμούμε να

⁴Έχουμε εισαγάγει κενά ώστε να διευκολύνουμε τον αναγνώστη.

ταξινομήσουμε και το πεδίο των τιμών; Θα μπορούσαμε να διαβάσουμε όλες τις τιμές στη μνήμη και να ταξινομήσουμε εκεί. Κάτι τέτοιο, όμως, μπορεί να επέφερε τον κίνδυνο να μην επαρκεί η μνήμη.

Για να το αντιμετωπίσουμε αυτό, θα ακολουθήσουμε τη λύση της μετατροπής τιμής σε κλειδί [LD10]. Ουσιαστικά, αυτό που συμβαίνει είναι ότι εκμεταλλευόμαστε το γεγονός ότι θα ταξινομηθεί το κλειδί και μετακινούμε τμήμα της τιμής στο κλειδί που παράγεται από τη φάση map, δημιουργώντας έτσι ένα σύνθετο κλειδί. Θα πρέπει να ορίσουμε τον τρόπο ταξινόμησης του σύνθετου κλειδιού αυτού κατά το πραγματικό κλειδί αρχικά και έπειτα κατά το τμήμα του κλειδιού που πήραμε από την τιμή. Στη συνέχεια, θα πρέπει να ορίσουμε τον διαμεριστή ώστε να κατακερματίσει το σύνθετο κλειδί μόνο ως προς το τμήμα που αφορούσε το αρχικό κλειδί.

3.5.5 Αλυσίδες εργασιών MapReduce

Πολλές φορές, ένα πρόβλημα δεν λύνεται μετά την ολοκλήρωση των βημάτων MapReduce. Αυτό σημαίνει ότι, μετά την ολοκλήρωση μίας εργασίας MapReduce μπορεί να χρειαστεί και άλλη ή άλλες εργασίες MapReduce. Σε αυτήν την περίπτωση, καθώς η έξοδος του βήματος reduce αποθηκεύεται, όπως είπαμε, στο κατανομημένο σύστημα αρχείων, ο mapper της επόμενης εργασίας MapReduce θα χρησιμοποιήσει αυτήν την έξοδο ως είσοδο, κ.ο.κ. Απλό παράδειγμα τέτοιου προβλήματος που απαιτεί πλέον του ενός βήματα για να επιλυθεί είναι ο εντοπισμός τιμών που ξεπερνούν τον μέσο όρο τιμών της εισόδου: Πρώτα θα πρέπει να υπολογιστεί ο μέσος όρος σε έναν κύκλο MapReduce και έπειτα θα πραγματοποιηθεί η σύγκριση με τον μέσο όρο.

Όσον αφορά την υλοποίηση τέτοιων λειτουργιών, αυτή εξαρτάται από την υλοποίηση που θα επιλεγεί να ακολουθηθεί. Έτσι, αν η υλοποίηση του αλγορίθμου πραγματοποιηθεί σε Java, η οποία είναι και η γλώσσα υλοποίησης του Hadoop, παρέχεται εγγενής μηχανισμός για τον προσδιορισμό της αλυσιδωτής εκτέλεσης. Αντιθέτως, αν η υλοποίηση πραγματοποιηθεί σε γλώσσα άλλη από την Java, οπότε και θα απαιτηθεί ο μηχανισμός streaming του Hadoop, είναι απαραίτητη η χρήση εξωτερικών πακέτων. Για παράδειγμα, αν η υλοποίηση πραγματοποιηθεί σε Python, για την πραγματοποίηση αλυσίδων εργασιών μπορεί να χρησιμοποιηθεί η βιβλιοθήκη MRJob⁵.

3.5.6 Μη ανάγκη χρήσης reducer

Υπάρχουν περιπτώσεις κατά τις οποίες δεν απαιτείται η χρήση του βήματος reduce, όπως στις περιπτώσεις που εξετάσαμε στην αρχή του κεφαλαίου αυτού. Αυτό είναι εφικτό να υλοποιηθεί από ένα σύστημα MapReduce. Για την ακρίβεια, υπάρχουν δύο εναλλακτικές. Η πρώτη είναι να μην χρησιμοποιηθούν καθόλου διεργασίες reduce. Σε αυτήν την περίπτωση, η έξοδος των mapper αποτελεί και το τελικό αποτέλεσμα.

Η άλλη εναλλακτική είναι να χρησιμοποιηθεί βήμα reduce το οποίο είναι *ταυτοτικό* (*identity*). Αυτό σημαίνει ότι ο κάθε reducer δεν επεξεργάζεται την είσοδο που λαμβάνει και απλώς την παράγει ως έξοδο. Η διαφορά σε αυτήν την περίπτωση είναι ότι, καθώς μεσολαβεί η φάση

⁵<https://mrjob.readthedocs.io/en/latest/>

μείξης και ταξινόμησης, η είσοδος των reducers θα είναι ταξινομημένη. Έτσι, σε περίπτωση που χρησιμοποιείται ένας reducer, η έξοδος θα είναι ταξινομημένη ως προς το κλειδί. Σε περίπτωση χρήσης πλέον του ενός reducer, θα υπάρχουν παρισσότερες από μία έξοδοι, κάθε μία με ταξινομημένα κλειδιά.

Τέλος, ενώ υπάρχει η δυνατότητα μη χρήσης reducer, κάτι τέτοιο δεν συμβαίνει με τους mappers. Αυτό επειδή αφενός οι reducers θα πρέπει να λαμβάνουν ζεύγη κλειδιών-τιμών, αφετέρου επειδή θα πρέπει αυτά να αποσταλούν από τους mappers στους reducers μέσω της διαδικασίας μείξης-ταξινόμησης. Σε περίπτωση που δεν θέλουμε κάποια επεξεργασία στα δεδομένα εισόδου, μπορούμε να χρησιμοποιήσουμε ταυτοτικό mapper, αντιστοίχως με τον ταυτοτικό reducer.

3.6 Σχεδιασμός αλγορίθμων MapReduce

Στο σημείο αυτό θα δούμε πώς μπορούμε να σχεδιάσουμε έναν αλγόριθμο MapReduce για την επίλυση ενός προβλήματος. Για τον σκοπό αυτό, θα χρησιμοποιήσουμε ένα παράδειγμα ως πρόβλημα. Στη συνέχεια, θα παραθέσουμε δύο υλοποιήσεις για το Apache Hadoop και τις επεξηγήσεις τους, μία σε Java και μία σε Python.

3.6.1 Περιγραφή του προβλήματος

Ας θεωρήσουμε ότι διατηρούμε τα δεδομένα του γράφου ενός κοινωνικού δικτύου τα οποία θέλουμε να αναλύσουμε. Τα δεδομένα αυτά αποθηκεύονται σε αρχεία ASCII με τη μορφή ζευγών, όπως αυτά εμφανίζονται στον Κώδικα 3.1. Σε κάθε γραμμή στο αρχείο υπάρχει ένα ζεύγος ονομάτων χρηστών το οποίο και αναπαριστά μία αμφίδρομη σχέση φιλίας μεταξύ αυτών. Έτσι, οι χρήστες Alexandros και Georgia είναι φίλοι μεταξύ τους. Ζητούμενο είναι να βρούμε το πλήθος των φίλων που έχει κάθε χρήστης του κοινωνικού δικτύου. Καθώς το πλήθος των χρηστών είναι πολύ μεγάλο, επιθυμούμε να υλοποιήσουμε τη λύση με τη χρήση MapReduce και να εκτελέσουμε το πρόγραμμά μας σε ένα κατανεμημένο σύστημα, όπως το Apache Hadoop.

```
Alex,Georgia
Apostolos,Alex
Anastasios,Apostolos
Anastasios,Alexandros
Georgia,Anastasios
```

Κώδικας 3.1: Αρχείο με την τοπολογία του κοινωνικού δικτύου.

3.6.2 Σχεδιασμός αλγορίθμου

Ας ξεκινήσουμε να σχεδιάζουμε τον αλγόριθμό μας με τη μελέτη των δεδομένων μας. Βλέπουμε ότι κάθε γραμμή των αρχείων εισόδου αποτελείται από ζεύγη ονομάτων χρηστών, κάτι που προσδιορίζεται εξάλλου και από την περιγραφή του προβλήματος. Αυτό που ζητείται από εμάς είναι να μπορέσουμε να μετρήσουμε το πλήθος των φίλων κάθε χρήστη. Ας θεωρήσουμε αρχικά για τη λύση μας ότι θα μεταχειριστούμε μια διεργασία reduce.

Καθώς τα αρχεία είναι πολύ μεγάλα, θα υποστούν επεξεργασία από διαφορετικές διεργασίες *map*, ενδεχομένως και σε διαφορετικούς υπολογιστές. Κάθε *mapper*, λοιπόν, θα πάρει κάποιες από τις γραμμές ως είσοδο και θα πρέπει να τις επεξεργαστεί. Το όνομα ενός χρήστη μπορεί να εμφανίζεται είτε πρώτο είτε δεύτερο. Ανεξάρτητα από τη θέση του, η εμφάνισή του υποδεικνύει μία σχέση φιλίας με τον άλλο χρήστη που θα πρέπει να καταγράψουμε. Άρα, εμείς θέλουμε να καταγράψουμε την εμφάνιση κάθε ονόματος χρήστη σε κάθε ζεύγος.

Με την περιγραφή που κάναμε, όμως, διαπιστώνουμε ότι το ίδιο όνομα χρήστη μάλλον θα συναντάται σε διαφορετικούς υπολογιστές. Άρα, για να μπορέσουμε να υπολογίσουμε το συνολικό πλήθος εμφανίσεων των χρηστών σε σχέσεις, άρα και των φιλιών, θα πρέπει με κάποιον τρόπο να μπορέσουμε να ομαδοποιήσουμε και να μετρήσουμε τα όμοια ονόματα. Λαμβάνοντας υπόψη τα παραπάνω, καταλήγουμε στο γεγονός ότι, για να μπορέσουμε να υπολογίσουμε το συνολικό πλήθος εμφανίσεων σε σχέσεις κάθε ονόματος χρήστη, θα πρέπει να ομαδοποιήσουμε όλες τις εμφανίσεις κάθε χρήστη σε έναν υπολογιστή και να υπολογίσουμε το τελικό άθροισμα εμφανίσεων για κάθε έναν χρήστη.

Αυτή η ομαδοποίηση πραγματοποιείται, όπως είπαμε, μέσω της φάσης μείξης-ταξινόμησης, κατά την οποία ίδια κλειδιά καταλήγουν στην ίδια διεργασία *reduce*. Εκεί, στη συνέχεια, θα μπορεί να γίνει η καταμέτρηση των εμφανίσεων κάθε ονόματος. Αυτό είναι πολύ κοντά σε αυτό που θέλουμε. Το μόνο που μένει είναι να ορίσουμε τα ζεύγη κλειδιών-τιμών με τέτοιο τρόπο ώστε οι εμφανίσεις κάθε χρήστη να υποστούν επεξεργασία μαζί.

Για να γίνει αυτό, αρκεί να θέσουμε στην έξοδο του *mapper* ως κλειδί το όνομα χρήστη και τιμή ίση με τη μονάδα. Αν και η τιμή δεν παίζει ιδιαίτερο ρόλο εδώ, εντούτοις βολεύει για να αθροίσουμε στη συνέχεια τις εμφανίσεις. Έχουμε, όμως, τελειώσει ή είναι κάτι που απομένει; Εξετάζοντας προσεκτικά τόσο τα δεδομένα όσο και τη λύση μας, διαπιστώνουμε ότι κάθε ζεύγος φιλίας αφορά δύο χρήστες, άρα η διεργασία *map* για κάθε γραμμή που επεξεργάζεται θα πρέπει να παράγει δύο ζεύγη κλειδιού-τιμής, ένα για κάθε όνομα χρήστη. Έτσι, για παράδειγμα, για τη φιλία Apostolos, Alexandros θα πρέπει να παραχθεί έξοδος τόσο για τον χρήστη Apostolos όσο και τον χρήστη Alexandros, δηλαδή τα ζεύγη <Apostolos, 1> και <Alexandros, 1>. Έτσι, η λύση μας, όσον αφορά τη φάση *map*, θα έχει τη μορφή που απεικονίζεται στον Αλγόριθμο 3.1.

Αλγόριθμος 3.1 Αλγόριθμος της διαδικασίας MAP για την επίλυση του προβλήματος της εύρεσης φίλων σε κοινωνικό δίκτυο.

MAP()

for all *line* $\in$ *Block* **do**

Username_1, *Username_2* = Split(*line*, ",")

 Emit <*Username_1*, 1>

 Emit <*Username_2*, 1>

end for

Το αποτέλεσμα θα είναι, μέσω της διαδικασίας μείξης-ταξινόμησης, τα ονόματα χρήστη να φτάσουν στον *reducer* ταξινομημένα, κάθε ένα ακολουθούμενο από μία λίστα άσων. Έτσι, στη συνέχεια μπορούμε για κάθε όνομα να μετρήσουμε το πλήθος των εμφανίσεων (που ισούται με το άθροισμα του πλήθους των άσων ως τιμών) και να εξαγάγουμε το αποτέλεσμα. Το αποτέλεσμα που θα εξαχθεί θα έχει και αυτό τη μορφή ζεύγους κλειδιού-τιμής, με κλειδί το

όνομα χρήστη και τιμή το άθροισμα των άσων που αναπαριστά τις εμφανίσεις του σε σχέσεις φιλίας. Έτσι, η λύση μας, όσον αφορά τη φάση reduce, θα έχει τη μορφή του Αλγορίθμου 3.2.

Αλγόριθμος 3.2 Αλγόριθμος της διαδικασίας REDUCE για την επίλυση του προβλήματος της εύρεσης φίλων σε κοινωνικό δίκτυο.

```

REDUCE()
for all Username  $\in$  Keys do
    RETRIEVE(Username, valueList = [1, 1,  $\dots$ , 1])
    Emit <Username, SUM(valueList)>
end for

```

Έχοντας σκιαγραφήσει την προσέγγιση που θα ακολουθήσουμε για να λύσουμε το παραπάνω πρόβλημα, μπορούμε πλέον να δούμε πώς αυτή θα υλοποιηθεί. Θα παρουσιάσουμε δύο υλοποιήσεις για το Hadoop MapReduce: μία σε Java, που είναι η εγγενής γλώσσα του Hadoop, καθώς είναι υλοποιημένο σε αυτή, και μία σε Python, ώστε να μελετήσουμε και τον μηχανισμό streaming που το Hadoop διαθέτει.

3.6.3 Υλοποίηση σε Java

Θα ξεκινήσουμε με την υλοποίηση σε Java, όπως φαίνεται στον Κώδικα 3.2. Εδώ, υλοποιούμε μία κλάση Social, η οποία επεκτείνει την κλάση Configured και υλοποιεί τη διεπαφή Tool. Με αυτόν τον τρόπο έχουμε τη δυνατότητα να περνάμε παραμέτρους συστήματος κατά την εκτέλεση. Στη συνέχεια, βλέπουμε δύο εμφωλευμένες κλάσεις. Στη γραμμή 2 είναι η κλάση SocialMapper, η οποία επεκτείνει την κλάση Mapper και εντός αυτής θα υπάρξει υλοποίηση της map, ενώ στη γραμμή 23 υπάρχει υλοποίηση της reduce μέσω της κλάσης SocialReducer.

Η κλάση SocialMapper αποτελείται στο παράδειγμά μας από δύο μεταβλητές κλάσης και μία μέθοδο map. Επίσης, βλέπουμε ότι λαμβάνει τέσσερις παραμέτρους. Οι δύο πρώτες παράμετροι αφορούν τους τύπους δεδομένων εισόδου της μεθόδου map, ενώ οι δύο τελευταίες τους τύπους δεδομένων εξόδου με τη μορφή ζευγών κλειδιού-τιμής. Με άλλα λόγια, η map θα διαβάζει LongWritable αριθμούς γραμμών ως κλειδί, το περιεχόμενο των γραμμών ως συμβολοσειρές με τη μορφή Text ως τιμή, και θα παράγει συμβολοσειρές ως κλειδί με τιμή κάποιον ακέραιο. Τα Text, IntWritable, LongWritable είναι οι τύποι του Hadoop για τους αντίστοιχους τύπους δεδομένων.

Η μεταβλητή mapKey της γραμμής 4 είναι τύπου Text καθώς θα χρησιμοποιηθεί ως κλειδί και, όπως είπαμε και πρωτύτερα, θα φιλοξενήσει το όνομα χρήστη. Η μεταβλητή mapValue είναι τύπου IntWritable, καθώς θα φιλοξενήσει την τιμή 1 για κάθε όνομα χρήστη που θα ορίσουμε ως κλειδί.

Η μέθοδος map, τώρα, θα καλείται για κάθε μία γραμμή δεδομένων που διαβάζεται από ένα block. Βλέπουμε ότι λαμβάνει ως παραμέτρους ένα κλειδί και μία τιμή, στα οποία αναφερθήκαμε πιο πριν, και ένα αντικείμενο τύπου Context το οποίο χρησιμοποιείται για επικοινωνία του κώδικά μας με το Hadoop. Στις γραμμές 10-16 βρίσκεται η κύρια υλοποίηση της λύσης μας. Κάθε μία γραμμή διασπάται ως προς το κόμμα σε δύο τμήματα που αντιστοιχούν στα ονόματα χρηστών. Στη συνέχεια, για κάθε ένα από αυτά παράγεται ένα ζεύγος κλειδιού-τιμής, με κλειδί το όνομα χρήστη και τιμή 1 (γραμμή 16).

Η κατάσταση είναι παρόμοια στην κλάση `SocialReducer` η οποία υλοποιεί τη λειτουργία `reduce`. Παίρνει τέσσερις παραμέτρους, που στην ουσία είναι δύο ζεύγη κλειδιών-τιμών. Οι τύποι του πρώτου ζεύγους αφορούν την έξοδο της φάσης `map`. Έτσι, καθώς κατά τη φάση αυτή η έξοδος θα έχει ως κλειδί `Text` και ως τιμή `IntWritable`, στη φάση `reduce` για κάθε κλειδί τύπου `Text` που θα λαμβάνεται, αντίστοιχος θα είναι και ο τύπος της τιμής (`IntWritable`). Το άλλο ζεύγος κλειδιού-τιμής που βρίσκεται στις παραμέτρους αφορά τους τύπους δεδομένων εξόδου της λειτουργίας `reduce`. Έτσι, τα ζεύγη που θα παράγονται θα έχουν κλειδί τύπου συμβολοσειράς και τιμή τύπου ακεραίου, που θα αναπαριστούν το όνομα χρήστη και το πλήθος των φίλων του.

Η κλάση αυτή αποτελείται από ένα πεδίο τύπου `IntWritable` που, όπως είπαμε, χρησιμοποιείται για την έξοδο του πλήθους των φίλων, και τη μέθοδο `reduce`. Οι παράμετροι της `reduce` αντιστοιχώς με αυτές της `map`, αφορούν την είσοδό της, μόνο που αυτήν τη φορά η είσοδος προέρχεται από το αποτέλεσμα της φάσης μείξης-ταξινόμησης και θα λαμβάνεται και μία λίστα τιμών που αντιστοιχούν σε αυτό το κλειδί, η προσπέλαση στις οποίες θα γίνεται μέσω `Iterator`, γι' αυτό και οι τιμές είναι τύπου `Iterable<IntWritable>`. Στις γραμμές 30-33 αρχικοποιείται μία μεταβλητή `sum` σε 0, η οποία θα αποθηκεύει το άθροισμα των άσων που αποτελούν οι τιμές `val` για ένα συγκεκριμένο κλειδί (γραμμή 32). Στη συνέχεια, στη γραμμή 34, τίθεται η τιμή του αθροίσματος στο αντικείμενο τύπου `IntWritable` που θα χρησιμοποιηθεί για έξοδο, η οποία παράγεται στη γραμμή 35 με κλειδί το όνομα χρήστη που αφορούν οι συγκεκριμένες τιμές.

Η μέθοδος `run`, τώρα, καλείται από τη μέθοδο `main`. Μέσα στη `run` (γραμμές 38-50) ορίζουμε τα στοιχεία εκτέλεσης του προγράμματος `MapReduce` που υλοποιούμε. Για να εκτελεστεί αυτό, θα πρέπει να οριστεί ένα νέο `Job` (γραμμή 39), το οποίο μέσω της `getConf()` λαμβάνει τις μεταβλητές περιβάλλοντος. Στη συνέχεια, με τη `SetJarByClass` της γραμμής 40 ορίζουμε την κύρια κλάση της συγκεκριμένης εργασίας, ενώ στη γραμμή 41 λέμε στο `Hadoop MapReduce` ότι επιθυμούμε η φάση `reduce` της εργασίας `job` να υλοποιηθεί από δύο διεργασίες. Στις γραμμές 42-45 ορίζονται αντιστοιχώς οι κλάσεις για τη φάση `map`, για τη φάση `reduce`, καθώς και οι τύποι δεδομένων που θα παραχθούν από το `job` για το κλειδί και την τιμή, αντίστοιχα. Στις γραμμές 46 και 47 ορίζουμε από ποια διαδρομή θα διαβάσει τα δεδομένα το `Hadoop MapReduce` και πού θα αποθηκεύσει τα αρχεία εξόδου. Τέλος, στη γραμμή 49 αποστέλλουμε την εργασία μας για εκτέλεση στη συστάδα και αναμένουμε την ολοκλήρωσή της.

```

1 public class Social extends Configured implements Tool {
2     public static class SocialMapper
3         extends Mapper<LongWritable, Text, Text, IntWritable> {
4         Text mapKey = new Text();
5         IntWritable mapValue = new IntWritable();
6
7         @Override
8         public void map(LongWritable key, Text value, Context context)
9             throws IOException, InterruptedException {
10             String line = value.toString();
11             try {
12                 String record[] = line.split(",");
13                 mapValue.set(1);
14                 for(int i=0; i< record.length;i++){
15                     mapKey.set(record[i]);

```

```

16         context.write(mapKey, mapValue);
17     }
18
19     } catch (Exception e) {
20         e.printStackTrace();
21     } } }
22
23     public static class SocialReducer
24         extends Reducer<Text,IntWritable,Text,IntWritable> {
25         private IntWritable result = new IntWritable();
26
27         @Override
28         public void reduce(Text key, Iterable<IntWritable> values, Context
context)
29             throws IOException, InterruptedException {
30             int sum = 0;
31             for (IntWritable val : values) {
32                 sum += val.get();
33             }
34             result.set(sum);
35             context.write(key, result);
36         } }
37
38     public int run(String[] args) throws Exception {
39         Job job = Job.getInstance(getConf());
40         job.setJarByClass(Social.class);
41         job.setNumReduceTasks(2);
42         job.setMapperClass(SocialMapper.class);
43         job.setReducerClass(SocialReducer.class);
44         job.setOutputKeyClass(Text.class);
45         job.setOutputValueClass(IntWritable.class);
46         FileInputFormat.addInputPath(job, new Path(args[0]));
47         FileOutputFormat.setOutputPath(job, new Path(args[1]));
48
49         int completion = job.waitForCompletion(true) ? 0 : 1;
50         return(completion);
51     }
52
53     public static void main(String[] args) throws Exception{
54         ToolRunner.run(new Configuration(), new Social(), args);
55     } }

```

Κώδικας 3.2: Υλοποίηση του αλγορίθμου σε Java.

3.6.4 Εκτέλεση υλοποίησης Java

Αφού είδαμε πώς υλοποιούμε τον αλγόριθμό μας σε Java, ας δούμε τώρα πώς μπορούμε να μεταγλωττίσουμε και να εκτελέσουμε τον κώδικά μας. Αρχικά, θα πρέπει να έχει οριστεί η μεταβλητή περιβάλλοντος HADOOP_CLASSPATH:

```
export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
```

Έπειτα, μπορούμε να μεταγλωττίσουμε. Θεωρούμε ότι στη μεταβλητή περιβάλλοντος PATH

έχει οριστεί η διαδρομή στην οποία βρίσκεται το εκτελέσιμο hadoop. Ειδικότερα, αυτό βρίσκεται στη διαδρομή ΚΑΤΑΛΟΓΟΣ_HADOOP/bin/.

```
hadoop com.sun.tools.javac.Main Social.java
```

Στη συνέχεια, δημιουργούμε ένα αρχείο .jar με όλες τις κλάσεις που δημιουργήθηκαν:

```
jar cf social.jar Social*.class
```

Και εκτελούμε:

```
hadoop jar social.jar Social input output
```

όπου input και output οι κατάλογοι εισόδου και εξόδου, αντίστοιχα. Στην περίπτωση μας, καθώς έχουμε ορίσει δύο διεργασίες reduce, η έξοδός μας θα αποτελείται από δύο αρχεία, τα part-r-00000 και part-r-00001, τα περιεχόμενα των οποίων βλέπουμε παρακάτω.

```
Alexandros 1
```

Κώδικας 3.3: part-r-00000.

```
Alex 2
Anastasios 3
Apostolos 2
Georgia 2
```

Κώδικας 3.4: part-r-00001.

Όπως μπορούμε να διαπιστώσουμε, το κάθε ένα αρχείο είναι ταξινομημένο ως προς το κλειδί, καθώς με αυτήν τη σειρά έφτασαν στην κάθε διεργασία reduce τα κλειδιά.

Κατά την εκτέλεση του προγράμματός μας, η χρήση του Tool interface μας παρέχει τη δυνατότητα να περάσουμε μια σειρά από παραμέτρους στο πρόγραμμά μας. Μία τέτοια παράμετρος είναι και το πλήθος των διεργασιών reduce, οι οποίες και θα εκτελέσουν τον κώδικα της αντίστοιχης φάσης. Είδαμε ότι μπορούμε να ορίσουμε αυτή την παράμετρο προγραμματιστικά. Τώρα θα δούμε πώς μπορούμε να την ορίσουμε κατά την εκτέλεση. Σε αυτήν την περίπτωση, δεν χρειάζεται η εντολή της γραμμής 41 του Κώδικα 3.2 job.setNumReduceTasks. Αντί αυτού θα εκτελέσουμε το πρόγραμμά μας με την εντολή του Κώδικα 3.5.

```
hadoop jar social.jar Social input output -Dmapreduce.job.reduces=5
```

Κώδικας 3.5: Ορισμός αριθμού διεργασιών reduce κατά την εκτέλεση προγράμματος με χρήση Tool interface.

Προσοχή! Κατά την εκτέλεση, ο κατάλογος προορισμού δεν θα πρέπει να υφίσταται, ώστε να δημιουργηθεί από το Hadoop κατά την εκτέλεση του προγράμματος. Σε αντίθετη περίπτωση, θα προκύψει σφάλμα, καθώς η εκτέλεση δεν θα μπορεί να πραγματοποιηθεί.

3.6.5 Τροποποίηση για χρήση συνδυαστή

Ο τρόπος με τον οποίο είναι υλοποιημένος ο αλγόριθμός μας είναι ορθός μεν, αλλά επιδέχεται βελτίωση, αναφορικά με την απόδοσή του. Καθώς κάθε χρήστης αναμένεται να έχει περισσότερους από έναν φίλους, κατά τη φάση map, αναμένεται να παραχθούν πολλά όμοια

ζεύγη κλειδιού-τιμής για κάθε ένα όνομα χρήστη της μορφής <username,1>, ..., <username,1>. Ο συνδυαστής που μπορούμε να χρησιμοποιήσουμε εδώ θα έχει την ίδια λειτουργία με αυτήν που θα επιτελεστεί κατά τη διεργασία reduce. Σε αυτήν την περίπτωση, δεν χρειάζεται να ξαναπρογραμματίσουμε την ίδια λειτουργία. Το μόνο που αρκεί να κάνουμε είναι να ορίσουμε μία κλήση εντός της run. Η κλήση αυτή φαίνεται στη γραμμή 3 στον Κώδικα 3.6. Κατά τα λοιπά, η εκτέλεση πραγματοποιείται με τον ίδιο τρόπο.

```
1 ...
2 job.setMapperClass(SocialMapper.class);
3 job.setCombinerClass(SocialReducer.class);
4 job.setReducerClass(SocialReducer.class);
5 ...
```

Κώδικας 3.6: Χρήση combiner με Java.

3.6.6 Υλοποίηση σε Python

Με αντίστοιχο τρόπο, μπορούμε να υλοποιήσουμε τον παραπάνω αλγόριθμο σε Python. Υπάρχει, όμως, μία διαφορά σε αυτήν την περίπτωση. Εδώ, θα πρέπει να γίνει χρήση του Hadoop Streaming, που είναι ο μηχανισμός που διαθέτει το Hadoop ώστε να επιτρέπει την εκτέλεση προγραμμάτων από γλώσσες εκτός της Java. Ουσιαστικά, το Hadoop επιτρέπει με αυτόν τον τρόπο να γραφτούν προγράμματα σε οποιαδήποτε γλώσσα προγραμματισμού, αρκεί αυτή να έχει τη δυνατότητα ανάγνωσης από το standard input και εγγραφής στο standard output. Αυτό σημαίνει ότι το πρόγραμμα σε Python θα γραφτεί με τέτοιον τρόπο ώστε να διαβάζει τα δεδομένα από το standard input και να τα γράφει στο standard output. Επιπλέον, θα πρέπει να δημιουργήσουμε δύο ξεχωριστά αρχεία: ένα για να εκτελείται από τις διεργασίες map και ένα για εκτέλεση από τις διεργασίες reduce.

Θα ξεκινήσουμε με την υλοποίηση της λειτουργίας map, όπως απεικονίζεται στον Κώδικα 3.7. Στη γραμμή 2 εισάγουμε το πακέτο sys. Αυτό μας είναι χρήσιμο ώστε να έχουμε πρόσβαση στην προκαθορισμένη είσοδο, όπως βλέπουμε στη γραμμή 3 (sys.stdin). Εκεί βρίσκουμε και τη μεταβλητή line, όπου και αποθηκεύεται κάθε γραμμή που διαβάζεται από την προκαθορισμένη είσοδο. Στη συνέχεια, στη γραμμή 4, από τη γραμμή που μόλις διαβάστηκε από την είσοδο αφαιρείται ο χαρακτήρας αλλαγής γραμμής. Στη γραμμή 5 διασπάται η γραμμή στον χαρακτήρα ",", ο οποίος και διαχωρίζει τα ονόματα χρηστών μεταξύ τους. Στη συνέχεια, για κάθε ένα από τα ονόματα χρηστών παράγεται στην προκαθορισμένη έξοδο ο συνδυασμός ζεύγους-κλειδιού <όνομα_χρήστη, 1>. Το κλειδί από την τιμή διαχωρίζεται με έναν στηλοθέτη, χαρακτήρα που αποτελεί και τον προκαθορισμένο χαρακτήρα διαχωρισμού κλειδιού από τιμή στο Hadoop Streaming.

```
1 #!/usr/bin/env python3
2 import sys
3 for line in sys.stdin:
4     line = line.strip()
5     usernames = line.split(",")
6     for username in usernames:
7         print(username, "\t", str(1))
```

Κώδικας 3.7: map.py.

Θα συνεχίσουμε με τη λειτουργία `reduce`, η οποία παρατίθεται στον Κώδικα 3.8. Κάτι που θα πρέπει να επισημάνουμε στην υλοποίηση σε Python, σε αντιπαράβολή με αυτή της Java, είναι ότι εδώ θα πρέπει να διαχωρίσουμε προγραμματιστικά τις τιμές από τα κλειδιά και να διακρίνουμε, επίσης με δικό μας κώδικα, τις αλλαγές των κλειδιών, λαμβάνοντας υπόψη και εκμεταλλευόμενοι το γεγονός ότι τα κλειδιά θα λαμβάνονται ταξινομημένα.

Ξεκινάμε από τις γραμμές 2-4 στον Κώδικα 3.8, όπου και εδώ εισάγουμε το πακέτο `sys`, όπως και στον Κώδικα 3.7. Επιπλέον, αρχικοποιούμε δύο μεταβλητές: τη `username` και την `prev_username`, οι οποίες θα μας χρησιμεύσουν ώστε να αντιληφθούμε την αλλαγή των κλειδιών. Η πρώτη αφορά το `username` της τρέχουσας γραμμής, ενώ η δεύτερη το `username` της προηγούμενης γραμμής. Αρχικά, τις θέτουμε και τις δύο στην τιμή `None`.

Στη συνέχεια, προχωράμε με ανάγνωση από την προκαθορισμένη είσοδο (γραμμή 6), γραμμή προς γραμμή, μόνο που τώρα η κάθε γραμμή που διαβάζουμε αποτελεί ζευγάρι κλειδιού-τιμής το οποίο προήλθε από τη φάση `map` μέσω της φάσης μείξης-ταξινόμησης. Τονίζουμε και πάλι ότι οι γραμμές αυτές θα είναι ταξινομημένες μεταξύ τους ως προς το κλειδί. Οι τιμές δεν συμμετέχουν στην ταξινόμηση αυτή.

Στη γραμμή 7 απαλείφουμε τον χαρακτήρα αλλαγής γραμμής, ενώ στη γραμμή 8 διαχωρίζουμε το κλειδί από την τιμή και τα αποθηκεύουμε στις αντίστοιχες μεταβλητές. Χρησιμοποιήσαμε στη φάση `map` τον χαρακτήρα στηλοθέτη ως διαχωριστικό και βάσει αυτού πραγματοποιούμε τον διαχωρισμό. Στη γραμμή 9 μετατρέπουμε την τιμή που διαβάσαμε (`one`) σε ακέραιο, ο οποίος στην περίπτωση μας θα είναι πάντα ίσος με 1, ώστε να μπορέσουμε στη συνέχεια, αθροίζοντας τις τιμές, να μετρήσουμε τις εμφανίσεις κάθε κλειδιού.

Στις γραμμές 10 έως 16 εντοπίζουμε και διαχειριζόμαστε την αλλαγή των κλειδιών. Για κάθε γραμμή που διαβάζουμε διατηρούμε το κλειδί της προηγούμενης (γραμμή 16), ώστε να μπορέσουμε να εντοπίσουμε την αλλαγή κλειδιού. Έχουμε δύο περιπτώσεις: την αλλαγή κλειδιού (γραμμές 10-13) και την ύπαρξη ίδιου κλειδιού μεταξύ δύο διαδοχικών γραμμών (14-15). Για την πρώτη περίπτωση, ο εντοπισμός αλλαγής κλειδιού μεταξύ διαδοχικών γραμμών πραγματοποιείται στη γραμμή 10, οπότε και θα πρέπει να παραγάγουμε έξοδο.

Αυτό, όμως, μόνο στην περίπτωση που το προηγούμενο όνομα χρήστη δεν είναι `None` (γραμμή 11), επειδή τότε θα πρόκειται για την πρώτη γραμμή. Καθώς παράγουμε έξοδο για το προηγούμενο κλειδί, αφού έχουμε συναντήσει κάποιο νέο (γραμμή 12), αν δεν υπήρχε ο έλεγχος αυτός θα παραγόταν έξοδος για το πρώτο ζευγάρι κλειδιού-τιμής, καθώς θα υπήρχε σύγκριση με την αρχική τιμή `None`.

```

1 #!/usr/bin/env python3
2 import sys
3 username = None
4 prev_username = None
5
6 for line in sys.stdin:
7     line=line.strip()
8     username, one = line.split("\t")
9     one = int(one)
10    if username != prev_username:
11        if prev_username!=None:
12            print(prev_username, "\t", occurrences)
13        occurrences = one

```

```

14     else:
15         occurrences += one
16         prev_username = username
17 print(prev_username, "\t", occurrences)

```

Κώδικας 3.8: *reduce.py*.

Στην περίπτωση αλλαγής κλειδιού, θα πρέπει να αρχίσουμε να μετράμε εκ νέου τις εμφανίσεις του κάθε κλειδιού, μεταξύ διαδοχικών γραμμών με το ίδιο κλειδί. Έτσι, για την πρώτη γραμμή που διαβάζουμε και εντοπίζουμε ένα κλειδί, αρχικοποιούμε τη μεταβλητή `occurrences` σε `one` (γραμμή 13). Αυτό το κάνουμε για κάθε αλλαγή κλειδιού, είτε πρόκειται για το πρώτο κλειδί που εντοπίζουμε είτε για οποιοδήποτε νέο κλειδί.

Λόγω της ταξινόμησης ως προς το κλειδί, που επιφέρει η φάση μείξης-ταξινόμησης, κάθε μοναδικό κλειδί μπορούμε να το συναντούμε μόνον σε διαδοχικές γραμμές του ενδιαμέσου αρχείου που διαβάζει ο κώδικας της φάσης `reduce`. Αν αλλάξει η τιμή του κλειδιού, δεν πρόκειται να συναντήσουμε το ίδιο κλειδί ξανά.

Στις γραμμές 14-15 έχουμε την περίπτωση για την οποία στις γραμμές τις οποίες διαβάζουμε η τιμή του κλειδιού δεν μεταβάλλεται. Έτσι, αυξάνουμε τη μεταβλητή `occurrences` κατά ένα. Με τη γραμμή 16 ασχοληθήκαμε πιο πριν.

Τέλος, στη γραμμή 17 παράγουμε την έξοδο για το τελευταίο κλειδί που εντοπίσαμε. Αυτό είναι απαραίτητο, καθώς παράγουμε έξοδο για το προηγούμενο κλειδί, κάθε φορά που εντοπίζουμε αλλαγή στην τιμή ενός κλειδιού. Όταν τελειώσουν οι γραμμές στην προκαθορισμένη είσοδο, θα έχει απομείνει το τελευταίο κλειδί με το πλήθος εμφανίσεών του, το οποίο και θα πρέπει να παραγάγουμε ξεχωριστά.

3.6.7 Εκτέλεση υλοποίησης σε Python

Πριν εκτελέσουμε το πρόγραμμά μας στην εγκατάστασή μας του Hadoop, μπορούμε να το δοκιμάσουμε με τη χρήση διασωληνώσεων Unix. Μπορούμε, λοιπόν, να προσομοιώσουμε την εκτέλεση όπως φαίνεται στον Κώδικα 3.9: Παραθέτουμε αρχικά τα περιεχόμενα των αρχείων στον κατάλογο `input`. Με διασωλήνωση ανακατευθύνουμε το περιεχόμενο ως είσοδο στο αρχείο `map.py` το οποίο είναι εκτελέσιμο. Στη συνέχεια, η έξοδος αυτού ανακατευθύνεται στην εντολή `sort` η οποία και ταξινομεί τα κλειδιά από την έξοδο που αποτελείται από ζεύγη κλειδιών-τιμών που παράγει η εκτέλεση του `map.py`. Η έξοδος της εκτέλεσης του `reduce.py` θα εμφανιστεί στην οθόνη. Με αντίστοιχο τρόπο, βέβαια, μπορεί να ανακατευθυνθεί και σε αρχείο.

```
cat input/* | ./map.py | sort | ./reduce.py
```

Κώδικας 3.9: Εκτέλεση με διασωληνώσεις Unix.

Για να εκτελέσουμε, τώρα, τον κώδικα που γράψαμε σε Python στη συστάδα μας, θα κάνουμε χρήση του μηχανισμού Hadoop Streaming. Ο τρόπος εκτέλεσης φαίνεται στον Κώδικα 3.10. Χρησιμοποιείται το αρχείο `hadoop-streaming-*.jar`. Ο αστερίσκος υποκαθιστά την έκδοση του αρχείου. Έπειτα, ακολουθεί η παράμετρος `-files` η οποία χρησιμεύει ώστε να μεταφέρει τοπικά σε κάθε κόμβο τα αρχεία που ακολουθούν. Έτσι, στη δική μας περίπτωση, θα μεταφερθούν σε κάθε κόμβο που θα πραγματοποιήσει την εκτέλεση τα `map.py` και `reduce.py`.

Η -files θα πρέπει να προηγείται των παρακάτω εντολών.

```
hadoop jar hadoop/tools/lib/hadoop-streaming-*.jar -files map.py,reduce.py
-mapper map.py -reducer reduce.py -input input -output output
```

Κώδικας 3.10: Εκτέλεση κώδικα Python με Hadoop Streaming.

Στη συνέχεια, έχουμε ορίσει τις εξής εντολές. Με -mapper υποδεικνύουμε το εκτελέσιμο αρχείο το οποίο υλοποιεί τη λειτουργία map, που στην περίπτωσή μας είναι το map.py. Με -reducer ορίζουμε το εκτελέσιμο που θα πραγματοποιήσει τη λειτουργία reduce. Εδώ είναι το reduce.py. Τα -input και -output ορίζουν τους καταλόγους με τα αρχεία εισόδου και εξόδου, αντίστοιχα. Στο παράδειγμά μας, αυτοί έχουν όνομα input και output.

```
hadoop jar hadoop/tools/lib/hadoop-streaming-*.jar -files map.py,reduce.py
-mapper map.py -reducer reduce.py -combiner reduce.py -input input
-output output -numReduceTasks 2
```

Κώδικας 3.11: Εκτέλεση κώδικα Python με Hadoop Streaming ορίζοντας επιπλέον παραμέτρους.

Ας δούμε τώρα πώς μπορούμε να προσθέσουμε επιπλέον παραμέτρους. Στο παράδειγμα εκτέλεσης που παρατίθεται στον Κώδικα 3.11 προσθέτουμε δύο επιπλέον παραμέτρους. Ορίζουμε συνδυαστή με την παράμετρο -combiner και πλήθος διεργασιών που θα εκτελέσουν τη φάση reduce με -numReduceTasks. Εδώ, χρησιμοποιούμε ως combiner το reduce.py και θέτουμε 2 διεργασίες reduce.

3.6.8 Πλήθος διεργασιών map και reduce

Το πλήθος των διεργασιών map εξαρτάται από το συνολικό πλήθος των blocks των αρχείων εισόδου.

Σύμφωνα με τον ιστότοπο του Hadoop [had], ο καλύτερος αριθμός διεργασιών reduce φαίνεται να είναι 0.95 ή 1.75 πολλαπλασιαζόμενο με ($\langle \text{αριθμό_κόμβων_στη_συστάδα} \rangle * \text{mapred.tasktracker.reduce.tasks.maximum} \rangle$).

Με 0.95, όλες οι διεργασίες reduce μπορούν να εκκινήσουν άμεσα τη λειτουργία τους λαμβάνοντας ως είσοδο την έξοδο των διεργασιών map. Με 1.75, οι γρηγορότεροι κόμβοι θα τελειώσουν τον πρώτο γύρο από το reduce που θα τους έχει ανατεθεί και θα εκκινήσουν δεύτερο, πραγματοποιώντας καλύτερη εξισορρόπηση φόρτου.

Με την αύξηση του πλήθους των διεργασιών που εκτελούν τη λειτουργία reduce, αφενός μεν δημιουργείται επιβάρυνση στο σύστημα, αφετέρου δε επιτυγχάνεται καλύτερη εξισορρόπηση φόρτου και, σε περίπτωση αστοχίας μιας διεργασίας reduce, η επίδραση θα είναι μικρή, καθώς αυτή θα έχει αναλάβει μικρό όγκο επεξεργασίας και θα μπορεί να αντικατασταθεί από άλλη. Οι παράγοντες που παρουσιάζονται παραπάνω δεν είναι ακέραιοι, αλλά ελαφρώς κάτω από αυτούς, ώστε να υπάρχει περιθώριο στο σύστημα για εργασίες που τυχόν αποτύχουν.

Αντιστοίχως, όπως είπαμε και στις αρχές του κεφαλαίου αυτού, δίνεται η δυνατότητα να μην υπάρχει καμία διεργασία reduce. Σε αυτήν την περίπτωση, οι έξοδοι πηγαίνουν κατευθείαν στο σύστημα αρχείων και δεν ταξινομούνται.

3.7 Συζήτηση

Στο κεφάλαιο αυτό μελετήσαμε το μοντέλο MapReduce και είδαμε πώς μπορούμε να σχεδιάσουμε έναν αλγόριθμο για το μοντέλο αυτό αλλά και να το υλοποιήσουμε για το Apache Hadoop, τόσο σε Java όσο και σε Python. Το Hadoop δεν είναι, όμως, το μόνο σύστημα που υποστηρίζει προγραμματιστικά αυτό το μοντέλο. Και το Apache Spark [spa] μπορεί να υλοποιήσει προγράμματα σχεδιασμένα για το μοντέλο MapReduce. Αν και σε εκείνη την περίπτωση η υλοποίηση, προγραμματιστικά, είναι διαφορετική, εντούτοις ο ίδιος αλγόριθμος μπορεί κάλλιστα να υλοποιηθεί και να εκτελεστεί κατανεμημένα.

Το μοντέλο MapReduce, λοιπόν, βρίσκει χρήση και εκτός του Hadoop. Αυτό μας οδηγεί στο συμπέρασμα ότι είναι χρήσιμο να αφομοιώσουμε αφηρημένες έννοιες και να αναπτύξουμε διαφορετικούς τρόπους σκέψης. Μπορεί η κατανόηση του τρόπου εργασίας στο συγκεκριμένο μοντέλο να φαίνεται αρχικά δύσκολη, αλλά μπορεί να βρει πληθώρα εφαρμογών.

3.8 Περαιτέρω μελέτη

Αναμφίβολα, το μοντέλο MapReduce χρήζει περαιτέρω μελέτης για να γίνει κατανοητό. Αυτό το κεφάλαιο αποσκοπεί πρώτιστα στη γνωριμία μαζί του. Ο αναγνώστης που ενδιαφέρεται να εμβαθύνει καλείται να διαβάσει πρωτίτως τη σχετική εργασία [DG04] και έπειτα το βιβλίο των Lin και Dyer [LD10] με πληθώρα παραδειγμάτων. Ενδιαφέροντα παραδείγματα υπάρχουν και στο [RU11], ενώ για πιο ενδελεχή μελέτη των μηχανισμών του Hadoop MapReduce ο αναγνώστης μπορεί να ανατρέξει στη σχετική βιβλιογραφία, όπως στα [Whi12, WSV14].

ΒΙΒΛΙΟΓΡΑΦΙΑ

[DG04] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *OSDI'04: Sixth Symposium on Operating System Design and Implementation*, pages 137–150, San Francisco, CA, 2004.

[had] Apache hadoop. <https://hadoop.apache.org/>. Accessed: 2023-09-30.

[LD10] Jimmy Lin and Chris Dyer. Data-intensive text processing with mapreduce. *Synthesis Lectures on Human Language Technologies*, 3(1):1–177, 2010.

[RU11] Anand Rajaraman and Jeffrey David Ullman. *Mining of massive datasets*. Cambridge University Press, 2011.

[spa] Spark standalone mode - spark documentation. <https://spark.apache.org/docs/latest/spark-standalone.html>. Accessed: 2023-09-30.

[Whi12] Tom White. *Hadoop: The definitive guide*. ” O'Reilly Media, Inc.”, 2012.

- [WSV14] Sameer Wadkar, Madhu Siddalingaiah, and Jason Venner. *Pro Apache Hadoop*. Springer, 2014.

ΚΕΦΑΛΑΙΟ 4

ΣΥΣΤΗΜΑΤΑ NOSQL

Περιεχόμενα

4.1	NoSQL βάσεις δεδομένων	77
4.1.1	Αρχές σχεδίασης	78
4.1.2	Διαμέριση και αναπαραγωγή δεδομένων	84
4.1.3	Συνέπεια δεδομένων	86
4.2	Key-value stores	89
4.2.1	Παραδείγματα key-value stores	90
4.3	Column-family stores	92
4.3.1	Παραδείγματα column-family Stores	93
4.4	Βάσεις δεδομένων εγγράφων	96
4.4.1	Παραδείγματα βάσεων δεδομένων εγγράφων	96
4.5	Βάσεις δεδομένων γραφημάτων	103
4.5.1	Παραδείγματα βάσεων δεδομένων γραφημάτων	105
4.6	Συστήματα διαχείρισης ετερογενών δεδομένων	114
4.6.1	Multi-model database	115
4.6.2	Polystores	115
4.6.3	Σύγκριση	117
4.7	Περαιτέρω μελέτη	117
	BIBΛΙΟΓΡΑΦΙΑ	118

Καρακασίδης, Α., Κολωνιάρη, Γ., Γούναρης, Α., & Παπαδόπουλος, Α. (2025). Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων [Μεταπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις. <http://dx.doi.org/10.57713/kallipos-1058>



Σύνοψη

Σε αυτό το κεφάλαιο παρουσιάζονται οι NoSQL βάσεις δεδομένων, δηλαδή τα συστήματα διαχείρισης δεδομένων που δεν στηρίζονται στο σχεσιακό μοντέλο και την SQL και έχουν αναπτυχθεί παράλληλα με την εξάπλωση των μεγάλων δεδομένων και του υπολογιστικού νέφους, ώστε να ταιριάζουν στα χαρακτηριστικά των μεγάλων δεδομένων. Συγκεκριμένα, στο κεφάλαιο αυτό αναπτύσσονται τα εξής θέματα:

- Αίτια και παράγοντες που οδήγησαν στην ανάπτυξη των NoSQL βάσεων δεδομένων και βασικές ιδιότητες των συστημάτων αυτών.
- Βασικές αρχές μοντελοποίησης δεδομένων και σχεδίασης NoSQL βάσεων δεδομένων, και λειτουργικότητα που παρέχουν.
- Ζητήματα όπως διαμέριση και αναπαραγωγή δεδομένων, και συνέπεια δεδομένων.
- Πλεονεκτήματα και μειονεκτήματα των NoSQL συστημάτων και πώς αυτά συγκρίνονται σε σχέση και με όλα τα παραπάνω θέματα με τα παραδοσιακά σχεσιακά συστήματα διαχείρισης βάσεων δεδομένων.
- Κύριες κατηγορίες των NoSQL συστημάτων και σύντομη αναφορά σε έναν ενδεικτικό αντιπρόσωπο της κάθε κατηγορίας, και συγκεκριμένα:
 - key-value stores και Riak,
 - column-family stores και Cassandra,
 - βάσεις δεδομένων εγγράφων και MongoDB, και
 - βάσεις δεδομένων γραφημάτων και Neo4j.
- Συστήματα διαχείρισης ετερογενών δεδομένων και συγκεκριμένα: multi-model databases και polystores.

Προαπαιτούμενη γνώση

Μέρος των γνώσεων που απαιτούνται για το κεφάλαιο αυτό, όπως τα ιδιαίτερα χαρακτηριστικά των μεγάλων δεδομένων αλλά και η διαχείριση κατανεμημένων αρχείων, έχουν συζητηθεί στα προηγούμενα κεφάλαια. Επιπλέον, για να κατανοήσει κανείς τα ζητήματα που ανακύπτουν κατά τον σχεδιασμό των NoSQL συστημάτων, θα πρέπει να έχει γνώση σχετικά με τα σχεσιακά συστήματα διαχείρισης βάσεων δεδομένων και το σχεσιακό μοντέλο. Αν και συμπεριλαμβάνονται εντός του κεφαλαίου σύντομες αναφορές στις βασικές ιδιότητες των σχεσιακών συστημάτων διαχείρισης βάσεων δεδομένων, για να γίνει κατανοητή η διαφορά τους με τα NoSQL συστήματα, συνιστάται ο αναγνώστης να εξοικειωθεί με αυτά. Επίσης, χρήσιμες, αν και όχι απαραίτητες, είναι και γνώσεις σχετικά με κατανεμημένες βάσεις δεδομένων αλλά και γενικότερα κατανεμημένα συστήματα.

4.1 NoSQL βάσεις δεδομένων

Ο όρος NoSQL χρησιμοποιείται για να αναφερθούμε σε οποιοδήποτε σύστημα διαχείρισης δεδομένων το οποίο δεν χρησιμοποιεί το σχεσιακό μοντέλο για τη μοντελοποίηση και αποθήκευση των δεδομένων και επίσης δεν χρησιμοποιεί τη γλώσσα SQL για τη διαχείρισή τους. Εναλλακτικά, ο όρος συχνά αποδίδεται και ως «Not only SQL» και περιγράφει μια πληθώρα διαφορετικών συστημάτων στα οποία χρησιμοποιούνται και άλλες τεχνολογίες διαχείρισης και μοντελοποίησης δεδομένων, πέρα από το σχεσιακό μοντέλο.

Η διάδοση αυτών των συστημάτων τα τελευταία χρόνια και η ανάπτυξη μεγάλου πλήθους διαφορετικών μοντέλων και συστημάτων οδηγούνται από τις μεγάλες απαιτήσεις που έχουν οι σύγχρονες διαδικτυακές εφαρμογές μεγάλης κλίμακας και η διαχείριση των μεγάλων δεδομένων. Τέτοιες εφαρμογές επιβάλλουν απαιτήσεις που τα σχεσιακά συστήματα διαχείρισης δεδομένων δεν έχουν σχεδιαστεί να διαχειρίζονται, με αποτέλεσμα να μην μπορούν πάντα να αντεπεξέλθουν ικανοποιητικά.

Οι σύγχρονες αυτές εφαρμογές διαχειρίζονται μεγάλα δεδομένα, δεδομένα δηλαδή με χαρακτηριστικά, όπως ήδη αναφέραμε σε προηγούμενο κεφάλαιο, τον τεράστιο όγκο τους, την ταχύτητα παραγωγής τους και την ποικιλία τους. Αυτά τα χαρακτηριστικά τα διαφοροποιούν από τα επιχειρησιακά δεδομένα που αρχικά σχεδιάστηκαν για να διαχειρίζονται οι σχεσιακές βάσεις δεδομένων. Με την ανάδειξη του Web 2.0, ένας τεράστιος όγκος δεδομένων που παράγεται στο διαδίκτυο δεν είναι ούτε εύκολο ούτε αποδοτικό, αλλά πολλές φορές ούτε και επιθυμητό να τυποποιηθεί αυστηρά για την αναπαραγωγή και διαχείριση από ένα σχεσιακό σύστημα διαχείρισης δεδομένων. Τα δεδομένα προέρχονται πλέον από μια πληθώρα πηγών, όπως αναρτήσεις χρηστών σε κοινωνικά δίκτυα, πολυμεσικά δεδομένα, δεδομένα αισθητήρων για εφαρμογές IoT ή και δορυφορικών εικόνων. Συνεπώς, τα δεδομένα είναι εν γένει ετερογενή και συνήθως αδόμητα ή ημιδομημένα και, άρα, απαιτούν μια μεγαλύτερη ευελιξία στη μοντελοποίησή τους από αυτήν που προσφέρει μια σχεσιακή βάση δεδομένων που σχεδιάστηκε αποκλειστικά και επιτυγχάνει την αποδοτική διαχείριση σχεσιακών δεδομένων.

Επιπρόσθετα, οι διαδικτυακές εφαρμογές απαιτούν συνήθως υψηλή διαθεσιμότητα. Για να αντεπεξέλθουν σε αυτήν την απαίτηση, τα συστήματα διαχείρισης δεδομένων καταφεύγουν στη διαμέριση ή την αναπαραγωγή των δεδομένων σε πολλαπλές βάσεις δεδομένων, οι οποίες διαμοιράζονται τον φόρτο των αιτημάτων από τους πελάτες. Ωστόσο, η χρήση συναλλαγών για την εξασφάλιση αυστηρής συνέπειας σε αυτήν την περίπτωση γίνεται πολλές φορές αδύνατη, με αποτέλεσμα να προκύψει η ανάγκη για συστήματα διαχείρισης δεδομένων που θα υποστηρίζουν πιο χαλαρές μορφές συνέπειας και συναλλαγών.

Τέλος, η ανάπτυξη του υπολογιστικού νέφους και η ελαστικότητα που παρέχει με το pay-as-you-go μοντέλο απαιτούν επίσης νέα συστήματα βάσεων δεδομένων με δυνατότητα εύκολης οριζόντιας κλιμάκωσης, σε αντίθεση με τις σχεσιακές βάσεις.

Για να συνοψίσουμε, η ανάγκη για ευελιξία στη μοντελοποίηση ετερογενών αδόμητων και ημιδομημένων δεδομένων, η υψηλή διαθεσιμότητα που πρέπει να παρέχεται στις νέες διαδικτυακές εφαρμογές μεγάλης κλίμακας αλλά και η ανάπτυξη υποδομών υπολογιστικού νέφους που προσφέρουν ελαστικότητα και υποστηρίζουν κατανομημένα μοντέλα υπολογισμού και αποθήκευσης αποτελούν τους βασικούς παράγοντες που έχουν φέρει στο προσκήνιο την απαίτηση για νέα συστήματα διαχείρισης δεδομένων και οδήγησαν στη ραγδαία ανάπτυξη

των NoSQL συστημάτων.

Τα NoSQL συστήματα, παρότι όλα σχεδιάστηκαν έχοντας τις παραπάνω απαιτήσεις ως οδηγό, ακολουθούν διαφορετικές προσεγγίσεις τόσο για τη μοντελοποίηση όσο και για την αποθήκευση και διαχείριση των δεδομένων. Ωστόσο, μοιράζονται κάποια κοινά χαρακτηριστικά και στις επόμενες ενότητες θα εστιάσουμε σε αυτά, πριν προχωρήσουμε σε μια παρουσίαση των διαφορετικών προσεγγίσεων.

4.1.1 Αρχές σχεδίασης

Για τη μοντελοποίηση των δεδομένων ενός συστήματος ή μιας εφαρμογής και τη σχεδίαση μιας βάσης δεδομένων, τα βήματα που ακολουθούνται είναι:

- Ανάλυση απαιτήσεων της εφαρμογής ή του συστήματος.
- Σχεδίαση εννοιολογικού μοντέλου. (μοντέλο οντοτήτων/συσχετίσεων – entity/relationship model)
- Μετατροπή από το εννοιολογικό στο λογικό (σχεσιακό μοντέλο – relational model) και κανονικοποίηση (normalization), αν αυτή απαιτείται.

Στη συνέχεια, για τη μετάβαση στο φυσικό μοντέλο, το λογικό μοντέλο υλοποιείται σε κάποιο σχεσιακό σύστημα διαχείρισης δεδομένων.

Η *ανάλυση των απαιτήσεων* του συστήματος έχει ως στόχο να καθοριστούν τα δεδομένα που θα πρέπει να διαχειρίζεται το σύστημα και οι σχέσεις μεταξύ τους. Στόχος της βάσης δεδομένων είναι να μοντελοποιήσει ολοκληρωμένα τη λειτουργία, για παράδειγμα, μιας επιχείρησης ή ενός οργανισμού και των δεδομένων που διαχειρίζεται. Επομένως, κατά τη διαδικασία της ανάλυσης απαιτήσεων καταγράφονται όλα τα «κομμάτια» πληροφορίας που παράγει και χρειάζεται το σύστημα για να προσφέρει ένα σύνολο από επιθυμητές λειτουργίες, καθώς και οι σχέσεις και οι αλληλεπιδράσεις μεταξύ αυτών των δεδομένων, που πιθανόν οδηγούν με τη σειρά τους και στη δημιουργία νέων δεδομένων.

Το *εννοιολογικό μοντέλο* έχει ως στόχο να προσφέρει μια υψηλού επιπέδου αφαίρεση του συνόλου των δεδομένων και των σχέσεών τους για τη σχεδίαση μιας βάσης δεδομένων που θα τα διαχειρίζεται. Αν θεωρήσουμε ως εννοιολογικό μοντέλο το *μοντέλο οντοτήτων/συσχετίσεων*, συνοπτικά έχουμε τα εξής. Τα αντικείμενα του πραγματικού ή ιδεατού κόσμου αποτελούν τις *οντότητες* του μοντέλου και οι σχέσεις μεταξύ των αντικειμένων συνθέτουν τις *συσχετίσεις* του μοντέλου. Αντικείμενα του ίδιου τύπου ομαδοποιούνται σε σύνολα οντοτήτων τα οποία περιγράφονται από ένα σύνολο γνωρισμάτων και προσδιορίζονται μονοσήμαντα από ένα ελάχιστο υποσύνολο τέτοιων γνωρισμάτων, που αποτελεί το *κλειδί* του συνόλου οντοτήτων. Αντιστοίχως, και οι συσχετίσεις ομαδοποιούνται σε σύνολα συσχετίσεων και περιγράφονται από γνωρίσματα.

Για τη σχεδίαση του *λογικού μοντέλου*, μετατρέπεται το εννοιολογικό μοντέλο στις αντίστοιχες *σχέσεις (πίνακες)*. Για τον προσδιορισμό του σχεσιακού μοντέλου, ορίζονται:

- Οι πίνακες που αντιστοιχούν στο μοντέλο, οι στήλες του κάθε πίνακα που αντιστοιχούν στα γνωρίσματα που αποθηκεύει, το πεδίο ορισμού κάθε στήλης.

- Το *πρωτεύον κλειδί* κάθε πίνακα που υλοποιεί τους περιορισμούς ακεραιότητας των δεδομένων και εξασφαλίζει ότι κάθε γραμμή/εγγραφή του πίνακα προσδιορίζεται μοναδικά.
- Τα *ξένα κλειδιά* του πίνακα που υλοποιούν τους περιορισμούς αναφορικής ακεραιότητας και εξασφαλίζουν ότι μια αλλαγή στα δεδομένα θα επηρεάσει όλα τα συσχετιζόμενα δεδομένα.

Η περιγραφή αυτή του τύπου των δεδομένων που αποθηκεύονται σε κάθε πίνακα αποτελεί και το *σχήμα* της βάσης δεδομένων.

Κατά τη σχεδίαση του λογικού μοντέλου, σημαντικό ρόλο παίζει η *κανονικοποίηση* των δεδομένων έτσι ώστε να αποφύγουμε την επανάληψη πληροφορίας εντός των πινάκων που δημιουργούνται. Το σχεσιακό μοντέλο έχει ως βασικό στόχο την αποφυγή επανάληψης πληροφορίας που επιφέρει διάφορα προβλήματα, προκαλώντας ασυνέπεια στα δεδομένα, προβλήματα στις ενημερώσεις και διαγραφές, και βέβαια αύξηση του όγκου των δεδομένων. Για αυτόν τον λόγο, έχουν οριστεί διαφορετικές *κανονικές μορφές* (*normal forms*), ξεκινώντας από τη λιγότερο αυστηρή *1η κανονική μορφή 1NF* και μέχρι και την *5η κανονική μορφή 5NF* που είναι πιο περιοριστική. Να σημειωθεί, ωστόσο, ότι, εάν έχει σχεδιαστεί ένα ορθό μοντέλο οντοτήτων/συσχετίσεων, τότε αυτό μπορεί να αντιστοιχιστεί σε ένα σχεσιακό μοντέλο που αποφεύγει την επανάληψη πληροφορίας και, έτσι, μπορεί να αποφευχθεί ένα επιπλέον βήμα κανονικοποίησης των πινάκων που δημιουργούνται. Μια αλγοριθμική διαδικασία καθορίζει τα βήματα μετατροπής κάθε στοιχείου του μοντέλου οντοτήτων/συσχετίσεων στο σχεσιακό μοντέλο.

Γίνεται φανερό ότι τα δύο πρώτα βήματα της διαδικασίας σχεδίασης δεν εξαρτώνται από το σχεσιακό μοντέλο, αλλά αποσκοπούν στον προσδιορισμό και την καλύτερη κατανόηση των δεδομένων της βάσης δεδομένων που θα δημιουργηθεί. Η διαφοροποίηση έγκειται στο 3ο βήμα της διαδικασίας, που για τις NoSQL βάσεις δεδομένων θα συνιστά τη μετατροπή του εννοιολογικού μοντέλου στο κάθε φορά κατάλληλο NoSQL μοντέλο, είτε ζεύγη κλειδιού-τιμών, είτε γράφημα, είτε έγγραφο, κ.ά. Οπότε, για τη σχεδίαση μιας NoSQL βάσης δεδομένων η διαδικασία που θα πρέπει να ακολουθηθεί κανείς είναι αντίστοιχη και τα αρχικά βήματα είναι κοινά.

Ας εστιάσουμε τώρα στις βασικές διαφορές των NoSQL συστημάτων που θα πρέπει να λάβει κανείς υπόψη στον σχεδιασμό του λογικού μοντέλου κάθε φορά, ανεξάρτητα από το συγκεκριμένο σύστημα που επιλέγει.

4.1.1.1 Συστήματα διαχείρισης δεδομένων χωρίς σχήμα

Η πρώτη απαίτηση που ορίσαμε για τον σχεδιασμό ενός NoSQL συστήματος δεδομένων είναι η *ευελιξία* στην αναπαράσταση ετερογενών δεδομένων, ενώ αντιθέτως, όπως προκύπτει στη σχεδίαση μιας σχεσιακής βάσης δεδομένων, υπάρχει αυστηρή τυποποίηση των δεδομένων μέσα από τον ορισμό του λογικού σχήματος. Η απαίτηση για ευελιξία οδήγησε τα NoSQL συστήματα στο να μην απαιτούν απαραίτητα την ύπαρξη ενός προκαθορισμένου σχήματος δεδομένων και να θεωρούνται *schemaless*.

Αρχικά, η μη ύπαρξη προκαθορισμένου σχήματος επιτρέπει την ενοποίηση στη βάση δε-

δομένων ετερογενών δεδομένων από διαφορετικές πηγές, που πιθανόν περιγράφονται από διαφορετικά χαρακτηριστικά. Για παράδειγμα, σε μια εταιρεία μπορεί το όνομα ενός εργαζομένου να δίνεται ως ονοματεπώνυμο, ενώ σε μια συνεργαζόμενη εταιρεία μπορεί να αποθηκεύεται ως δύο ξεχωριστά γνωρίσματα: όνομα και επώνυμο. Σε μια βάση που θα θέλαμε να συμπεριλάβουμε τους εργαζόμενους και από τις δύο εταιρείες θα θέλαμε να υποστηρίξουμε και τις δύο μορφές, αλλά η ύπαρξη ενός προκαθορισμένου σχήματος θα επέβαλλε μία από τις δύο επιλογές. Επίσης, ας υποθέσουμε ότι στην πρώτη εταιρεία διατηρούμε τον τόπο γέννησης των εργαζομένων, ενώ στη δεύτερη εταιρεία δεν καταγράφεται αυτή η πληροφορία. Το κοινό σχήμα θα έπρεπε να περιλαμβάνει το πεδίο «τόπος γέννησης», το οποίο θα έπρεπε να μείνει κενό για όλους τους εργαζόμενους της δεύτερης εταιρείας. Όπως είναι γνωστό, η απουσία τιμών αποτελεί άλλο ένα πρόβλημα που κατά το δυνατό προσπαθούμε να αποφύγουμε στη σχεδίαση μιας βάσης δεδομένων.

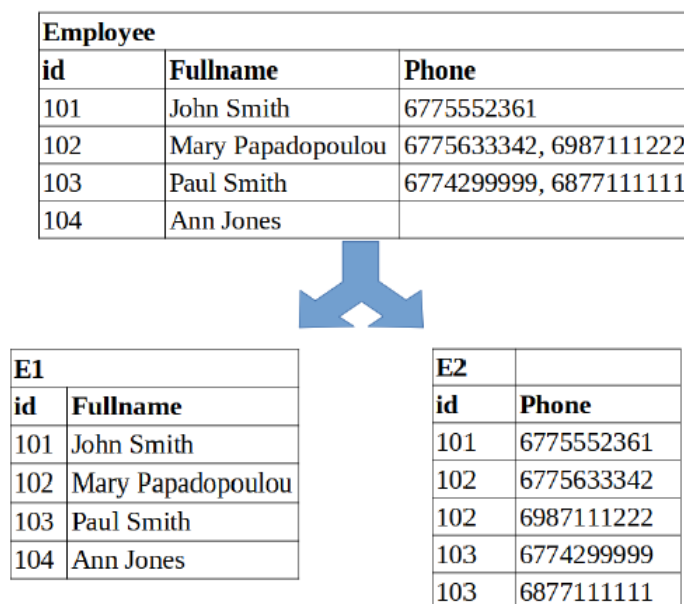
Επιπρόσθετα, σε ένα σύστημα που λειτουργεί σε ένα περιβάλλον που εξελίσσεται και αλλάζει συνεχώς, όπως οι σύγχρονες διαδικτυακές εφαρμογές, αναμένουμε ότι θα συμβαίνουν και συχνές αλλαγές στα χαρακτηριστικά και τις ιδιότητες των δεδομένων που αποθηκεύει. Στο προηγούμενο παράδειγμα των εργαζομένων, ενώ οι εταιρείες αρχικά διατηρούσαν στα στοιχεία επικοινωνίας το τηλέφωνο και τη διεύθυνση των εργαζομένων, ας υποθέσουμε ότι θέλουν να προσθέσουν πλέον και το πεδίο email. Αυτό θα σήμαινε μια αλλαγή στο προκαθορισμένο σχήμα της βάσης μας. Αλλά και πιο μεγάλης κλίμακας αλλαγές, όπως η προσθήκη νέων συσχετίσεων ή η κατάργηση υπαρχουσών συσχετίσεων, μπορεί να καταλήξουν στην ανάγκη για ορισμό ενός νέου σχήματος που θα ανταποκρίνεται στις νέες απαιτήσεις. Συνεπώς, θα θέλαμε μια ευελιξία στο σχήμα των δεδομένων που θα επιτρέπει εύκολες και κυρίως γρήγορες αλλαγές, χωρίς να απαιτείται ο επανασχεδιασμός του συστήματος.

Ωστόσο, η έλλειψη ενός σχήματος μπορεί να προκαλέσει και προβλήματα τόσο στη συνέπεια όσο και στη διαχείριση των δεδομένων από τις εφαρμογές, εάν δεν έχουν ορθή εικόνα των δεδομένων που αποθηκεύονται. Τα NoSQL συστήματα, λοιπόν, θεωρούνται μεν συστήματα που δεν απαιτούν τη χρήση ενός προκαθορισμένου σχήματος, ωστόσο προαιρετικά τα περισσότερα από αυτά μπορούν να υποστηρίξουν τη χρήση προκαθορισμένου σχήματος δεδομένων, αλλά και τον προσδιορισμό περιορισμών που αποσκοπούν στον έλεγχο της εγκυρότητας των αποθηκευμένων δεδομένων.

Εφόσον τα NoSQL συστήματα δεν απαιτούν την ύπαρξη σχήματος, δεν υπάρχει ο αντίστοιχος διαχωρισμός μεταξύ σχήματος και δεδομένων που συναντάμε σε μια σχεσιακή βάση δεδομένων. Αντιθέτως, τα δεδομένα συνήθως είναι *αυτοπεριγραφικά*, όπως ορίζεται και στο μοντέλο των ημιδομημένων δεδομένων. Έτσι, κάθε «κομμάτι» πληροφορίας έρχεται μαζί με την περιγραφή του. Για παράδειγμα, η τιμή «Μαρία» περιγράφεται από το κλειδί «όνομα», δημιουργώντας ένα ζεύγος κλειδιού-τιμής (*key-value*) «όνομα: Μαρία» που αποτελεί ένα από τα μοντέλα ημιδομημένων δεδομένων, αλλά αντιστοιχεί και στο μοντέλο δεδομένων μίας από τις κατηγορίες των NoSQL συστημάτων, των *key-value stores*.

4.1.1.2 Ενσωμάτωση δεδομένων

Κατά τη σχεδίαση του λογικού μοντέλου για μια σχεσιακή βάση δεδομένων, δώσαμε ιδιαίτερη έμφαση στην κανονικοποίηση για την αποφυγή επανάληψης πληροφορίας. Τα NoSQL



Σχήμα 4.1: Παράδειγμα διάσπασης πίνακα.

```
{id:101, fullname: "John Smith", phone: ["6775552361"]}  
{id:102, fullname: "Mary Papadopoulou", phone: ["6775633342", "6987111222"]}  
{id:103, fullname: "Paul Smith", phone: ["6774299999", "6877111111"]}  
{id:104, fullname: "Ann Jones"}
```

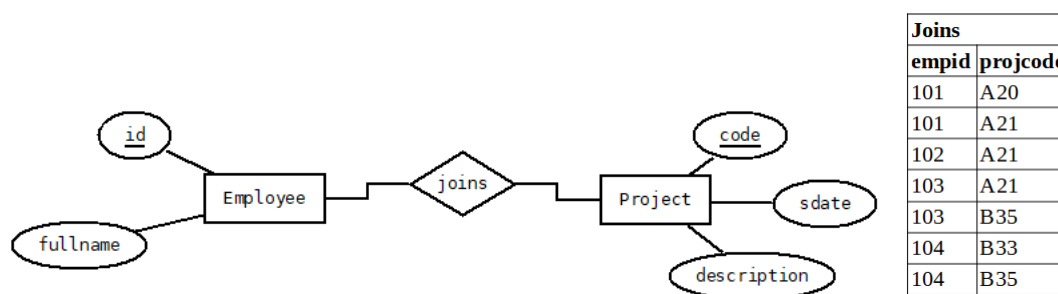
Σχήμα 4.2: Παράδειγμα απλής ενσωμάτωσης τιμών.

συστήματα δεν απαιτούν κανονικοποίηση. Αντιθέτως, ο σχεδιασμός τους υποστηρίζει την επανάληψη πληροφορίας, για να πετύχει καλύτερη απόδοση στην επεξεργασία ερωτημάτων στη βάση. Αντί για κανονικοποίηση, στα NoSQL συστήματα θα συναντήσουμε τις έννοιες της *συνάθροισης* (aggregation) ή *ενσωμάτωσης* (embeddedness).

Ας δούμε ένα απλό παράδειγμα για να εξηγήσουμε τις έννοιες αυτές και πού αποσκοπούν. Έστω ότι για τους εργαζόμενους που αναφέραμε και στην προηγούμενη ενότητα θέλουμε να διατηρούμε τα τηλέφωνα τους. Επειδή ένας εργαζόμενος πιθανόν έχει πολλούς αριθμούς τηλεφώνου, σε μια σχεσιακή βάση δεδομένων δεν θα μπορούσε ο πίνακας που αντιστοιχεί στον εργαζόμενο να έχει ένα πεδίο «τηλέφωνο», αφού σύμφωνα με την 1η κανονική μορφή κάθε πεδίο ενός πίνακα περιέχει ατομικές τιμές. Επομένως, θα οδηγούμασταν στη διάσπαση του πίνακα που αφορά τον εργαζόμενο σε δύο πίνακες, όπως φαίνεται στο Σχήμα 4.1.

Συνέπεια αυτής της διάσπασης, ωστόσο, είναι ότι για να ανακτήσει κανείς τα τηλέφωνα ενός εργαζομένου θα πρέπει να εφαρμοστεί ένα *ερώτημα σύζευξης* (join) μεταξύ των πινάκων E1 και E2. Για να αποφευχθεί το κόστος της σύζευξης και να επιτευχθεί ταχύτερη ανάκτηση των δεδομένων, η εναλλακτική προσέγγιση θα μπορούσε να αποθηκεύει όλα τα τηλέφωνα που διαθέτει ένας εργαζόμενος ως μια λίστα. Συγκεκριμένα, το πεδίο, εδώ το τηλέφωνο, θα έχει ως τιμή μια λίστα από αντικείμενα, τα οποία στο συγκεκριμένο παράδειγμα θα είναι απλοί τηλεφωνικοί αριθμοί (Σχήμα 4.2). Έτσι, όταν ανακτούμε έναν εργαζόμενο, θα μπορούμε να ανακτούμε και όλα του τα τηλέφωνα.

Ας σκεφτούμε ένα πιο πολύπλοκο παράδειγμα. Έστω ότι κάθε εργαζόμενος απασχολείται



Σχήμα 4.3: Πολλά-προς-πολλά συσχέτιση (αριστερά) στο μοντέλο οντοτήτων συσχετίσεων και (δεξιά) μετατροπή στο σχεσιακό μοντέλο.

σε κάποια έργα (projects), για τα οποία διατηρείται πληροφορία όπως ο κωδικός τους, μια περιγραφή, ημερομηνία έναρξης και εκτιμώμενη διάρκεια. Αν θεωρήσουμε ότι ένας εργαζόμενος συμμετέχει σε πολλά έργα και σε ένα έργο συμμετέχουν πολλοί εργαζόμενοι, τότε στο μοντέλο οντοτήτων/συσχετίσεων προκύπτει μια πολλά-προς-πολλά συσχέτιση (Σχήμα 4.3, αριστερά), η οποία στο σχεσιακό μοντέλο θα μετατραπεί σε έναν νέο πίνακα (Σχήμα 4.3, δεξιά). Και πάλι, για την ανάκτηση των στοιχείων π.χ. της Mary αλλά και των έργων στα οποία απασχολείται, θα πρέπει να εκτελέσουμε σύζευξη μεταξύ των πινάκων του «Εργαζομένου» (Employee) και «συμμετέχει» (joins). Αν μάλιστα θέλουμε και τις αναλυτικές πληροφορίες για κάθε έργο, τότε θα πρέπει το αποτέλεσμα της πρώτης σύζευξης να συζευχθεί με τον πίνακα «Έργο» (Project), στον οποίο αποθηκεύονται τα σχετικά στοιχεία των έργων.

Μια εναλλακτική προσέγγιση για να αποφύγουμε τις συζεύξεις είναι να ενσωματώσουμε τις πληροφορίες για τα έργα στα οποία απασχολείται ο κάθε εργαζόμενος στις ιδιότητες του εργαζομένου. Έτσι, χρησιμοποιώντας την ιδέα που είχαμε και για την αποθήκευση των πολλαπλών τηλεφωνικών αριθμών του, ο εργαζόμενος θα αποκτήσει μια ιδιότητα «projects», η οποία θα έχει ως τιμή μια λίστα με τους κωδικούς των έργων στα οποία απασχολείται. Επομένως, η ανάκτηση των κωδικών θα γίνεται απευθείας με την ανάκτηση των στοιχείων του εργαζομένου.

Μπορούμε να επεκτείνουμε και περισσότερο αυτήν την ιδέα, ενσωματώνοντας όχι μόνο τον κωδικό του έργου στα στοιχεία του εργαζομένου, αλλά όλα τα στοιχεία κάθε έργου στα οποία απασχολείται. Πλέον, το πεδίο «projects» του εργαζομένου θα αποτελείται από μια λίστα σύνθετων στοιχείων (αντικειμένων), που το καθένα θα περιγράφει ένα από τα έργα στα οποία απασχολείται, όπως φαίνεται στο Σχήμα 4.4. Η μοντελοποίηση αυτή ενσωματώνει (embeds) την περιγραφή ενός αντικειμένου (οντότητας έργου) εντός μιας άλλης (της οντότητας εργαζομένου) ή αλλιώς συγκεντρώνει/συναθροίζει (aggregates) τις περιγραφές των έργων εντός της περιγραφής του εργαζομένου.

Από τη μία πλευρά, αυτό έχει ως αποτέλεσμα το επιθυμητό, η ανάκτηση όλων των στοιχείων των έργων ενός εργαζομένου να γίνεται απευθείας, χωρίς να χρειάζεται να συνδυαστούν και να ανακτηθούν και άλλα δεδομένα, άρα η ανάκτηση γίνεται πολύ πιο αποδοτικά. Από την άλλη, η ενσωμάτωση αυτή προκαλεί μεγάλη επανάληψη πληροφορίας, διότι τα χαρακτηριστικά ενός έργου θα αποθηκεύονται πολλές φορές, δηλαδή για κάθε εργαζόμενο ο οποίος εργάζεται σε ένα έργο.

Για να κατανοήσουμε το γιατί η επιλογή της ενσωμάτωσης και, άρα, επανάληψης πληροφορίας που δεν συστήνεται για τις σχεσιακές βάσεις δεδομένων είναι η προτιμητέα στα

```
{id:101, fullname: "John Smith", projects:[
    {code: A20, type: "R&D", description: "An interesting project"},
    {code: A21, type: "R&D", description: "A more interesting project"}]}
{id:102, fullname: "Mary Papadopoulou", projects:[
    {code: A21, type: "R&D", description: "A more interesting project"}]}
{id:103, fullname: "Paul Smith", projects:[
    {code: A21, type: "R&D", description: "A more interesting project"},
    {code: B35, type: "Management", description: "Another project"}]}
...
```

Σχήμα 4.4: Παράδειγμα ενσωμάτωσης των έργων στον εργαζόμενο.

NoSQL συστήματα, θα πρέπει να θυμηθούμε και πάλι τις απαιτήσεις που μας οδήγησαν στην ανάγκη σχεδίασης νέων συστημάτων διαχείρισης δεδομένων. Θεωρούμε, λοιπόν, ότι οι εφαρμογές μας θα διαχειρίζονται δεδομένα κατανεμημένα σε πολλαπλές μηχανές στο διαδίκτυο ή και στο υπολογιστικό νέφος. Συνεπώς, ο συνδυασμός δεδομένων για διαφορετικές οντότητες πιθανόν απαιτεί την ανάκτηση δεδομένων από πολλαπλές απομακρυσμένες βάσεις, αυξάνοντας σημαντικά το κόστος επεξεργασίας του ερωτήματος και τον χρόνο απόκρισης για τον χρήστη του συστήματος. Σε ένα τέτοιο σενάριο, η επιλογή να «θυσιάσουμε» τον επιπλέον χώρο αποθήκευσης για να πετύχουμε καλύτερους χρόνους απόκρισης στα ερωτήματα είναι συχνά αποδεκτή. Εάν συνυπολογίσουμε το ότι έτσι κι αλλιώς τα συστήματα αυτά δεν απαιτούν αυστηρή συνέπεια, στα οποία η επανάληψη πληροφορίας θα προκαλούσε προβλήματα και καθυστερήσεις κατά την ενημέρωση και τη διαγραφή, η συνάθροιση ή ενσωμάτωση δεδομένων αποτελεί τη δημοφιλέστερη επιλογή μοντελοποίησης σε NoSQL συστήματα.

Βέβαια, εδώ προκύπτει άλλη μία απόφαση που καλείται να πάρει ο σχεδιαστής. Εάν σκεφτούμε το παράδειγμά μας, ο σχεδιαστής καλείται να αποφασίσει για τα ακόλουθα:

1. Θα ενσωματώσει τα έργα στα δεδομένα κάθε εργαζομένου που απασχολείται σε αυτά (Σχήμα 4.4);
2. Θα ενσωματώσει τους εργαζομένους στα δεδομένα κάθε έργου στα οποία απασχολούνται (Σχήμα 4.5);
3. Θα ενσωματώσει και τα έργα στα δεδομένα κάθε εργαζομένου, αλλά και τους εργαζομένους στα στοιχεία κάθε έργου;

Το κίνητρο που μας οδήγησε στο να ενσωματώσουμε αρχικά τα στοιχεία των έργων στα στοιχεία του εργαζομένου, δηλαδή να ακολουθήσουμε την επιλογή (1), ήταν το ερώτημα «θέλω για τη Mary τα στοιχεία της και όλα τα στοιχεία των έργων στα οποία απασχολείται η Mary». Εάν θεωρούσαμε το ερώτημα «θέλω για το έργο με κωδικό A21 τα στοιχεία του και όλα τα στοιχεία των εργαζομένων που απασχολούνται σε αυτό», θα οδηγούμασταν στην επιλογή (2). Ενώ εάν θέλαμε να υποστηρίξει το σύστημά μας εξίσου και τις δύο επιλογές, τότε πιθανότατα θα υιοθετούσαμε την επιλογή (3), δηλαδή οι εργαζόμενοι θα έχουν τη μορφή που φαίνεται στο Σχήμα 4.4, και ταυτόχρονα τα έργα θα έχουν τη μορφή που φαίνεται στο Σχήμα 4.5.

Είναι φανερό ότι τα ερωτήματα που αναμένουμε στον φόρτο του συστήματος είναι αυτά που θα πρέπει να κατευθύνουν τις επιλογές, όταν η σχεδίαση αφορά την ενσωμάτωση σε ένα NoSQL σύστημα. Η βελτιστοποίηση των ερωτημάτων που έρχονται με μεγάλη συχνότητα στο


```
{code: A20, type: "R&D", description: "An interesting project", joined:[
  {id:101, fullname: "John Smith"}]}
{code: A21, type: "R&D", description: "A more interesting project", joined:[
  {id:101, fullname: "John Smith"},
  {id:102, fullname: "Mary Papadopoulou"},
  {id:103, fullname: "Paul Smith"}]}
{code: B35, type: "Management", description: "Another project", joined:[
  {id:103, fullname: "Paul Smith"},
  ...]}
...
```

Σχήμα 4.5: Παράδειγμα ενσωμάτωσης των εργαζομένων στο έργο

σύστημα θα καθορίσει τι είδους ενσωμάτωση απαιτείται. Βέβαια, η επιλογή (3) στο προηγούμενο παράδειγμα μοιάζει να αποτελεί την καλύτερη επιλογή από αυτήν τη σκοπιά, αφού είναι αποδοτικότερη για κάθε τέτοιο ερώτημα και, άρα, δεν χρειάζεται να γνωρίζουμε εκ των προτέρων τα αναμενόμενα ερωτήματα. Ωστόσο, αν τα δεδομένα που αφορούν μια οντότητα μεγαλώσουν υπερβολικά, με αποτέλεσμα να μην μπορούμε να τα χειριστούμε αποδοτικά στη μνήμη ενός μηχανήματος λόγω της αλόγιστης χρήσης της τεχνικής της ενσωμάτωσης, τότε θα αναγκαστεί το σύστημα να διαμερίσει τα δεδομένα αυτών των οντοτήτων και πλέον θα χαθεί το πλεονέκτημα που προσπαθούμε να επιτύχουμε, διότι θα απαιτείται να συνδυαστεί και πάλι πληροφορία από διαφορετικές προελεύσεις για την επεξεργασία των ερωτημάτων.

Συνεπώς, στις NoSQL βάσεις δεδομένων, σε αντίθεση με τις σχεσιακές βάσεις, χρησιμοποιούμε την ενσωμάτωση κατά τη μοντελοποίηση, ωστόσο αυτή πρέπει να χρησιμοποιείται με μέτρο. Επιπλέον, κατά την ανάλυση απαιτήσεων, εκτός από την καταγραφή του είδους των πληροφοριών που θα πρέπει να διαχειρίζεται το σύστημα και των σχέσεων μεταξύ τους, θα πρέπει επίσης ο σχεδιαστής να αναλύει το είδος των ερωτημάτων που αναμένεται να ικανοποιεί το σύστημα, έτσι ώστε η μοντελοποίηση να κατευθυνθεί προς τη βελτιστοποίηση των ερωτημάτων που θα απαντώνται συχνότερα στο σύστημα.

4.1.2 Διαμέριση και αναπαραγωγή δεδομένων

Με τους όγκους των μεγάλων δεδομένων, όπως είπαμε, μια μονολιθική βάση δεδομένων δεν μπορεί να υποστηρίξει αποδοτικά τις σύγχρονες εφαρμογές. Συνεπώς, υπάρχει ανάγκη για τη χρήση κατανεμημένων λύσεων που θα μοιράσουν τον φόρτο του συστήματος σε διαφορετικές μηχανές για να επιτευχθεί η *εξισορρόπηση φόρτου (load balance)*. Παράλληλα, η απαίτηση για υψηλή διαθεσιμότητα απαιτεί την αναπαραγωγή των δεδομένων σε διαφορετικούς διακομιστές, με επιπλέον στόχο και πάλι την εξισορρόπηση του φόρτου.

4.1.2.1 Sharding

Η τεχνική που χωρίζει μια λογική βάση δεδομένων σε πολλαπλά μέρη ονομάζεται *διαμέριση (partitioning)*. Ο όρος είναι γενικός και, αν και συνήθως αναφέρεται στην κατανομή μιας λογικής βάσης δεδομένων σε πολλαπλούς διακομιστές, δηλαδή διαφορετικές μηχανές, μπορεί να τον συναντήσουμε και σε άλλες περιπτώσεις. Η διαμέριση μιας βάσης μπορεί να είναι

είτε *οριζόντια διαμέριση* (*horizontal partitioning*), στην οποία οι γραμμές ενός πίνακα διαμερίζονται σε διαφορετικούς διακομιστές, είτε *κάθετη διαμέριση* (*vertical partitioning*), όπου οι στήλες ενός πίνακα ομαδοποιούνται (μαζί με τη στήλη του κλειδιού) και διαμερίζονται σε διαφορετικούς διακομιστές.

Ειδικότερα, στα NoSQL συστήματα χρησιμοποιείται η τεχνική της οριζόντιας διαμέρισης σε διαφορετικούς φυσικούς διακομιστές, που έχει επικρατήσει με τον όρο *sharding*. Το *sharding* διαμοιράζει έναν μεγάλο πίνακα σε υποσύνολα γραμμών που μοιράζονται σε πολλαπλούς φυσικούς διακομιστές. Το σχήμα του πίνακα, αν αναφερόμαστε σε σχεσιακές βάσεις, σε αυτήν την περίπτωση επαναλαμβάνεται μαζί με κάθε *θραύσμα* (*shard*) του διαμερισμένου πίνακα. Ο τρόπος που τα δεδομένα μοιράζονται σε διαφορετικούς διακομιστές εξαρτάται από την τιμή ενός κλειδιού, του *κλειδιού διαμέρισης*, όπως θα το αναφέρουμε. Το κλειδί αυτό μπορεί να αναφέρεται σε ένα γνώρισμα του πίνακα, π.χ. το αναγνωριστικό ενός εργαζομένου, ή και σε συνδυασμό γνωρισμάτων.

Μπορούμε να διακρίνουμε διαφορετικά είδη *sharding*, ανάλογα με τον τρόπο που μοιράζονται τα δεδομένα στους πολλαπλούς διακομιστές και τη μέθοδο που χρησιμοποιείται για την αναζήτηση και ανάκτησή τους, όταν απαιτούνται από κάποια εφαρμογή.

Το *sharding βασισμένο σε κλειδί* (*key-based* ή *hash-based*) βασίζεται στην τεχνική του κατακερματισμού. Μια συνάρτηση εφαρμόζεται στο κλειδί κατακερματισμού (το οποίο αποτελεί και το κλειδί διαμέρισης) και το αποτέλεσμά της καθορίζει το θραύσμα στο οποίο θα τοποθετηθεί κάθε γραμμή των δεδομένων. Η ίδια συνάρτηση κατακερματισμού χρησιμοποιείται για να εντοπίσουμε το θραύσμα στο οποίο θα πρέπει να αναζητήσουμε κάθε γραμμή δεδομένων για την ανάκτησή τους, χωρίς να απαιτείται η διατήρηση κάποιου καταλόγου. Το κλειδί διαμερισμού σε αυτήν την περίπτωση συνιστάται να είναι στατικό, διότι αλλαγές στην τιμή του θα απαιτούσαν τη μετακίνηση των δεδομένων μεταξύ θραυσμάτων. Με την καλή επιλογή του κλειδιού διαμέρισης, μια καλή συνάρτηση κατακερματισμού θα πετύχει την ισοκατανομή των δεδομένων στους διαφορετικούς διακομιστές και, άρα, και την εξισορρόπηση του φόρτου μεταξύ τους.

Το *δυναμικό sharding* ή το *sharding που βασίζεται σε εύρος τιμών* (*range-based*) χωρίζει τα δεδομένα ανάλογα με το εύρος της τιμής του κλειδιού διαμέρισης. Για παράδειγμα, μπορούμε να χωρίσουμε τους εργαζομένους με κλειδί διαμερισμού τον μισθό τους. Προκαθορίζοντας διαστήματα τιμών που αντιστοιχούν σε κάθε θραύσμα, μια γραμμή τοποθετείται στο θραύσμα του διαστήματος στο οποίο ανήκει ο μισθός του εργαζομένου που περιγράφει. Για την υλοποίηση αυτής της τεχνικής απαιτείται ένας κατάλογος που διατηρεί την αντιστοίχιση κάθε θραύσματος με το εύρος τιμών του κλειδιού διαμέρισης που θα αντιστοιχιστούν σε αυτό, έτσι ώστε να είναι δυνατή η αναζήτηση και ανάκτηση των δεδομένων που απαιτούνται από μια εφαρμογή. Για την καλύτερη εξισορρόπηση, το κλειδί διαμέρισης σε αυτήν την περίπτωση πρέπει να επιλεγεί έτσι, ώστε να περιέχει πολλές διαφορετικές τιμές και αυτές οι τιμές να είναι κατανομημένες κατά το δυνατό ομοιόμορφα στα δεδομένα. Με αυτόν τον τρόπο, θα πετύχουμε και πάλι την ισοκατανομή των δεδομένων σε διαφορετικούς διακομιστές.

Το *sharding με χρήση καταλόγου* (*directory-based*) ακολουθεί την ίδια τεχνική με το *sharding* που βασίζεται σε εύρη τιμών, αλλά, αντί να διαμοιράζουμε με βάση εύρη της τιμής του κλειδιού διαμέρισης, αντιστοιχίζουμε ρητά τις τιμές του κλειδιού με ένα θραύσμα σε έναν κατάλογο που διατηρείται για αυτόν τον σκοπό. Η μέθοδος αυτή προσφέρει ευελιξία, αφού η

αντιστοίχιση των τιμών του κλειδιού με τα θραύσματα μπορεί να γίνει με οποιονδήποτε αλγόριθμο είναι επιθυμητό και, επίσης, δεν απαιτείται το κλειδί διαμέρισης να έχει τις ιδιότητες που περιγράψαμε στο sharding με βάση το εύρος τιμών. Ωστόσο, η τεχνική αυτή απαιτεί κατά την αναζήτηση δεδομένων πρώτα να γίνεται αναζήτηση στον σχετικό κατάλογο, και αυτό επιφέρει ένα επιπλέον κόστος. Παράλληλα, απαιτείται και η διατήρηση αυτού του καταλόγου με το δικό της επιπλέον κόστος για ενημερώσεις.

Οι αλγόριθμοι που μπορούμε να χρησιμοποιήσουμε για τη διαμέριση στην τελευταία περίπτωση μπορεί, για παράδειγμα, να βασίζονται σε γεωγραφικά κριτήρια. Αν έχουμε δεδομένα που αφορούν διαφορετικές γεωγραφικές περιοχές, π.χ. εργαζομένους σε διαφορετικές πόλεις ή χώρες, μπορούμε να διαμερίσουμε τα δεδομένα τους αντιστοιχίζοντας ένα θραύσμα σε κάθε πόλη ή χώρα. Επίσης, κανείς μπορεί να βασιστεί και στις σχέσεις των δεδομένων και να προσπαθήσει να τοποθετήσει συσχετιζόμενα δεδομένα στο ίδιο θραύσμα, έτσι ώστε να βελτιώσει την απόδοση κατά την ανάκτησή τους.

4.1.2.2 Αναπαραγωγή δεδομένων

Η τεχνική της *αναπαραγωγής* ή *αντιγραφής* (*replication*) των δεδομένων σε διαφορετικές μηχανές χρησιμοποιείται εναλλακτικά ή και συμπληρωματικά μαζί με το sharding για να πετύχει την εξισορρόπηση του φόρτου μεταξύ διαφορετικών διακομιστών, αλλά και να βελτιώσει τη διαθεσιμότητα των δεδομένων ακόμα και σε περίπτωση σφάλματος σε κάποιον ή κάποιους από τους διακομιστές. Στον αντίποδα, το κόστος που επιφέρει η χρήση αναπαραγωγής δεδομένων συνίσταται στη διατήρηση της συνέπειας μεταξύ των διαφορετικών αντιγράφων των δεδομένων. Οι ενημερώσεις σε ένα αντίγραφο θα πρέπει να διαδίδονται και στα υπόλοιπα αντίγραφα ή, με άλλα λόγια, τα αντίγραφα θα πρέπει να συγχρονίζονται. Διαφορετικές τεχνικές χρησιμοποιούνται για τον συγχρονισμό των αντιγράφων και θα συζητήσουμε σχετικά στα επόμενα κεφάλαια.

Μπορούμε να διακρίνουμε μεταξύ *πλήρους αναπαραγωγής* (*full replication*) και *μερικής αναπαραγωγής* (*partial replication*). Κατά την πλήρη αναπαραγωγή αντιγράφονται όλα τα δεδομένα σε όλους τους διακομιστές. Με αυτήν την τεχνική επιτυγχάνεται η υψηλότερη διαθεσιμότητα, αφού οποιουδήποτε διακομιστής μπορεί να χρησιμοποιηθεί για την επεξεργασία οποιουδήποτε ερωτήματος. Η απόδοση του συστήματος συνολικά, επίσης, βελτιώνεται με την καλύτερη δυνατή εξισορρόπηση του φόρτου μεταξύ των διακομιστών. Ωστόσο, η πλήρης αναπαραγωγή επιφέρει και το μεγαλύτερο κόστος συγχρονισμού. Κάθε ενημέρωση θα πρέπει να διαδίδεται σε όλα τα αντίγραφα. Η μερική αναπαραγωγή χρησιμοποιείται σε μια προσπάθεια εξορθολογισμού του κέρδους και του κόστους της αναπαραγωγής. Κάποια δεδομένα αναπαράγονται σε ένα υποσύνολο ή και στο σύνολο των διακομιστών. Η επιλογή των δεδομένων που θα αναπαραχθούν εξαρτάται από το πόσο συχνά ανακτώνται αυτά τα δεδομένα ή πόσο κρίσιμος είναι ο χρόνος απόκρισης κατά την ανάκτησή τους.

4.1.3 Συνέπεια δεδομένων

Στις σχεσιακές βάσεις δεδομένων οι συναλλαγές που χρησιμοποιούνται για να εξυπηρετήσουν πολλαπλούς χρήστες της βάσης δεδομένων, χωρίς να δημιουργούνται προβλήματα στη

συνέπεια και την ακεραιότητα των δεδομένων, ικανοποιούν τις λεγόμενες *ACID* ιδιότητες:

- **Ατομικότητα (Atomicity).** Οι συναλλαγές εκτελούνται είτε εξ ολοκλήρου είτε καθόλου.
- **Συνέπεια (Consistency).** Μια συναλλαγή οδηγεί τη βάση δεδομένων πάντα από μια συνεπή κατάσταση σε μια άλλη συνεπή κατάσταση.
- **Απομόνωση (Isolation).** Οι συναλλαγές που εκτελούνται ταυτόχρονα δεν επηρεάζουν η μία την εκτέλεση της άλλης.
- **Διάρκεια (Durability).** Αν μια συναλλαγή ολοκληρωθεί, τότε οι αλλαγές που έχει κάνει στη βάση δεδομένων είναι μόνιμες.

Η εγγύηση αυτών των ιδιοτήτων επιβάλλει τη χρήση μηχανισμών ελέγχου του ταυτοχρονισμού, είτε μέσω της χρήσης *κλειδώματος (locking)* των δεδομένων για να αποτρέψουν την ταυτόχρονη χρήση από πολλαπλές συναλλαγές, επιβάλλοντας στις συναλλαγές να περιμένουν τότε τα δεδομένα που ζητούν θα γίνουν διαθέσιμα, είτε με τη χρήση αισιόδοξων τεχνικών που επιτρέπει την εκτέλεση όλων των συναλλαγών και μετά ελέγχει πόσες από αυτές θα χρειαστεί να ακυρωθούν αν παραβιάζουν τις *ACID* ιδιότητες.

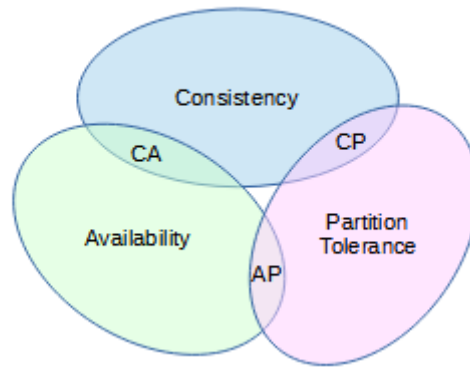
Όπως αναφέραμε και στην εισαγωγική περιγραφή των NoSQL συστημάτων και των στόχων που έχουν, σε αντίθεση με τα σχεσιακά συστήματα διαχείρισης δεδομένων, η αυστηρή συνέπεια των δεδομένων δεν αποτελεί συνήθως προτεραιότητά τους. Αντιθέτως, η υψηλή διαθεσιμότητα αλλά και η ανοχή σε διαμερίσεις του δικτύου που θα απέτρεπαν την πρόσβαση σε κάποιους διακομιστές αποκτούν ιδιαίτερη σημασία. Ο Eric Brewer, το 2000, διατύπωσε την υπόθεση ότι οι τρεις ιδιότητες της συνέπειας, διαθεσιμότητας και ανοχής στις διαμερίσεις του δικτύου δεν μπορούν να συνυπάρχουν σε ένα κατακεντρωμένο σύστημα μεγάλης κλίμακας. Αργότερα, η υπόθεση αυτή αποδείχτηκε από τους Gilbert και Lynch [GL02], συνιστώντας το γνωστό και ως θεώρημα *CAP (Consistency, Availability, Partition Tolerance)*.

Ορισμός 4.1 (Θεώρημα CAP). Σε οποιοδήποτε κατακεντρωμένο σύστημα διαχείρισης δεδομένων μεγάλης κλίμακας, μόνο δύο ιδιότητες από τις τρεις, συνέπεια, διαθεσιμότητα και ανοχή σε διαμερίσεις, μπορούν να εξασφαλιστούν ταυτόχρονα.

Ειδικότερα, οι τρεις ιδιότητες ορίζονται ως:

- **Συνέπεια (Consistency).** Τα δεδομένα που διαβάσει οποιαδήποτε συναλλαγή σε ένα κατακεντρωμένο σύστημα με αναπαραγωγή δεδομένων βρίσκονται στην έγκυρη τρέχουσα κατάσταση, ανεξάρτητα από το σε ποιον κόμβο πραγματοποιείται η ανάγνωση.
- **Διαθεσιμότητα (Availability).** Οι εφαρμογές δεν αποτυγχάνουν ποτέ και έχουν λογικούς χρόνους απόκρισης.
- **Ανοχή στη διαμέριση (Partition tolerance).** Η λειτουργία του συστήματος δεν θα πρέπει να επηρεάζεται από την αποτυχία συγκεκριμένων διακομιστών ή της επικοινωνίας μεταξύ τους.

Σύμφωνα με το θεώρημα, κάθε σύστημα πρέπει να επιλέξει δύο από τις τρεις ιδιότητες (Σχήμα 4.6), και έτσι μπορούμε να χωρίσουμε τα συστήματα αναλόγως σε:



Σχήμα 4.6: Διάγραμμα του θεωρήματος CAP.

1. συστήματα που επιλέγουν συνέπεια και διαθεσιμότητα (CA),
2. συστήματα που επιλέγουν συνέπεια και ανοχή στις διαμερίσεις του δικτύου (CP), και
3. συστήματα που επιλέγουν διαθεσιμότητα και ανοχή σε διαμερίσεις δικτύου (AP).

Η πρώτη κατηγορία (CA) αντιστοιχεί στα σχεσιακά συστήματα διαχείρισης δεδομένων που υποστηρίζουν συναλλαγές με ACID ιδιότητες. Οι άλλες δύο επιλογές είναι συνηθέστερες στα NoSQL συστήματα που υποστηρίζουν τόσο τη διαμέριση όσο και την αναπαραγωγή δεδομένων.

Ο ίδιος ο Brewer επέστρεψε στο θεώρημά του, ωστόσο, και εξήγησε το πώς αυτή η επιλογή των δύο από τις τρεις ιδιότητες είναι ιδιαίτερα περιοριστική και απλοποιεί τις σχέσεις μεταξύ των τριών ιδιοτήτων [Bre12]. Στην πραγματικότητα, αυτό που συμβαίνει είναι ότι ένα σύστημα δεν μπορεί να πετύχει τέλεια διαθεσιμότητα και συνέπεια όταν συμβαίνουν διαμερίσεις στο δίκτυο.

Τα NoSQL συστήματα συνήθως θυσιάζουν την αυστηρή συνέπεια και υποστηρίζουν πιο χαλαρές μορφές συνέπειας, όπως την *τελική συνέπεια* (*eventual consistency*). Η μορφή αυτή της συνέπειας εγγυάται ότι, εάν δεν συμβεί καμία ενημέρωση στα δεδομένα μιας κατανεμημένης βάσης δεδομένων για κάποιον χρόνο, τότε όλες οι ενημερώσεις που είχαν γίνει θα διαδοθούν τελικά σε όλα τα κατανεμημένα αντίγραφα της.

Με τον ορισμό της τελικής συνέπειας, οδηγούμαστε σε ένα νέο είδος συστημάτων που υποστηρίζουν τις *BASE* ιδιότητες στις συναλλαγές τους:

- **Βασικά διαθέσιμο (Basically Available).** Το σύστημα θα πρέπει να επιστρέφει απάντηση σε οποιοδήποτε αίτημα, ακόμα κι αν αυτή η απάντηση είναι μήνυμα αποτυχίας.
- **Μαλακή κατάσταση (Soft State).** Το σύστημα μπορεί να αλλάζει καταστάσεις όταν και όσο δέχεται νέα δεδομένα.
- **Τελική συνέπεια (Eventually Consistent).** Μετά από μη φραγμένη καθυστέρηση, οι αλλαγές των δεδομένων θα διαδοθούν σε όλα τα αντίγραφα του συστήματος.

Σύμφωνα με τις BASE ιδιότητες, οι κόμβοι στην κατακευματισμένη βάση μπορούν να διατηρούν αντικρουόμενα δεδομένα για κάποιο χρονικό διάστημα, τα οποία τελικά θα ενημερωθούν με κάποια καθυστέρηση. Εφόσον πολλές εφαρμογές μεγάλων δεδομένων υποστηρίζουν πολλά περισσότερα ερωτήματα ανάγνωσης σε σχέση με τα αντίστοιχα ερωτήματα ενημέρωσης, αλλά και οι απαιτήσεις συνέπειας δεν είναι τόσο αυστηρές, αυτό το BASE μοντέλο συναλλαγών μπορεί με επιτυχία να χειριστεί τις συναλλαγές στις NoSQL βάσεις δεδομένων, αν και αυξάνει την προγραμματιστική πολυπλοκότητα των εφαρμογών, μεταφέροντάς την από το σύστημα διαχείρισης βάσεων δεδομένων στην ίδια την εφαρμογή. Ένας προγραμματιστής πλέον πρέπει να αναπτύσσει εφαρμογές που πιθανό να χρειαστεί να χειριστούν παρωχημένα ή και αντικρουόμενα δεδομένα.

4.2 Key-value stores

Τα *key-value stores* αποτελούν τα συστήματα που χρησιμοποιούν το απλούστερο μοντέλο για τα δεδομένα τους, αυτό που περιγράψαμε ήδη για τα ημιδομημένα δεδομένα και αποτελείται από ζεύγη κλειδίων-τιμής. Ένα key-value store αποθηκεύει συλλογές από τέτοια ζεύγη κλειδίων-τιμής. Χάρη στην απλότητα αυτή, τα key-value stores αποτελούν μια πολύ ελκυστική επιλογή και έχουν πολλούς εκπροσώπους.

Οι τιμές στα ζεύγη μπορούν να είναι αντικείμενα οποιουδήποτε τύπου, από απλά αλφαριθμητικά έως άλλα σύνθετα αντικείμενα. Αυτά τα συστήματα, ουσιαστικά, είναι σχεδιασμένα ώστε να αγνοούν τον τύπο των δεδομένων, δηλαδή τα key-value stores αντιμετωπίζουν κάθε τιμή ως ένα δυαδικό αντικείμενο. Ωστόσο, πολλά από τα συστήματα παρέχουν επιπλέον υποστήριξη για κάποιους τύπους δεδομένων, όπως δευτερεύοντα ευρετήρια.

Αντίστοιχα βασική με το μοντέλο τους είναι και η λειτουργικότητα που παρέχουν. Οι βασικές εντολές για την προσπέλαση των δεδομένων είναι: SET για την αποθήκευση των δεδομένων, GET για την ανάκτησή τους και DEL για τη διαγραφή τους. Παρέχουν, δηλαδή, πολύ απλά APIs για τις βασικές *CRUD λειτουργίες* (*CREATE, READ, UPDATE, DELETE*) που υποστηρίζει κάθε σύστημα διαχείρισης βάσεων δεδομένων.

Για παράδειγμα, για την αποθήκευση ενός εργαζομένου και των γνωρισμάτων τους έχουμε:

```
1 SET Employee001:name Alexandros
2 SET Employee001:email alexandros@mycompany.com
3 SET Employee002:name Anastasios
4 SET Employee002:email anastasios@mycompany.com
5 ...
```

Ο παραπάνω κώδικας δημιουργεί δύο αντικείμενα, Employee001 και Employee002, που αντιστοιχούν σε δύο εργαζομένους, και δίνει τιμές σε δύο γνωρίσματα, το *name* και το *email*, που προσδιορίζονται από τα αντίστοιχα κλειδιά *Employee00x.name* και *Employee00x.email*.

Αντιστοίχως, για να ανακτήσουμε την τιμή ενός κλειδιού χρησιμοποιείται απλώς μια εντολή GET για το αντίστοιχο κλειδί:

```
1 GET Employee001:email
2 > alexandros@mycompany.com
```

Τέλος, για τη διαγραφή του email από τον δεύτερο εργαζόμενο έχουμε τη χρήση του DEL ως εξής:

```
1 DEL Employee002:email
2 > OK
```

Όπως φαίνεται από τα παραπάνω, όλες οι προσπελάσεις στα δεδομένα σε ένα key-value store γίνονται μέσω ενός μοναδικού κλειδιού.

Τα key-value stores δεν χρησιμοποιούν προκαθορισμένο σχήμα για τα δεδομένα, οπότε η προσθήκη κλειδιών μπορεί να γίνεται άμεσα με την εισαγωγή νέων δεδομένων, προσφέροντας μεγάλη ευελιξία σε αυτά τα συστήματα. Επιπλέον, τα συστήματα αυτά δεν υλοποιούν κανενός είδους περιορισμούς αναφορικής ακεραιότητας, γεγονός που επιταχύνει τη διαχείριση δεδομένων, αφού αποφεύγονται σχετικοί έλεγχοι. Ωστόσο, μπορεί να χρησιμοποιηθεί ένας απλός ιεραρχικός τρόπος ονοματοδοσίας για να πετύχουμε την οργάνωση των δεδομένων, όπως είδαμε και παραπάνω, χρησιμοποιώντας το κλειδί *Employee001:email* ώστε να αναφερθούμε στο email ενός εργαζομένου.

Ως αποτέλεσμα, τα συστήματα αυτά πετυχαίνουν πολύ εύκολη οριζόντια κλιμάκωση μέσω του sharding. Το hash-based sharding χρησιμοποιείται ως επί το πλείστον και συγκεκριμένα μια μορφή κατακερματισμού γνωστή ως *consistent hashing*. Στην τεχνική του consistent hashing, εκτός από τα δεδομένα, η ίδια συνάρτηση κατακερματισμού χρησιμοποιείται και για τη διευθυνσιοδότηση των κόμβων της συστάδας που φιλοξενεί τους διακομιστές στους οποίους διαμερίζονται τα δεδομένα. Συνήθως, σε αυτήν την τεχνική οι κόμβοι και τα δεδομένα αναπαρίστανται σε έναν ιδεατό δακτύλιο, σύμφωνα με τη διεύθυνση που τους αναθέτει η συνάρτηση κατακερματισμού. Τα δεδομένα ανατίθενται σε κόμβους σύμφωνα με τα αποτελέσματα της συνάρτησης κατακερματισμού στο κλειδί διαμέρισης, και συγκεκριμένα στον επόμενο κόμβο στον δακτύλιο, σύμφωνα με τη φορά του ρολογιού. Με την τεχνική του consistent hashing μπορούμε να πετύχουμε, πέρα από κλιμάκωση στα δεδομένα, και οριζόντια κλιμάκωση στο πλήθος των κόμβων της συστάδας, αφού δεν απαιτείται ο επανασχεδιασμός της τεχνικής κατακερματισμού, αλλά οι αλλαγές αντιμετωπίζονται με ενημερώσεις σε γειτονικούς μόνο κόμβους στον ιδεατό δακτύλιο.

Από τη μία, η ευελιξία και η απλότητα που χαρακτηρίζουν τα key-value stores τα καθιστούν μια πολύ ελκυστική επιλογή. Από την άλλη, η απουσία σχήματος μπορεί να προκαλέσει προβλήματα και επιβάρυνση στο κόστος διαχείρισής τους.

4.2.1 Παραδείγματα key-value stores

Τα key-value stores είναι πολύ δημοφιλή. Γνωστοί εκπρόσωποί τους συμπεριλαμβάνουν το Riak [Ria], το Memcached DB [Fit04], το Berkeley DB [Ber] και φυσικά το Amazon DynamoDB [Siv12]. Στη συνέχεια, θα περιγράψουμε πιο αναλυτικά το Riak.

Το Riak βασίζεται στις προδιαγραφές που ακολούθησε και το Amazon DynamoDB, που αποτέλεσε ένα από τα πρώτα NoSQL συστήματα. Το Riak επιτρέπει τον ορισμό δεδομένων ως: (α) δυαδικά αντικείμενα, (β) έγγραφα, όπως XML, JSON, ή κείμενο, (γ) προσαρμοσμένους τύπους δεδομένων, και (δ) έναν τύπο δεδομένων που ονομάζεται CRDT (convergent replicated data type) και συμπεριλαμβάνει χάρτες, σύνολα και μετρητές.

Πίνακας 4.1: Ορολογία Riak και σχεσιακών βάσεων δεδομένων.

Σχεσιακή ΒΔ	Riak
στιγμιότυπο βάσης	συστάδα
πίνακας	κάδος (bucket)
εγγραφή	ζεύγος κλειδιού-τιμής
πρωτεύον κλειδί	κλειδί

Για τους τύπους δεδομένων που ορίζει, πέρα από την πρόσβαση μέσω του κλειδιού, το Riak επιτρέπει και τον ορισμό δευτερευόντων ευρετηρίων, έτσι ώστε να γίνεται πιο αποδοτικά ο εντοπισμός των δεδομένων, ακόμα και όταν η αναζήτηση γίνεται όχι μέσα από το κλειδί τους, αλλά μέσα από τον προσδιορισμό κριτηρίων στις τιμές τους. Για παράδειγμα, θα θέλαμε να εντοπίσουμε όλους τους εργαζομένους που στο domain του email τους έχουν το «company.com».

Μια αντιστοίχιση της οργάνωσης των δεδομένων στο Riak σε σχέση με μια σχεσιακή βάση δεδομένων φαίνεται στον Πίνακα 4.1. Ο κάδος (bucket) αποτελεί, ουσιαστικά, έναν τρόπο διαμερισμού των κλειδιών σε λογικές μονάδες ή, με άλλα λόγια, σε έναν χώρο ονομάτων (namespace). Έτσι, για παράδειγμα, θα μπορούσαμε να ορίσουμε ένα bucket για όλα τα δεδομένα της εταιρείας μας, ή και να ορίσουμε διαφορετικά buckets για κάθε είδος δεδομένων, όπως ένα bucket για τους εργαζομένους και ένα για τα έργα τους.

```

1  Bucket bucket=client.fetchBucket(bucketName).execute();
2  DomainBucket<Employee> empBucket=DomainBucket.builder(bucket, Employee.
   class).build

```

Για την υλοποίηση των δευτερευόντων ευρετηρίων, το Riak συμπεριλαμβάνει το Solr, μια ανοικτού κώδικα μηχανή αναζήτησης κειμένου. Παράλληλα, για ημιδομημένα δεδομένα (XML και JSON), το Riak υποστηρίζει και αναζήτηση για συγκεκριμένα πεδία των εγγράφων που οι τιμές τους ικανοποιούν κάποια κριτήρια.

Το Riak περιλαμβάνει τρεις διανομές, το Riak KV, το key-value store που περιγράφουμε, το Riak TS σχεδιασμένο για δεδομένα χρονοσειρών, και το Riak CS που προσφέρει μια κατανεμημένη λύση στο υπολογιστικό νέφος.

Αφού κατεβάσουμε το Riak KV, η εγκατάστασή του στο Ubuntu γίνεται εύκολα με το apt από το packagecloud. Στη συνέχεια και αφού διαμορφωθεί έστω μία συστάδα με έναν τουλάχιστον κόμβο, μπορούμε να χρησιμοποιήσουμε το key-value store.

Σχετικά με τη συνέπεια, το Riak υποστηρίζει eventual consistency. Επίσης, για τη βελτίωση της διαθεσιμότητας χρησιμοποιείται και αναπαραγωγή σε περίπτωση που το σύστημά μας αποτελείται από πολλούς κόμβους/συστάδες. Για τη ρύθμιση των συγκρούσεων που συμβαίνουν κατά τις εγγραφές του ίδιου δεδομένου, είτε η πιο πρόσφατη εγγραφή κερδίζει τις υπόλοιπες είτε όλες οι τιμές που προκαλούν τη σύγκρουση επιστρέφονται στον πελάτη, ο οποίος θα πρέπει να ρυθμίσει τη σύγκρουση. Οι ρυθμίσεις σχετικά με τα επίπεδα συνέπειας των δεδομένων μπορούν να καθοριστούν κατά τη δημιουργία των buckets.

Χάρη στην απλότητα που το χαρακτηρίζει, το Riak, αλλά και τα key-value stores γενικά, είναι κατάλληλα για εφαρμογές όπως: αποθήκευση web sessions τα οποία ανακτώνται με ένα κλειδί, στοιχεία χρηστών και προτιμήσεις τους, αλλά και την υλοποίηση εφαρμογών καλα-

θιού αγοράς σε ιστότοπους ηλεκτρονικού εμπορίου. Από την άλλη, καλό είναι να αποφεύγονται όταν τα δεδομένα έχουν πολύπλοκες σχέσεις μεταξύ τους ή όταν η ανάκτηση θα πρέπει να υποστηρίζει πιο εκφραστικά ερωτήματα από απλές GET αιτήσεις.

4.3 Column-family stores

Τα *column-family stores* χτίζουν πάνω στο μοντέλο των key-value stores παρέχοντας επιπλέον δομή στα δεδομένα, μέσω της οργάνωσής τους σε ομάδες στηλών (*column families*).

Στα σχεσιακά συστήματα διαχείρισης βάσεων δεδομένων η προσπέλαση στα δεδομένα γίνεται κατά γραμμές, και με τον ίδιο τρόπο οργανώνεται και η αποθήκευση των δεδομένων ενός πίνακα στον σκληρό δίσκο, ώστε να πετύχουμε την αποδοτική ανάκτησή τους. Αντιθέτως, στα *column-family stores* η προσπέλαση στα δεδομένα γίνεται κατά στήλη, τεχνική που στην πράξη έχει αποδειχτεί πιο αποδοτική. Η βασική ιδέα εδώ είναι ότι σπάνια σε ένα ερώτημα απαιτούμε όλα τα γνωρίσματα (στήλες) ενός πίνακα, αλλά ομάδες από στήλες ζητούνται συχνά μαζί. Συνεπώς, για να βελτιώσουμε την απόδοση κατά την ανάκτηση αυτών των ομάδων από στήλες, πρέπει να οργανώσουμε και την αποθήκευσή τους με αντίστοιχο τρόπο, κατά στήλες και όχι κατά γραμμές. Ομάδες στηλών αποθηκεύονται συνεχόμενα και στον δίσκο και αποτελούν τη δομική μονάδα προσπέλασης για τα δεδομένα μας.

Κάθε εγγραφή (γραμμή) σε ένα *column-family store* έχει το κλειδί της, αλλά και κάθε στήλη μπορεί να προσπελαστεί με το κλειδί της. Πολλές φορές, κάθε ζεύγος κλειδιού-τιμής δεδομένων έχει και μια *χρονοσφραγίδα* (*timestamp*). Οι χρονοσφραγίδες προσφέρουν πολλαπλή λειτουργικότητα και μπορούν να χρησιμοποιηθούν εάν χρειαστεί να ακυρώσουμε δεδομένα μετά από κάποιον χρόνο, να αντιμετωπίσουμε συγκρούσεις, καθώς και ώστε να διατηρούμε πολλαπλές εκδόσεις των δεδομένων. Οι στήλες εντός μιας *column family* αποτελούνται από δεδομένα του ίδιου τύπου, γεγονός που εξυπηρετεί την αποδοτική ανάκτηση και επεξεργασία τους.

Με άλλα λόγια, τα δεδομένα οργανώνονται σε *column families*, οι οποίες περιέχουν γραμμές δεδομένων και απλές στήλες. Για παράδειγμα, για μια εταιρεία με στοιχεία εργαζομένων, μπορούμε να θεωρήσουμε μια *column family* που θα περιέχει τις στήλες όνομα, επώνυμο και email, μια δεύτερη με την ημερομηνία πρόσληψης, ημερομηνία γέννησης, και μια τρίτη *column family* με τον μισθό και τις ώρες εργασίας των εργαζομένων. Σε κάθε τέτοια *column family* έχουμε γραμμές δεδομένων που έχουν ένα μοναδικό κλειδί, το αναγνωριστικό του εργαζομένου (όπως στο σχεσιακό μοντέλο) και, αντίστοιχα, κάθε γραμμή περιέχει τριπλέτες κλειδιού, τιμής και χρονοσφραγίδας που αντιστοιχούν σε μια συγκεκριμένη στήλη της *column family*. Είναι φανερό ότι οι *column families*, ουσιαστικά, υπαγορεύουν ένα έμμεσο σχήμα στα δεδομένα που αποθηκεύονται σε ένα *column-family store*.

Τα *column-family stores* προσφέρουν την ίδια λειτουργικότητα με τις βασικές εντολές SET /GET /DEL που είδαμε και στα key-value stores. Ωστόσο, τα συστήματα αυτά παρέχουν και εκτεταμένη λειτουργικότητα με παραπάνω εντολές για τον ορισμό *column families*, αλλά και γλώσσες ερωτημάτων με χαρακτηριστικό παράδειγμα το Cassandra [LM10] και την Cassandra Query Language (CQL), η οποία ακολουθεί μια σύνταξη βασισμένη στο παράδειγμα της SQL, υποστηρίζοντας εντολές όπως SELECT, INSERT, UPDATE και DELETE.

Ας δούμε κάποια πλεονεκτήματα και μειονεκτήματα της οργάνωσης των δεδομένων με αυτόν τον τρόπο. Καταρχάς, ερωτήματα που έχουν βέλτιστη απόδοση στα column-family stores είναι συναθροιστικά ερωτήματα (aggregation queries) που υπολογίζουν κάποιο μέτρο, όπως άθροισμα, μέσο όρο, μέγιστο ή ελάχιστο για τις τιμές της στήλης.

Ένα ακόμη πλεονέκτημα αυτής της οργάνωσης στηλών με ίδιο τύπο δεδομένων είναι η δυνατότητα συμπίεσης που παρέχουν. Η συμπίεση έχει ως στόχο τη μείωση του πλεονασμού ανάμεσα στις τιμές των δεδομένων. Αν και ο πλεονασμός που εμφανίζεται στα δεδομένα δεν αλλάζει προφανώς αν αποθηκεύσουμε τα δεδομένα κατά γραμμές ή κατά στήλες, οι επαναλαμβανόμενες τιμές, όταν εμφανίζονται «κοντά» μεταξύ των δεδομένων, προσφέρουν μεγαλύτερα περιθώρια κέρδους κατά τη συμπίεση. Επομένως, η αποθήκευση κατά στήλες εξυπηρετεί αυτόν τον στόχο καλύτερα με τιμές του ίδιου τύπου να αποθηκεύονται συνεχόμενα.

Ακόμη, τα column-family stores έχουν πολύ μεγάλη δυνατότητα οριζόντιας κλιμάκωσης, αφού το μοντέλο τους μπορεί εύκολα να κατανεμηθεί σε πλήθος κόμβων μιας συστάδας υπολογιστών και μπορούν να υποστηρίξουν εφαρμογές παράλληλης επεξεργασίας.

Το μειονέκτημα της αποθήκευσης κατά στήλες είναι ότι για την εισαγωγή μιας εγγραφής, η οποία σε μια σχεσιακή βάση δεδομένων ιδεατά θα απαιτούσε μια I/O πράξη, αφού η γραμμή είναι η δομική μονάδα ανάγνωσης και εγγραφής δεδομένων και τα στοιχεία μιας γραμμής γράφονται συνεχόμενα στον δίσκο, σε ένα column-family store, αυτή η πράξη απαιτεί την ενημέρωση πολλών column families και, άρα, πολλές I/O πράξεις.

Συνεπώς, τα συστήματα αυτά είναι καλύτερα για εφαρμογές όπου δεν γίνονται συχνά μεμονωμένες εισαγωγές νέων δεδομένων, αλλά αντίθετα συνιστώνται όταν χρησιμοποιείται μαζική φόρτωση δεδομένων, η οποία γίνεται πολύ αποδοτικά. Εφαρμογές αποθηκών δεδομένων που φορτώνουν περιοδικά νέα δεδομένα και έχουν έναν φόρτο ερωτημάτων συνάθροισης αποτελούν χαρακτηριστικές εφαρμογές που προσφέρονται για υλοποίηση μέσω ενός column-family store.

Πριν περάσουμε στην αναφορά κάποιων παραδειγμάτων τέτοιων συστημάτων, αξίζει να αναφέρουμε και μια επέκτασή τους, στην οποία οι τιμές στις ομάδες στηλών αποθηκεύονται ταξινομημένες. Η επέκταση αυτή προσφέρει άλλο ένα επίπεδο οργάνωσης στα δεδομένα και έχει ως αποτέλεσμα να έχουμε ακόμα μεγαλύτερα περιθώρια κέρδους ως προς τη συμπίεση των δεδομένων. Ειδικότερα, σε αυτήν την περίπτωση μπορεί κανείς, αντί των τιμών των δεδομένων, να διατηρεί τη διαφορά (delta) μιας τιμής από την προηγούμενή της.

4.3.1 Παραδείγματα column-family Stores

Στα column-family stores συγκαταλέγονται δημοφιλή συστήματα, όπως HBase [Geo11], Amazon SimpleDB [sim] και Cassandra [LM10]. Στη συνέχεια, θα χρησιμοποιήσουμε ως παράδειγμα το Cassandra για να δούμε πώς αντιμετωπίζει τα διαφορετικά θέματα αποθήκευσης και διαχείρισης δεδομένων.

Το Cassandra είναι μια ανοικτού λογισμικού κατανεμημένη βάση δεδομένων που ανήκει στην κατηγορία των column-family stores και προσφέρει γραμμική δυνατότητα κλιμάκωσης και μεγάλη ανοχή σε σφάλματα. Πολλές λύσεις βασισμένες στο υπολογιστικό νέφος επιλέ-

Row Key	Personal Info		Employment Info		
john	Fullname John Smith	Phone 6775552361	Hired 10/9/2015	Position LabTechnician	Salary 2800
ann	Fullname Ann Jones		Hired 10/2/2010	Position Researcher	Salary 3000

Σχήμα 4.7: Column families για εργαζομένους.

γουν το Cassandra ως το backend τους. Αναπτύχθηκε αρχικά από το Facebook/Meta και έγινε έργο ανοικτού λογισμικού της Apache το 2008. Είναι υλοποιημένο σε Java και αποτελεί μια cross-platform λύση.

Χρησιμοποιεί το μοντέλο δεδομένων που βασίζεται στην οργάνωση δεδομένων σε ιδεατούς πίνακες από στήλες οι οποίες ομαδοποιούνται σε οικογένειες/ομάδες στηλών (column families). Οι εγγραφές, οι γραμμές του πίνακα δηλαδή, προσδιορίζονται και ευρετηριοποιούνται με τη βοήθεια του κλειδιού της γραμμής. Οπότε και γραμμές που εκτείνονται σε πολλές στήλες ή column families δεικτοδοτούνται από το ίδιο μοναδικό κλειδί. Οι στήλες και οι column families ορίζονται κατά τη σχεδίαση της βάσης, οπότε σε αντίθεση με άλλα NoSQL συστήματα το Cassandra υποστηρίζει ένα δομημένο σύστημα αποθήκευσης δεδομένων. Το Cassandra χρησιμοποιεί τον όρο *keyspace* για να αναφερθεί στο σύνολο αυτών των column families μιας βάσης, το οποίο είναι και το ανάλογο της βάσης δεδομένων σε ένα σχεσιακό σύστημα διαχείρισης δεδομένων. Στο Σχήμα 4.7 διακρίνουμε στοιχεία εργαζομένων που διακρίνονται σε δύο column families, τα προσωπικά στοιχεία, και τις πληροφορίες της απασχόλησής του.

Επιπλέον, το Cassandra υποστηρίζει πολλαπλές εκδοχές από το ίδιο δεδομένο. Τα δεδομένα, δηλαδή, αποθηκεύονται μαζί με τη χρονοσφραγίδα τους και, έτσι, έχουμε υποστήριξη πολλαπλών εκδοχών. Όταν υπάρχουν πολλαπλές εκδοχές των δεδομένων, οι τιμές αποθηκεύονται σε φθίνουσα σειρά με βάση τη χρονοσφραγίδα, ενώ υπάρχει και όριο για το πλήθος των εκδόσεων που μπορούν να διατηρούνται εντός κάθε column family.

Ως column-family store το Cassandra υποστηρίζει τα απλά APIs, SET/GET/DEL, για τη διαχείριση των δεδομένων, αλλά επιπλέον, όπως αναφέραμε, υποστηρίζει και την CQL (Cassandra Query Language), μια γλώσσα υψηλού επιπέδου εμπνευσμένη από την SQL η οποία παρέχει διαφάνεια ως προς τον πραγματικό χειρισμό των δεδομένων που γίνεται από το σύστημα. Αφού η CQL βασίζεται στην SQL, υποστηρίζει πολλούς αντίστοιχους με αυτήν τελεστές, όπως CREATE, SELECT, UPDATE, INSERT, DROP, κ.λπ., αλλά και WHERE, ORDER BY. Ωστόσο, αποτελεί απλούστερη εκδοχή της SQL αφού δεν υποστηρίζει συζεύξεις (joins) και συνήθως οι συνθήκες που εμφανίζονται στο WHERE είναι απλούστερες σε σχέση με τις σχεσιακές βάσεις και την SQL.

Για να δημιουργήσουμε τον πίνακα για τα προσωπικά δεδομένα των εργαζομένων που εμφανίζονται στο Σχήμα 4.7, μπορούμε αρχικά να χρησιμοποιήσουμε την παρακάτω εντολή

CREATE:

```
CREATE TABLE PersonalInfo (
  id TEXT PRIMARY KEY
  fullname TEXT,
  phone TEXT);
```

Στη CQL, όπως και το Cassandra, οι έννοιες του πίνακα και του column family στην πράξη είναι ισοδύναμες, οπότε η παραπάνω εντολή θα δημιουργήσει την παραπάνω column family *PersonalInfo*. Για να εισαγάγουμε στη συνέχεια τα στοιχεία των εργαζομένων στον πίνακα που δημιουργήσαμε, μπορούμε να χρησιμοποιήσουμε μια εντολή INSERT, όπως φαίνεται παρακάτω:

```
INSERT INTO PersonalInfo(id, fullname, phone)
VALUES ('alex', 'Alex Smith', '678911111');
```

Ερωτήματα πάνω στα δεδομένα μπορούν με τη σειρά τους να γίνουν με την εντολή SELECT. Επομένως, για την ανάκτηση όλων των δεδομένων του *PersonalInfo* μπορούμε να χρησιμοποιήσουμε την παρακάτω εντολή:

```
SELECT * from PersonalInfo;
```

Για να φιλτράρουμε τα δεδομένα που θέλουμε να ανακτηθούν, θα πρέπει να προσθέσουμε τον τελεστή WHERE με τις κατάλληλες συνθήκες. Όπως είπαμε, ωστόσο, η CQL εμφανίζει απλούστερες συνθήκες στα κριτήρια ανάκτησης και αυτό συμβαίνει επειδή το Cassandra επιβάλλει και αρκετούς περιορισμούς στο τι μπορεί να χρησιμοποιήσει κανείς ως συνθήκη για την επιλογή δεδομένων. Για το πρωτεύον κλειδί, που χρησιμοποιείται και για τον διαμερισμό των δεδομένων στο Cassandra, υποστηρίζονται μόνο συνθήκες ισότητας.

```
SELECT fullname, phone from PersonalInfo WHERE id='alex';
```

Το παραπάνω ερώτημα θα μας επιστρέψει τις τιμές του ονόματος και του email για τον εργαζόμενο με *id='alex'*.

Σχετικά με τη συνέπεια των δεδομένων, το Cassandra υλοποιεί το μοντέλο της προσαρμόσιμης συνέπειας (tunable consistency), που υποστηρίζει πολλαπλά επίπεδα συνέπειας τα οποία μπορούν να επιλεγθούν από κάθε ξεχωριστή εφαρμογή. Η συνέπεια προσδιορίζεται μέσω τριών παραμέτρων:

- **N**: το πλήθος των αντιγράφων των δεδομένων που διατηρούνται, γνωστό και ως *παράγοντας αναπαραγωγής (replication factor)*.
- **W**: το πλήθος των αντιγράφων που πρέπει να ενημερωθούν από μία εγγραφή πριν αυτή θεωρηθεί ολοκληρωμένη.
- **R**: το πλήθος των αντιγράφων που πρέπει να προσπελαστούν για να θεωρηθεί ολοκληρωμένη μια ανάγνωση.

Ο παράγοντας αναπαραγωγής ορίζεται σε επίπεδο keyspace, δηλαδή αφορά όλα τα column families μιας βάσης. Από εκεί και πέρα, όπως αναφέραμε, η συνέπεια είναι προσαρμόσιμη, οπότε τόσο για ανάγνωση όσο και για εγγραφή μπορεί μια εφαρμογή να επιλέξει από πολύ

αυστηρή συνέπεια, που απαιτεί την ανάγνωση ή αντίστοιχα την εγγραφή όλων των αντιγράφων για να ολοκληρωθεί, μέχρι την πιο χαλαρή συνέπεια, που ολοκληρώνεται αν εγγραφεί ή προσπελαστεί έστω και ένα αντίγραφο.

4.4 Βάσεις δεδομένων εγγράφων

Άλλη μία κατηγορία των NoSQL συστημάτων είναι οι *βάσεις δεδομένων εγγράφων* (*document based databases*). Σε αυτά τα συστήματα τα δεδομένα δομούνται σε *έγγραφα* (*documents*) που ακολουθούν το παράδειγμα των αυτοπεριγραφικών ημιδομημένων μοντέλων δεδομένων. Τα έγγραφα αυτά, για παράδειγμα, μπορεί να είναι είτε XML είτε JSON έγγραφα και παρέχουν μια σημασιολογική ομαδοποίηση των δεδομένων. Με αυτήν τη μοντελοποίηση οι βάσεις δεδομένων εγγράφων δίνουν τη δυνατότητα περιγραφής της δομής των δεδομένων, χωρίς όμως να χρησιμοποιούν σχήμα δεδομένων και χωρίς να επιβάλλουν σχετικούς περιορισμούς ακεραιότητας και αναφορικής ακεραιότητας στα δεδομένα.

Βασισμένοι στο μοντέλο των key-value stores, μπορούμε να περιγράψουμε μια βάση δεδομένων εγγράφων ως μια βάση που αποθηκεύει ζεύγη από κλειδιά που αποτελούν τα μοναδικά αναγνωριστικά των εγγράφων (DocID), και τιμές οι οποίες αντιστοιχούν στα έγγραφα που αποθηκεύονται στη βάση.

Τα ίδια τα έγγραφα αποτελούνται από μια συλλογή από ζεύγη κλειδιών-τιμής όπου κάθε τιμή μπορεί να αποτελείται από άλλα σύνθετα αντικείμενα, όπως λίστες τιμών ή λίστες από άλλα ζεύγη κλειδιών-τιμής. Στα έγγραφα αυτά, γενικά, επιτρέπεται η *εμφώλευση* (*nesting*) σε οποιοδήποτε βάθος και συναντάμε την εφαρμογή των εννοιών της ενσωμάτωσης δεδομένων για τη γρήγορη ανάκτηση δεδομένων που ανακτώνται από κοινού συχνά.

Όπως είπαμε, και σε συμφωνία με τα key-value stores, οι βάσεις δεδομένων εγγράφων δεν έχουν σχήμα για τα έγγραφά τους, οπότε μπορούν σε οποιοδήποτε έγγραφο να προστεθούν αυθαίρετα ζεύγη κλειδιών-τιμής. Ωστόσο, η οργάνωση των αυθαίρετων αυτών ζευγών σε έγγραφα και η εμφώλευση των στοιχείων επιτρέπουν τη σαφώς καλύτερη λογική οργάνωση των δεδομένων και, έτσι, διευκολύνουν τη διαχείρισή τους από τις εφαρμογές.

Αν και στη θεωρία οι βάσεις δεδομένων εγγράφων μπορούν να υποστηρίξουν αυστηρή συνέπεια, συνήθως οι διαδικτυακές εφαρμογές στις οποίες συναντώνται δεν έχουν αυστηρές απαιτήσεις συνέπειας και, αντιθέτως, υιοθετούν το μοντέλο της eventual consistency. Από την άλλη, τα δεδομένα σε βάσεις εγγράφων μπορούν εύκολα να διαμεριστούν μεταξύ των κόμβων μιας συστάδας χρησιμοποιώντας sharding και συνηθίζεται η επεξεργασία τους με παράλληλα μοντέλα επεξεργασίας, όπως το MapReduce.

4.4.1 Παραδείγματα βάσεων δεδομένων εγγράφων

Στο πλαίσιο των βάσεων δεδομένων εγγράφων, θα μελετήσουμε ως αντιπροσωπευτικό παράδειγμα τη βάση MongoDB [MOnb]. Σύμφωνα και με τους δημιουργούς της, η MongoDB είναι μια *κλιμακώσιμη, υψηλής απόδοσης, ανοικτού κώδικα, χωρίς σχήμα βάση δεδομένων εγγράφων*. Το όνομά της, άλλωστε, προέρχεται από τη λέξη humongous, δηλαδή τεράστια. Η MongoDB επίσης είναι πολύ δημοφιλής, κατέχει μάλιστα τα τελευταία χρόνια την πρω-

Πίνακας 4.2: Ορολογία MongoDB και σχεσιακών βάσεων δεδομένων.

Σχεσιακή ΒΔ	MongoDB
στιγμιότυπο βάσης πίνακας	στιγμιότυπο MongoDB συλλογή
εγγραφή/γραμμή στήλη	έγγραφο ζεύγος κλειδιού-τιμής

τιά σε σχέση με όλες τις NoSQL βάσεις δεδομένων στα Database Engine Rankings (<https://db-engines.com/en/ranking>) στα οποία περισσότερα από 380 συστήματα διαχείρισης βάσεων δεδομένων βαθμολογούνται σε σχέση με τη δημοφιλία τους. Σήμερα, βρίσκεται στην 5η θέση συνολικά, αλλά τα συστήματα διαχείρισης δεδομένων που την ξεπερνούν ακολουθούν όλα το σχεσιακό μοντέλο.

Η MongoDB αναπτύχθηκε το 2007 από την 10gen, που αργότερα ονομάστηκε MongoDB. Όπως είπαμε, είναι ανοικτού κώδικα και είναι υλοποιημένη σε C++. Πρόκειται για μια schema-less βάση δεδομένων και, ως αποτέλεσμα, δεν διαθέτει κάποια γλώσσα ορισμού δεδομένων. Η MongoDB βασίζεται στον κατακερματισμό, για να είναι πιο αποδοτική, και αποθηκεύει τα κλειδιά κατακερματισμού μαζί με οποιαδήποτε κλειδιά και τιμές. Τα δεδομένα στη MongoDB αποθηκεύονται ως έγγραφα *BSON* (*binary JSON*), ενώ παρέχει APIs σε πολλές προγραμματιστικές γλώσσες. Συνεπώς, τα δεδομένα της αναπαρίστανται ως JSON έγγραφα, δηλαδή εμφωλευμένα ζεύγη κλειδιών-τιμής. Το σύστημα αποδίδει σε κάθε έγγραφο ένα μοναδικό αναγνωριστικό (*_id*) και δεν υπάρχει ανάγκη για χρήση κάποιου άλλου μοναδικού κλειδιού.

Συνεπώς, ως βάση δεδομένων εγγράφων, στη MongoDB η βασική δομική μονάδα είναι το έγγραφο, το οποίο αποτελείται από ζεύγη κλειδιών-τιμής. Ακόμα, έγγραφα που ανήκουν στον ίδιο τύπο/οντότητα ομαδοποιούνται σε *συλλογές* (*collections*). Στον Πίνακα 4.2 συνοψίζεται η αντιστοιχία των όρων που χρησιμοποιούνται στη MongoDB, έναντι των αντίστοιχων όρων στις σχεσιακές βάσεις δεδομένων.

Ας δούμε λίγο πιο αναλυτικά αυτά τα βασικά χαρακτηριστικά της οργάνωσης των δεδομένων στη MongoDB.

- Σε κάθε στιγμιότυπο της MongoDB μπορεί κανείς να έχει μηδέν ή πολλές βάσεις δεδομένων. Αυτή η αρχή είναι αντίστοιχη με ό,τι συμβαίνει στις σχεσιακές βάσεις δεδομένων, και μπορεί κανείς να θεωρήσει αυτές τις βάσεις δεδομένων ως containers υψηλού επιπέδου.
- Μια βάση μπορεί να περιέχει μηδέν ή περισσότερες συλλογές.
- Μια συλλογή αποτελείται από μηδέν ή περισσότερα έγγραφα.
- Ένα έγγραφο αποτελείται από ένα ή περισσότερα πεδία.

Όπως αναφέραμε, μια συλλογή στη MongoDB ουσιαστικά ομαδοποιεί έγγραφα που σημασιολογικά αντιστοιχούν στην ίδια οντότητα/αντικείμενο. Ωστόσο, αυτό δεν σημαίνει ότι υποχρεωτικά τα έγγραφα μιας συλλογής θα έχουν ακριβώς την ίδια δομή, δηλαδή ότι θα περιέχουν τα ίδια κλειδιά/πεδία. Αντιθέτως, κάθε έγγραφο μπορεί δυνητικά να περιέχει ένα

μοναδικό σύνολο από κλειδιά εντελώς διαφορετικά από τα υπόλοιπα έγγραφα της συλλογής. Αυτό αποτελεί απόρροια των σχεδιαστικών στόχων που υπαγόρευσαν την υλοποίηση της MongoDB. Δεν υπάρχουν αυστηρή τυποποίηση και προκαθορισμένο σχήμα, αλλά αντίθετως παρέχεται ευελιξία στην αναπαράσταση των δεδομένων. Συνεπώς, τα έγγραφα της ίδιας συλλογής μπορούν να διαφοροποιούνται οσοδήποτε. Για παράδειγμα, σε μια συλλογή για τους εργαζομένους μιας εταιρείας, όπου κάθε έγγραφο περιγράφει έναν διαφορετικό εργαζόμενο, μπορεί να υπάρχουν έγγραφα που περιέχουν πεδίο email, και άλλα που δεν έχουν. Μπορεί να υπάρχουν εργαζόμενοι που έχουν πολλά τηλέφωνα, και άλλοι που δεν έχουν κανένα, κ.ο.κ.

Εκτός από τα παραπάνω στοιχεία οργάνωσης των δεδομένων, η MongoDB υλοποιεί και ευρετήρια. Συγκεκριμένα, αυτόματα κατά τη δημιουργία των εγγράφων κατασκευάζεται ένα ευρετήριο στο *_id* τους, που υλοποιεί άλλωστε και έναν ρόλο ανάλογο αυτού του πρωτεύοντος κλειδιού για τη MongoDB. Εκτός από αυτό το ευρετήριο, όμως, η MongoDB παρέχει στον χρήστη τη δυνατότητα να ορίσει και τα δικά του ευρετήρια, είτε για λόγους απόδοσης είτε ως έναν έμμεσο τρόπο για να επιβάλει περιορισμούς μοναδικότητας στα δεδομένα. Τα ευρετήρια αυτά μπορεί να κατασκευάζονται είτε σε ένα πεδίο-κλειδί των εγγράφων είτε και να είναι σύνθετα ευρετήρια που κατασκευάζονται πάνω σε συνδυασμούς γνωρισμάτων.

Για την υλοποίηση των ευρετηρίων η MongoDB χρησιμοποιεί δενδρικές δομές, και συγκεκριμένα B+ δένδρα, τα οποία αποθηκεύουν διατεταγμένες τις τιμές των δεδομένων. Χρησιμοποιώντας μια τέτοια δενδρική δομή ευρετηρίου, η MongoDB μπορεί να υποστηρίξει πιο αποδοτικά όχι μόνο ερωτήματα αναζήτησης συγκεκριμένης τιμής (*lookup queries*), αλλά και ερωτήματα για ένα εύρος τιμών ενός κλειδιού (*range queries*), σε αντίθεση με τα ευρετήρια που βασίζονται στον κατακερματισμό και δεν μπορούν να υποστηρίξουν *range queries*. Για παράδειγμα, ένα ευρετήριο στο πεδίο «ηλικία» εργαζομένων, μπορεί να χρησιμοποιηθεί για να επιταχύνει τόσο το ερώτημα «Βρες όλους τους εργαζομένους που είναι 28 ετών» όσο και το ερώτημα «Βρες όλους τους εργαζομένους που είναι μεταξύ 30 και 40 ετών».

Η MongoDB παρέχει στον χρήστη ένα κέλυφος (*shell*) για να επικοινωνεί με τον διακομιστή, το *mongo shell*. Αυτό είναι ουσιαστικά ένα διαδραστικό περιβάλλον που υποστηρίζει εντολές στη μορφή Javascript. Πέρα από αυτό, η MongoDB παρέχει και έναν γραφικό διαδραστικό πελάτη, το Mongo Compass [Com], για πιο εύκολη και φιλική προς τον χρήστη χρήση. Οι νεότερες εκδόσεις του Compass έχουν ενσωματωμένο και το *mongo shell*.

Παρακάτω απαριθμούμε τις *shell* εντολές για τις βασικές CRUD λειτουργίες που υποστηρίζει η Mongo.

- Create:
 - `db.collection.insertOne(document)`
 - `db.collection.insertMany(document)`
- Read:
 - `db.collection.find(filter, projection)`
 - `db.collection.findOne(filter, projection)`
- Update:

- `db.collection.updateOne(filter, update, options)`
- `db.collection.updateMany(filter, update, options)`
- Delete:
 - `db.collection.deleteOne(filter)`
 - `db.collection.deleteMany(filter)`
 - `db.collection.drop()`

Ας αναλύσουμε αρχικά τις εντολές για τη δημιουργία δεδομένων (CREATE). Βλέπουμε ότι η MongoDB παρέχει εντολές για την εισαγωγή ενός (`insertOne`) ή περισσότερων (`insertMany`) εγγράφων. Η παράμετρος που παίρνει η εντολή είναι το *document*, δηλαδή το έγγραφο ή τα έγγραφα που θα εισαχθούν στη βάση. Με τον όρο έγγραφο εδώ εννοούμε το σύνολο από τα ζεύγη κλειδιών-τιμών που το απαρτίζουν. Προσέξτε, ότι δεν υπάρχει εντολή για τη δημιουργία μιας συλλογής. Η συλλογή, *collection* στην εντολή δημιουργίας, δημιουργείται αυτόματα, όταν εισάγουμε το πρώτο έγγραφο σε αυτήν. Στο παρακάτω παράδειγμα έχουμε τη δημιουργία δύο εγγράφων εργαζομένων, τα οποία εισάγονται σε μια συλλογή «employees». Η συλλογή αυτή δημιουργείται από την πρώτη εντολή (αν δεν υπάρχει ήδη), ενώ η δεύτερη εντολή προσθέτει άλλο ένα έγγραφο στην υπάρχουσα συλλογή.

```
1 db.employees.insertOne({name:'John Smith', dob:new Date(1988, 2, 20), phone
   :['6775552361'], email:'jsmith@company.com'});
2 db.employees.insertOne({name:'Mary Papadopoulou', dob:new Date(1994, 1, 15),
   phone:['6775633342', '6877111111']});
```

Παρατηρούμε ότι στο έγγραφο που δίνουμε προς εισαγωγή δεν καθορίζουμε εμείς κάποιο αναγνωριστικό. Αυτό, όπως είπαμε, δίνεται αυτόματα από τη MongoDB, με αποτέλεσμα το έγγραφο που θα δημιουργηθεί, από την πρώτη εντολή για παράδειγμα, να είναι της μορφής:

```
{
  "_id": {
    "$oid": "5ec2f0bf5dd6d89aa2ac2a1c"
  },
  "name": "John Smith",
  "dob": {
    "$date": "1988-02-20"
  },
  "phone": [
    "6775552361"
  ],
  "email": "jsmith@company.com"
}
```

Εναλλακτικά, θα μπορούσαμε να δώσουμε εμείς το *_id* των εγγράφων.

Οι εντολές *find* και *findOne* χρησιμοποιούνται για την ανάκτηση δεδομένων από τη συλλογή *collection*. Η παράμετρος *filter* είναι προαιρετική και λειτουργεί ως φίλτρο, προσδιορίζοντας τα κριτήρια της αναζήτησης. Μπορεί να θεωρήσει κανείς αυτό το φίλτρο ανάλογο του

Πίνακας 4.3: Συγκριτικοί, λογικοί και άλλοι τελεστές στη MongoDB.

Τελεστής	Περιγραφή
\$eq	συγκριτικός τελεστής ισότητας =
\$ne	συγκριτικός τελεστής διάφορο $\neq$
\$gt	συγκριτικός τελεστής μεγαλύτερο >
\$gte	συγκριτικός τελεστής μεγαλύτερο ή ίσο $\geq$
\$in	τελεστής που ελέγχει αν η τιμή ανήκει σε έναν πίνακα (λίστα) $\in$
\$nin	τελεστής που ελέγχει αν η τιμή δεν ανήκει σε έναν πίνακα (λίστα) $\notin$
\$or	λογικός τελεστής OR
\$and	λογικός τελεστής AND
\$not	λογικός τελεστής NOT
\$exists	τελεστής που ελέγχει αν υπάρχει κάποιο πεδίο $\exists$
\$nexists	τελεστής που ελέγχει αν δεν υπάρχει κάποιο πεδίο $\nexists$

WHERE της SQL, όπου εμφανίζονται οι συνθήκες που πρέπει να ικανοποιούν οι εγγραφές που ανακτώνται. Προαιρετική είναι και η παράμετρος *projection*, η οποία καθορίζει ποια πεδία των εγγράφων που ικανοποιούν τα κριτήρια του *filter* πρόκειται να συμπεριλαμβάνονται στο αποτέλεσμα, με άλλα λόγια, ποια πεδία θα προβληθούν και είναι ανάλογη του SELECT της SQL. Η *findOne* επιστρέφει μόνο το πρώτο αποτέλεσμα που ικανοποιεί τα κριτήρια του *filter* αν υπάρχουν, ενώ η *find* επιστρέφει όλα τα αποτελέσματα που ικανοποιούν τα κριτήρια αναζήτησης και τα τοποθετεί σε έναν κέρσορα προς προσπέλαση. Αν δεν καθοριστούν κριτήρια αναζήτησης, τότε επιστρέφονται όλα ή το πρώτο έγγραφο της συλλογής, ανάλογα με το ποια από τις δύο εντολές ανάκτησης δεδομένων χρησιμοποιήθηκε. Επίσης, αν δεν προσδιοριστούν πεδία προβολής, τότε επιστρέφεται όλο το έγγραφο που ικανοποιεί ένα ερώτημα.

Οι συνθήκες που μπορούν να οριστούν σε ένα φίλτρο σχηματίζονται με τη χρήση συγκριτικών τελεστών, λογικών τελεστών, κ.ά. Για να γίνει πιο κατανοητή η χρήση της ανάκτησης δεδομένων, ας εξετάσουμε κάποια ενδεικτικά παραδείγματα.

```
db.employees.find({name:"Mary Papadopoulou"});
```

Στο παραπάνω ερώτημα καθορίζουμε μόνο το φίλτρο με τη συνθήκη το πεδίο *name* να έχει τιμή «Mary Papadopoulou» και το αποτέλεσμα είναι τα έγγραφα που ικανοποιούν αυτήν τη συνθήκη. Εφόσον δεν καθορίζουμε πεδία προβολής, θα επιστραφούν ολόκληρα τα έγγραφα. Το ανάλογο SQL ερώτημα είναι:

```
select *
from employees
where name="Mary Papadopoulou";
```

όπου ο πίνακας *employees* είναι το σχεσιακό ανάλογο της συλλογής *employees*.

Στο επόμενο παράδειγμα θα χρησιμοποιήσουμε και την προβολή.

```
db.employees.find({salary:{$gt:2500}}, {name:1});
```

Η εντολή αυτή καθορίζει ένα φίλτρο $\{salary : \{ \$gt : 2000 \} \}$ και ένα πεδίο προς προβολή, θέτοντας $\{name : 1\}$. Το φίλτρο χρησιμοποιεί τον συγκριτικό τελεστή “\$gt” που αντιστοιχεί στο “>” (greater than). Στον Πίνακα 4.3 αναφέρονται κάποιοι ενδεικτικοί τελεστές, αλλά μπορούμε να βρούμε περισσότερους στο εγχειρίδιο του mongo shell [Mona].

Η διαγραφή είναι ανάλογη της ενημέρωσης. Με την *deleteOne* διαγράφεται το πρώτο έγγραφο που ταιριάζει στο φίλτρο που δίνεται, με την *deleteMany* διαγράφονται όλα τα έγγραφα που ταιριάζουν στο φίλτρο, ενώ η *drop* χρησιμοποιείται για να διαγραφεί όλη η συλλογή *collection*.

Πριν ολοκληρώσουμε την πολύ συνοπτική αναφορά μας στις λειτουργίες της MongoDB, θα αναφερθούμε σε μία ακόμα κατηγορία ερωτημάτων, τα συναθροιστικά ερωτήματα. Στην SQL έχουμε το GROUP BY για ομαδοποίηση των εγγραφών ενός πίνακα και τις συναθροιστικές συναρτήσεις, όπως MAX, MIN, COUNT. Οπότε, για παράδειγμα, εάν θέλουμε να μετρήσουμε πόσοι είναι οι εργαζόμενοι από κάθε πόλη, μπορούμε να θέσουμε το SQL ερώτημα:

```
select count(*)
from employees
group by city;
```

Για να παρέχει αυτήν τη λειτουργικότητα, η MongoDB χρησιμοποιεί την εντολή:

```
db.collection.aggregate(pipeline, options)
```

Η *aggregate*, όπως βλέπουμε, έχει ως όρισμα ένα pipeline που ορίζει στάδια επεξεργασίας για τα έγγραφα της συλλογής. Έτσι, το παράδειγμα που είδαμε σε SQL θα γραφεί ως:

```
db.employees.aggregate (
[
{$group:{
_id:"$city",
number:{$sum:1$}}
}
]);
```

Το παραπάνω ερώτημα αποτελείται μόνο από ένα στάδιο, το *\$group* που πραγματοποιεί την ομαδοποίηση με βάση το πεδίο που καθορίζεται στο *_id*, δηλαδή εδώ το *city*. Το αποτέλεσμα αποτελείται από ένα έγγραφο για κάθε διαφορετική πόλη. Επιπλέον, το ερώτημα δημιουργεί σε κάθε έγγραφο του αποτελέσματος ένα νέο πεδίο *number*, στο οποίο αθροίζεται 1 κάθε φορά που ένα έγγραφο προστίθεται στην ομάδα και, άρα, έχουμε τον μετρητή των εργαζομένων ανά πόλη.

Πέρα από το στάδιο *\$group*, υποστηρίζονται και πολλά άλλα στάδια. Ενδεικτικά, αναφέρουμε:

- *\$match* που επιβάλλει κάποιο φίλτρο στα έγγραφα,
- *\$sort* που, όπως έχουμε δει, ταξινομεί τα αποτελέσματα,
- *\$project* που καθορίζει ποια πεδία θα έχουν τα έγγραφα του αποτελέσματος, και
- *\$unwind* που «ξεδιπλώνει» ένα πεδίο πίνακα έτσι ώστε να προκύψει ένα έγγραφο για κάθε στοιχείο που υπάρχει στον πίνακα.

Ας υποθέσουμε ότι τα έγγραφά μας έχουν και ένα πεδίο *hobbies* που ορίζεται ως μία λίστα (πίνακας) από τα hobbies κάθε εργαζομένου, π.χ. σινεμά, τρέξιμο, κ.λπ. Έστω ότι θέλουμε να μετρήσουμε πόσοι εργαζόμενοι έχουν το κάθε hobby:

```
db.employees.aggregate(  
[  
{$unwind: '$hobbies$'},  
{$group: {  
  _id: "$hobbies",  
  number: { $sum: 1$ },  
  emps: { $addToSet$: '$name$' }  
}}  
]);
```

Στο παραπάνω ερώτημα έχουμε δύο στάδια, *\$unwind* και *\$group*. Πρώτα αναπτύσσεται ο πίνακας των *hobbies* και μετά τα στοιχεία του χρησιμοποιούνται για την ομαδοποίηση. Επιπλέον, δημιουργείται άλλο ένα πεδίο στα έγγραφα του αποτελέσματός μας, το *emps*, που είναι ένας πίνακας με τα ονόματα των εργαζομένων που έχουν το κάθε hobby.

Είναι προφανές ότι η λειτουργικότητα που παρέχει η MongoDB μας δίνει πολλές δυνατότητες, και δεν είναι δυνατό να την καλύψουμε σε ένα κεφάλαιο, αλλά προσπαθήσαμε να φωτίσουμε τις διαφορετικές δυνατότητες που παρέχονται.

Όσον αφορά τη διαχείριση των δεδομένων, η Mongo χρησιμοποιεί sharding και αναπαραγωγή. Για το sharding, μπορεί να χρησιμοποιηθεί είτε hash-based είτε range-based sharding. Για την αναπαραγωγή των δεδομένων, δημιουργείται ένα πρωτεύον και ένα ή πολλά δευτερεύοντα αντίγραφα σε διαφορετικούς κόμβους της συστάδας υπολογιστών που υλοποιούν τη βάση. Τέλος, για τη συνέπεια, η MongoDB εξ ορισμού παρέχει αυστηρή συνέπεια χρησιμοποιώντας κλειδώματα σε επίπεδο εγγράφου. Οι εγγραφές πετυχαίνουν όταν ολοκληρωθεί η εγγραφή στο πρωτεύον αντίγραφο.

4.5 Βάσεις δεδομένων γραφημάτων

Η τελευταία βασική κατηγορία NoSQL συστημάτων διαχείρισης βάσεων δεδομένων είναι οι *βάσεις δεδομένων γραφημάτων* (*graph databases*). Οι βάσεις δεδομένων γραφημάτων διαφέρουν σημαντικά από τις τρεις άλλες κατηγορίες που παρουσιάσαμε μέχρι τώρα, τα key-value stores, τα column-family stores και τις βάσεις δεδομένων εγγράφων. Στις βάσεις δεδομένων γραφημάτων τα δεδομένα μοντελοποιούνται ως *γραφήματα* (*graphs*) και, σε αντίθεση με τα προηγούμενα συστήματα όπου αποθηκεύονται ξεχωριστές οντότητες στη βάση δεδομένων, εδώ η έμφαση δίνεται στη συσχέτιση των δεδομένων.

Ειδικότερα, η ανάγκη για τις βάσεις δεδομένων γραφημάτων προκύπτει από την απαίτηση για τη διαχείριση δεδομένων στα οποία σημαντικό ρόλο και αξία έχουν οι σχέσεις μεταξύ τους. Το πιο χαρακτηριστικό παράδειγμα αποτελούν τα δεδομένα ενός κοινωνικού δικτύου, όπως το Facebook. Σε ένα τέτοιο κοινωνικό δίκτυο αποθηκεύεται πληροφορία για τους χρήστες του δικτύου, ωστόσο πολύ σημαντικό είναι να αποτυπωθούν και οι σχέσεις φιλίας μεταξύ τους, καθώς και οι αλληλεπιδράσεις τους. Σε ένα μοντέλο γραφημάτων οι χρήστες του κοινωνικού δικτύου θα μοντελοποιηθούν ως οι κόμβοι του γραφήματος και οι ακμές του θα μοντελοποιούν τις σχέσεις φιλίας ή και τις αλληλεπιδράσεις μεταξύ τους. Αντίστοιχη μοντελοποίηση χρησιμοποιείται για τον παγκόσμιο ιστό, στον οποίο οι ιστοσελίδες αποτελούν τους κόμβους του παραγόμενου γραφήματος, ενώ οι υπερσύνδεσμοι μοντελοποιούνται ως οι κα-

τευθυνόμενες ακμές του γραφήματος. Γενικά, οποιοδήποτε σύνολο δεδομένων αντιστοιχεί σε ένα δίκτυο του πραγματικού κόσμου, όπως οδικά δίκτυα, βιβλιογραφικά δίκτυα αναφορών, δίκτυα νευρώνων, κ.ά., μπορούν να αναπαρασταθούν και να μοντελοποιηθούν με τη χρήση γραφημάτων.

Οπότε, μια βάση δεδομένων γραφημάτων μοντελοποιεί τα δεδομένα που αποθηκεύονται σε αυτή ως ένα γράφημα και, άρα, τα βασικά στοιχεία του μοντέλου είναι οι κόμβοι και οι ακμές (σχέσεις) του γραφήματος. Αναλόγως με τη βάση δεδομένων γραφημάτων που έχουμε, το βασικό αυτό μοντέλο μπορεί να επεκτείνεται με τη χρήση ιδιοτήτων για τα δεδομένα, όπως συμβαίνει στο *γράφημα ιδιοτήτων* (*property graph*) στη Neo4j [sys] που θα περιγράψουμε αναλυτικά στη συνέχεια.

Το χαρακτηριστικό οποιασδήποτε βάσης δεδομένων γραφημάτων είναι ότι τόσο το μοντέλο δεδομένων, όπως είπαμε, όσο και η λειτουργικότητα που προσφέρει βασίζονται στα γραφήματα και τις αντίστοιχες λειτουργίες για γραφήματα. Έχουμε, λοιπόν, λειτουργικότητα που αφορά την ανάκτηση των *γειτόνων* ενός κόμβου στο γράφημα (των κόμβων δηλαδή με τους οποίους συνδέεται με μία ακμή ένας κόμβος), την εύρεση *μονοπατιών* στο γράφημα, *υπογραφημάτων*, κ.λπ. Βέβαια, όπως και στις σχεσιακές βάσεις δεδομένων, έχουμε υποστήριξη για τις βασικές CRUD πράξεις, δηλαδή δημιουργία (CREATE), ανάγνωση (READ), ενημέρωση (UPDATE), και διαγραφή (DELETE), αλλά αφορούν πλέον τα δομικά στοιχεία του μοντέλου του γραφήματος. Έχουμε δηλαδή δημιουργία, ανάγνωση, ενημέρωση και διαγραφή κόμβων και ακμών.

Παρότι, όπως είπαμε, οι βάσεις δεδομένων γραφημάτων είναι η κατάλληλη επιλογή για τη μοντελοποίηση δεδομένων στα οποία ιδιαίτερη σημασία έχουν οι διασυνδέσεις μεταξύ τους, οι σχεσιακές βάσεις δεδομένων και το σχεσιακό μοντέλο έχουν φυσικά και αυτές τη δυνατότητα αναπαράστασης των διασυνδέσεων των δεδομένων. Το πρόβλημα που συναντούν οι σχεσιακές βάσεις είναι όταν έχουμε γραφήματα μεγάλου βάθους όπου πλέον η *διάσχιση* του γραφήματος, δηλαδή η προσπέλαση των κόμβων του γραφήματος ακολουθώντας τις ακμές του, καθίσταται υπολογιστικά πολύ ακριβή.

Ας σκεφτούμε το παράδειγμα ενός κοινωνικού δικτύου. Οι φιλίες σε ένα σχεσιακό μοντέλο δεδομένων θα αποθηκεύονται σε έναν πίνακα που θα συσχετίζει τα κλειδιά των αντίστοιχων χρηστών που συνδέονται με μια σχέση φιλίας. Αν θέλουμε να ανακτήσουμε τους φίλους των φίλων ενός χρήστη, τότε θα πρέπει να πραγματοποιήσουμε σύζευξη του πίνακα φιλίας με τον εαυτό του (self-join). Αν θέλουμε να προχωρήσουμε ένα βήμα περαιτέρω και να ανακτήσουμε τους χρήστες σε απόσταση τρία από έναν χρήστη, ή αλλιώς τους φίλους των φίλων των φίλων ενός χρήστη, θα πρέπει το αποτέλεσμα της προηγούμενης σύζευξης να το συζεύξουμε και πάλι με τον πίνακα της φιλίας. Το αποτέλεσμα κάθε διαδοχικής σύζευξης θα είναι ολοένα και μεγαλύτερο, και έτσι γίνεται φανερό ότι αυτή η μέθοδος δεν μπορεί να κλιμακωθεί σε μεγάλα γραφήματα. Ακόμα και η ανάκτηση των άμεσων φίλων ενός χρήστη (δηλαδή των γειτονικών του κόμβων) που μπορούν να εντοπιστούν με την ανάγνωση του πίνακα φιλίας απαιτεί τη χρήση ευρετηρίων, και το κόστος που επιφέρει αυξάνεται σε μεγάλα γραφήματα με πολλές σχέσεις μεταξύ των χρηστών τους.

Οι βάσεις δεδομένων γραφημάτων, από την άλλη, ικανοποιούν την *ιδιότητα γειτνίασης χωρίς ευρετήριο* (*index-free adjacency property*), η οποία ουσιαστικά σημαίνει ότι η ανάκτηση των γειτόνων ενός κόμβου στο γράφημα γίνεται χωρίς να απαιτεί τη χρήση ευρετηρίων. Απο-

τέλεσμα αυτής της ιδιότητας είναι ότι το κόστος της διάσχισης μιας ακμής (ενός βήματος – hop) στο γράφημα δεν εξαρτάται από το μέγεθος του γραφήματος, δηλαδή από το πλήθος των κόμβων του γραφήματος. Αυτό συνεπάγεται τη δυνατότητα των βάσεων δεδομένων γραφημάτων να πραγματοποιούν πολύ αποδοτικά τη διάσχιση των γραφημάτων και, άρα, να απαντούν σε ερωτήματα της μορφής «Βρες τους φίλους των φίλων μου» πολύ αποδοτικά και σε οποιοδήποτε βάθος αναζήτησης.

Βέβαια, ευρετήρια χρησιμοποιούνται και στις βάσεις δεδομένων γραφημάτων, για παράδειγμα στις ιδιότητες των κόμβων (αν υπάρχουν), έτσι ώστε να μπορούν να υλοποιηθούν πιο αποδοτικά ερωτήματα που αναζητούν κόμβους που έχουν κάποιες συγκεκριμένες ιδιότητες.

Η δυσκολία που εμφανίζεται στις βάσεις δεδομένων γραφημάτων έχει να κάνει με την εφαρμογή του sharding. Σε αντίθεση με τις τρεις άλλες κατηγορίες NoSQL συστημάτων των οποίων τα μοντέλα δεν υλοποιούν διασυνδέσεις μεταξύ των δεδομένων που αποθηκεύουν, και άρα η διαμέρισή τους είναι προφανής, στα γραφήματα ο στόχος είναι ακριβώς η μοντελοποίηση των διασυνδέσεων μεταξύ των διαφορετικών αντικειμένων στη βάση. Έτσι, αν διαμερίσουμε ένα γράφημα μεταξύ των υπολογιστικών κόμβων μιας συστάδας, η διάσχιση ενός τμήματος του γραφήματος και ακόμα και η ανάκτηση των γειτόνων ενός κόμβου πιθανότατα θα απαιτήσει την πρόσβαση σε διαφορετικούς υπολογιστικούς κόμβους της συστάδας. Συνεπώς, οι στρατηγικές για τη διαμέριση του γραφήματος δεν είναι τόσο προφανείς και στοχεύουν στο να ελαχιστοποιήσουν το πλήθος των μεταβάσεων μεταξύ των κόμβων της συστάδας κατά την πραγματοποίηση διασχίσεων στο γράφημα. Αν και θεωρητικά υπάρχουν σχετικοί αλγόριθμοι για τη διαμέριση γραφημάτων από τον τομέα της θεωρίας γραφημάτων, η πολυπλοκότητά τους είναι τέτοια, που στην πράξη για τα μεγάλης κλίμακας γραφήματα που διαχειρίζονται οι βάσεις δεδομένων γραφημάτων καταφεύγουμε σε ευρετικές μεθόδους.

4.5.1 Παραδείγματα βάσεων δεδομένων γραφημάτων

Η πιο δημοφιλής βάση δεδομένων γραφημάτων είναι η Neo4j [sys]. Η Neo4j αποτελεί μια ανοικτού κώδικα, cross-platform βάση δεδομένων γραφημάτων που είναι υλοποιημένη σε Java. Προσφέρει μεγάλη κλιμάκωση και μπορεί να υποστηρίξει γραφήματα με δισεκατομμύρια κόμβους, ενώ υποστηρίζεται και από μια πολύ ενεργή κοινότητα που προσθέτει συνεχώς επιπλέον λειτουργικότητα μέσω ανάπτυξης νέων εκδόσεων και plugins, όπως είναι η βιβλιοθήκη Graph Data Science (GDS) library [GDS] για την ανάλυση γραφημάτων, το NeoSemantics [n10] για την υποστήριξη οντολογιών και γραφημάτων γνώσεων, και το εργαλείο Neo4j Bloom [Blo] για οπτικοποίηση.

Όσον αφορά τη συνέπεια των δεδομένων, η Neo4j υποστηρίζει συναλλαγές που ικανοποιούν τις ACID ιδιότητες. Το επίπεδο απομόνωσης που παρέχεται εξ ορισμού είναι το *read committed*, το οποίο σημαίνει ότι μια συναλλαγή διαβάσει μόνο δεδομένα που έχουν ενημερωθεί από ολοκληρωμένες συναλλαγές. Για την υποστήριξη του ταυτοχρονισμού, η Neo4j χρησιμοποιεί κλειδώματα τόσο σε επίπεδο κόμβων όσο και σχέσεων. Επίσης, χρησιμοποιεί master/slave αναπαραγωγή.

Η Neo4j παρέχει μια web-based διεπαφή για τη διαχείριση της βάσης δεδομένων, ενώ υποστηρίζει και πολυάριθμα APIs για πολλές προγραμματιστικές γλώσσες.

Το μοντέλο δεδομένων που χρησιμοποιεί η Neo4j, όπως αναφέραμε, είναι το γράφημα ιδιο-

τήτων (property graph). Οπότε, τα δομικά στοιχεία του μοντέλου είναι πιο αναλυτικά:

- **Κόμβοι.** Οι κόμβοι του γραφήματος αντιστοιχούν στις οντότητες των δεδομένων.
- **Σχέσεις.** Οι σχέσεις, δηλαδή οι ακμές του γραφήματος, έχουν όνομα, αποτελούν κατευθυνόμενα τόξα μεταξύ ενός ζεύγους κόμβων, και αντιστοιχούν σε σημασιολογικές συνδέσεις μεταξύ των αντίστοιχων οντοτήτων.
- **Ιδιότητες.** Οι ιδιότητες αντιστοιχούν σε γνωρίσματα τα οποία περιγράφουν τις οντότητες αλλά και τις σχέσεις του γραφήματος. Οπότε, τόσο οι κόμβοι όσο και οι σχέσεις έχουν ιδιότητες. Αυτές αναπαρίστανται μέσα από ζεύγη κλειδιού-τιμής.

Οι κόμβοι στο γράφημα μπορούν να έχουν μία ή περισσότερες ετικέτες (*labels*). Ο ρόλος τους είναι να ομαδοποιούν ουσιαστικά ομοειδείς κόμβους/οντότητες. Έτσι, ένας κόμβος μπορεί να έχει, για παράδειγμα, ετικέτα *person*, και επίσης ετικέτα *employee*. Ενώ, όμως, οι ετικέτες είναι προαιρετικές και μπορεί να είναι πολλαπλές για τους κόμβους, οι σχέσεις έχουν υποχρεωτικά όνομα ή τύπο, όπως αναφέρεται πολλές φορές, και αυτός ο τύπος είναι μοναδικός για κάθε σχέση.

Όπως όλες οι NoSQL βάσεις δεδομένων γραφημάτων, και η Neo4j δεν επιβάλλει την αυστηρή τυποποίηση των δεδομένων και την ύπαρξη σχήματος. Αντιθέτως, παρέχεται προαιρετικά η υποστήριξη ορισμού σχήματος για λόγους απόδοσης αλλά και μοντελοποίησης. Επομένως, οι κόμβοι ακόμα και αν έχουν την ίδια ετικέτα, ή οι σχέσεις, ακόμη και αν έχουν τον ίδιο τύπο, δεν είναι υποχρεωτικό να έχουν το ίδιο σύνολο ιδιοτήτων, αλλά υποστηρίζεται ευελιξία στην περιγραφή των δεδομένων. Επίσης, εξ ορισμού δεν επιβάλλονται περιορισμοί πρωτεύοντος κλειδιού ή μοναδικότητας, αν και όπως είπαμε υπάρχει η δυνατότητα να επιβληθούν αντίστοιχοι περιορισμοί αν αυτό απαιτείται από τις εφαρμογές. Τον ρόλο του πρωτεύοντος κλειδιού υλοποιεί ένα μοναδικό αναγνωριστικό (*id*) που ανατίθεται αυτόματα από τη βάση δεδομένων σε κάθε κόμβο και ακμή που δημιουργείται στο σύστημα.

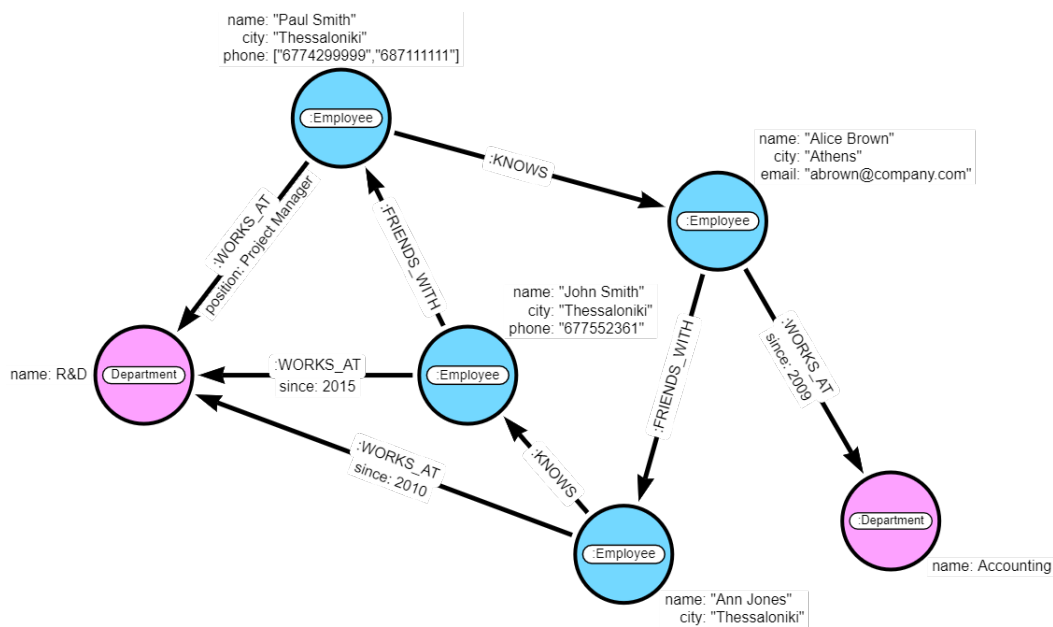
Στο Σχήμα 4.8 βλέπουμε ένα γράφημα που αναπαριστά εργαζομένους μιας εταιρείας, τα τμήματα στα οποία ανήκουν, καθώς και τις σχέσεις μεταξύ τους.

Ένα από τα μεγάλα πλεονεκτήματα της Neo4j είναι ότι υποστηρίζει μια υψηλού επιπέδου, δηλωτική γλώσσα για τη διαχείριση του γραφήματος, τη Cypher [Cyp]. Η Cypher είναι εμπνευσμένη από την SQL και περιγράφει πρότυπα ή μοτίβα (*patterns*) σε γραφήματα οπτικά, με τη χρήση ενός συντακτικού ASCII-art. Για παράδειγμα, ένα πρότυπο που θα οριζόταν ως μια σχέση τύπου *:KNOWS* μεταξύ δύο κόμβων, θα αναπαρίστατο από τη Cypher ως:

(a)-[:KNOWS]->(b)

όπου οι δύο κόμβοι αναπαρίστανται με παρενθέσεις και η αναμεταξύ τους σχέση με το βέλος, με τύπο που φαίνεται παραπάνω. Τα *a* και *b* εδώ είναι μεταβλητές, και θα δούμε τη λειτουργία τους παρακάτω.

Οι κόμβοι και οι σχέσεις αποτελούν τα δομικά στοιχεία της Cypher που τη βοηθούν να συνθέτει τα πιο πολύπλοκα πρότυπα, τα οποία με τη σειρά τους αποτελούν άλλωστε και το θεμελιώδες στοιχείο των ερωτημάτων. Τα πρότυπα μπορεί να είναι πολύ απλά, απλοί κόμβοι για παράδειγμα, μονοπάτια, δηλαδή μια ακολουθία από κόμβους που συνδέονται μεταξύ τους με σχέσεις (ακμές στο γράφημα), αλλά και πιο σύνθετα πρότυπα που αποτελούνται από



Σχήμα 4.8: Γράφημα ιδιοτήτων.

συνδυασμό άλλων προτύπων και ουσιαστικά ορίζουν κάποιο υπογράφημα. Σημαντικό για τη διαμόρφωση των μονοπατιών είναι ότι οι σχέσεις πρέπει πάντα να περικλείονται από ένα ζεύγος κόμβων, οπότε δεν μπορεί να υπάρξει πρότυπο που να ξεκινάει ή να τελειώνει με μία ακμή.

Η λειτουργία της Cypher, λοιπόν, έγκειται στο να αναζητά και να ταιριάζει αυτά τα πρότυπα στο γράφημα της βάσης δεδομένων. Ως προς το συντακτικό που χρησιμοποιείται για τη διαμόρφωση των προτύπων, όπως είδαμε παραπάνω, οι κόμβοι συμβολίζονται με τα σύμβολα της παρένθεσης (). Πιο αναλυτικά, το συντακτικό υποστηρίζει για τους κόμβους:

- *Ανώνυμους κόμβους* που συμβολίζονται με τις κενές παρενθέσεις () και χρησιμοποιούνται όταν θέλουμε να προσδιορίσουμε την ύπαρξη κάποιου κόμβου σε ένα πρότυπο, αλλά αυτός ο κόμβος δεν είναι σχετικός με το ερώτημά μας. Δηλαδή, δεν χρησιμοποιείται ούτε για επιπλέον φιλτράρισμα με κάποια συνθήκη ούτε και θέλουμε υποχρεωτικά να μας επιστραφεί αυτός ή ιδιότητές του στα αποτελέσματα.
- *Κόμβους που ανατίθενται σε μία μεταβλητή* και συμβολίζονται ως (*variable*). Η εμβέλεια της ανάθεσης αυτής αφορά μόνο το τρέχον ερώτημα.
- *Κόμβους με δεδομένη ετικέτα* που συμβολίζονται ως (*:* *label*). Με αυτόν τον τρόπο, καθορίζεται ότι μόνο κόμβοι που διαθέτουν τη συγκεκριμένη ετικέτα θα ταιριάξουν στον συγκεκριμένο κόμβο του προτύπου. Ο κόμβος μπορεί να είναι είτε ανώνυμος είτε και να έχει ανατεθεί σε μεταβλητή, οπότε και ο συμβολισμός είναι (*variable : label*). Ακόμη, το συντακτικό επιτρέπει να επεκταθεί αυτός ο περιορισμός σε κόμβους που θα πρέπει να έχουν πολλαπλές ετικέτες, οπότε έχουμε: (*variable : label1 : label2*), και αντίστοιχα για ανώνυμους κόμβους.
- *Κόμβους που έχουν κάποια ιδιότητα με συγκεκριμένη τιμή* και συμβολίζονται ως (*variable {property : 'value'}*) (ή και αντίστοιχα για ανώνυμους κόμβους). Και αυτή η περίπτωση

μπορεί να συνδυαστεί και με τον περιορισμό για ύπαρξη συγκεκριμένης ετικέτας στους κόμβους, οπότε μπορούμε να έχουμε εκφράσεις σαν αυτή: (*variable* : *label*{*property* : '*value*'}).

Για τις ακμές, το αντίστοιχο συντακτικό έχει ως εξής:

- *Ανώνυμες σχέσεις* που συμβολίζονται είτε ως $-[]->$ είτε ως $-->$ και η χρήση τους είναι αντίστοιχη με των ανώνυμων κόμβων.
- *Σχέσεις που ανατίθενται σε μια μεταβλητή* και συμβολίζονται ως $-[variable]->$.
- *Σχέσεις με δεδομένο τύπο* που συμβολίζονται ως $-[: TYPE]->$ και ταιριάζουν μόνο σε σχέσεις που έχουν αυτόν τον τύπο στο γράφημα. Ο περιορισμός αυτός μπορεί να επιβληθεί και σε σχέση που έχει ανατεθεί σε μεταβλητή και συμβολίζεται ως $-[variable : TYPE]->$. Όπως είπαμε, οι σχέσεις έχουν μοναδικό τύπο, οπότε δεν υποστηρίζονται ερωτήματα για ακμές με πολλαπλούς τύπους όπως έχουμε για κόμβους με πολλαπλές ετικέτες. Ωστόσο, η Cypher παρέχει τη δυνατότητα της διάξευξης μεταξύ τύπων, δηλαδή μπορεί ένα πρότυπο να περιέχει μια σχέση που ανήκει είτε στον TYPE1 είτε στον TYPE2, και σε αυτήν την περίπτωση θα το συμβολίζαμε ως $-[: TYPE1 | : TYPE2]->$
- *Σχέσεις που έχουν κάποια ιδιότητα η οποία έχει με συγκεκριμένη τιμή* και συμβολίζονται ως $-[variable : TYPE\{property : 'value'\}]->$, και μπορούν και αυτές να συνδυαστούν και με ανώνυμες σχέσεις με ή χωρίς τύπο.
- *Σχέσεις που επαναλαμβάνονται* και συμβολίζονται ως $-[*m..n]->$. Με αυτόν τον τρόπο, επιτρέπεται μεταξύ δύο κόμβων να υπάρχει ουσιαστικά ένα μονοπάτι που μπορεί να αποτελείται από m έως n βήματα, δηλαδή ακμές. Αν καθορίσουμε τον τύπο της ακμής, τότε το μονοπάτι θα πρέπει να αποτελείται από ακμές του ίδιου τύπου, ενώ αν παραλείψουμε είτε το άνω είτε το κάτω όριο το κριτήριο διαμορφώνεται αντίστοιχα.

Αν και ορίσαμε το συντακτικό για τις σχέσεις προσδιορίζοντας μόνο κατευθυνόμενες σχέσεις, μπορεί να υπάρχουν ερωτήματα στα οποία δεν μας ενδιαφέρει η κατεύθυνση της σχέσης, αλλά μόνο να συνδέονται δύο κόμβοι. Για να υποστηρίξει και αυτά τα ερωτήματα, η Cypher ορίζει και μονοπάτια χωρίς κατεύθυνση, που αντίστοιχα συμβολίζονται ως $-[]-$ ή $--$, και όλες οι αντίστοιχες παραλλαγές που είδαμε παραπάνω. Προσέξτε, αυτό δεν σημαίνει ότι στο γράφημα υπάρχουν μη κατευθυνόμενες σχέσεις, αλλά ότι ένα πρότυπο με μια μη κατευθυνόμενη ακμή μπορεί να ταιριάζει σε μια ακμή άσχετα με την κατεύθυνσή της, αν πληροί τα υπόλοιπα κριτήρια που θέτει το πρότυπο.

Στη συνέχεια, θα δούμε συνοπτικά κάποιες από τις δυνατότητες της γλώσσας και θα εξηγήσουμε τη λειτουργία της μέσα από κάποια παραδείγματα.

Για τα ερωτήματα ανάγνωσης, η βασική δομή είναι της μορφής:

```
MATCH pattern
WHERE conditions
RETURN nodes|edges|paths|properties [ORDER BY] [SKIP] [LIMIT]
```

Μπορεί κανείς εύκολα να διακρίνει την ομοιότητα με την SQL. Ας δούμε αναλυτικά τι κάνει ο κάθε τελεστής, καθώς και κάποιοι επιπλέον τελεστές που δεν ορίσαμε στη βασική δομή.

- **MATCH.** Στο MATCH καθορίζονται ένα ή περισσότερα πρότυπα που θα πρέπει να ταιριάζουν στο γράφημα.
- **OPTIONAL MATCH.** Το OPTIONAL MATCH έχει ακριβώς την ίδια λειτουργία με το MATCH, μόνο που είναι προαιρετικό. Επομένως, εάν δεν προκύψει ταιρίασμα στο γράφημα, τα αντίστοιχα δεδομένα συμπληρώνονται με NULL. Μπορεί να θεωρηθεί ως το ανάλογο της εξωτερικής σύζευξης (outer join) της SQL.
- **WHERE.** Στο WHERE καθορίζονται οι συνθήκες που επιβάλλονται και φιλτράρουν επιπλέον τα δεδομένα που έχουν ταιριάζει στο MATCH.
- **WITH.** Το WITH χρησιμοποιείται για να δημιουργήσει pipelines μεταξύ ερωτημάτων ή τελεστών. Ουσιαστικά, διοχετεύει αποτελέσματα από έναν τελεστή στον επόμενο.
- **RETURN.** Στο RETURN καθορίζονται τα αποτελέσματα που θα επιστρέψει το ερώτημα, και προσφέρει λειτουργικότητα παρόμοια με το SELECT της SQL. Τα αποτελέσματα αυτά μπορεί να είναι σύνολα κόμβων, σύνολα ιδιοτήτων, σύνολα ακμών ή και μονοπάτια.
- **ORDER BY.** Το ORDER BY καθορίζει με βάση ποιο ή ποια πεδία θα ταξινομηθούν τα αποτελέσματα. Μπορεί κανείς να καθορίσει, όπως και στην SQL είτε την επιλογή ASC για αύξουσα ταξινόμηση είτε την DESC για φθίνουσα.
- **SKIP.** Το SKIP παραλείπει ένα πλήθος από αποτελέσματα από το τελικό αποτέλεσμα.
- **LIMIT.** Το LIMIT περιορίζει το πλήθος των αποτελεσμάτων που επιστρέφονται.

Εάν θέλουμε να δομήσουμε ένα πιο πολύπλοκο ερώτημα, τότε η απλή δομή που είδαμε παραπάνω εμπλουτίζεται με τους επιπλέον τελεστές ως:

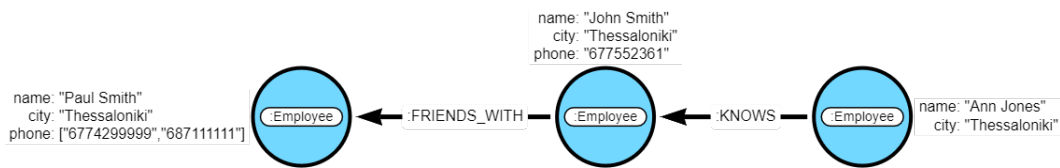
```
MATCH [WHERE]
[OPTIONAL MATCH] [WHERE]
[WITH]
RETURN [ORDER BY] [SKIP] [LIMIT]
```

Αντιστοίχως, για ερωτήματα εγγραφής έχουμε την παρακάτω δομή:

```
(CREATE | MERGE) *
[SET | DELETE | REMOVE | FOREACH] *
RETURN [ORDER BY] [SKIP] [LIMIT]
```

Και οι επιπλέον τελεστές έχουν την παρακάτω λειτουργία:

- **CREATE.** Το CREATE δημιουργεί νέους κόμβους ή σχέσεις.



Σχήμα 4.9: Γράφημα αποτελέσματος για το ερώτημα εργαζόμενοι της Θεσσαλονίκης.

- **MERGE.** Το MERGE αναζητά ένα πρότυπο στο γράφημα και, αν δεν καταφέρει να το ταιριάξει, τότε το δημιουργεί.
- **SET.** Το SET χρησιμοποιείται για την ενημέρωση ετικετών ή ιδιοτήτων κόμβων και σχέσεων.
- **DELETE.** Το DELETE διαγράφει κόμβους ή σχέσεις στο γράφημα.
- **REMOVE.** Το REMOVE διαγράφει ετικέτες ή ιδιότητες από κόμβους και σχέσεις.
- **FOREACH.** Το FOREACH χρησιμοποιείται για να πραγματοποιηθεί μια πράξη εγγραφής (δημιουργία, ενημέρωση ή διαγραφή) στα στοιχεία ενός μονοπατιού ή μιας λίστας που έχει δημιουργηθεί ως αποτέλεσμα κάποιου ερωτήματος.

Εκτός από εντολές ανάγνωσης ή εγγραφής, η Cypher υποστηρίζει και εντολές που συνδυάζουν τα δύο. Πρώτα γίνεται κάποια αναζήτηση ταίριασμα στο γράφημα και μετά ακολουθεί μία ή περισσότερες εγγραφές.

Για να δημιουργηθεί, λοιπόν, το γράφημα που εικονίζεται στο Σχήμα 4.8, μπορούμε να χρησιμοποιήσουμε τις CREATE εντολές που φαίνονται παρακάτω.

```
CREATE (n0:`Employee` {name: "John Smith", city: "Thessaloniki", phone:
  "677552361"}),
(n1:`Employee` {name: "Ann Jones", city: "Thessaloniki"}),
(n2:`Employee` {name: "Paul Smith", city: "Thessaloniki", phone:
  ["6774299999", "6871111111"]}),
(n3:`Employee` {name: "Alice Brown", city: "Athens", email: "abrown@company.
  com"}),
(n4:Department {name: "R&D"}),
(n5:`Department` {name: "Accounting"})
CREATE (n0)-[:WORKS_AT {since: 2015}]->(n4),
(n2)-[:WORKS_AT {position: "Project Manager"}]->(n4),
(n0)-[:FRIENDS_WITH]->(n2),
(n1)-[:KNOWS]->(n0),
(n3)-[:FRIENDS_WITH]->(n1),
(n2)-[:KNOWS]->(n3)-[:WORKS_AT {since: 2009}]->(n5),
(n1)-[:WORKS_AT {since: 2010}]->(n4);
```

Αρχικά, δημιουργούμε τους κόμβους με τις ετικέτες τους και τις ιδιότητές τους, και τους αναθέτουμε σε κάποιες μεταβλητές, που στη συνέχεια χρησιμοποιούμε σε ένα δεύτερο CREATE (αλλά εντός του ίδιου ερωτήματος) για να δημιουργήσουμε και τις ακμές μεταξύ τους με τις αντίστοιχες ιδιότητές τους.

Για την αναζήτηση και ανάκτηση δεδομένων, τα πιο απλά ερωτήματα που μπορούμε να σχηματίσουμε αφορούν αποκλειστικά κόμβους. Για παράδειγμα, εάν θέλουμε να βρούμε τους εργαζομένους που έχουν ως πόλη τη Θεσσαλονίκη, το ερώτημα Cypher είναι:



Σχήμα 4.10: Γράφημα αποτελέσματος για το ερώτημα τμήματα με εργαζομένους από την Αθήνα.

```

MATCH (e:Employee)
WHERE e.city='Thessaloniki'
RETURN e

```

Στο MATCH το πρότυπο που σχηματίζουμε ταιριάζει σε όλους τους κόμβους με ετικέτα *Employee*, οι οποίοι στη συνέχεια φιλτράρονται σύμφωνα με τη συνθήκη *e.city='Thessaloniki'*. Ο συμβολισμός που χρησιμοποιείται για να αναφερθούμε στις ιδιότητες ενός κόμβου ή μιας σχέσης είναι με αναφορά στη μεταβλητή που έχει ανατεθεί στον κόμβο, εδώ το *e*, και μετά “.” και το όνομα της ιδιότητας, εδώ *city*. Στο RETURN προσδιορίζουμε ότι θέλουμε να επιστραφούν οι κόμβοι που έχουν ανατεθεί στο *e* και ικανοποιούν το ερώτημα. Το αποτέλεσμα του ερωτήματος φαίνεται στο Σχήμα 4.9. Προσέξτε, ότι, ενώ στο RETURN έχουμε ζητήσει μόνο τους κόμβους, το αποτέλεσμα της Neo4j είναι ένα γράφημα. Συγκεκριμένα, όταν ζητάμε ένα σύνολο από κόμβους στο αποτέλεσμα, επιστρέφεται το *επαγόμενο υπογράφημα* που συνθέτουν αυτοί οι κόμβοι, δηλαδή το γράφημα που σχηματίζεται από τους κόμβους που καθορίζουμε και όλες τις σχέσεις (ακμές) που υπάρχουν μεταξύ τους στο γράφημα της βάσης.

Πέρα από κόμβους, μπορούμε στα πρότυπα να σχηματίζουμε, όπως είπαμε, και μονοπάτια. Έστω ότι θέλουμε να ανακτήσουμε τα τμήματα στα οποία εργάζονται εργαζόμενοι από την Αθήνα:

```

MATCH (e:Employee)-[:WORKS_AT]->(d:Department)
WHERE e.city='Athens'
RETURN d, e

```

Στο πρότυπο του ερωτήματος αυτού προσδιορίζουμε ότι μας ενδιαφέρουν ζεύγη κόμβων με ετικέτες *Employee* και *Department* που συνδέονται μέσω σχέσης τύπου *:WORKS_AT*. Αυτό το πρότυπο ταιριάζει με όλα τα ζεύγη εργαζομένων τμημάτων, π.χ. ο Paul Smith που εργάζεται στο R&D, η Ann Jones επίσης στο R&D, στο Σχήμα 4.8. Αλλά, με την επιπλέον συνθήκη η πόλη του εργαζομένου να είναι η Αθήνα, το μόνο ζεύγος που ικανοποιεί το κριτήριο είναι το Alice Brown, Accounting. Και επειδή στο αποτέλεσμα ζητάμε και τα τμήματα και τους εργαζομένους που ικανοποιούν τις συνθήκες μας, θα επιστρέφει το υπογράφημα που σχηματίζεται από τον κόμβο της Alice και του Accounting (Σχήμα 4.10).

Ας εξετάσουμε ένα πιο πολύπλοκο πρότυπο, με συνθήκη στις ιδιότητες της σχέσης:

```

MATCH (d:Department)<-[:WORKS_AT]-(e:Employee)-[:KNOWS]-(e1:Employee)
WHERE w.since>2010 and d.name='R&S&D'
RETURN DISTINCT e1.name

```

Στο παραπάνω ερώτημα ζητάμε τους εργαζομένους (*e1*) που γνωρίζουν εργαζόμενοι (*e*) που εργάζονται στο τμήμα (*d*) R&D μετά το 2010. Επειδή θέλουμε και συνθήκη που αφορά ιδιότητα της σχέσης *:WORKS_AT*, αναθέτουμε και τη σχέση σε μια μεταβλητή, *w*. Προσέξτε, επίσης, ότι δεν έχουμε καθορίσει κατεύθυνση στη σχέση *:KNOWS* στο πρότυπο, οπότε ταιριάζει στο γράφημα είτε εάν η ακμή έχει κατεύθυνση από τον *e* στον *e1* είτε το αντίστροφο.

Τέλος, στο RETURN ζητάμε μόνο τα διακριτά ονόματα των εργαζομένων που ταιριάζουν στην *e1*. Εάν το ερώτημα τρέξει πάνω στο γράφημα του παραδείγματός μας, η μόνη απάντηση που θα πάρουμε είναι το Ann Jones. Σε αυτήν την περίπτωση, η Neo4j θα επιστρέψει τα αποτελέσματα σε κειμενική μορφή.

Να σημειώσουμε εδώ, ότι η Neo4j γενικά παρέχει τη δυνατότητα επιστροφής των αποτελεσμάτων τόσο σε μορφή γραφήματος, στη web-based εφαρμογή πελάτη που παρέχει, όσο και σε κείμενο (JSON) ή πίνακα (απλό κείμενο).

Εκτός από το να προσδιορίζουμε συνθήκες σε κάποια σχέση, μπορούμε να προσδιορίζουμε και συνθήκες χωρίς να υπάρχει κάποια σχέση. Για παράδειγμα, έστω ότι ζητούνται οι εργαζόμενοι που δεν έχουν κανέναν φίλο στην εταιρεία:

```
MATCH (e:Employee)
WHERE NOT (e)-[:FRIENDS_WITH]-(:Employee)
RETURN e
```

Το πρότυπο στο MATCH σε αυτό το ερώτημα προσδιορίζει μόνο κόμβους με ετικέτα *Employee*. Στο WHERE, όμως, προσδιορίζουμε με την άρνηση (NOT) ότι δεν πρέπει να υπάρχει ακμή τύπου *:FRIENDS_WITH* που να συνδέει τους εργαζομένους με οποιονδήποτε άλλον κόμβο. Δεν έχει καθοριστεί κατεύθυνση και πάλι στη σχέση, και επιπλέον, επειδή δεν μας αφορά με ποιον θα είναι φίλος ο εργαζόμενός μας, χρησιμοποιούμε έναν ανώνυμο κόμβο.

Εάν θα θέλαμε να προσδιορίσουμε έναν συγκεκριμένο εργαζόμενο, εδώ το ερώτημά μας θα έπρεπε να είναι λίγο διαφορετικό. Έστω ότι ζητούνται οι εργαζόμενοι που δεν είναι φίλοι με τον Paul Smith. Πρέπει να προσέξουμε ότι οι όποιες μεταβλητές θα πρέπει να ορίζονται στο MATCH και δεν είναι δυνατό να οριστεί νέα μεταβλητή στο WHERE. Οπότε, μια προσέγγιση που θα μπορούσαμε να ακολουθήσουμε εδώ είναι η παρακάτω:

```
MATCH (e:Employee), (e1:Employee)
WHERE e1.name='Paul Smith' AND NOT (e)-[:FRIENDS_WITH]-(e1)
RETURN e
```

Σε αυτό το ερώτημα συνδυάζουμε δύο πρότυπα στο MATCH. Γενικά, αυτό είναι αποδεκτό, αλλά απαιτεί προσοχή διότι, εάν τα δύο πρότυπα δεν έχουν κάποιο κοινό στοιχείο, δημιουργούνται όλοι οι δυνατοί συνδυασμοί των προτύπων, στο συγκεκριμένο παράδειγμα όλων των κόμβων, και σε μεγάλα γραφήματα αυτό δεν είναι καθόλου αποδοτικό. Εναλλακτικά, για να αποφύγουμε το καρτεσιανό γινόμενο στο MATCH, μπορούμε να γράψουμε το ερώτημα ως:

```
MATCH (e:Employee)
WHERE NOT (e)-[:FRIENDS_WITH]-(:Employee{name:'Paul Smith'})
RETURN e
```

Σε σχέση με την ομαδοποίηση, η προσέγγιση της Cypher είναι πολύ απλή. Έστω ότι ζητείται το πλήθος των εργαζομένων ανά τμήμα:

```
MATCH (e:Employee)-[:WORKS_AT]->(d:Department)
RETURN d.name, COUNT(*) AS number
ORDER BY number DESC
```

Στο MATCH το πρότυπο απλώς προσδιορίζει τους εργαζομένους μαζί με το τμήμα στο οποίο εργάζονται. Στη συνέχεια, η ομαδοποίηση γίνεται κατευθείαν στο RETURN, όπου ουσιαστικά για κάθε όνομα τμήματος (ή για κάθε τμήμα) μετρώνται τα ταιριάσματα που έχει αυ-

τός ο κόμβος, δηλαδή το τμήμα, σύμφωνα με το MATCH. Αυτά τα ταιριάσματα αντιστοιχούν στους εργαζομένους στο συγκεκριμένο τμήμα. Τελικά, τα αποτελέσματα αυτά ταξινομούνται σε φθίνουσα σειρά με βάση το πλήθος, από το πιο πολυπληθές τμήμα σε αυτό με τους λιγότερους εργαζομένους. Συνεπώς, το αποτέλεσμα του ερωτήματος είναι: (R&D, 3), (Accounting, 1).

Στην περίπτωση που θέλουμε να προσθέσουμε και κάποια συνθήκη σε επίπεδο ομάδας, μπορούμε να το κάνουμε με τη χρήση του WITH. Έστω ότι αλλάζουμε το προηγούμενο ερώτημα και ζητούνται πλέον τα τμήματα τα οποία έχουν τουλάχιστον 3 εργαζομένους.

```
MATCH (e:Employee)-[:WORKS_AT]->(d:Department)
WITH d, COUNT(*) AS number
WHERE number>=3
RETURN d.name, number
ORDER BY number DESC
```

Το πρότυπο στο MATCH δεν αλλάζει, και στο WITH πραγματοποιείται η ομαδοποίηση και το μέτρημα που είχαμε στο προηγούμενο ερώτημα στο RETURN. Όμως, το WITH δεν επιστρέφει τα αποτελέσματα όπως το RETURN, αλλά τα διοχετεύει σε ένα επόμενο ερώτημα ή τελεστή, στη συγκεκριμένη περίπτωση στο WHERE, όπου μπορεί πλέον να ελεγχθεί η συνθήκη για το πλήθος των εργαζομένων ανά τμήμα. Στη συνέχεια, το RETURN και το ORDER BY είναι όμοια με το προηγούμενο παράδειγμα, αλλά πλέον το μόνο αποτέλεσμα που επιστρέφεται είναι το (R&D, 3).

Πέρα από τα ερωτήματα ανάκτησης, παρέχονται, όπως είδαμε, και ερωτήματα ενημέρωσης και διαγραφής. Έστω π.χ. ότι θα πρέπει να προστεθεί μια ιδιότητα *email* στον Paul Smith. Το ερώτημα θα πρέπει πρώτα να ταιριάξει τον κόμβο που αντιστοιχεί στον συγκεκριμένο εργαζόμενο και μετά να προσθέσει την ιδιότητα και να της δώσει τιμή.

```
MATCH (e:Employee)
WHERE e.name='Paul Smith'
SET e.email='psmith@company.com'
```

Με αντίστοιχο τρόπο μπορεί να προστεθεί και ετικέτα σε κάποιους κόμβους, όπως στο παρακάτω παράδειγμα, στο οποίο σε όλους τους εργαζομένους προστίθεται η ετικέτα *Person*.

```
MATCH (e:Employee)
SET e:Person
```

Για τη διαγραφή ιδιότητας ή ετικέτας στα παραπάνω ερωτήματα θα αντικαθιστούμε το SET με το REMOVE.

Ας δούμε, τέλος, και τις διαγραφές κόμβων και ακμών όπου χρησιμοποιείται το DELETE. Για να διαγραφούν όλοι οι κόμβοι δεν αρκεί μετά το MATCH να θέσουμε DELETE *e*. Μια τέτοια εντολή θα επέστρεφε μήνυμα λάθους, και ο λόγος είναι οι ακμές που ξεκινούν ή καταλήγουν στους συγκεκριμένους κόμβους. Η Neo4j αντιμετωπίζει τις σχέσεις ως περιορισμούς αναφορικής ακεραιότητας, το οποίο είναι λογικό διότι, εάν μετατρέψει κανείς ένα σχεσιακό μοντέλο στο αντίστοιχο γράφημα ιδιοτήτων, οι περιορισμοί ξένων κλειδιών θα υποδείξουν τις σχέσεις που θα δημιουργηθούν μεταξύ των οντοτήτων. Επομένως, για να πραγματοποιηθεί η διαγραφή ενός κόμβου, πρέπει πρώτα να *αποσυνδεθεί* (DETACH) από τις ακμές του. Άρα, το ορθό ερώτημα διαγραφής είναι:

```
MATCH (n)
DETACH
DELETE n
```

Σίγουρα, δεν έχουν καλυφθεί όλες οι δυνατότητες της Cypher, αλλά προσπαθήσαμε μέσα από τη μικρή αυτή εισαγωγή να καταδείξουμε τις δυνατότητες που παρέχει αλλά και τη βασική ιδέα πίσω από τα πρότυπα στα οποία στηρίζεται. Μάλιστα, η γλώσσα *GQL* (*Graph Query Language*) [GQL] η οποία αναπτύσσεται ως το πρότυπο για τις γλώσσες ερωτημάτων γραφημάτων βρίσκει στην Cypher το πιο κοντινό της υλοποιημένο παράδειγμα, και τα συστήματα βάσεων δεδομένων γραφημάτων γίνονται ολοένα και πιο δημοφιλή, με εφαρμογές ιδιαίτερα στην ανάλυση δεδομένων.

4.6 Συστήματα διαχείρισης ετερογενών δεδομένων

Όπως έχουμε ήδη τονίσει, στα βασικά χαρακτηριστικά των μεγάλων δεδομένων ανήκει η ποικιλία ή ποικιλομορφία και ετερογένεια. Μέσα από την απαρίθμηση των διαφορετικών συστημάτων που χρησιμοποιούν διαφορετικά μοντέλα για τα δεδομένα, είδαμε ότι «ταιριάζουν» σε κάποια από τα δεδομένα ενός συστήματος και τονίσαμε ότι κάποια συστήματα είναι πιο κατάλληλα για κάποιες εφαρμογές και κάποια για κάποιες άλλες. Οπότε, αναλόγως με την εφαρμογή που θέλει να αναπτύξει κανείς και τα δεδομένα που θα διαχειρίζεται η εφαρμογή, επιλέγει το κατάλληλο μοντέλο.

Τι συμβαίνει, όμως, εάν για τον ίδιο στόχο, οργανισμό ή επιχείρηση και τις εφαρμογές του θα θέλαμε ιδανικά να διαχειριζόμαστε και να συνδυάζουμε πολλές διαφορετικές μορφές δεδομένων, δομημένα, ημιδομημένα και αδόμητα, χωρίς να πρέπει απαραίτητα να τα μετασχηματίζουμε ώστε να συμμορφώνονται με τις απαιτήσεις μιας βάσης δεδομένων και του μοντέλου που χρησιμοποιεί, είτε πρόκειται για το σχεσιακό μοντέλο είτε για μια βάση εγγράφων, ή γραφημάτων, κ.ο.κ.; Άλλωστε, ένας τέτοιος μετασχηματισμός είναι πολύ πιθανό να φέρει αντίστροφα από τα επιθυμητά αποτελέσματα, εφόσον τα δεδομένα πλέον δεν θα αποθηκεύονται με τον τρόπο που θα επιτρέπει την πιο αποδοτική τους επεξεργασία, αφού ένα κοινό μοντέλο δεν μπορεί να είναι το βέλτιστο για όλες τις περιπτώσεις.

Ας θεωρήσουμε, για παράδειγμα, μια εφαρμογή ηλεκτρονικού εμπορίου κάποιου ηλεκτρονικού καταστήματος που έχει να διαχειριστεί: σχεσιακά δεδομένα που αντιστοιχούν στα στοιχεία των πελατών της, δεδομένα σε δομή γραφήματος που αντιστοιχούν σε πληροφορία από κοινωνικά δίκτυα, ιεραρχικά δεδομένα που αντιστοιχούν στον κατάλογο και τις κατηγορίες των προϊόντων του καταστήματος, καθώς και κειμενικά δεδομένα που αντιστοιχούν στις κριτικές των πελατών.

Για το κάθε ένα από τα είδη δεδομένων που πραγματεύεται η εφαρμογή, πιθανώς θα γινόταν διαφορετική επιλογή για το ποιο είναι το καταλληλότερο μοντέλο και, κατά συνέπεια, σύστημα διαχείρισης δεδομένων για την αποθήκευση και επεξεργασία τους. Για παράδειγμα, τα δεδομένα των πελατών θα ταίριαζε να αποθηκευτούν σε μια σχεσιακή βάση, ενώ τα δεδομένα του κοινωνικού δικτύου σε μια βάση δεδομένων γραφημάτων.

Πώς θα πετύχουμε, ωστόσο, μια ενιαία αντιμετώπιση; Μια πρώτη επιλογή είναι να χρησιμοποιηθεί ένα σύστημα διαχείρισης δεδομένων στο οποίο θα ενσωματώσουμε όλα τα δε-

δομένα (μετά την κατάλληλη μοντελοποίηση). Στο προηγούμενο παράδειγμα μπορούμε να επιλέξουμε το σχεσιακό μοντέλο, αλλά τότε δεν θα έχουμε την καλύτερη απόδοση όταν θα επεξεργαζόμαστε τα δεδομένα του κοινωνικού γραφήματος. Ενώ μπορούμε να μετασχηματίσουμε τα δεδομένα του κοινωνικού δικτύου σε σχεσιακά δεδομένα, άλλα ερωτήματα πλοήγησης στο γράφημα θα εκτελούνται πολύ λιγότερο αποδοτικά από το εάν είχαμε επιλέξει μια βάση δεδομένων γραφημάτων για αυτά. Ωστόσο, για τις πληροφορίες των πελατών η σχεσιακή βάση είναι αυτή που θα μας προσέφερε αντίστοιχα την πιο αποδοτική λύση.

Συνεπώς, αυτή η προσέγγιση δεν είναι ικανοποιητική, οπότε κανείς μπορεί να ακολουθήσει δύο εναλλακτικές: είτε να χρησιμοποιήσει μια βάση δεδομένων που υποστηρίζει πολλά μοντέλα (multi-model database) είτε ένα πολυ-αποθηκευτικό σύστημα διαχείρισης (polystore). Στις επόμενες ενότητες, θα περιγράψουμε σύντομα τις δύο αυτές λύσεις.

4.6.1 Multi-model database

Στην περίπτωση μιας multi-model database, έχουμε ένα ενιαίο σύστημα διαχείρισης δεδομένων το οποίο, όμως, υποστηρίζει την αποθήκευση δεδομένων που ανήκουν σε διαφορετικά μοντέλα δεδομένων. Η ίδια βάση δεδομένων, δηλαδή, είναι σε θέση να διαχειρίζεται, για παράδειγμα, και σχεσιακά δεδομένα, και κειμενικά δεδομένα, αλλά και ημιδομημένα δεδομένα σε μορφή XML ή και RDF, δηλαδή και ιεραρχίες και γραφήματα.

Όλα τα μοντέλα δεδομένων που υποστηρίζουν τα NoSQL συστήματα διαχείρισης δεδομένων που έχουμε αναφέρει έως τώρα μπορούν να υποστηριχτούν στο πλαίσιο μιας βάσης δεδομένων πολλαπλών μοντέλων. Μάλιστα, η τάση που έχει επικρατήσει είναι τα δημοφιλέστερα συστήματα διαχείρισης βάσεων δεδομένων να προσφέρουν πλέον υποστήριξη για πολλαπλά μοντέλα δεδομένων. Η Oracle, για παράδειγμα, στην έκδοση Oracle 19C [Ora19], παρέχει μια multi-model database η οποία υποστηρίζει ένα εύρος από διαφορετικά μοντέλα δεδομένων, τύπους και δομές δεδομένων και παρέχει επίσης τελεστές για την επεξεργασία και την ανάλυσή τους. Συγκεκριμένα, υποστηρίζει σχεσιακά δεδομένα, δεδομένα στην κύρια μνήμη, θρυμματισμένα δεδομένα, δεδομένα στη μορφή εγγράφων, και συγκεκριμένα XML, JSON και έγγραφα κειμένου, χωρικά δεδομένα και δεδομένα γραφημάτων ή σε τριπλέτες (RDF). Παράλληλα, υποστηρίζει τελεστές και APIs για κάθε τέτοιο τύπο δεδομένων, καθώς και αντίστοιχες γλώσσες περιγραφής και επεξεργασίας των δεδομένων. Για παράδειγμα, υπάρχει υποστήριξη SPARQL 1.1 για την αναπαράσταση απλών και σύνθετων εκφράσεων μονοπατιών, αλλά και ενσωματωμένη υλοποίηση της XQuery για να υποστηρίζονται ερωτήματα σε XML δεδομένα.

Το επιχείρημα υπέρ αυτών των προσεγγίσεων είναι ότι επιτυγχάνεται συνολικά η πιο αποδοτική διαχείριση των διαφορετικών δεδομένων λαμβάνοντας υπόψη τα διαφορετικά τους χαρακτηριστικά, αλλά κάτω από ένα κοινό σύστημα που προσφέρει ενιαία υποστήριξη συνέπειας, ασφάλειας, διαθεσιμότητας και απόδοσης.

4.6.2 Polystores

Στον αντίποδα των multi-model databases, και με βάση το επιχείρημα «Μια λύση δεν ταιριάζει σε όλα», έχουμε την εναλλακτική των polystores.

Το polystore αποτελεί μια προσέγγιση στην οποία χρησιμοποιούνται από κοινού διαφορετικές τεχνολογίες και μοντέλα αποθήκευσης και διαχείρισης δεδομένων, αναλόγως με τις απαιτήσεις των εφαρμογών που τα αξιοποιούν. Αποτελούν μια επέκταση των παράλληλων και των ομοσπονδιακών βάσεων δεδομένων (federated databases). Ωστόσο, αντιθέτως με τα πρώτα αυτά συστήματα, τα polystores δεν απαιτούν την ενοποίηση των δεδομένων που διαχειρίζονται κάτω από ένα κοινό μοντέλο.

Τα polystores σχεδιάστηκαν για να αποτελέσουν μια λύση στο πρόβλημα της ετερογένειας, δίνοντας τη δυνατότητα τα δεδομένα κάθε τύπου να αποθηκεύονται με τον βέλτιστο για αυτά τα δεδομένα τρόπο και επιπλέον να γίνεται και η επεξεργασία τους ή και η ανάλυσή τους χρησιμοποιώντας και πάλι το βέλτιστο μοντέλο επεξεργασίας. Επομένως, είναι σε θέση να υποστηρίζουν ταυτόχρονα διαφορετικά συστήματα διαχείρισης δεδομένων, καθώς και διαφορετικά προγραμματιστικά μοντέλα και μοντέλα δεδομένων. Ουσιαστικά, τα polystores προσφέρουν μια κοινή διεπαφή για τα διαφορετικά αποθηκευτικά και προγραμματιστικά μοντέλα που υλοποιούνται σε ένα χαμηλότερο επίπεδο, ενώ παράλληλα πραγματοποιούν και την ενορχήστρωσή τους.

Συνήθως, τα polystores βασίζονται σε κάποιο επίπεδο ενδιάμεσου λογισμικού (middleware) που έχει τον ρόλο της ενοποίησης των δεδομένων από τις διαφορετικές πηγές και τη χρήση wrappers για κάποια από τα διαφορετικά συστήματα. Η επεξεργασία ερωτημάτων μπορεί να γίνει απευθείας σε κάποια από τα υποκείμενα συστήματα, ενώ χρησιμοποιούνται wrappers για κάποια άλλα.

Θα περιγράψουμε σε μεγαλύτερο βαθμό ένα από αυτά τα συστήματα, το BigDAWG [Big, DES⁺15], το οποίο αποτελεί ένα αντιπροσωπευτικό παράδειγμα μιας polystore αρχιτεκτονικής, για να γίνουν κατανοητά τα βασικά της χαρακτηριστικά.

Το BigDAWG υποστηρίζει μια κοινή διεπαφή για τα υποκείμενα συστήματα διαχείρισης δεδομένων που προσφέρει τα εξής χαρακτηριστικά: α) διαφάνεια τοποθεσίας, β) σημασιολογική πληρότητα, και γ) δυνατότητα ολιστικής προσπέλασης δεδομένων.

Η διαφάνεια τοποθεσίας σημαίνει ότι το polystore κρύβει από τον χρήστη το πού είναι πραγματικά αποθηκευμένα τα δεδομένα. Συνεπώς, οι χρήστες μπορούν να θέτουν ερωτήματα χρησιμοποιώντας κάποια δηλωτική γλώσσα, τα οποία θα προσπελάσουν δεδομένα σε διαφορετικά συστήματα διαχείρισης δεδομένων, χωρίς να γνωρίζουν τις λεπτομέρειες σχετικά με το πού και σε ποιο σύστημα διαχείρισης δεδομένων βρίσκονται τα ζητούμενα δεδομένα ή και για το ποια μηχανή εκτέλεσης θα πρέπει να τα επεξεργαστεί.

Η ιδιότητα αυτή του BigDAWG βασίζεται στην κύρια μονάδα αφαίρεσης που χρησιμοποιεί, την έννοια του *island of information* που αναφέρεται στο σύνολο των βάσεων δεδομένων που θα προσπελαστούν μέσω της ίδιας γλώσσας ερωτημάτων.

Η σημασιολογική πληρότητα σημαίνει ότι τα υποκείμενα συστήματα διαχείρισης δεδομένων που βρίσκονται κάτω από το polystore δεν χάνουν καθόλου από τη λειτουργικότητα που προσφέρουν και μπορούν να παρέχουν στους χρήστες τους. Ο ορισμός πολλαπλών islands of information επιτρέπει στο BigDawg να ικανοποιεί αυτήν την ιδιότητα, έτσι ώστε η λειτουργικότητα που παρέχει τελικά στους χρήστες του να αποτελεί την ένωση της λειτουργικότητας που παρέχουν τα υποκείμενα συστήματα.

Η τελευταία ιδιότητα έχει ως στόχο οι χρήστες να μπορούν να έχουν προσπέλαση σε δεδο-

μένα που είναι αποθηκευμένα σε ένα από τα υποκείμενα συστήματα διαχείρισης δεδομένων μέσα από πολλαπλές νήσους πληροφορίας, δηλαδή μέσα από διαφορετικές δηλωτικές γλώσσες ερωτημάτων.

Στην τρέχουσα διανομή του το BigDAWG υποστηρίζει τρεις βάσεις δεδομένων, τη σχεσιακή βάση PostgreSQL, την SciDB [Riv17], μια βάση δεδομένων που βασίζεται σε πίνακες και χρησιμοποιείται για επιστημονικά και γεωχωρικά δεδομένα, και την Accumulo [acc], ένα key-value store.

4.6.3 Σύγκριση

Τόσο οι multi-model databases όσο και τα polystores σχεδιάστηκαν για να αντιμετωπίσουν την ετερογένεια των δεδομένων που διαχειρίζονται κοινές εφαρμογές.

Στα κοινά τους χαρακτηριστικά μπορούμε να διακρίνουμε την υποστήριξη πολλαπλών μοντέλων δεδομένων και την υποστήριξη για καθολική επεξεργασία ερωτημάτων. Ωστόσο, οι δύο αυτές προσεγγίσεις έχουν και κάποιες σημαντικές διαφορές, με πιο σημαντική ότι, ενώ οι multi-model databases αποτελούν ένα ενιαίο σύστημα διαχείρισης δεδομένων, μια μηχανή, ένα backend, τα polystores αποτελούν μια σύνθεση από πολλαπλά αυτόνομα συστήματα διαχείρισης δεδομένων. Ως συνέπεια, η υποστήριξη καθολικών συναλλαγών και ενημερώσεων επιτυγχάνεται μόνο από τις multi-model databases, ενώ δεν υποστηρίζονται καθολικά ακόμα από τα polystores. Από την άλλη, η μετεγκατάσταση δεδομένων αποτελεί πολύ πιο απλή διαδικασία στα polystores σε σχέση με τις multi-model databases.

Γενικότερα, οι multi-model databases αποτελούν ένα εμπορικό προϊόν και βρίσκονται πλέον σε ένα ώριμο στάδιο ανάπτυξης, ενώ τα polystores βρίσκονται ακόμα σε εξέλιξη. Το μόνο βέβαιο είναι ότι η απαίτηση για ενιαία διαχείριση ετερογενών ποικίλων δεδομένων δεν θα πάψει και ανάλογες λύσεις θα συνεχίσουν να αναπτύσσονται για να αντεπεξέλθουν σε αυτήν την ποικιλομορφία των δεδομένων, αλλά και στις διαφορετικές ανάγκες ανάλυσης και επεξεργασίας που ανακύπτουν.

4.7 Περαιτέρω μελέτη

Στο κεφάλαιο αυτό περιγράψαμε τα NoSQL συστήματα βάσεων δεδομένων. Περιγράψαμε σύντομα τα βασικά τους χαρακτηριστικά και το πώς αντιμετωπίζουν θέματα όπως η μοντελοποίηση δεδομένων, η συνέπεια δεδομένων, η αναπαραγωγή και η διαμέριση δεδομένων. Στη συνέχεια, αναφερθήκαμε στις τέσσερις βασικές κατηγορίες των NoSQL συστημάτων και χρησιμοποιήσαμε ένα ενδεικτικό παράδειγμα για κάθε κατηγορία. Ωστόσο, αν και ο στόχος ήταν να παρέχουμε για κάθε κατηγορία συστημάτων μια ολοκληρωμένη εικόνα του πώς αντιμετωπίζει τα ζητήματα που είναι καίρια στη σχεδίαση ενός συστήματος διαχείρισης βάσεων δεδομένων, σε ένα κεφάλαιο δεν μπορούμε να συμπεριλάβουμε όλη τη σχετική πληροφορία, όταν υπάρχουν ολόκληρα βιβλία που καλύπτουν μόνο ένα σύστημα από τους εκπροσώπους μίας κατηγορίας.

Συνεπώς, ο αναγνώστης που ενδιαφέρεται για τις NoSQL βάσεις δεδομένων μπορεί να βρει πολύ χρήσιμα τα βιβλία [Har15], [SF13] και [MK19], που καλύπτουν αναλυτικά όλα τα

ζητήματα σχεδίασης ενός NoSQL συστήματος, αλλά και αναλυτικές αναφορές στις τέσσερις βασικές κατηγορίες, καθώς και εκτενείς συγκρίσεις με τα σχεσιακά συστήματα διαχείρισης βάσεων δεδομένων.

Πέρα από τη γενική εικόνα, όποιος αναγνώστης θα ήθελε να εμβαθύνει σε κάποια από τις βασικές κατηγορίες και τα αντιπροσωπευτικά συστήματα που παρουσιάσαμε μπορεί να ανατρέξει για key-value stores και το Riak στο [Mey11], για column-family stores και το Cassandra στα [CH22], [Bro15] και [Nee15], για βάσεις δεδομένων εγγράφων και τη MongoDB στα [BBC19], [PAHN20], και [MPHH10], και για βάσεις δεδομένων γραφημάτων και τη Neo4j στα [RWE15], [Goe15] και [VB14], αλλά και ειδικότερα για τη γλώσσα Cypher στο [Ant22], και την ανάλυση γραφημάτων μέσω της Neo4j και του Spark στο [NH19].

Επίσης, για να συμπληρώσει ο αναγνώστης τις γνώσεις του για τα συστήματα διαχείρισης βάσεων δεδομένων και ειδικότερα των παραδοσιακών σχεσιακών συστημάτων διαχείρισης δεδομένων, μπορεί να ανατρέξει σε πολλά βιβλία, όπως στο [RGG03] και στο [ENEN15].

Τέλος, για την πιο εκτενή μελέτη των ζητημάτων του ταυτοχρονισμού, της συνέπειας των δεδομένων αλλά και της αναπαραγωγής και της διαμέρισης, όπως αντιμετωπίστηκαν αρχικά από τις σχεσιακές κατανεμημένες βάσεις δεδομένων, και άλλα σχετικά θέματα για τις κατανεμημένες βάσεις που θα βοηθήσουν τον αναγνώστη να σχηματίσει μια πιο ολοκληρωμένη εικόνα για τις εναλλακτικές που υπάρχουν στη σχεδίαση κατανεμημένων λύσεων διαχείρισης δεδομένων, μπορεί να βρει κανείς στο [ÖV⁺99] αλλά και στα πιο πρόσφατα [R⁺09] και [RH10] που καλύπτουν σε βάθος όλα τα σχετικά θέματα.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [acc] Apache accumulo. <https://accumulo.apache.org/>.
- [Ant22] Ravindranatha Anthapu. *Graph Data Processing with Cypher: A practical guide to building graph traversal queries using the Cypher syntax on Neo4j*. Packt Publishing, 2022.
- [BBC19] Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow. *MongoDB: the definitive guide: powerful and scalable data storage*. O'Reilly Media, 2019.
- [Ber] Oracle berkeley db. <https://www.oracle.com/in/database/technologies/related/berkeleydb.html>.
- [Big] Bigdawg polystore. <https://bigdawg.mit.edu/>.
- [Blo] Neo4j bloom. <https://neo4j.com/product/bloom/>.
- [Bre12] Eric Brewer. Cap twelve years later: How the "rules" have changed. *Computer*, 45(2):23–29, 2012.
- [Bro15] Mat Brown. *Learning Apache Cassandra*. Packt Publishing Ltd, 2015.
- [CH22] Jeff Carpenter and Eben Hewitt. *Cassandra: The Definitive Guide,(Revised)*. "O'Reilly Media, Inc.", 2022.

- [Com] Mongo compass. <https://www.mongodb.com/products/tools/compass>.
- [Cyp] Neo4j cypher query language. <https://neo4j.com/docs/getting-started/cypher-intro/>.
- [DES⁺15] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. The big-dawg polystore system. *SIGMOD Rec.*, 44(2):11–16, aug 2015.
- [ENEN15] R. Elmasri, S.B. Navathe, R. Elmasri, and S.B. Navathe. Fundamentals of database systems. In *Advances in Databases and Information Systems*, page 139. Springer, 2015.
- [Fit04] Brad Fitzpatrick. Distributed caching with memcached. *Linux journal*, 2004(124):5, 2004.
- [GDS] Neo4j graph data science library. <https://neo4j.com/docs/graph-data-science/current/>.
- [Geo11] Lars George. *HBase: the definitive guide: random access to your planet-size data.* ” O’Reilly Media, Inc.”, 2011.
- [GL02] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, 2002.
- [Goe15] Ankur Goel. *Neo4j cookbook*. Packt Publishing Ltd, 2015.
- [GQL] Graph query language, gql. <https://www.gqlstandards.org/>.
- [Har15] Guy Harrison. *Next Generation Databases: NoSQLand Big Data*. Apress, 2015.
- [LM10] Avinash Lakshman and Prashant Malik. Cassandra: A decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, 44(2):35–40, 2010.
- [Mey11] Mathias Meyer. Riak handbook. Technical report, Technical report, Basho, 2011.
- [MK19] Andreas Meier and Michael Kaufmann. *SQL & NoSQL databases*. Springer, 2019.
- [Mona] Mongo shell documentation. <https://www.mongodb.com/docs/mongodb-shell/>.
- [MONb] MongoDB. <https://www.mongodb.com/>.
- [MPHH10] Peter Membrey, Eelco Plugge, DUPTim Hawkins, and Tim Hawkins. *The definitive guide to MongoDB: the noSQL database for cloud and desktop computing*. Springer, 2010.
- [n10] n10s, neosemantics. <https://neo4j.com/labs/neosemantics/>.
- [Nee15] Nishant Neeraj. *Mastering Apache Cassandra*. Packt Publishing Ltd, 2015.

- [NH19] Mark Needham and Amy E Hodler. *Graph algorithms: practical examples in Apache Spark and Neo4j*. O'Reilly Media, 2019.
- [Ora19] Oracle. Multimodel database. Technical report, 2019.
- [ÖV⁺99] M Tamer Özsu, Patrick Valduriez, et al. *Principles of distributed database systems*, volume 2. Springer, 1999.
- [PAHN20] Amit Phaltankar, Juned Ahsan, Michael Harrison, and Liviu Nedov. *MongoDB Fundamentals: A hands-on guide to using MongoDB and Atlas in the real world*. Packt Publishing Ltd, 2020.
- [R⁺09] Chhanda Ray et al. *Distributed database systems*. Pearson Education India, 2009.
- [RGG03] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.
- [RH10] Saeed K Rahimi and Frank S Haug. *Distributed database management systems: A Practical Approach*. John Wiley & Sons, 2010.
- [Ria] Riak. <https://riak.com/index.html>.
- [Riv17] Jonathan Rivers. Scidb: An array-native computational database for heterogeneous, multi-dimensional data sets. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 3206–3210, 2017.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. " O'Reilly Media, Inc.", 2015.
- [SF13] Pramod J Sadalage and Martin Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.
- [sim] Amazon simpledb. <https://aws.amazon.com/simpledb/>.
- [Siv12] Swaminathan Sivasubramanian. Amazon dynamodb: a seamlessly scalable non-relational database service. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 729–730, 2012.
- [sys] Neo4j Data Graph Platform. <https://neo4j.com/>. Accessed: 2022-02-20.
- [VB14] Rik Van Bruggen. *Learning Neo4j*. Packt Publishing Ltd, 2014.

ΕΙΣΑΓΩΓΗ ΣΤΟ ΣΥΣΤΗΜΑ ΑΡΑΧΗ SPARK

Περιεχόμενα

5.1	Εισαγωγή	122
5.2	Η Μεγάλη εικόνα	123
5.3	RDDs, DataFrames και DataSets	124
5.4	Διεργασία οδήγησης, εκτελεστές και διαμερίσεις	125
5.5	Εγκατάσταση και χρήση	128
5.5.1	Εφαρμογές Spark με τη γλώσσα Scala	131
5.5.2	Εφαρμογές Spark με τη γλώσσα Python	137
5.6	Μετασχηματισμοί και ενέργειες	139
5.6.1	Μετασχηματισμοί	139
5.6.2	Ενέργειες	144
5.7	Επίβλεψη εκτέλεσης εφαρμογών Spark	145
5.8	Περαιτέρω μελέτη	147
	BIBΛΙΟΓΡΑΦΙΑ	147



Σύνοψη

Σε αυτό το κεφάλαιο παρουσιάζουμε τα κύρια συστατικά στοιχεία του Spark και εξετάζουμε τον τρόπο εκτέλεσης των εφαρμογών. Επίσης, εντοπίζουμε τους βασικούς παίκτες: τη διεργασία οδήγησης, τους εκτελεστές και τις διαμερίσεις. Στη συνέχεια, παρουσιάζουμε τον τρόπο εκτέλεσης εφαρμογών Spark και πώς μπορούμε να δουλέψουμε με τη γλώσσα Scala ή τη γλώσσα Python. Είναι προφανές ότι σε ένα μόνο κεφάλαιο δεν μπορούμε να καλύψουμε όλες τις λεπτομέρειες και, επομένως, ο ενδιαφερόμενος αναγνώστης θα πρέπει να ανατρέξει σε βιβλιογραφικές αναφορές που εστιάζουν στο Spark και μελετούν το σύστημα σε βάθος.

Προαπαιτούμενη γνώση

Το κεφάλαιο αποτελεί μια εισαγωγή στο σύστημα Apache Spark και δεν απαιτούνται ιδιαίτερες γνώσεις. Ωστόσο, πάντα βοηθούν μερικές βασικές γνώσεις προγραμματισμού.

5.1 Εισαγωγή

Τα μεγάλα δεδομένα (big data) είναι παντού! Σήμερα, βιώνουμε μια εκθετική αύξηση του μεγέθους των δεδομένων που συλλέγονται και υποβάλλονται σε επεξεργασία, καθώς και μια αυξανόμενη πολυπλοκότητα των εργασιών που πρέπει να εφαρμοστούν σε αυτά τα δεδομένα. Στην πραγματικότητα, ο όρος *big data* τείνει να γίνει συνώνυμος του όρου *data*. Επί του παρόντος, η συντριπτική πλειονότητα των εφαρμογών απαιτεί την ανάλυση μεγάλων, σύνθετων και εξελισσόμενων συνόλων δεδομένων. Για να μπορέσουμε να αντιμετωπίσουμε αυτές τις προκλήσεις, χρειαζόμαστε κάποια πραγματικά ισχυρά όπλα. Ένα από αυτά τα όπλα είναι το σύστημα Apache Spark.

Το σύστημα Apache Spark αποτελεί μια ενοποιημένη, κατανεμημένη μηχανή επεξεργασίας δεδομένων. Η τελευταία έκδοσή του είναι η 3.5.4. Λαμβάνοντας υπόψη ότι η έκδοση 1.0 κυκλοφόρησε το 2014, μπορεί κανείς να παρατηρήσει ότι υπάρχει μια πολύ μεγάλη δραστηριότητα σε αυτό το έργο. Σε αυτό το κεφάλαιο, θα παρουσιάσουμε το σύστημα Apache Spark περιγράφοντας εν συντομία τα κύρια χαρακτηριστικά του, τις βιβλιοθήκες που προσφέρονται και τα APIs που υποστηρίζει.

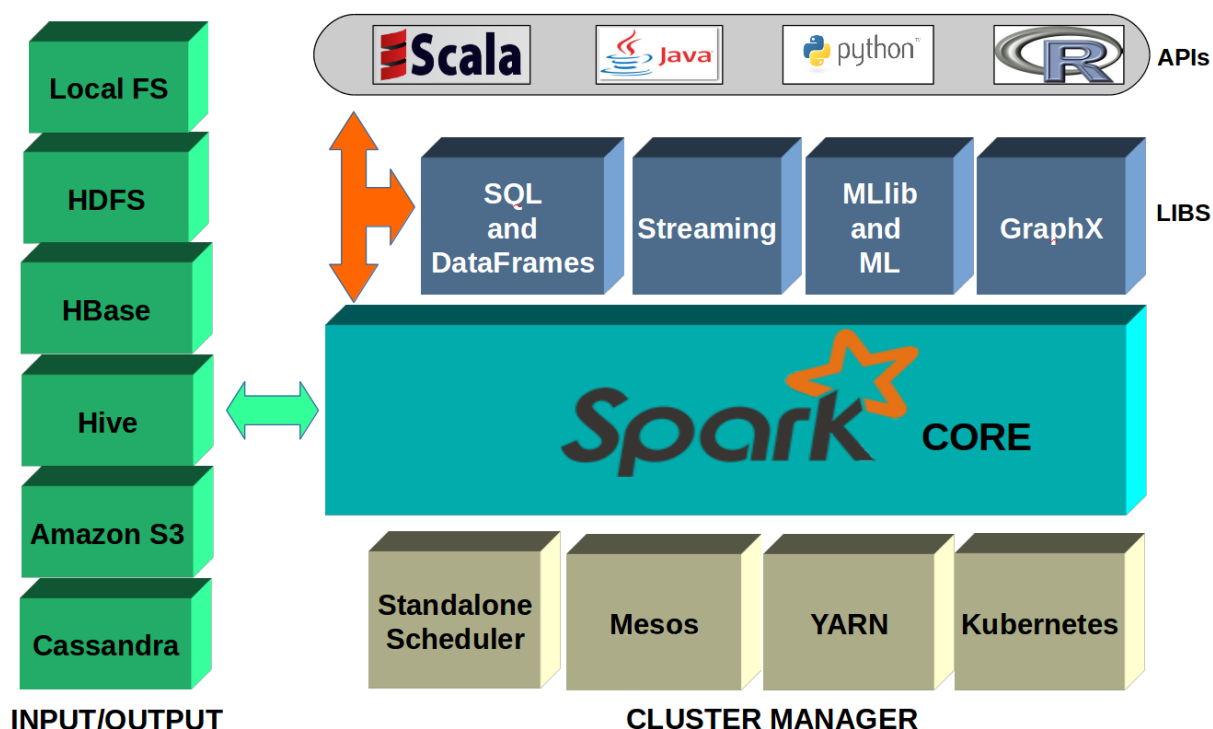
Το Spark έχει δημιουργηθεί για να υποστηρίξει την ανάπτυξη εφαρμογών που απαιτούν την επεξεργασία και την ανάλυση δεδομένων μεγάλου όγκου. Αυτά τα μεγάλα σύνολα δεδομένων μπορούν να δημιουργηθούν από διαφορετικούς τομείς εφαρμογών. Για παράδειγμα, τα δεδομένα μπορεί να αντιστοιχούν σε αρχεία καταγραφής, μηνύματα email, έγγραφα κειμένου σε XML, μετρήσεις αισθητήρα, σχέσεις φίλων σε ένα κοινωνικό δίκτυο, δομημένα δεδομένα από βάσεις δεδομένων, ακολουθίες DNA και πολλά άλλα. Το πλούσιο σύνολο εργαλείων που περιλαμβάνονται στο Spark καθώς και η δυνατότητα επεξεργασίας διαφορετικών τύπων δεδομένων δικαιολογούν απόλυτα τον όρο *unified*, που σημαίνει ότι δεν χρειάζεται να εκτελεστούν υπολογιστικά ακριβές λειτουργίες ETL (Extract-Transform-Load), δεδομένου ότι τα δεδομένα, στις περισσότερες περιπτώσεις, μπορούν να υποβληθούν σε επεξεργασία εντός του Spark, χωρίς την ανάγκη χρήσης εξωτερικών εργαλείων. Ωστόσο, αυτό δεν σημαίνει ότι

δεν μπορούν να χρησιμοποιηθούν εξωτερικά εργαλεία, εάν αυτό κριθεί απαραίτητο.

5.2 Η Μεγάλη εικόνα

Ένα από τα πιο σημαντικά χαρακτηριστικά του Spark είναι ότι υποστηρίζει διαφορετικές γλώσσες προγραμματισμού, όπως Scala, Java, Python και R. Αυτό σημαίνει ότι, ανάλογα με τις επιλογές του προγραμματιστή, μπορούμε να επιλέξουμε τη γλώσσα που θέλουμε να χρησιμοποιήσουμε. Επιπλέον, υποστηρίζει διαφορετικούς διαχειριστές πόρων (resource managers): standalone, YARN, Mesos, Kubernetes. Παρατηρούμε ότι υπάρχει υψηλός βαθμός ευελιξίας στη ρύθμιση του περιβάλλοντος επιλογής για συνεργασία με το Spark. Μία από τις πιο ευρέως χρησιμοποιούμενες ρυθμίσεις είναι η χρήση HDFS για αποθήκευση, του YARN ως διαχειριστή πόρων και του Spark ως της μηχανής διαχείρισης και ανάλυσης δεδομένων. Ωστόσο, είναι δυνατές και άλλες εναλλακτικές λύσεις.

Ένα άλλο εξαιρετικό χαρακτηριστικό του Spark είναι ότι υποστηρίζει πολλούς διαφορετικούς τρόπους φόρτωσης και αποθήκευσης δεδομένων. Για παράδειγμα, μπορεί κανείς να επεξεργαστεί δεδομένα που είναι αποθηκευμένα στα υποσυστήματα: HDFS, τοπικό σύστημα αρχείων, HBase, Hive, Cassandra, κ.λπ. Αυτό είναι ένα εξαιρετικό χαρακτηριστικό, δεδομένου ότι επιτρέπει στο Spark να έχει πρόσβαση σε δεδομένα από διαφορετικές πηγές εισόδου και να στέλνει τα δεδομένα σε διαφορετικούς αποδέκτες.



Σχήμα 5.1: Τα βασικά τμήματα του Spark.

Τα κύρια στοιχεία και οι αλληλεπιδράσεις του Spark φαίνονται στο Σχήμα 5.1. Κάποιος μπορεί να χρησιμοποιήσει πολλές διαφορετικές γλώσσες για ανάπτυξη εφαρμογών, ιδίως Scala, Java, Python και R. Εκτός από το κύριο τμήμα του Spark, το Spark Core, υπάρχουν

τέσσερις βιβλιοθήκες που υποστηρίζονται με διαφορετικές λειτουργίες: οι δομές SQL και DataFrames βασίζονται στη δομή επεξεργασία και ανάλυση, η βιβλιοθήκη ροών δεδομένων για χειρισμό εξελισσόμενων δεδομένων, η βιβλιοθήκη ML για μηχανική εκμάθηση και η βιβλιοθήκη GraphX για επεξεργασία και ανάλυση δεδομένων δικτύου. Αυτή η ποικιλία εργαλείων είναι πολύ βολική και παρέχει μεγάλη ευελιξία στην ανάπτυξη εφαρμογών.

5.3 RDDs, DataFrames και DataSets

Το Spark εκθέτει τρία διαφορετικά APIs για πρόσβαση, επεξεργασία και ανάλυση δεδομένων: το RDD API, το DataFrame API και το DataSet API. Το πιο σημαντικό πράγμα που πρέπει να θυμόμαστε είναι ότι RDDs, Dataframes και Datasets είναι κατανεμημένα αντικείμενα με διαφορετικά χαρακτηριστικά, προσφέροντας διαφορετική λειτουργικότητα. Αυτό σημαίνει ότι ορισμένες λειτουργίες εφαρμόζονται μόνο σε RDD, ενώ ορισμένες άλλες εφαρμόζονται μόνο σε DataFrames ή DataSets. Ουσιαστικά, αυτές είναι διαφορετικές αφαιρέσεις ή αναπαραστάσεις των υποκείμενων δεδομένων.

Τα RDD (Resilient Distributed Datasets) αποτελούν το API χαμηλού επιπέδου που παρέχει μεγάλη ευελιξία σε σχέση με τις λειτουργίες που μπορούν να εφαρμοστούν στα δεδομένα. Το RDD είναι μια *αμετάβλητη κατανεμημένη συλλογή αντικειμένων*. Ο όρος *immutable* σημαίνει ότι το περιεχόμενο μιας RDD δεν μπορεί να αλλάξει ή να ενημερωθεί. Επομένως, ο μόνος τρόπος για να αλλάξουμε τα περιεχόμενα ενός RDD είναι να το μετατρέψουμε σε ένα άλλο, χρησιμοποιώντας έναν ή περισσότερους από τους διαθέσιμους *μετασχηματισμούς* (transformations). Για παράδειγμα, αν τα δεδομένα είναι μια μεγάλη ακολουθία ακέραιων αριθμών, αυτή μπορεί να αναπαρασταθεί ως ένα RDD από ακέραιους. Μια μεγάλη συλλογή τριάδων <θερμοκρασία, πίεση, υγρασία>, μπορεί να αναπαρασταθεί ως RDD από πλειάδες της μορφής (θερμοκρασία, πίεση, υγρασία). Τέλος, μια συλλογή λέξεων μπορεί να αναπαρασταθεί ως ένα RDD από συμβολοσειρές (string). Ένα RDD είναι κατανεμημένο, που σημαίνει ότι η συλλογή των δεδομένων χωρίζεται σε πολλά τμήματα, αποθηκευμένα σε διαφορετικά μηχανήματα, προσφέροντας τη δυνατότητα επεξεργασίας πολλών τμημάτων παράλληλα. Το RDD API είναι πολύ ευέλικτο, αλλά χαρακτηρίζεται ως χαμηλού επιπέδου.

Ένα DataFrame είναι μια *δομημένη αμετάβλητη κατανεμημένη συλλογή αντικειμένων*. Η κύρια διαφορά με ένα RDD είναι ότι ένα DataFrame είναι *δομημένο*, που σημαίνει ότι τα δεδομένα αντιπροσωπεύονται από ένα σύνολο *γνωρισμάτων* (χαρακτηριστικών) που αποθηκεύονται ως μια ακολουθία από *ονομαζόμενες στήλες* στην αναπαράσταση του DataFrame. Για παράδειγμα, αντί να χρησιμοποιήσουμε τριάδες της μορφής (θερμοκρασία, πίεση, υγρασία), μπορούμε να χρησιμοποιήσουμε ένα πλαίσιο δεδομένων με τρεις στήλες, *θερμοκρασία*, *πίεση* και *υγρασία*. Το DataFrame προσφέρει αφαίρεση υψηλού επιπέδου, καθώς επιτρέπει την εκτέλεση ερωτημάτων SQL μέσω των δεδομένων. Αυτή είναι μια εξαιρετικά χρήσιμη λειτουργία, λαμβάνοντας υπόψη ότι πολλές δομημένες συλλογές δεν αντιμετωπίζονται από βάσεις δεδομένων, αλλά αποθηκεύονται ως csv ή παρόμοια αρχεία σε μεγάλες ομάδες. Το DataFrame API προσφέρει την ευκαιρία να αποκτήσουμε πρόσβαση σε αυτά τα μεγάλα αρχεία χρησιμοποιώντας SQL και να εφαρμόσουμε ενδιαφέροντα ερωτήματα ανάλυσης, κάτι που αποτελεί μεγάλο πλεονέκτημα.

Τα RDDs και τα DataFrames είναι αρκετά διαφορετικά και υποστηρίζουν διαφορετικές

λειτουργίες. Το DataSet API είναι μια προσπάθεια συνδυασμού RDDs και DataFrames, λαμβάνοντας τα καλά χαρακτηριστικά από κάθε αφαίρεση: την ευελιξία και την τυποποίηση των δεδομένων από τα RDDs και τη δόμηση των δεδομένων των DataFrames. Το DataSet API υποστηρίζεται από τις γλώσσες προγραμματισμού Scala και Java, ενώ Python και R υποστηρίζουν μόνο το DataFrame API. Επιπλέον, η Python υποστηρίζει επίσης το RDD API. Για το υπόλοιπο αυτού του βιβλίου, θα επικεντρωθούμε κυρίως στο DataFrame API και το RDD API.

5.4 Διεργασία οδήγησης, εκτελεστές και διαμερίσεις

Παρόλο που είναι πολύ νωρίς για να εμβαθύνουμε στις λεπτομέρειες σχετικά με τον τρόπο εκτέλεσης μιας εφαρμογής Spark, είναι καλή ιδέα να συζητήσουμε σύντομα για ορισμένες σημαντικές πτυχές της αρχιτεκτονικής του Spark. Ας υποθέσουμε ότι έχουμε αναπτύξει μια εφαρμογή Spark και πρέπει να την εκτελέσουμε σε ένα σύνολο μηχανών που σχηματίζουν συλλογικά μία *συστάδα* (cluster). Οι μηχανές της συστάδας καλούνται *κόμβοι εργασίας* (worker nodes). Το πρώτο σημαντικό στοιχείο της εκτέλεσης της εφαρμογής Spark είναι η *διεργασία οδήγησης* (driver process), δηλαδή μια διεργασία που εκτελείται σε έναν από τους κόμβους της συστάδας η οποία εκτελεί τη συνάρτηση `main()` της εφαρμογής Spark. Προκειμένου η εφαρμογή Spark να επιτρέπει παράλληλη επεξεργασία, χρειαζόμαστε ένα σύνολο από *εκτελεστές* (Executors). Κάθε εκτελεστής αντιστοιχεί σε ένα JVM που μπορεί να εκτελέσει πολλαπλές *εργασίες* (Tasks). Η διεργασία οδήγησης εκτελείται σε ξεχωριστό JVM.

Μία από τις πιο σημαντικές έννοιες στο Spark είναι ο αριθμός των *διαμερίσεων* Partitions των δεδομένων. Κάθε κατανεμημένη συλλογή (RDD, DataFrame, DataSet) χωρίζεται σε ένα σύνολο διαμερίσεων (ή αλλιώς κατατμήσεων) αυθαίρετου μεγέθους. Για παράδειγμα, ένα αρχείο csv μεγέθους 1 GB μπορεί να χωριστεί σε 10 διαμερίσεις του 0,1 GB η καθεμία. Στην πραγματικότητα, ο αριθμός των διαμερίσεων καθορίζει το επίπεδο παραλληλισμού. Πιο συγκεκριμένα, εάν όλα τα δεδομένα ανήκουν σε μία μόνο διαμέριση, τότε δεν υπάρχει τρόπος να επιβληθεί η παράλληλη εκτέλεση. Γενικά, εάν τα δεδομένα χωρίζονται σε P διαμερίσεις, τότε το μέγιστο επίπεδο παραλληλισμού είναι P . Ωστόσο, το επίπεδο του παραλληλισμού αποφασίζεται επίσης από τον αριθμό των εκτελεστών, ας πούμε E , που λειτουργούν στο κατανεμημένο σύστημα. Σε γενικές γραμμές, πρέπει να εγγυηθούμε ότι $E \leq P$, διαφορετικά οι επιπλέον εκτελεστές δεν θα χρησιμοποιηθούν κατά την εκτέλεση της εφαρμογής.

Ας δούμε ένα απλό παράδειγμα. Ας υποθέσουμε ότι έχουμε ένα αρχείο κειμένου και πρέπει πρώτα να εξαλείψουμε όλες τις λέξεις με λιγότερους από 4 χαρακτήρες και, στη συνέχεια, να μετρήσουμε τον αριθμό των υπόλοιπων λέξεων στο αρχείο. Αν και δεν έχουμε συζητήσει ακόμη για μετασχηματισμούς, ενέργειες και πώς μπορούμε να επεξεργαστούμε μια κατανεμημένη συλλογή, το παράδειγμα είναι εύκολα κατανοητό. Σημειώστε επίσης ότι θα χρησιμοποιήσουμε τη γλώσσα Scala και το RDD API. Ας υποθέσουμε ότι το αρχείο κειμένου ονομάζεται `leonardo.txt`. Παρακάτω παρουσιάζουμε μια απλή εφαρμογή Spark που θα μας βοηθήσει να κατανοήσουμε τις προαναφερθείσες έννοιες.

```
1  /* SimpleApp.scala */
2  import org.apache.spark.{SparkConf, SparkContext}
3
4  object SimpleApp {
```

```

5      def main(args: Array[String]): Unit = {
6          println("Starting execution of SimpleApp ... ")
7
8          // Create the Spark Configuration.
9          val sparkConf = new SparkConf()
10             .setMaster("local[4]")
11             .setAppName("SimpleApp")
12
13          // Create the Spark Context.
14          val sc = new SparkContext(sparkConf)
15
16          // Create an RDD from the text file. Use four partitions
17          val txtFileRDD = sc.textFile("leonardo.txt", 4)
18
19          // Remove small words and count all remaining words.
20          val numWords = txtFileRDD
21             .flatMap(line => line.split(' '))
22             .filter(word => word.length() > 3)
23             .count
24
25          // Print some useful info about the data.
26          println("Number of partitions of RDD: " + txtFileRDD.getNumPartitions
27      )
28          println("Number of words: " + numWords)
29          println("Application SimpleApp completed successfully!")
30      }

```

Το πρώτο πράγμα που συζητούμε είναι οι έννοιες Spark Configuration και Spark Context. Ουσιαστικά, το Spark Configuration καθορίζει τον τρόπο με τον οποίο διαμορφώνεται η εφαρμογή Spark. Σε αυτό το παράδειγμα, ορίζουμε δύο σημαντικές μεταβλητές: το όνομα της εφαρμογής Spark και του master. Η εφαρμογή μας ονομάζεται SimpleApp και την ορίσαμε χρησιμοποιώντας τη συνάρτηση `setAppName()`. Ο master αναφέρεται, ουσιαστικά, στον διαχειριστή συμπλέγματος που πρόκειται να χρησιμοποιήσουμε. Στην περίπτωση μας θα εκτελέσουμε την εφαρμογή τοπικά, οπότε καθορίζουμε ότι ο master στην ουσία τρέχει στο τοπικό μηχάνημα και ότι θα χρησιμοποιήσουμε τέσσερις (4) πυρήνες. Αυτός είναι ο λόγος για τη σύνταξη `setMaster(local [4])`. Αυτό σημαίνει ότι ενδέχεται να εφαρμόσουμε τέσσερις διαφορετικές εργασίες ταυτόχρονα στα δεδομένα, όπως θα αναλύσουμε αργότερα στο βιβλίο. Σημειώστε ότι, συνήθως, δεν καθορίζουμε τον master μέσα στον πηγαίο κώδικα, προκειμένου να μπορέσουμε να εκτελέσουμε την εφαρμογή σε διαφορετικά συστήματα.

Το Spark Context (παραδοσιακά αντιπροσωπεύεται από τη μεταβλητή `sc`) είναι η κύρια πύλη μας για να μπορούμε να εκτελούμε κατανεμημένους υπολογισμούς εάν χρησιμοποιούμε το RDD API. Στην πραγματικότητα, το Spark Context περιέχεται σε μια άλλη μεταβλητή, που ονομάζεται Spark Session, που απαιτείται από το DataFrame API και το DataSet API. Ωστόσο, ακόμη και αν δημιουργήσουμε το Spark Session, έχουμε ακόμα πρόσβαση στο Spark Context (περισσότερα για αυτό αργότερα). Το Spark Context είναι μέρος της διαδικασίας προγράμματος οδήγησης (όπου εκτελείται η κύρια λειτουργία της εφαρμογής Spark). Περιττό να πούμε ότι, αν δεν χρησιμοποιήσουμε το Spark Context, ο κώδικας θα τρέξει στον κόμβο που τρέχει τη διαδικασία driver, πράγμα που σημαίνει ότι δεν θα έχουμε καθόλου παραλληλισμό. Σίγουρα

δεν είναι αυτό που θέλουμε. Χρησιμοποιούμε το Spark για να επωφεληθούμε από τους πολλαπλούς πόρους του συμπλέγματος και, επομένως, η χρήση του Spark Context (ή του Spark Session) είναι υποχρεωτική.

Στο παράδειγμά μας, χρησιμοποιούμε το Spark Context για να ορίσουμε ένα RDD με σκοπό την επεξεργασία ενός αρχείου κειμένου. Μόλις οριστεί το RDD, όλα όσα πρέπει να εφαρμόσουμε στο αρχικό αρχείο κειμένου πρέπει να «περάσουν» από τη μεταβλητή. Επίσης, σε αυτό το σημείο έχουμε την ευελιξία να καθορίσουμε τον αριθμό των τμημάτων ή διαμερίσεων για το RDD. Στο παράδειγμά μας, το αρχείο `leonardo.txt` είναι πολύ μικρό, πράγμα που σημαίνει ότι από προεπιλογή θα δημιουργηθεί μόνο μία (1) διαμέριση. Εάν θέλουμε να αυξήσουμε αυτόν τον αριθμό, ενδέχεται να περάσουμε μια δεύτερη παράμετρο στο `textFile` (3 στην περίπτωσή μας), για να καθορίσουμε τον αριθμό διαμερίσεων που χρειαζόμαστε.

```
val txtFileRDD = sc.textFile("leonardo.txt", 3)
```

Εάν δεν καθορίσουμε το πλήθος των διαμερίσεων, το Spark θα το αποφασίσει λαμβάνοντας υπόψη το μέγεθος του αρχείου και το προεπιλεγμένο μέγεθος για κάθε διαμέρισμα. Για παράδειγμα, εάν το αρχείο εισαγωγής βρίσκεται στο HDFS (Hadoop Distributed File System), τότε ο αριθμός των διαμερίσεων ισούται με τον αριθμό των HDFS blocks, όπου συνήθως το προεπιλεγμένο μέγεθος block είναι 64 MB ή 128 MB. Σημειώστε, επίσης, ότι ο αριθμός των διαμερίσεων μπορεί να αλλάξει καθώς εκτελούμε λειτουργίες στα δεδομένα (περισσότερα για αυτό αργότερα).

Στη συνέχεια, με βάση τις απαιτήσεις, πρέπει να αφαιρέσουμε όλες τις λέξεις από το αρχείο κειμένου με 3 ή λιγότερους χαρακτήρες και, στη συνέχεια, να μετρήσουμε τον αριθμό των υπόλοιπων λέξεων. Για να το κάνουμε αυτό, θα χρησιμοποιήσουμε την ακόλουθη δήλωση:

```
val numWords = txtFileRDD
  .flatMap(line => line.split(' '))
  .filter(word => word.length() > 3)
  .count
```

Σημειώστε τη συναρτησιακή σημειογραφία (functional notation) στην παραπάνω εντολή. Θα χρησιμοποιούμε με συνέπεια το συναρτησιακό μοντέλο προγραμματισμού. Εάν νιώθετε άβολα με αυτό τώρα, περιμένετε μέχρι να το συνηθίσετε. Είναι πολύ ισχυρό και οδηγεί σε κομψό κώδικα. Ουσιαστικά, το `txtFileRDD` είναι μια συλλογή γραμμών, όπου κάθε γραμμή είναι μια συμβολοσειρά. Αποφεύγοντας λεπτομέρειες σε αυτό το στάδιο, η δήλωση `flatMap(line => line.split(' '))` μετατρέπει κάθε γραμμή σε μια συλλογή λέξεων. Στη συνέχεια, απλώς καταργούμε όλες τις λέξεις με 3 ή λιγότερους χαρακτήρες χρησιμοποιώντας το `filter(word => word.length() > 3)` και, τέλος, εφαρμόζουμε `count` για να μετρήσει τον αριθμό των υπόλοιπων λέξεων.

Εάν εκτελέσουμε την εφαρμογή, λαμβάνουμε τα ακόλουθα αποτελέσματα:

```
Starting execution of SimpleApp ...
Number of partitions of RDD: 3
Number of words: 140011
Application SimpleApp completed successfully!
```

Παρατηρούμε ότι ο αριθμός των κατατμήσεων είναι 3. Μπορούμε να πάρουμε αυτήν την τιμή εκτελώντας τη δήλωση `txtFileRDD.getNumPartitions`. Αυτή η τιμή καθορίζει

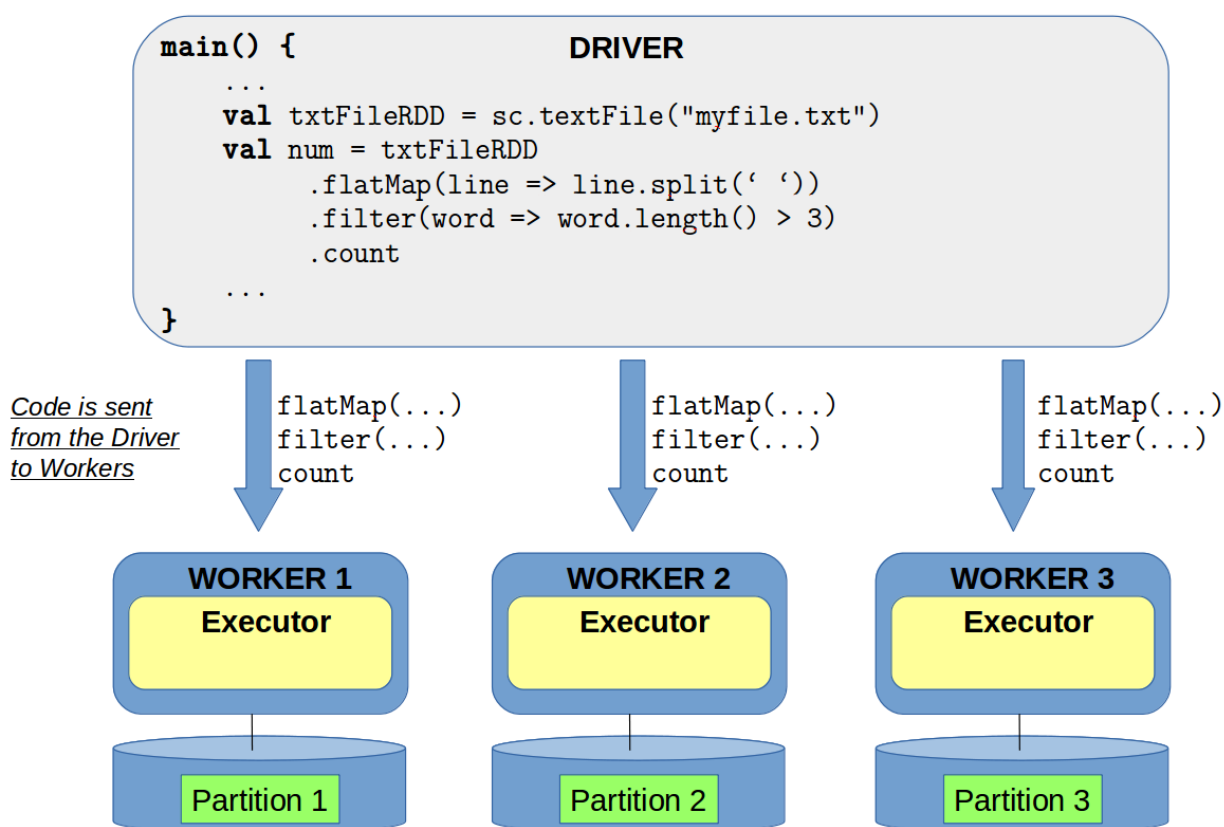
το μέγιστο επίπεδο παραλληλισμού που μπορεί να επιτύχουμε, δηλαδή το πολύ 3 διαμερίσματα μπορούν να υποστούν επεξεργασία παράλληλα. Εάν εκτελέσουμε την εφαρμογή τοπικά (δηλαδή με τοπικό Master), τότε ουσιαστικά ο παραλληλισμός επιτυγχάνεται από τους διαθέσιμους πυρήνες της μονάδας επεξεργασίας. Εάν εκτελέσουμε την εφαρμογή σε μία συστάδα μηχανημάτων (π.χ. όπου είναι εγκατεστημένο το Hadoop), τότε ο παραλληλισμός επιτυγχάνεται κυρίως με την ικανότητα πολλών Spark executors σε διαφορετικά μηχανήματα (αν και ενδέχεται να έχουμε περισσότερους εκτελεστές ανά μηχανήμα). Σε αυτήν την περίπτωση, πρέπει να χρησιμοποιηθεί ένας άλλος διαχειριστής πόρων, όπως ο YARN ή ο Mesos. Σημειώστε ότι στην τοπική λειτουργία το πρόγραμμα οδήγησης και ο εκτελεστής αντιμετωπίζονται από την ίδια διαδικασία.

Για να εξηγήσουμε καλύτερα τις προαναφερθείσες έννοιες (πάλι αποφεύγοντας σκόπιμα λεπτομέρειες χαμηλού επιπέδου), εξηγούμε πώς επιτυγχάνεται η εκτέλεση του SimpleApp σε ένα σύμπλεγμα όπου υπάρχουν τρεις (3) διαθέσιμοι Spark executors. Για ευκολία, θα υποθέσουμε επίσης ότι τα δεδομένα εισαγωγής χωρίζονται σε τρία (3) διαμερίσματα, όπου κάθε executor χειρίζεται ακριβώς ένα (1) διαμέρισμα. Ας υποθέσουμε επίσης ότι όλα τα διαμερίσματα είναι ήδη διαθέσιμα στη δευτερεύουσα αποθήκευση των κόμβων όπου βρίσκονται οι executors. Όπως εξηγήθηκε προηγουμένως, η εφαρμογή θα αρχίσει να εκτελείται σε έναν κόμβο του συμπλέγματος και η εκτέλεσή του θα ελεγχθεί από τη διαδικασία οδήγησης (driver). Ένα σημαντικό πράγμα που πρέπει να θυμόμαστε είναι ότι ο driver θα στείλει στους executors όλες τις απαραίτητες πληροφορίες για να είναι σε θέση να εκτελέσει τις απαιτούμενες εργασίες. Προς το παρόν, ας υποθέσουμε ότι μια εργασία ορίζεται ως μια λειτουργία που πρέπει να εφαρμοστεί σε ένα υποσύνολο των δεδομένων.

Όπως παρατηρούμε στο Σχήμα 5.2 ο driver θα στείλει στους executors όλες τις απαραίτητες πληροφορίες σχετικά με τις λειτουργίες που πρέπει να εκτελεστούν στο `txtFileRDD`. Σε αυτό το παράδειγμα, χρειαζόμαστε τις λειτουργίες `flatMap()`, `filter()` και `count`. Κάθε εκτελεστής εφαρμόζει αυτές τις λειτουργίες στα δικά του δεδομένα. Αυτό είναι ένα σημαντικό χαρακτηριστικό που πρέπει να θυμόμαστε: το ίδιο κομμάτι κώδικα εκτελείται ταυτόχρονα, αλλά σε διαφορετικά τμήματα των δεδομένων. Σημειώστε επίσης ότι η *τοπικότητα των δεδομένων* (*data locality*) είναι ένα πολύ σημαντικό θέμα. Η μεταφορά δεδομένων από το ένα μηχανήμα στο άλλο είναι πολύ ακριβή, ειδικά όταν μιλάμε για μεγάλα σύνολα δεδομένων. Επομένως, το Spark προσπαθεί να μετακινήσει όσο το δυνατό λιγότερα δεδομένα. Σημειώστε επίσης ότι η μεταφορά δεδομένων αυξάνει την κυκλοφορία του δικτύου, που είναι ένας από τους σημαντικότερους παράγοντες κόστους στους κατανεμημένους υπολογισμούς.

5.5 Εγκατάσταση και χρήση

Στη συνέχεια, περιγράφουμε εν συντομία την εγκατάσταση του Spark και του σχετικού λογισμικού σε συστήματα που βασίζονται σε Unix. Θα παράσχουμε πληροφορίες και για τις γλώσσες προγραμματισμού Scala και Python. Σημειώστε ότι η τοπική χρήση του Spark είναι εντελώς διαφορετική από τη χρήση του Spark σε ένα σύμπλεγμα (cluster). Ωστόσο, η τοπική ανάπτυξη είναι πολύ σημαντική για την εκμάθηση του Spark. Επιπλέον, ο κώδικας που πρέπει να δημιουργήσουμε για να εκτελέσουμε την εφαρμογή μας τοπικά, είναι ακριβώς ο ίδιος με τον κώδικα που θα εκτελεστεί σε ένα σύμπλεγμα μηχανών που χρησιμοποιούν μεγά-



Σχήμα 5.2: Εκτέλεση της εφαρμογής SimpleApp.

τερα σύνολα δεδομένων. Ωστόσο, ενδέχεται να προκύψουν ζητήματα απόδοσης κατά την εκτέλεση σε ένα σύμπλεγμα, που μπορεί να απαιτήσει περαιτέρω βελτιστοποιήσεις. Στη συνέχεια, περιγράφουμε τα βήματα που πρέπει να ακολουθήσουμε για να εγκαταστήσουμε και να εκτελέσουμε το σύστημα Apache Spark στον τοπικό μας υπολογιστή (είτε φορητό είτε επιτραπέζιο). Υποθέτουμε ότι η έκδοση Spark που χρησιμοποιείται είναι η 3.0.1 (τελευταία σταθερή έκδοση, που κυκλοφόρησε στις 8 Σεπτεμβρίου 2020). Επίσης, θα επικεντρωθούμε στο Scala 2.12.8 και στην Python 3 για τα παραδείγματα κώδικα.

Σε αυτό το κεφάλαιο, δεν θα χρησιμοποιήσουμε HDFS. Όλα τα παραδείγματα υποθέτουν ότι τα αρχεία δεδομένων εισόδου βρίσκονται στο τοπικό σύστημα αρχείων. Ωστόσο, καθώς σε πολλές περιπτώσεις το Spark χρησιμοποιείται πάνω από το HDFS, στο επόμενο κεφάλαιο θα περιγράψουμε τον τρόπο εκτέλεσης εφαρμογών Spark που έχουν πρόσβαση σε αρχεία που είναι αποθηκευμένα στο HDFS.

Επιπλέον, σε αυτό το βιβλίο, θα συζητήσουμε τον τρόπο εργασίας με το Spark σε συστήματα που βασίζονται σε Unix. Συγκεκριμένα, υποθέτουμε ότι το Ubuntu Linux 20.04 LTS είναι το κεντρικό λειτουργικό σύστημα. Ωστόσο, άλλες διανομές Linux μπορούν να χρησιμοποιηθούν εξίσου καλά. Οι αναγνώστες που ενδιαφέρονται να χρησιμοποιήσουν το Spark σε συστήματα που βασίζονται σε Windows μπορούν να συμβουλευτούν τους πολυάριθμους ιστότοπους που ασχολούνται με αυτό το ζήτημα. Σημειώστε επίσης, ότι ο σκοπός αυτού του κεφαλαίου είναι να προετοιμάσει το περιβάλλον για να μπορέσουμε να προγραμματίσουμε στο Spark.

Το πρώτο πράγμα που πρέπει να κάνετε είναι να εγκαταστήσουμε την Java στον υπολογιστή σας. Το Spark 2.4.4 τρέχει σε Java 8. Για να εγκαταστήσουμε την Java 8, ακολουθούμε

τις παρακάτω οδηγίες. Σημειώστε ότι αυτές οι οδηγίες αναφέρονται στη διανομή Ubuntu. Ωστόσο, μπορούμε να βρούμε παρόμοιες οδηγίες και για άλλες διανομές.

```
sudo add-apt-repository ppa:webupd8team/java
sudo apt update
sudo apt install oracle-java8-installer oracle-java8-set-default
java -version
```

Η απάντηση μετά την εκτέλεση της προηγούμενης εντολής αναμένεται να είναι όπως παρακάτω:

```
java version "1.8.0_171"
Java(TM) SE Runtime Environment (build 1.8.0_171-b11)
Java HotSpot(TM) 64-Bit Server VM (build 25.171-b11, mixed mode)
```

Στη συνέχεια, πρέπει να εγκαταστήσουμε το σύστημα Apache Spark. Μπορούμε να μεταβούμε στον ιστότοπο του Apache Spark και να κατεβάσουμε την έκδοση για Hadoop 2.7. (<https://spark.apache.org/downloads.html>). Μπορούμε να ακολουθήσουμε το σύνδεσμο <https://archive.apache.org/dist/spark/spark-2.4.4/> Από τη στιγμή που έχουμε το αρχείο `tgz` στον υπολογιστή μας, πρέπει να το αποσυμπιέσουμε σε έναν φάκελο. Για τη συνέχεια, θα υποθέσουμε ότι το Spark βρίσκεται στον φάκελο `bdc` του χρήστη `ozzy`

```
/home/ozzy/bdc/spark-2.4.4-bin-hadoop2.7
```

Συνεχίζουμε με κάποιες απαραίτητες ρυθμίσεις. Σε ένα terminal (κονσόλα) γράφουμε τις ακόλουθες εντολές, ώστε να κατασκευάσουμε το αρχείο `log4j.properties`:

```
cd /home/ozzy/bdc/spark-2.4.4-bin-hadoop2.7/conf
cp log4j.properties.template log4j.properties
```

Στη συνέχεια, επεξεργαζόμαστε το αρχείο με κάποιον επιμελητή κειμένου (text editor), ώστε να αντικαταστήσουμε την εντολή `INFO` με την εντολή `ERROR`. Η αλλαγή αυτή θα οδηγήσει το Spark να τυπώνει στην οθόνη μόνο μηνύματα λάθους. Συγκεκριμένα, πρέπει να αλλάξουμε τη γραμμή του αρχείου `log4j.rootCategory=INFO, console` σε `log4j.rootCategory=ERROR, console`.

Το επόμενο βήμα περιλαμβάνει την ενημέρωση του αρχείου `.bashrc` που βρίσκεται στον κατάλογο `home`, εισάγοντας τις ακόλουθες γραμμές:

```
export PATH=$PATH:/home/ozzy/bdc/spark-2.4.4-bin-hadoop2.7/bin
export PATH=$PATH:/home/ozzy/bdc/spark-2.4.4-bin-hadoop2.7/sbin
export JAVA_HOME=/usr/lib/jvm/java-8-oracle
export SPARK_HOME=/home/ozzy/bdc/spark-2.4.4-bin-hadoop2.7
```

Εκτελούμε την ακόλουθη εντολή, ώστε να εμφανιστούν οι αλλαγές:

```
source .bashrc
```

5.5.1 Εφαρμογές Spark με τη γλώσσα Scala

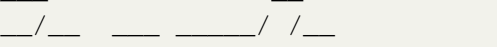
Ο απλούστερος τρόπος να χρησιμοποιήσουμε το Spark με τη γλώσσα Scala είναι χρησιμοποιώντας το κέλυφος του Spark (spark shell). Σε κονσόλα γράφουμε:

```
spark-shell
```

Ως απάντηση αναμένουμε κάτι σαν αυτό:

```
19/12/18 23:43:43 WARN Utils: Your hostname, HPProBook resolves to a loopback
    address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
19/12/18 23:43:43 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
    another address
19/12/18 23:43:43 WARN NativeCodeLoader: Unable to load native-hadoop library
    for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
    setLogLevel(newLevel).
Spark context Web UI available at http://192.168.1.12:4040
Spark context available as 'sc' (master = local[*], app id = local
    -1576705431368).
Spark session available as 'spark'.
```

Welcome to



version 2.4.4

```
Using Scala version 2.11.12 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_171)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Εάν δείτε την παραπάνω οθόνη, τότε συγχαρητήρια! Μπορείτε να ξεκινήσετε να γράφετε κώδικα χρησιμοποιώντας το Spark! Αν και το κέλυφος Spark δεν είναι ένα πλήρες IDE, αποτελεί ένα πολύ χρήσιμο εργαλείο για τα πρώτα μας βήματα στο Spark, επειδή μας επιτρέπει να δοκιμάσουμε γρήγορα ορισμένα τμήματα κώδικα. Για μικρά προγράμματα, το κέλυφος θα βοηθήσει πολύ. Ωστόσο, καθώς τα προγράμματά μας θα γίνονται μεγαλύτερα, αργά ή γρήγορα θα συνειδητοποιήσουμε ότι χρειαζόμαστε ένα πιο φιλικό προς τον χρήστη περιβάλλον για να αναπτύξουμε τις εφαρμογές μας με μεγαλύτερη ευκολία και περισσότερη βοήθεια.

Εάν σκοπεύουμε να χρησιμοποιήσουμε το Spark με τη γλώσσα Scala, πρέπει να κάνουμε λήψη μιας κατάλληλης έκδοσης της γλώσσας Scala. Κανονικά, για το Spark 2.4.4 χρειαζόμαστε Scala 2.12.x. Ωστόσο, δεδομένου ότι αρκετοί χρήστες έχουν αντιμετωπίσει κάποια προβλήματα, θα χρησιμοποιήσουμε την έκδοση Scala 2.11.8. Μπορούμε, ωστόσο, να δοκιμάσουμε Scala 2.12.x αν προτιμάμε. Για λήψη της Scala 2.11.8 χρησιμοποιούμε τον σύνδεσμο <https://downloads.lightbend.com/scala/2.11.8/scala-2.11.8.tgz>.

Μπορούμε να αποσυμπιέσουμε το αρχείο σε οποιονδήποτε φάκελο. Ας υποθέσουμε ότι αυτός ο φάκελος είναι ο `/home/ozzy/bdc/scala-2.11.8`.

Στο αρχείο `.bashrc` εισάγουμε τις ακόλουθες γραμμές:

```
export PATH=$PATH:/home/ozzy/bdc/scala-2.11.8/bin
```

Για να ενεργοποιηθούν οι αλλαγές, γράφουμε:

```
source .bashrc
```

Για να βεβαιωθούμε ότι η Scala υπάρχει στο σύστημά μας, γράφουμε:

```
scala -version
```

Θα πρέπει στην έξοδο να δούμε κάτι σαν αυτό:

```
Scala code runner version 2.11.8 -- Copyright 2002-2016, LAMP/EPFL
```

Στη συνέχεια, μπορούμε να τρέξουμε το κέλυφος της Scala (scala shell) ώστε να γράψουμε μικρά τμήματα κώδικα σε Scala:

```
scala
```

Στην έξοδο αναμένουμε κάτι σαν αυτό:

```
Welcome to Scala 2.11.8 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_171).
Type in expressions for evaluation. Or try :help.
scala>
```

Εάν θέλουμε να γράψουμε μια απλή εφαρμογή ή να δοκιμάσουμε κάποιες ιδέες, τότε το κέλυφος Scala είναι εντάξει. Ωστόσο, εγκαταστήσαμε τη γλώσσα Scala για να τη χρησιμοποιήσουμε με το Spark. Στη συνέχεια, θα περιγράψουμε δύο διαφορετικούς τρόπους δημιουργίας εφαρμογών Spark χρησιμοποιώντας τη γλώσσα Scala. Ο πρώτος τρόπος περιλαμβάνει τη μεταγλώττιση και τη δημιουργία αρχείων `jar` χρησιμοποιώντας το Simple Build Tool (`sbt`) μέσω μιας κονσόλας, και, στη συνέχεια, εκτελώντας τα χρησιμοποιώντας το εργαλείο `spark-submit`. Ο δεύτερος τρόπος απαιτεί την εγκατάσταση ενός IDE που υποστηρίζει Scala (π.χ. IntelliJ IDEA). Η δημιουργία και η εκτέλεση του κώδικα γίνονται πιο εύκολες. Ωστόσο, οι δύο μέθοδοι μπορούν να συνδυαστούν: μπορούμε να χρησιμοποιήσουμε το IDE για να δημιουργήσουμε την εφαρμογή μας και να δώσουμε εντολή στο IDE να δημιουργήσει το αρχείο `.jar`. Στη συνέχεια, μπορούμε είτε να τρέξουμε το εκτελέσιμο αρχείο μέσα από το IDE είτε χρησιμοποιώντας το εργαλείο `spark-submit`.

Αρχικά, ας δούμε πώς χρησιμοποιείται το `sbt` για τη μεταγλώττιση και τη δημιουργία εφαρμογών Spark γραμμένων σε Scala. Το `sbt` είναι διαθέσιμο από <https://www.scala-sbt.org/download.html>. Εξάγουμε το ληφθέν αρχείο σε έναν φάκελο της επιλογής μας. Στην περίπτωση μας, θα υποθέσουμε ότι το `sbt` έχει εξαχθεί στο `/home/ozzy/bdc/sbt`. Στη συνέχεια, ανοίγουμε το `.bashrc` αρχείο και συμπεριλαμβάνουμε την ακόλουθη γραμμή:

```
export PATH=$PATH:/home/ozzy/bdc/sbt/bin
```

Για να ενεργοποιηθούν οι αλλαγές, δίνουμε:

```
source .bashrc
```

Τώρα, είμαστε έτοιμοι να προχωρήσουμε στη μεταγλώττιση του κώδικα και τη δημιουργία εκτελέσιμων αρχείων. Για να δοκιμάσουμε τη διαδικασία χρειαζόμαστε μία μικρή εφαρμογή. Υποθέτουμε ότι ο φάκελος `/home/ozzy/bdc/sparklab/scala` περιέχει την εφαρμογή μας. Μέσα σε αυτόν τον φάκελο δημιουργούμε έναν νέο με όνομα `WordCount`. Η εφαρμογή που θα χρησιμοποιήσουμε μετρά το πλήθος εμφανίσεων της κάθε λέξης σε ένα αρχείο κειμένου. Κάθε `sbt project` απαιτεί ένα αρχείο `build.sbt`. Σε έναν επιμελητή κειμένου αντιγράφουμε τις ακόλουθες γραμμές και σώζουμε το αρχείο με το όνομα `build.sbt` στον φάκελο `/home/ozzy/bdc/sparklab/scala/WordCount`.

```
name := "WordCount"
version := "1.0"
scalaVersion := "2.11.8"
scalaHome := Some(file("/home/ozzy/bdc/scala-2.11.8"))
libraryDependencies += "org.apache.spark" %% "spark-core" % "2.4.4"
```

Στη συνέχεια, πρέπει να ορίσουμε τα δεδομένα εισόδου. Θα χρησιμοποιήσουμε ένα μικρό αρχείο κειμένου το οποίο περιέχει τις σημειώσεις του Leonardo da Vinci και είναι ελεύθερα διαθέσιμο από το Project Gutenberg. Μπορούμε να βρούμε το αρχείο στον σύνδεσμο: <http://www.gutenberg.org/files/5000/5000-8.txt>. Σώζουμε το αρχείο στον φάκελο `/home/ozzy/bdc/sparklab/scala/WordCount` και στη συνέχεια το μετονομάζουμε σε `leonardo.txt`. Προφανώς, μπορούμε να χρησιμοποιήσουμε οποιοδήποτε άλλο αρχείο κειμένου. Ο φάκελος του project περιέχει δύο αρχεία: `build.sbt` και `leonardo.txt`. Στη συνέχεια, δημιουργούμε το αρχείο που περιέχει τον πηγαίο κώδικα σε Scala. Πρώτα, πρέπει να δημιουργήσουμε τον κατάλογο `src/main/scala` μέσα στον κατάλογο `WordCount`. Ο πηγαίος κώδικας που παρουσιάζεται στη συνέχεια πρέπει να αποθηκευθεί μέσα στον φάκελο `/home/ozzy/bdc/sparklab/scala/WordCount/src/main/scala` στο αρχείο με όνομα `WordCount.scala`.

```
1  /* WordCount.scala */
2
3  import org.apache.spark.{SparkConf, SparkContext}
4
5  object WordCount {
6
7      def main(args: Array[String]): Unit = {
8
9          println("Starting execution of WordCount ... ")
10
11          // Create the spark configuration
12          val sparkConf = new SparkConf()
13              .setMaster("local[2]")
14              .setAppName("WordCount")
15
16          // Create the spark context
17          val sc = new SparkContext(sparkConf)
18
19          // Create an RDD from the text file
20          val txtFileRDD = sc.textFile("leonardo.txt", 4)
21
22          // Apply MapReduce to generate word counts
23          val wordCountRDD = txtFileRDD
```

```

24     .flatMap(line => line.split(" ")) // split line to words
25     .map(word => (word,1)) // map words to (word,1) pairs
26     .reduceByKey(_+_ ) // reduce
27
28     // Save the result to the "output" folder
29     wordCountRDD.saveAsTextFile("output")
30
31     println("Application WordCount completed successfully!")
32 }
33 }

```

Τώρα, είμαστε έτοιμοι να δοκιμάσουμε τον κώδικα. Πρώτα, κατασκευάζουμε το αρχείο jar. Χρησιμοποιώντας κονσόλα, πηγαίνουμε στον φάκελο WordCount και ελέγχουμε τα περιεχόμενα:

```

cd /home/ozzy/bdc/sparklab/scala/WordCount
ls -las

```

Το αποτέλεσμα αναμένουμε να είναι όπως παρακάτω:

```

1 ozzy@myPC:~/bdc/sparklab/scala/WordCount$ ls -las
2 total 1408
3    4 drwxrwxr-x  3 ozzy ozzy    4096 Dec  20 11:18 .
4    4 drwxrwxr-x 10 ozzy ozzy    4096 Dec  12 00:37 ..
5    4 -rwxrwxr-x  1 ozzy ozzy    214 Dec  20 11:06 build.sbt
6 1392 -rwxrwxr-x  1 ozzy ozzy 1423801 Dec   8  2014 leonardo.txt
7    4 drwxrwxr-x  3 ozzy ozzy    4096 Dec   5  2018 src
8 ozzy@myPC:~/bdc/sparklab/scala/WordCount$

```

Για να προχωρήσουμε, γράφουμε:

```
sbt package
```

Κατά τη διαδικασία αυτή, θα δούμε στην έξοδο διάφορα μηνύματα, όπως παρακάτω:

```

[info] Updated file /home/ozzy/bdc/sparklab/scala/WordCount/project/build.
      properties: set sbt.version to 1.1.0
[info] Loading project definition from /home/ozzy/bdc/sparklab/scala/
      WordCount/project
[info] Updating ProjectRef(uri("file:/home/ozzy/bdc/sparklab/scala/WordCount/
      project/"), "root-wordcount-build")...
[info] Done updating.
[info] Loading settings from build.sbt ...
[info] Set current project to WordCount (in build file:/home/ozzy/bdc/
      sparklab/scala/WordCount/)
[info] Updating ...
[info] Done updating.
[info] Compiling 1 Scala source to /home/ozzy/bdc/sparklab/scala/WordCount/
      target/scala-2.11/classes ...
[info] Done compiling.
[info] Packaging /home/ozzy/bdc/sparklab/scala/WordCount/target/scala-2.11/
      wordcount_2.11-1.0.jar ...
[info] Done packaging.
[success] Total time: 8 s, completed Dec 20, 2019 11:30:46 AM

```

Ελέγχοντας τα περιεχόμενα του φακέλου, θα διαπιστώσουμε ότι έχουν δημιουργηθεί δύο νέοι υπο-φάκελοι με ονομασίες: `project` και `target`.

```
ozzy@myPC:~/bdc/sparklab/scala/WordCount$ ls -las
total 1408
  4 drwxrwxr-x  3 ozzy ozzy    4096 Dec  20 11:18 .
  4 drwxrwxr-x 10 ozzy ozzy    4096 Dec 12 00:37 ..
  4 -rwxrwxr-x  1 ozzy ozzy     214 Dec  20 11:06 build.sbt
1392 -rwxrwxr-x  1 ozzy ozzy 1423801 Dec   8 2014 leonardo.txt
  4 drwxrwxr-x  3 ozzy ozzy    4096 Dec  20 11:30 project
  4 drwxrwxr-x  3 ozzy ozzy    4096 Dec   5 2018 src
  4 drwxrwxr-x  4 ozzy ozzy    4096 Dec  20 11:30 target
ozzy@myPC:~/sparklab/scala/WordCount$
```

Το εκτελέσιμο αρχείο που έχει δημιουργηθεί βρίσκεται στον φάκελο `/home/ozzy/bdc/sparklab/scala/WordCount/target/scala-2.11/` με όνομα αρχείου `word-count_2.11-1.0.jar`. Εάν το αρχείο αυτό υπάρχει, τότε η διαδικασία είναι επιτυχής και είμαστε έτοιμοι να προχωρήσουμε με την εκτέλεση της εφαρμογής μας.

Το εργαλείο `spark-submit` αποτελεί ένα από τα σημαντικότερα εργαλεία κονσόλας για εφαρμογές Spark. Με το εργαλείο αυτό μπορούμε να εκτελέσουμε μία εφαρμογή Spark τοπικά ή σε κάποιον cluster. Επίσης, έχουμε τη δυνατότητα να ορίσουμε τιμές για σημαντικές παραμέτρους όπως για παράδειγμα το μέγεθος της μνήμης που θα χρησιμοποιηθεί από τον οδηγό και τους εκτελεστές, το πλήθος των εκτελεστών, κ.λπ. Περισσότερες πληροφορίες για το εργαλείο `spark-submit` υπάρχουν στο επίσημο website του Apache Spark: <https://spark.apache.org/docs/latest/submitting-applications.html>.

Εκτελούμε την εφαρμογή δίνοντας στην κονσόλα:

```
spark-submit ./target/scala-2.11/wordcount_2.11-1.0.jar
```

Αναμένουμε στην έξοδο μηνύματα όπως τα ακόλουθα:

```
19/12/22 11:41:27 WARN Utils: Your hostname, HPProBook resolves to a loopback
address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
19/12/22 11:41:27 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
19/12/22 11:41:27 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
Starting execution of WordCount ...
Application WordCount completed successfully!
```

Η έξοδος της εφαρμογής βρίσκεται στον φάκελο `/home/ozzy/bdc/sparklab/scala/WordCount/output`. Ας εξετάσουμε τα περιεχόμενα του φακέλου αυτού:

```
ls -las output
```

```
  4 drwxrwxr-x  2 ozzy ozzy    4096 Dec  20 21:34 .
  4 drwxrwxr-x  6 ozzy ozzy    4096 Dec  20 21:34 ..
104 -rw-r--r--  1 ozzy ozzy 103682 Dec  20 21:34 part-00000
  4 -rw-r--r--  1 ozzy ozzy     820 Dec  20 21:34 .part-00000.crc
100 -rw-r--r--  1 ozzy ozzy 100419 Dec  20 21:34 part-00001
  4 -rw-r--r--  1 ozzy ozzy     796 Dec  20 21:34 .part-00001.crc
100 -rw-r--r--  1 ozzy ozzy  98729 Dec  20 21:34 part-00002
  4 -rw-r--r--  1 ozzy ozzy     780 Dec  20 21:34 .part-00002.crc
```

```
100 -rw-r--r-- 1 ozzy ozzy 100747 Dec 20 21:34 part-00003
  4 -rw-r--r-- 1 ozzy ozzy    796 Dec 20 21:34 .part-00003.crc
  0 -rw-r--r-- 1 ozzy ozzy      0 Dec 20 21:34 _SUCCESS
  4 -rw-r--r-- 1 ozzy ozzy      8 Dec 20 21:34 ._SUCCESS.crc
```

Αγνοώντας τα αρχεία ελέγχου crc, υπάρχουν τέσσερα αρχεία που περιέχουν την έξοδο του WordCount και ένα αρχείο σημαίας με το όνομα `_SUCCESS` που υποδηλώνει ότι η εφαρμογή ολοκληρώθηκε με επιτυχία. Τα υπόλοιπα αρχεία περιέχουν τις συχνότητες των λέξεων. Ας ελέγξουμε ένα από τα αρχεία, π.χ., `part-00000`. Στην κονσόλα γράφουμε:

```
tail output/part-00000
```

Το αποτέλεσμα αποτελείται από τις δέκα τελευταίες γραμμές του αρχείου `part-00000`:

```
(parenthesis.,1)
(basilica,2)
(movements,,2)
(hemisphere.,2)
(Dardania,1)
(Acropolis,1)
(absolutely,4)
(loved.,1)
(young,32)
(ruin,,1)
```

Κάθε ζεύγος αντιστοιχεί σε μία λέξη και έναν αριθμό που δηλώνει τον αριθμό των εμφανίσεων. Για παράδειγμα, η λέξη «Ακρόπολη» εμφανίζεται μία φορά στο κείμενο. Σημειώστε επίσης, ότι μερικές λέξεις είναι περιέργες! Για παράδειγμα, «ημισφαίριο.», «αγαπημένο.», «καταστροφή.». Αυτές οι λέξεις περιέχουν σημεία στίξης. Ωστόσο, αυτό είναι φυσικό, αφού δεν αφαιρέσαμε τα σημεία στίξης από το κείμενο. Επίσης, ορισμένες λέξεις περιέχουν κεφαλαία γράμματα (όπως η λέξη το «Ακρόπολη»), επειδή δεν εφαρμόσαμε κανενός είδους προεπεξεργασία για τη μετατροπή κεφαλαίων χαρακτήρων σε πεζούς.

Ένα ενδιαφέρον σημείο που πρέπει να εξηγήσουμε είναι ο λόγος για τον οποίο έχουμε τέσσερα αρχεία στην έξοδο. Αυτό οφείλεται στο ότι στον πηγαίο κώδικα, ορίσαμε ότι το αρχείο εισόδου θα χωριστεί σε τέσσερις διαμερίσεις (partitions). Ελέγχοντας προσεκτικά τον πηγαίο κώδικα, διαπιστώνουμε ότι υπάρχει η γραμμή:

```
val txtFile = sc.textFile(inputFile, 4)
```

Ο αριθμός 4 που εμφανίζεται ως δεύτερη παράμετρος της συνάρτησης `textFile` ορίζει τη χρήση τεσσάρων διαμερίσεων. Όπως θα αναλύσουμε σε άλλο μέρος του βιβλίου, το πλήθος των διαμερίσεων (σε συνδυασμό με το πλήθος των εκτελεστών) ορίζει το επίπεδο του παραλληλισμού που θα έχει η εφαρμογή.

Μέχρι αυτό το σημείο, είναι βέβαιο ότι θα υπάρχουν πολλές απορίες. Ελπίζουμε πραγματικά ότι αυτές οι απορίες θα απαντηθούν αργότερα, καθώς προχωράτε στη μελέτη του βιβλίου. Προς το παρόν, εάν έχετε φτάσει σε αυτό το σημείο με επιτυχία, έχετε κάνει το πρώτο (και το πιο σημαντικό) βήμα προς τον προγραμματισμό στο Spark χρησιμοποιώντας τη γλώσσα προγραμματισμού Scala. Στην επόμενη ενότητα, θα επαναλάβουμε τη διαδικασία εστιάζοντας στη γλώσσα Python. Ωστόσο, τονίζουμε ότι για τη συνέχεια της μελέτης στα επόμενα κεφάλαια θα χρησιμοποιήσουμε τη γλώσσα Scala.

5.5.2 Εφαρμογές Spark με τη γλώσσα Python

Αρχικά, θα εγκαταστήσουμε τη γλώσσα Python 3. Αν ήδη έχουμε Python στο σύστημά μας, μπορούμε να προχωρήσουμε στο επόμενο βήμα. Εναλλακτικά, μπορούμε να εγκαταστήσουμε Anaconda για Python 3. Για την εγκατάσταση της Python 3, εκτελούμε τις ακόλουθες εντολές.

```
sudo apt install software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.7
python --version
```

Αναμένουμε να δούμε:

Python 3.7.4

Η Python 3 έχει εγκατασταθεί, αλλά μπορεί να χρειαστούμε επιπλέον βιβλιοθήκες τις οποίες πρέπει να εγκαταστήσουμε. Ας ξεκινήσουμε τώρα το κέλυφος PySpark:

pyspark

Αναμένουμε την ακόλουθη απάντηση:

```
Python 3.7.4 (default, Aug 13 2019, 20:35:49)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
19/12/21 23:50:41 WARN Utils: Your hostname, HPProBook resolves to a loopback
address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
19/12/21 23:50:41 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
19/12/21 23:50:42 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
Welcome to

  ____      _
 /  _ \    / \
/_  _ \    /  \
 \_/\_ \  /_/\_ \
  \_/\_ \ /_/\_ \
   \_/\_ \

version 2.4.4

Using Python version 3.7.4 (default, Aug 13 2019 20:35:49)
SparkSession available as 'spark'.
>>>
```

Για να εκτελέσουμε εφαρμογές Spark από την κονσόλα, μπορούμε να χρησιμοποιήσουμε το εργαλείο `spark-submit`. Θα εργαστούμε με παρόμοιο τρόπο όπως στην περίπτωση της γλώσσας Scala. Πρώτα, θα δημιουργήσουμε έναν φάκελο για την εφαρμογή Python, μετά θα ετοιμάσουμε τα απαραίτητα αρχεία και, τελικά, θα εκτελέσουμε την εφαρμογή χρησιμοποιώντας το `spark-submit`. Αρχικά, δημιουργούμε τον φάκελο `python` μέσα στον φάκελο `sparklab`. Αυτός ο φάκελος θα χρησιμοποιηθεί για την αποθήκευση όλων των εφαρμογών Python που σχετίζονται με το Spark. Στη συνέχεια, δημιουργούμε τον φάκελο `WordCount`

μέσα στο φάκελο python. Θα τοποθετήσουμε όλα τα απαραίτητα αρχεία μέσα στον φάκελο /home/ozzy/bdc/sparklab/python/WordCount. Ξεκινάμε δημιουργώντας το αρχείο πηγαίου κώδικα WordCount.py. Τα περιεχόμενα αυτού του αρχείου είναι τα εξής:

```

1  # WordCount.py
2
3  from pyspark import SparkContext, SparkConf
4
5  if __name__ == "__main__":
6
7      print("Starting execution of WordCount ... ")
8
9      # Create the Spark context with the necessary configuration
10     sc = SparkContext(master="local", appName="WordCount")
11
12     # Create an RDD from the text file
13     txtFileRDD = sc.textFile("leonardo.txt", 4)
14
15     # Apply MapReduce to generate word counts
16     wordCountRDD = txtFileRDD \
17         .flatMap(lambda line: line.split(" ")) \
18         .map(lambda word: (word, 1)) \
19         .reduceByKey(lambda a,b:a+b)
20
21     # Save the result to the "output" folder
22     wordCountRDD.saveAsTextFile("output/")
23
24     print("Application WordCount completed successfully!")

```

Πρέπει, επίσης, να αντιγράψουμε και το αρχείο leonardo.txt στον φάκελο της εφαρμογής. Τα περιεχόμενα του φακέλου πρέπει να είναι ως εξής:

```

4 drwxrwxr-x 2 ozzy ozzy      4096 Dec  22 18:06 .
4 drwxrwxr-x 4 ozzy ozzy      4096 Dec  22 10:57 ..
1396 -rwxrwxr-x 1 ozzy ozzy 1423801 Dec  22 10:54 leonardo.txt
4 -rw-rw-r-- 1 ozzy ozzy      669 Dec  22 13:38 WordCount.py

```

Για την εκτέλεση της εφαρμογής, γράφουμε:

```
spark-submit WordCount.py
```

Κατά την εκτέλεση, εμφανίζονται διάφορα μηνύματα όπως παρακάτω:

```

19/12/22 18:09:33 WARN Utils: Your hostname, HPProBook resolves to a loopback
address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
19/12/22 18:09:33 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to
another address
19/12/22 18:09:33 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
Starting execution of WordCount ...
Application WordCount completed successfully!

```

Όπως ίσως έχετε παρατηρήσει, η έξοδος της εφαρμογής βρίσκεται στον φάκελο output. Σημειώστε, επίσης, ότι έχουμε δώσει εντολή στο Spark να χρησιμοποιήσει τέσσερις (4) καταμήσεις για το αρχείο εισόδου. Αυτός είναι ο λόγος για τον οποίο η έξοδος αποτελείται από

τέσσερα αρχεία εξόδου και μία σημαία (_SUCCESS). Ας ελέγξουμε τις τελευταίες 10 γραμμές του αρχείου εξόδου `part-00000` πληκτρολογώντας:

```
tail output/part-00000
```

Η έξοδος είναι όπως παρακάτω:

```
('ASCII', 1)
('form', 1)
('gross', 1)
("DON'T", 1)
('accepts', 1)
('the:', 1)
('scanning', 1)
('equipment', 1)
('hart@pobox.com', 1)
('hardware', 1)
```

5.6 Μετασχηματισμοί και ενέργειες

Στην προηγούμενη ενότητα, περιγράφηκαν συνοπτικά μερικές βασικές συναρτήσεις που μπορούμε να εκτελέσουμε σε ένα RDD, όπως οι συναρτήσεις `flatMap`, `filter`, `count`. Τώρα που είμαστε σε θέση να τρέξουμε απλές εφαρμογές Spark, είναι ώρα να εξηγήσουμε με περισσότερες λεπτομέρειες τον τρόπο με τον οποίο δομείται μια εφαρμογή Spark.

Ένα από τα βασικά χαρακτηριστικά του Spark είναι ότι οι συναρτήσεις που μπορούμε να εκτελέσουμε σε ένα RDD δεν έχουν όλες τις ίδιες ιδιότητες. Για την ακρίβεια, οι συναρτήσεις χωρίζονται σε δύο βασικές κατηγορίες: 1) μετασχηματισμοί (transformations) και 2) ενέργειες (actions). Για τη συνέχεια του κεφαλαίου θα εστιάσουμε στη γλώσσα Scala. Σημειώνεται ότι τα ονόματα των συναρτήσεων στην Python είναι ακριβώς τα ίδια, μόνο που η σύνταξη είναι διαφορετική, όπως έχουμε παρατηρήσει και σε προηγούμενα παραδείγματα.

5.6.1 Μετασχηματισμοί

Όπως έχει αναφερθεί σε προηγούμενο σημείο, ένα RDD αποτελεί συλλογή αντικειμένων που δεν μπορούμε να αλλάξουμε. Αυτό όμως δεν σημαίνει ότι δεν μπορούμε να το μετασχηματίσουμε. Η διαχείριση κατανεμημένων αντικειμένων με δυνατότητες ανάγνωσης (read) και εγγραφής (write) δημιουργεί θέματα συνέπειας και, επομένως, επιβάλλει την εφαρμογή μηχανισμών κλειδώματος (locking) με στόχο την προστασία των δεδομένων και τη χρήση αμοιβαίου αποκλεισμού σε περίπτωση που δύο διεργασίες προσπαθούν ταυτόχρονα να πραγματοποιήσουν εγγραφή στην ίδια περιοχή δεδομένων. Άρα, υποχρεώνοντας τα κατανεμημένα αντικείμενα να είναι μόνο για ανάγνωση (read-only ή immutable), βοηθούμε την πιο αποδοτική κατανεμημένη επεξεργασία.

Όμως, με ποιον τρόπο θα αλλάξουμε τα περιεχόμενα ενός RDD από τη στιγμή που είναι μόνο για ανάγνωση; Η απάντηση δίνεται με τη χρήση μετασχηματισμών που μπορούμε να χρησιμοποιήσουμε. Ένας *μετασχηματισμός* είναι μια συνάρτηση που δέχεται ως είσοδο ένα ή δύο RDDs και επιστρέφει ένα άλλο RDD ως έξοδο. Ας υποθέσουμε ότι έχουμε ένα RDD

που αποτελείται από ακέραιους αριθμούς με περιεχόμενα: $\langle 1, 5, 2, 4, 1, 6, 2 \rangle$. Θα εφαρμόσουμε μερικούς μετασχηματισμούς και θα εξετάσουμε τον τρόπο λειτουργίας για τον κάθε ένα, καθώς και το αποτέλεσμα που επιφέρει. Ο κατάλογος των μετασχηματισμών είναι πολύ μεγάλος, οπότε εδώ θα δώσουμε μόνο μερικά χαρακτηριστικά παραδείγματα. Για μία πλήρη λίστα μετασχηματισμών θα πρέπει ο αναγνώστης να ανατρέξει στην επίσημη τεκμηρίωση του Spark (<http://spark.apache.org>).

Θα ξεκινήσουμε με τη συνάρτηση `map()`. Η συνάρτηση αυτή αποτελεί μετασχηματισμό ο οποίος δέχεται στην είσοδο ένα RDD, παίρνει ως παράμετρο μία άλλη συνάρτηση `f()` και μετασχηματίζει κάθε στοιχείο του RDD με βάση τη συνάρτηση `f()`. Ας υποθέσουμε ότι θέλουμε να πολλαπλασιάσουμε το κάθε στοιχείο του RDD με τον αριθμό 3 και να δημιουργήσουμε ένα νέο RDD με όνομα `outputRDD`. Το αποτέλεσμα θα είναι $\langle 3, 15, 6, 12, 3, 18, 6 \rangle$ και ο αντίστοιχος κώδικας είναι:

```
val outputRDD = inputRDD.map(element => 3*element)
```

Στην υλοποίηση του μετασχηματισμού `map()` υπάρχει μια κρυφή ανακύκλωση. Πράγματι, η συνάρτηση που δίνουμε ως παράμετρο στον μετασχηματισμό `map()` εκτελείται μία φορά για κάθε στοιχείο του RDD εισόδου. Άρα, η συνάρτηση θα εκτελεστεί τόσες φορές όσα είναι τα στοιχεία του RDD εισόδου.

Έστω τώρα ότι πρέπει να κρατήσουμε μόνο τα στοιχεία που είναι μεγαλύτερα από τον αριθμό 3. Εδώ θα πρέπει να εφαρμοστεί ένα φίλτρο που να δίνει στην έξοδο μόνο τα στοιχεία για τα οποία ενδιαφερόμαστε. Η κατάλληλη συνάρτηση είναι η `filter()`, η οποία δέχεται στην είσοδο ένα RDD και παίρνει ως παράμετρο μια συνάρτηση `bf()`, η οποία επιστρέφει `true` ή `false`. Πιο συγκεκριμένα, το αποτέλεσμα θα είναι $\langle 5, 4, 6 \rangle$ και η διατύπωση έχει ως εξής:

```
val outputRDD = inputRDD.filter(element => element > 3)
```

Ομοίως και εδώ, ο μετασχηματισμός `filter()` εκτελεί τη συνάρτηση που δίνουμε ως παράμετρο τόσες φορές όσα είναι τα στοιχεία του RDD στην είσοδο.

Οι εκφράσεις `element => 3 * element` και `element => element > 3` καλούνται *ανώνυμες συναρτήσεις*, διότι περιγράφουν ακριβώς τον μετασχηματισμό του κάθε αντικειμένου και δεν έχουν όνομα. Ωστόσο, πάντα έχουμε τη δυνατότητα να ορίσουμε μια συνάρτηση με συγκεκριμένο όνομα και να την καλέσουμε μέσα στη συνάρτηση `map()` ή `filter()` χρησιμοποιώντας το όνομά της. Αυτό βοηθάει πολύ στην περίπτωση όπου η συνάρτηση που θέλουμε να χρησιμοποιήσουμε είναι πολύπλοκη. Για τις απλές περιπτώσεις, η χρήση ανώνυμων συναρτήσεων οδηγεί σε συντομότερο και πιο ευανάγνωστο πηγαίο κώδικα.

Με βάση τη συναρτησιακή σημειογραφία, έχουμε τη δυνατότητα να χρησιμοποιήσουμε πολλούς μετασχηματισμούς σε σειρά (τον έναν δίπλα στον άλλον), έτσι ώστε η έξοδος του προηγούμενου μετασχηματισμού να τροφοδοτεί την είσοδο του επόμενου. Για παράδειγμα, εάν θέλουμε να εκτελέσουμε πρώτα τον μετασχηματισμό `map()` και, στη συνέχεια, τον μετασχηματισμό `filter()`, τότε μπορούμε να γράψουμε:

```
val outputRDD = inputRDD.map(element => 3*element).filter(element =>
element > 3)
```

Ας εξετάσουμε στη συνέχεια τον μετασχηματισμό `flatMap()` τον οποίο είχαμε συναντήσει σε προηγούμενο παράδειγμα. Θα υποθέσουμε ότι στην είσοδο έχουμε ένα RDD που αποτελείται από γραμμές ενός αρχείου κειμένου (text). Θέλουμε να δημιουργήσουμε ένα RDD στην έξοδο το οποίο αποτελείται από όλες τις λέξεις του αρχείου κειμένου. Επομένως, θα πρέπει με κάποιον τρόπο να διαχωρίσουμε την κάθε γραμμή του αρχείου σε λέξεις. Έστω ότι έχουμε ένα αρχείο κειμένου που αποτελείται από τις ακόλουθες δύο γραμμές:

```
this is a line
this is another line
```

Για να επεξεργαστούμε το αρχείο, αρχικά, πρέπει να δημιουργήσουμε ένα RDD, το οποίο θα αποτελείται από αντικείμενα τύπου `String`, και κάθε αντικείμενο θα είναι μία γραμμή από το αρχείο κειμένου. Ας υποθέσουμε ότι έχουμε έτοιμο ένα αντικείμενο Spark context (`sc`). Ο κώδικας που απαιτείται για να δημιουργήσουμε ένα RDD με τις λέξεις του αρχείου κειμένου είναι:

```
val txtFileRDD = sc.textFile("myfile.txt", 5)
val wordsRDD = txtFileRDD.flatMap(line => line.split(' '))
```

Για να κατανοήσουμε τον τρόπο λειτουργίας του μετασχηματισμού `flatMap()`, ας δούμε πρώτα ποιο θα ήταν το αποτέλεσμα εάν αντί για `flatMap()` χρησιμοποιούσαμε τον μετασχηματισμό `map()`. Έστω λοιπόν ο κώδικας:

```
val txtFileRDD = sc.textFile("myfile.txt", 5)
val listsRDD = txtFileRDD.map(line => line.split(' '))
```

Μπορείτε να δείτε τη διαφορά στο αποτέλεσμα; Επειδή ο μετασχηματισμός `map()` είναι ένα-προς-ένα, η κάθε γραμμή του αρχείου κειμένου μετατρέπεται σε **μία μόνο λίστα** με τις λέξεις της γραμμής. Αυτό έχει ως επακόλουθο στην απάντηση να πάρουμε ένα RDD από λίστες λέξεων, και όχι ένα RDD από λέξεις όπως θα θέλαμε. Άρα, το αποτέλεσμα που θα πάρουμε είναι:

```
< ["this", "is", "a", "line"], ["this", "is", "another", "line"] >
```

Παρατηρούμε ότι, αντί να έχουμε ένα RDD από `String` αντικείμενα, έχουμε ένα RDD από αντικείμενα τύπου `List[String]` (λίστες). Η χρήση του μετασχηματισμού `flatMap()` μας λύνει τα χέρια, καθώς μπορούμε να πάρουμε απευθείας το αποτέλεσμα που θέλουμε. Ο μετασχηματισμός `flatMap()`, στην ουσία, καταργεί τις εσωτερικές λίστες και δίνει απευθείας ένα RDD από λέξεις. Αυτό σημαίνει ότι ο μετασχηματισμός `flatMap()` είναι ένα-προς-πολλά, σε αντίθεση με τον μετασχηματισμό `map()` που είναι ένα-προς-ένα. Εάν χρησιμοποιήσουμε `flatMap()` στο προηγούμενο παράδειγμα, η απάντηση θα είναι:

```
< "this", "is", "a", "line", "this", "is", "another", "line" >
```

Ας δούμε μερικά ακόμη παραδείγματα μετασχηματισμών. Έστω ότι θέλουμε στην έξοδο τα μοναδικά στοιχεία ενός RDD, δηλαδή το κάθε στοιχείο να εμφανίζεται το πολύ μία φορά. Για παράδειγμα, στο RDD με τις λέξεις του προηγούμενου παραδείγματος, έστω ότι θέλουμε

να διαγράψουμε τις πολλαπλές εμφανίσεις μίας λέξης. Εδώ μπορούμε να χρησιμοποιήσουμε τον μετασχηματισμό `distinct()` ως εξής:

```
val txtFileRDD = sc.textFile("myfile.txt", 5)
val distinctWordsRDD = txtFileRDD.flatMap(line => line.split(' ')).
distinct()
```

Το αποτέλεσμα θα έχει ως εξής:

```
< "this", "another", "is", "a", "line" >
```

Σε πολλές περιπτώσεις, απαιτείται η εφαρμογή δειγματοληψίας στο αρχικά δεδομένα, με σκοπό να δοκιμάσουμε την εφαρμογή αλγορίθμων σε ένα υποσύνολο, για λόγους ταχύτητας. Σε τέτοιες περιπτώσεις, μπορούμε να εφαρμόσουμε τον μετασχηματισμό `sample()` ο οποίος λαμβάνει ως είσοδο ένα RDD και επιστρέφει ένα RDD το μέγεθος του οποίου είναι ένα ποσοστό του αρχικού. Για παράδειγμα, έστω ότι από το RDD που περιέχει τις λέξεις του αρχείου κειμένου θέλουμε να κρατήσουμε ένα δείγμα που να ανταποκρίνεται στο 20% των λέξεων του αρχικού RDD (χωρίς να χρησιμοποιήσουμε τον μετασχηματισμό `distinct()`). Αυτό μπορούμε να το πετύχουμε ως εξής:

```
val txtFileRDD = sc.textFile("myfile.txt", 5)
val wordsRDD = txtFileRDD.flatMap(line => line.split(' '))
val sampledRDD = wordsRDD.sample(false, 0.2, 42)
```

Στο παράδειγμα αυτό, η συνάρτηση `sample()` δέχεται τα εξής ορίσματα:

1. Το πρώτο όρισμα (`false`) αναφέρει ότι η δειγματοληψία θα πραγματοποιηθεί χωρίς αντικατάσταση, δηλαδή, εάν κάποιο στοιχείο επιλεγεί, παραμένει εκτός και δεν μπορεί να επιλεγεί δεύτερη φορά. Προφανώς, αυτό ισχύει για τα στοιχεία που είναι μοναδικά στο RDD. Εάν ένα στοιχείο εμφανίζεται πολλές φορές, τότε προφανώς μπορεί να επιλεγεί και πάλι, αφού κάθε εμφάνιση του ίδιου στοιχείου θεωρείται ξεχωριστή.
2. Το δεύτερο όρισμα (`0.2`) δηλώνει το ποσοστό του αρχικού RDD που θέλουμε. Το `0.2` σημαίνει ότι στην έξοδο θα πάρουμε περίπου το 20% των στοιχείων του αρχικού RDD. Επομένως, το τελικό RDD μπορεί να μην ανταποκρίνεται ακριβώς στο 20% του αρχικού RDD, εξαιτίας του τρόπου με τον οποίο λειτουργεί η δειγματοληψία σε κατανεμημένο περιβάλλον.
3. Το τρίτο όρισμα (`42`) είναι ο σπόρος (`seed`) για τη γεννήτρια τυχαίων αριθμών, έτσι ώστε κάθε φορά που εκτελείται ο μετασχηματισμός να λαμβάνουμε τα ίδια στοιχεία στο δείγμα.

Εκτός από τους μετασχηματισμούς που επιδρούν σε ένα RDD, υπάρχουν και μετασχηματισμοί που δέχονται δύο RDDs στην είσοδο. Για παράδειγμα, ας υποθέσουμε ότι έχουμε δύο RDDs με ακέραιους αριθμούς με περιεχόμενα: `< 1, 2, 3, 4 >` και `< 1, 2, 3, 5, 7 >`. Υποθέτουμε ότι θέλουμε στην έξοδο την τομή των δύο RDDs. Εάν τα ονόματα των μεταβλητών είναι `rdd1` και `rdd2` τότε μπορούμε να γράψουμε:

```
val intersectionRDD = rdd1.intersection(rdd2)
```

Γενικά, όταν ένας μετασχηματισμός απαιτεί στην είσοδο δύο RDDs, τότε εφαρμόζουμε τον μετασχηματισμό σε ένα από τα δύο RDDs και δίνουμε ως παράμετρο το δεύτερο RDD. Με παρόμοιο τρόπο μπορούμε να υπολογίσουμε την ένωση (union), διαφορά (subtract) και καρτεσιανό γινόμενο (cartesian). Ας δούμε πώς μπορούμε να γράψουμε όλους τους μετασχηματισμούς και ποιο θα είναι το αποτέλεσμα σε κάθε περίπτωση.

```
val intersectionRDD = rdd1.intersection(rdd2)
val subtractRDD = rdd1.subtract(rdd2)
val unionRDD = rdd1.union(rdd2)
val cartesianRDD = rdd1.cartesian(rdd2)
```

Στην περίπτωση της τομής, το αποτέλεσμα είναι: $\langle 1, 2, 3 \rangle$, για την ένωση θα έχουμε ως αποτέλεσμα $\langle 1, 2, 3, 4, 1, 2, 3, 4, 5, 7 \rangle$, για τη διαφορά θα έχουμε ως αποτέλεσμα $\langle 4 \rangle$ και, τέλος, το καρτεσιανό γινόμενο θα επιστρέψει ένα RDD με όλα τα δυνατά ζεύγη στοιχείων από τα δύο RDDs, δηλαδή $\langle (1, 1), (1, 2), (1, 3), (1, 5), (1, 7), (2, 1), (2, 2), (2, 5), \dots \rangle$.

Οι μετασχηματισμοί, ανάλογα με την πολυπλοκότητά τους, χωρίζονται σε δύο βασικές κατηγορίες: μετασχηματισμοί *στενής συσχέτισης* και μετασχηματισμοί *ευρείας συσχέτισης*. Έστω ότι ένα RDD αποτελείται από 3 διαμερίσεις P_1, P_2, P_3 και υποθέτουμε ότι η κάθε διαμέριση έχει ανατεθεί σε διαφορετικό εκτελεστή που υποθέτουμε ότι βρίσκεται σε διαφορετικό κόμβο της συστάδας. Έστω, επίσης, ότι θέλουμε να εκτελέσουμε την πράξη `map()`, ώστε να μετασχηματίσουμε το κάθε στοιχείο του RDD, και ότι οι διαμερίσεις του RDD εξόδου είναι C_1, C_2 και C_3 . Είναι προφανές, ότι για να παραχθεί η διαμέριση C_1 απαιτείται μόνο η P_1 . και γενικότερα η παραγωγή της διαμέρισης C_j εξαρτάται μόνο από τη διαμέριση P_j . Στην περίπτωση αυτή, ο μετασχηματισμός καλείται στενής συσχέτισης, καθώς είναι καθορισμένο το σύνολο των διαμερίσεων του αρχικού RDD που επηρεάζουν το τελικό RDD. Στην ίδια κατηγορία μετασχηματισμών ανήκουν οι `filter()`, `flatMap()`, `sample()` κ.ά.

Ωστόσο, δεν έχουν όλοι οι μετασχηματισμοί την ίδια απλότητα. Υπάρχουν μετασχηματισμοί στους οποίους δεν μπορούμε να εντοπίσουμε τις διαμερίσεις του αρχικού RDD που θα επηρεάσουν το τελικό RDD, και ενδεχομένως να απαιτούνται όλες οι διαμερίσεις του αρχικού RDD για να παραχθεί μία διαμέριση του τελικού RDD. Οι μετασχηματισμοί αυτοί καλούνται μετασχηματισμοί ευρείας συσχέτισης και χαρακτηριστικά παραδείγματα είναι οι `sortByKey()`, `distinct()`, `intersection()`, `join()` κ.ά. Η σημαντικότητα των μετασχηματισμών ευρείας συσχέτισης είναι πολύ μεγάλη, καθώς καθορίζουν το πλήθος των shuffles που θα εκτελεστούν κατά την εκτέλεση των εφαρμογών.

Μια εφαρμογή Spark αποτελείται από μία ή περισσότερες εργασίες (jobs). Κάθε εργασία (job), αποτελείται από μία ακολουθία μετασχηματισμών που τελειώνει με μία ενέργεια. Η εκτέλεση κάθε εργασίας οργανώνεται σε στάδια (stages). Οι λειτουργίες που μπορούν να εκτελεστούν παράλληλα μπορούν να ομαδοποιηθούν στο ίδιο στάδιο. Ωστόσο, πολλές φορές, για να ξεκινήσει η εκτέλεση μία λειτουργίας πρέπει πρώτα να έχει ολοκληρωθεί η εκτέλεση της προηγούμενης, οπότε πρέπει να δημιουργηθούν περισσότερα στάδια τα οποία εκτελούνται ακολουθιακά. Το πλήθος των σταδίων, στην ουσία, ορίζεται από τους μετασχηματισμούς ευρείας συσχέτισης. Έτσι, η εκτέλεση μιας εργασίας ξεκινά με το 1ο στάδιο και, εάν στην πορεία συναντήσουμε μετασχηματισμό ευρείας συσχέτισης, ξεκινά το επόμενο στάδιο. Τονί-

ζουμε ότι όσο περισσότερους μετασχηματισμούς ευρείας συσχέτισης έχουμε, τόσο περισσότερα στάδια έχουμε και τόσο μεγαλύτερο το κόστος επεξεργασίας.

5.6.2 Ενέργειες

Ένα από τα βασικά στοιχεία του Spark είναι ότι οι μετασχηματισμοί δεν εκτελούνται παρά μόνο εάν στο τέλος υπάρχει μια συνάρτηση που χαρακτηρίζεται ως *ενέργεια* (action), η οποία στην ουσία δίνει το έναυσμα για την εκτέλεση της ακολουθίας των μετασχηματισμών. Επομένως, ο τρόπος με τον οποίο λειτουργεί το Spark χαρακτηρίζεται από την ιδιότητα της *τεμπέλικης αξιολόγησης* (lazy evaluation). Σύμφωνα με την ιδιότητα αυτή, δεν εκτελείται κανένας μετασχηματισμός εάν δεν υπάρχει κάποια ενέργεια.

Αρχικά, θα περιγράψουμε μερικές ενέργειες και, στη συνέχεια, θα επιχειρηματολογήσουμε για τη χρησιμότητα της τεμπέλικης αξιολόγησης. Μία από τις πιο βασικές ενέργειες είναι η συνάρτηση `count()`, η οποία μετρά το πλήθος των στοιχείων ενός RDD. Αξίζει να σημειώσουμε, ότι υπάρχει ένας απλός τρόπος να ξεχωρίσουμε εάν μια συνάρτηση είναι μετασχηματισμός ή ενέργεια. Ένας μετασχηματισμός επιδρά σε ένα RDD και επιστρέφει ένα RDD. Μια ενέργεια επιδρά σε ένα RDD αλλά δεν επιστρέφει RDD στην έξοδο, αλλά μια τιμή, ένα Array, μια συμβολοσειρά ή κάτι άλλο. Τονίζεται ότι, ενώ το RDD βρίσκεται στη συστάδα και υφίσταται επεξεργασία από τους εκτελεστές (executors), η τιμή επιστροφής μιας ενέργειας αποθηκεύεται στη διαδικασία οδήγησης (driver) που εκτελεί τη συνάρτηση `main()`.

Μία άλλη ενέργεια που χρησιμοποιείται πολλές φορές είναι η `collect()`, η οποία ανάλογα με την περίπτωση μπορεί να είναι και *επικίνδυνη*. Η συνάρτηση `collect()`, διαβάζει όλο το RDD και το επιστρέφει σε τοπική μεταβλητή στη διαδικασία οδήγησης. Είναι προφανές ότι, εάν το RDD έχει σχετικά μικρό μέγεθος και η ελεύθερη κύρια μνήμη στο μηχάνημα που εκτελείται η διαδικασία οδήγησης επαρκεί, δεν θα υπάρξει κάποιο πρόβλημα. Όμως, τις περισσότερες φορές, τα RDDs είναι μεγάλα σε μέγεθος και είναι αδύνατο να αποθηκευθούν στην κύρια μνήμη ενός μόνο μηχανήματος. Επομένως, η χρήση της `collect()` θα πρέπει να γίνεται με μεγάλη προσοχή, για την αποφυγή σφαλμάτων μνήμης. Άλλες ενέργειες που μπορούν να χρησιμοποιηθούν είναι `saveAsTextFile()`, `take()`, `reduce()`, `countByKey()`, κ.ά. Περισσότερες λεπτομέρειες θα βρει ο ενδιαφερόμενος αναγνώστης στην επίσημη τεκμηρίωση του Apache Spark <https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>.

Στη συνέχεια, θα εξηγήσουμε αναλυτικότερα τον μηχανισμό της τεμπέλικης αξιολόγησης. Ας υποθέσουμε ότι θέλουμε να εκτελέσουμε μία ακολουθία από μετασχηματισμούς και η ενέργεια που υπάρχει στο τέλος είναι η `take(5)`. Αυτό σημαίνει ότι στην έξοδο θέλουμε τα πέντε πρώτα στοιχεία του RDD. Για παράδειγμα, θα μπορούσαμε να έχουμε τον ακόλουθο κώδικα:

```
val inputRDD = sc.textFile("myfile.txt")
val applesRDD = inputRDD.filter(x => x.contains("apple"))
val orangesRDD = inputRDD.filter(x => x.contains("orange"))
val unionRDD = applesRDD.union(orangesRDD)

val numLines = unionRDD.count()
```



```
println(numLines)
unionRDD.take(5).foreach(println)
```

Στην ουσία, από ένα αρχείο κειμένου απομονώνουμε τις γραμμές που περιέχουν τη λέξη «apple» και τις γραμμές που περιέχουν τη λέξη «orange». Έτσι δημιουργούνται δύο διαφορετικά RDDs, τα `applesRDD` και `orangesRDD`. Στη συνέχεια, χρησιμοποιώντας τη συνάρτηση `union()` παίρνουμε την ένωση των δύο RDDs. Μέχρι και αυτό το σημείο, το Spark δεν έχει εκτελέσει καμία συνάρτηση, επειδή όλες είναι μετασχηματισμοί. Ωστόσο, αυτό που έχει καταχωρίσει είναι οι εξαρτήσεις που έχουν τα RDDs μεταξύ τους. Στη συνέχεια, βλέπουμε δύο ενέργειες, την `count()` και την `take()`. Όταν συναντήσουμε την ενέργεια `count()`, θα εκτελεστούν όλοι οι μετασχηματισμοί της ακολουθίας μέχρι να εξαχθεί το τελικό αποτέλεσμα και να μετρηθούν τα στοιχεία στο τελικό RDD. Όμως, παρατηρούμε ότι η δεύτερη ενέργεια `take(5)` δίνει στην έξοδο τα πρώτα πέντε στοιχεία του RDD. Αυτό σημαίνει ότι δεν χρειάζεται να εκτελεστεί η πράξη `union()` μέχρι να ολοκληρωθεί το συνολικό αποτέλεσμα, αλλά η εκτέλεση μπορεί να διακοπεί όταν στο αποτέλεσμα βρεθούν πέντε στοιχεία. Καταλαβαίνουμε ότι, με αυτόν τον τρόπο, έχουμε μεγάλη οικονομία στον χρόνο επεξεργασίας, καθώς η εκτέλεση τερματίζεται πολύ νωρίτερα. Επίσης, όταν έχουμε τη συνολική εικόνα της επεξεργασίας, μπορούμε να εφαρμόσουμε τεχνικές βελτιστοποίησης, οι οποίες δεν μπορούν να εφαρμοστούν όταν εκτελούνται άμεσα οι μετασχηματισμοί. Λαμβάνοντας υπόψη ότι οι εργασίες που εκτελούνται στο Spark σχετίζονται με την επεξεργασία μεγάλων δεδομένων, ο μηχανισμός της τεμπέλικης αξιολόγησης μπορεί να μειώσει σημαντικά τον χρόνο εκτέλεσης των εφαρμογών και να συμβάλει στην εξοικονόμηση πόρων.

5.7 Επίβλεψη εκτέλεσης εφαρμογών Spark

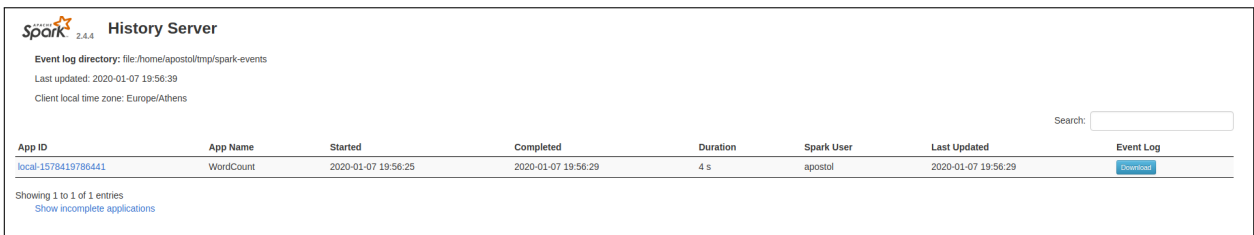
Όταν εκτελείται μια εφαρμογή Spark, είναι σημαντικό να μπορούμε να παρακολουθούμε την πρόοδό της και να λαμβάνουμε κάποια στατιστικά στοιχεία σχετικά με τον τρόπο που εκτελούνται οι εφαρμογές. Για να διευκολυνθεί αυτό, πρόκειται να ρυθμίσουμε τον *διακομιστή ιστορικού* (History Server), ο οποίος είναι ένα εξαιρετικό εργαλείο που επιτρέπει την ανασκόπηση της εκτέλεσης μέσω ενός προγράμματος περιήγησης Ιστού. Η βασική ιδέα είναι ότι όταν εκτελείται μια εφαρμογή Spark παράγει μια ακολουθία από αρχεία καταγραφής. Αυτά τα αρχεία καταγραφής μπορούν αργότερα να μελετηθούν χρησιμοποιώντας μια εξειδικευμένη διεπαφή Web που είναι διαθέσιμη στη διεύθυνση `http://localhost:18080`.

Αρχικά, θα ενεργοποιήσουμε το μηχανισμό καταγραφής του Spark. Όλα τα αρχεία καταγραφής θα αποθηκεύονται στον φάκελο `/home/oszy/tmp/spark-events`.

```
mkdir /home/oszy/bdc/tmp
mkdir /home/oszy/bdc/tmp/spark-events
```

Στη συνέχεια, πρέπει να καθορίσουμε τον φάκελο στον οποίο οι εφαρμογές Spark θα αποθηκεύουν τα αρχεία καταγραφής. Πάμε μέσα στον φάκελο διαμόρφωσης του Spark `/home/oszy/bdc/spark-2.4.4bin-hadoop2.7/conf`. Μέσα σε αυτόν τον φάκελο υπάρχει ένα αρχείο με το όνομα `sparkdefaults.conf.template`. Πρέπει να αντιγράψουμε αυτό το αρχείο στο `spark-defaults.conf`:

```
cd /home/oszy/bdc/spark-2.4.4-bin-hadoop2.7/conf
```



spark 2.4.4 History Server

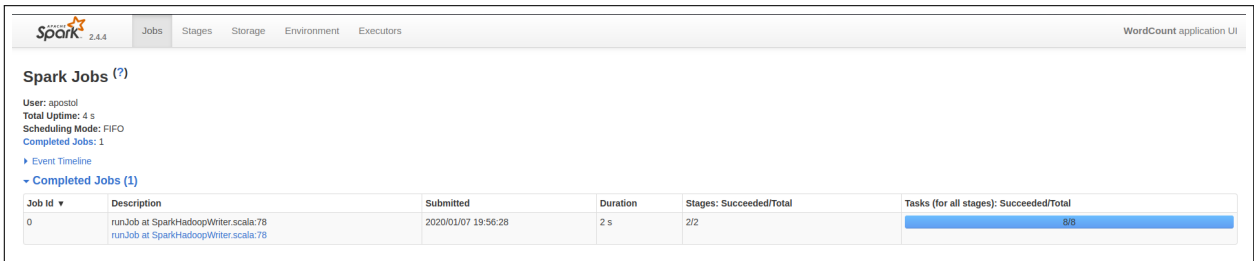
Event log directory: file:/home/apostol/tmp/spark-events
 Last updated: 2020-01-07 19:56:39
 Client local time zone: Europe/Athens

Search:

App ID	App Name	Started	Completed	Duration	Spark User	Last Updated	Event Log
local-1578419786441	WordCount	2020-01-07 19:56:25	2020-01-07 19:56:29	4 s	apostol	2020-01-07 19:56:29	Download

Showing 1 to 1 of 1 entries
[Show incomplete applications](#)

Σχήμα 5.3: Τα περιεχόμενα του συνδέσμου `http://localhost:18080`.



Spark Jobs (?)

User: apostol
 Total Uptime: 4 s
 Scheduling Mode: FIFO
 Completed Jobs: 1

▶ Event Timeline

~ Completed Jobs (1)

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2020/01/07 19:56:28	2 s	2/2	8/8

Σχήμα 5.4: Λεπτομέρειες για την εκτέλεση της εφαρμογής *WordCount*.

```
cp spark-defaults.conf.template spark-defaults.conf
```

Στη συνέχεια, επεξεργαζόμαστε το αρχείο `spark-defaults.conf` εισάγοντας τις ακόλουθες γραμμές:

```
spark.eventLog.enabled      true
spark.eventLog.dir          file:/home/ozzy/tmp/spark-events
spark.history.fs.logDirectory file:/home/ozzy/tmp/spark-events
```

Για την εκκίνηση του Spark history server εκτελούμε:

```
start-history-server file:/home/ozzy/bdc/tmp/spark-events
```

Για να επιβεβαιώσουμε ότι ο history server λειτουργεί κανονικά μπορούμε να εκτελέσουμε την εντολή `jobs` στην κονσόλα. Αναμένουμε στην έξοδο:

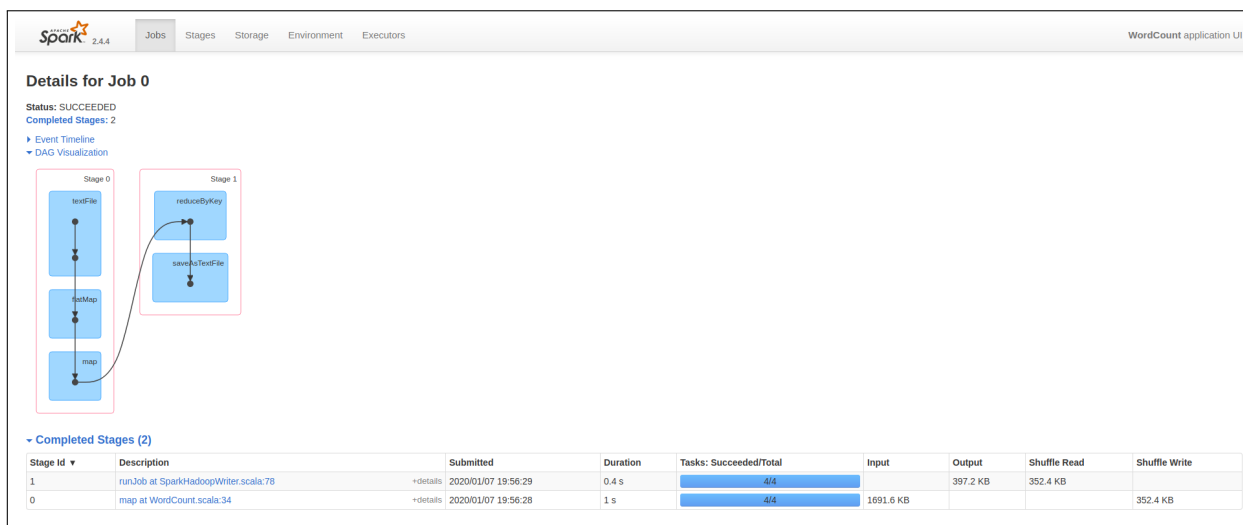
```
1 6675 Jps
2 6084 HistoryServer
```

Τώρα, θα εκτελέσουμε μια εφαρμογή Spark και μετά θα ελέγξουμε πώς ακριβώς εκτελέστηκε. Επομένως, πρέπει να επιστρέψουμε στην εφαρμογή *WordCount* (θα χρησιμοποιήσουμε την έκδοση Scala εδώ) και, στη συνέχεια, να χρησιμοποιήσουμε το `spark-submit`:

```
cd /home/ozzy/bdc/sparklab/python/WordCount
spark-submit ./target/scala-2.11/wordcount_2.11-1.0.jar
```

Μετά την ολοκλήρωση της εφαρμογής *WordCount*, μπορούμε να ελέγξουμε τον τρόπο εκτέλεσης της εφαρμογής επιθεωρώντας τα αρχεία καταγραφής. Ας επισκεφτούμε το `http://localhost:18080` χρησιμοποιώντας οποιοδήποτε πρόγραμμα περιήγησης Ιστού. Η έξοδος θα πρέπει να μοιάζει με αυτό που φαίνεται στο Σχήμα 5.3.

Εάν κάνουμε κλικ στο πεδίο App ID, η έξοδος είναι όπως αυτή που φαίνεται στο Σχήμα 5.4. Σε αυτό το σχήμα, παρατηρούμε κάποιες πρόσθετες πληροφορίες σχετικά με την εκτέλεση της εφαρμογής *WordCount*. Για παράδειγμα, βλέπουμε ότι η εφαρμογή απαίτησε 2 δευτερό-



Σχήμα 5.5: Λεπτομέρειες για τα στάδια για μία συγκεκριμένη εργασία.

λεπτα για να εκτελεστεί και ότι η εφαρμογή ολοκληρώθηκε σε δύο (2) στάδια. Δεν έχουμε συζητήσει ακόμη για την έννοια του σταδίου, αλλά αυτό θα γίνει στο επόμενο κεφάλαιο.

Για να λάβουμε περισσότερες λεπτομέρειες σχετικά με την εκτέλεση, εάν κάνουμε κλικ στον σύνδεσμο `runJob at SparkHadoopWriter.scala:78`, θα δούμε στην έξοδο κάτι παρόμοιο με το Σχήμα 5.5. Σε αυτό το σχήμα βλέπουμε ξεκάθαρα ότι εμπλέκονται δύο στάδια και, επίσης, παρατηρούμε τις συναρτήσεις που εκτελούνται σε κάθε στάδιο. Αξίζει να αφιερωθεί λίγος χρόνος για την εξέταση των πληροφοριών που παρέχονται σχετικά με την εκτέλεση της εφαρμογής κάνοντας κλικ στους διαθέσιμους συνδέσμους. Για να σταματήσουμε την εκτέλεση του διακομιστή ιστορικού, πρέπει να δώσουμε την εντολή:

```
stop-history-server.sh
```

Για περισσότερες πληροφορίες σχετικά με τον διακομιστή ιστορικού, ο ενδιαφερόμενος αναγνώστης θα πρέπει να ανατρέξει στην επίσημη τεκμηρίωση του Apache Spark στον σύνδεσμο <https://spark.apache.org/docs/latest/monitoring.html>.

5.8 Περαιτέρω μελέτη

Στο τρέχον κεφάλαιο εξετάστηκαν τα βασικά συστατικά στοιχεία του Spark και ο τρόπος εκτέλεσης των εφαρμογών. Επίσης, εντοπίσαμε τους βασικούς παίκτες: τη διεργασία οδήγησης (driver), τους εκτελεστές (executors) και τις διαμερίσεις (partitions). Επιπλέον, παρουσιάσαμε τον τρόπο εκτέλεσης εφαρμογών Spark και το πώς μπορούμε να δουλέψουμε με τη γλώσσα Scala ή τη γλώσσα Python. Είναι προφανές ότι σε ένα μόνο κεφάλαιο δεν μπορούμε να καλύψουμε όλες τις λεπτομέρειες και, επομένως, ο ενδιαφερόμενος αναγνώστης θα πρέπει να ανατρέξει σε βιβλιογραφικές αναφορές που εστιάζονται στο Spark και μελετούν το σύστημα σε βάθος. Μερικές από αυτές είναι οι: [ZXW⁺16], [KW17], [Sal17], [CZ18]. Επίσης, προτείνουμε τη μελέτη της επίσημης τεκμηρίωσης του Apache Spark στον σύνδεσμο <https://spark.apache.org/>.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [CZ18] Bill Chambers and Matei Zaharia. *Spark: The definitive guide: Big data processing made simple*. O'Reilly Media, Inc., 2018.
- [KW17] Holden Karau and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. O'Reilly Media, Inc., 1st edition, 2017.
- [Sal17] Neelesh Srinivas Salian. *Spark Operations Cookbook: Solving the Practical Challenges of Spark Implementation*. O'Reilly Media, Inc., 1st edition, 2017.
- [ZXW⁺16] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, and et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

ΚΕΦΑΛΑΙΟ 6

ΠΡΟΧΩΡΗΜΕΝΑ ΘΕΜΑΤΑ ΑΡΑΧΕ SPARK

Περιεχόμενα

6.1	Εισαγωγή	150
6.2	Βελτίωση επίδοσης	150
6.2.1	Διατήρηση δεδομένων	150
6.2.2	Μετάδοση δεδομένων	154
6.2.3	Συσσώρευση δεδομένων	156
6.3	Επεξεργασία δομημένων δεδομένων	158
6.4	Εξόρυξη δεδομένων και ανακάλυψη γνώσης	164
6.5	Περαιτέρω μελέτη	168
	ΒΙΒΛΙΟΓΡΑΦΙΑ	169



Σύνοψη

Στο προηγούμενο κεφάλαιο, παρουσιάσαμε τα βασικά χαρακτηριστικά του συστήματος Spark και εστιάσαμε σε απλές εφαρμογές και τεχνικές επεξεργασίας. Στο κεφάλαιο αυτό, προχωράμε βαθύτερα στο σύστημα Spark και μελετάμε σημαντικά χαρακτηριστικά του συστήματος που σχετίζονται με τη βελτίωση των επιδόσεων του συστήματος, την επεξεργασία δεδομένων μεγάλης κλίμακας και την εξόρυξη γνώσης από μεγάλα δεδομένα.

Προαπαιτούμενη γνώση

Προτείνουμε στον αναγνώστη να μελετήσει το υλικό του προηγούμενου κεφαλαίου, το οποίο περιέχει μια εισαγωγή στα βασικά χαρακτηριστικά του Spark, πριν μελετήσει το τρέχον κεφάλαιο.

6.1 Εισαγωγή

Το σύστημα Spark έχει πολλές δυνατότητες αποδοτικής επεξεργασίας μεγάλων δεδομένων. Ενώ στο προηγούμενο κεφάλαιο εξετάσαμε μερικά από τα βασικά του χαρακτηριστικά, στο κεφάλαιο αυτό μελετάμε σε μεγαλύτερο βάθος κάποια από τα χαρακτηριστικά του που το έχουν καταστήσει ένα από τα πιο πετυχημένα συστήματα επεξεργασίας μεγάλων δεδομένων. Αναλυτικότερα, εστιάζουμε σε τρεις διαφορετικούς άξονες: 1) χρήση τεχνικών για τη βελτίωση της απόδοσης, 2) χρήση μετασχηματισμών σε DataFrames και της γλώσσας SQL και 3) χρήση αλγορίθμων μηχανικής μάθησης με στόχο την εξόρυξη γνώσης.

6.2 Βελτίωση επίδοσης

Αν και, όπως θα δούμε παρακάτω, το Spark διαθέτει σημαντικούς μηχανισμούς βελτιστοποίησης της επίδοσης, ειδικά σε ό,τι αφορά την επεξεργασία δομημένων δεδομένων (μέσω του DataFrame API), θα πρέπει να προσέξουμε ιδιαίτερα κατά την ανάπτυξη εφαρμογών, διότι μπορούν να εφαρμοστούν επιπλέον μηχανισμοί βελτίωσης της επίδοσης. Εδώ θα εξετάσουμε τους ακόλουθους μηχανισμούς βελτίωσης επίδοσης: διατήρηση δεδομένων (persistence), μετάδοση δεδομένων (broadcasting) και συσσώρευση δεδομένων (accumulation).

Οι μηχανισμοί αυτοί είναι ανεξάρτητοι από τον εκάστοτε αλγόριθμο που χρησιμοποιείται για την επεξεργασία ή ανάλυση δεδομένων και μπορούν να ερμηνευθούν ως καλές πρακτικές που μπορούν να βοηθήσουν σημαντικά στη μείωση του χρόνου εκτέλεσης.

6.2.1 Διατήρηση δεδομένων

Η διατήρηση δεδομένων στο Apache Spark, γνωστή και ως προσωρινή αποθήκευση (caching), είναι ένα βασικό χαρακτηριστικό του Apache Spark που ενισχύει την απόδοση και την αποτελεσματικότητα των εργασιών επεξεργασίας κατανεμημένων δεδομένων. Επιτρέπει στο Spark

να αποθηκεύει ενδιάμεσα δεδομένα ή δεδομένα με συχνή πρόσβαση στη μνήμη ή στον δίσκο, μειώνοντας την ανάγκη επανυπολογισμού ή επαναφόρτωσης των ίδιων δεδομένων πολλές φορές κατά την εκτέλεση μιας εφαρμογής. Αυτό βελτιώνει τη συνολική ταχύτητα και τη χρήση πόρων των εργασιών Spark, καθιστώντας το μια θεμελιώδη τεχνική βελτιστοποίησης στην επεξεργασία μεγάλων δεδομένων. Τα βασικά στοιχεία της διατήρησης δεδομένων είναι τα ακόλουθα:

- **Προσωρινή αποθήκευση δεδομένων.** Το σύστημα διατήρησης δεδομένων του Spark επιτρέπει την προσωρινή αποθήκευση δεδομένων στη μνήμη ή στον δίσκο. Τα δεδομένα μπορεί να είναι οργανωμένα σε RDDs (Resilient Distributed Datasets), DataFrames ή Datasets, τα οποία αποτελούν τις κύριες αφαιρέσεις για την εργασία με κατανεμημένα δεδομένα στο Spark.
- **Αποθήκευση στην κύρια μνήμη.** Ένα από τα βασικά πλεονεκτήματα του Spark persistence είναι η ικανότητά του να αποθηκεύει δεδομένα στη μνήμη. Όταν τα δεδομένα αποθηκεύονται προσωρινά στη μνήμη, οι επόμενες λειτουργίες που απαιτούν τα δεδομένα μπορούν να έχουν πρόσβαση σε αυτά πολύ πιο γρήγορα από την ανάγνωση από τον δίσκο ή τον επανυπολογισμό τους. Αυτή η αποθήκευση στη μνήμη βελτιώνει σημαντικά την ταχύτητα των επαναληπτικών αλγορίθμων και τη διαδραστική εξερεύνηση δεδομένων.
- **Αποθήκευση στον δίσκο.** Σε περιπτώσεις όπου τα δεδομένα δεν χωρούν εξ ολοκλήρου στη μνήμη, το Spark επιτρέπει στους χρήστες να διατηρούν δεδομένα στον δίσκο. Ενώ η αποθήκευση του δίσκου είναι πιο αργή από τη μνήμη, παρέχει ανοχή σφαλμάτων και διασφαλίζει ότι ακόμη και μεγάλα σύνολα δεδομένων μπορούν να υποβληθούν σε αποτελεσματική επεξεργασία.
- **Επίπεδα διατήρησης.** Το Spark προσφέρει πολλαπλά επίπεδα διατήρησης των δεδομένων, επιτρέποντας στους χρήστες να επιλέγουν την αντιστάθμιση μεταξύ χρήσης μνήμης και ταχύτητας επεξεργασίας. Τα επίπεδα διατήρησης είναι MEMORY_ONLY, MEMORY_ONLY_SER, MEMORY_AND_DISK, MEMORY_AND_DISK_SER, κ.ά. Κάθε επίπεδο εξισορροπεί διαφορετικά τη χρήση της κύριας μνήμης και του δίσκου.
- **Γενεαλογία και ανοχή στα σφάλματα.** Το Spark διατηρεί πληροφορίες γενεαλογίας για κάθε RDD, το οποίο είναι ένα κατευθυνόμενο άκυκλο γράφημα (DAG) που αντιπροσωπεύει τους μετασχηματισμούς που εφαρμόζονται στα δεδομένα. Αυτές οι πληροφορίες γενεαλογίας εξασφαλίζουν ανοχή στα σφάλματα. Εάν για κάποιους λόγους χαθεί μια διαμέριση ενός αποθηκευμένου RDD, το Spark μπορεί να την υπολογίσει εκ νέου χρησιμοποιώντας τους αρχικούς μετασχηματισμούς.
- **Διαχείριση κρυφής μνήμης.** Ενώ η προσωρινή αποθήκευση μπορεί να ενισχύσει σημαντικά την απόδοση, είναι σημαντικό να διαχειριζόμαστε αποτελεσματικά τα δεδομένα της κρυφής μνήμης. Οι χρήστες μπορούν να διατηρήσουν ή να αφαιρέσουν ρητά αποθηκευμένα δεδομένα όταν δεν χρειάζονται πλέον, ώστε να απελευθερωθούν πόροι μνήμης ή δίσκου.

- **Δυναμική προσωρινή αποθήκευση.** Το Spark επιτρέπει τη δυναμική αποθήκευση στην κρυφή μνήμη, πράγμα που σημαίνει ότι μπορούμε να αποθηκεύουμε προσωρινά ή να αποσυναρμολογούμε δεδομένα κατά την εκτέλεση μιας εργασίας Spark. Αυτή η ευελιξία είναι χρήσιμη όταν χρειάζεται να αποθηκεύουμε προσωρινά μόνο συγκεκριμένα τμήματα δεδομένων, με βάση τη ροή εκτέλεσης.
- **Παρακολούθηση και βελτιστοποίηση.** Διατίθενται εργαλεία και τεχνικές παρακολούθησης που βοηθούν τους χρήστες να παρακολουθούν την αποτελεσματικότητα της προσωρινής αποθήκευσης στις εφαρμογές Spark. Η δημιουργία προφίλ και η βελτιστοποίηση είναι απαραίτητες, για να διασφαλιστεί ότι η προσωρινή αποθήκευση βελτιώνει τη συνολική απόδοση της εφαρμογής.
- **Περιπτώσεις χρήσης.** Η ιδιότητα της διατήρησης δεδομένων είναι ευεργετική σε ένα ευρύ φάσμα περιπτώσεων χρήσης, συμπεριλαμβανομένων των επαναληπτικών αλγορίθμων μηχανικής μάθησης, της επεξεργασίας γραφημάτων, της διαδραστικής ανάλυσης δεδομένων και της αποθήκευσης δεδομένων. Με τη διατήρηση μειώνεται η υπολογιστική επιβάρυνση και ελαχιστοποιείται η περιττή επεξεργασία δεδομένων.

Συμπερασματικά, η διατήρηση δεδομένων στο Spark είναι ένα κρίσιμο χαρακτηριστικό για τη βελτιστοποίηση της απόδοσης των εφαρμογών επεξεργασίας μεγάλων δεδομένων. Αποθηκεύοντας σημαντικά δεδομένα στη μνήμη ή στον δίσκο και διαχειριζόμενοι αυτά αποτελεσματικά, οι χρήστες του Spark μπορούν να μειώσουν σημαντικά τους χρόνους υπολογισμού και να βελτιώσουν την αποτελεσματικότητα των ροών εργασίας επεξεργασίας δεδομένων, καθιστώντας το ένα ισχυρό εργαλείο για επεκτάσιμη και υψηλής απόδοσης επεξεργασία δεδομένων.

Στη συνέχεια, δίνεται ένα παράδειγμα εφαρμογής της διατήρησης δεδομένων, το οποίο εστιάζει στο RDD API. Στο προηγούμενο κεφάλαιο, μελετήθηκαν οι δύο βασικές κατηγορίες συναρτήσεων του Spark, οι *μετασχηματισμοί* και οι *ενέργειες*. Όπως έχει αναφερθεί, λόγω της τεμπέλικης επεξεργασίας, οι μετασχηματισμοί σε ένα RDD θα εκτελεστούν μόνο εάν υπάρχει κάποια ενέργεια. Τι θα συμβεί, όμως, εάν υπάρχει και δεύτερη ενέργεια η οποία πρέπει να εκτελεστεί στο ίδιο RDD; Εάν δεν ληφθεί κάποια μέριμνα, η ακολουθία μετασχηματισμών θα εκτελεστεί και πάλι, επιβαρύνοντας σημαντικά το κόστος επεξεργασίας της εφαρμογής.

Έστω ότι έχουμε το ακόλουθο τμήμα κώδικα, όπου αρχικά δημιουργείται το RDD με όνομα `txtFileRDD` το οποίο βασίζεται στο αρχείο κειμένου `leonardo.txt`. Στη συνέχεια, δημιουργείται το RDD `myRDD` το οποίο βασίζεται στο `txtFileRDD` και απλώς αποθηκεύει όλες τις λέξεις του αρχείου κειμένου με αριθμό χαρακτήρων μεγαλύτερο από 3. Ακολουθώντας, εκτελούνται δύο διαφορετικές ενέργειες πάνω στο `myRDD`: ενέργεια `count` για τη μέτρηση των λέξεων του `myRDD` και ενέργεια `saveAsTextFile` για την αποθήκευση του `myRDD` σε ένα αρχείο κειμένου.

```
val txtFileRDD = sc.textFile("leonardo.txt")
val myRDD = txtFileRDD
    .flatMap(line => line.split(' '))
    .filter(word => word.length() > 3)

val wordCounter = myRDD.count // πρώτη ενέργεια
myRDD.saveAsTextFile("myfile.txt") // δεύτερη ενέργεια
```

Εάν δεν ληφθεί κάποια μέριμνα, κατά την εκτέλεση της ενέργειας `count` θα εκτελεστούν οι μετασχηματισμοί `textFile`, `flatMap` και `filter`, έτσι ώστε να προκύψει το RDD `myRDD` για να μπορεί να μετρηθεί το συνολικό πλήθος των λέξεων. Επιπλέον, η δεύτερη ενέργεια `saveAsTextFile` θα ενεργοποιήσει την ίδια ακριβώς εκτέλεση μετασχηματισμών, διότι το RDD `myRDD` δεν είναι διαθέσιμο για επεξεργασία, καθώς το Spark, μετά την εκτέλεση μιας ενέργειας, δεν διατηρεί αυτόματα τα δεδομένα για επιπλέον επεξεργασία. Εδώ ακριβώς έρχεται η ιδιότητα της διατήρησης δεδομένων να δώσει λύση στο πρόβλημά μας.

Ας δούμε πώς μπορεί να αλλάξει λίγο ο κώδικας εισάγοντας μία επιπλέον γραμμή, πριν από την εκτέλεση της πρώτης ενέργειας `count`. Θα χρησιμοποιήσουμε τη συνάρτηση `persist`, η οποία επιτρέπει τη διατήρηση των δεδομένων στην κύρια μνήμη και στον δίσκο, όπως έχει αναφερθεί και προηγουμένως.

```
val txtFileRDD = sc.textFile("leonardo.txt")
val myRDD = txtFileRDD
  .flatMap(line => line.split(' '))
  .filter(word => word.length() > 3)

myRDD.persist("StorageLevel.MEMORY_AND_DISK") // εντολή διατήρησης

val wordCounter = myRDD.count // πρώτη ενέργεια
myRDD.saveAsTextFile("myfile.txt") // δεύτερη ενέργεια
```

Η αλλαγή αυτή είναι πολύ σημαντική. Στην ουσία, αναφέρουμε στο Spark ότι, μετά την εκτέλεση της ακολουθίας συναρτήσεων για τη μέτρηση των λέξεων, τα δεδομένα του `myRDD` θα διατηρηθούν, διότι θα τα χρειαζούμαστε πάλι. Είναι πολύ σημαντικό να γίνει κατανοητό ότι, τις περισσότερες φορές η διατήρηση των δεδομένων δεν μπορεί να γίνει μόνο στην κύρια μνήμη, λόγω μεγέθους των δεδομένων. Για τον λόγο αυτόν, συνήθως, χρησιμοποιούμε την παράμετρο `MEMORY_AND_DISK`, με αποτέλεσμα κάποια δεδομένα να διατηρούνται στην κύρια μνήμη και κάποια άλλα στον δίσκο. Είναι προφανές ότι θα θέλαμε όλα τα δεδομένα να βρίσκονται στην κύρια μνήμη, ωστόσο αυτό δεν είναι εφικτό για προφανείς λόγους.

Ιδιαίτερη προσοχή απαιτείται στην αποδέσμευση του χώρου όταν δεν χρειαζόμαστε πλέον τα δεδομένα του RDD. Δηλαδή, με συνεχόμενες εκτελέσεις της `persist` δεσμεύονται πολύτιμοι πόροι του συστήματος. Εάν δεν χρειαζόμαστε πλέον τα δεδομένα του RDD, μια καλή πρακτική είναι να χρησιμοποιούμε τη συνάρτηση `unpersist` έτσι ώστε να ελευθερώσουμε πόρους (κύρια μνήμη και δίσκο) έτσι ώστε να μπορούν να χρησιμοποιηθούν από άλλες εφαρμογές. Για το συγκεκριμένο παράδειγμα, παρακάτω δίνεται ο πλήρης κώδικας ο οποίος περιλαμβάνει και την εντολή αποδέσμευσης.

```
val txtFileRDD = sc.textFile("leonardo.txt")
val myRDD = txtFileRDD
  .flatMap(line => line.split(' '))
  .filter(word => word.length() > 3)

myRDD.persist("StorageLevel.MEMORY_AND_DISK") // εντολή διατήρησης

val wordCounter = myRDD.count // πρώτη ενέργεια
myRDD.saveAsTextFile("myfile.txt") // δεύτερη ενέργεια

myRDD.unpersist() // εντολή αποδέσμευσης
```

Εάν δεν αναφέρουμε κάποιο συγκεκριμένο επίπεδο αποθήκευσης, χρησιμοποιείται η αποθήκευση στην κύρια μνήμη (MEMORY_ONLY). Επιπλέον, εάν θέλουμε, μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `cache()` η οποία, στην ουσία, εκτελεί την ίδια εργασία με την `persist(StorageLevel.MEMORY_ONLY)`.

6.2.2 Μετάδοση δεδομένων

Σε πολλές περιπτώσεις, ένα τμήμα δεδομένων που βρίσκεται στον driver πρέπει να μεταφερθεί στους executors, ώστε να μπορούν να εκτελεστούν οι σχετικοί μετασχηματισμοί και ενέργειες. Φυσικά, εάν δεν γράψουμε επιπλέον κώδικα, η μεταφορά δεδομένων θα πραγματοποιηθεί, αλλά πολλές φορές μεταδίδεται μεγάλος όγκος δεδομένων στο δίκτυο, κάτι που επηρεάζει αρνητικά τον χρόνο εκτέλεσης της εφαρμογής.

Τονίζεται ότι ένα από τα σημαντικότερα κόστη στις κατανεμημένες εφαρμογές είναι το κόστος μετάδοσης δεδομένων μέσω του δικτύου. Λαμβάνοντας υπόψη ότι το δίκτυο αποτελεί κοινό πόρο σε ένα κατανεμημένο σύστημα, θα πρέπει οι εφαρμογές να περιορίσουν στο ελάχιστο τον όγκο δεδομένων που μεταδίδονται μέσω του δικτύου.

Στο πλαίσιο αυτό, το Spark διαθέτει έναν μηχανισμό *μετάδοσης δεδομένων* (broadcasting) ο οποίος μπορεί να χρησιμοποιηθεί σε περιπτώσεις που θέλουμε να επέμβουμε στον τρόπο με τον οποίο μεταδίδονται δεδομένα από τον driver στους executors. Τα δεδομένα που μεταδίδονται πρέπει να είναι μόνο για ανάγνωση (read-only) και θα πρέπει να μπορούν να αποθηκευθούν στην κύρια μνήμη των κόμβων όπου εκτελούνται οι Executors.

Ας υποθέσουμε ότι στη συνάρτηση `main()` της εφαρμογής μας, ορίζουμε έναν πίνακα `myArray` ο οποίος περιέχει ένα σύνολο από ακέραιους αριθμούς. Υποθέτουμε ότι το μέγεθος αυτού του πίνακα είναι αρκετά μεγάλο, π.χ. 2 GB. Επίσης, υποθέτουμε ότι ο πίνακας αυτός συμμετέχει σε έναν μετασχηματισμό `filter()` ο οποίος πρέπει να εκτελεστεί σε ένα RDD. Αυτό που ζητούμε είναι η `filter()` να κρατήσει μόνο τους αριθμούς που περιέχονται στο RDD και επίσης υπάρχουν στο `myArray`. Εάν δεν είχαμε στη διάθεσή μας τον μηχανισμό μετάδοσης, θα μπορούσαμε να λύσουμε το πρόβλημα χρησιμοποιώντας τον ακόλουθο κώδικα:

```
val r = scala.util.Random

val myRDD = Array.fill(1000000){r.nextInt(99)}.parallelize

val myArray = Array.fill(10000){r.nextInt(99)}

val numElements = myRDD.filter(x => myArray.contains(x)).count
```

Στον παραπάνω κώδικα, αρχικά, χρησιμοποιούμε τυχαίους αριθμούς για να δημιουργήσουμε ακέραιους που αποθηκεύονται στο `myRDD` και για τη δημιουργία του `myArray`. Εδώ να παρατηρήσουμε ότι αυτό γίνεται καθαρά για το παράδειγμα. Στην πράξη, τα στοιχεία του `myArray` μπορεί να διαβάζονται από κάποιο αρχείο, ενώ το `myRDD` μπορεί να προϋπάρχει. Αυτό που θέλουμε στο τέλος είναι το πλήθος των στοιχείων, οπότε χρησιμοποιούμε την ενέργεια `count`. Για να μπορέσει να ξεκινήσει η εκτέλεση των εργασιών στους executors θα πρέπει ο driver να στείλει τις απαραίτητες πληροφορίες. Το σύνολο των πληροφοριών

που απαιτούνται έτσι ώστε οι executors να μπορέσουν να εκτελέσουν τις εργασίες τους καλείται *κλειστότητα* (closure). Στο παράδειγμά μας επομένως, θα πρέπει ο driver να στείλει στους executors και το στοιχείο myArray.

Ο αναγνώστης στο σημείο αυτό μπορεί να αναρωτηθεί «Πού ακριβώς είναι το πρόβλημα;». Η κλειστότητα θα μεταδοθεί τόσες φορές όσες είναι οι διαμερίσεις του myRDD. Δηλαδή, εάν έχουμε ένα RDD που είναι κατανεμημένο σε 10 κόμβους και κάθε κόμβος περιέχει 100 διαμερίσεις, η κλειστότητα θα μεταδοθεί μέσω του δικτύου $10 \times 100 = 1.000$ φορές. Με τη μετάδοση δεδομένων η κλειστότητα θα μεταδοθεί μόνο 10 φορές, όσοι είναι οι κόμβοι. Αντιλαμβανόμαστε ότι το κέρδος που θα έχουμε θα είναι τεράστιο, καθώς αποφεύγουμε σημαντική σπατάλη των πόρων του συστήματος.

Στο σημείο αυτό έρχεται να δώσει λύση η μετάδοση δεδομένων. Συγκεκριμένα, για τα δεδομένα που θέλουμε να αναμεταδώσουμε (στην περίπτωση μας τον πίνακα myArray) πρέπει να δημιουργηθεί μια ειδική μεταβλητή. Στη συνέχεια, εάν θέλουμε να χρησιμοποιήσουμε τον πίνακα myArray, θα πρέπει αντί αυτού να χρησιμοποιηθεί η ειδική μεταβλητή που έχει δημιουργηθεί. Στον παρακάτω κώδικα βλέπουμε ότι δεν χρησιμοποιείται πλέον η μεταβλητή myArray, αλλά η μεταβλητή bArray η οποία έχει δημιουργηθεί με τη βοήθεια της συνάρτησης `sc.broadcast(myArray)`. Η χρήση της ειδικής μεταβλητής πραγματοποιείται με τη σύνταξη `bArray.value`.

```
val r = scala.util.Random

val myRDD = Array.fill(1000000){r.nextInt(99)}.parallelize

val myArray = Array.fill(10000){r.nextInt(99)}

val bArray = sc.broadcast(myArray) // δήλωση ειδικής μεταβλητής

val numElements = myRDD.filter(x => bArray.value.contains(x)).count
```

Απαιτείται προσοχή διότι, όταν τα δεδομένα που μεταδίδονται έχουν μεγάλο όγκο, ενδέχεται να δεσμεύσουν μεγάλα ποσά μνήμης και έτσι να προκληθούν προβλήματα. Επίσης, είναι σημαντικό να τονίσουμε ότι οι μεταβλητές μετάδοσης είναι μόνο για ανάγνωση και ότι αποθηκεύονται μόνο μία φορά στη μνήμη των κόμβων του κατανεμημένου συστήματος. Επομένως, εάν θέλουμε να αλλάξουμε το περιεχόμενο της μεταβλητής, θα πρέπει η διαδικασία να πραγματοποιηθεί από την αρχή, ώστε να πραγματοποιηθεί ενημέρωση των δεδομένων στους εκτελεστές.

Ας περιγράψουμε στη συνέχεια την ακολουθία των βημάτων κατά τη χρήση μεταβλητών μετάδοσης.

- Για να δημιουργήσουμε μια μεταβλητή μετάδοσης (broadcast variable), χρησιμοποιούμε τη μέθοδο `broadcast()`. Αυτή η μέθοδος παίρνει το σύνολο δεδομένων ως όρισμα και επιστρέφει μια μεταβλητή μετάδοσης.
- Η διαδικασία οδηγός σειριοποιεί (serialize) τη μεταβλητή μετάδοσης και τη στέλνει σε όλους τους κόμβους εργασίας της συστάδας.
- Με τη λήψη της μεταβλητής μετάδοσης, κάθε κόμβος εργασίας την αποσειριοποιεί (deserialize) και την αποθηκεύει στην τοπική του μνήμη.

- Όταν οι εργασίες (π.χ. `map`, `filter`) εκτελούνται σε κόμβους εργασίας, μπορούν να έχουν πρόσβαση στη μεταβλητή μετάδοσης τοπικά, χωρίς να απαιτείται μεταφορά δεδομένων μέσω δικτύου.
- Οι εργασίες που εκτελούνται σε κόμβους εργασίας μπορούν να έχουν αποτελεσματική πρόσβαση στη μεταβλητή μετάδοσης σαν να ήταν τοπική μεταβλητή. Αυτό ελαχιστοποιεί την ανάγκη για μεταφορά δεδομένων μέσω του δικτύου, μειώνοντας την καθυστέρηση και βελτιώνοντας την απόδοση.
- Η μεταβλητή μετάδοσης μπορεί να επαναχρησιμοποιηθεί σε πολλαπλές εργασίες που εκτελούνται στον ίδιο κόμβο εργασίας, βελτιώνοντας περαιτέρω την αποτελεσματικότητα.
- Το Spark διαχειρίζεται αυτόματα τον κύκλο ζωής των μεταβλητών μετάδοσης. Τα δεδομένα μετάδοσης διαγράφονται από τη μνήμη των κόμβων εργασίας όταν δεν χρησιμοποιούνται πλέον.

Συνοψίζοντας, η χρήση της μετάδοσης δίνει μεγάλη ευελιξία στην ανάπτυξη εφαρμογών, καθώς βοηθά στον προσδιορισμό των δεδομένων που είτε θα ακολουθήσουν τον παραδοσιακό τρόπο χειρισμού μέσω της κλειστότητας είτε θα μεταδοθούν με ειδικό τρόπο μέσω του μηχανισμού μετάδοσης.

6.2.3 Συσσώρευση δεδομένων

Οι *συσσωρευτές* (accumulators) αποτελούν ένα σημαντικό χαρακτηριστικό του Apache Spark, και χρησιμοποιούνται ως καταναμημένα αντικείμενα, ανεκτικά σε σφάλματα, για τη συγκέντρωση τιμών σε πολλαπλές εργασίες μέσα σε μια εφαρμογή Spark. Είναι ιδιαίτερα χρήσιμα για εργασίες όπως είναι η μέτρηση ή η παράλληλη άθροιση δεδομένων, όπως η καταμέτρηση γεγονότων ή η συγκέντρωση αποτελεσμάτων σε ένα καταναμημένο σύνολο δεδομένων. Οι συσσωρευτές μπορούν να ενημερώνονται μόνο εντός των εργασιών που εκτελούνται και προορίζονται μόνο για ανάγνωση από τη διεργασία οδήγησης, εξασφαλίζοντας ασφαλή και ταυτόχρονη συσσώρευση, χωρίς την ανάγκη ρητού συγχρονισμού. Μόλις ολοκληρωθούν όλες οι εργασίες, η διεργασία οδήγησης μπορεί να έχει πρόσβαση στην τελική συσσωρευμένη τιμή. Αυτή η δυνατότητα απλοποιεί πολύπλοκους καταναμημένους υπολογισμούς, ενισχύει την ανοχή σφαλμάτων και συμβάλλει στην ικανότητα του Spark να επεξεργάζεται με παράλληλο τρόπο δεδομένα μεγάλης κλίμακας.

Ας δούμε στη συνέχεια μερικές λεπτομέρειες για τη χρήση συσσωρευτών σε μια εφαρμογή Spark με τη γλώσσα Scala. Ένας συσσωρευτής, αρχικά, δηλώνεται και αρχικοποιείται στη διεργασία οδήγησης. Για παράδειγμα, εάν θέλουμε να δημιουργήσουμε έναν συσσωρευτή που μετρά κάποια στοιχεία, είναι λογικό η τιμή του να αρχικοποιηθεί στο 0. Εάν ο συσσωρευτής είναι λίστα, επίσης είναι λογικό να αρχικοποιηθεί στην κενή λίστα. Εάν και το Spark παρέχει ενσωματωμένους συσσωρευτές για βασικούς τύπους δεδομένων, όπως ακέραιους και πραγματικούς, μπορούμε επίσης να δημιουργήσουμε προσαρμοσμένους συσσωρευτές για πιο σύνθετους τύπους δεδομένων. Απαιτείται ιδιαίτερη προσοχή, διότι οι λειτουργίες που εφαρμόζονται στους συσσωρευτές θα πρέπει να ικανοποιούν την αντιμεταθετική και

την επιμεριστική ιδιότητα. Αυτό ισχύει διότι η σειρά με την οποία θα εκτελεστούν οι λειτουργίες στον συσσωρευτή μπορεί να διαφέρει από τη μία εκτέλεση στην άλλη, ωστόσο το τελικό αποτέλεσμα θα πρέπει να είναι το ίδιο. Επίσης τονίζουμε ότι δεν επιτρέπεται στις εργασίες που εκτελούνται στους εκτελεστές να διαβάσουν την τιμή του συσσωρευτή. Η ανάγνωση του συσσωρευτή επιτρέπεται μόνο να γίνεται από τη διεργασία οδήγησης.

Οι δύο βασικοί τύποι συσσωρευτών που υποστηρίζονται από το Spark στη γλώσσα Scala είναι οι: `longAccumulator` και `doubleAccumulator`. Ας δούμε τον τρόπο χρήσης ενός συσσωρευτή τον οποίο θέλουμε να χρησιμοποιήσουμε για να μετρήσουμε σε ένα αρχείο κειμένου πόσες φορές εμφανίζεται ο χαρακτήρας `*`. Είναι προφανές ότι θα μπορούσαμε να χρησιμοποιήσουμε μετασχηματισμούς όπως `filter` και να μετρήσουμε το πλήθος των χαρακτήρων χρησιμοποιώντας την ενέργεια `count`. Ωστόσο, η χρήση συσσωρευτών, ειδικά σε προβλήματα που έχουν να κάνουν με μέτρηση, δίνει πολύ πιο ευέλικτες λύσεις, ενώ ταυτόχρονα επιτρέπει τη μέτρηση διαφορετικών ποσοτήτων με τη χρήση διαφορετικών συσσωρευτών. Ο παρακάτω κώδικας Scala υλοποιεί το ζητούμενο:

```
val txtFileRDD = sc.textFile("leonardo.txt")

val acc = sc.longAccumulator("Star_Counter") // δήλωση συσσωρευτή

txtFileRDD.foreach { line =>
    if (line.contains("*"))
        acc.add(1) // ενημέρωση της τιμής του συσσωρευτή
}

println(acc.value) // χρήση της τιμής του συσσωρευτή
```

Εκτός από τους βασικούς τύπους συσσωρευτών που υποστηρίζονται από το Spark, δίνεται η δυνατότητα στον προγραμματιστή να αναπτύξει νέους τύπους συσσωρευτών, χρησιμοποιώντας την κλάση `AccumulatorV2`. Για παράδειγμα, εάν θέλουμε να υλοποιήσουμε έναν συσσωρευτή που να αποθηκεύει στοιχεία σε μία λίστα, θα πρέπει να δημιουργήσουμε μία κλάση αντικειμένων που κληρονομεί από την κλάση `AccumulatorV2` και υλοποιεί τις μεθόδους που απαιτούνται ώστε να υποστηριχθεί η λειτουργικότητα της λίστας. Για παράδειγμα, για την υλοποίηση ενός συσσωρευτή τύπου λίστας που αποθηκεύει συμβολοσειρές, μπορούμε να βασιστούμε στον ακόλουθο κώδικα:

```
class ListAccumulator[T] extends AccumulatorV2[T, List[T]] {
    // Υλοποίηση μεθόδων συσσωρευτή λίστας
    // ...
}

val listAccumulator = new ListAccumulator[String]
```

Στην περίπτωση υλοποίησης νέου τύπου συσσωρευτή, θα πρέπει να υλοποιηθούν οπωσδήποτε οι μέθοδοι `reset`, `isZero`, `copy`, `merge`, `add` οι οποίες κληρονομούνται από την αφαιρετική κλάση `AccumulatorV2`.

6.3 Επεξεργασία δομημένων δεδομένων

Μέχρι τώρα, χρησιμοποιήθηκε το RDD API, το οποίο θεωρείται χαμηλού επιπέδου. Πράγματι, εάν δουλεύουμε με RDDs έχουμε πολύ μεγάλη ευελιξία ως προς τις λειτουργίες που μπορούν να εφαρμοστούν στα δεδομένα. Μελετήσαμε, επίσης, κάποιες δυνατότητες βελτιστοποίησης που έχουμε κατά την επεξεργασία μεγάλων δεδομένων χρησιμοποιώντας το RDD API. Ωστόσο, με τη χρήση του RDD API δεν έχουμε τη δυνατότητα να χρησιμοποιήσουμε τους αυτοματοποιημένους μηχανισμούς του Spark που στοχεύουν στη βελτιστοποίηση. Στην ενότητα αυτή θα εστιάσουμε στο DataFrame API, το οποίο χρησιμοποιείται για την επεξεργασία και ανάλυση *δομημένων δεδομένων*, τα οποία χαρακτηρίζονται από γραμμές και στήλες, όπως συμβαίνει και σε έναν τυπικό πίνακα ενός συστήματος βάσεων δεδομένων.

Ένα DataFrame ορίζεται ως μια κατανεμημένη συλλογή δεδομένων η οποία διαθέτει γραμμές και στήλες. Για παράδειγμα, έστω ότι έχουμε ένα αρχείο τύπου csv το οποίο περιέχει κάποια στοιχεία για τους εργαζόμενους μιας εταιρείας. Υποθέτουμε ότι έχουμε τις στήλες `id`, `name`, `city` και `age`. Τα περιεχόμενα του αρχείου csv έχουν ως εξής:

```
id,name,city,age
1,Emily,Los Angeles,29
2,John,Los Angeles,28
3,Alice,New York,24
4,Bob,New York,32
5,Michael,Chicago,35
```

Αρχικά, θα δημιουργήσουμε ένα Spark DataFrame για τα δεδομένα αυτά και, στη συνέχεια, θα προχωρήσουμε με την επεξεργασία των στοιχείων εκτελώντας κάποια βασικά ερωτήματα. Υποθέτουμε ότι τα δεδομένα μας είναι αποθηκευμένα στον τρέχοντα κατάλογο σε ένα αρχείο csv με όνομα `people.csv`. Για την ανάγνωση του αρχείου csv χρησιμοποιούμε τη συνάρτηση `read` με τις επιλογές να χρησιμοποιήσουμε επικεφαλίδα (`option("header", "true")`) και οι τύποι δεδομένων των στηλών να προσδιοριστούν αυτόματα κατά τη διαδικασία της ανάγνωσης (`option("inferSchema", "true")`).

```
import org.apache.spark.sql.{SparkSession, DataFrame}

val spark = SparkSession.builder()
  .appName("Simple DataFrame Example")
  .config("spark.master", "local")
  .getOrCreate()

val csvFilePath = "people.csv"

val df: DataFrame = spark.read
  .option("header", "true") // Use the first row as column names
  .option("inferSchema", "true") // Infer the schema automatically
  .csv(csvFilePath)
```

Μετά την εκτέλεση της συνάρτησης `read`, η μεταβλητή `df` αναφέρεται σε ένα DataFrame. Δύο πολύ χρήσιμες συναρτήσεις που μπορούμε να χρησιμοποιήσουμε σε ένα DataFrame είναι

οι `printSchema` και `show`. Με την `printSchema` μπορούμε να δούμε το πλήθος και τον τύπο δεδομένων των στηλών του `DataFrame`, ενώ με την `show` μπορούμε να τυπώσουμε τις πρώτες γραμμές του `DataFrame`.

```
df.printSchema() // εκτύπωση του σχήματος του DataFrame

df.show(3) // εκτύπωση των τριών πρώτων γραμμών του DataFrame
```

Η εκτέλεση της `printSchema` δίνει το ακόλουθο αποτέλεσμα, στο οποίο περιέχονται τα ονόματα των στηλών και οι αντίστοιχοι τύποι δεδομένων. Για παράδειγμα, οι στήλες `id` και `age` είναι τύπου `Integer`, ενώ οι υπόλοιπες στήλες είναι τύπου `String`. Τονίζουμε ότι έχουμε τη δυνατότητα να καθορίσουμε το σχήμα του `DataFrame` χωρίς να χρησιμοποιηθεί ο αυτόματος μηχανισμός.

```
root
 |-- id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- age: integer (nullable = true)
```

Μια άλλη πολύ χρήσιμη συνάρτηση (που αποτελεί ενέργεια με βάση την ορολογία του Spark) είναι η συνάρτηση `show`, η οποία δέχεται ως παράμετρο έναν ακέραιο αριθμό που προσδιορίζει το πλήθος των γραμμών του `DataFrame` που θέλουμε στην έξοδο. Παρακάτω δίνουμε το αποτέλεσμα της εκτέλεσης της συνάρτησης `show(3)`:

```
+---+-----+-----+---+
| id| name|      city|age|
+---+-----+-----+---+
|  1|Emily|Los Angeles| 29|
|  2| John|Los Angeles| 28|
|  3|Alice|  New York| 24|
+---+-----+-----+---+
```

Το βασικό πλεονέκτημα των δομημένων δεδομένων είναι το ότι μπορούμε να εκτελούμε ερωτήματα σε SQL, όπως συμβαίνει στα συστήματα βάσεων δεδομένων. Αυτό το χαρακτηριστικό των `DataFrames` αποτελεί σημαντική διαφορά σε σχέση με τα `RDDs`, τα οποία υποστηρίζουν λειτουργίες χαμηλού επιπέδου. Επομένως, σε περίπτωση που έχουμε δομημένα δεδομένα, η καλύτερη πρακτική είναι η χρήση `DataFrames` για την αναπαράσταση και επεξεργασία τους. Ωστόσο, εάν θέλουμε να εκτελέσουμε κάποια εξειδικευμένη λειτουργία που δεν υποστηρίζεται άμεσα από το `DataFrame API`, μπορούμε να χρησιμοποιήσουμε το `RDD API` χωρίς πρόβλημα.

Ας ξεκινήσουμε με κάποια απλά ερωτήματα τα οποία θέλουμε να εκτελεστούν στο `DataFrame` που έχουμε δημιουργήσει. Το Spark διαθέτει μια ξεχωριστή βιβλιοθήκη για την επεξεργασία δομημένων δεδομένων μέσω της SQL. Η πρόσβαση στις λειτουργίες της βιβλιοθήκης αυτής πραγματοποιείται με την εισαγωγή της εντολής `import org.apache.spark.sql._`.

Ωστόσο, τις περισσότερες φορές, χρειαζόμαστε ένα υποσύνολο των λειτουργιών της βιβλιοθήκης, οπότε καλό είναι να κάνουμε `import` μόνο τη λειτουργικότητα που χρειαζόμαστε στην εφαρμογή.

Η επεξεργασία των δεδομένων σε ένα `DataFrame` μπορεί να πραγματοποιηθεί με δύο βασικές τεχνικές. Η πρώτη τεχνική χρησιμοποιεί μετασχηματισμούς και ενέργειες που υποστηρίζονται από το `DataFrame` API, και η δεύτερη χρησιμοποιεί απευθείας ερωτήματα SQL τα οποία εισάγονται ως συμβολοσειρές και, ακολούθως, εκτελούνται από το Spark. Θα εξετάσουμε και τις δύο τεχνικές στη συνέχεια.

Από τη στιγμή που έχουμε στη διάθεσή μας ένα `DataFrame`, ας εξετάσουμε κάποιους από τους μετασχηματισμούς που μπορούμε να χρησιμοποιήσουμε. Έστω ότι θέλουμε να δώσουμε στην έξοδο όλες τις πληροφορίες για τα άτομα με ηλικία μεγαλύτερη των 28 ετών. Για τον σκοπό αυτόν, θα χρησιμοποιήσουμε τον μετασχηματισμό `filter` ώστε να κρατήσουμε μόνο τις επιθυμητές γραμμές από το `DataFrame`. Εναλλακτικά, μπορούμε να χρησιμοποιήσουμε και τον μετασχηματισμό `where`. Ο κώδικας και το αντίστοιχο αποτέλεσμα της εκτέλεσης της `filter` παρουσιάζονται παρακάτω:

```
val result = df.filter(df("age") > 28)
result.show()
```

```
+---+-----+-----+---+
| id|   name|   city|age|
+---+-----+-----+---+
|  1|  Emily|Los Angeles| 29|
|  4|   Bob|   New York| 32|
|  5|Michael|   Chicago| 35|
+---+-----+-----+---+
```

Εάν θέλουμε να κρατήσουμε συγκεκριμένες στήλες, μπορούμε να χρησιμοποιήσουμε τον μετασχηματισμό `select`. Για παράδειγμα, έστω ότι θέλουμε να κρατήσουμε μόνο τις στήλες `name` και `age`. Ο αντίστοιχος κώδικας έχει ως εξής:

```
val result = df.filter(df("age") > 28).select(df("name"), df("age"))
result.show()
```

ενώ το αποτέλεσμα θα είναι:

```
+-----+---+
|   name|age|
+-----+---+
|  Emily| 29|
|   Bob| 32|
|Michael| 35|
+-----+---+
```

Μία πολύ σημαντική λειτουργία στην επεξεργασία δομημένων δεδομένων είναι η *συνάθροιση* (aggregation). Έστω ότι ενδιαφερόμαστε για τη μέση ηλικία των ατόμων ανά πόλη. Για την

επεξεργασία αυτού του ερωτήματος, θα πρέπει να συγκεντρώσουμε όλες τις εγγραφές που αναφέρονται στην ίδια πόλη, και στη συνέχεια, να υπολογίσουμε τη μέση τιμή της ηλικίας.

```
val result = df.groupBy(df("city")).agg(avg(df("age")))
result.printSchema()
result.show()
```

Το αποτέλεσμα του μετασχηματισμού `groupBy` είναι ένα άλλο `DataFrame`, το οποίο αποθηκεύεται στη μεταβλητή `result`. Η εντολή `groupBy` ακολουθείται από τη συνάρτηση `agg`, η οποία χρησιμοποιείται για τη δήλωση της συνάρτησης που θα χρησιμοποιηθεί. Παρακάτω δίνουμε το αποτέλεσμα της εκτέλεσης της συνάρτησης `printSchema()` και της συνάρτησης `show()`:

```
root
|-- city: string (nullable = true)
|-- avg(age): double (nullable = true)
```

```
+-----+-----+
|      city|avg(age)|
+-----+-----+
|Los Angeles|    28.5|
|   Chicago|    35.0|
|   New York|    28.0|
+-----+-----+
```

Σε περίπτωση που απαιτείται ο προσδιορισμός των *διακριτών στοιχείων*, μπορεί να χρησιμοποιηθεί η συνάρτηση `distinct`. Για παράδειγμα, εάν θέλουμε στην έξοδο τις πόλεις έτσι ώστε η κάθε πόλη να εμφανίζεται μία φορά το πολύ, μπορούμε να χρησιμοποιήσουμε τον ακόλουθο κώδικα:

```
df.select(df("city")).distinct().show()
```

ο οποίος δίνει το ακόλουθο αποτέλεσμα:

```
+-----+
|      city|
+-----+
|Los Angeles|
|   Chicago|
|   New York|
+-----+
```

Στη συνέχεια, ας δούμε πώς μπορούμε να χρησιμοποιήσουμε τους μετασχηματισμούς του Spark για να υλοποιήσουμε μία σύνδεση (`join`) ανάμεσα σε ένα `DataFrame` και τον εαυτό του. Συγκεκριμένα, εδώ θέλουμε να δώσουμε στην έξοδο όλα τα ζεύγη ατόμων που διαμένουν

στην ίδια πόλη. Στην περίπτωση αυτή, μπορούμε να χρησιμοποιήσουμε τον μετασχηματισμό `join` ως εξής:

```
df.as("t1").join(df.as("t2"), $"t1.city" === $"t2.city")
    .select("t1.name", "t2.name", "t1.city")
    .orderBy("t1.city")
    .show()
```

Παρατηρούμε ότι έχουν χρησιμοποιηθεί δύο ψευδώνυμα για το ίδιο `DataFrame`, έτσι ώστε να μπορέσει να εκτελεστεί η πράξη `self-join`. Στη συνέχεια, επιλέγουμε με τον μετασχηματισμό `select` τις στήλες που χρειαζόμαστε στην έξοδο και, τέλος, με την ενέργεια `show` τυπώνουμε ένα τμήμα του `DataFrame`. Παρατηρήστε ότι χρησιμοποιήσαμε και τον μετασχηματισμό `orderBy`, ώστε τα αποτελέσματα να τυπωθούν με αύξουσα λεξικογραφική σειρά ως προς το πεδίο `city`.

Τα παραδείγματα που μελετήσαμε έως τώρα χρησιμοποιούν το `DataFrame API` και συγκεκριμένους μετασχηματισμούς (π.χ. `select`, `filter`), ώστε να επεξεργαστούμε δομημένα δεδομένα. Ωστόσο, πολλές φορές είναι πιο βολικό να μπορούμε να εκτελούμε απευθείας ερωτήματα SQL, όπως ακριβώς θα γινόταν σε ένα σύστημα βάσεων δεδομένων. Αυτό μας αποδεσμεύει από τη χρήση συγκεκριμένων μετασχηματισμών του `DataFrame API` και επιτρέπει την πιο εύκολη χρήση του `Spark` από χρήστες που προέρχονται από την περιοχή των βάσεων δεδομένων και γνωρίζουν πολύ καλά τη γλώσσα ερωτημάτων SQL.

Ωστόσο, πριν από την εκτέλεση ερωτημάτων SQL σε ένα `DataFrame`, θα πρέπει να δημιουργήσουμε ένα ειδικό αντικείμενο, έτσι ώστε να μπορεί να χρησιμοποιηθεί ως *πίνακας*. Αυτό ισχύει για όλα τα `DataFrames` που θέλουμε να χρησιμοποιήσουμε ως πίνακες στα ερωτήματα SQL. Στο παράδειγμά μας, θα δημιουργήσουμε αυτό το ειδικό αντικείμενο και θα το ονομάσουμε `people`. Ας εξετάσουμε τον ακόλουθο κώδικα:

```
// Δημιουργία προσωρινής όψης
df.createOrReplaceTempView("people")

// Χρήση της όψης σε ερώτημα SQL
val result: DataFrame = spark.sql("SELECT * FROM people WHERE Age > 28")

// Εμφάνιση αποτελέσματος
result.show()
```

Αρχικά, δημιουργείται η *προσωρινή όψη* (temporary view) με όνομα `people` χρησιμοποιώντας τη συνάρτηση `createOrReplaceTempView()`. Στη συνέχεια, χρησιμοποιούμε μια συμβολοσειρά για να ορίσουμε το ερώτημα SQL που θέλουμε να εκτελέσουμε. Στην περίπτωση μας, αυτό το ερώτημα προσδιορίζει τα στοιχεία των ατόμων με ηλικία μεγαλύτερη από 28 έτη. Το ερώτημα έχει ως εξής: `"SELECT * FROM people WHERE Age > 28"`.

Στα συστήματα βάσεων δεδομένων μία από τις βασικές λειτουργίες που υποστηρίζονται είναι η βελτιστοποίηση ερωτημάτων SQL. Το σύστημα προσπαθεί να προσδιορίσει τόσο τη σειρά εκτέλεσης των λειτουργιών όσο και τον αλγόριθμο που θα χρησιμοποιηθεί, ώστε το τελικό κόστος εκτέλεσης να είναι το ελάχιστο δυνατό. Η διαδικασία αυτή παραδοσιακά καλείται *βελτιστοποίηση ερωτήματος* (query optimization). Το `Spark` διαθέτει έναν πολύ εξελιγ-

μένο βελτιστοποιητή ερωτημάτων που καλείται Catalyst και αναλαμβάνει τη βελτιστοποίηση τόσο των ερωτημάτων που εκφράζονται μέσω μετασχηματισμών στο DataFrame API όσο και των ερωτημάτων που ορίζονται μέσω Spark SQL χρησιμοποιώντας τη συνάρτηση `sql()`. Στη συνέχεια, παρουσιάζονται συνοπτικά τα βασικότερα χαρακτηριστικά του βελτιστοποιητή Catalyst.

- Ο βελτιστοποιητής Catalyst λειτουργεί σε δύο φάσεις: λογική βελτιστοποίηση και φυσική βελτιστοποίηση. Στη φάση της λογικής βελτιστοποίησης, εφαρμόζονται λογικοί μετασχηματισμοί στο πλάνο του ερωτήματος, όπως η εκτέλεση των κατηγορημάτων όσο γίνεται χαμηλότερα, για την απλοποίηση και τη βελτίωση της δομής του πλάνου ερωτήματος χωρίς να ληφθούν υπόψη οι φυσικές λεπτομέρειες εκτέλεσης. Στη φάση της φυσικής βελτιστοποίησης, δημιουργείται ένα αποτελεσματικό πλάνο φυσικής εκτέλεσης που βασίζεται στο λογικό πλάνο, λαμβάνοντας υπόψη παράγοντες όπως την κατανομή δεδομένων και τις μεθόδους προσπέλασης.
- Ο Catalyst χρησιμοποιεί ένα πλαίσιο βελτιστοποίησης που βασίζεται σε κανόνες. Αποτελείται από ένα σύνολο κανόνων βελτιστοποίησης που εφαρμόζονται στο πλάνο του ερωτήματος για να το ξαναγράψουν ή να το μετατρέψουν σε πιο αποτελεσματική μορφή. Αυτοί οι κανόνες εφαρμόζονται επαναληπτικά έως ότου δεν μπορεί να επιτευχθεί περαιτέρω βελτιστοποίηση.
- Ο Catalyst ωθεί τις συνθήκες φίλτρου όσο το δυνατόν πιο κοντά στην πηγή δεδομένων. Αυτό μειώνει τον όγκο των δεδομένων που πρέπει να υποβληθούν σε επεξεργασία κατά την εκτέλεση του ερωτήματος.
- Απλοποιεί τις εκφράσεις που περιέχουν σταθερές, αξιολογώντας τις κατά τον χρόνο μεταγλώττισης. Αυτό μειώνει την υπολογιστική επιβάρυνση χρόνου εκτέλεσης.
- Ο Catalyst κλαδεύει περιττές στήλες από το πλάνο ερωτήματος, διασφαλίζοντας ότι υποβάλλονται σε επεξεργασία μόνο οι απαιτούμενες στήλες. Αυτή η βελτιστοποίηση μειώνει τον όγκο των δεδομένων και βελτιώνει την απόδοση των ερωτημάτων.
- Ξεκινώντας από την έκδοση Spark 2.0, ο Catalyst εισήγαγε τη βελτιστοποίηση βάσει κόστους. Εκτιμά το κόστος διαφορετικών πλάνων εκτέλεσης και επιλέγει το σχέδιο με το χαμηλότερο εκτιμώμενο κόστος. Αυτό βοηθά στη λήψη πιο ενημερωμένων αποφάσεων σχετικά με τις στρατηγικές εκτέλεσης ερωτημάτων.
- Ο Catalyst είναι επεκτάσιμος, επιτρέποντας στους προγραμματιστές να ορίζουν προσαρμοσμένους κανόνες βελτιστοποίησης και στρατηγικές προσαρμοσμένες σε συγκεκριμένες περιπτώσεις χρήσης ή πηγές δεδομένων.
- Είναι σχεδιασμένος να είναι ανεξάρτητος από την πλατφόρμα, πράγμα που σημαίνει ότι μπορεί να χρησιμοποιηθεί με διαφορετικές μηχανές εκτέλεσης. Αν και χρησιμοποιείται συνήθως με τη μηχανή εκτέλεσης Tungsten του Spark, μπορεί να ενσωματωθεί και σε άλλες μηχανές.
- Ο Catalyst αποθηκεύει προσωρινά τα πλάνα εκτέλεσης ερωτημάτων, μειώνοντας την ανάγκη επαναπροσδιορισμού πλάνων εκτέλεσης για παρόμοια ερωτήματα, γεγονός

που μπορεί να οδηγήσει σε σημαντικές βελτιώσεις απόδοσης, ειδικά όταν ο φόρτος εργασίας είναι παρόμοιος.

6.4 Εξόρυξη δεδομένων και ανακάλυψη γνώσης

Ένα από τα δυνατά σημεία του Spark, εκτός από την αποδοτική επεξεργασία μεγάλων δεδομένων, είναι και η δυνατότητα εξόρυξης δεδομένων με αποδοτικούς κατανεμημένους αλγορίθμους. Το Spark (ανάλογα με την έκδοση) διαθέτει δύο βιβλιοθήκες που υλοποιούν αλγορίθμους μηχανικής μάθησης: ML και MLlib. Η ύπαρξη δύο βιβλιοθηκών οφείλεται στο γεγονός ότι η ML χρησιμοποιείται με το DataFrame API ενώ η MLlib χρησιμοποιείται με το RDD API. Ωστόσο, τονίζουμε ότι η βιβλιοθήκη για το RDD API έχει ήδη μπει σε κατάσταση συντήρησης, που σημαίνει ότι δεν θα προστίθενται νέες δυνατότητες. Μπορεί κάποιος, εάν θέλει, να χρησιμοποιήσει το RDD API, αλλά οι οδηγίες αναφέρουν ότι θα πρέπει να χρησιμοποιείται το DataFrame API.

Στην ενότητα αυτή θα περιγράψουμε τον τρόπο υλοποίησης μιας εφαρμογής Spark που βασίζεται στο DataFrame API και στοχεύει στην πρόβλεψη για το εάν κάποια ανασκόπηση (review) μιας ταινίας είναι θετική ή αρνητική. Για τη λύση του προβλήματος θα χρησιμοποιήσουμε μια τεχνική μηχανικής μάθησης με επίβλεψη (supervised) και, συγκεκριμένα, θα λύσουμε το πρόβλημα χρησιμοποιώντας τεχνικές ταξινόμησης ή κατάταξης (classification) όπου έχουμε δύο κλάσεις: η πρώτη κλάση αφορά ανασκοπήσεις με αρνητικό συναίσθημα, ενώ η δεύτερη κλάση αφορά ανασκοπήσεις με θετικό συναίσθημα. Το πρόβλημα αυτό εμπίπτει στο γενικότερο πρόβλημα της *ανάλυσης συναισθήματος* (sentiment analysis) που έχει πολλές εφαρμογές στην ανάλυση δεδομένων κοινωνικών δικτύων και όχι μόνο. Άρα, πρέπει με βάση το περιεχόμενο της ανασκόπησης, που είναι ένα μικρό αρχείο κειμένου, να προβλέψουμε εάν το συναίσθημα είναι θετικό ή αρνητικό.

Αρχικά, διαβάζουμε το αρχείο `review.csv` και δημιουργούμε ένα `DataFrame` με όνομα `basicDF` και τυπώνουμε το σχήμα του (με τη συνάρτηση `printSchema()`):

```
import org.apache.spark.ml.evaluation.MulticlassClassificationEvaluator
import org.apache.spark.sql.{SparkSession}
import org.apache.spark.ml.classification.{LogisticRegression, NaiveBayes}
}
import org.apache.spark.ml.feature.{Tokenizer, HashingTF, IDF}
import org.apache.spark.sql.functions._

// Create the spark session first
val spark = SparkSession.builder().master("local").appName("tfidfApp").
getOrCreate()
import spark.implicits._
val inputFile = "./reviews.csv"

println("reading from input file: " + inputFile)

// Read the contents of the csv file in a dataframe
val basicDF = spark.read.option("header", "false").csv(inputFile)
basicDF.printSchema()
```

```
root
|-- _c0: string (nullable = true)
|-- _c1: string (nullable = true)
|-- _c2: string (nullable = true)
```

Θα εφαρμόσουμε μετονομασία στις στήλες του `basicDF`, ώστε να δώσουμε αντιπροσωπευτικά ονόματα:

```
val newColumnNames = Seq("rId", "rText", "rLabel")
val reviewsDF = basicDF.toDF(newColumnNames: _*)
reviewsDF.printSchema()
reviewsDF.show()
```

```
root
|-- rId: string (nullable = true)
|-- rText: string (nullable = true)
|-- rLabel: string (nullable = true)
```

```
+---+-----+-----+
|rId|          rText|  rLabel|
+---+-----+-----+
|  1|This movie is rea... |positive|
|  2|Vanilla Sky was a... |positive|
|  3|Joseph Brady and ... |positive|
|  4|10 ITEMS OR LESS ... |positive|
|  5|I saw this movie ... |positive|
|  6|Elvira, Mistress ... |positive|
|  7|I'm a pretty old ... |positive|
|  8|This movie brings... |positive|
|  9|In Micro Phonies ... |positive|
| 10|Terminator is the ...|positive|
+---+-----+-----+
```

Στη συνέχεια, πρέπει να δημιουργήσουμε ένα χαρακτηριστικό διάνυσμα για κάθε ανασκόπηση. Εδώ θα χρησιμοποιήσουμε τη μεθοδολογία `tf-idf`, η οποία χρησιμοποιεί τη συχνότητα εμφάνισης των λέξεων σε κάθε έγγραφο, καθώς και το πλήθος των εγγράφων όπου εμφανίζεται η λέξη. Σε πρώτη φάση, από το κείμενο του κάθε εγγράφου λαμβάνουμε τις λέξεις από τις οποίες αποτελείται και δημιουργούμε μία νέα στήλη στο `DataFrame`. Το νέο `DataFrame` καλείται `wordsDF`.

```
val tokenizer = new Tokenizer().setInputCol("rText").setOutputCol("rWords")
val wordsDF = tokenizer.transform(reviewsDF)
wordsDF.printSchema()
```

```

root
|-- rId: string (nullable = true)
|-- rText: string (nullable = true)
|-- rLabel: string (nullable = true)
|-- rWords: array (nullable = true)
|   |-- element: string (containsNull = true)

```

Χρησιμοποιώντας τις τεχνικές HashingTF και IDF, για κάθε έγγραφο δημιουργείται ένα διάνυσμα το οποίο χαρακτηρίζει το έγγραφο. Τέλος, μετατρέπουμε τον χαρακτηρισμό positive/negative σε 1.0/0.0, ώστε να μπορεί να χρησιμοποιηθεί από τους αλγορίθμους κατάταξης. Το νέο DataFrame που παράγεται από την προηγούμενη διαδικασία καλείται newDF.

```

val hashingTF = new HashingTF().setInputCol("rWords").setOutputCol("rRawFeatures")
val featurizedDF = hashingTF.transform(wordsDF)
featurizedDF.printSchema()

val idf = new IDF().setInputCol("rRawFeatures").setOutputCol("rFeatures")
val idfM = idf.fit(featurizedDF)
val completeDF = idfM.transform(featurizedDF)
completeDF.printSchema()

val udf_toDouble = udf( (s: String) => if(s=="positive") 1.0 else 0.0 )

val newDF = completeDF.select($"rId", $"rText", $"rWords", $"rRawFeatures", $"rFeatures", $"rLabel", udf_toDouble($"rLabel").as("rrLabel"))
newDF.printSchema()

```

```

root
|-- rId: string (nullable = true)
|-- rText: string (nullable = true)
|-- rWords: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- rRawFeatures: vector (nullable = true)
|-- rFeatures: vector (nullable = true)
|-- rLabel: string (nullable = true)
|-- rrLabel: double (nullable = false)

```

Τα δεδομένα τώρα είναι σε μορφή ώστε να μπορούν να χρησιμοποιηθούν από έναν αλγόριθμο κατάταξης, ώστε να προσδιορίσουμε κατά πόσο το χαρακτηριστικό διάνυσμα tf-idf που έχουμε χρησιμοποιήσει για την αναπαράσταση των εγγράφων δίνει καλά αποτελέσματα. Η βιβλιοθήκη ML του Spark υποστηρίζει πολλούς αλγορίθμους κατάταξης. Θα χρησιμοποιήσουμε δύο από αυτούς τους αλγορίθμους, τους Naive Bayes και Logistic Regression. Αρχικά, μετονομάζουμε τη στήλη rFeatures σε features και τη στήλη rrLabel σε label. Αυτό δεν είναι απαραίτητο, καθώς στους αλγορίθμους μπορούμε να καθορίσουμε ποια στήλη περιέχει τα χαρακτηριστικά διανύσματα και ποια περιέχει την ετικέτα. Ωστόσο, εάν αυτό δεν το καθορίσουμε, χρησιμοποιούνται αυτομάτως τα ονόματα features και label, οπότε θα πρέπει να προϋπάρχουν στο DataFrame.

```
val df = newDF.select("rId", "rFeatures", "rrLabel")
                .withColumnRenamed("rFeatures", "features")
                .withColumnRenamed("rrLabel", "label")
```

Το επόμενο βήμα είναι να προχωρήσουμε στην αξιολόγηση χρησιμοποιώντας την απλή τεχνική της τυχαίας διάσπασης του DataFrame σε σύνολο εκπαίδευσης (training set) και σύνολο ελέγχου (test set). Η τεχνική που χρησιμοποιούμε βασίζεται στη συνάρτηση `randomSplit()`. Στο παράδειγμα θα χρησιμοποιήσουμε το 70% των δεδομένων για εκπαίδευση και το 30% για έλεγχο.

```
val Array(trainingData, testData) = df.randomSplit(Array(0.7, 0.3), seed
= 1234L)
```

Ορίζουμε το μοντέλο που θα χρησιμοποιήσουμε, εφαρμόζουμε τη συνάρτηση `fit()` στο σύνολο εκπαίδευσης και, στη συνέχεια, τη συνάρτηση `transform()` στο σύνολο ελέγχου. Το αποτέλεσμα που παράγεται αποθηκεύεται στο DataFrame `predictionsNB`.

```
val NBmodel = new NaiveBayes().fit(trainingData)
val predictionsNB = NBmodel.transform(testData)
predictionsNB.printSchema()
predictionsNB.select("rId", "features", "label", "prediction").show(10)
```

Παρακάτω φαίνεται το σχήμα του `predictionsNB`, καθώς και οι δέκα πρώτες γραμμές του:

```
root
 |-- rId: string (nullable = true)
 |-- features: vector (nullable = true)
 |-- label: double (nullable = false)
 |-- rawPrediction: vector (nullable = true)
 |-- probability: vector (nullable = true)
 |-- prediction: double (nullable = false)
```

Παρατηρούμε ότι στο DataFrame έχει δημιουργηθεί μία νέα στήλη με όνομα `prediction`, η οποία περιέχει τις προβλέψεις για το σύνολο ελέγχου. Επίσης, έχουν δημιουργηθεί και άλλες στήλες, όπως `probability` και `rawPrediction`, οι οποίες μπορούν να χρησιμοποιηθούν για την αξιολόγηση του μοντέλου.

```
+----+-----+-----+-----+
| rId|          features|label|prediction|
+----+-----+-----+-----+
|   1|(262144,[3376,142...|  1.0|         0.0|
|  10|(262144,[67740,95...|  1.0|         1.0|
|1001|(262144,[8538,880...|  0.0|         0.0|
|1017|(262144,[991,1097...|  0.0|         0.0|
| 102|(262144,[4629,646...|  1.0|         1.0|
|1020|(262144,[2276,392...|  0.0|         0.0|
```

```
|1021| (262144, [2437, 293...| 0.0| 0.0|
|1022| (262144, [3928, 844...| 0.0| 0.0|
|1024| (262144, [476, 3421...| 0.0| 0.0|
|1026| (262144, [1097, 146...| 0.0| 0.0|
+----+-----+-----+-----+
```

Προχωράμε τώρα στην αξιολόγηση του αποτελέσματος του αλγορίθμου Naive Bayes. Εδώ θα χρησιμοποιήσουμε τους έτοιμους αξιολογητές που υποστηρίζει το Spark και, έτσι, δεν θα χρειαστεί να υλοποιήσουμε τη διαδικασία αξιολόγησης. Θα χρησιμοποιήσουμε τον αξιολογητή `MulticlassClassificationEvaluator` και θα εστιάσουμε στο μέτρο της ακρίβειας (accuracy). Το μέτρο αυτό χρειάζεται προσοχή, επειδή σε περίπτωση που δεν έχουμε ισορροπημένες κλάσεις δεν μπορούμε να βασιστούμε στα αποτελέσματα. Για το συγκεκριμένο παράδειγμα, ο αλγόριθμος Naive Bayes δίνει ένα αποτέλεσμα ακρίβειας 0,78.

```
val evaluatorNB = new MulticlassClassificationEvaluator()
    .setLabelCol("label")
    .setPredictionCol("prediction")
    .setMetricName("accuracy")

val accuracyNB = evaluatorNB.evaluate(predictionsNB)
println("Accuracy of Naive Bayes: " + accuracyNB)
```

Με παρόμοιο τρόπο μπορούμε να δουλέψουμε και για την εφαρμογή του αλγορίθμου Logistic Regression, όπως παρουσιάζεται στον ακόλουθο κώδικα:

```
val LRmodel = new LogisticRegression()
    .setMaxIter(100)
    .setRegParam(0.1)
    .setElasticNetParam(0.0)
    .fit(trainingData)

val predictionsLR = LRmodel.transform(testData)
predictionsLR.printSchema()
predictionsLR.show(10)

val accuracyLR = evaluatorNB.evaluate(predictionsLR)
println("Accuracy of Logistic Regression: " + accuracyLR)
```

Παρατηρούμε ότι χρησιμοποιείται ο ίδιος αξιολογητής που κατασκευάσαμε πριν για τον αλγόριθμο Naive Bayes, ενώ πάλι το μέτρο που χρησιμοποιούμε είναι η ακρίβεια. Ο ενδιαφερόμενος αναγνώστης καλείται να ανατρέξει στη σχετική τεκμηρίωση του Apache Spark και συγκεκριμένα στους αλγορίθμους κατάταξης που υποστηρίζονται (<https://spark.apache.org/docs/latest/ml-classification-regression.html>).

6.5 Περαιτέρω μελέτη

Στο κεφάλαιο αυτό, μελετήσαμε περισσότερο το σύστημα Spark και παρουσιάσαμε μερικά σημαντικά χαρακτηριστικά του συστήματος που σχετίζονται με τη βελτίωση των επιδόσεων, την επεξεργασία δεδομένων μεγάλης κλίμακας χρησιμοποιώντας DataFrames και την εξό-

ρυξη γνώσης από μεγάλα δεδομένα. Περισσότερες πληροφορίες για τις αλγοριθμικές τεχνικές που υποστηρίζονται από το Spark υπάρχουν στις βιβλιογραφικές αναφορές [ZXW⁺16], [KW17], [Sal17], [RLOW17], [CZ18].

BIBΛΙΟΓΡΑΦΙΑ

- [CZ18] Bill Chambers and Matei Zaharia. *Spark: The definitive guide: Big data processing made simple*. O'Reilly Media, Inc., 2018.
- [KW17] Holden Karau and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. O'Reilly Media, Inc., 1st edition, 2017.
- [RLOW17] Sandy Ryza, Uri Laserson, Sean Owen, and Josh Wills. *Advanced Analytics with Spark: Patterns for Learning from Data at Scale*. O'Reilly, 2 edition, 2017.
- [Sal17] Neelesh Srinivas Salian. *Spark Operations Cookbook: Solving the Practical Challenges of Spark Implementation*. O'Reilly Media, Inc., 1st edition, 2017.
- [ZXW⁺16] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, and et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

ΚΕΦΑΛΑΙΟ 7

ΕΠΕΞΕΡΓΑΣΙΑ ΡΟΩΝ ΔΕΔΟΜΕΝΩΝ

Περιεχόμενα

7.1	Εισαγωγή	172
7.2	Συστήματα επεξεργασίας ροών δεδομένων	174
7.2.1	Apache Kafka	174
7.2.2	Apache Spark	178
7.2.3	Apache Flink	179
7.2.4	Apache Storm	181
7.2.5	Amazon Kinesis	185
7.3	Περαιτέρω μελέτη	188
	ΒΙΒΛΙΟΓΡΑΦΙΑ	188



Σύνοψη

Οι τεχνικές διαχείρισης και ανάλυσης ροών δεδομένων έχουν εφαρμογή σε περιπτώσεις όπου τα δεδομένα ανανεώνονται σχετικά γρήγορα, με αποτέλεσμα να δημιουργούνται ζητήματα απόδοσης, αλλά και μνήμης πολλές φορές. Υπάρχει μεγάλη ερευνητική δραστηριότητα στον χώρο τις τελευταίες δεκαετίες και έχουν προταθεί αλγόριθμοι και δομές δεδομένων που στοχεύουν στη γρήγορη επεξεργασία των ροών δεδομένων, πολλές φορές, χρησιμοποιώντας προσεγγιστικές τεχνικές [Mun05]. Επίσης, μεγάλες προκλήσεις προκύπτουν και στην περίπτωση που θέλουμε να εφαρμόσουμε αλγορίθμους εξόρυξης γνώσης και μηχανικής μάθησης σε δεδομένα ροών [BGHP18, RJD19]. Στο κεφάλαιο αυτό, εστιάζουμε σε τεχνολογίες που χρησιμοποιούνται για τη διαχείριση και ανάλυση ροών δεδομένων και περιγράφουμε συνοπτικά μερικές από τις πλατφόρμες που εφαρμόζονται ευρύτατα. Μερικές βασικές αλγοριθμικές τεχνικές για την επεξεργασία ροών δεδομένων θα συναντήσουμε σε επόμενο κεφάλαιο.

Προαπαιτούμενη γνώση

Για την κατανόηση του περιεχομένου του κεφαλαίου αυτού, δεν απαιτούνται ειδικές γνώσεις. Ωστόσο, ο αναγνώστης θα διευκολυνθεί εάν έχει κάποια εμπειρία στην ανάπτυξη εφαρμογών σε γλώσσες όπως Java, Scala ή Python.

7.1 Εισαγωγή

Η διαχείριση και ανάλυση ροών είναι δύο σημαντικές έννοιες στον τομέα της επεξεργασίας δεδομένων σε πραγματικό χρόνο. Η επεξεργασία σε μια ροή μπορεί να αναφέρεται σε μια απλή εργασία, για παράδειγμα στη μέτρηση του πλήθους των λέξεων σε μια ροή κειμένου, ή σε πιο πολύπλοκες εργασίες ανάλυσης, όπως στην εύρεση ανωμαλιών σε μια ροή γεγονότων που αναφέρεται στη λειτουργία ενός αεροσκάφους.

Οι δύο αυτές βασικές λειτουργίες χρησιμοποιούνται ευρύτατα για τη διαχείριση συνεχών ροών δεδομένων, επιτρέποντας την έγκαιρη λήψη αποφάσεων και την εξαγωγή πολύτιμης πληροφορίας και γνώσης από μεταβαλλόμενες πηγές δεδομένων. Με απλά λόγια, μια *ροή δεδομένων* (data stream) είναι μια χρονικά μεταβαλλόμενη συλλογή δεδομένων η οποία, στις περισσότερες περιπτώσεις, είναι ατέρμονη. Σε κάποιες περιπτώσεις, η ταχύτητα αλλαγής των δεδομένων είναι μικρή και δεν προκαλεί ιδιαίτερα ζητήματα. Σε άλλες περιπτώσεις, όμως, η ταχύτητα με την οποία λαμβάνονται νέα δεδομένα ή μεταβάλλονται τα υπάρχοντα είναι μεγάλη και δημιουργούνται σημαντικές προκλήσεις στην απόδοση του συστήματος.

Οι εφαρμογές που απαιτούν τη διαχείριση και ανάλυση ροών δεδομένων είναι πολλές. Χαρακτηριστικά παραδείγματα είναι η ανάλυση tweets με σκοπό τον προσδιορισμό συναισθημάτων σε πραγματικό χρόνο [BHPG11], η μελέτη δεδομένων πρόγνωσης καιρού και η ενημέρωση των μοντέλων ανάλογα με τα πιο πρόσφατα δεδομένα, η ανάλυση χρονικά μεταβαλλόμενων δικτύων (π.χ. κοινωνικών δικτύων) και η πρόβλεψη της εξέλιξής τους, η ανάλυση δεδομένων δορυφορικών εικόνων, η ανάλυση οικονομικών δεδομένων με σκοπό την ανίχνευση απάτης (fraud detection). Γενικά, οπουδήποτε έχουμε χρονικά μεταβαλλόμενα δε-

δομένα θα μπορούσαν να χρησιμοποιηθούν τεχνικές διαχείρισης και ανάλυσης ροών δεδομένων.

Θυμίζουμε ότι η ταχύτητα (velocity) με την οποία αλλάζουν τα δεδομένα αποτελεί μία από τις τρεις βασικές προκλήσεις των μεγάλων δεδομένων (τα τρία V των μεγάλων δεδομένων: volume, velocity, variety). Για τον λόγο αυτόν, τα μοντέρνα συστήματα διαχείρισης και ανάλυσης δεδομένων υποστηρίζουν εξειδικευμένες λειτουργίες για τη διαχείριση ροών δεδομένων. Στη συνέχεια, παραθέτουμε μερικά βασικά χαρακτηριστικά των ροών δεδομένων και των αντίστοιχων συστημάτων επεξεργασίας τους.

- Οι ροές δεδομένων είναι συνεχείς και τις περισσότερες φορές ατέρμονες, με δεδομένα που φτάνουν σε πραγματικό χρόνο ή σχεδόν σε πραγματικό χρόνο. Στη γενική περίπτωση, υποθέτουμε ότι οι ροές δεν τερματίζουν ποτέ, με αποτέλεσμα να είναι δυνητικά άπειρες σε διάρκεια. Φυσικά, μπορούμε να τερματίσουμε την ανάγνωση από μια ροή, ανάλογα με τις ανάγκες της εφαρμογής μας.
- Οι ροές δεδομένων μπορούν να παράγουν δεδομένα με υψηλή ταχύτητα, συχνά με τα στοιχεία δεδομένων να καταφθάνουν με γρήγορους ρυθμούς. Αυτό το χαρακτηριστικό απαιτεί δυνατότητες πολύ γρήγορης επεξεργασίας για να συμβαδίζει με τη ροή δεδομένων, έτσι ώστε να μην υπάρχουν καθυστερήσεις και να μη συσσωρεύονται δεδομένα στην είσοδο του συστήματος.
- Ο όγκος των δεδομένων σε μια ροή μπορεί να ποικίλλει σημαντικά με την πάροδο του χρόνου. Μπορεί να υπάρχουν περίοδοι υψηλής εισροής δεδομένων και περίοδοι χαμηλότερης δραστηριότητας.
- Τα στοιχεία σε μια ροή έχουν μια χρονική σειρά, που σημαίνει ότι η ακολουθία με την οποία φτάνουν τα δεδομένα είναι σημαντική για την ανάλυση. Η διατήρηση της χρονολογικής σειράς είναι ζωτικής σημασίας για την ακριβή ερμηνεία.
- Οι ροές δεδομένων ενδέχεται να μην έχουν προκαθορισμένο τελικό σημείο ή όριο μεγέθους. Μπορούν να αναπτυχθούν επ' αόριστον και η διαχείριση της αποθήκευσης και της επεξεργασίας για τέτοιες ροές μπορεί να είναι δύσκολη.
- Οι ροές δεδομένων μπορεί να περιέχουν διαφορετικούς τύπους δεδομένων, όπως κείμενο, αριθμητικά δεδομένα, δεδομένα αισθητήρων, εικόνες, βίντεο, κ.ά. Ο χειρισμός αυτής της ποικιλίας απαιτεί συχνά εξειδικευμένες τεχνικές επεξεργασίας.
- Οι ροές δεδομένων μπορεί να περιέχουν θόρυβο, δηλαδή άσχετα ή λανθασμένα στοιχεία δεδομένων αναμειγμένα με χρήσιμες πληροφορίες. Για την ακριβή ανάλυση, μπορεί να απαιτηθούν τεχνικές μείωσης ή καθαρισμού θορύβου.
- Η ανάλυση ροών δεδομένων απαιτεί συχνά επεξεργασία σε πραγματικό χρόνο ή σχεδόν σε πραγματικό χρόνο για τη λήψη έγκαιρων αποφάσεων ή τον εντοπισμό μοτίβων καθώς συμβαίνουν γεγονότα.
- Τα συστήματα επεξεργασίας ροών λειτουργούν συχνά με περιορισμένους πόρους, όπως η μνήμη και η υπολογιστική ισχύς. Η αποτελεσματικότητα και η διαχείριση των πόρων είναι ζωτικής σημασίας.

- Εξειδικευμένοι αλγόριθμοι επεξεργασίας ροών δεδομένων χρησιμοποιούνται για την αποτελεσματική επεξεργασία τους. Αυτοί οι αλγόριθμοι έχουν σχεδιαστεί για να λειτουργούν με περιορισμένη μνήμη και να επεξεργάζονται δεδομένα με ένα μόνο πέρασμα.
- Σε ορισμένες περιπτώσεις, τα παλαιότερα δεδομένα σε μια ροή ενδέχεται να γίνουν λιγότερο σχετικά ή σημαντικά και μπορεί να χρειαστεί να απορριφθούν, για να δημιουργηθεί ελεύθερος χώρος για νέα δεδομένα. Αυτό είναι γνωστό ως αποσύνθεση δεδομένων ή γήρανση δεδομένων.
- Τα συστήματα επεξεργασίας ροής πρέπει να είναι επεκτάσιμα, για να χειρίζονται ροές δεδομένων υψηλής ταχύτητας. Μπορεί να απαιτείται κατανομημένη και παράλληλη επεξεργασία για την ικανοποίηση των απαιτήσεων απόδοσης.

7.2 Συστήματα επεξεργασίας ροών δεδομένων

Στη συνέχεια, θα περιγράψουμε συνοπτικά μερικά συστήματα που υποστηρίζουν διαχείριση και ανάλυση ροών δεδομένων και χρησιμοποιούνται ευρύτατα από μεγάλους οργανισμούς και επιχειρήσεις. Θα περιγράψουμε συνοπτικά το κάθε σύστημα και θα δώσουμε και ένα παράδειγμα χρήσης. Τα συστήματα με τα οποία θα ασχοληθούμε είναι: Apache Kafka, Apache Spark, Apache Flink, Apache Storm, και Amazon Kinesis.

7.2.1 Apache Kafka

Το σύστημα Apache Kafka είναι μια πλατφόρμα επεξεργασίας ροής γεγονότων (events) ανοικτού κώδικα που αναπτύχθηκε από το Apache Software Foundation. Έχει σχεδιαστεί για να χειρίζεται ροές δεδομένων σε πραγματικό χρόνο αποτελεσματικά, αξιόπιστα και σε μεγάλη κλίμακα. Το σύστημα Kafka χρησιμοποιείται ευρέως για την κατασκευή *αγωγών δεδομένων* (data pipelines) σε πραγματικό χρόνο και εφαρμογών που βασίζονται σε γεγονότα. Το σύστημα Kafka αποτελεί μια βασική τεχνολογία για τη δημιουργία υποδομών δεδομένων σε πραγματικό χρόνο και εφαρμογών που βασίζονται σε συμβάντα σε διάφορους τομείς, συμπεριλαμβανομένων των χρηματοοικονομικών υπηρεσιών, του ηλεκτρονικού εμπορίου, των μέσων κοινωνικής δικτύωσης, και του Διαδικτύου των Πραγμάτων (IoT). Η ικανότητά του να χειρίζεται τα δεδομένα με υψηλή απόδοση, ανθεκτικότητα και επεκτασιμότητα το καθιστά πολύ βασικό συστατικό των σύγχρονων αρχιτεκτονικών επεξεργασίας και ανάλυσης δεδομένων.

Ας δούμε περισσότερες λεπτομέρειες για τον τρόπο λειτουργίας του συστήματος αυτού. Ένα σύστημα επεξεργασίας ροής γεγονότων καταγράφει τα γεγονότα με τη σειρά και αυτές οι ροές αποθηκεύονται για επεξεργασία, ανάλυση και απόκριση σε πραγματικό χρόνο ή για ανάκτηση αργότερα. Επιπλέον, οι ροές γεγονότων μπορούν να δρομολογηθούν σε διαφορετικές εφαρμογές, ανάλογα με τις ανάγκες. Η ροή γεγονότων εξασφαλίζει συνεχή ροή και ερμηνεία δεδομένων, έτσι ώστε οι σωστές πληροφορίες να βρίσκονται στο σωστό μέρος, τη σωστή χρονική στιγμή.

Για να επιτευχθεί αυτό, το σύστημα Kafka εκτελείται σε συστάδα (cluster) σε έναν ή περισσότερους διακομιστές, που μπορούν να είναι εγκατεστημένοι σε πολλά διαφορετικά κέντρα δεδομένων (data centers), και παρέχει τη λειτουργικότητά του με κατανεμημένο, εξαιρετικά επεκτάσιμο, ελαστικό, ανεκτικό σε σφάλματα και ασφαλή τρόπο. Επιπλέον, το σύστημα Kafka μπορεί να αναπτυχθεί σε φυσικές μηχανές (bare metal), εικονικές μηχανές, κοντέινερ, καθώς και στο cloud. Οι βασικοί *παίκτες* στο σύστημα Kafka είναι οι εξής:

Μεσολαβητής (broker). Οι μεσολαβητές είναι οι διακομιστές που συνθέτουν τη συστάδα Kafka. Κάθε μεσολαβητής είναι υπεύθυνος για τη λήψη, την αποθήκευση και την εξυπηρέτηση δεδομένων. Οι μεσολαβητές κατανέμονται σε πολλαπλές μηχανές ή κόμβους, με στόχο την ανοχή στα σφάλματα και την επεκτασιμότητα.

Θέμα (topic). Τα θέματα είναι λογικά κανάλια ή κατηγορίες όπου δημοσιεύονται δεδομένα. Οι *παραγωγοί* δημοσιεύουν μηνύματα στις διάφορες θεματικές περιοχές και οι *καταναλωτές* εγγράφονται σε αυτά, για να λαμβάνουν και να επεξεργάζονται τα σχετικά μηνύματα. Τα θέματα μπορούν να έχουν πολλαπλές κατατμήσεις για παράλληλη επεξεργασία και επεκτασιμότητα.

Παραγωγός (producer). Οι παραγωγοί είναι εφαρμογές ή συστήματα που στέλνουν δεδομένα ή μηνύματα σε θέματα του Kafka. Οι παραγωγοί μπορούν να καθορίσουν ένα κλειδί για τα μηνύματα, το οποίο καθορίζει την κατάτμηση στην οποία αποστέλλεται ένα μήνυμα.

Καταναλωτής (consumer). Οι καταναλωτές είναι εφαρμογές ή συστήματα που εγγράφονται στα διαθέσιμα θέματα του Kafka και επεξεργάζονται μηνύματα. Το σύστημα Kafka υποστηρίζει δύο τύπους καταναλωτών: ομάδες καταναλωτών και απλούς καταναλωτές. Οι ομάδες καταναλωτών επιτρέπουν την παράλληλη επεξεργασία των μηνυμάτων μέσα σε μια ομάδα, ενώ οι απλοί καταναλωτές παρέχουν περισσότερο έλεγχο, αλλά στερούνται συντονισμού ομάδας.

Ομάδα καταναλωτών (consumer group). Μια ομάδα καταναλωτών είναι μια λογική ομάδα καταναλωτών που καταναλώνουν συλλογικά μηνύματα από ένα ή περισσότερα θέματα του συστήματος Kafka. Το σύστημα Kafka διασφαλίζει ότι κάθε κατάτμηση ενός θέματος καταναλώνεται μόνο από έναν καταναλωτή σε μια ομάδα καταναλωτών, επιτρέποντας την εξισορρόπηση φόρτου και την ανοχή στα σφάλματα.

Κατάτμηση (partition). Κάθε θέμα του Kafka χωρίζεται σε ένα ή περισσότερα διαμερίσματα. Οι κατατμήσεις είναι η βασική μονάδα παραλληλισμού στο σύστημα Kafka και επιτρέπουν ταυτόχρονες προσπελάσεις (αναγνώσεις και εγγραφές). Οι κατατμήσεις διανέμονται σε διαφορετικούς μεσολαβητές (brokers) και κάθε κατάτμηση έχει μια μοναδική ακολουθία μηνυμάτων.

Ελεγκτής Kafka (Kafka controller). Ο ελεγκτής Kafka είναι ένα ενσωματωμένο εργαλείο που είναι υπεύθυνο για τη διαχείριση των διαμερισμάτων, των αστοχιών των μεσολαβητών, και των μεταδεδομένων συστάδας. Εξασφαλίζει ότι τα διαμερίσματα και τα αντίγραφα κατανέμονται ομοιόμορφα μεταξύ των μεσολαβητών και χειρίζεται διαφορετικά σενάρια ανακατεύθυνσης.

Για την ανάπτυξη εφαρμογών Kafka θα πρέπει, επομένως, να προσδιοριστούν πλήρως όλα τα λειτουργικά στοιχεία που συμμετέχουν. Ωστόσο, τα δύο βασικότερα είναι οι παραγωγοί και οι καταναλωτές και ο τρόπος με τον οποίο επιτυγχάνεται η κατανάλωση των μηνυμάτων που παράγονται από τους παραγωγούς.

Το σύστημα Kafka υποστηρίζει βιβλιοθήκες για διάφορες γλώσσες προγραμματισμού, καθιστώντας το ευέλικτο και προσβάσιμο σε ένα ευρύ φάσμα προγραμματιστών. Αυτές οι βιβλιοθήκες επιτρέπουν την ενσωμάτωση του Kafka σε ένα ευρύ φάσμα εφαρμογών και συστημάτων, ανεξάρτητα από τη γλώσσα προγραμματισμού που χρησιμοποιείται. Η υποστήριξη πολλών γλωσσών από το σύστημα Kafka είναι ένα από τα δυνατά του σημεία, καθιστώντας το κατάλληλο για διαφορετικές εφαρμογές. Συγκεκριμένα, μερικές από τις γλώσσες προγραμματισμού που υποστηρίζονται είναι οι εξής: Java, C/C++, Scala, Python, Rust, Haskell, Go, Ruby, κ.ά. Ο ενδιαφερόμενος αναγνώστης μπορεί να βρει περισσότερες λεπτομέρειες για την ανάπτυξη εφαρμογών Kafka στις βιβλιογραφικές αναφορές [Gar13, NSP17, Kum20], καθώς και στον επίσημο ιστότοπο του Kafka στο <https://kafka.apache.org/>.

Εδώ θα εστιάσουμε στην περιγραφή μίας πολύ απλής εφαρμογής, που σχετίζεται με τη δημιουργία παραγωγών και καταναλωτών. Για να δείξουμε τον τρόπο υλοποίησης θα χρησιμοποιήσουμε τη γλώσσα Python για τον σκοπό αυτόν, αλλά μπορεί να χρησιμοποιηθεί οποιαδήποτε από τις γλώσσες προγραμματισμού που υποστηρίζονται. Πιο συγκεκριμένα, θα υλοποιήσουμε δύο παραγωγούς και έναν καταναλωτή. Με παρόμοιο τρόπο μπορούν να χρησιμοποιηθούν και περισσότεροι παραγωγοί/καταναλωτές, ανάλογα με τις ανάγκες της εκάστοτε εφαρμογής.

```

1 # Κώδικας για Producer1
2 from confluent_kafka import Producer
3
4 # Ορισμός Kafka Broker
5 broker_address = 'localhost:9090'
6
7 # Δημιουργία αντικειμένου Kafka Producer
8 producer1 = Producer({'bootstrap.servers': bootstrap_servers})
9
10 # Ορισμός θέματος για το οποίο στέλνει ο Producer1
11 topic = 'BasicTopic'
12
13 # Παραγωγή μηνυμάτων
14 for i in range(1, 100):
15     message = f"Producer 1 message {i}"
16     producer1.produce(topic, key='key', value=message)
17     print(f"Produced: {message}")
18
19 # Αναμονή για αποστολή όλων των εκκρεμών μηνυμάτων
20 producer1.flush()

```

```
1 # Κώδικας για Producer2
2 from confluent_kafka import Producer
3
4 # Ορισμός Kafka Broker
5 broker_address = 'localhost:9090'
6
7 # Δημιουργία αντικειμένου Kafka Producer
8 producer2 = Producer({'bootstrap.servers': bootstrap_servers})
9
10 # Ορισμός θέματος για το οποίο στέλνει ο Producer2
11 topic = 'BasicTopic'
12
13 # Παραγωγή μηνυμάτων
14 for i in range(100, 200):
15     message = f"Producer 2 message {i}"
16     producer2.produce(topic, key='key', value=message)
17     print(f"Produced: {message}")
18
19 # Αναμονή για αποστολή όλων των εκκρεμών μηνυμάτων
20 producer2.flush()
```

Οι παραγωγοί, αρχικά, θέτουν στη μεταβλητή `broker_address` τη διεύθυνση του μεσο-λαβητή και, στη συνέχεια, στέλνουν μηνύματα στο θέμα για το οποίο ενδιαφέρονται, στη συγκεκριμένη περίπτωση το `BasicTopic`. Για το παράδειγμά μας, ο `Producer1` στέλνει μηνύματα με κωδικούς από 1 έως και 99 και ο `Producer2` στέλνει μηνύματα με κωδικούς από 100 έως και 199. Από την άλλη πλευρά, ο καταναλωτής εγγράφεται στο θέμα `BasicTopic` και αρχίζει τη διαδικασία κατανάλωσης και επεξεργασίας των μηνυμάτων.

```
1 # Κώδικας για Consumer1
2 from confluent_kafka import Consumer, KafkaError
3
4 # Ορισμός Kafka Broker
5 broker_address = 'localhost:9090'
6
7 # Δημιουργία αντικειμένου Kafka Consumer
8 consumer1 = Consumer({
9     'bootstrap.servers': broker_address,
10    'group.id': 'my-group',
11    'auto.offset.reset': 'earliest'
12 })
13
14 # Εγγραφή στο θέμα BasicTopic
15 topic = 'BasicTopic'
16 consumer1.subscribe([topic])
17
18 # Κατανάλωση και επεξεργασία μηνυμάτων
19 try:
20     while True:
21         msg = consumer1.poll(1.0) # Αναμονή για 1 δευτερόλεπτο για νέο
            μήνυμα.
22
23         if msg is None:
24             continue
```

```

25         if msg.error():
26             if msg.error().code() == KafkaError._PARTITION_EOF:
27                 print('Reached end of partition')
28             else:
29                 print(f'Error: {msg.error()}')
30         else:
31             print(f'Consumer 1 received: {msg.value().decode("utf-8")}')
32
33 except KeyboardInterrupt:
34     pass
35 finally:
36     consumer1.close()

```

7.2.2 Apache Spark

Το σύστημα Spark υποστηρίζει την επεξεργασία και ανάλυση ροών δεδομένων. Το σύστημα Kafka, που εξετάσαμε στην προηγούμενη ενότητα, παρέχει βασικούς μηχανισμούς επεξεργασίας και διαχείρισης ροών, αλλά δεν μπορεί να χρησιμοποιηθεί για πιο πολύπλοκες εργασίες. Τονίζουμε εδώ, ότι πολύ συχνά βλέπουμε σε εφαρμογές το Spark να συνεργάζεται με το σύστημα Kafka.

Η αρχική προσέγγιση που ακολούθησε το Spark είναι η υποστήριξη ροών δεδομένων χρησιμοποιώντας το RDD API μέσω του Spark Streaming. Η άφιξη νέων δεδομένων μπορεί να μοντελοποιηθεί με μικρά τμήματα δεδομένων (μικρά RDDs), τα οποία μπορούν να καταναλωθούν από τις εφαρμογές. Ωστόσο, το Spark Streaming δεν υποστηρίζεται πλέον (<https://spark.apache.org/docs/latest/streaming-programming-guide.html>), και θεωρείται μια προσέγγιση χαμηλού επιπέδου, καθώς ο προγραμματιστής πρέπει να φροντίσει για πολλές λεπτομέρειες υλοποίησης. Επομένως, ο ορθόδοξος τρόπος επεξεργασίας και ανάλυσης ροών δεδομένων στο Spark είναι μέσω του Structured Streaming το οποίο βασίζεται στο DataFrame API.

Το Structured Streaming είναι μια δυνατή και επεκτάσιμη μηχανή η οποία παρέχει ένα υψηλού επιπέδου API για την επεξεργασία και ανάλυση ροών δεδομένων σε πραγματικό χρόνο με την ίδια ευκολία όπως και η επεξεργασία κατά τμήματα (batch processing).

Το Structured Streaming αντιμετωπίζει τις διαφορετικές πηγές δεδομένων ως πίνακες. Υποστηρίζει μια ποικιλία εξωτερικών πηγών δεδομένων, συμπεριλαμβανομένων των Kafka, Flume, HDFS, Twitter/X, Kinesis, κ.ά., καθιστώντας το μια πολύ καλή και ευέλικτη λύση για την πρόσληψη πληροφορίας από διάφορες ροές δεδομένων. Τα ερωτήματα επεξεργασίας και ανάλυσης χρησιμοποιούν το DataFrame API και τη γλώσσα ερωτημάτων SQL. Αυτό είναι πολύ σημαντικό, επειδή διευκολύνεται σημαντικά η ανάπτυξη εφαρμογών, καθώς η επεξεργασία ροών βασίζεται επάνω σε γνωστές λειτουργίες του DataFrame API. Επίσης, τονίζεται ότι το Structured Streaming υποστηρίζει ανοχή σε ενδεχόμενα σφάλματα, χρησιμοποιώντας σημεία ελέγχου (check points) και αρχεία καταγραφής (write-ahead logs).

Στη συνέχεια, θα περιγράψουμε ένα απλό παράδειγμα χρήσης του Structured Streaming, όπου τα δεδομένα βασίζονται σε μία συνεχή ροή κειμένου. Αυτό που θέλουμε στην έξοδο είναι να μετρήσουμε το πλήθος των συμβολοσειρών που έχουν περισσότερους από τέσσερις

χαρακτήρες. Θα υποθέσουμε ότι η ροή διαβάζεται από τη διεύθυνση `localhost:9098`. Τα αποτελέσματα θέλουμε να εκτυπώνονται στην οθόνη.

```
1
2 import org.apache.spark.sql.SparkSession
3
4 // Δημιουργία spark session
5 val spark = SparkSession.builder()
6     .appName("StructuredStreamingSocketExample")
7     .master("local[*]")
8     .getOrCreate()
9
10 // Ορισμός σχήματος του DataFrame, μία στήλη τύπου STRING
11 val schema = "value STRING"
12
13 // Ανάγνωση δεδομένων από socket ως ροή δεδομένων με τη μορφή DataFrame
14 val streamDF = spark.readStream
15     .format("socket")
16     .option("host", "localhost")
17     .option("port", 9098)
18     .schema(schema)
19     .load()
20
21 // Λειτουργία που θέλουμε να εφαρμοστεί στη ροή
22 val result = streamDF
23     .as[String]
24     .flatMap(_.split(" "))
25     .filter(length(col("value")) > 4)
26     .groupBy("value")
27     .count()
28
29 // Ορισμός ερωτήματος
30 val query = result.writeStream
31     .outputMode("complete")
32     .format("console")
33     .start()
34
35 // Έναρξη εκτέλεσης του ερωτήματος
36 query.awaitTermination()
```

7.2.3 Apache Flink

Το Apache Flink (<https://flink.apache.org/>) είναι μια μηχανή επεξεργασίας και ανάλυσης μεγάλων δεδομένων με πολλές δυνατότητες επεξεργασίας ροών δεδομένων, χωρίς να χρησιμοποιείται διαφορετικό πλαίσιο. Τόσο η επεξεργασία δεδομένων με τη μορφή ροής όσο και η επεξεργασία σε τμήματα (batches) υλοποιούνται στο ίδιο πλαίσιο, χωρίς να απαιτείται ιδιαίτερη μεταχείριση. Αυτό σημαίνει ότι το Flink μπορεί να χρησιμοποιηθεί για επεξεργασία δεδομένων σε πραγματικό χρόνο και σε παραδοσιακές εργασίες επεξεργασίας κατά τμήματα, καθιστώντας το μια ευέλικτη επιλογή για διάφορες περιπτώσεις χρήσης.

Το Flink διαθέτει ένα πλούσιο οικοσύστημα το οποίο περιλαμβάνει βιβλιοθήκες μηχανι-

κής μάθησης (FlinkML), επεξεργασία γραφημάτων (Gelly), επεξεργασία σύνθετων συμβάντων (CEP), και πολλά άλλα. Οι βιβλιοθήκες αυτές επεκτείνουν τις δυνατότητες του Flink και επιτρέπουν την υλοποίηση διαφορετικών εφαρμογών επεξεργασίας και ανάλυσης δεδομένων. Το Flink έχει σχεδιαστεί για καταναμημένη και επεκτάσιμη επεξεργασία. Μπορεί να κλιμακωθεί οριζόντια για να επεξεργαστεί μεγάλους όγκους δεδομένων, ενώ προσφέρει ενσωματωμένους μηχανισμούς ανοχής στα σφάλματα, διασφαλίζοντας έτσι ότι οι εργασίες επεξεργασίας θα συνεχίσουν κανονικά την εκτέλεσή τους, ακόμη και στην περίπτωση αστοχιών.

Επίσης, το Flink (όπως και το Spark) παρέχει ένα SQL API που επιτρέπει στους χρήστες να εκφράζουν λειτουργίες επεξεργασίας ροής χρησιμοποιώντας ερωτήματα SQL, καθιστώντας το προσβάσιμο σε ένα ευρύτερο φάσμα χρηστών, συμπεριλαμβανομένων των αναλυτών δεδομένων και των προγραμματιστών που αναπτύσσουν εφαρμογές SQL.

Στη συνέχεια, θα δώσουμε ένα παράδειγμα σε Scala για το πώς μπορούμε να χρησιμοποιήσουμε το Flink για την επεξεργασία μιας ροής κειμένου, με στόχο να μετρήσουμε το συνολικό πλήθος των λέξεων που υπάρχουν στη ροή. Το παράδειγμα αυτό είναι παρόμοιο με αυτό που χρησιμοποιήσαμε στην προηγούμενη ενότητα για το Apache Spark.

```
1 import org.apache.flink.streaming.api.scala._
2
3 object FlinkStreamingSocketWordCount {
4
5     def main(args: Array[String]): Unit = {
6
7         // Περιβάλλον εκτέλεσης
8         val env = StreamExecutionEnvironment.getExecutionEnvironment
9
10        // Ορισμός της πηγής (hostname, port)
11        val hostname = "localhost"
12        val port = 9098
13
14        // Δημιουργία socket
15        val socketDataStream = env.socketTextStream(hostname, port)
16
17        // Μέτρηση λέξεων
18        val wordCountStream = socketDataStream
19            .flatMap { _.toLowerCase.split("\\W+") }
20            .filter { _.nonEmpty }
21            .map { (_, 1) }
22            .keyBy(0)
23            .sum(1)
24
25        // Εκτύπωση αποτελεσμάτων
26        wordCountStream.print()
27
28        // Εκτέλεση της εργασίας
29        env.execute("FlinkStreamingSocketWordCount")
30    }
31 }
```

7.2.4 Apache Storm

Το Apache Storm (<https://storm.apache.org/>) είναι ένα σύστημα ανοικτού κώδικα που υποστηρίζει κατανεμημένη επεξεργασία ροών δεδομένων σε πραγματικό χρόνο, σχεδιασμένο για την επεξεργασία μεγάλου όγκου ροών δεδομένων με υψηλή αξιοπιστία και πολύ καλές επιδόσεις. Αναπτύχθηκε αρχικά από την Twitter Inc., ενώ τώρα είναι ένα έργο που αναπτύσσεται στο πλαίσιο του Apache Software Foundation.

Η μηχανή Apache Storm έχει σχεδιαστεί για την επεξεργασία ροών δεδομένων σε πραγματικό χρόνο, καθιστώντας την κατάλληλη για περιπτώσεις χρήσης που απαιτούν επεξεργασία χαμηλής καθυστέρησης και γρήγορες απαντήσεις κατά την επεξεργασία των εισερχόμενων δεδομένων. Η μηχανή Storm υποστηρίζει την κατανομή των δεδομένων και μπορεί να κλιμακωθεί οριζόντια, για την επεξεργασία ροών δεδομένων υψηλής απόδοσης και μεγάλης κλίμακας, ενώ μπορεί να λειτουργήσει σε συστάδες υπολογιστών (clusters).

Τονίζεται, επίσης, ότι η μηχανή Storm παρέχει ενσωματωμένους μηχανισμούς ανοχής σε σφάλματα. Εάν ένας κόμβος της συστάδας αποτύχει, το Storm θα εκχωρήσει εκ νέου τις προβληματικές εργασίες σε άλλους κόμβους, για να διασφαλίσει ότι η επεξεργασία δεδομένων θα συνεχιστεί χωρίς διακοπή. Τονίζεται, επίσης, ότι το Storm απαιτεί την παρουσία του εργαλείου ZooKeeper (<https://zookeeper.apache.org/>).

Το Apache Storm είναι ένα σύστημα που έχει μια ενεργή κοινότητα και ένα αναπτυσσόμενο οικοσύστημα βιβλιοθηκών και επεκτάσεων που επεκτείνουν τη λειτουργικότητά του, ενώ ταυτόχρονα διευκολύνουν τη χρήση του σε διάφορα πεδία εφαρμογών. Χρησιμοποιείται σε διάφορους τομείς, συμπεριλαμβανομένων των οικονομικών, των μέσων κοινωνικής δικτύωσης, του Διαδικτύου των Πραγμάτων, κ.ά., για εργασίες όπως ανάλυση σε πραγματικό χρόνο, συστάσεις (recommendations), και παρακολούθηση εφαρμογών και υπηρεσιών σε πραγματικό χρόνο.

Ας περιγράψουμε συνοπτικά στη συνέχεια τα βασικά συστατικά στοιχεία του Storm, τα οποία συνεργάζονται αρμονικά μεταξύ τους για την υποστήριξη της λειτουργικότητας.

Τοπολογία (topology). Η τοπολογία στη μηχανή Storm είναι η κύρια υπολογιστική μονάδα. Στην ουσία, αντιπροσωπεύει ένα *κατευθυνόμενο άκυκλο γράφημα* (DAG) που αποτελείται από στόμια και μπουλόνια. Τα στόμια είναι πηγές ροών δεδομένων και τα μπουλόνια είναι μονάδες επεξεργασίας που καταναλώνουν, μετασχηματίζουν και εκπέμπουν δεδομένα. Οι τοπολογίες καθορίζουν τον τρόπο με τον οποίο τα δεδομένα ρέουν από το ένα στοιχείο στο άλλο.

Στόμιο (spout). Τα στόμια αποτελούν, στην ουσία, τις πηγές των ροών δεδομένων σε μια τοπολογία Storm. Μπορούν να διαβάσουν δεδομένα από διάφορες πηγές, όπως Kafka, Twitter/X ή από μια προσαρμοσμένη πηγή δεδομένων. Τα στόμια εκπέμπουν πλειάδες (tuples), που αποτελούν τα θεμελιώδη στοιχεία δεδομένων στο Storm.

Μπουλόνι (bolt). Τα μπουλόνια είναι οι μονάδες επεξεργασίας στο Storm. Λαμβάνουν πλειάδες από στόμια ή άλλα μπουλόνια, επεξεργάζονται τα δεδομένα και εκπέμπουν νέες πλειά-

δες ή εκτελούν άλλες λειτουργίες. Τα μπουλόνια μπορεί να είναι με ή χωρίς κατάσταση και μπορούν να εκτελούν διάφορους τύπους μετασχηματισμών στα δεδομένα και υπολογισμούς.

Πλειάδα (tuple). Οι πλειάδες αποτελούν τη βασική δομική μονάδα πληροφορίας στο Storm. Είναι μια ταξινομημένη λίστα στοιχείων, όπως μια πλειάδα στα μαθηματικά. Οι πλειάδες εκπέμπονται από στόμια και μπουλόνια και περιέχουν τα προς επεξεργασία δεδομένα. Οι πλειάδες υποβάλλονται σε επεξεργασία παράλληλα στην τοπολογία.

Νέφος (nimbus). Το νέφος είναι ο κύριος κόμβος (master Node) μιας συστάδας του Storm, ο οποίος είναι υπεύθυνος για τη διανομή κώδικα και τη διαμόρφωση στους κόμβους εργασίας, την ανάθεση εργασιών σε κόμβους και την παρακολούθηση της συνολικής καλής λειτουργίας της συστάδας.

Διεργασίες εργατών (worker process). Οι διεργασίες εργατών εκτελούνται σε κόμβους εργασίας και εκτελούν εργασίες που έχουν ανατεθεί από το νέφος (nimbus). Κάθε διεργασία εργάτη μπορεί να περιέχει πολλούς εκτελεστές, οι οποίοι είναι υπεύθυνοι για την παράλληλη λειτουργία των μπουλονιών και των στομιών.

Επόπτης (Supervisor). Οι επόπτες εκτελούνται σε κόμβους εργασίας στη συστάδα του Storm. Είναι υπεύθυνοι για την έναρξη και τη διαχείριση των διεργασιών εργασίας. Κάθε επόπτης διαχειρίζεται μία ή περισσότερες διεργασίες.

ZooKeeper. Το εργαλείο Apache ZooKeeper χρησιμοποιείται για τον συντονισμό και διαχείριση κατανεμημένων ρυθμίσεων σε μία συστάδα Storm. Το Storm χρησιμοποιεί το ZooKeeper για να διατηρήσει την κατάσταση της συστάδας και για τη διαδικασία εκλογής αρχηγού. Επίσης, παίζει καθοριστικό ρόλο στην υψηλή διαθεσιμότητα και αξιοπιστία των δεδομένων και των εργασιών που εκτελούνται.

Storm UI. Το Storm UI είναι μια διεπαφή που παρέχει μια συλλογή εργαλείων για την παρακολούθηση της κατάστασης και της απόδοσης των τοπολογιών και των συστάδων. Εμφανίζει πολλές χρήσιμες πληροφορίες, όπως τον αριθμό των επεξεργασμένων πλειάδων, μετρήσεις σφαλμάτων και στατιστικές πληροφορίες της χρήσης των διαφορετικών πόρων.

Αρχεία καταγραφής (log files): Το Storm δημιουργεί αρχεία καταγραφής που περιέχουν πληροφορίες σχετικά με την εκτέλεση τοπολογιών και διεργασιών εργασίας. Τα αρχεία καταγραφής είναι πολύτιμα για τον εντοπισμό σφαλμάτων και την παρακολούθηση της κατάστασης του συστήματος.

Το Apache Storm υποστηρίζει, κυρίως, δύο γλώσσες προγραμματισμού για τη δημιουργία και τη διαμόρφωση τοπολογιών και στοιχείων. Η Java είναι η κύρια και πιο συχνά χρησιμοποιούμενη γλώσσα προγραμματισμού για την ανάπτυξη τοπολογιών και στοιχείων στο σύ-

στημα Apache Storm. Οι βασικές βιβλιοθήκες του Storm και τα περισσότερα παραδείγματα είναι γραμμένα σε Java. Μπορούμε να χρησιμοποιήσουμε την Java για να ορίσουμε τα βασικά στοιχεία του Storm και να δημιουργήσουμε τις τοπολογίες που περιγράφουν τον τρόπο ροής των δεδομένων μέσω των αγωγών επεξεργασίας (processing pipelines). Η Clojure είναι μια άλλη γλώσσα που υποστηρίζεται από το Apache Storm. Η Clojure είναι μια *συναρτησιακή γλώσσα προγραμματισμού* (functional programming language), που αποτελεί διάλεκτο της γλώσσας προγραμματισμού Lisp, και εκτελείται στην εικονική μηχανή Java Virtual Machine (JVM). Αν και η Clojure δεν χρησιμοποιείται ευρέως όσο η Java για την ανάπτυξη εφαρμογών στην πλατφόρμα Storm, παρέχει έναν εναλλακτικό τρόπο ορισμού τοπολογιών και στοιχείων. Πολλοί προγραμματιστές προτιμούν τη γλώσσα Clojure, λόγω της συναρτησιακής της φύσης, η οποία διευκολύνει τη συγγραφή συνοπτικού κώδικα.

Αν και η Java και η Clojure αποτελούν τις βασικές γλώσσες προγραμματισμού που χρησιμοποιούνται για την ανάπτυξη εφαρμογών σε Apache Storm, μπορούν να χρησιμοποιηθούν και άλλες γλώσσες μέσω της χρήσης του πρωτοκόλλου MultiLang, το οποίο αποτελεί ένα πρωτόκολλο επικοινωνίας που υποστηρίζεται από το Apache Storm για την ενσωμάτωση στοιχείων υλοποιημένων σε διαφορετικές γλώσσες προγραμματισμού, όπως Python, Ruby, JavaScript (<https://storm.apache.org/releases/2.2.0/Multilang-protocol.html>).

Στη συνέχεια, δίνουμε ένα πολύ απλό παράδειγμα υλοποιημένο σε Java. Θα χρησιμοποιήσουμε το κλασικό παράδειγμα που μετρά το πλήθος εμφανίσεων όλων των λέξεων από ένα σύνολο προτάσεων, οι οποίες διαβάζονται ως ροή και η επεξεργασία τους γίνεται σταδιακά.

```
1 import org.apache.storm.Config;
2 import org.apache.storm.LocalCluster;
3 import org.apache.storm.StormSubmitter;
4 import org.apache.storm.generated.StormTopology;
5 import org.apache.storm.spout.SpoutOutputCollector;
6 import org.apache.storm.task.TopologyContext;
7 import org.apache.storm.topology.BasicOutputCollector;
8 import org.apache.storm.topology.OutputFieldsDeclarer;
9 import org.apache.storm.topology.TopologyBuilder;
10 import org.apache.storm.topology.base.BaseBasicBolt;
11 import org.apache.storm.topology.base.BaseRichSpout;
12 import org.apache.storm.tuple.Fields;
13 import org.apache.storm.tuple.Tuple;
14 import org.apache.storm.tuple.Values;
15 import org.apache.storm.utils.Utils;
16 import java.util.Map;
17
18 public class WordCountTopology {
19     public static class SentenceSpout extends BaseRichSpout {
20         private SpoutOutputCollector collector;
21         private String[] demotxt = {
22             "This is the first sentence",
23             "This is the second sentence",
24             "Apache Storm is great!"
25         };
26         private int index = 0;
27
28         @Override
```

```

29     public void open(Map conf, TopologyContext context,
SpoutOutputCollector collector) {
30         this.collector = collector;
31     }
32
33     @Override
34     public void nextTuple() {
35         if (index < demotxt.length) {
36             collector.emit(new Values(demotxt[index]));
37             index++;
38         } else {
39             Utils.sleep(1000); // Αναμονή για 1 δευτερόλεπτο
40         }
41     }
42
43     @Override
44     public void declareOutputFields(OutputFieldsDeclarer declarer) {
45         declarer.declare(new Fields("sentence"));
46     }
47 }
48
49 public static class WordCountBolt extends BaseBasicBolt {
50     private Map<String, Integer> wordCounts = new HashMap<>();
51
52     @Override
53     public void execute(Tuple input, BasicOutputCollector collector) {
54         String sentence = input.getStringByField("sentence");
55         String[] words = sentence.split(" "); // Διαχωρισμός λέξεων
56
57         for (String word : words) {
58             wordCounts.put(word, wordCounts.getOrDefault(word, 0) + 1);
59             collector.emit(new Values(word, wordCounts.get(word)));
60         }
61     }
62
63     @Override
64     public void declareOutputFields(OutputFieldsDeclarer declarer) {
65         declarer.declare(new Fields("word", "count"));
66     }
67 }
68
69 public static void main(String[] args) throws Exception {
70     TopologyBuilder builder = new TopologyBuilder();
71
72     builder.setSpout("sentence-spout", new SentenceSpout());
73     builder.setBolt("wordcount-bolt", new WordCountBolt())
74         .fieldsGrouping("sentence-spout", new Fields("sentence"));
75
76     Config myconfig = new Config();
77     myconfig.setDebug(true);
78
79     LocalCluster mycluster = new LocalCluster();
80     mycluster.submitTopology("wordcount-topology", myconfig,

```

```

81         builder.createTopology();
82         Utils.sleep(20000); // Εκτέλεση της τοπολογίας για 20 δευτερόλεπτα
83         mycluster.killTopology("wordcount-topology");
84         mycluster.shutdown();
85
86     }
87 }

```

Το αντικείμενο `SentenceSpout` στέλνει τις φράσεις ως πλειάδες, έτσι ώστε να προσομοιώσει μια ροή δεδομένων. Το αντικείμενο `WordCountBolt` μετατρέπει την κάθε φράση σε μία συλλογή από λέξεις, όπου ως διαχωριστής χρησιμοποιείται ο κενός χαρακτήρας. Τονίζουμε εδώ, ότι εάν θέλουμε να εφαρμοστεί οποιαδήποτε προ-επεξεργασία στο κείμενο (π.χ., διαγραφή σημείων στίξης) θα πρέπει να εφαρμοστεί πριν από τον διαχωρισμό σε λέξεις. Στη συνέχεια, υπολογίζεται το πλήθος εμφανίσεων της κάθε λέξης και δημιουργούνται οι αντίστοιχες πλειάδες στην έξοδο, οι οποίες περιέχουν δύο πεδία, τη λέξη και το πλήθος εμφανίσεων. Η τοπολογία είναι υπεύθυνη για τη σύνδεση ανάμεσα στο στόμιο και το μπουλόνι, ομαδοποιώντας προτάσεις ανά πεδίο και ορίζοντας τα πεδία των πλειάδων εξόδου.

Η τοπολογία, όπως έχει υλοποιηθεί, εκτελείται τοπικά και επεξεργάζεται προτάσεις για 20 δευτερόλεπτα και, στη συνέχεια, τερματίζει. Εάν θέλουμε να εκτελεστεί σε κάποια συστάδα, τότε θα πρέπει να γίνει κάποια τροποποίηση στο αντικείμενο `mycluster` που έχουμε ορίσει προηγουμένως. Συγκεκριμένα, στο παράδειγμα υποθέτουμε ότι η διεύθυνση της συστάδας περνάει ως παράμετρος στη συνάρτηση `main()` και, ακολούθως, χρησιμοποιείται στη συνάρτηση `submitTopologyWithProgressBar`. Εδώ χρησιμοποιούνται πέντε (5) κόμβοι εργασίας στη συστάδα.

```

1     config.setNumWorkers(5);
2     StormSubmitter.submitTopologyWithProgressBar(args[0], config, builder.
        createTopology());

```

7.2.5 Amazon Kinesis

Οι τεχνολογίες επεξεργασίας και ανάλυσης ροών δεδομένων που εξετάσαμε στις προηγούμενες ενότητες βασίζονται σε ανοικτό κώδικα (open source) και είναι προσβάσιμες άμεσα από οποιονδήποτε ενδιαφερόμενο. Στην ενότητα αυτή θα περιγράψουμε συνοπτικά μια πλατφόρμα της Amazon, το Amazon Kinesis (<https://aws.amazon.com/kinesis/>), που συμπεριλαμβάνεται στη μεγαλύτερη σουίτα εφαρμογών της Amazon, τη σουίτα Amazon Web Services (AWS).

Οι υπηρεσίες που προσφέρονται από το Kinesis είναι πλήρως ενσωματωμένες στο οικοσύστημα AWS, επιτρέποντας στους χρήστες να ενσωματώνουν απρόσκοπτα την επεξεργασία ροών δεδομένων με άλλες υπηρεσίες AWS, όπως Amazon S3, Amazon Redshift, Amazon Elasticsearch, κ.ά. Αυτή η ενοποίηση διευκολύνει σημαντικά τους πελάτες του AWS να δημιουργήσουν αγωγούς επεξεργασίας δεδομένων και εφαρμογές σε πραγματικό χρόνο στο περιβάλλον cloud AWS.

Το Amazon Kinesis παρέχει διάφορες βιβλιοθήκες και υποστηρίζει πολλές γλώσσες προγραμματισμού για την ανάπτυξη εφαρμογών, έτσι ώστε να πραγματοποιείται επεξεργασία και ανάλυση των δεδομένων της ροής. Οι γλώσσες προγραμματισμού που υποστηρίζονται

επίσημα από το Kinesis είναι οι: Java, Python, Ruby, Go, Node.js, C#. Επίσης, δυνητικά μπορεί να υποστηριχθεί οποιαδήποτε γλώσσα, καθώς το Kinesis μπορεί να δεχθεί αιτήματα HTTP μέσω του αντίστοιχου API που προσφέρει.

Το Amazon Kinesis προσφέρει τρεις διαφορετικές πλατφόρμες υπηρεσιών που σχετίζονται με ροές δεδομένων. Αυτές είναι: Kinesis Data Streams, Kinesis Firehose, και Kinesis Video Streams. Το Kinesis Firehose αποτελεί μια πλατφόρμα για λειτουργίες Extract-Transform-Load (ETL) σε ροές δεδομένων, ενώ το Kinesis Video Streams εστιάζει στην επεξεργασία και ανάλυση video. Εμείς θα εστιάσουμε στην πλατφόρμα Data Streams, η οποία αποτελεί το βασικό συστατικό στοιχείο της πλατφόρμας Amazon Kinesis και προσφέρει τεχνικές για την ανάλυση, επεξεργασία, αποθήκευση ροών δεδομένων οποιασδήποτε κλίμακας. Επίσης, με τη χρήση του Kinesis Data Analytics έχουμε τη δυνατότητα να επεξεργαστούμε και να αναλύσουμε ροές δεδομένων χρησιμοποιώντας τη γλώσσα ερωτημάτων SQL. Πριν περιγράψουμε ένα συγκεκριμένο παράδειγμα χρήσης, ας δούμε ποια είναι τα βασικά τμήματα του Kinesis Data Streams και πώς μπορεί να συνεργαστεί με την υπηρεσία Kinesis Data Analytics.

Ροή (stream). Μια ροή είναι το βασικό κανάλι δεδομένων στο Amazon Kinesis. Λειτουργεί ως αγωγός που μας επιτρέπει να λαμβάνουμε και να αποθηκεύουμε συνεχώς εγγραφές δεδομένων σε πραγματικό χρόνο. Οι ροές αποτελούνται από ένα ή περισσότερα θραύσματα.

Θραύσμα (shard). Ένα θραύσμα είναι μια βασική μονάδα χωρητικότητας δεδομένων σε μια ροή Kinesis. Κάθε θραύσμα μπορεί να απορροφήσει εγγραφές δεδομένων με συγκεκριμένο ρυθμό και να αποθηκεύσει έναν συγκεκριμένο όγκο δεδομένων. Η χωρητικότητα ενός ρεύματος καθορίζεται από τον αριθμό των θραυσμάτων που έχει. Μπορούμε να κλιμακώσουμε τον αριθμό των θραυσμάτων προς τα πάνω ή προς τα κάτω, για να προσαρμόσουμε διαφορετικούς φόρτους εργασίας δεδομένων.

Εγγραφή (record). Μια εγγραφή δεδομένων (αλλιώς πλειάδα) είναι η μικρότερη μονάδα δεδομένων σε μια ροή Kinesis. Οι εγγραφές είναι αμετάβλητες και αποτελούνται από ένα ωφέλιμο φορτίο δεδομένων και έναν αριθμό σειράς. Το ωφέλιμο φορτίο μπορεί να είναι σε οποιαδήποτε μορφή, όπως JSON ή δυαδικό, και συνήθως αντιπροσωπεύει ένα συμβάν ή μια πληροφορία. Οι αριθμοί ακολουθίας εκχωρούνται στις εγγραφές με τη σειρά που προστίθενται στη ροή και μπορούν να χρησιμοποιηθούν για τη σειρά των εγγραφών και την κατάργηση των αντιγράφων.

Παραγωγός (producer). Ένας παραγωγός είναι μια εφαρμογή που προωθεί τις εγγραφές δεδομένων σε μια ροή Kinesis. Οι παραγωγοί μπορεί να είναι διάφορες πηγές δεδομένων, όπως συσκευές IoT, εφαρμογές ή συλλέκτες αρχείων καταγραφής. Οι παραγωγοί χρησιμοποιούν το Kinesis API για να τοποθετήσουν εγγραφές στη ροή δεδομένων.

Καταναλωτής (consumer). Ένας καταναλωτής είναι μια εφαρμογή που ανακτά και επεξεργάζεται εγγραφές δεδομένων από μια ροή Kinesis. Οι καταναλωτές διαβάζουν δεδομένα από τη ροή σε πραγματικό χρόνο ή σχεδόν σε πραγματικό χρόνο και εκτελούν ενέργειες με βάση τα δεδομένα, όπως ανάλυση, αποθήκευση ή οπτικοποίηση.

Kinesis client library (KCL). Η KCL είναι μια βιβλιοθήκη η οποία παρέχεται από την AWS και απλοποιεί την ανάπτυξη εφαρμογών. Χειρίζεται εργασίες όπως είναι ο συντονισμός θραυσμάτων, σημεία ελέγχου (check points) και η παράλληλη επεξεργασία αρχείων.

Επαναλήπτης θραύσματος (shard iterator). Ένας επαναλήπτης θραύσματος είναι ένας δείκτης σε μια συγκεκριμένη θέση μέσα σε ένα θραύσμα της ροής. Οι καταναλωτές χρησιμοποιούν επαναλήπτες θραύσματος για την ανάγνωση δεδομένων από τη ροή. Υπάρχουν διάφοροι τύποι επαναληπτών, συμπεριλαμβανομένων των TRIM_HORIZON (παλαιότερα δεδομένα) και LATEST (πιο πρόσφατα δεδομένα).

Η υπηρεσία Kinesis Data Streams υποστηρίζει τη βασική λειτουργικότητα για την επεξεργασία ροών δεδομένων. Ωστόσο, στην πράξη χρειαζόμαστε πιο δυνατά εργαλεία, με στόχο την εξόρυξη γνώσης από δεδομένα ροών. Τα δεδομένα από την υπηρεσία Kinesis Data Streams μπορούν να οδηγηθούν στην υπηρεσία Kinesis Data Analytics για την εκτέλεση πιο πολύπλοκων εργασιών, οι οποίες μπορούν να περιγραφούν χρησιμοποιώντας τη γλώσσα ερωτημάτων SQL. Ας δούμε ένα απλό παράδειγμα δημιουργίας μιας ροής δεδομένων και, στη συνέχεια, την επεξεργασία της ροής από το Kinesis Data Analytics. Αρχικά, υποθέτουμε ότι έχουμε δημιουργήσει μια ροή δεδομένων στο AWS με όνομα `MyDataStream`. Το επόμενο βήμα είναι να υλοποιήσουμε μια εφαρμογή για να τροφοδοτήσουμε με δεδομένα τη ροή. Εδώ, θα χρησιμοποιήσουμε τη γλώσσα Python και θα χρειαστούμε το αντίστοιχο AWS SDK, το Boto3 (<https://aws.amazon.com/sdk-for-python/>). Στο παράδειγμά μας, η ροή αποτελείται από πλειάδες με δύο πεδία, το πεδίο `id` και το πεδίο `year`.

```
1 import boto3
2 import json
3 import time
4
5 # Αρχικοποίηση του πελάτη Kinesis
6 kinesis_client = boto3.client('kinesis', region_name='eu-south-2')
7
8 # Δημιουργία ροής δεδομένων με απλά δεδομένα
9 myData = [
10     {"id": 1, "year": 1821},
11     {"id": 2, "year": 1986},
12     {"id": 3, "year": 2010},
13     {"id": 4, "year": 2023},
14 ]
15
16 # Διαβάζουμε τα records ένα προς ένα--
17 for record in myData:
18
19     record_data = json.dumps(record)
20
21     # Τοποθέτηση του record στη ροή
22     kinesis_client.put_record(
23         StreamName='MyDataStream',
24         Data=record_data,
```



```

25     PartitionKey=record['id']
26 )
27
28     print(f"Record {record['id']} put into the source stream.")
29
30 # Αναμονή για 1 δευτερόλεπτο
31 time.sleep(1)

```

Στη συνέχεια, πρέπει να δημιουργηθεί μια εφαρμογή Kinesis Data Analytics στο AWS, η οποία θα χρησιμοποιηθεί για να καταναλώσει τα δεδομένα της ροής και να εκτελέσει την επιθυμητή εργασία στα δεδομένα της ροής. Στην εφαρμογή αυτή θα πρέπει να δηλώσουμε τη ροή από την οποία θα γίνεται η ανάγνωση των δεδομένων. Για το παράδειγμά μας, θα υποθέσουμε ότι το όνομα "SOURCE_STREAM" αναφέρεται στη ροή MyDataStream που έχει δημιουργηθεί προηγουμένως. Είναι ευνόητο ότι, εάν θέλουμε να διαβάσουμε δεδομένα από πολλές διαφορετικές ροές, θα πρέπει να έχουμε ορίσει τις αντίστοιχες ονομασίες. Το ερώτημα SQL που θέλουμε να εκτελείται στη ροή είναι πολύ απλό. Θέλουμε να εμφανίσουμε τις εγγραφές της ροής δεδομένων για τις οποίες το πεδίο year είναι μεγαλύτερο από την τιμή 2000.

```

1     CREATE OR REPLACE STREAM "DestinationSQLStream" (
2         "id" INTEGER,
3         "year" INTEGER
4     );
5
6     CREATE OR REPLACE PUMP "DestinationSQLStream" AS
7     INSERT INTO "DestinationSQLStream"
8     SELECT STREAM "id", "year"
9     FROM "SOURCE_STREAM"
10    WHERE "year" > 2000;

```

Το Kinesis Data Analytics αναλύει συνεχώς τα δεδομένα ροής που προέρχονται από την πηγή, εφαρμόζει τα ερωτήματα SQL και, τέλος, στέλνει τα αποτελέσματα στη ροή προορισμού (ροή εξόδου). Τα δεδομένα της ροής εξόδου μπορούν να οδηγηθούν σε επιπλέον επεξεργασία, οπτικοποίηση ή ό,τι άλλο απαιτείται από την εφαρμογή.

7.3 Περαιτέρω μελέτη

Η διαχείριση και η ανάλυση ροών δεδομένων αποτελούν βασικά εργαλεία για πολλές σύγχρονες εφαρμογές. Το κεφάλαιο αυτό αποτελεί μία εισαγωγή στα συστήματα επεξεργασίας ροών δεδομένων και στον τρόπο με τον οποίο αναπτύσσουμε σχετικές εφαρμογές. Δόθηκε έμφαση σε πέντε διαφορετικά συστήματα που θεωρούνται state-of-the-art στον χώρο. Ο αναγνώστης που θέλει να εμβαθύνει στην περιοχή θα πρέπει να μελετήσει τη βιβλιογραφία και από την τεχνολογική πλευρά αλλά και από την αλγοριθμική πλευρά, έτσι ώστε να έχει μια ολοκληρωμένη γνώση για το αντικείμενο. Δύο βιβλία που θα βοηθήσουν πολύ τον αναγνώστη που δεν έχει προηγούμενη εμπειρία στον χώρο των ροών δεδομένων είναι τα [Psa17] και [ACL18]. Η αναφορά [GZ10] αποτελεί μια πολύ καλή εισαγωγή στην περιοχή των ροών δεδομένων. Αλγοριθμικά θέματα επεξεργασίας ροών δεδομένων μελετώνται στην αναφορά [GGR18].

BIBΛΙΟΓΡΑΦΙΑ

- [ACL18] Tyler Akidau, Slava Chernyak, and Reuven Lax. *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media, Inc., 1st edition, 2018.
- [BGHP18] Albert Bifet, Ricard Gavaldà, Geoffrey Holmes, and Bernhard Pfahringer. *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, Cambridge, MA, 2018.
- [BHPG11] Albert Bifet, Geoff Holmes, Bernhard Pfahringer, and Ricard Gavaldà. Detecting sentiment change in twitter streaming data. In Tom Diethe, Jose Balcazar, John Shawe-Taylor, and Cristina Tirnauca, editors, *Proceedings of the Second Workshop on Applications of Pattern Analysis*, volume 17 of *Proceedings of Machine Learning Research*, pages 5–11, CIEM, Castro Urdiales, Spain, 19–21 Oct 2011. PMLR.
- [Gar13] Nishant Garg. *Apache Kafka*. Packt Publishing, 2013.
- [GGR18] Minos Garofalakis, Johannes Gehrke, and Rajeev Rastogi. *Data Stream Management: Processing High-Speed Data Streams*. Springer Publishing Company, Incorporated, 2018.
- [GZ10] Lukasz Golab and M. Tamer Zsu. *Data Stream Management*. Morgan & Claypool Publishers, 2010.
- [Kum20] Abhishek Kumar. *Designing Event-Driven Systems with Apache Kafka*. O'Reilly Media, 2020.
- [Mut05] S. Muthukrishnan. Data streams: Algorithms and applications. *Foundations and Trends in Theoretical Computer Science*, 1(2):117–236, 2005.
- [NSP17] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: The Definitive Guide Real-Time Data and Stream Processing at Scale*. O'Reilly Media, Inc., 1st edition, 2017.
- [Psa17] A. Psaltis. *Streaming Data: Understanding the real-time pipeline*. Manning, 2017.
- [RJD19] L. Rutkowski, M. Jaworski, and P. Duda. *Stream Data Mining: Algorithms and Their Probabilistic Properties*. Studies in Big Data. Springer International Publishing, 2019.

ΚΕΦΑΛΑΙΟ 8

ΑΝΑΛΥΣΗ ΔΙΚΤΥΩΝ

Περιεχόμενα

8.1	Εισαγωγή	192
8.2	Συστήματα	194
8.2.1	Συστήματα NoSQL	194
8.2.2	Συστήματα μεγάλης κλίμακας	198
8.3	Περαιτέρω μελέτη	206
	BIBΛΙΟΓΡΑΦΙΑ	207



Σύνοψη

Τα δίκτυα αποτελούν δομές οργάνωσης οντοτήτων. Σε ένα δίκτυο υπάρχουν κόμβοι οι οποίοι συνδέονται με ακμές. Τα χαρακτηριστικά των κόμβων και των ακμών ποικίλλουν ανάλογα με τον τύπο του δικτύου και την εφαρμογή. Τα δίκτυα, πολλές φορές, αναφέρονται ως γραφήματα (graphs) που αποτελούν τη μαθηματική έννοια του όρου. Η ανάλυση δικτύων έχει τις ρίζες της στη Θεωρία Γραφημάτων (graph theory), επιστήμη που γεννήθηκε πριν από πολλά χρόνια και πατέρας της ήταν ο Euler (https://en.wikipedia.org/wiki/Leonhard_Euler). Λαμβάνοντας υπόψη τη μεγάλη χρησιμότητα των δικτύων στις σύγχρονες απαιτήσεις, στο κεφάλαιο αυτό περιγράφουμε συνοπτικά τεχνολογίες ανάλυσης δεδομένων δικτύων.

Προαπαιτούμενη γνώση

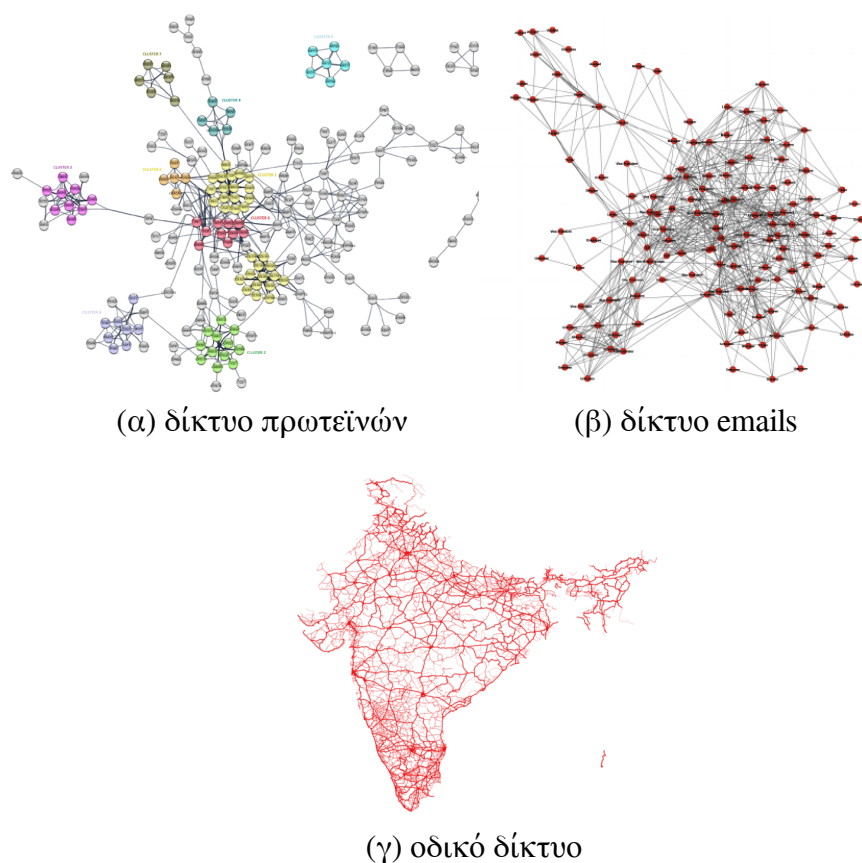
Ο αναγνώστης καλό θα είναι να έχει βασικές γνώσεις γραφημάτων, όπως αυτές διδάσκονται σε αντίστοιχα μαθήματα (π.χ., Δομές Δεδομένων, Διακριτά Μαθηματικά, Θεωρία Γραφημάτων). Ωστόσο, δεν απαιτούνται εξειδικευμένες γνώσεις για την κατανόηση του περιεχομένου του τρέχοντος κεφαλαίου.

8.1 Εισαγωγή

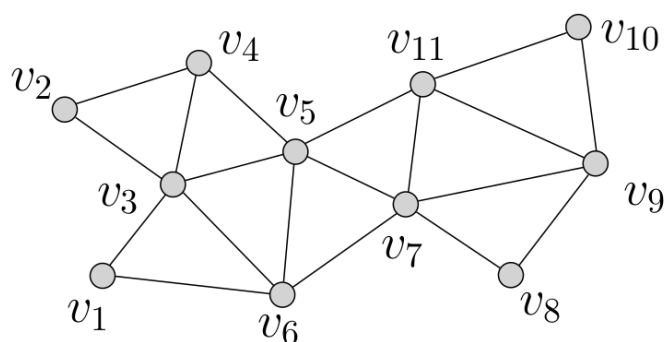
Η ανάλυση δικτύων (network analysis) αποτελεί κατά βάση υπο-περιοχή της επιστήμης δικτύων (network science). Η επιστήμη δικτύων [BP16] είναι ένα διεπιστημονικό πεδίο μελέτης που επικεντρώνεται στην ανάλυση, τη μοντελοποίηση και την κατανόηση των δικτύων. Ένα δίκτυο, σε αυτό το πλαίσιο, αναφέρεται σε μια συλλογή διασυνδεδεμένων κόμβων ή οντοτήτων, που συχνά αντιπροσωπεύονται ως κορυφές (ή κόμβοι) και ακμές (ή σύνδεσμοι) που συνδέουν τις κορυφές. Τα δίκτυα μπορούν να λάβουν διάφορες μορφές, όπως κοινωνικά δίκτυα, δίκτυα επικοινωνίας, δίκτυα μεταφορών, βιολογικά δίκτυα, κ.ά. (βλ. Σχήμα 8.1). Εξειδίκευση της επιστήμης των δικτύων αποτελεί η περιοχή της εξόρυξης γνώσης από δεδομένα γραφημάτων (graph mining) [CH06], η οποία εστιάζει σε τεχνικές ανάλυσης και εξόρυξης με στόχο την εξαγωγή χρήσιμης γνώσης.

Συνήθως, ένα γράφημα συμβολίζεται ως $G(V, E)$ ή $G=(V, E)$, όπου $V(G)$ είναι το σύνολο των κορυφών και $E(G)$ είναι το σύνολο των ακμών. Η μεταβλητή G μπορεί να παραιοφθεί εάν είναι φανερό σε ποιο γράφημα αναφερόμαστε. Στη βιβλιογραφία, το πλήθος των κορυφών (αντιστοίχως, των ακμών) ενός γραφήματος, συνήθως, συμβολίζεται με $n=|V|$ (αντιστοίχως, με $m=|E|$) και ονομάζεται τάξη (order) (αντιστοίχως, μέγεθος (size)) του γραφήματος. Στο Σχήμα 8.2 παρουσιάζεται ένα γράφημα G με τάξη $n=11$ και μέγεθος $m=19$. Το σύνολο των κορυφών είναι $V(G)=\{v_1, v_2, \dots, v_8\}$ και το σύνολο των ακμών είναι $E(G)$ και αποτελείται από τα ζεύγη κορυφών που συνδέονται, π.χ. (v_1, v_3, v_2, v_3) κ.λπ.

Τονίζουμε ότι τα γραφήματα μπορούν να έχουν διαφορετικές ιδιότητες, αναλόγως με τα δεδομένα που μοντελοποιούνται. Έτσι, οι κορυφές μπορούν να έχουν ιδιότητες (attributes), όπως επίσης οι ακμές μπορούν να έχουν τις δικές τους ιδιότητες (π.χ., κάποιο βάρος). Επίσης,



Σχήμα 8.1: Παραδείγματα δικτύων από διαφορετικές εφαρμογές: (α) δίκτυο πρωτεϊνών ([Ata20]), (β) δίκτυο emails ([CM09]), (γ) οδικό δίκτυο (https://commons.wikimedia.org/wiki/File:India_major_road_network_map.png).



Σχήμα 8.2: Παράδειγμα γραφήματος.

οι ακμές μπορεί να έχουν κατευθύνσεις (βέλη), που σημαίνει ότι η φορά παίζει σημαντικό ρόλο στην επεξεργασία των ερωτημάτων, αλλά και στον προσδιορισμό της απάντησης.

Τα τελευταία χρόνια, πολλά συστήματα για την ανάλυση γραφημάτων έχουν αναπτυχθεί για να καλύψουν τις αυξανόμενες ανάγκες, τόσο στη βιομηχανία όσο και στον ακαδημαϊκό χώρο, για τη μελέτη πολύπλοκων γραφημάτων. Η ανάλυση θα επιτρέψει στις μελλοντικές εξελίξεις τέτοιων συστημάτων να βελτιστοποιηθούν, για τη χρήση τους σε προβλήματα του πραγματικού κόσμου.

Ας δούμε μερικές αντιπροσωπευτικές αναλυτικές εργασίες που θα μπορούσαμε να εφαρμόσουμε σε ένα δίκτυο. Μία πολύ σημαντική εργασία, η οποία εφαρμόζεται σε δίκτυα που αναπαρίστανται με *κατευθυνόμενα γραφήματα*, είναι ο έλεγχος για το αν υπάρχει μονοπάτι από έναν κόμβο v προς έναν άλλο κόμβο u το οποίο να σέβεται τις κατευθύνσεις των ακμών. Τα ερωτήματα αυτά είναι γνωστά στη βιβλιογραφία ως *ερωτήματα προσβασιμότητας* (reachability queries). Μια πολύ ωραία ανασκόπηση της περιοχής μπορεί να βρεθεί στη βιβλιογραφική αναφορά [AW10].

Η *ανακάλυψη κοινοτήτων* (community discovery) αποτελεί μία άλλη οπτική γωνία της διαδικασίας της *ομαδοποίησης* (clustering), ή αλλιώς *συσταδοποίησης*, όπου ο στόχος είναι να δημιουργηθούν ομάδες οντοτήτων με βάση τις αμοιβαίες συσχετίσεις ομοιότητας. Το πρόβλημα βρίσκει πολλές εφαρμογές στον παγκόσμιο ιστό, στα κοινωνικά δίκτυα και γενικά σε τομείς όπου υπάρχει ενδιαφέρον για τον εντοπισμό ομάδων. Υπάρχει πολύ υλικό στη βιβλιογραφία σχετικά με την ανακάλυψη κοινοτήτων. Η πηγή [BP16] αποτελεί μία πολύ καλή αφετηρία.

Επίσης, η ανάλυση δεδομένων σε *χρονικά μεταβαλλόμενα δίκτυα* [ILDS16] (time-evolving networks) χαρακτηρίζεται ως πολύ σημαντική διαδικασία και με πάρα πολλές εφαρμογές, καθώς συνήθως τα δίκτυα του πραγματικού κόσμου μεταβάλλονται με τον χρόνο (είτε αργά είτε πολύ γρήγορα) και απαιτείται η ανανέωση των αποτελεσμάτων. Δεν είναι τυχαίο, ότι λόγω του μεγάλου μεγέθους των δικτύων που συναντούμε σε εφαρμογές του πραγματικού κόσμου, χρησιμοποιούνται τεχνικές επεξεργασίας που βασίζονται στις ροές δεδομένων, με στόχο την ταχύτατη επεξεργασία και τη μείωση του απαιτούμενου χώρου.

8.2 Συστήματα

Στις προηγούμενες παραγράφους περιγράψαμε ένα πολύ μικρό τμήμα των προβλημάτων που εφαρμόζονται σε δίκτυα του πραγματικού κόσμου. Στις επόμενες ενότητες θα εστιάσουμε σε συστήματα που στοχεύουν στη διαχείριση και ανάλυση δεδομένων γραφημάτων. Θα εστιάσουμε σε δύο κατηγορίες συστημάτων: σε συστήματα που ανήκουν στη σφαίρα των συστημάτων NoSQL και σε συστήματα που έχουν ως βάση πλατφόρμες επεξεργασίας μεγάλων δεδομένων.

8.2.1 Συστήματα NoSQL

Στο Κεφάλαιο 4 μελετήσαμε συστήματα βάσεων δεδομένων που χαρακτηρίζονται ως NoSQL. Μία από τις τέσσερις βασικές κατηγορίες συστημάτων NoSQL είναι οι *βάσεις δεδομένων γραφημάτων* (graph stores, graph databases). Η Neo4j (<https://neo4j.com/>) αποτελεί ένα από τα πιο διαδεδομένα συστήματα επεξεργασίας γραφημάτων. Ωστόσο, δεν είναι το μοναδικό, και στη συνέχεια θα αναφερθούμε συνοπτικά σε παρόμοια συστήματα του χώρου.

8.2.1.1 ArangoDB

Το σύστημα ArangoDB (<https://arangodb.com/>) είναι μια ευέλικτη και ισχυρή βάση δεδομένων NoSQL πολλαπλών μοντέλων, που ξεχωρίζει για την ικανότητά της να χειρίζεται

απρόσκοπτα πολλαπλά μοντέλα δεδομένων, συμπεριλαμβανομένων των εγγράφων, γραφημάτων και δεδομένων key-value, όλα μέσα σε μια μηχανή βάσης δεδομένων. Αυτή η ευελιξία καθιστά το σύστημα ArangoDB μια πολύ ελκυστική επιλογή για ένα ευρύ φάσμα εφαρμογών, από την απλή αποθήκευση εγγράφων έως τη μοντελοποίηση σύνθετων δεδομένων γραφημάτων.

Ένα από τα βασικά χαρακτηριστικά του ArangoDB είναι η γλώσσα ερωτημάτων του, η AQL (ArangoDB Query Language), η οποία έχει σχεδιαστεί για να υποστηρίζει τόσο διασχίσεις γραφημάτων όσο και σύνθετα ερωτήματα σε διάφορα μοντέλα δεδομένων. Η γλώσσα AQL επιτρέπει στους προγραμματιστές να εκφράζουν πολύπλοκες σχέσεις και χειρισμούς δεδομένων με ευκολία, καθιστώντας την ιδιαίτερα κατάλληλη για εφαρμογές που περιλαμβάνουν περίπλοκες συνδέσεις δεδομένων.

Το σύστημα ArangoDB είναι χτισμένο με βασικό γνώμονα την επεκτασιμότητα και προσφέρει επιλογές ανάπτυξης τόσο σε έναν διακομιστή όσο και σε περισσότερους. Αυτό σημαίνει ότι μπορεί να διαχειριστεί αποτελεσματικά τα αυξανόμενα σύνολα δεδομένων και τον υψηλό φόρτο ερωτημάτων, καθιστώντας το κατάλληλο στις περιπτώσεις όπου απαιτείται υψηλή απόδοση και οριζόντια κλιμάκωση. Επιπλέον, το σύστημα ArangoDB παρέχει δυνατότητες όπως την αναζήτηση πλήρους κειμένου και τη γεωχωρική ευρετηρίαση (geospatial indexing), επεκτείνοντας περαιτέρω τις δυνατότητές του.

Ένα άλλο αξιοσημείωτο χαρακτηριστικό είναι το ευέλικτο σχήμα του, το οποίο επιτρέπει στους προγραμματιστές να εξελίσσουν μοντέλα δεδομένων με την πάροδο του χρόνου. Αυτή η ευελιξία είναι ζωτικής σημασίας για εφαρμογές που απαιτούν συνεχή προσαρμογή στις μεταβαλλόμενες απαιτήσεις των δεδομένων. Το σύστημα ArangoDB περιλαμβάνει, επίσης, ισχυρούς μηχανισμούς ασφαλείας, συμπεριλαμβανομένων του ελέγχου ταυτότητας χρήστη και εξουσιοδότησης, για την προστασία τόσο των δεδομένων όσο και των πόρων.

Είτε υλοποιούμε εφαρμογές για κοινωνική δικτύωση, συστήματα συστάσεων, διαχείριση περιεχομένου ή οποιαδήποτε άλλη περίπτωση χρήσης που περιλαμβάνει διαφορετικά και διασυνδεδεμένα δεδομένα, το σύστημα ArangoDB προσφέρει μια ισχυρή και προσαρμόσιμη λύση για να καλύψει τις απαιτούμενες ανάγκες. Ο συνδυασμός της υποστήριξης πολλαπλών μοντέλων, των δυνατοτήτων αναζήτησης, της επεκτασιμότητας και της ευκολίας χρήσης του, το καθιστά μια πολύτιμη προσθήκη στον χώρο των NoSQL συστημάτων διαχείρισης βάσεων δεδομένων.

Ας δούμε ένα παράδειγμα διατύπωσης ενός ερωτήματος στο σύστημα ArangoDB με το οποίο θέλουμε στην έξοδο τους κόμβους ενός γραφήματος που βρίσκονται σε απόσταση το πολύ 2 από έναν κόμβο αφετηρία. Η απόσταση προσδιορίζεται με το μήκος του μονοπατιού για να μεταβούμε από έναν κόμβο σε έναν άλλον. Το γράφημά μας αποτελείται από κόμβους που αναπαριστούν χρήστες, και ακμές που αναπαριστούν τη σχέση φιλίας μεταξύ των χρηστών.

```
LET userId = "u42"  
LET maxDepth = 2  
  
FOR v, e, p IN 1..@maxDepth OUTBOUND @userId GRAPH "socialGraph"  
  RETURN DISTINCT p.vertices[*]._key
```


8.2.1.2 JanusGraph

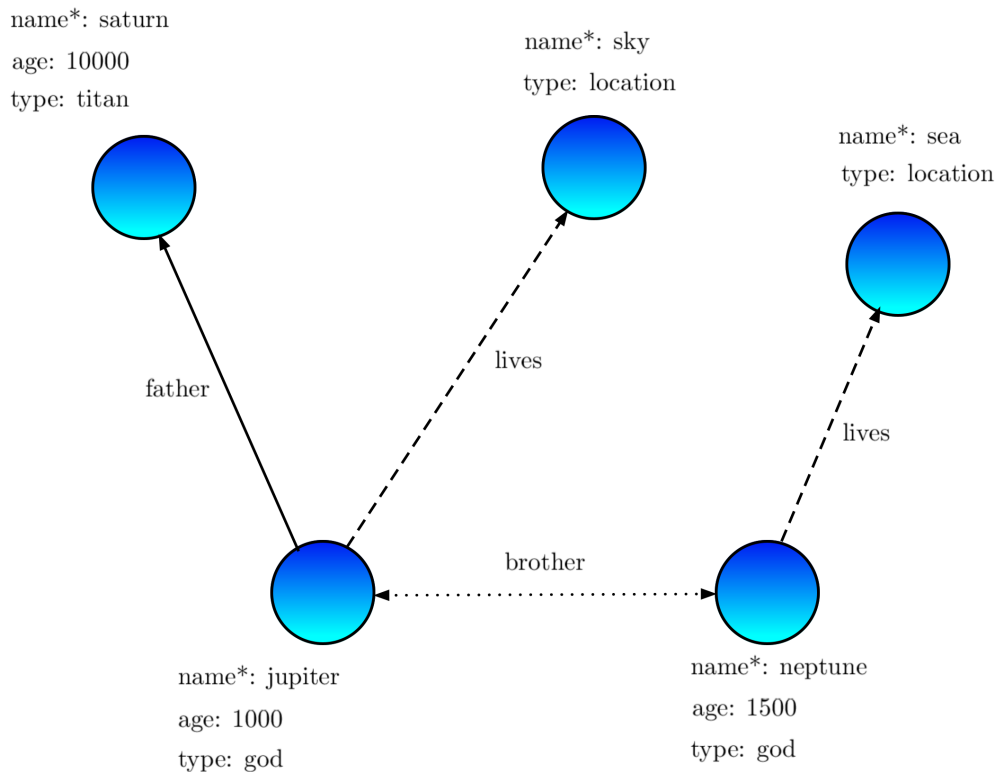
Το σύστημα JanusGraph (<https://janusgraph.org/>) είναι ένα κλιμακούμενο σύστημα βάσεων δεδομένων γραφημάτων, με δυνατότητες βελτιστοποίησης για την αποθήκευση και την αναζήτηση γραφημάτων που περιέχουν εκατοντάδες δισεκατομμύρια κορυφές και ακμές κατανεμημένες σε μία συστάδα πολλών μηχανών. Το σύστημα JanusGraph έχει τις ρίζες του σε ένα προηγούμενο σύστημα με όνομα TitanDB (<https://espeed.github.io/titandb/>), τη βάση δεδομένων κατανεμημένων γραφημάτων που κυκλοφόρησε αρχικά το 2012, για να επιτρέπει στους χρήστες να βρίσκουν συνδέσεις μεταξύ μεγάλων συνόλων δεδομένων που αποτελούνται από δισεκατομμύρια κορυφές και ακμές. Το σύστημα TitanDB αναπτύχθηκε αρχικά από μια εταιρεία που ονομάζεται Aurelius, η οποία εξαγοράστηκε το 2015 από την DataStax, την εταιρεία που βρίσκεται πίσω από το σύστημα Apache Cassandra.

Ένα από τα ξεχωριστά χαρακτηριστικά του συστήματος JanusGraph είναι η ικανότητά του να κλιμακώνεται οριζόντια, καθιστώντας το μια κατάλληλη επιλογή για τη διαχείριση συνεχώς διευρυνόμενων συνόλων δεδομένων γραφημάτων. Υπερέχει σε κατανεμημένα περιβάλλοντα, διασφαλίζοντας ανοχή στα σφάλματα και υψηλή διαθεσιμότητα, χαρακτηριστικά που είναι ζωτικής σημασίας για κρίσιμες εφαρμογές. Το JanusGraph παρέχει μεγάλη ευελιξία μέσω της υποστήριξης διαφορετικών συστημάτων αποθήκευσης (backend storage), συμπεριλαμβανομένων των Apache Cassandra, Apache HBase, BerkeleyDB, ScyllaDB. Αυτή η ευελιξία δίνει στους χρήστες τη δυνατότητα να επιλέγουν τη λύση αποθήκευσης που ταιριάζει καλύτερα με τις απαιτήσεις των εκάστοτε εφαρμογών, ενώ διασφαλίζεται ότι η λειτουργικότητα που προσφέρεται δεν εξαρτάται από το εκάστοτε σύστημα αποθήκευσης που χρησιμοποιείται. Επίσης, υποστηρίζει τη μηχανή Elasticsearch, η οποία είναι μια ευέλικτη και ισχυρή κατανεμημένη μηχανή αναζήτησης και ανάλυσης ανοικτού κώδικα (<https://www.elastic.co/elasticsearch/>).

Ένα επίσης πολύ σημαντικό χαρακτηριστικό του συστήματος JanusGraph είναι ότι υποστηρίζει τη γλώσσα ερωτημάτων Gremlin (<https://tinkerpop.apache.org/gremlin.html>) και, επομένως, μπορεί να χρησιμοποιηθεί στο πλαίσιο του Apache Tinkerpop (<https://tinkerpop.apache.org/>), το οποίο είναι ένα σύστημα επεξεργασίας γραφημάτων που μπορεί να χρησιμοποιηθεί τόσο για τις βασικές λειτουργίες διαχείρισης γραφημάτων (OLTP) όσο και για πολύπλοκα ερωτήματα ανάλυσης (OLAP). Η υποστήριξη που προσφέρει το σύστημα JanusGraph για τη γλώσσα γραφημάτων Gremlin διευκολύνει τη διατύπωση ερωτημάτων που αφορούν πολύπλοκες διασχίσεις (traversals), και την ανάλυση δεδομένων σε πραγματικό χρόνο, καθιστώντας το JanusGraph κατάλληλο για εφαρμογές που απαιτούν γρήγορη επεξεργασία σε μεγάλα σύνολα δεδομένων.

Στη συνέχεια, παραθέτουμε ένα παράδειγμα χρήσης του συστήματος JanusGraph χρησιμοποιώντας τη γλώσσα Gremlin. Αρχικά, δημιουργείται ένα γράφημα που αναφέρεται σε όντα της ελληνικής μυθολογίας και στις μεταξύ τους συσχετίσεις. Το γράφημα αυτό χρησιμοποιείται και ως παράδειγμα στα έγγραφα τεκμηρίωσης του συστήματος JanusGraph, και παρουσιάζεται στο Σχήμα 8.3. Αρχικά, δημιουργείται ένα γράφημα, στη συνέχεια εισάγουμε μερικές κορυφές και ακμές και, ακολούθως, εκτελούμε μερικά απλά ερωτήματα.

Θα υποθέσουμε ότι το σύστημα αποθήκευσης που θα χρησιμοποιήσουμε είναι η Cassandra, οπότε θα προσδιορίσουμε CQL (Cassandra Query Language) για τη δημιουργία του αντι-



Σχήμα 8.3: Τμήμα του γραφήματος της μυθολογίας (πηγή <https://docs.janusgraph.org/getting-started/basic-usage/>).

κειμένου `graph`. Επίσης, θέλουμε να χρησιμοποιήσουμε `ElasticSearch` ως σύστημα αναζήτησης. Επομένως, στη συνάρτηση `JanusGraphFactory.open()` περνούμε ως παράμετρο `'conf/janusgraph-cql-es.properties'`. Στη συνέχεια, χρησιμοποιούμε το αντικείμενο `GraphOfTheGodsFactory` για να φορτώσουμε το γράφημα. Ο κώδικας της συνάρτησης `GraphOfTheGodsFactory.load()` βρίσκεται στον σύνδεσμο <https://github.com/JanusGraph/janusgraph/blob/master/janusgraph-core/src/main/java/org/janusgraph/example/>.

Οι ετικέτες που υποστηρίζονται από τον κώδικα `GraphOfTheGodsFactory` είναι «titan», «location», «god», «demigod», «human», «monster», ενώ οι ετικέτες για τις ακμές είναι «father», «mother», «battled», «battlesByTime», «lives», «pet», «brother». Φυσικά, υπάρχει η δυνατότητα να δημιουργήσουμε νέες ετικέτες ανάλογα με τις ανάγκες μας.

```

// Ορισμός γραφήματος
graph = JanusGraphFactory.open('conf/janusgraph-cql-es.properties')
GraphOfTheGodsFactory.load(graph)

// Ορισμός διασχίσης
myTraversal = graph.traversal()

// Προσθήκη μερικών κόμβων
newGod = myTraversal.addV('god').property('name', 'newGod').next()
newHuman = myTraversal.addV('human').property('name', 'newHuman').next()
newTitan = myTraversal.addV('tital').property('name', 'newTitan').next()
  
```

```
// Προσθήκη μερικών ακμών
graph.V(newGod).addE('battled').to(newHuman).next()
graph.V(newGod).addE('battled').to(newTitan).next()

// Εύρεση των ονομάτων των όντων που πολέμησε ο θεός newGod
monstersBattled = graph.V(newGod).out('battled').values('name')

// Εκτύπωση αποτελεσμάτων
monstersBattled.each { println it }
```

8.2.2 Συστήματα μεγάλης κλίμακας

Τα συστήματα που θα μελετήσουμε στην ενότητα αυτή χαρακτηρίζονται ως συστήματα μεγάλης κλίμακας, διότι βασίζονται *εκ γενετής* σε κατανεμημένες πλατφόρμες επεξεργασίας δεδομένων μεγάλης κλίμακας. Ο διαχωρισμός αυτός δεν συνεπάγεται ότι τα συστήματα που μελετήσαμε προηγουμένως δεν μπορούν να εφαρμοστούν σε δεδομένα μεγάλης κλίμακας. Ωστόσο, τα συστήματα παρουσιάζονται διαφορετικά, για να τονιστεί η μεγάλη συγγένεια των συστημάτων αυτών με πλατφόρμες όπως Hadoop, Spark και AWS.

8.2.2.1 Apache Giraph

Το σύστημα Apache Giraph (<https://giraph.apache.org/>) είναι μια μηχανή επεξεργασίας γραφημάτων ανοικτού κώδικα, σχεδιασμένο για να λειτουργεί σε κατανεμημένα υπολογιστικά περιβάλλοντα, ιδιαίτερα σε συστάδες Hadoop. Ο πρωταρχικός σκοπός του είναι να αναλύει και να χειρίζεται αποτελεσματικά δεδομένα δομημένων γραφημάτων μεγάλης κλίμακας. Το Giraph βασίζεται στο μοντέλο Bulk Synchronous Parallel (BSP), στο οποίο ο υπολογισμός πραγματοποιείται σε συγχρονισμένα βήματα, επιτρέποντας τη συντονισμένη επεξεργασία μεταξύ των κατανεμημένων κόμβων. Αυτό το μοντέλο είναι κατάλληλο για αλγορίθμους γραφημάτων, καθώς απλοποιεί τον συγχρονισμό και την επικοινωνία σε σύνθετες εργασίες επεξεργασίας.

Ένα από τα βασικά χαρακτηριστικά του Giraph είναι το μοντέλο προγραμματισμού με επίκεντρο την κορυφή (vertex-centric). Οι προγραμματιστές εκφράζουν αλγορίθμους γραφημάτων ορίζοντας τη λογική υπολογισμού στο επίπεδο των μεμονωμένων κορυφών και των ακμών που συνδέονται με αυτές. Το Giraph υποστηρίζει ένα ευρύ φάσμα αλγορίθμων γραφημάτων οι οποίοι είναι υλοποιημένοι σύμφωνα με το μοντέλο MapReduce, όπως PageRank, συνεκτικές συνιστώσες (connected components), συντομότερα μονοπάτια (shortest paths), καθιστώντας το μια ευέλικτη επιλογή για διάφορες εργασίες ανάλυσης γραφημάτων. Επιπλέον, προσφέρει μηχανισμούς ανοχής σε σφάλματα για τον χειρισμό αστοχιών στους υπολογιστικούς κόμβους, διασφαλίζοντας στιβαρότητα σε κατανεμημένα περιβάλλοντα μεγάλης κλίμακας. Το Giraph ενσωματώνεται ομοιόμορφα στο οικοσύστημα του Hadoop, καθώς λειτουργεί συνεργατικά με άλλα υποσυστήματα, όπως το σύστημα αρχείων HDFS και τον διαχειριστή συστάδας YARN, ενώ βελτιώνεται συνεχώς από μια πολύ ενεργή κοινότητα ανοικτού κώδικα. Αποτελεί πολύτιμο σύστημα για οργανισμούς που εστιάζουν σε σύνθετες εργασίες ανάλυσης δεδομένων γραφημάτων διαφορετικών τύπων.

Το Giraph υποστηρίζει κυρίως τη γλώσσα προγραμματισμού Java. Το μεγαλύτερο μέρος του βασικού κώδικα (codebase) και των εφαρμογών του Giraph επικεντρώνεται στην Java. Οι προγραμματιστές χρησιμοποιούν τη γλώσσα Java για να γράψουν προσαρμοσμένες κλάσεις για τις κορυφές του γραφήματος και τα μηνύματα που ανταλλάσσονται και να υλοποιήσουν αλγόριθμους γραφημάτων.

Ωστόσο, είναι σημαντικό να σημειωθεί ότι, ενώ η Java είναι η κύρια γλώσσα για το Apache Giraph, το οικοσύστημα Hadoop στο οποίο λειτουργεί το Giraph υποστηρίζει διάφορες γλώσσες για την επεξεργασία μεγάλων δεδομένων. Για παράδειγμα, μπορούμε να χρησιμοποιήσουμε γλώσσες όπως Python, R, κ.ά. μέσω του Hadoop Streaming ή ενσωματώνοντας βιβλιοθήκες όπως την PyGiraph ή την Hama για Python και άλλες γλώσσες. Αυτά τα εργαλεία επιτρέπουν την υλοποίηση τμήματος του κώδικα σε γλώσσες εκτός της Java.

```
import org.apache.giraph.aggregators.DoubleSumAggregator;
import org.apache.giraph.aggregators.TextAppendAggregator;
import org.apache.giraph.edge.Edge;
import org.apache.giraph.graph.BasicComputation;
import org.apache.giraph.graph.Vertex;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;

public class PageRankComputation extends BasicComputation<LongWritable,
    DoubleWritable, DoubleWritable, Text> {

    @Override
    public void compute(Vertex<LongWritable, DoubleWritable, DoubleWritable>
        vertex, Iterable<Text> messages) {
        if (getSuperstep() == 0) {
            // Initialize all vertices with PageRank 1.0 at the beginning
            vertex.setValue(new DoubleWritable(1.0 / getTotalNumVertices()));
        } else {
            double sum = 0.0;
            for (Text message : messages) {
                sum += Double.parseDouble(message.toString());
            }
            // Υπολογισμός PageRank για την κορυφή
            double newPageRank = 0.15 / getTotalNumVertices() + 0.85 * sum;
            vertex.setValue(new DoubleWritable(newPageRank));
        }

        // Αποστολή της νέας τιμής PageRank στους γείτονες
        if (getSuperstep() < 20) {
            sendMessageToAllEdges(new Text(vertex.getId().toString() + ":"
                + vertex.getValue()));
        } else {
            vertex.voteToHalt();
        }
    }
}
```

Εδώ, θα δώσουμε ένα παράδειγμα χρήσης του Giraph που στοχεύει στην υλοποίηση μιας

απλής εκδοχής του αλγορίθμου PageRank [PBMW98], ο οποίος χρησιμοποιείται για την ποσοτικοποίηση της σημαντικότητας των κόμβων ενός γραφήματος. Στη μεταβλητή `newPageRank` αποθηκεύεται η νέα τιμή του PageRank για μία κορυφή και υπολογίζεται με βάση τις τιμές PageRank των γειτόνων της κορυφής. Ο επαναληπτικός αυτός αλγόριθμος εκτελείται για 20 επαναλήψεις. Η πιθανότητα κατά τη διάρκεια του τυχαίου περιπάτου να επιλέξουμε έναν γειτονικό κόμβο έχει τιμή 0,85 ενώ η πιθανότητα τηλεμεταφοράς (teleporting) έχει τιμή 0,15. Στο τέλος των επαναλήψεων, η κάθε κορυφή περιέχει την τελική τιμή του PageRank που της αντιστοιχεί.

8.2.2.2 Apache Spark

Το σύστημα Apache Spark έχει μελετηθεί σε προηγούμενα κεφάλαια. Το συναντάμε και στην ενότητα αυτή, εξαιτίας της σημαντικής λειτουργικότητας που παρέχει στην ανάλυση δικτύων, μέσω των βιβλιοθηκών που υποστηρίζει για την ανάλυση γραφημάτων. Οι βιβλιοθήκες αυτές είναι οι: GraphX (<https://spark.apache.org/graphx/>) και GraphFrames (https://graphframes.github.io/graphframes/docs/_site/index.html). Τονίζουμε ότι η βιβλιοθήκη GraphX βασίζεται στο RDD API και είναι μία από τις επίσημες βιβλιοθήκες του Spark, ενώ η βιβλιοθήκη GraphFrames βασίζεται στο DataFrame API και δεν υποστηρίζεται επίσημα από το Spark. Επίσης, αξίζει να αναφερθεί ότι, εάν χρησιμοποιείται η Python για την ανάπτυξη εφαρμογών, τότε θα πρέπει οπωσδήποτε να χρησιμοποιηθεί η βιβλιοθήκη GraphFrames, διότι η GraphX δεν υποστηρίζει τη γλώσσα προγραμματισμού Python.

Οι βιβλιοθήκες GraphX και GraphFrames επιτρέπουν τη δημιουργία, διαχείριση και ανάλυση γραφημάτων με κατανεμημένο τρόπο, σε περιβάλλον συστάδας. Οι κατανεμημένοι αλγόριθμοι επιτρέπουν την επεξεργασία πολύ μεγάλων γραφημάτων, που θα ήταν αδύνατο να χειριστούμε σε μία μόνο μηχανή επεξεργασίας, λόγω του μεγάλου όγκου δεδομένων αλλά και της πολυπλοκότητας των εργασιών που πρέπει να εκτελεστούν σε αυτά. Στους αλγόριθμους που είναι υλοποιημένοι συμπεριλαμβάνονται οι PageRank, συνεκτικές συνιστώσες, εύρεση τριγώνων, προώθηση ετικέτας (label propagation), συντομότερα μονοπάτια, Breadth-First Search (BFS), κ.ά.

Στη συνέχεια, παραθέτουμε ένα παράδειγμα χρήσης της βιβλιοθήκης GraphFrames. Θα χρησιμοποιήσουμε την έκδοση 0.8.1. Τα δεδομένα αφορούν μετακινήσεις με ποδήλατο και το γράφημα αποτελείται από κορυφές που δηλώνουν τις στάσεις όπου μπορούν να βρεθούν τα ποδήλατα, ενώ μία ακμή μεταξύ δύο κορυφών δηλώνει τη μετακίνηση ενός ή περισσότερων ποδηλάτων από τη μία κορυφή (αφετηρία) στην άλλη (προορισμός). Περισσότερες λεπτομέρειες για το συγκεκριμένο σύνολο δεδομένων μπορούν να βρεθούν στο βιβλίο [CZ18] και συγκεκριμένα στο Κεφάλαιο 30, ενώ τα σχετικά δεδομένα είναι διαθέσιμα στον σύνδεσμο <https://github.com/databricks/Spark-The-Definitive-Guide/>. Επίσης, υποθέτουμε ότι η εκτέλεση πραγματοποιείται εντός του `spark-shell`, όπου έχουμε τις μεταβλητές `sc` για το `spark context` και `spark`, για το `spark session`. Για την έναρξη του `spark shell` έτσι ώστε να υποστηρίζεται η βιβλιοθήκη GraphFrames, μπορούμε να χρησιμοποιήσουμε την ακόλουθη εντολή:

```
spark-shell --packages graphframes:graphframes:0.8.1-spark3.0-s_2.12
```

Αρχικά, διαβάζουμε τα δύο αρχεία `csv` έτσι ώστε να δημιουργηθούν δύο DataFrames, ένα

για τις κορυφές του γραφήματος και ένα για τις ακμές. Στη συνέχεια, με τη συνάρτηση `printSchema` παρουσιάζονται οι στήλες για το κάθε `DataFrame` και, τέλος, εμφανίζουμε μερικές γραμμές και στήλες από το κάθε ένα.

```
// Ορισμός αρχείων
val stations_file_location = "201508_station_data.csv"
val trip_file_location = "201508_trip_data.csv"

// Δημιουργία DataFrames για κορυφές και ακμές
val bikeStations = spark.read.format("csv")
    .option("inferSchema", "true")
    .option("header", "true")
    .option("delimiter", ",")
    .load(stations_file_location)

val tripData = spark.read.format("csv")
    .option("inferSchema", "true")
    .option("header", "true")
    .option("delimiter", ",")
    .load(trip_file_location)

bikeStations.printSchema()
bikeStations.select("station_id", "name", "lat", "long").show(10)

tripData.printSchema()
tripData.select("Trip ID", "Start Station", "End Station", "Duration").show
(10)
```

root

```
|-- station_id: integer (nullable = true)
|-- name: string (nullable = true)
|-- lat: double (nullable = true)
|-- long: double (nullable = true)
|-- dockcount: integer (nullable = true)
|-- landmark: string (nullable = true)
|-- installation: string (nullable = true)
```

```
+-----+-----+-----+-----+
|station_id|          name|          lat|          long|
+-----+-----+-----+-----+
|          2|San Jose Diridon ...|37.329732|-121.901782|
|          3|San Jose Civic Ce...|37.330698|-121.888979|
|          4|Santa Clara at Al...|37.333988|-121.894902|
|          5|    Adobe on Almaden|37.331415|  -121.8932|
|          6|    San Pedro Square|37.336721|-121.894074|
|          7|Paseo de San Antonio|37.333798|-121.886943|
|          8| San Salvador at 1st|37.330165|-121.885831|
|          9|          Japantown|37.348742|-121.894715|
```

```
|          10|   San Jose City Hall|37.337391|-121.886995|
|          11|           MLK Library|37.335885| -121.88566|
+-----+-----+-----+-----+
```

```
root
```

```
|-- Trip ID: integer (nullable = true)
|-- Duration: integer (nullable = true)
|-- Start Date: string (nullable = true)
|-- Start Station: string (nullable = true)
|-- Start Terminal: integer (nullable = true)
|-- End Date: string (nullable = true)
|-- End Station: string (nullable = true)
|-- End Terminal: integer (nullable = true)
|-- Bike #: integer (nullable = true)
|-- Subscriber Type: string (nullable = true)
|-- Zip Code: string (nullable = true)
```

```
+-----+-----+-----+-----+
|Trip ID|      Start Station|      End Station|Duration|
+-----+-----+-----+-----+
| 913460|Harry Bridges Pla...|San Francisco Cal...|    765|
| 913459|San Antonio Shopp...|Mountain View Cit...|   1036|
| 913455|      Post at Kearny|    2nd at South Park|    307|
| 913454|   San Jose City Hall| San Salvador at 1st|    409|
| 913453|Embarcadero at Fo...|Embarcadero at Sa...|    789|
| 913452|Yerba Buena Cente...|San Francisco Cal...|    293|
| 913451|Embarcadero at Fo...|Embarcadero at Sa...|    896|
| 913450|Embarcadero at Sa...|  Steuart at Market|    255|
| 913449|      Beale at Market|Temporary Transba...|    126|
| 913448|      Post at Kearny|South Van Ness at...|    932|
+-----+-----+-----+-----+
```

Για λόγους ευκολίας, προχωράμε στη μετονομασία κάποιων στηλών των DataFrames και δημιουργούμε δύο νέα DataFrames, stationVertices και tripEdges.

```
val stationVertices = bikeStations.withColumnRenamed("name", "id").
distinct()

val tripEdges = tripData.withColumnRenamed("Start Station", "src")
                        .withColumnRenamed("End Station", "dst")
```

Από τη στιγμή που έχουμε στη διάθεσή μας ένα DataFrame για τις κορυφές και ένα DataFrame για τις ακμές του γραφήματος, μπορούμε να κατασκευάσουμε το γράφημα χρησιμοποιώντας τη βιβλιοθήκη GraphFrames ως εξής:


```
import org.graphframes._
val graph = GraphFrame(stationVertices, tripEdges)
```

Στη συνέχεια, θα δούμε πώς μπορούμε να εφαρμόσουμε αλγορίθμους ανάλυσης γραφημάτων με τις δυνατότητες που μας παρέχει η βιβλιοθήκη GraphFrames. Αρχικά, ενδιαφερόμαστε να υπολογίσουμε τα τρίγωνα στο γράφημα, και συγκεκριμένα σε πόσα τρίγωνα συμμετέχει η κάθε κορυφή. Το πλήθος των τριγώνων στα οποία συμμετέχει μία κορυφή u δηλώνει το πόσο καλά συνδεδεμένοι είναι οι γείτονες της u . Το πλήθος των τριγώνων αποθηκεύεται στη στήλη `trcount` του `DataFrame triangles`.

```
val triangles = graph.triangleCount.run().withColumnRenamed("count", "trcount")
triangles.select("id", "station_id", "trcount").show(10)
```

```
+-----+-----+-----+
|              id|station_id|trcount|
+-----+-----+-----+
|      2nd at Folsom|        62|    496|
|California Ave Ca...|        36|     51|
|Washington at Kea...|        46|      0|
|Powell at Post (U...|        71|    496|
|Golden Gate at Polk|        59|    496|
|Yerba Buena Cente...|        68|    496|
|Market at Sansome|        77|    496|
|      MLK Library|        11|    108|
|Spear at Folsom|        49|    496|
|      Japantown|         9|    108|
+-----+-----+-----+
```

Εάν ενδιαφερόμαστε για τις δέκα κορυφές με το μεγαλύτερο πλήθος τριγώνων, μπορούμε να εκτελέσουμε τον ακόλουθο κώδικα:

```
triangles.select("id", "trcount")
          .orderBy(desc("trcount"))
          .show(10)
```

```
+-----+-----+
|              id|trcount|
+-----+-----+
|San Francisco Cal...|    500|
|Market at 4th|    498|
|Townsend at 7th|    496|
|2nd at Folsom|    496|
|Spear at Folsom|    496|
|Civic Center BART...|    496|
|Commercial at Mon...|    496|
|Powell at Post (U...|    496|
```



```
|Yerba Buena Cente...|      496|
| Golden Gate at Polk|      496|
+-----+-----+
```

Ας δούμε τώρα πώς θα μπορούσαμε να εκτελέσουμε τον αλγόριθμο PageRank στο προηγούμενο γράφημα. Υπάρχουν δύο βασικοί τρόποι εκτέλεσης του αλγορίθμου PageRank. Ο πρώτος τρόπος προσδιορίζει επακριβώς το πλήθος των επαναλήψεων. Στο παράδειγμά μας, χρησιμοποιούμε 10 επαναλήψεις (`maxIter(10)`). Είναι προφανές ότι το τελικό αποτέλεσμα μπορεί να μην είναι το ίδιο με αυτό που θα λαμβάναμε εάν αφήναμε τον αλγόριθμο να συγκλίνει. Ωστόσο, ενδέχεται το πλήθος των επαναλήψεων που θα απαιτηθούν μέχρι τη σύγκλιση να είναι μεγάλο, με αποτέλεσμα να αυξηθεί σημαντικά ο χρόνος εκτέλεσης, κάτι που είναι πιο πιθανό να συμβεί σε πολύ μεγάλα γραφήματα. Ο δεύτερος τρόπος εκτέλεσης θέτει μία ανοχή, στο παράδειγμά μας 0,01 (`tol(0.01)`). Η ανοχή αυτή χρησιμοποιείται ως εξής: εάν μετά από μία εκτέλεση επανάληψης οι νέες τιμές που έχουν προκύψει διαφέρουν από τις προηγούμενες λιγότερο από 0,01, τότε ο αλγόριθμος τερματίζεται.

```
val ranks = graph.pageRank
    .resetProbability(0.15).maxIter(10)
    .run()

ranks.vertices.orderBy(desc("pagerank")).select("id", "pagerank").show(false)

val ranks2 = graph.pageRank
    .resetProbability(0.15).tol(0.01)
    .run()

ranks2.vertices.orderBy(desc("pagerank")).select("id", "pagerank").show(false)
```

Τα αποτελέσματα των δύο εκτελέσεων δίνονται παρακάτω, και παρατηρούμε ότι έχουν μικρές διαφορές. Γενικά, είναι πιο ασφαλές να χρησιμοποιείται η μέθοδος εκτέλεσης με την ανοχή, έτσι ώστε το πλήθος των επαναλήψεων να ρυθμίζεται αυτόματα.

```
+-----+-----+
|id                                             |pagerank      |
+-----+-----+
|San Jose Diridon Caltrain Station            |4.051504835990006 |
|San Francisco Caltrain (Townsend at 4th)     |3.351183296428596 |
|Mountain View Caltrain Station              |2.51439077101558  |
|Redwood City Caltrain Station                |2.3263087713711887|
|San Francisco Caltrain 2 (330 Townsend)      |2.231144291369861 |
|Harry Bridges Plaza (Ferry Building)        |1.8251120118882567|
|2nd at Townsend                             |1.5821217785039052|
|Santa Clara at Almaden                      |1.5730074084907735|
|Townsend at 7th                             |1.5684565805340842|
|Embarcadero at Sansome                     |1.5414242087748846|
|Steuart at Market                          |1.4659451265543435|
```

Palo Alto Caltrain Station	1.418905165291439
Market at Sansome	1.3854248857995
San Pedro Square	1.378338977470068
Temporary Transbay Terminal (Howard at Beale)	1.3677942011839381
University and Emerson	1.3362687304968772
Mountain View City Hall	1.2145886778331225
Stanford in Redwood City	1.1704516264869989
San Antonio Shopping Center	1.1316004674331854
Market at 10th	1.1020452302927386

id	pagerank
San Jose Diridon Caltrain Station	4.0008391706249045
San Francisco Caltrain (Townsend at 4th)	3.3104342428401057
Mountain View Caltrain Station	2.546931345140747
Redwood City Caltrain Station	2.254919435863379
San Francisco Caltrain 2 (330 Townsend)	2.2084537589424436
Harry Bridges Plaza (Ferry Building)	1.8136826356918814
2nd at Townsend	1.5746756702712958
Santa Clara at Almaden	1.5617318333925763
Townsend at 7th	1.556585045048147
Embarcadero at Sansome	1.5327611522476383
Steuart at Market	1.460066213227452
Palo Alto Caltrain Station	1.416057945511353
Market at Sansome	1.3758451244891643
San Pedro Square	1.369134920777329
Temporary Transbay Terminal (Howard at Beale)	1.3601243507175402
University and Emerson	1.3396293698364254
Mountain View City Hall	1.2301326309160583
San Antonio Shopping Center	1.1485688398403715
Stanford in Redwood City	1.145549532049726
San Antonio Caltrain Station	1.109189041630095

Θα ολοκληρώσουμε τα παραδείγματα ανάλυσης γραφημάτων με το Spark, με την εκτέλεση ενός αλγορίθμου εύρεσης κοινοτήτων (community detection) που βασίζεται στην προώθηση ετικέτας (label propagation). Ο στόχος είναι, μετά από έναν αριθμό επαναλήψεων, οι κορυφές του γραφήματος που ανήκουν στην ίδια κοινότητα να έχουν την ίδια ετικέτα. Ο αλγόριθμος παράγει στην έξοδο ένα DataFrame (εδώ καλείται `com`) το οποίο περιέχει μία επιπλέον στήλη με όνομα `label`, η οποία περιέχει ακέραιους αριθμούς που δηλώνουν την κοινότητα στην οποία ανήκει η κάθε κορυφή (κορυφές με την ίδια ετικέτα ανήκουν στην ίδια κοινότητα).

```
val com = graph.labelPropagation.maxIter(5).run()
com.printSchema()
com.select("id", "label").show(10)
```

```
root
|-- station_id: integer (nullable = true)
|-- id: string (nullable = true)
|-- lat: double (nullable = true)
|-- long: double (nullable = true)
|-- dockcount: integer (nullable = true)
|-- landmark: string (nullable = true)
|-- installation: string (nullable = true)
|-- label: long (nullable = true)
```

```
+-----+-----+
|          id|          label|
+-----+-----+
|      2nd at Folsom|1657857376256|
|      Market at 10th|1657857376256|
|California Ave Ca...| 429496729600|
|Washington at Kea...| 17179869184|
|Redwood City Publ...| 730144440320|
|Powell at Post (U...|1657857376256|
|Golden Gate at Polk|1657857376256|
|Adobe on Almaden| 292057776128|
|Broadway St at Ba...|1657857376256|
|Yerba Buena Cente...|1657857376256|
+-----+-----+
```

8.3 Περαιτέρω μελέτη

Τα συστήματα διαχείρισης και ανάλυσης γραφημάτων χρησιμοποιούνται ευρύτατα από τις σύγχρονες εφαρμογές, κυρίως λόγω της έντονης παρουσίας των γραφημάτων σε προβλήματα του πραγματικού κόσμου. Στο κεφάλαιο αυτό, εξετάσαμε μερικά μόνο από τα διαθέσιμα συστήματα. Ωστόσο, υπάρχουν πολλές εναλλακτικές λύσεις, ανάλογα με τις περιπτώσεις εφαρμογής για τις οποίες ενδιαφερόμαστε. Για παράδειγμα, η περίπτωση χρήσης μπορεί να αφορά γράφημα ιδιοτήτων (property graph), γραφήματα RDF ή και τα δύο. Επομένως, η επιλογή του κατάλληλου συστήματος θα πρέπει να γίνεται με προσοχή. Επίσης, πολλές φορές, χρησιμοποιούνται περισσότερα του ενός συστήματα για να καλύψουν τις ανάγκες.

Ο αναγνώστης που επιθυμεί να εμβαθύνει στην περιοχή αυτή μπορεί να μελετήσει τα βιβλία [RWE15], [BP20] και [Bad22], καθώς και τα άρθρα [AG08] και [BC20] για μια πιο ολοκληρωμένη γνώση στο αντικείμενο. Επίσης, αξίζει να διερευνηθεί και η χρήση συστημάτων βάσεων δεδομένων γραφημάτων, όπως:

- Cayley (<https://cayley.io/>),
- OrientDB (<http://orientdb.org/>),
- Dgraph (<https://dgraph.io/>),
- AllegroGraph(<https://allegrograph.com/>),
- HugeGraph(<https://hugegraph.apache.org/>),
- TigerGraph (<https://www.tigergraph.com/>),
- TerminusDB (<https://terminusdb.com/>).

Επίσης, αξίζει να σημειωθεί ότι στο πλαίσιο του AWS υπάρχει η υπηρεσία Amazon Neptune (<https://aws.amazon.com/neptune/>), η οποία είναι μια πλήρως διαχειριζόμενη υπηρεσία βάσεων δεδομένων γραφημάτων που παρέχεται από την πλατφόρμα Amazon Web Services (AWS). Το Neptune υποστηρίζει τόσο το μοντέλο γραφήματος ιδιοτήτων (property graph) όσο και μοντέλα RDF. Επιπλέον, το Amazon Neptune ενσωματώνεται απρόσκοπτα σε άλλες υπηρεσίες AWS, όπως το Amazon S3, το AWS Identity and Access Management (IAM) και το Amazon CloudWatch, για να παρέχει βελτιωμένη λειτουργικότητα και παρακολούθηση. Επίσης, το Neptune μπορεί να αξιοποιηθεί σε συνδυασμό με την υπηρεσία AWS Lambda, η οποία προσφέρει τη δυνατότητα να ενεργοποιηθούν λειτουργίες χωρίς διακομιστή (serverless) ως απόκριση σε συμβάντα βάσης δεδομένων γραφημάτων, προσφέροντας δυνατότητες επεξεργασίας γραφημάτων σε πραγματικό χρόνο.

BIBΛΙΟΓΡΑΦΙΑ

- [AG08] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Comput. Surv.*, 40(1), 2008.
- [Ata20] Sevcan Atay. Integrated transcriptome meta-analysis of pancreatic ductal adenocarcinoma and matched adjacent pancreatic tissues. *PeerJ*, 8, 10 2020.
- [AW10] Charu C. Aggarwal and Haixun Wang. *Managing and Mining Graph Data*. Springer Publishing Company, Incorporated, 1st edition, 2010.
- [Bad22] D.A. Bader. *Massive Graph Analytics*. Chapman & Hall/CRC Data Science Series. CRC Press, 2022.
- [BC20] Amitabha Bhattacharyya and Durgapada Chakravarty. (graph database: A survey). In *2020 International Conference on Computer, Electrical and Communication Engineering (ICCECE)*, pages 1–8, 2020.
- [BP16] A.L. Barabási and M.Á. Pálfai. *Network Science*. Cambridge University Press, 2016.
- [BP20] D. Bechberger and J. Perryman. *Graph Databases in Action*. In Action. Manning Publications, 2020.

- [CH06] Diane J. Cook and Lawrence B. Holder. *Mining Graph Data*. John Wiley & Sons, 2006.
- [CM09] Ben Collingsworth and Ronaldo Menezes. *Identification of Social Tension in Organizational Networks*, pages 209–223. 04 2009.
- [CZ18] Bill Chambers and Matei Zaharia. *Spark: The definitive guide: Big data processing made simple*. O'Reilly Media, Inc., 2018.
- [ILDS16] Anand Padmanabha Iyer, Li Erran Li, Tathagata Das, and Ion Stoica. Time-evolving graph processing at scale. In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems*, GRADES '16, New York, NY, USA, 2016. Association for Computing Machinery.
- [PBMW98] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical report, Stanford Digital Library Technologies Project, 1998.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. ” O'Reilly Media, Inc.”, 2015.

ΔΙΑΧΕΙΡΙΣΗ ΠΟΡΩΝ ΚΑΙ ΕΚΤΕΛΕΣΗ ΕΦΑΡΜΟΓΩΝ

Περιεχόμενα

9.1	Εισαγωγή	210
9.2	Γενικά για τη διαχείριση πόρων	210
9.3	Hadoop YARN	211
9.3.1	Οντότητες του YARN	211
9.3.2	Λειτουργία του YARN	213
9.3.3	Χρονοπρογραμματισμός στο YARN	215
9.3.4	Βασική διαχείριση του YARN	215
9.3.5	Βασική διαχείριση κόμβων	216
9.3.6	Βασική διαχείριση υπηρεσιών	218
9.3.7	Βασική διαχείριση εφαρμογών	218
9.4	Apache Mesos	220
9.4.1	Οντότητες του Mesos	220
9.4.2	Λειτουργία του Mesos	222
9.5	Kubernetes	224
9.5.1	Έννοιες του Kubernetes	225
9.5.2	Workloads	225
9.5.3	Άκατοι	225
9.5.4	Κόμβοι	227
9.5.5	Συστατικά στοιχεία του Kubernetes	227
9.5.6	Εκτέλεση εφαρμογής στο Kubernetes	229
9.6	Spark standalone cluster manager	230
9.6.1	Οντότητες του Apache Spark	230
9.6.2	Λειτουργία του διαχειριστή συστάδας του Spark	231
9.7	Περαιτέρω μελέτη	232
	BIBΛΙΟΓΡΑΦΙΑ	233

Καρακασίδης, Α., Κολωνιάρη, Γ., Γούναρης, Α., & Παπαδόπουλος, Α. (2025). Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων [Μεταπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις. <http://dx.doi.org/10.57713/kallipos-1058>

Σύνοψη

Το κεφάλαιο αυτό ασχολείται με τα εξής θέματα:

- Παρουσίαση του Apache YARN, μέρους του Apache Hadoop, με παραδείγματα εντολών διαχείρισης.
- Παρουσίαση του συστήματος Apache Mesos.
- Παρουσίαση του συστήματος Kubernetes.
- Παρουσίαση του αυτόνομου διαχειριστή cluster του Apache Spark.

Προαπαιτούμενη γνώση

Για να μπορέσει ο αναγνώστης να κατανοήσει τους όρους και τις έννοιες που περιγράφονται σε αυτό το κεφάλαιο, θα πρέπει να κατέχει βασικές γνώσεις Αρχιτεκτονικής Υπολογιστών, Λειτουργικών Συστημάτων και Δικτύων. Επίσης, θα ήταν χρήσιμο να έχει διαβάσει τα σχετικά κεφάλαια του παρόντος βιβλίου για το μοντέλο MapReduce και το Apache Spark. Επιπλέον, για να εκτελέσει τις εντολές του YARN που περιγράφονται, θα πρέπει να είναι εξοικειωμένος με το λειτουργικό σύστημα Linux και να υπάρχει εγκατάσταση του HDFS.

9.1 Εισαγωγή

Για την ολοκλήρωση μεγάλων έργων εργάζονται πολλοί άνθρωποι. Είναι, φυσικά, αδύνατο όλοι αυτοί να μπορέσουν να επιτύχουν το επιθυμητό αποτέλεσμα χωρίς την ύπαρξη κάποιου να συντονίζει τις ενέργειές τους, ενός *συντονιστή*. Η κατάσταση είναι αντίστοιχη και σε ένα παράλληλο ή σε ένα καταναμημένο σύστημα, όπου εκτελούνται πολλαπλές διεργασίες με στόχο την ολοκλήρωση ενός σκοπού. Χρειάζονται, λοιπόν, μηχανισμοί για τον συντονισμό και την ολοκλήρωση των επιμέρους ενεργειών ενός έργου. Στο κεφάλαιο αυτό, θα μελετήσουμε μηχανισμούς που έχουν αυτόν ακριβώς τον σκοπό: να παρέχουν συντονισμό στις εκτελούμενες διεργασίες κατανέμοντας πόρους όπως είναι η μνήμη, ο δίσκος, η χωρητικότητα δικτύου και, φυσικά, οι διαθέσιμοι επεξεργαστές. Αρχικά θα μελετήσουμε μηχανισμούς που εμπεριέχονται στα συστήματα που έχουμε συναντήσει έως τώρα, το YARN, που αποτελεί μέρος του Apache Hadoop. Έπειτα, θα εστιάσουμε σε αυτόνομα συστήματα για τη διαχείριση πόρων. Θα μελετήσουμε το Apache Mesos και στη συνέχεια θα συνεχίσουμε με το Kubernetes, το οποίο εστιάζει στη διαχείριση φόρτου εφαρμογών που εκτελούνται σε περιέκτες (containers). Τέλος, θα παρουσιάσουμε τον μηχανισμό του Apache Spark για την εκτέλεση εργασιών.

9.2 Γενικά για τη διαχείριση πόρων

Κάθε υπολογιστής διαθέτει συγκεκριμένους πόρους. Ανεξάρτητα από το πλήθος των CPUs, την ποσότητα της κύριας μνήμης ή το μέγεθος του δίσκου, οι πόροι αυτοί είναι συγκεκριμέ-

νοι και δεν μπορούν να αλλάξουν χωρίς φυσική παρέμβαση. Από την άλλη μεριά, το πλήθος των εφαρμογών που μπορεί να εκτελούνται σε έναν υπολογιστή μπορεί να μεταβληθεί πολύ ευκολότερα με βάση τις ανάγκες. Έτσι, σε μία συστάδα μπορεί να καταστεί αναγκαία η εκτέλεση περισσότερων εφαρμογών από αυτές για τις οποίες υπήρξε αρχικός σχεδιασμός. Το ζήτημα που προκύπτει αφορά τους πόρους που μπορούμε ή επιθυμούμε να διαθέσουμε σε μια εφαρμογή. Για να μπορέσουμε να το πραγματοποιήσουμε αυτό, ένας βασικός τρόπος είναι να διαχωρίσουμε τις εφαρμογές σε διαφορετικούς υπολογιστές. Κάτι τέτοιο, όμως, έχει ως αποτέλεσμα, σύμφωνα και με αυτά που περιγράψαμε πιο πάνω, τη στατική διάθεση των πόρων, με αποτέλεσμα όχι μόνο τη μείωση της ευελιξίας και τη μη ορθή κατανομή αυτών, αλλά και τη δυσκολία διαχείρισης πολλαπλών διακομιστών.

Μία πρώτη λύση σε αυτά τα προβλήματα προέκυψε με τους μηχανισμούς των εικονικών μηχανών. Ένας μηχανισμός εικονικών μηχανών εκτελείται ως εφαρμογή στο λειτουργικό σύστημα του υπολογιστή και με τη σειρά του εκτελεί *εικόνες (images)* λειτουργικών συστημάτων. Η εφαρμογή αυτή ονομάζεται *υπερεπόπτης (hypervisor)*. Έτσι έχουμε εικονικούς υπολογιστές που εκτελούνται ως λογισμικό. Αυτό δίνει τη δυνατότητα να διαθέσουμε συγκεκριμένους πόρους σε κάθε μία εικονική μηχανή κατά το δοκούν. Επιπλέον, αυτή η διάταξη επιτρέπει την ανεξαρτησία από το υλικό, καθώς υπάρχει η δυνατότητα μεταφοράς των εικονικών μηχανών σε διαφορετικούς ή ισχυρότερους φυσικούς υπολογιστές.

Εξέλιξη του μηχανισμού των εικονικών μηχανών αποτελούν οι *περιέκτες (containers)*. Η λογική είναι παρόμοια, υπό την έννοια ότι κάθε περιέκτης είναι απομονωμένος από τον άλλο, του κατανέμονται οι πόροι που μπορεί να χρησιμοποιήσει και είναι μεταφέρσιμος. Επιπλέον, τους περιέκτες διαχειρίζεται η αντίστοιχη εφαρμογή περιεκτών, όπως και ο υπερεπόπτης διαχειρίζεται τις εικονικές μηχανές.

Η κύρια διαφορά έγκειται στο γεγονός ότι, αντί να εκτελείται ένα ολόκληρο λειτουργικό σύστημα και μέσα σε αυτό να εκτελούνται οι εφαρμογές, όπως συμβαίνει με τις εικονικές μηχανές, ο περιέκτης περιλαμβάνει σε μία εικόνα (*container image*) όλα εκείνα τα απαραίτητα στοιχεία για την ορθή εκτέλεση της εφαρμογής: βιβλιοθήκες, αρχικές τιμές, αλλά και πρόσθετο λογισμικό. Οι περιέκτες είναι αμετάβλητοι και δεν διατηρούν κατάσταση. Όλοι οι περιέκτες έχουν πρόσβαση στο ίδιο λειτουργικό σύστημα. Έτσι, ένας περιέκτης είναι λιγότερο απαιτητικός σε πόρους από μια εικονική μηχανή.

9.3 Hadoop YARN

Το πρώτο σύστημα που θα μελετήσουμε είναι το YARN. Το YARN δεν είναι αυτόνομο σύστημα, αλλά αποτελεί μέρος του Hadoop και επιτελεί δύο ρόλους. Ο πρώτος είναι η διαχείριση πόρων της συστάδας, ενώ ο δεύτερος είναι ο προγραμματισμός και η παρακολούθηση των εργασιών που εκτελούνται στη συστάδα. Για να ανταποκριθεί στους ρόλους αυτούς, το YARN περιλαμβάνει μία σειρά από οντότητες, τις οποίες θα περιγράψουμε παρακάτω.

9.3.1 Οντότητες του YARN

Οι οντότητες που απαρτίζουν το YARN είναι κατά βάση τέσσερις: ο *ResourceManager*, ο *ApplicationMaster*, ο *NodeManager* και οι *περιέκτες (Containers)*. Ας δούμε τον ρόλο που επιτελεί κάθε μία από τις οντότητες αυτές.

9.3.1.1 ResourceManager

Καθώς, όπως είπαμε και πιο πριν, σε μια συστάδα μπορεί να συνυπάρχουν και να εκτελούνται ταυτόχρονα περισσότερες από μία εφαρμογές, απαιτείται ένας μηχανισμός για τη διαχείρισή τους. Στο YARN αυτός ο μηχανισμός ονομάζεται *διαχειριστής πόρων (ResourceManager)* που είναι ο καθολικός και μόνος υπεύθυνος για την κατανομή των πόρων μεταξύ όλων των εφαρμογών του συστήματος. Αντικειμενικός σκοπός του είναι όλοι οι πόροι της συστάδας να βρίσκονται διαρκώς σε χρήση.

Για την επίτευξη του σκοπού αυτού, ο ResourceManager αποτελείται από δύο μηχανισμούς. Ο πρώτος είναι ο *χρονοπρογραμματιστής*, ενώ ο δεύτερος είναι ο *διαχειριστής εφαρμογών (ApplicationManager)* που διαχειρίζεται τις εργασίες χρήστη στη συστάδα. Καθώς ο ResourceManager αποτελεί βασικό μηχανισμό του YARN, γίνεται σαφές ότι πιθανή κατάρρευσή του θα καταστήσει το YARN μη λειτουργικό. Για τον λόγο αυτόν, υπάρχει η δυνατότητα διαμόρφωσης του ResourceManager σε διάταξη υψηλής διαθεσιμότητας (High Availability) (High Availability - HA).

Για την υλοποίηση αυτής της διάταξης, χρησιμοποιείται ένας τουλάχιστον επιπλέον κόμβος ως *αναμένων ResourceManager (Standby ResourceManager)* και μία συστάδα Zookeeper. Ο ενεργός (Active) ResourceManager ενημερώνει τακτικά το Zookeeper με την κατάστασή του. Αν αποτύχει, το Zookeeper εκλέγει νέο ενεργό ResourceManager από τους αναμένοντες. Ο νέος ενεργός ενημερώνεται για την κατάσταση του προηγούμενου ενεργού από το Zookeeper. Όταν υπάρχει μια τέτοια διάταξη, τα αντίστοιχα στοιχεία ρυθμίσεων του YARN θα πρέπει να περιλαμβάνουν μια λίστα με τους πιθανούς ResourceManagers, ώστε οι διάφορες οντότητες να μπορούν να εντοπίζουν ποιος από αυτούς είναι ενεργός.

Ας μελετήσουμε τώρα τους δύο επιμέρους μηχανισμούς του ResourceManager, ξεκινώντας από τον χρονοπρογραμματιστή.

Χρονοπρογραμματιστής. Ο χρονοπρογραμματιστής (scheduler) είναι υπεύθυνος για την κατανομή πόρων στις διάφορες εκτελούμενες εφαρμογές. Για να το επιτύχει αυτό, υλοποιεί διάφορους αλγόριθμους κατανομής πόρων, όπως αλγόριθμο εστιασμένο στη δίκαιη κατανομή πόρων. Για να διανείμει τους πόρους, χρησιμοποιεί την αφηρημένη έννοια του περιέκτη πόρων, που ενσωματώνει τύπους πόρων όπως μνήμη, CPU, δίσκο και δίκτυο. Έτσι, πραγματοποιείται ανάθεση περιεκτών στις εφαρμογές, κάθε ένας από τους οποίους περιλαμβάνει τους απαιτούμενους πόρους.

Διαχειριστής εφαρμογών. Ο διαχειριστής εφαρμογών (ApplicationManager) είναι υπεύθυνος για την αποδοχή των αιτημάτων υποβολής εργασιών. Επίσης, διαπραγματεύεται τον περιέκτη για την εκτέλεση του ApplicationMaster και την επανεκκίνηση του ApplicationMaster σε περίπτωση αποτυχίας.

9.3.1.2 NodeManager

Ο *διαχειριστής κόμβου (NodeManager)* αποτελεί την οντότητα που εκτελείται σε κάθε κόμβο-εργάτη και είναι υπεύθυνος για την εκκίνηση των περιεκτών εφαρμογών στους κόμβους-

εργάτες. Έτσι, σε μία συστάδα όπου εκτελείται το YARN, εκτελούνται τόσοι Nodemanagers όσοι και οι κόμβοι-εργάτες. Μία από τις εργασίες του είναι η παρακολούθηση της χρήσης των πόρων του κόμβου (CPU, μνήμη, δίσκος, δίκτυο) που εκτελείται, ώστε να τα αναφέρει στον ResourceManager. Όταν εκκινείται, ο NodeManager εγγράφεται στον ResourceManager και, στη συνέχεια, του στέλνει παλμούς με την κατάστασή του και περιμένει οδηγίες. Επίσης, μπορεί να τερματιστεί εάν του ζητηθεί από τον ResourceManager.

9.3.1.3 ApplicationMaster

Ο *master εφαρμογής* (*ApplicationMaster*) αποτελεί μια οντότητα η οποία είναι συγκεκριμένη ανά πλατφόρμα που μπορεί μέσω της υλοποίησής της να συνεργαστεί με το YARN. Είναι υπεύθυνος για την παρακολούθηση των εργασιών μιας εφαρμογής και μπορεί να διαχειρίζεται ομάδα εφαρμογών. Ο ρόλος του είναι να συνεργάζεται τόσο με τον ResourceManager όσο και με τους NodeManagers. Με τον πρώτο διαπραγματεύεται πόρους με τη μορφή περιεκτών, ενώ με τους δεύτερους επικοινωνεί ώστε να πραγματοποιήσει την εκτέλεση και την παρακολούθηση εργασιών.

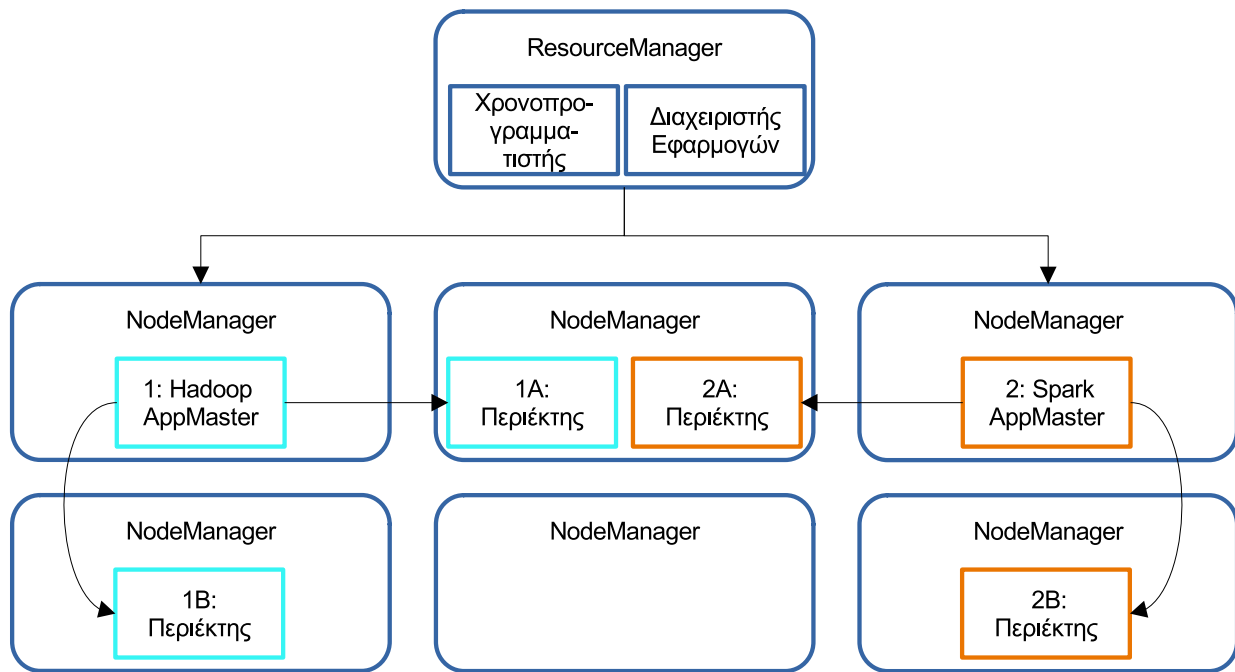
Είναι λογικό να δημιουργείται η απορία για τους λόγους που οδήγησαν σε αυτήν την αρχιτεκτονική επιλογή, δηλαδή ο ApplicationMaster να υλοποιείται ανά framework και να μην είναι μια κεντρικοποιημένη οντότητα [ME14]. Ο πρώτος λόγος είναι η δυνατότητα κλιμάκωσης, καθώς η ύπαρξη διαφορετικής οντότητας ανά εφαρμογή για την εκτέλεση και παρακολούθησή της μπορεί να επιτρέψει την εκτέλεση πολλών εφαρμογών, χωρίς την ύπαρξη ενός μοναδικού σημείου συμφόρησης. Ο δεύτερος λόγος είναι η δυνατότητα επεκτασιμότητας, διότι η προσέγγιση αυτή δίνει τη δυνατότητα να υποστηριχθούν πολλά διαφορετικά συστήματα, καθώς το κάθε ένα θα υλοποιεί τον δικό του ApplicationMaster για τις εφαρμογές του. Πέρα από τα πλεονεκτήματα αυτά, υπάρχει και ένα λεπτό σημείο το οποίο χρήζει προσοχής. Καθώς ο ApplicationMaster υλοποιείται ανά πλατφόρμα, αποτελεί εξωτερικό κώδικα, με συνέπεια να μην είναι έμπιστη εφαρμογή. Για τον λόγο αυτόν, τόσο ο ResourceManager όσο και οι NodeManagers διαθέτουν μηχανισμούς προστασίας από τους ApplicationMasters.

9.3.2 Λειτουργία του YARN

Αφού περιγράψαμε τις βασικές οντότητες που απαρτίζουν το YARN, ας δούμε πώς αυτές συμμετέχουν στη λειτουργία του. Θα περιγράψουμε, αρχικά, πώς πραγματοποιείται η διαχείριση των πόρων και, έπειτα, τον τρόπο χρονοπρογραμματισμού των εργασιών.

9.3.2.1 Διαχείριση πόρων στο YARN

Το μοντέλο πόρων στο οποίο βασίζεται το YARN έχει ως κεντρική έννοια τον *περιέκτη* (Container). Ένας περιέκτης είναι το δικαίωμα σε μια εφαρμογή για χρήση συγκεκριμένων πόρων (π.χ. μνήμη, CPU) σε έναν συγκεκριμένο υπολογιστή [ME14] και περιλαμβάνει κάποιους ελάχιστους πόρους (π.χ. 2 GB μνήμης και 2 CPUs). Έτσι, κάθε κόμβος μπορεί να θεωρηθεί ότι περιλαμβάνει πλέον του ενός περιέκτες, οι οποίοι μπορεί να εκτελούν διαφορετικές εφαρμογές στον ίδιο κόμβο.



Σχήμα 9.1: Παράδειγμα YARN: υποβολή εφαρμογών στον ResourceManager.

Μια εφαρμογή ξεκινά με τη δημιουργία του ApplicationMaster, για την εκτέλεση του οποίου χρειάζεται επίσης ένας περιέκτης. Μετά την επιτυχή εκκίνησή του, ο ApplicationMaster διαπραγματεύεται με τον ResourceManager για επιπλέον περιέκτες ώστε να εκτελεστεί η εφαρμογή του, κάτι το οποίο μπορεί να γίνεται και δυναμικά κατά τον χρόνο εκτέλεσης. Μια εφαρμογή (μέσω του ApplicationMaster) μπορεί να ζητήσει πόρους με πολύ συγκεκριμένες απαιτήσεις [ME14], όπως όνομα πόρου (συμπεριλαμβανομένου του ονόματος διακομιστή), ποσότητα μνήμης, πλήθος CPUs, κ.ά. Ο χρονοπρογραμματιστής ανταποκρίνεται σε ένα αίτημα πόρων εκχωρώντας ένα κοντέινερ, το οποίο ικανοποιεί τις απαιτήσεις που ορίζονται από τον ApplicationMaster.

Για να κάνουμε πιο κατανοητό αυτόν τον μηχανισμό, θα χρησιμοποιήσουμε το παράδειγμα που απεικονίζεται στο Σχήμα 9.1. Εδώ βλέπουμε μία συστάδα η οποία αποτελείται από ένα σύνολο διακομιστών, οι οποίοι απεικονίζονται με το μπλε περίγραμμα. Έστω ότι έχουμε δύο εφαρμογές οι οποίες επιθυμούν να εκτελεστούν στη συστάδα. Η μία είναι εφαρμογή του Hadoop MapReduce, με το κυανό χρώμα, ενώ η άλλη, με το πορτοκαλί χρώμα, είναι εφαρμογή του Spark.

Αρχικά, ο ResourceManager εκχωρεί έναν περιέκτη για την εκκίνηση του ApplicationMaster ανά εφαρμογή που πρόκειται να εκτελεστεί: έναν περιέκτη για τον ApplicationMaster του Hadoop και έναν για του Spark. Κατά την εκκίνηση του ApplicationMaster, ο ResourceManager καταχωρίζει την εφαρμογή στον ApplicationManager. Αφού εκκινηθεί, ο κάθε ApplicationMaster διαπραγματεύεται με τον ResourceManager για τους κατάλληλους πόρους, σύμφωνα με την απαίτηση της εφαρμογής του. Έπειτα, μετά την κατανομή πόρων από τον ResourceManager, ο ApplicationMaster ζητά από τον κάθε NodeManager να εκκινήσει τον περιέκτη που έχει εκχωρηθεί από τον ResourceManager. Με την επιτυχή εκκίνηση των περιεκτών, η εφαρμογή εκτελείται εντός του περιέκτη και ο ApplicationManager αναφέρει στον ResourceManager

την κατάσταση της εφαρμογής. Κατά την ολοκλήρωση της εφαρμογής, ο ApplicationMaster ενημερώνει τον ResourceManager για την ολοκλήρωση, ώστε να τερματίσει και τον δικό του περιέκτη.

9.3.3 Χρονοπρογραμματισμός στο YARN

Το YARN διαθέτει τρεις μηχανισμούς για τον χρονοπρογραμματισμό των εργασιών [ME14]. Αυτοί είναι ο FIFO (First in - First Out), ο μηχανισμός χωρητικότητας (capacity) και ο δίκαιος (fair). Ας δούμε κάποιες λεπτομέρειες για κάθε έναν από αυτούς.

9.3.3.1 Ο μηχανισμός χρονοπρογραμματισμού FIFO

Ο μηχανισμός χρονοπρογραμματισμού FIFO είναι ο πιο απλοϊκός από τους τρεις μηχανισμούς. Κατά τη χρήση του, δεν υπάρχει προτεραιότητα και οι εργασίες που υποβάλλονται για εκτέλεση ανακτώνται από ουρά και εκτελούνται με τη σειρά που προσέρχονται. Ο μηχανισμός αυτός είναι κατάλληλος για μικρά φορτία, αλλά μπορεί να προκαλέσει προβλήματα διαθεσιμότητας σε μεγάλες συστάδες.

9.3.3.2 Ο μηχανισμός χρονοπρογραμματισμού χωρητικότητας

Ο δεύτερος μηχανισμός χρονοπρογραμματισμού που θα μελετήσουμε είναι ο μηχανισμός χωρητικότητας. Ο μηχανισμός αυτός κατανέμει τους διαθέσιμους πόρους στις τρέχουσες εφαρμογές λαμβάνοντας υπόψη τις ανάγκες και απαιτήσεις της κάθε μίας. Οι εφαρμογές μπορούν να χρησιμοποιήσουν διαθέσιμους πόρους που δεν χρησιμοποιούνται από καμία άλλη εφαρμογή. Υπάρχουν πολλαπλές ουρές προτεραιότητας και σε κάθε μία εκχωρείται μία χωρητικότητα με την έννοια ότι μια συγκεκριμένη χωρητικότητα πόρων θα είναι στη διάθεση των εφαρμογών. Όλες οι εργασίες που υποβάλλονται σε μια ουρά θα έχουν πρόσβαση στους πόρους που έχουν διατεθεί στην ουρά αυτή.

9.3.3.3 Ο δίκαιος μηχανισμός χρονοπρογραμματισμού

Ο τρίτος χρονοπρογραμματιστής του YARN είναι ο δίκαιος. Όλες οι εφαρμογές λαμβάνουν, κατά μέσο όρο, ίσο μερίδιο πόρων με την πάροδο του χρόνου. Εάν εκτελείται μία μεμονωμένη εργασία, θα λάβει όλους τους διαθέσιμους πόρους στη συστάδα. Καθώς, όμως, αυξάνεται ο αριθμός των εργασιών, θα παρέχονται πόροι στις εργασίες, έτσι ώστε κάθε χρήστης να έχει ένα δίκαιο μερίδιο της συστάδας. Η δικαιοσύνη ορίζεται ως προς τον χρήστη. Έτσι, ένας χρήστης που υποβάλλει περισσότερες εργασίες από έναν άλλο χρήστη δεν λαμβάνει περισσότερους πόρους.

9.3.4 Βασική διαχείριση του YARN

Αφού είδαμε τους μηχανισμούς λειτουργίας του YARN και εξοικειωθήκαμε με τις έννοιες και τις οντότητές του, μπορούμε να περιγράψουμε κάποιες βασικές εντολές διαχείρισής του. Για να μπορέσουμε να διαχειριστούμε το YARN, θα υποθέσουμε ότι αυτό έχει εγκατασταθεί

σωστά, στο πλαίσιο της εγκατάστασης μιας συστάδας Hadoop, και πως έχουν οριστεί οι μεταβλητές περιβάλλοντος με τέτοιον τρόπο που να επιτρέπουν την εκκίνηση του YARN από οποιαδήποτε διαδρομή¹.

9.3.5 Βασική διαχείριση κόμβων

Θα ξεκινήσουμε παρουσιάζοντας εντολές για τη βασική διαχείριση κόμβων στον YARN.

Πληροφορίες κόμβων του YARN Για να εκκινήσουμε το YARN, αρκεί να εκτελέσουμε την εντολή που παρατίθεται παρακάτω.

```
start-yarn.sh
```

Για να προχωρήσουμε με τις εντολές διαχείρισης του YARN, θα θεωρήσουμε ότι και το αντίστοιχο εκτελέσιμο μπορεί να εκκινηθεί από οποιαδήποτε διαδρομή². Μπορούμε να δούμε όλες τις διαθέσιμες εντολές απλώς εκτελώντας:

```
yarn
```

Για λόγους επίδειξης θα θεωρήσουμε το Παράδειγμα 9.1.

Παράδειγμα 9.1. Έστω ότι έχουμε μία στοιχειώδη συστάδα του YARN σε ψευδοκατανεμημένη διαμόρφωση. Αυτό σημαίνει ότι η συστάδα αποτελείται από έναν μοναδικό κόμβο ο οποίος εκτελεί και τον ResourceManager και τον NodeManager. Μια τέτοια διάταξη μπορεί να χρησιμοποιηθεί για εκπαιδευτικούς σκοπούς ή για δοκιμές.

Μπορούμε να ελέγξουμε τους κόμβους που είναι ενεργοί στη συστάδα που έχουμε διαθέσιμη εκτελώντας:

```
yarn node -list
```

Με την εκτέλεση της παραπάνω εντολής, εμφανίζονται όλοι οι ενεργοί κόμβοι στη συστάδα, καθώς και άλλες πληροφορίες, όπως το id τους και το πλήθος των περιεκτών που εκτελούνται αυτήν τη στιγμή. Οι πληροφορίες αυτές προέρχονται έπειτα από επικοινωνία με τον ResourceManager. Υπάρχει, όμως, το ενδεχόμενο να μην εμφανίζονται όλοι οι κόμβοι της συστάδας μας με αυτήν την εντολή. Αυτό μπορεί να συμβαίνει στην περίπτωση που κάποιος κόμβος δεν είναι ενεργός ή αντιμετωπίζει κάποιο πρόβλημα.

Κάτι τέτοιο μπορούμε να δούμε στο Σχήμα 9.2. Εδώ, ενώ υπάρχει ένας κόμβος στην ψευδοκατανεμημένη διάταξή μας, αυτός, για κάποιον λόγο, δεν εμφανίζεται στη λίστα των ενεργών κόμβων. Το αποτέλεσμα της εκτέλεσης της εντολής απεικονίζεται στο Σχήμα 9.2.

Για να εμφανίσουμε όλους τους κόμβους στη συστάδα, ανεξαρτήτως της κατάστασής τους, εκτελούμε το παρακάτω:

```
yarn node -list -all
```

¹Θεωρώντας ότι έχουμε μεταφορτώσει μια πλήρη έκδοση του Hadoop την οποία έχουμε αποσυμπίεσει, ώστε ο βασικός κατάλογος να ονομάζεται `hadoop`, το σενάριο εκκίνησης (και τερματισμού, αντίστοιχα) του YARN βρίσκεται στη διαδρομή `hadoop/sbin/`

²Κατ' αντιστοιχία με πριν, το εκτελέσιμο διαχείρισης του yarn βρίσκεται στη διαδρομή `hadoop/bin/`

```
2023-09-28 20:12:37,778 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at master/127.0.0.1:8032
Total Nodes:0
Node-Id Node-State Node-Http-Address Number-of-Running-Containers
```

Σχήμα 9.2: Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager, ο οποίος όμως δεν είναι σε κατάσταση RUNNING.

Εκτελώντας την παραπάνω εντολή, μπορούμε να παρατηρήσουμε ότι αυτήν τη φορά όντως εμφανίζεται ο κόμβος, όπως μπορούμε να δούμε στο Σχήμα 9.3, σε κατάσταση UNHEALTHY.

```
2023-09-28 20:17:12,564 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at master/127.0.0.1:8032
Total Nodes:1
Node-Id Node-State Node-Http-Address Number-of-Running-Containers
bigdata-virtualbox:33973 UNHEALTHY bigdata-virtualbox:8042 0
```

Σχήμα 9.3: Παράδειγμα YARN: Αποτέλεσμα list -all σε συστάδα με έναν NodeManager, ο οποίος είναι σε κατάσταση UNHEALTHY.

Ένας κόμβος περιέρχεται σε κατάσταση UNHEALTHY όταν ο χρησιμοποιούμενος χώρος του δίσκου στον διακομιστή στον οποίο βρίσκεται ξεπεράσει το 90%³. Εκτός από UNHEALTHY και RUNNING, ένας κόμβος μπορεί να είναι σε μία από τις εξής καταστάσεις: NEW, UNHEALTHY, DECOMMISSIONED, LOST, REBOOTED, DECOMMISSIONING, SHUT-DOWN. Μπορούμε να επαναφέρουμε τον κόμβο σε κατάσταση RUNNING απελευθερώνοντας χώρο στον δίσκο.

Στο Σχήμα 9.4 εμφανίζεται το αποτέλεσμα της list με τον NodeManager σε κατάσταση RUNNING, μετά την επαναφορά του ελεύθερου χώρου στον δίσκο.

```
2023-09-28 23:12:42,215 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at master/127.0.0.1:8032
Total Nodes:1
Node-Id Node-State Node-Http-Address Number-of-Running-Containers
bigdata-virtualbox:44283 RUNNING bigdata-virtualbox:8042 0
```

Σχήμα 9.4: Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager, ο οποίος είναι σε κατάσταση UNHEALTHY.

Επιπλέον, χρησιμοποιώντας την επιλογή -showDetails, όπως φαίνεται παρακάτω, μπορούμε να δούμε λεπτομερείς πληροφορίες για έναν κόμβο και τους πόρους που χρησιμοποιούνται, αλλά και αυτούς που είναι διαθέσιμοι.

```
yarn node -list -showDetails
```

Στο Σχήμα 9.5 εμφανίζεται το αποτέλεσμα της list με τον NodeManager σε κατάσταση RUNNING, με χρήση της επιλογής showDetails. Βλέπουμε ότι ο συγκεκριμένος NodeManager διαθέτει 8 GB μνήμης και 8 πυρήνες. Επίσης, βλέπουμε ότι δεν χρησιμοποιούνται πόροι για την εκτέλεση περιεκτών. □

Εκκίνηση και τερματισμός μεμονωμένων κόμβων. Συνεχίζοντας, ας δούμε με ποιον τρόπο μπορούμε να τερματίσουμε κάποιον μεμονωμένο κόμβο του YARN σε ένα σύστημα το οποίο ήδη εκτελείται. Με την παρακάτω εντολή μπορούμε να τερματίσουμε τον NodeManager στον κόμβο όπου θα εκτελεστεί η εντολή αυτή:

³Πρόκειται για παράμετρο του YARN η οποία μπορεί να τροποποιηθεί.

Node-Id	Node-State	Node-Http-Address	Number-of-Running-Containers
bigdata-virtualbox:44283	RUNNING	bigdata-virtualbox:8042	0
Detailed Node Information :			
Configured Resources : <memory:8192, vCores:8>			
Allocated Resources : <memory:0, vCores:0>			
Resource Utilization by Node : PMem:1696 MB, VMem:1835 MB, VCores:1.1996001			
Resource Utilization by Containers : PMem:0 MB, VMem:0 MB, VCores:0.0			
Node-Labels :			

Σχήμα 9.5: Παράδειγμα YARN: αποτέλεσμα list σε συστάδα με έναν NodeManager και επιλογή -showDetails.

```
yarn --daemon stop nodemanager
```

Και με την επόμενη εντολή μπορούμε, αντίστοιχα, να εκκινήσουμε έναν σταματημένο NodeManager:

```
yarn --daemon start nodemanager
```

Μπορούμε να εκκινήσουμε και να τερματίσουμε μεμονωμένα τον ResourceManager, εάν όπου nodemanager βάλουμε resourcemanager.

9.3.6 Βασική διαχείριση υπηρεσιών

Εκτός από αυτές τις δύο βασικές για τη λειτουργία του YARN οντότητες, μπορούμε να χρησιμοποιήσουμε την ίδια μορφή εντολής ώστε να εκκινήσουμε και να τερματίσουμε μια σειρά από βοηθητικές υπηρεσίες του YARN. Μία από αυτές είναι ο Timelineserver⁴.

Ο Timelineserver χρησιμοποιείται για την εμφάνιση πληροφοριών με ενιαίο τρόπο τόσο για τις εκτελούμενες εφαρμογές του YARN όσο και για αυτές που έχουν εκτελεστεί στο παρελθόν. Και αυτός εκκινείται με:

```
yarn --daemon start timelineserver
```

και τερματίζεται, αντίστοιχα, εάν αντικαταστήσουμε το start με stop.

9.3.7 Βασική διαχείριση εφαρμογών

Με αντίστοιχο τρόπο μπορούμε να διαχειριστούμε εφαρμογές στο YARN. Ας θεωρήσουμε το Παράδειγμα 9.2.

Παράδειγμα 9.2. Έστω ότι εκτελούμε ένα από τα παραδείγματα που περιέχονται στο πακέτο του Hadoop και συγκεκριμένα τον προσεγγιστικό υπολογισμό του π . Αυτό επιτυγχάνεται με την παρακάτω εντολή:

```
hadoop jar hadoop/share/hadoop/mapreduce/hadoop-*-examples*.jar pi 10 100
```

Ο χαρακτήρας μπαλαντέρ *** χρησιμοποιείται για να μειώσει το μήκος της εντολής, αλλά και να καταστήσει δυνατή την εκτέλεση ανεξαρτήτως έκδοσης του Hadoop, καθώς στο όνομα

⁴[https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/](https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/TimelineServer.html)
TimelineServer.html

του συγκεκριμένου jar αρχείου περιλαμβάνεται και η έκδοση του Hadoop από την οποία προέρχεται.

Μπορούμε τώρα, ανοίγοντας ένα νέο τερματικό, και όσο εκτελείται το παραπάνω παράδειγμα, να εκτελέσουμε την παρακάτω εντολή:

```
yarn application -list
```

Το αποτέλεσμα αυτής της εντολής εμφανίζεται στο Σχήμα 9.6.

```
2023-09-29 23:16:35,754 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at master/127.0.0.1:8032
Total number of applications (application-types: [], states: [SUBMITTED, ACCEPTED, RUNNING] and tags: []):1
```

Application-Id	Application-Name	Application-Type	User	Queue	State
Final-State	Progress	Tracking-URL			
application_1696017934927_0004	QuasiMonteCarlo	MAPREDUCE	bigdata	default	ACCEPTED
UNDEFINED	0%	N/A			

Σχήμα 9.6: Λίστα εκτέλεσης εφαρμογών στο YARN.

Εδώ μπορούμε να δούμε ότι εκτελείται η εφαρμογή που έχουμε εκκινήσει με όνομα Quasi-MonteCarlo για τον χρήστη bigdata, ότι έχει λάβει το αναγνωριστικό

application_1696017934927_0004

και ότι έχει γίνει αποδεκτή προς εκτέλεση από τον ResourceManager.

Έχοντας το αναγνωριστικό της εφαρμογής, μπορούμε να προβούμε σε μια σειρά από ενέργειες. Για παράδειγμα, μπορούμε να δούμε λεπτομέρειες εκτέλεσης της εφαρμογής αυτής.

Για να δούμε λεπτομέρειες εκτέλεσης της εφαρμογής μας, θα πρέπει να χρησιμοποιήσουμε το αναγνωριστικό που της έχει δοθεί. Έτσι θα πρέπει να εκτελέσουμε:

```
yarn application -status application_1696017934927_0004
```

Το αποτέλεσμα αυτής της εντολής εμφανίζεται στο Σχήμα 9.7.

```
Application Report :
  Application-Id : application_1696017934927_0004
  Application-Name : QuasiMonteCarlo
  Application-Type : MAPREDUCE
  User : bigdata
  Queue : default
  Application Priority : 0
  Start-Time : 1696018594487
  Finish-Time : 0
  Progress : 5%
  State : RUNNING
  Final-State : UNDEFINED
  Tracking-URL : http://bigdata-virtualbox:46189
  RPC Port : 44563
  AM Host : bigdata-virtualbox
  Aggregate Resource Allocation : 109330 MB-seconds, 85 vcore-seconds
  Aggregate Resource Preempted : 0 MB-seconds, 0 vcore-seconds
  Log Aggregation Status : DISABLED
  Diagnostics :
  Unmanaged Application : false
  Application Node Label Expression : <Not set>
  AM container Node Label Expression : <DEFAULT_PARTITION>
  TimeoutType : LIFETIME ExpiryTime : UNLIMITED RemainingTime : -1seconds
```

Σχήμα 9.7: Λεπτομέρειες εκτελούμενης εφαρμογής στο YARN.

Εδώ μπορούμε να δούμε μια σειρά από επιπλέον πληροφορίες, όπως την τρέχουσα κατάσταση της εφαρμογής, τον χρόνο έναρξης, τον χρόνο επεξεργαστή, κ.ά.

Τέλος, εκτός από την παρακολούθηση των εφαρμογών, μπορούμε να επέμβουμε στην εκτέλεσή τους. Έτσι, μπορούμε να τερματίσουμε βίαια μια εφαρμογή ως εξής:

```
yarn application -kill application_1696017934927_0004
```

Με την παραπάνω εντολή, μπορούμε να τερματίσουμε τη συγκεκριμένη εφαρμογή. □

9.4 Apache Mesos

Το Apache Mesos αποτελεί ένα αυτόνομο σύστημα διαχείρισης και παρακολούθησης πόρων γενικού σκοπού σχεδιασμένο για datacenters το οποίο έχει γνωρίσει ευρεία αποδοχή, καθώς, μεταξύ άλλων, παρέχει και τη δυνατότητα ανάπτυξης εφαρμογών βασιζόμενων σε αυτό. Είναι ενδεικτικό ότι έχει χρησιμοποιηθεί από οργανισμούς όπως το eBay, το Paypal, το Twitter/X, κ.ά. Όπως είναι αναμενόμενο, το Mesos έχει σχεδιαστεί για να εκτελείται σε συστάδα.

Προσφέρει τη δυνατότητα αντιμετώπισης των μηχανών της συστάδας ως μιας ενιαίας ομάδας πόρων. Αυτό επιτυγχάνεται δημιουργώντας ένα επίπεδο αφαίρεσης πάνω από τη συστάδα, έτσι ώστε αυτή να αντιμετωπίζεται ως ένας ενιαίος διακομιστής. Όπως σε έναν τυπικό υπολογιστή υπάρχουν πολλαπλοί πυρήνες στον επεξεργαστή και δεν έχει ιδιαίτερη σημασία σε ποιον από αυτούς θα εκτελεστεί μια εφαρμογή, έτσι και το MESOS ακολουθεί παρόμοια λογική [Ign16]. Υποστηρίζει περιέκτες, κάτι που επιτρέπει την απομόνωση των εφαρμογών.

Με το Mesos, υπάρχει η δυνατότητα πολλαπλά κατανεμημένα συστήματα να εκτελεστούν απομονωμένα σε μια ενιαία συστάδα, τους πόρους της οποίας διαχειρίζεται το Mesos. Έτσι, σύμφωνα και με την παραπάνω λογική, αντί να χρησιμοποιούνται διαφορετικές συστάδες για την εγκατάσταση και εκτέλεση κατανεμημένων εφαρμογών, μπορεί να χρησιμοποιηθεί μία συστάδα η οποία θα εκτελεί εφαρμογές απομονωμένες μεταξύ τους. Επιπλέον, το Mesos επιτρέπει την απομόνωση εργασιών χρησιμοποιώντας είτε ομάδες ελέγχου (control groups) του Linux είτε περιέκτες Docker. Κάθε framework εκτελεί το task του μέσα στον δικό του περιέκτη.

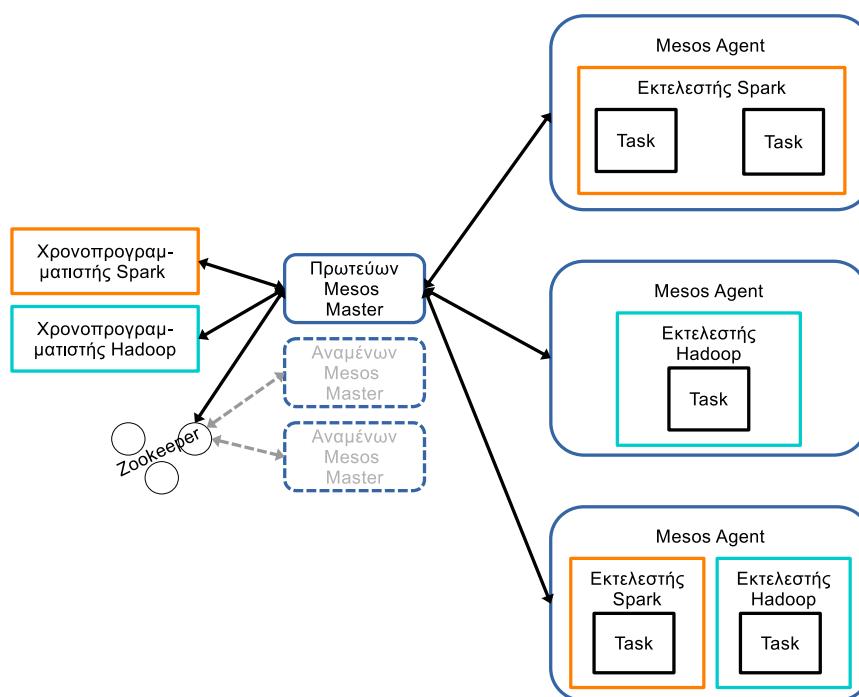
9.4.1 Οντότητες του Mesos

Από πλευράς αρχιτεκτονικής, το Mesos αποτελείται από τρεις κύριες οντότητες: τους masters, τους slaves (agents) και τις εφαρμογές, οι οποίες στο οικοσύστημα του Mesos απαντώνται ως “frameworks”.

9.4.1.1 Οι masters στο Mesos

Ένας ή περισσότεροι masters διαχειρίζονται τους slaves που εκτελούνται στους κόμβους της συστάδας. Καθώς μπορεί να υπάρχουν πλέον του ενός masters, ένας από αυτούς είναι ο κύριος master, ενώ οι υπόλοιποι σε κατάσταση αναμονής. Για να αποφασιστεί ποιος από τους κόμβους αυτούς θα είναι ο κύριος master, χρησιμοποιείται ένα άλλο framework, το Zookeeper⁵.

⁵<https://zookeeper.apache.org/>



Σχήμα 9.8: Αρχιτεκτονική Mesos.

Οι masters πρέπει να έχουν άφθονη κύρια μνήμη διαθέσιμη, καθώς διατηρούν σε αυτήν τα μεταδεδομένα των εργασιών που εκτελούνται.

Ο κύριος master, χρησιμοποιώντας αρθρωτό αλγόριθμο χρονοπρογραμματισμού, προσφέρει στις εφαρμογές πόρους βάσει κάποιας *πολιτικής* που ακολουθείται από αυτόν. Οι προσφορές μπορούν να γίνουν δεκτές ή να απορριφθούν. Μια προσφορά είναι ένα σύνολο πόρων σε κάποιον agent που γίνεται διαθέσιμο σε κάποιο framework. Με άλλα λόγια, ο master προσφέρει σε μια εφαρμογή πόρους που βρίσκονται διαθέσιμοι σε κάποιον agent. Υπάρχουν δύο ενσωματωμένες πολιτικές. Η *δίκαιη πολιτική*, και η *πολιτική προτεραιότητας*. Τέλος, ο μηχανισμός πολιτικών είναι επεκτάσιμος. Αυτό σημαίνει ότι παρέχεται η δυνατότητα να οριστούν νέα, προσαρμοσμένα σχήματα πολιτικών.

9.4.1.2 Οι slaves

Οι slaves εκτελούν τις εργασίες μίας εφαρμογής-framework σε απομόνωση. Σε περίπτωση ύπαρξης πλέον του ενός master, υποβάλλουν ερωτήματα στο Zookeeper για να εντοπίσουν τον κύριο master. Γνωστοποιούν στον κύριο master τους πόρους τους. Οι πόροι χρησιμοποιούνται από τα tasks των frameworks. Εκτός από πόρους, ένας slave έχει και κάποια γνωρίσματα (attributes). Τα γνωρίσματα αυτά περιγράφουν κάποια χαρακτηριστικά του slave, είτε υλικού είτε λογισμικού. Π.χ. λειτουργικό σύστημα και έκδοσή του. Τα attributes μεταβιβάζονται στα frameworks.

9.4.1.3 Τα frameworks

Τα frameworks είναι οι εφαρμογές οι οποίες εκτελούνται στο Mesos και εκτελούν ένα πλήθος από εργασίες (tasks), τα οποία και καταναλώνουν τους πόρους που διατίθενται από τον

master. Μια εφαρμογή που εκτελείται στο Mesos αποτελείται από δύο δομικά συστατικά: τον χρονοπρογραμματιστή και τον εκτελεστή.

Στο Mesos υπάρχει διαχωρισμός των λειτουργιών του χρονοπρογραμματισμού και της κατανομής πόρων [Kak15]. Την κατανομή πόρων, δηλαδή το ποια εφαρμογή θα λάβει ποιους πόρους, την αναλαμβάνει ο master. Τον χρονοπρογραμματισμό, δηλαδή το πώς θα χρησιμοποιηθούν αυτοί οι πόροι, τον αναλαμβάνουν οι χρονοπρογραμματιστές των εφαρμογών. Αξίζει να τονίσουμε στο σημείο αυτό, ότι ο χρονοπρογραμματιστής εκτελείται ανά εφαρμογή και όχι κεντρικά από τον server. Ο λόγος για αυτήν την αρχιτεκτονική επιλογή είναι ότι το Mesos προορίζεται για να εκτελεί ένα πλήθος εφαρμογών με διαφορετικές απαιτήσεις.

9.4.1.4 Οι *executors*

Ένας *executor* είναι ένας περιέκτης που εκτελεί *tasks* (εργασίες). Κάθε μία εργασία είναι αυτό που εκτελείται στο Mesos.

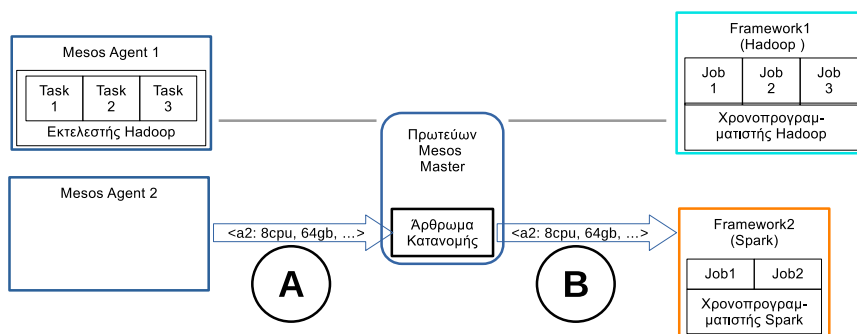
9.4.2 Λειτουργία του Mesos

Τα στοιχεία που μόλις περιγράφηκαν απεικονίζονται στο σχήμα 9.8. Συγκεκριμένα, υπάρχουν δύο frameworks που απεικονίζονται, μαζί με τους χρονοπρογραμματιστές τους: το Spark, με το πορτοκαλί χρώμα, και το Hadoop, με το κυανό χρώμα. Μπορούμε επίσης να δούμε τον κύριο master στο κέντρο του σχήματος, τους masters σε αναμονή και το Zookeeper, το οποίο επιλέγει master σε περίπτωση κατάρρευσης του πρωτεύοντος. Έπειτα, στα δεξιά, βλέπουμε τρεις slaves (agents) οι οποίοι επικοινωνούν με τον πρωτεύοντα master. Στον πρώτο, βρίσκεται ένας εκτελεστής του Spark στον οποίο εκτελούνται δύο εργασίες του Spark. Στον δεύτερο, ένας εκτελεστής του Hadoop με μία εργασία, ενώ στον τρίτο slave υπάρχουν δύο εκτελεστές: ένας για το Spark και ένας για το Hadoop, οι οποίοι εκτελούν από μία εργασία ο κάθε ένας.

Συνοπτικά, η λειτουργία του Mesos πραγματοποιείται σε δύο στάδια [Kak15]: Αρχικά, κάθε slave ενημερώνει περιοδικά τον master για τους διαθέσιμους πόρους του: μνήμη, επεξεργαστές, δίσκους κ.λπ. Έτσι, γνωρίζει τους διαθέσιμους πόρους ανά slave στο σύστημα. Κατά την εκκίνησή του, ένα framework γνωστοποιεί στον master την ύπαρξή του. Ο master ενεργοποιεί το άρθρωμα κατανομής και διαθέτει πόρους στο framework, το οποίο με τη σειρά του τους αποδέχεται, εάν είναι κατάλληλοι για τις εργασίες που έχει να εκτελέσει. Σε αντίθετη περίπτωση, τους απορρίπτει περιμένοντας για νέα προσφορά. Στην περίπτωση που η προσφορά γίνει δεκτή, ο slave δεσμεύει τους πόρους που έχουν προσφερθεί και εκκινεί τους εκτελεστές του framework, οι οποίοι με τη σειρά τους εκκινούν τις εργασίες του framework. Ο χρονοπρογραμματιστής παρακολουθεί την εκτέλεση των εργασιών και εκκινεί νέες, ενώ παράλληλα εξακολουθεί να λαμβάνει προσφορές πόρων, τις οποίες δέχεται ή απορρίπτει. Τέλος, εάν το framework έχει ολοκληρώσει την εργασία του, θα ενημερώσει τον master ώστε να σταματήσει να λαμβάνει προσφορές πόρων.

Ο χρονοπρογραμματιστής μπορεί να δεχθεί ή να απορρίψει μια προσφορά πόρων από τον master. Ο εκτελεστής επιτρέπει την εκτέλεση *tasks* υπό τη μορφή σεναρίων φλοιού ή Docker containers. Σε έναν εκτελεστή μπορεί να εκτελούνται περισσότερες από μία εργασίες.

Για τον χρονοπρογραμματισμό (κατανομή πόρων), το Mesos χρησιμοποιεί ένα *άρθρωμα κα-*



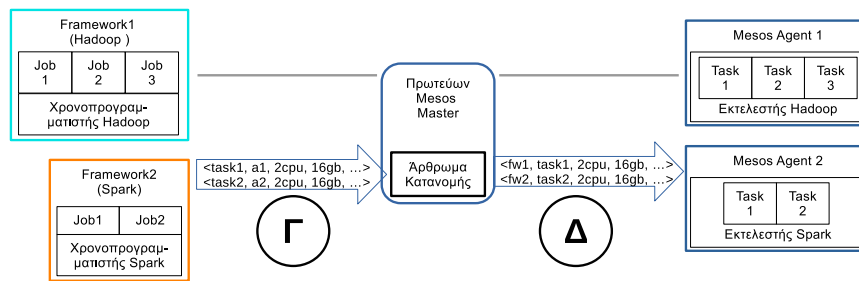
Σχήμα 9.9: Παράδειγμα εκτέλεσης Mesos - Μέρος 1.

τανομής (allocation module). Στη βασική του διαμόρφωση, αυτό χρησιμοποιεί έναν αλγόριθμο κατανομής πόρων που ονομάζεται DRF (Dominant Resource Fairness) [GZH⁺11], ο οποίος, επιγραμματικά, προσπαθεί να εξισορροπήσει την κατανομή πόρων για κάθε τύπο πόρου. Ο μηχανισμός κατανομής αποτελεί άρθρωμα, καθώς υπάρχει η δυνατότητα αντικατάστασής του από έναν προσαρμοσμένο μηχανισμό. Πέρα, όμως, από την αντικατάσταση, υπάρχει η δυνατότητα παραμετροποίησης του βασικού μηχανισμού με τη χρήση ρόλων (roles), βαρών (weights) και κρατήσεων πόρων (resource reservations). Έτσι, μπορεί να πραγματοποιηθεί κράτηση πόρων για συγκεκριμένους ρόλους, χρησιμοποιώντας βάρη για την προτεραιότητα των ρόλων αυτών.

Πιο αναλυτικά, οι ρόλοι χρησιμοποιούνται για τη δημιουργία, επί της ουσίας, υπο-συστάδων. Οι agents του Mesos μπορούν να αναθέσουν πόρους σε συγκεκριμένους ρόλους, κάτι που αποτελεί και την κράτηση των πόρων. Τώρα, όσον αφορά τα βάρη, οι ρόλοι με υψηλότερα βάρη λαμβάνουν πόρους ανάλογους του βάρους τους. Οι κρατήσεις, από την άλλη, εξασφαλίζουν σε συγκεκριμένα frameworks πάντα τους πόρους που απαιτούν.

Ας δώσουμε ένα παράδειγμα του τρόπου με τον οποίο μια εφαρμογή (framework), έστω το Apache Spark, εκτελεί τις εργασίες της. Για να γίνει αυτό εφικτό, είναι λογικό ότι απαιτούνται πόροι. Όπως είπαμε και πιο πριν, οι πόροι αυτοί γνωστοποιούνται από τους slaves στον πρωτεύοντα master. Το Σχήμα 9.9 απεικονίζει αυτήν τη διαδικασία. Έτσι, στη φάση (Α), ο slave 2 (Mesos Agent 2) ενημερώνει τον master ότι διαθέτει ελεύθερα 64 GB μνήμης και 8 CPUs. Το άρθρωμα κατανομής του master προσφέρει αυτούς τους πόρους στην εφαρμογή Spark που εκτελείται. Στη φάση (Β), ο master ενημερώνει το Spark framework για τους πόρους που του προσφέρει ο slave 2. Το Spark framework χρειάζεται να εκτελέσει 2 tasks με 16 GB μνήμης και 2 CPUs το κάθε ένα. Παρατηρώντας το Σχήμα 9.10, βλέπουμε ότι, στη φάση (Γ), το Spark framework ενημερώνει τον master για τους πόρους που χρειάζεται και ο master αποστέλλει τις εργασίες στον slave 2, στη φάση (Δ) [HKZ⁺11], ο οποίος διαθέτει τους σχετικούς πόρους στους εκτελεστές του Spark. Από τη διαδικασία αυτή βλέπουμε ότι απομένουν 2 CPUs και 32 GB μνήμης. Αυτούς τους πόρους ο master μπορεί να τους διαθέσει σε άλλο framework.

Πέρα από τα παραπάνω συστατικά στοιχεία, η λειτουργία του Mesos προϋποθέτει την ύπαρξη μιας σειράς εξωτερικών βοηθητικών υπηρεσιών, έτσι ώστε να επιτύχει την οπτική και τη λειτουργικότητα ενός ενιαίου συστήματος. Θα πρέπει, λοιπόν, να υπάρχει ένα σύστημα αρχείων με δυνατότητα διαμοιρασμού. Αυτό μπορεί να είναι το HDFS ή κάποια επιλογή στο Νέφος, όπως το Amazon S3.



Σχήμα 9.10: Παράδειγμα εκτέλεσης Mesos - Μέρος 2.

9.4.2.1 Ανοχή σε σφάλματα

Το Mesos είναι εγγενώς κατανομημένο και εγγυάται ανοχή στα σφάλματα. Η ανοχή σε σφάλματα έχει την έννοια ότι η αστοχία ενός κόμβου του συστήματος δεν παύει τη λειτουργία του συστήματος, το οποίο έχει τη δυνατότητα να ανακάμψει από αυτό το σφάλμα.

Το Mesos έχει δύο μηχανισμούς που του επιτρέπουν ανοχή σε σφάλματα. Ο ένας μηχανισμός είναι η χρήση σημείων ελέγχου, ενώ ο άλλος η ανάκαμψη των slaves. Κατά τη λήψη σημείου ελέγχου, πληροφορίες για την κατάσταση της συστάδας αποθηκεύονται στον δίσκο. Η ανάκαμψη των slaves επιτρέπει σε αυτούς να διαβάσουν πληροφορίες από τους δίσκους και να επανασυνδεθούν με τους εκτελεστές, σε περίπτωση που οι slaves καταρρεύσουν ή επανεκκινηθούν.

9.4.2.2 Υψηλή διαθεσιμότητα

Η υψηλή διαθεσιμότητα σημαίνει ότι το σύστημα είναι λειτουργικό για μεγάλες χρονικές περιόδους, κοντά στο 100% του χρόνου λειτουργίας του. Όπως και στην περίπτωση του YARN, επιτυγχάνεται με την ύπαρξη πολλαπλών διακομιστών σε κατάσταση αναμονής, οι οποίοι, σε περίπτωση βλάβης του κυρίως master, εκλέγουν νέο κυρίως master μέσω του Zookeeper. Τόσο οι slaves όσο και οι εφαρμογές χρησιμοποιούν το Zookeeper για τον εντοπισμό του κυρίως master.

9.5 Kubernetes

Το Kubernetes περιγράφεται συχνά ως *ενορχηστρωτής (orchestrator)*. Πρόκειται για την αντίστοιχη έννοια του μουσικού που αναλαμβάνει τη μουσική επένδυση ενός τραγουδιού. Εκεί, υπάρχει μια πληθώρα από όργανα και τους αντίστοιχους μουσικούς. Στο τραγούδι, κάθε μουσικός θα εκτελέσει με το αντίστοιχο μουσικό όργανο το τμήμα της σύνθεσης που του αναλογεί. Ο ενορχηστρωτής θα καθοδηγήσει τους μουσικούς ως προς τον τρόπο που θα εκτελέσουν το τμήμα του μουσικού κομματιού.

Έχουμε ήδη εξηγήσει τον ρόλο των περιεκτών. Το Kubernetes είναι ένα σύστημα που επιτρέπει την εκτέλεση περιεκτών παρέχοντας δυνατότητες ανθεκτικότητας και ανάκαμψης από σφάλματα. Αναπτύχθηκε από την Google, είναι ανοικτού κώδικα και βασίζεται σε έναν

μηχανισμό μόνιμης αποθήκευσης και προσφέρει μια σειρά από δυνατότητες.⁶ Έτσι, μπορεί να προσφέρει ανακάλυψη υπηρεσιών, προσδίδοντας σε κάθε περιέκτη τη δική του διεύθυνση IP. Μπορεί να πραγματοποιήσει κατανομή φόρτου μεταξύ των περιεκτών. Έχει τη δυνατότητα αυτοματοποιημένης διαχείρισης αποθήκευσης, επιτρέποντας την αυτοματοποιημένη προσάρτηση όχι μόνο τοπικών πόρων αποθήκευσης, αλλά και πόρων αποθήκευσης στο νέφος. Έχει τη δυνατότητα αυτοματοποιημένης αύξησης ή μείωσης των ενεργών περιεκτών μιας εφαρμογής, ανακαταθέτοντας τους διαθέσιμους πόρους μεταξύ τους. Επιτρέπει την ορθή διαχείριση των πόρων, αναθέτοντας σε κάθε περιέκτη τους πόρους που θα ορίσει ο χρήστης. Επιπλέον, προσφέρει τη δυνατότητα ανάκαμψης από κατάρρευση, επανεκκινώντας περιέκτες που έχουν καταρρεύσει και τερματίζοντας αυτούς που δεν ανταποκρίνονται. Τέλος, επιτρέπει την ασφαλή αποθήκευση πληροφοριών.

9.5.1 Έννοιες του Kubernetes

Εδώ θα παραθέσουμε μία σειρά από έννοιες που θα επιτρέψουν την κατανόηση της λειτουργίας και των επιμέρους μηχανισμών του Kubernetes.

9.5.2 Workloads

Με τον όρο *φόρτος εργασίας* (*Workload*) αναφερόμαστε σε μια εφαρμογή η οποία εκτελείται στο Kubernetes. Τα φορτία εργασίας εκτελούνται πάντα εντός των ακάτων. Κάθε άκατος αποτελεί ένα σύνολο εκτελούμενων περιεκτών. Για τη διαχείριση των ακάτων μιας εφαρμογής, χρησιμοποιούνται οι *πόροι φόρτων εργασίας* (*Workload Resources*) που διαχειρίζονται συνολικά τις ακάτους, οι οποίοι μέσω ελεγκτών υλοποιούν την επιθυμητή από τον χρήστη πολιτική εκτέλεσης. Μπορούν να χρησιμοποιηθούν οι βασικοί πόροι φόρτων εργασίας που είναι διαθέσιμοι στο Kubernetes ή πόροι εργασίας προερχόμενοι από εξωτερικές πηγές.

9.5.3 Άκατοι

Η *άκατος* (*pod*) είναι η στοιχειώδης υπολογιστική μονάδα του Kubernetes και αποτελείται από ένα σύνολο περιεκτών που μοιράζονται πόρους δικτύου και αποθήκευσης, καθώς και οδηγίες για τον τρόπο εκτέλεσης των περιεκτών. Οι άκατοι εκκινούν περιβάλλοντα εκτέλεσης εγκατεστημένα στους κόμβους και κάθε μία προορίζεται να εκτελεί ένα στιγμιότυπο μιας εφαρμογής. Καθώς οι άκατοι δημιουργούνται μέσω των πόρων φόρτων εργασίας, σε περίπτωση που είναι επιθυμητό να υπάρχουν πλέον του ενός στιγμιότυπα μιας εφαρμογής με στόχο την εξισορρόπηση φόρτου, τότε θα πρέπει να οριστούν αντίστοιχα πολλαπλές τέτοιες άκατοι και όχι πολλαπλοί περιέκτες εντός μιας ακάτου, ώστε να γίνει ορθή κατανομή φόρτου στους κόμβους.

Μια άκατος μπορεί να έχει δύο μορφές: ακάτους που εκτελούν έναν μοναδικό περιέκτη και ακάτους που εκτελούν πολλαπλούς συνεργαζόμενους περιέκτες. Ας δούμε τις δύο αυτές περιπτώσεις πιο αναλυτικά. Καθώς το Kubernetes διαχειρίζεται ακάτους και όχι περιέκτες, η άκατος που περιλαμβάνει μοναδικό περιέκτη είναι ο τρόπος με τον οποίο ένας περιέκτης

⁶<https://kubernetes.io/docs/concepts/overview>

μπορεί να εκτελεστεί στο Kubernetes. Η δεύτερη εναλλακτική, όπου έχουμε πολλαπλούς περιέκτες εντός μίας ακάτου, αφορά περιπτώσεις όπου πολλαπλοί περιέκτες θα πρέπει να συνεργαστούν για την επίτευξη ενός σκοπού. Σε αυτήν την περίπτωση, η άκατος προσφέρει ένα πλαίσιο μέσα στο οποίο οι περιέκτες αυτοί μπορούν να συνεργαστούν και να μοιράζονται δικτυακούς και αποθηκευτικούς πόρους. Ενόσω εκτελούνται οι περιέκτες μίας ακάτου, δεν μπορούν να τροποποιηθούν οι ρυθμίσεις της. Θα πρέπει να τερματιστεί και να ξεκινήσει νέα άκατος από την αρχή, σε περίπτωση που επιθυμούμε να την ενημερώσουμε.

9.5.3.1 Εκτέλεση

Μια άκατος μπορεί να βρίσκεται σε μια από πέντε πιθανές φάσεις εκτέλεσης. Έτσι, μπορεί να είναι εκκρεμής, εκτελούμενη, επιτυχής, ανεπιτυχής ή άγνωστη. Ας δούμε αυτές τις φάσεις μία προς μία. Μια άκατος χαρακτηρίζεται ως *εκκρεμής* όταν, ενώ έχει γίνει αποδεκτή από το Kubernetes, δεν έχουν εκκινηθεί όλοι οι περιέκτες που την απαρτίζουν. Ένας πιθανός λόγος για τον οποίο μια άκατος μπορεί να βρίσκεται σε αυτήν την κατάσταση είναι η μη ολοκλήρωση της λήψης των εικόνων των προς εκτέλεση περιεκτών της. Όταν όλοι οι περιέκτες μιας ακάτου έχουν δημιουργηθεί και ένας τουλάχιστον εκκινείται, εκτελείται ή επανεκκινείται, η άκατος είναι *εκτελούμενη*. Στην επιτυχή ολοκλήρωση όλων των περιεκτών της, η κατάσταση στην οποία βρίσκεται η άκατος ονομάζεται *επιτυχής*. Εάν ένας τουλάχιστον περιέκτης δεν ολοκληρωθεί επιτυχώς, η κατάστασή της ονομάζεται *ανεπιτυχής*, ενώ αν δεν μπορεί να ανακτηθεί η κατάσταση μίας ακάτου, τότε χαρακτηρίζεται ως *άγνωστη*. Αντίστοιχα με τις ακάτους, και η κατάσταση των περιεκτών περιγράφεται με μία σειρά από χαρακτηρισμούς. Έτσι, ένας περιέκτης μπορεί να είναι σε κατάσταση αναμονής, εκτέλεσης ή τερματισμένος. Οι ονομασίες είναι αυτοπεριγραφικές.

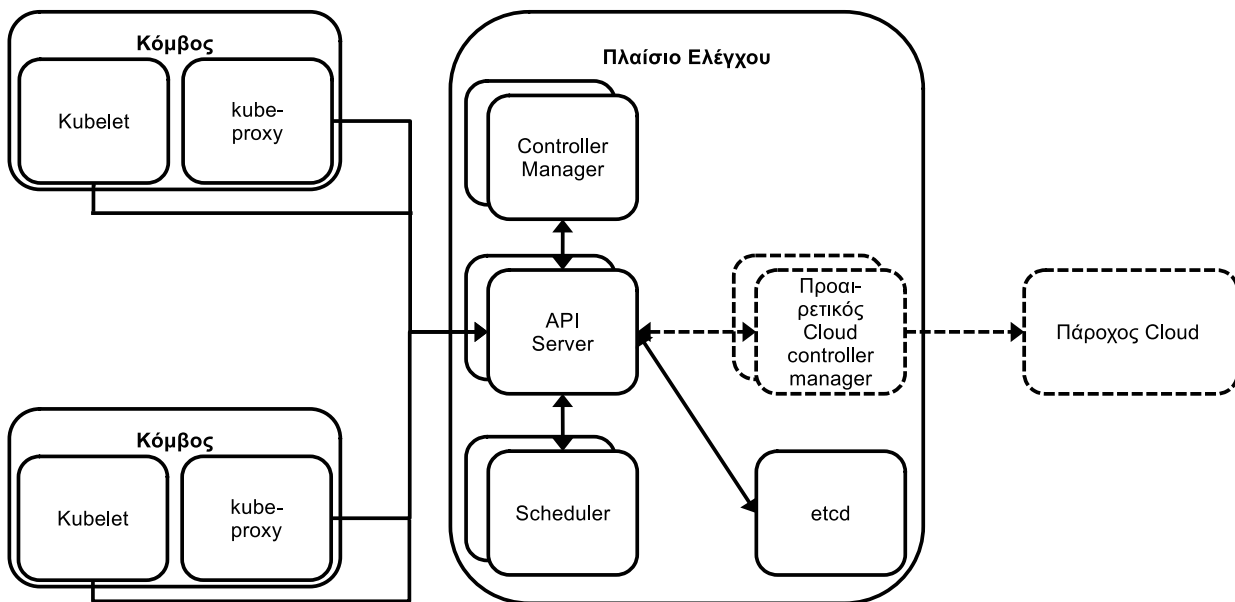
9.5.3.2 Συνδεσιμότητα

Κάθε άκατος έχει τη δική της διεύθυνση IP η οποία και είναι μοναδική σε όλη τη συστάδα. Με αυτόν τον τρόπο μια άκατος μπορεί να αντιμετωπιστεί όπως μια εικονική μηχανή. Αυτή είναι η διεύθυνση IP των εκτελούμενων περιεκτών σε αυτήν. Επίσης, οι περιέκτες έχουν κοινό εύρος θυρών. Οι άκατοι μπορούν να επικοινωνούν άμεσα μεταξύ τους, αλλά και με όλες τις υπηρεσίες σε έναν κόμβο.

Καθώς άκατοι τερματίζονται και δημιουργούνται διαρκώς και κάθε άκατος έχει τη δική της IP διεύθυνση, θα πρέπει να υπάρχει η δυνατότητα απρόσκοπτης επικοινωνίας. Για τον λόγο αυτόν, υπάρχουν οι *υπηρεσίες* (*Services*) οι οποίες προσφέρουν αξιοπιστία στην επικοινωνία εντός ενός συνόλου ακάτων, έχοντας σταθερό όνομα DNS και πραγματοποιώντας και εξισορρόπηση φόρτου. Κάθε συστάδα του Kubernetes έχει τη δική της υπηρεσία DNS, για να μπορεί να την εντοπίζει κάθε εκτελούμενος περιέκτης σε κάθε άκατο.

9.5.3.3 Μόνιμη αποθήκευση

Η τοπική αποθήκευση σε μια άκατο είναι προσωρινή και διαρκεί όσο η άκατος αυτή υφίσταται. Μπορεί να χρησιμοποιηθεί για την ανταλλαγή δεδομένων μεταξύ των κόμβων. Σε περίπτωση, όμως, που είναι απαραίτητη η μόνιμη αποθήκευση δεδομένων, το Kubernetes



Σχήμα 9.11: Αρχιτεκτονική Kubernetes.

υποστηρίζει τόμους αποθήκευσης (volumes) οι οποίοι μπορεί να αφορούν σκληρούς δίσκους, πόρους στο νέφος, ή κατανεμημένα συστήματα αρχείων.

9.5.4 Κόμβοι

Στο Kubernetes οι εφαρμογές, που όπως προείπαμε ονομάζονται workloads, εκτελούνται με την τοποθέτηση περιεκτών εντός των ακάτων οι οποίες φιλοξενούνται στους κόμβους. Για να μπορέσει το επίπεδο ελέγχου να έχει επίγνωση της κατάστασης των κόμβων, αυτοί του αποστέλλουν παλμούς, ώστε τυχόν αποτυχίες των κόμβων να μπορέσουν να γίνουν άμεσα αντιληπτές.

9.5.5 Συστατικά στοιχεία του Kubernetes

Μια συστάδα στο Kubernetes αποτελείται, αντίστοιχα και με τα υπόλοιπα συστήματα που παρουσιάσαμε, από μηχανές που έχουν τον ρόλο του master, που στην περίπτωση αυτή ονομάζεται *επίπεδο ελέγχου* (Control Plane). Αρκεί μία μηχανή για τον ρόλο του master, αν και σε διατάξεις υψηλής διαθεσιμότητας (High Availability, HA), συνήθως χρησιμοποιούνται τρεις [Car22] και από μηχανές που έχουν τον ρόλο του εργάτη (slave). Οι δεύτερες ονομάζονται κόμβοι (nodes) και εκτελούν τις εφαρμογές εντός των περιεκτών. Για την εξασφάλιση ανοχής σε σφάλματα και υψηλής διαθεσιμότητας, το επίπεδο ελέγχου εκτελείται σε πολλές μηχανές και σε μία συστάδα εκτελούνται πολλαπλοί κόμβοι.

9.5.5.1 Στοιχεία του επιπέδου ελέγχου

Τα στοιχεία του επιπέδου ελέγχου διαχειρίζονται συνολικά τα στοιχεία της συστάδας [kub]:

- **Cloud-controller-manager.** Το πρώτο από τα στοιχεία του επιπέδου ελέγχου είναι ο

cloud-controller-manager, ο οποίος είναι υπεύθυνος για την επικοινωνία της συστάδας με το API του παρόχου του συστήματος Νέφους που τη φιλοξενεί και είναι λειτουργικό μόνο όταν η συστάδα φιλοξενείται σε Νέφος. Για την ανοχή σε σφάλματα και την εξισορρόπηση φόρτου, μπορούν να εκτελούνται περισσότερες από μία διεργασίες cloud-controller-manager.

- **Μόνιμη αποθήκευση της συστάδας.** Για τη διατήρηση των δεδομένων σε μόνιμη αποθήκευση, χρειάζεται μια βάση δεδομένων. Το etcd αποτελεί τη βάση δεδομένων που χρησιμοποιεί το Kubernetes για τη μόνιμη αποθήκευση στον δίσκο όλων των δεδομένων της συστάδας. Προσφέρει συνέπεια και υψηλή διαθεσιμότητα, δίνοντας προτεραιότητα στη συνέπεια.
- **Kube-apiserver** Ο kube-apiserver είναι το σημείο επαφής για το API του Kubernetes και αποτελεί το κύριο στοιχείο του επιπέδου ελέγχου. Για εξισορρόπηση φόρτου, μπορούν να εκτελούνται περισσότεροι από ένας kube-apiservers. Προσφέρει ένα HTTP API το οποίο επιτρέπει την επικοινωνία μεταξύ των διαφόρων συστατικών της συστάδας, των χρηστών και εξωτερικών μηχανισμών
- **Kube-controller-manager.** Στο Kubernetes υπάρχει πλήθος διαφορετικών ελεγκτών που παρακολουθούν τη λειτουργία των επιμέρους στοιχείων του. Ενδεικτικά, υπάρχει ο *ελεγκτής κόμβων (Node Controller)*, για την παρακολούθηση και διαχείριση των κόμβων, ο οποίος διατηρεί μία λίστα με τις διαθέσιμες μηχανές (φυσικές ή εικονικές) στη συστάδα. Επιπλέον, ο ελεγκτής κόμβων είναι υπεύθυνος για την κατάσταση των κόμβων, παρακολουθώντας την κατάστασή τους μέσω παλμών. Έτσι, απουσία παλμών μπορεί να σημαίνει την πιθανή κατάρρευση κάποιου κόμβου. Έπειτα, υπάρχει ο *ελεγκτής εργασιών (Job controller)*, ο οποίος παρακολουθεί τα αντικείμενα εργασιών και δημιουργεί ακάτους για την εκτέλεσή τους, και άλλοι, όπως οι ελεγκτές Deployment, StatefulSet και ReplicaSet. Για την απλούστερη διαχείριση ελεγκτών, αυτοί έχουν ενσωματωθεί σε μία εργασία, στον kube-controller-manager, ο οποίος τους εκτελεί και τους διαχειρίζεται. Κάθε ελεγκτής ακολουθεί την εξής διαδικασία [Ρου23]: αρχικά, ενημερώνεται για την κατάσταση στην οποία βρίσκονται τα αντικείμενα που παρακολουθεί. Έπειτα, εντοπίζει την κατάσταση στην οποία βρίσκονται τα εν λόγω αντικείμενα. Στη συνέχεια, εντοπίζει τις διαφορές μεταξύ των δύο αυτών καταστάσεων και, τέλος, επιλύει αυτές τις διαφορές ώστε το κάθε αντικείμενο να επέλθει στη ζητούμενη κατάσταση.
- **Kube-scheduler.** Ο χρονοπρογραμματιστής kube (kube-scheduler) έχει ως ρόλο την κατανομή των ακάτων που δημιουργούνται για την εκτέλεση των εργασιών στους κόμβους της συστάδας. Για να το κάνει αυτό, λαμβάνει υπόψη μία σειρά από παράγοντες, όπως είναι οι απαιτήσεις σε πόρους, οι διαθέσιμοι πόροι, κ.ά.

9.5.5.2 Στοιχεία των κόμβων

Τα παρακάτω στοιχεία εκτελούνται σε κάθε κόμβο σε μία συστάδα Kubernetes [kub]:

- **Container runtime.** Το περιβάλλον εκτέλεσης περιέκτη (*Container Runtime*) διαχειρίζεται την εκτέλεση των περιεκτών στο Kubernetes. Το Kubernetes μπορεί να υποστηρίξει διάφορα περιβάλλοντα εκτέλεσης, τα οποία υλοποιούν τη διεπαφή περιβάλλοντος εκτέλεσης του Kubernetes (Kubernetes CRI).
- **kubelet.** Το kubelet είναι ο μηχανισμός που διαχειρίζεται την εκτέλεση των περιεκτών μίας ακάτου και εκτελείται σε κάθε κόμβο. Παρακολουθεί τον API server για τυχόν νέες εργασίες, τις οποίες και αναλαμβάνει. Εάν ο κόμβος στον οποίο βρίσκεται το kubelet δεν μπορεί να εκτελέσει μια νέα εργασία, το kubelet ενημερώνει το επίπεδο ελέγχου το οποίο είναι υπεύθυνο στη συνέχεια να επιλέξει έναν άλλον κόμβο.
- **kube-proxy.** Ο μεσολαβητής kube (kube-proxy) διαχειρίζεται τις επικοινωνίες δικτύου των ακάτων στη συστάδα. Εκτελείται σε κάθε κόμβο της συστάδας.

9.5.6 Εκτέλεση εφαρμογής στο Kubernetes

Για να εκτελεστεί μία εφαρμογή στο Kubernetes, προϋποτίθεται μία σειρά από βήματα [Rou23]. Έτσι, αρχικά, η εφαρμογή θα πρέπει να ενσωματωθεί σε μια εικόνα περιέκτη και να αποθηκευθεί σε ένα μητρώο κατάλληλο για τον σκοπό αυτόν. Στη συνέχεια, θα πρέπει να τοποθετηθεί σε μια άκατο για να εκτελεστεί. Ο ενδεδειγμένος τρόπος για να γίνει αυτό είναι μέσω των ελεγκτών, όπως αναφέρθηκε και πιο πριν, με πιο απλή περίπτωση αυτήν της χρήσης του *ελεγκτή ανάπτυξης (Deployment Controller)*.

Κάθε ελεγκτής προσφέρει διαφορετικούς μηχανισμούς. Έτσι, ο ελεγκτής ανάπτυξης προσφέρει μηχανισμούς κλιμάκωσης δημιουργώντας αντίγραφα, ανοχής σε σφάλματα, και ενημερώσεων για εφαρμογές που δεν διατηρούν την κατάστασή τους. Αυτά ορίζονται σε ένα αρχείο YAML-YAML Ain't Markup Language. Χρησιμοποιώντας το εργαλείο γραμμής εντολών του Kubernetes, ο API server λαμβάνει το αίτημα αντιμετωπίζοντάς το ως την επιθυμητή κατάσταση της εφαρμογής. Έπειτα, εντοπίζει σε ποιον ελεγκτή θα πρέπει να αποσταλεί, ώστε η επιθυμητή κατάσταση να υλοποιηθεί. Για παράδειγμα, εάν θέλουμε ένας διακομιστής να εκτελείται σε 3 αντίγραφα ακάτων και το ένα καταρρεύσει, η επιθυμητή κατάσταση θα προβλέπει την εκτέλεση ενός αντιγράφου λιγότερου. Το Kubernetes, βλέποντας τη διαφορά, θα προσπαθήσει να υλοποιήσει την επιθυμητή κατάσταση. Επιπλέον, υπάρχει η δυνατότητα να αλλάξουμε αυτές τις επιλογές, ακόμα και την εικόνα περιέκτη.

Ο χρονοπρογραμματισμός στο Kubernetes πραγματοποιείται ως εξής [Car22]: τα αιτήματα των χρηστών για πόρους αποστέλλονται στον master. Με τη λήψη, προωθούνται στον χρονοπρογραμματιστή, ο οποίος αποφασίζει σε ποιους κόμβους-εργάτες θα δημιουργηθούν και θα εκτελεστούν οι συγκεκριμένες άκατοι που θα περιγράφονται στο αίτημα. Έπειτα, ο χρονοπρογραμματιστής ενημερώνει τα kubelets των συγκεκριμένων κόμβων, ώστε να δημιουργήσουν τις ακάτους βάσει των προδιαγραφών που τίθενται στο αίτημα του χρήστη. Όταν ολοκληρωθεί η δημιουργία της ακάτου σε έναν κόμβο, ξεκινά η εκτέλεσή της και το kubelet του κόμβου αυτού επικοινωνεί με τον API server του master για να τον ενημερώσει για αυτό το γεγονός.

9.6 Spark standalone cluster manager

Το Apache Spark, εκτός από τη δυνατότητα αυτόνομης εκτέλεσης, έχει σχεδιαστεί με τη δυνατότητα εκτέλεσης σε συστάδα. Μπορεί το ίδιο να μην αποτελεί σύστημα διαχείρισης πόρων, αλλά διαθέτει τον δικό του μηχανισμό για τον σκοπό αυτόν. Καθώς είναι ένα ευρέως διαδεδομένο σύστημα και περιγράφεται αναλυτικά σε αυτό το βιβλίο, θα κάνουμε ειδική μνεία στον βασικό ενσωματωμένο χρονοπρογραμματιστή του. Βέβαια, το Spark μπορεί να συνεργαστεί και με τους εξωτερικούς διαχειριστές πόρων που αναφέρονται σε αυτό το κεφάλαιο.

9.6.1 Οντότητες του Apache Spark

Ας ξεκινήσουμε με τις βασικές έννοιες και οντότητες του Spark⁷, οι οποίες απεικονίζονται στο Σχήμα 9.12.

9.6.1.1 Ο driver

Ο driver αποτελεί το βασικό πρόγραμμα που εκτελείται στο Spark που συγγράφει ο χρήστης και δημιουργεί ένα αντικείμενο που ονομάζεται SparkContext, το οποίο αναλαμβάνει την επικοινωνία με τη συστάδα. Συγκεκριμένα, το SparkContext επικοινωνεί με κάποιον διαχειριστή πόρων, όπως αυτούς που αναφέρονται σε αυτό το κεφάλαιο, προκειμένου να αποκτήσει πόρους, καθώς και με τους διακομιστές οι οποίοι θα αναλάβουν να εκτελέσουν τμήματα του προγράμματος ξεκινώντας executors.

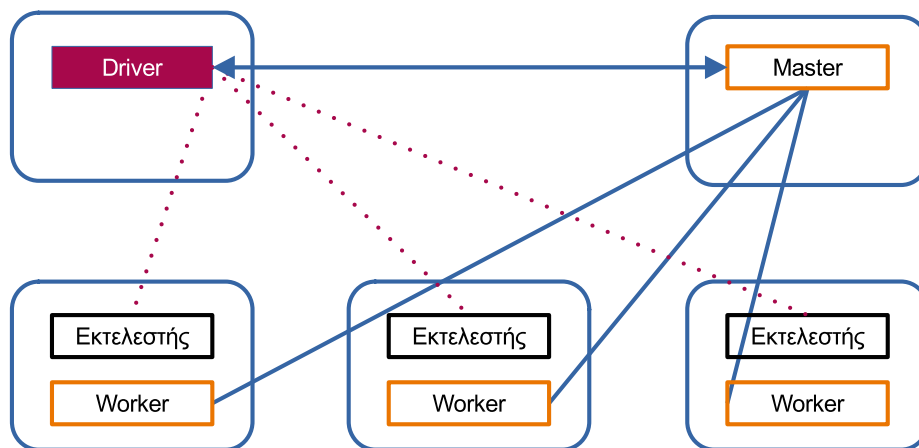
9.6.1.2 Ο master

Ο master είναι υπεύθυνος για να διαχειρίζεται τους διαθέσιμους πόρους στη συστάδα που εκτελούν τις εφαρμογές του Spark. Υπάρχουν τρεις διαμορφώσεις εκτέλεσης του Spark σε συστάδα, κάθε μία από τις οποίες προσδιορίζει και τη διαμόρφωση των πόρων [CZ18], που αφορά τον χώρο εκτέλεσης του driver: η διαμόρφωση συστάδας (Cluster Mode), η διαμόρφωση πελάτη (Client Mode) και η τοπική διαμόρφωση (Local Mode). Παρακάτω θα δούμε την κάθε μία από αυτές τι περιπτώσεις ξεχωριστά.

Διαμόρφωση συστάδας. Η διαμόρφωση συστάδας αποτελεί ίσως την πλέον διαδεδομένη προσέγγιση εκτέλεσης εφαρμογών στο Spark. Ο driver εκτελείται σε έναν κόμβο της συστάδας. Στο Σχήμα 9.12, εάν ο διακομιστής στον οποίο βρίσκεται ο driver είναι μέρος της συστάδας του Spark, τότε πρόκειται για διαμόρφωση συστάδας.

Διαμόρφωση πελάτη. Η διαμόρφωση πελάτη είναι παρόμοια με τη διαμόρφωση συστάδας. Η διαφορά έγκειται στην τοποθεσία εκτέλεσης του driver του Spark, το οποίο, αυτήν τη φορά, παραμένει στον υπολογιστή-πελάτη που υπέβαλε το αίτημα. Αυτό έχει ως αποτέλεσμα ο διακομιστής του πελάτη, εκτός της συστάδας, να είναι υπεύθυνος για τη διατήρηση εκτέλεσης

⁷<https://spark.apache.org/docs/latest/cluster-overview.html>



Σχήμα 9.12: Αρχιτεκτονική Spark χρησιμοποιώντας τον αυτόνομο διαχειριστή συστάδας του.

της εφαρμογής. Στο Σχήμα 9.12, εάν ο διακομιστής στον οποίο βρίσκεται ο driver δεν είναι μέρος της συστάδας του Spark, τότε πρόκειται για διαμόρφωση πελάτη.

Τοπική διαμόρφωση. Πρόκειται για την πλέον συνηθισμένη προσέγγιση που ακολουθείται για την εκμάθηση του Spark. Σε αυτήν την περίπτωση, όλη η εφαρμογή εκτελείται σε ένα μόνο μηχάνημα και ο παραλληλισμός επιτυγχάνεται μέσω ελαφρών διεργασιών. Είναι προφανές ότι μια τέτοια διάταξη δεν μπορεί να επιτύχει την κλιμάκωση των δύο προηγούμενων περιπτώσεων.

9.6.1.3 O worker

Κάθε worker διαχειρίζεται τους πόρους στον διακομιστή όπου βρίσκεται. Master και worker μπορεί να εκτελούνται στον ίδιο διακομιστή.

9.6.1.4 Executors

Οι executors είναι οι διεργασίες που εκτελούν τις εργασίες που περιγράφονται στον driver και έχουν ανατεθεί από το SparkContext. Τις εκτελούν και αναφέρουν την κατάσταση (επιτυχία ή αποτυχία) και τα αποτελέσματά τους.

9.6.2 Λειτουργία του διαχειριστή συστάδας του Spark

Για να περιγράψουμε τη λειτουργία του διαχειριστή cluster του Spark, ας θεωρήσουμε ότι έχουμε μία διαμόρφωση συστάδας, με τον driver να εκτελείται εντός της συστάδας του Spark. Ένα jar που περιέχει τον μεταγλωττισμένο κώδικα ή ένα script εκτελούνται στον τοπικό υπολογιστή και υποβάλλεται αίτημα στον master του διαχειριστή συστάδας (cluster manager), ζητώντας πόρους (περιέκτη) για τον driver. Μετά την υποβολή του κώδικά του στον διαχειριστή συστάδας, αυτός τοποθετεί τη διεργασία του driver σε έναν κόμβο εντός της συστάδας. Η αρχική διεργασία πελάτη που υπέβαλε το αίτημα τερματίζεται και αρχίζει η εκτέλεση του κώδικα χρήστη στη συστάδα. Δημιουργείται το SparkSession που αναλαμβάνει την επικοινωνία με τον master, ο οποίος ξεκινά τις διεργασίες των executors σε όλη τη συστάδα και στέλνει τις

σχετικές πληροφορίες σχετικά με τις τοποθεσίες τους στη διεργασία του driver. Ο driver και οι κόμβοι που περιλαμβάνουν τους executors επικοινωνούν μεταξύ τους, εκτελώντας κώδικα και μετακινώντας δεδομένα. Ο driver προγραμματίζει εργασίες σε κάθε εργάτη, ο οποίος ανταποκρίνεται αναφέροντας την κατάσταση αυτών των εργασιών και το εάν ολοκληρώθηκαν ή απέτυχαν. Η διεργασία driver ολοκληρώνεται είτε με επιτυχία είτε με αποτυχία, και ο master τερματίζει τη λειτουργία των executors.

9.7 Περαιτέρω μελέτη

Στο κεφάλαιο αυτό μελετήσαμε μια σειρά συστημάτων για τη διαχείριση πόρων. Παρουσιάσαμε τα αρχιτεκτονικά τους στοιχεία και τους μηχανισμούς με τους οποίους επιτυγχάνεται η εκτέλεση αλλά και η διαχείριση των πόρων. Αρχικά, παρουσιάσαμε κάποιους βασικούς όρους, όπως αυτόν του περιέκτη.

Διαπιστώσαμε ότι, παρά τις διαφορετικές προσεγγίσεις, ονομασίες και μηχανισμούς, όλα αυτά τα συστήματα βασίζονται στο μοντέλο master / slave, κατά το οποίο ένας κόμβος αναλαμβάνει τον συντονισμό της εκτέλεσης (master) και οι υπόλοιποι πραγματοποιούν την εκτέλεση (slaves). Είδαμε, επίσης, ότι σε πολλές περιπτώσεις υπάρχει πρόνοια για την κατάρρευση κόμβων, ώστε να υπάρχει υψηλή διαθεσιμότητα. Τότε, υπάρχουν πλέον του ενός masters σε κατάσταση αναμονής, οι οποίοι αναλαμβάνουν σε περίπτωση που ο διακομιστής που εκτελεί χρέη master καταρρεύσει.

Μελετήσαμε το YARN, το οποίο αποτελεί μηχανισμό του Hadoop, αλλά μπορεί να χρησιμοποιηθεί και στα πλαίσια άλλων εφαρμογών. Είδαμε μια σειρά εντολών για τη διαχείριση των εφαρμογών και των συστατικών τους στοιχείων μέσα από μια σειρά παραδειγμάτων. Κεντρική οντότητα στο YARN είναι ο ResourceManager.

Γνωρίσαμε το Mesos, ένα σύστημα που μεσουρανούσε την προηγούμενη δεκαετία ως μηχανισμός διαχείρισης πόρων σε ένα datacenter, καθώς παρείχε τη δυνατότητα στις εφαρμογές να αντιλαμβάνονται του πόρους μιας συστάδας όπως τους πόρους ενός υπολογιστή.

Πραγματοποιήσαμε μια εισαγωγή στο Kubernetes, το οποίο έχει κατασκευαστεί από την Google και, κατά τη στιγμή συγγραφής αυτού του βιβλίου, είναι το πλέον διαδεδομένο σύστημα διαχείρισης πόρων. Το Kubernetes βασίζεται στον μηχανισμό των περιεκτών και χρησιμοποιείται τόσο για ανάπτυξή τους σε μια συστάδα όσο και ενορχήστρωσή τους πραγματοποιώντας κατανομή φόρτου. Κεντρική έννοια στο Kubernetes είναι η άκατος (Pod).

Τέλος, για λόγους πληρότητας, παρουσιάσαμε και τον ενσωματωμένο μηχανισμό διαχείρισης πόρων του Apache Spark, ο οποίος μπορεί να χρησιμοποιηθεί σε βασικές διατάξεις.

Ο ενδιαφερόμενος αναγνώστης μπορεί να ανατρέξει στις αντίστοιχες δημοσιεύσεις για κάθε σύστημα. Επίσης, καθώς όλα τα συστήματα είναι ανοιχτού κώδικα, υλικό υπάρχει στους αντίστοιχους ιστοτόπους. Έτσι, για το YARN μπορεί κάποιος να μελετήσει τα [VMD⁺13] και [yar]. Για το Mesos μπορεί κανείς να συμβουλευτεί τα [HKZ⁺11, mes]. Περισσότερες πληροφορίες για το Spark υπάρχουν στα [ZXW⁺16] και [spa]. Για το Kubernetes υλικό υπάρχει στον ιστότοπο του συστήματος [kub]. Πολύ λεπτομερείς αναλύσεις των παραπάνω συστημάτων υπάρχουν και σε βιβλία. Ενδεικτικά, αναφέρουμε τα [ME14, Ign16, Kak15, CZ18, Pou23]. Τέλος, για τις τελευταίες ερευνητικές κατευθύνσεις αναφορικά με τα συστήματα που παρουν-

σιάστηκαν σε αυτό το κεφάλαιο, ο ενδιαφερόμενος αναγνώστης μπορεί να συμβουλευτεί μία σειρά ερευνητικών εργασιών [Car22, KLY21, PS21, KRG20, TZZ⁺20].

BIBΛΙΟΓΡΑΦΙΑ

- [Car22] Carmen Carrión. Kubernetes scheduling: Taxonomy, ongoing issues and challenges. *ACM Computing Surveys*, 55(7):1–37, 2022.
- [CZ18] Bill Chambers and Matei Zaharia. *Spark: The definitive guide: Big data processing made simple*. O'Reilly Media, Inc., 2018.
- [GZH⁺11] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*, 2011.
- [HKZ⁺11] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for {Fine-Grained} resource sharing in the data center. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, 2011.
- [Ign16] Roger Ignazio. *Mesos in action*. Simon and Schuster, 2016.
- [Kak15] Dharmesh Kakadia. *Apache Mesos Essentials*. Packt Publishing, 2015.
- [KLY21] Eunsook Kim, Kyungwoon Lee, and Chuck Yoo. On the resource management of kubernetes. In *2021 International Conference on Information Networking (ICOIN)*, pages 154–158. IEEE, 2021.
- [KRG20] Pradyumna Kaushik, Srinidhi Raghavendra, Madhusudhan Govindaraju, and Devesh Tiwari. Exploring the potential of using power as a first class parameter for resource allocation in apache mesos managed clouds. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, pages 216–226. IEEE, 2020.
- [kub] Kubernetes concepts. <https://kubernetes.io/docs/concepts/>. Accessed: 2023-09-30.
- [ME14] Arun C. Murthy and Doug Eadline. *Apache Hadoop YARN: moving beyond MapReduce and batch processing with Apache Hadoop 2*. Pearson Education, 2014.
- [mes] Apache mesos architecture. <https://mesos.apache.org/documentation/latest/architecture/>. Accessed: 2023-09-30.
- [Pou23] Nigel Poulton. *The kubernetes book*. NIGEL POULTON LTD, 2023.
- [PS21] Vaibhav Pandey and Poonam Saini. A heuristic method towards deadline-aware energy-efficient mapreduce scheduling problem in hadoop yarn. *Cluster Computing*, 24:683–699, 2021.

- [spa] Spark standalone mode - spark documentation. <https://spark.apache.org/docs/latest/spark-standalone.html>. Accessed: 2023-09-30.
- [TZZ⁺20] Zhuo Tang, Ailing Zeng, Xuedong Zhang, Li Yang, and Kenli Li. Dynamic memory-aware scheduling in spark computing environment. *Journal of Parallel and Distributed Computing*, 141:10–22, 2020.
- [VMD⁺13] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing*, pages 1–16, 2013.
- [yar] Apache hadoop yarn. <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>. Accessed: 2023-09-30.
- [ZXW⁺16] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, and et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

ΠΑΡΑΛΛΗΛΙΣΜΟΣ ΜΕ MAPREDUCE ΣΕ GPUS

Περιεχόμενα

10.1	Αποδοτικοί αλγόριθμοι στο MapReduce	237
10.1.1	Πολυπλοκότητα αλγορίθμων MapReduce	241
10.1.2	Κατώτερα όρια στον βαθμό αντιγραφής	243
10.1.3	Παραδείγματα εφαρμογής της μεθοδολογίας ανάλυσης	244
10.2	Θέματα αρχιτεκτονικής και προγραμματιστικού μοντέλου CUDA	247
10.2.1	Οργάνωση νημάτων	248
10.2.2	Ιεραρχίες μνήμης	250
10.2.3	Περισσότερα για το SIMT μοντέλο	251
10.3	Παραδείγματα διαχείρισης δεδομένων σε GPUs	252
10.3.1	Πολλαπλασιασμός πινάκων	253
10.3.2	Επεξεργασία πινάκων και εξαγωγή στατιστικών	255
10.3.3	Συχνές λανθασμένες αντιλήψεις	258
10.4	Περαιτέρω μελέτη	260
	ΒΙΒΛΙΟΓΡΑΦΙΑ	260



Σύνοψη

Το κεφάλαιο αυτό ασχολείται με τα παρακάτω θέματα:

- Ανάλυση αλγορίθμων MapReduce, έννοια της αντιγραφής και εύρεση κατώτερων ορίων και παραδείγματα.
- Αρχιτεκτονική του προγραμματιστικού μοντέλου CUDA με έμφαση σε θέματα οργάνωσης νημάτων, ιεραρχίας μνήμης και μοντέλου εκτέλεσης εντολών.
- Παραδείγματα διαχείρισης δεδομένων σε GPU.

Προαπαιτούμενη γνώση

Η προαπαιτούμενη γνώση περιέχεται ήδη σε προηγούμενα κεφάλαια αυτού του βιβλίου. Ωστόσο, ο αναγνώστης θα διευκολυνθεί εάν διαθέτει βασικές γνώσεις παράλληλου υπολογισμού, τεχνικών πολλαπλασιασμού πινάκων και διαχείρισης δεδομένων.

Εισαγωγή

Στο κεφάλαιο αυτό θα ασχοληθούμε με δύο βασικά θέματα. Αρχικά, θα ασχοληθούμε με κατάλληλο και αποδοτικό σχεδιασμό παράλληλων αλγορίθμων στο ευρύτερο υπολογιστικό πλαίσιο του MapReduce. Η βάση για τη συζήτηση αυτή είναι το 2ο κεφάλαιο του ελεύθερου βιβλίου των Jure Leskovec, Anand Rajaraman και Jeffrey D. Ullman με τίτλο *Mining of Massive Datasets* [LRU19]. Θα μας απασχολήσουν ερωτήματα όπως το πώς μπορούμε να μετρήσουμε την αποδοτικότητα της σχεδίασης ενός προγράμματος MapReduce και το εάν μπορούμε να έχουμε θεωρητικά όρια ώστε να είμαστε σε θέση να ισχυριστούμε την ύπαρξη ή μη βελτιστότητας.

Στη συνέχεια, θα ασχοληθούμε με έναν άλλο τρόπο παράλληλου υπολογισμού, τη χρήση καρτών γραφικών. Η συζήτηση στο κεφάλαιο αυτό χρησιμοποιεί το NVIDIA CUDA (Compute Unified Device Architecture) ως το μοντέλο αναφοράς για την αρχιτεκτονική και τον προγραμματισμό σε GPU. Οι ίδιες βασικές αρχές ισχύουν και για άλλους προμηθευτές, όπως AMD, και σε άλλα μοντέλα προγραμματισμού, όπως το OpenCL. Τα κύρια πλεονεκτήματα του CUDA είναι η συνεχής υποστήριξη και ανάπτυξή του, η ευρύτερη κοινότητά του και το γεγονός ότι είναι πιο φιλικό προς τον προγραμματιστή. Το υλικό για το συγκεκριμένο κεφάλαιο είναι βασισμένο στη διδακτορική διατριβή του Χρ. Μπέλλα (<https://ikee.lib.auth.gr/record/337473/files/GRI-2022-33918.pdf>) [Bel22] και σε βιβλία, όπως:

- *Programming Massively Parallel Processors: A Hands-on Approach* των Kirk και Hwu [KH17]
- *Programming on Parallel Machines* του Matloff [Mat17].

Οι κάρτες γραφικών (GPUs) είναι σήμερα η βασική μέθοδος εκπαίδευσης μοντέλων βαθιάς μάθησης. Δεν είναι μόνο οι πολλοί πυρήνες που παρέχουν, αλλά και το αυξημένο εύρος ζώ-

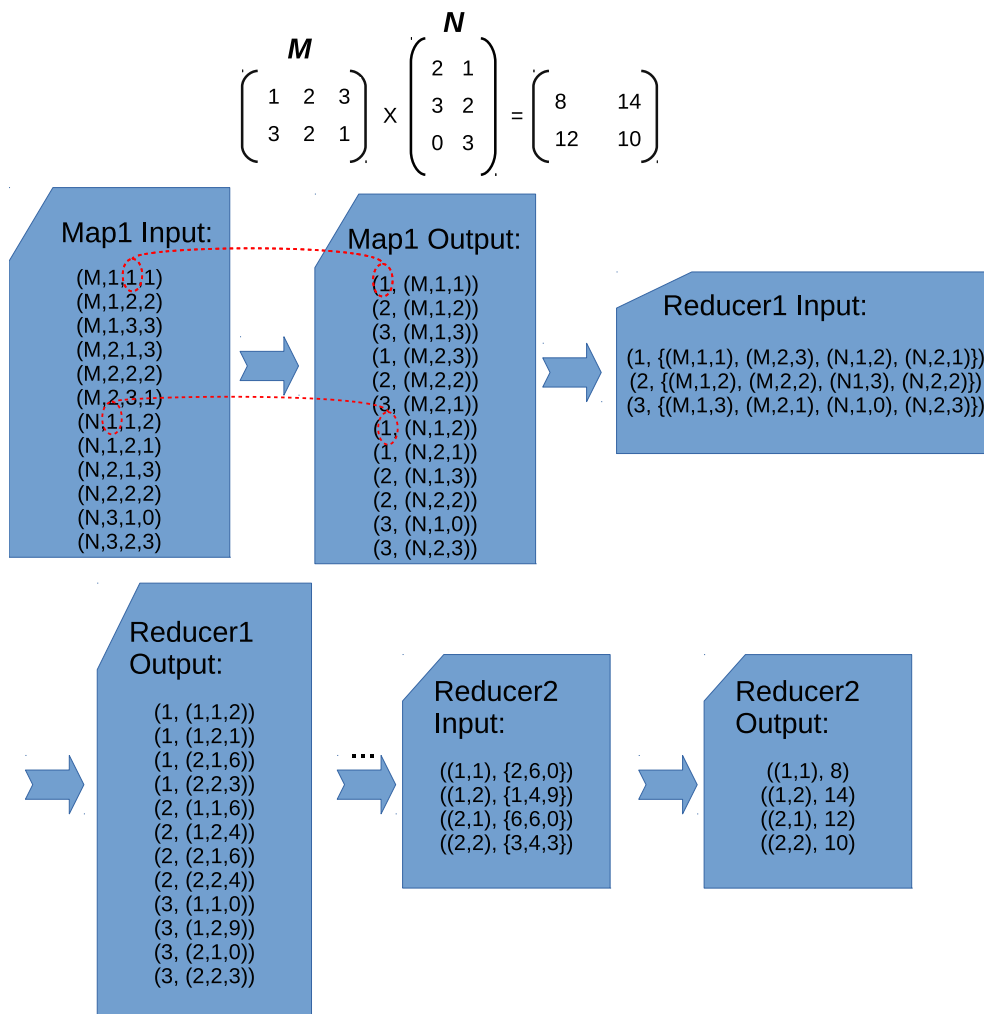
νης ανάμεσα στην κύρια μνήμη μιας κάρτας και των επεξεργαστών της σε σύγκριση με το αντίστοιχο εύρος ζώνης μιας σύγχρονης CPU. Ένα άλλο προτέρημά τους είναι η ενεργειακή απόδοση. Όμως, εκτελώντας υπολογισμούς σε GPUs δεν σημαίνει ότι καταργούνται οι CPUs ως εναλλακτικές. Το αντίθετο, λόγω των συμπληρωματικών πλεονεκτημάτων των δύο αυτών επιλογών, στην πράξη καλό είναι να συνδυάζονται.

Κλείνοντας, είναι απαραίτητο να διευκρινίσουμε ότι το όφελος του παραλληλισμού στη μείωση του χρόνου εκτέλεσης και στην επιτάχυνση προγραμμάτων περιορίζεται από το μη παραλληλοποιήσιμο κομμάτι του προγράμματος, κάτι που είναι γνωστό ως νόμος του Amdahl. Π.χ., εάν ένα πρόγραμμα τρέχει σε 10 δευτερόλεπτα σε έναν επεξεργαστή, εκ των οποίων τα 5 είναι μη παραλληλοποιήσιμο κομμάτι, τότε, εάν έχουμε γραμμική επιτάχυνση και 10 επεξεργαστές, το παραλληλοποιήσιμο κομμάτι μπορεί να πέσει στα 0,5 δευτερόλεπτα. Όμως, ο συνολικός χρόνος θα είναι 5,5 δευτερόλεπτα, δηλαδή επιτάχυνση κατά μόνο 1,81 φορές. Σε μια ακραία περίπτωση, ως ένα δεύτερο παράδειγμα, όπου μόνο το 10% του χρόνου εκτέλεσης ενός προγράμματος δεν είναι παραλληλοποιήσιμο, εάν με χρήση πολλών επεξεργαστών εκμηδενιστεί το παραλληλοποιήσιμο κομμάτι, η συνολική επιτάχυνση δεν μπορεί να ξεπεράσει τις 10 φορές.

10.1 Αποδοτικοί αλγόριθμοι στο MapReduce

Το ευρύτερο υπολογιστικό πλαίσιο που εισήγαγε το MapReduce, είτε υλοποιείται άμεσα από συστήματα, όπως το Hadoop, είτε χρησιμοποιείται μέσα από τις επεκτάσεις του, όπως το σύστημα Spark και Flink, διευκολύνει από τη μία πλευρά τον εύκολο παράλληλο υπολογισμό πολλών συναρτήσεων και αλγορίθμων, αλλά από την άλλη εγείρει νέα θέματα αναφορικά με την αποδοτική υλοποίηση. Π.χ., είναι γνωστοί δύο βασικοί αλγόριθμοι στο πλαίσιο MapReduce για τον πολλαπλασιασμό πινάκων.

Ο ένας αλγόριθμος αντιστοιχίζει μία αρχική εγγραφή του πίνακα, δηλαδή ένα στοιχείο του πίνακα, σε ένα ζεύγος κλειδιού-τιμής, αλλά απαιτεί δύο κύκλους εργασιών map και reduce. Ένα παράδειγμα παρουσιάζεται στο Σχήμα 10.1. Έστω M ένας πίνακας με το στοιχείο m_{ij} στη γραμμή i και στη στήλη j , και N ένας πίνακας με το στοιχείο n_{jk} στη γραμμή j και στη στήλη k . Τότε το γινόμενο $P = MN$ είναι ο πίνακας P , με το στοιχείο p_{ik} στη γραμμή i και στη στήλη k , όπου $p_{ik} = \sum_j m_{ij}n_{jk}$. Τα στοιχεία m_{ij} μπορούν να αναπαρασταθούν ως σχέση, όπως φαίνεται στο σχήμα, όπου η τελευταία τιμή σε κάθε τετράδα είναι η τιμή του στοιχείου. Π.χ., η τετράδα $(N, 3, 1, 0)$ υποδηλώνει ότι $n_{3,1} = 0$. Τα M και N είναι στην πραγματικότητα bits που δηλώνουν από ποιον από τους δύο πίνακες προέρχεται η τιμή. Σημειώνεται ότι μια τέτοια αναπαράσταση αφαιρώντας τα μηδενικά είναι κατάλληλη για αραιούς πίνακες. Για κάθε στοιχείο m_{ij} του πίνακα M η πρώτη map παράγει το ζεύγος κλειδιού-τιμής $(j, (M, i, m_{ij}))$. Ομοίως για κάθε στοιχείο n_{jk} του πίνακα N παράγει το ζεύγος κλειδιού-τιμής $(j, (N, k, n_{jk}))$. Έπειτα, η reduce για κάθε κλειδί j εξετάζει τη σχετική λίστα τιμών και εξάγει ένα ζεύγος κλειδιού-τιμής για κάθε ζεύγος τιμών, όπου η μία τιμή προέρχεται από τον M και η άλλη από τον N . Πιο συγκεκριμένα, από το ζεύγος (M, i, m_{ij}) και (N, k, n_{jk}) παράγεται ένα ζεύγος κλειδιού-τιμής με κλειδί το j και τιμή ίση με $(i, k, m_{ij}n_{jk})$, δηλαδή η τιμή είναι σύνθετη και περιλαμβάνει στο τέλος το γινόμενο αυτών των στοιχείων, $m_{ij}n_{jk}$. Η δεύτερη συνάρτηση map λαμβάνει υπόψη μόνο τις τιμές $(i, k, m_{ij}n_{jk})$ και θέτει τα δύο πρώτα στοιχεία



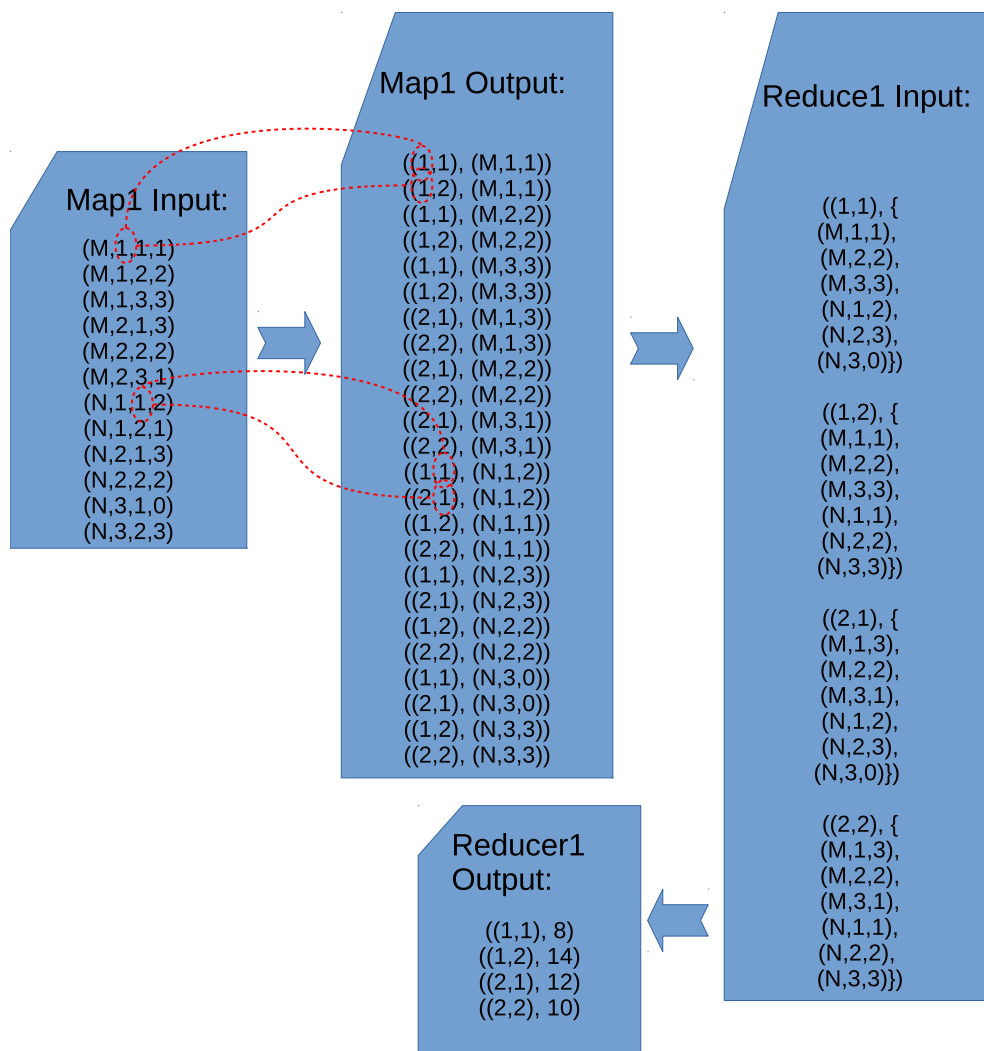
Σχήμα 10.1: Παράδειγμα πολλαπλασιασμού πινάκων σε δύο κύκλους εργασιών MapReduce.

ως κλειδί. Η δεύτερη reduce για κάθε κλειδί (i, k) παράγει το άθροισμα της λίστας των τιμών που σχετίζονται με το συγκεκριμένο κλειδί. Το αποτέλεσμα είναι ένα ζεύγος $((i, k), p_{ik})$.

Ο δεύτερος αλγόριθμος εκτελεί τον πολλαπλασιασμό σε έναν κύκλο εργασιών, αλλά παράγει πολλαπλά ζεύγη κλειδιού-τιμής για κάθε αρχικό στοιχείο των δύο πινάκων, όπως φαίνεται στο παράδειγμα του Σχήματος 10.2. Για κάθε στοιχείο m_{ij} του M η map παράγει όλα τα ζεύγη κλειδιού-τιμής $((i, k), (M, j, m_{ij}))$ για $k = 1, 2, \dots$, μέχρι το πλήθος των στηλών του N . Ομοίως, για κάθε στοιχείο n_{jk} του N παράγονται όλα τα ζεύγη κλειδιού-τιμής $((i, k), (N, j, n_{jk}))$ για $i = 1, 2, \dots$, μέχρι το πλήθος των γραμμών του M . Η reduce για κάθε κλειδί (i, k) θα έχει συσχετισθεί με μία λίστα με όλες τις τιμές (M, j, m_{ij}) και (N, j, n_{jk}) για όλες τις δυνατές τιμές του j . Η συνάρτηση Reduce πρέπει να πολλαπλασιάσει τις δύο τιμές στη λίστα, μία από τον M και μία από τον N , με την ίδια τιμή j για κάθε j . Έπειτα, αυτά τα γινόμενα αθροίζονται και το αποτέλεσμα γίνεται ζεύγος με το (i, k) στην έξοδο της συνάρτησης Reduce.

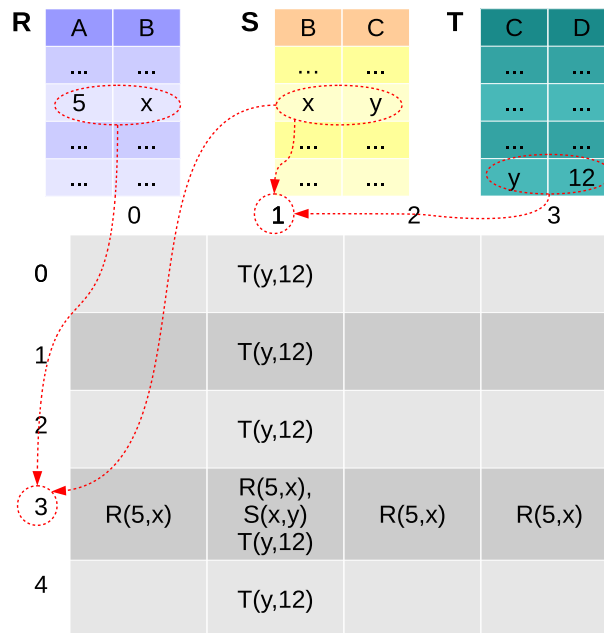
Υπάρχουν και άλλα χαρακτηριστικά των δύο εκδοχών που παίζουν ρόλο, αλλά και μόνο τα παραπάνω αρκούν για να αναδειχθεί το γεγονός ότι η απάντηση στο ποιος από τους δύο αλγορίθμους είναι αποδοτικότερος δεν είναι προφανής.

Πριν προχωρήσουμε, είναι σημαντικό να τονιστεί ότι αντίστοιχες εκδοχές εμφανίζονται ευρύτερα στη διαχείριση δεδομένων. Ένα χαρακτηριστικό και πιο σύνθετο παράδειγμα εί-



Σχήμα 10.2: Αντίστοιχο παράδειγμα πολλαπλασιασμού πινάκων σε έναν κύκλο MapReduce.

ναι η πολλαπλή σύνδεση ή σύνδεση πολλαπλών δρόμων (multi-way join). Έστω, π.χ., μια έκφραση σχεσιακής άλγεβρας με δύο φυσικές συνδέσεις πάνω σε τρεις σχέσεις R , S και T ως εξής: $R(A, B) \bowtie S(B, C) \bowtie T(C, D)$. Μπορούμε, προφανώς, να επιλέξουμε μια τεχνική που πρώτα εκτελεί τη μία σύνδεση και μετά την άλλη. Ο κλασικότερος τρόπος εκτέλεσης φυσικών συνδέσεων στο MapReduce είναι να παράγεται ένα ζεύγος κλειδιού-τιμής ανά πλειάδα εισόδου, όπου το κλειδί είναι η τιμή του πεδίου σύνδεσης. Με αυτόν τον τρόπο, πλειάδες από διαφορετικές σχέσεις που συνδέονται μεταξύ τους καταλήγουν στην ίδια εργασία reduce, η οποία μπορεί εύκολα να υπολογίσει το ακριβές τελικό αποτέλεσμα. Ένας εναλλακτικός τρόπος υπολογισμού αυτής της σύνδεσης είναι σε μία εργασία MapReduce, η οποία συνδέει τις τρεις σχέσεις απευθείας. Ας υποθέσουμε ότι σχεδιάζουμε να χρησιμοποιήσουμε k reducers για αυτόν τον υπολογισμό. Τότε, μπορούμε να διαλέξουμε νούμερα b και c που αντιπροσωπεύουν τον αριθμό των κάδων, όπου θα κατακερματίσουμε τις τιμές σύνδεσης B και C , αντίστοιχως, ούτως ώστε $bc = k$. Έστω h η συνάρτηση κατακερματισμού που αντιστοιχίζει τις τιμές B στους b κάδους, και έστω g μία άλλη συνάρτηση κατακερματισμού που στέλνει τις τιμές C στους c κάδους. Σε ένα τέτοιο σενάριο, μία πλειάδα από τη σχέση S , που περιέχει και τα δύο πεδία σύνδεσης, θα καταλήξει σε έναν reducer μόνο εφαρμόζοντας τις συναρτήσεις h και g στα χαρακτηριστικά B και C , αντίστοιχα. Όμως μία πλειάδα από τον πίνακα R θα



Σχήμα 10.3: Είκοσι εργασίες *reduce* μαζί εκτελούν μία σύνδεση τριών δρομών.

καταλήξει σε c reducers, καθώς, για την εγγύηση ορθών αποτελεσμάτων, θα πρέπει να αντιγραφεί σε όλους τους reducers που αντιστοιχούν στον κάδο που ορίζεται από την εφαρμογή της h στο πεδίο B , που είναι το μόνο πεδίο σύνδεσης που περιέχει η συγκεκριμένη πλειάδα. Ομοίως, και μία πλειάδα της T αποστέλλεται σε b reducers.

Παράδειγμα 10.1. Υποθέτουμε ότι $b = 5$ και $c=4$, ώστε $k=20$. Οι είκοσι reducers μπορούν να θεωρηθούν ως διατεταγμένοι σε ένα ορθογώνιο, όπου ο κάθετος άξονας αντιστοιχεί στις τιμές του b και ο οριζόντιος στις τιμές του c , όπως φαίνεται στο Σχήμα 10.3. Εκεί, βλέπουμε μία υποθετική πλειάδα $S(x, y)$ για την οποία ισχύει $h(x)=3$ και $g(y)=1$. Αυτή η πλειάδα στέλνεται από την εργασία *map* μόνο στον reducer (3,1). Μπορούμε επίσης να δούμε μία πλειάδα $R(5, x)$. Καθώς $h(x)=3$, αυτή η πλειάδα στέλνεται σε όλους τους reducers (3, i) για $i = 1, 2, 3, 4$. Τέλος, βλέπουμε μία πλειάδα $T(y, 12)$. Καθώς $g(y)=1$, αυτή η πλειάδα στέλνεται σε όλους τους reducers ($j, 1$) για $j = 1, 2, 3, 4, 5$. Παρατηρούμε ότι αυτές οι τρεις πλειάδες συνδέονται και συναντώνται σε ακριβώς έναν reducer, αυτόν με κλειδί (3, 1). □

Γενικότερα, κατά τον σχεδιασμό αλγορίθμων στο MapReduce, πρέπει να λαμβάνονται υπόψη τα ακόλουθα:

- το συνολικό μέγεθος των αντικειμένων εισόδου που επεξεργάζονται όλες οι εργασίες *map* ή *reduce*, το οποίο μπορεί να μετρηθεί είτε σε πλήθος εγγραφών είτε σε bytes,
- το μέγεθος της εισόδου που μία μεμονωμένη εργασία *reduce* καλείται να επεξεργαστεί σε μία κλήση της, το οποίο είναι ουσιαστικά το μέγεθος της λίστας τιμών που αντιστοιχεί σε ένα συγκεκριμένο κλειδί.

Το σκεπτικό πίσω από την επιλογή του μεγέθους της εισόδου ως ενδεικτική μετρική κόστους βασίζεται στο ότι, τις πιο πολλές φορές, αυτά τα δεδομένα μεταφέρονται μέσω δικτύου και η ταχύτητα μεταφοράς, π.χ. 1 Gb/second, είναι πολύ πιο μικρή από την ταχύτητα εκτέλεσης

εντολών από έναν συνηθισμένο επεξεργαστή, ενώ επίσης το δίκτυο εξυπηρετεί παράλληλα τη μεταφορά της εισόδου σε πολλές εργασίες και συνεπώς, εμφανίζεται το φαινόμενο της συμφόρησης. Αντίστοιχα, και η ταχύτητα μεταφοράς δεδομένων από τον σκληρό δίσκο (και ευρύτερα από τον μόνιμο αποθηκευτικό χώρο) στην κύρια μνήμη είναι σημαντικά μικρότερη από την ταχύτητα επεξεργασίας.

Επίσης, το μέγεθος εισόδου μίας εργασίας reduce είναι σημαντικό, καθώς ένα μικρό μέγεθος αναγκάζει πολλές φορές τη δημιουργία πολλών κλειδιών, ενώ μεγάλο μέγεθος οδηγεί ενδεχομένως στον χειρισμό δεδομένων που δεν χωρούν εξ ολοκλήρου στην κύρια μνήμη και έρχεται σε αντίθεση με το σκεπτικό επιτάχυνσης της εκτέλεσης μέσω παράλληλου υπολογισμού. Είναι σημαντικό μία εργασία reduce να μπορεί να καταχωρίζει όλα τα δεδομένα εισόδου της στην κύρια μνήμη.

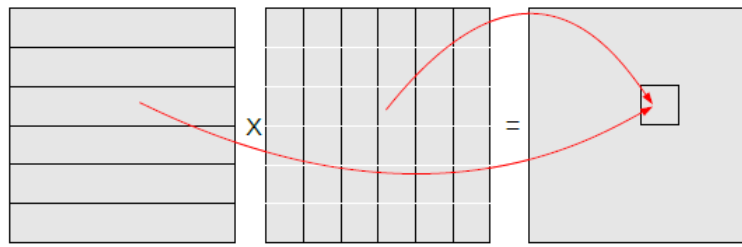
Παράδειγμα 10.2. Έστω ότι μια εφαρμογή θέλει να αναλύσει δεδομένα 5.000 φαρμάκων. Στα δεδομένα αυτά περιλαμβάνεται σύντομο ιατρικό ιστορικό των ασθενών που το έλαβαν και η εγγραφή για κάθε φάρμακο έχει μέγεθος 2 MB. Το συνολικό μέγεθος των δεδομένων είναι, συνεπώς, 10 GB. Η ανάλυση που επιδιώκεται αφορά τις αλληλεπιδράσεις μεταξύ φαρμάκων και, στην πράξη, περιλαμβάνει τον έλεγχο ανάμεσα σε όλα τα ζεύγη εγγραφών. Μία απλοϊκή προσέγγιση για τον έλεγχο δύο εγγραφών φαρμάκων i και j κατά το στάδιο reduce θα ήταν να δημιουργηθεί ένα κλειδί (i, j) στη map τόσο για την εγγραφή i όσο και για την εγγραφή j . Αυτό αυτομάτως συνεπάγεται ότι για κάθε εγγραφή i δημιουργούνται 4.999 κλειδιά στη map. Συνολικά, ανάμεσα στη map και στη reduce μεταφέρονται $5.000 * 4.999 * 2$ MBs, δηλαδή περίπου 50 TBs. Αυτός ο όγκος δεδομένων είναι ιδιαίτερα αυξημένος για ένα δίκτυο με ταχύτητα μεταφοράς της τάξης του 1 Gb/second και η επεξεργασία αυτών των λίγων χιλιάδων εγγραφών καθίσταται ιδιαίτερα χρονοβόρα.

Μία πιο προσεγμένη υλοποίηση θα ανέθετε σε κάθε εργασία reduce τον έλεγχο περισσότερων ζευγών φαρμάκων. Έστω ότι τα φάρμακα μοιράζονται σε 20 ισομεγέθεις ομάδες των 250 εγγραφών η κάθε μία. Τότε, η έξοδος της map για κάθε εγγραφή φαρμάκου i θα είναι το ζεύγος (g_i, g_j) , $j \neq i$, όπου g_i η ομάδα του i και g_j κάθε άλλη ομάδα. Με αυτόν τον τρόπο, κάθε εγγραφή φαρμάκου στη map θα παραγάγει μόνο 19 κλειδιά εξόδου, μειώνοντας τον όγκο δεδομένων που μεταφέρονται από τη map στη reduce κατά 2 τάξεις μεγέθους: $5.000 * 19 * 2$ MBs, δηλαδή 190 GBs. Όμως, κάθε reducer επιβαρύνεται πλέον περισσότερο δεχόμενος ως είσοδο αντί για δύο εγγραφές, όπως πριν, δύο ομάδες εγγραφών συνολικού μεγέθους 1GB (από 4MB). □

Αυτό το παράδειγμα είναι αρκετό για να δείξει ότι το συνολικό μέγεθος των αντικειμένων εισόδου μίας εργασίας map ή reduce, στη γενική περίπτωση, εξαρτάται και μάλιστα μπορεί να συσχετίζεται αρνητικά με το μέγεθος εισόδου που μία μεμονωμένη εργασία reduce καλείται να επεξεργαστεί. Αυτή η παρατήρηση οδήγησε στην ανάλυση των αλγορίθμων και των προβλημάτων στο πλαίσιο MapReduce, που παρουσιάζονται παρακάτω.

10.1.1 Πολυπλοκότητα αλγορίθμων MapReduce

Οι δύο παράγοντες που αναφέρθηκαν στην προηγούμενη ενότητα, δηλαδή το συνολικό μέγεθος εισόδου της map και της reduce και το μέγεθος εισόδου μίας μεμονωμένης εργασίας re-



Σχήμα 10.4: Χωρίζοντας τους πίνακες σε ζώνες σε έναν γενικό αλγόριθμο πολλαπλασιασμού πινάκων ενός περάσματος.

duce, έχουν οδηγήσει στην ανάπτυξη μιας θεωρίας πολυπλοκότητας για αλγορίθμους MapReduce, που βασίζεται σε δύο μετρικές: στον βαθμό αντιγραφής, που συμβολίζεται ως r , και στο μέγιστο δυνατό μέγεθος του reducer, το οποίο συμβολίζουμε με q . Η πρώτη μετρική ορίζεται ως το πλήθος των ζευγών κλειδιού-τιμής που παράγονται από όλες τις εργασίες map και λειτουργούν ως είσοδος σε όλες τις εργασίες reduce, διαιρούμενο με το πλήθος εισόδου της map. Με άλλα λόγια, ο βαθμός αντιγραφής είναι η μέση επικοινωνία από εργασίες map σε εργασίες reduce ανά μονάδα εισόδου. Επίσης, το q ορίζεται ως το μέγιστο πλήθος τιμών που μπορεί να εμφανίζεται στη λίστα ενός οποιουδήποτε κλειδιού.

Για παράδειγμα, σε έναν αλγόριθμο πολλαπλασιασμού δύο πινάκων $n \times n$ M και N ενός κύκλου MapReduce, όπου κάθε στοιχείο εξόδου υπολογίζεται από έναν διαφορετικό reducer, τότε κάθε στοιχείο καθενός από τους δύο αρχικούς πίνακες εισόδου πρέπει να αντιγραφεί σε n κλειδιά. Αυτό συμβαίνει επειδή για κάθε στοιχείο m_{ij} του M παράγονται n ζεύγη κλειδιού-τιμής, ένα για κάθε ζεύγος (i, k) , όπου $1 \leq k \leq n$. Ομοίως, για κάθε στοιχείο του άλλου πίνακα N , n_{jk} , παράγονται n ζεύγη τιμών-κλειδιών (i, k) για κάθε i , όπου $1 \leq i \leq n$. Συνεπώς, $r = n$. Επίσης, $q = 2n$, καθώς για κάθε κλειδί (i, k) υπάρχουν n τιμές από κάθε πίνακα.

Το παραπάνω ζεύγος τιμών r και q αποτελεί έναν τρόπο αποτίμησης της αποδοτικότητας ενός αλγορίθμου MapReduce. Αλλά, οφείλουμε να τονίσουμε ότι αντιπροσωπεύει μόνο έναν συγκεκριμένο αλγόριθμο για πολλαπλασιασμό πινάκων ενός περάσματος, και το σημαντικό είναι, όπως θα δούμε, πως για όλους ισχύει $qr \geq 2n^2$.

Πριν από αυτό το σημείο, θα εξετάσουμε έναν γενικό αλγόριθμο πολλαπλασιασμού δύο πινάκων M και N , που για ευκολία θα συνεχίσουμε να τους θεωρούμε τετραγωνικούς $n \times n$. Στον γενικό αλγόριθμο, ένας reducer δεν θα είναι υπεύθυνος για την παραγωγή ενός μόνο στοιχείου του πίνακα εξόδου, αλλά για περισσότερα. Για να το πετύχουμε αυτό, μπορούμε να ομαδοποιήσουμε τις γραμμές και στήλες από τους δύο πίνακες εισόδου σε ζώνες. Το κλειδί που λαμβάνει κάθε reducer είναι πάλι ένα ζεύγος, που περιλαμβάνει μία ζώνη από γραμμές του πρώτου πίνακα και μία ζώνη στηλών του δεύτερου πίνακα. Με αυτόν τον τρόπο, ένας reducer είναι σε θέση να παραγάγει ένα τετράγωνο στοιχείων του πίνακα εξόδου, όπως φαίνεται στο Σχήμα 10.4.

Πιο αναλυτικά, ομαδοποιούμε τις γραμμές του M σε g ζώνες n/g σειρών η κάθε μία και τις στήλες του N σε g ζώνες με n/g στήλες η καθεμία. Παράδειγμα αυτής της ομαδοποίησης απεικονίζεται στο σχήμα. Τα κλειδιά αντιστοιχούν σε δύο ομάδες (ζώνες), μία από τον M και μία από τον N . Παρόμοιο σκεπτικό χρησιμοποιήσαμε και στο Παράδειγμα 10.2.

Στη συνάρτηση Map, για κάθε στοιχείο του M δημιουργούνται g ζεύγη κλειδιού-τιμής. Η τιμή σε κάθε περίπτωση είναι το ίδιο το στοιχείο μαζί με τον αριθμό της αρχικής γραμμής και στήλης, ώστε να μπορεί να αναγνωρισθεί από τη συνάρτηση Reduce. Το κλειδί είναι το ζευγάρι της ομάδας στην οποία ανήκει το στοιχείο, σε συνδυασμό με οποιαδήποτε από τις ομάδες του πίνακα N , από το 1 έως το g . Ομοίως, για κάθε στοιχείο του N η συνάρτηση Map δημιουργεί επίσης g ζεύγη κλειδιού-τιμής, όπου το κλειδί είναι το ζεύγος της ομάδας αυτού του στοιχείου σε συνδυασμό με οποιαδήποτε από τις g ομάδες του M , και η τιμή είναι το ίδιο το στοιχείο συν τη γραμμή και τη στήλη του.

Στη συνάρτηση Reduce, ο reducer που αντιστοιχεί στο κλειδί (i, j) , όπου i είναι μία ομάδα του M και j είναι μία ομάδα του N , παίρνει μία λίστα τιμών που αποτελείται από όλα τα στοιχεία στην i -οστή ομάδα του M και την j -οστή ομάδα του N . Έχει, έτσι, όλες τις τιμές που χρειάζεται για να υπολογίσει τα στοιχεία του πίνακα εξόδου, των οποίων η γραμμή είναι μία από αυτές τις γραμμές που περιλαμβάνει η i -οστή ζώνη του M και των οποίων η στήλη είναι μία από αυτές που περιλαμβάνονται στην j -οστή ζώνη του N . Για παράδειγμα, το Σχήμα 10.4 υποδεικνύει την τρίτη ομάδα του πίνακα M και την τέταρτη ομάδα του N , που συνδυάζονται για να υπολογισθεί ένα τετράγωνο του πίνακα εξόδου στον reducer $(3, 4)$.

Κάθε reducer πρέπει να μπορεί να λαμβάνει $n(n/g)$ στοιχεία από κάθε έναν από τους δύο πίνακες, οπότε $q = 2n^2/g$. Ο βαθμός αντιγραφής είναι g , καθώς κάθε στοιχείο κάθε πίνακα αποστέλλεται σε g reducers. Δηλαδή, $r = g$. Συνδυάζοντας το $r = g$ με το $q = 2n^2/g$, μπορούμε να συμπεράνουμε ότι $r = 2n^2/q$. Δηλαδή, ο βαθμός αντιγραφής μεταβάλλεται αντιστρόφως ανάλογα με το μέγεθος του reducer.

10.1.2 Κατώτερα όρια στον βαθμό αντιγραφής

Ένα ερώτημα που τίθεται είναι το κατά πόσο αυτός ο αλγόριθμος πολλαπλασιασμού πινάκων ενός περάσματος είναι ο καλύτερος που μπορεί να υπάρξει αναφορικά με το κόστος εισόδου της Reduce, όπως αυτό αποτυπώνεται μέσω της μετρικής r , δεδομένων των περιορισμών στο μέγιστο μέγεθος ενός reducer q . Για να απαντηθεί αυτό το ερώτημα, μπορούμε να χρησιμοποιήσουμε μια μεθοδολογία πέντε βημάτων:

1. Αποδεικνύουμε ένα ανώτερο όριο για το πόσες εξόδους μπορεί ένας reducer με q εισόδους να καλύψει. Ονομάζουμε αυτό το όριο $g(q)$. Αυτό το βήμα μπορεί να είναι δύσκολο, αλλά για παραδείγματα όπως ο πολλαπλασιασμός πινάκων είναι στην πραγματικότητα αρκετά απλό.
2. Προσδιορίζουμε τον συνολικό αριθμό των εξόδων που παράγονται από το πρόβλημα.
3. Έστω ότι υπάρχουν k reducers και ο i -οστός reducer έχει εισόδους $q_i < q$. Παρατηρούμε ότι το $\sum_{i=1}^k g(q_i)$ δεν πρέπει να είναι μικρότερο από τον αριθμό των εξόδων που υπολογίστηκε στο βήμα (2).
4. Χειριζόμαστε την ανισότητα από το (3) για να πάρουμε ένα κατώτερο όριο στο $\sum_{i=1}^k q_i$. Συχνά, το τέχνασμα που χρησιμοποιείται σε αυτό το βήμα είναι η αντικατάσταση ορισμένων παραγόντων του q_i από το άνω όριο q , αλλά αφήνοντας έναν μόνο παράγοντα q_i στον όρο για το i .

5. Καθώς το $\sum_{i=1}^k q_i$ είναι η συνολική επικοινωνία από εργασίες Map σε εργασίες Reduce, διαιρούμε το κάτω όριο από το (4) σε αυτήν την ποσότητα με τον αριθμό των εισόδων. Το αποτέλεσμα είναι ένα κατώτερο όριο στον βαθμό αντιγραφής.

Στο συγκεκριμένο παράδειγμα, το μέγιστο πλήθος στοιχείων εξόδου από έναν reducer επιτυγχάνεται όταν ένας reducer λαμβάνει k πλήρεις γραμμές του M και k πλήρεις στήλες του N . Τότε $q = 2nk$ και καλύπτονται k^2 έξοδοι. Δηλαδή, το $g(q)$, ο μέγιστος αριθμός των εξόδων που καλύπτονται από έναν reducer που λαμβάνει q εισόδους, είναι $q^2/4n^2$. Αυτό αποτελεί το πρώτο βήμα της μεθοδολογίας.

Για το βήμα (2), γνωρίζουμε ότι ο αριθμός των εξόδων είναι n^2 . Στο βήμα (3) παρατηρούμε ότι εάν υπάρχουν k reducers, με τον i -οστό reducer να λαμβάνει $q_i \leq q$ εισόδους, τότε

$$\sum_{i=1}^k q_i^2/4n^2 \geq n^2$$

ή

$$\sum_{i=1}^k q_i^2 \geq 4n^4$$

Από αυτήν την ανισότητα και δεδομένου ότι $q_i < q$, μπορούμε να πάρουμε:

$$q \sum_{i=1}^k q_i \geq 4n^4$$

ενώ το μέγεθος της εισόδου είναι n^2 και $r = \frac{\sum_{i=1}^k q_i}{n^2}$, οπότε προκύπτει ότι:

$$r \geq 2n^2/q$$

Η ερμηνεία της παραπάνω έκφρασης είναι ότι δεν μπορεί να υπάρξει αλγόριθμος πολλαπλασιασμού δύο πινάκων σε ένα πέρασμα με r μικρότερο από αυτό που ήδη πετύχαμε με την τεχνική ομαδοποίησης σε g ομάδες γραμμών και στηλών, σύμφωνα με το Σχήμα 10.4.

Η παραπάνω μεθοδολογία ανάλυσης συγκεκριμένων αλγορίθμων και προβλημάτων, ανεξάρτητα από συγκεκριμένο αλγόριθμο υλοποίησης, γενικεύεται σε πολλές άλλες περιπτώσεις. Εάν, για παράδειγμα, εφαρμοστεί στον αλγόριθμο πολλαπλασιασμού πινάκων δύο περασμάτων, είναι ενδιαφέρον ότι το συνολικό κόστος επικοινωνίας, όπως αποτυπώνεται από τη μετρική r , είναι μικρότερο για το ίδιο q .

10.1.3 Παραδείγματα εφαρμογής της μεθοδολογίας ανάλυσης

Θα εξετάσουμε δύο παραδείγματα εφαρμογής της μεθοδολογίας. Το πρώτο συνεχίζει το Παράδειγμα 10.2. Έστω, λοιπόν, ένα σύνολο δεδομένων με n εγγραφές που χωρίζεται σε g ομάδες με n/g εγγραφές η κάθε ομάδα, ενώ θέλουμε να συγκρίνουμε όλες τις εγγραφές μεταξύ τους. Ένας reducer αναλαμβάνει 2 ομάδες g_i και g_j , οπότε $q = 2n/g$. Κάθε εγγραφή αποστέλλεται σε $g - 1$, άρα προσεγγιστικά $r \approx g$. Αντικαθιστώντας το g στην προηγούμενη σχέση,

προκύπτει ότι $r = 2n/q$.

Σημειώνουμε ότι δεν υπάρχει ξεχωριστός reducer για τη σύγκριση των εγγραφών της ίδιας ομάδας, καθώς είναι εύκολο αυτό να ανατεθεί με συγκεκριμένο τρόπο σε έναν από τους υπάρχοντες reducers. Για παράδειγμα, ο reducer του ζεύγους (g_i, g_{i+1}) μπορεί να θεωρηθεί υπεύθυνος για τη σύγκριση των εγγραφών εντός της g_i .

Το ερώτημα που τίθεται είναι το πόσο κοντά είναι αυτή η συμπεριφορά στη βέλτιστη. Αρχικά, θα ακολουθήσουμε ένα απλό σκεπτικό πέρα από την προτεινόμενη μεθοδολογία εύρεσης του κατώτερου ορίου αντιγραφής. Θεωρούμε ότι κάθε εγγραφή i πρέπει να συγκριθεί με τις υπόλοιπες εγγραφές σε τουλάχιστον έναν reducer μεγέθους q . Σε κάθε έναν reducer η εγγραφή i μπορεί να συγκριθεί με το πολύ $q - 1$ άλλες εγγραφές, οπότε πρέπει να αποσταλεί σε τουλάχιστον $(n - 1)/(q - 1)$ reducers, οπότε $r \geq \lceil (n - 1)/(q - 1) \rceil$, που προσεγγιστικά δίνει $r \geq \lceil n/q \rceil$, δηλαδή το μισό από ό,τι η μέθοδος στο Παράδειγμα 10.2.

Η εφαρμογή της μεθοδολογίας υπολογισμού του κατώτερου ορίου καταλήγει στο ίδιο αποτέλεσμα. Στο πρώτο βήμα, ορίζεται το $g(q)$ [προσοχή στη διαφορετική χρήση του συμβόλου g : το $g(q)$ δεν πρέπει να συγχέεται με το πλήθος των ομάδων g]. Σε κάθε περίπτωση, όταν ένας reducer λαμβάνει q εγγραφές, υπολογίζει προσεγγιστικά $q^2/2$ ζεύγη. Όλα τα ζεύγη που πρέπει να υπολογιστούν στην έξοδο είναι $n^2/2$.

Στο τρίτο βήμα παρατηρούμε ότι, εάν υπάρχουν k reducers, με τον i -οστό reducer να λαμβάνει $q_i \leq q$ εισόδους, τότε

$$\sum_{i=1}^k q_i^2/2 \geq n^2/2$$

ή

$$\sum_{i=1}^k q_i^2 \geq n^2$$

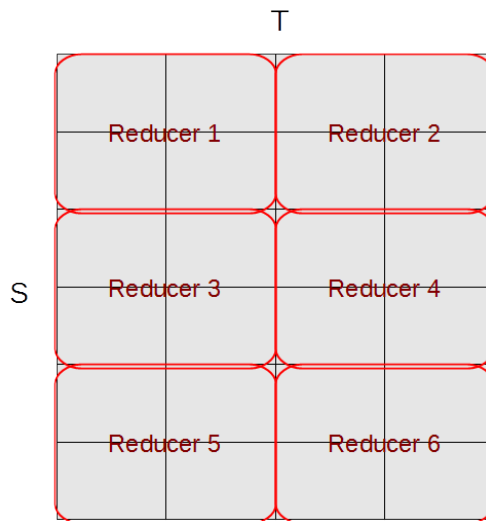
Ακριβώς αντίστοιχα με το προηγούμενο παράδειγμα εφαρμογής της μεθοδολογίας, από αυτήν την ανισότητα, και δεδομένου ότι $q_i < q$, μπορούμε να πάρουμε:

$$q \sum_{i=1}^k q_i \geq n^2$$

Για τον βαθμό αντιγραφής ισχύει $r = \sum_{i=1}^k q_i/n$. Οπότε, προκύπτει ότι:

$$r \geq n/q$$

Το δεύτερο παράδειγμα έχει να κάνει με μια γενική σύνδεση δύο σχέσεων $S \bowtie_{\theta} T$. Η διαφορά με το προηγούμενο παράδειγμα είναι ότι χρειάζεται η σύγκριση όλων των ζευγών, όπου κάθε μέρος του ζεύγους έρχεται από διαφορετικό πληθυσμό και όχι από τον ίδιο. Για την καλύτερη κατανόηση του προβλήματος, θα χρησιμοποιήσουμε την έννοια του πίνακα σύνδεσης (join matrix), όπου κάθε γραμμή του πίνακα αντιστοιχεί σε μία εγγραφή (ή ομάδα εγγραφών) της σχέσης S , και κάθε στήλη σε μία εγγραφή (ή ομάδα εγγραφών) της σχέσης T . Ένα κελί του πίνακα αντιστοιχεί σε ένα ζεύγος και θα πρέπει, εάν έχουμε k reducers στη διάθεσή μας, να



Σχήμα 10.5: Παράδειγμα πίνακα σύνδεσης και ανάθεσης εργασίας γενικής σύνδεσης σε reducers.

καλυφθεί πλήρως ο πίνακας. Ένα παράδειγμα δίνεται στο Σχήμα 10.5, όπου $|S| = 6$, $|T| = 4$ και $k = 6$.

Στο συγκεκριμένο πρόβλημα, το μέγεθος της εισόδου είναι $|S| + |T|$ και της εξόδου $|S||T|$. Έστω ότι ένας reducer λαμβάνει q εγγραφές εισόδου. Τότε, παράγει το μεγαλύτερο πλήθος εγγραφών εξόδου όταν οι μισές εγγραφές είναι από την S και οι άλλες μισές από την T . Π.χ., εάν λάβει 10 εγγραφές, 5 από τη μία σχέση και 5 από την άλλη, μπορεί να παραγάγει 25 εγγραφές εξόδου. Αλλά, εάν είναι 6 και 4, τότε το μέγεθος εξόδου μειώνεται στα 24. Όσο μεγαλώνει η ανισοκατανομή, τόσο λιγότερες πλειάδες εξόδου παράγονται: με 9 εγγραφές από την S και 1 από την T , το μέγεθος εξόδου είναι 9. Συνεπώς, $g(q) = (q/2)^2 = q^2/4$.

Για να καλυφθεί όλη η απαιτούμενη έξοδος, θα πρέπει:

$$\sum_{i=1}^k g(q_i) \geq |S||T|$$

που μπορεί να γραφεί και ως:

$$\sum_{i=1}^k q_i \frac{g(q_i)}{q_i} \geq |S||T|$$

Δεδομένου ότι ο ρυθμός αύξησης της $g(q)$ είναι μεγαλύτερος από αυτόν της q , που είναι το μέγιστο μέγεθος ενός reducer, η παραπάνω ανισότητα γίνεται

$$\sum_{i=1}^k q_i \frac{g(q)}{q} \geq |S||T|$$

Διαιρώντας με το μέγεθος εισόδου και αντικαθιστώντας τη μεταβλητή $g(q)$ με $q^2/4$, έχουμε

$$\frac{\sum_{i=1}^k q_i}{(|S| + |T|)} \frac{q}{4} \geq \frac{|S||T|}{|S| + |T|}$$

από όπου εξάγεται ότι:

$$r \geq \frac{4}{q} \frac{|S||T|}{|S| + |T|}$$

Έστω ότι αναπτύσσουμε μια τεχνική η οποία διαμοιράζει τον πίνακα συνδέσεων με τέτοιον τρόπο, ώστε κάθε reducer να καλύπτει ένα τετράγωνο με πλευρά $\alpha = \sqrt{\frac{|S||T|}{k}}$, ακριβώς όπως στο Σχήμα 10.5. Αυτό προφανώς δεν είναι πάντα εφικτό, αλλά ας υποθέσουμε ότι είναι τέτοια τα μεγέθη των σχέσεων, που αυτό μπορεί να πραγματοποιηθεί. Σε αυτήν την περίπτωση, ας υποθέσουμε ότι $|S| = \alpha c_S$ και $|T| = \alpha c_T$. Στο σχήμα, $c_S = 3$ και $c_T = 2$. Κάθε εγγραφή της σχέσης S αντιγράφεται στη map c_T φορές και, αντίστοιχα, κάθε εγγραφή της T αντιγράφεται c_S φορές. Οπότε, ο βαθμός αντιγραφής της συγκεκριμένης τεχνικής είναι:

$$r = \frac{|S|c_T + |T|c_S}{|S| + |T|} = \frac{2}{\alpha} \frac{|S||T|}{|S| + |T|}$$

Όμως, επειδή $q = 2\alpha$, προκύπτει:

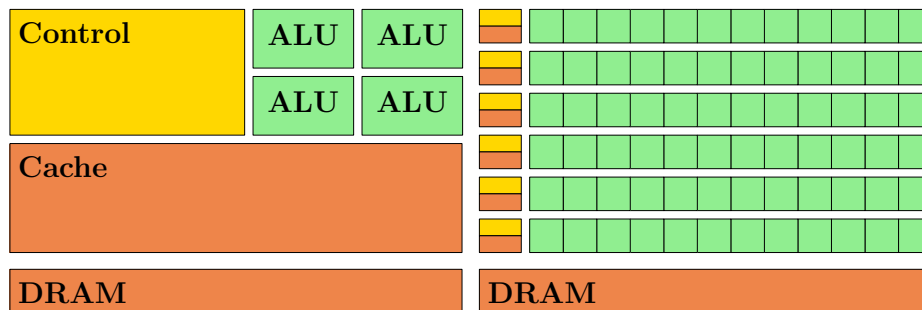
$$r = \frac{4}{q} \frac{|S||T|}{|S| + |T|}$$

Με άλλα λόγια, η συγκεκριμένη ανάθεση εργασίας στους reducers είναι η βέλτιστη αναφορικά με τον βαθμό αντιγραφής, καθώς αυτός ο βαθμός αντιστοιχεί στο κατώτερο όριο του συγκεκριμένου προβλήματος.

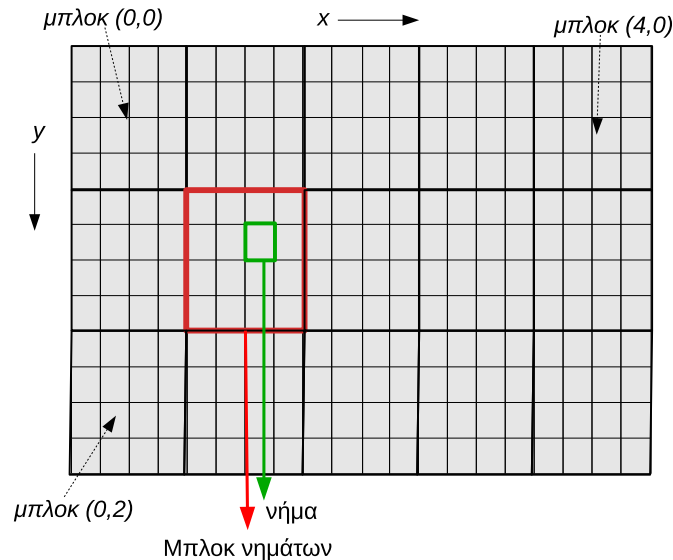
10.2 Θέματα αρχιτεκτονικής και προγραμματιστικού μοντέλου CUDA

Μια σύγχρονη CPU περιέχει πολλούς πυρήνες, κάθε ένας από τους οποίους μπορεί να εκτελεί εντολές ανεξάρτητα. Το σύνολο εντολών επιτρέπει την ταυτόχρονη εκτέλεση πολλαπλών νημάτων, αλλά έχει περιορισμένο παραλληλισμό λόγω των εξαρτήσεων μεταξύ των νημάτων αναφορικά με τα δεδομένα που επεξεργάζονται και την προσπέλασή τους. Για να αυξηθεί η υπολογιστική ισχύς, οι CPUs έχουν εξοπλιστεί με ακόμη περισσότερους πυρήνες. Ωστόσο, όσο αυξάνεται ο αριθμός των πυρήνων τόσο αυξάνεται και ο χρόνος που απαιτείται για τη μετάβαση από το ένα νήμα στο άλλο. Αυτό το φαινόμενο είναι γνωστό ως *καθυστέρηση* (latency). Ο χρόνος καθυστέρησης αυξάνεται γραμμικά με τον αριθμό των πυρήνων και, έτσι, γρήγορα ο αυξημένος βαθμός παραλληλισμού γίνεται αιτία συμφόρησης που περιορίζει την απόδοση. Από την άλλη πλευρά, οι GPUs έχουν σχεδιαστεί για παράλληλους υπολογισμούς και διαφέρουν θεμελιωδώς από τις CPUs στο ότι χρησιμοποιούν περισσότερους πόρους υλικού για υπολογισμούς και λιγότερους πόρους για κρυφές μνήμες και έλεγχο ροής. Όπως φαίνεται στο Σχήμα 10.6, μια GPU έχει πολύ περισσότερες μονάδες επεξεργασίας, που αναφέρονται επίσης ως πυρήνες, αλλά λιγότερη κρυφή μνήμη και μονάδες ελέγχου σε σύγκριση με μια CPU. Αυτή η σχεδιαστική απόφαση ευνοεί την εκτέλεση πράξεων και προγραμμάτων υψηλής έντασης υπολογισμού, σε αντίθεση με προγράμματα με πολύπλοκη διακλάδωση.

Οι GPUs χωρίζουν τους πυρήνες σε ομάδες που ονομάζονται *πολυεπεξεργαστές ροής* (Streaming Multiprocessors, SMs). Κάθε SM περιλαμβάνει επίσης άλλες μονάδες, όπως ALU, μονάδες εντολών, κρυφές μνήμες για λειτουργίες φόρτωσης/αποθήκευσης και ακολουθεί το πλαίσιο παράλληλης επεξεργασίας *μία εντολή - πολλαπλά νήματα* (Single Instruction Multiple



Σχήμα 10.6: Διαφορές στην αρχιτεκτονική μιας CPU (αριστερά) και GPU (δεξιά) [NVI].



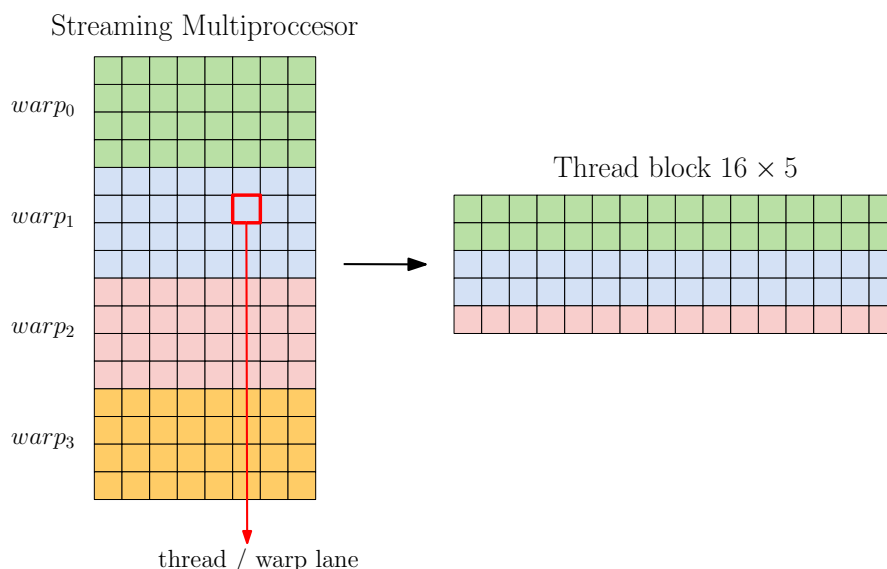
Σχήμα 10.7: Παράδειγμα ενός δισδιάστατου πλέγματος πυρήνα με $5 \times 3 = 15$ blocks, όπου το κάθε ένα αποτελείται από 16 νήματα σε επίσης 2 διαστάσεις.

Threads, SIMT). Συνολικά, οι πυρήνες GPU εκτελούν την ίδια εντολή σε διαφορετικά στοιχεία δεδομένων, ενώ οι πυρήνες της CPU συνήθως εκτελούν διαφορετικές εντολές, αυξάνοντας έτσι την ανάγκη για πιο προηγμένο έλεγχο ροής προγραμμάτων. Γενικά, οι σύγχρονες CPUs έχουν σχεδιαστεί για να ελαχιστοποιούν τον χρόνο καθυστέρησης μεμονωμένων νημάτων, ενώ οι GPUs στοχεύουν στη μεγιστοποίηση της απόδοσης και του ρυθμού πράξεων ανά μονάδα χρόνου.

Στην ορολογία της CUDA, η CPU και η κύρια μνήμη αναφέρονται ως *κεντρικός υπολογιστής* ή *host*, ενώ η GPU και η δική της μνήμη αναφέρονται ως *device*. Στο κεφάλαιο αυτό, οι όροι CPU και κεντρικός υπολογιστής (αντίστοιχα GPU και συσκευή) χρησιμοποιούνται εναλλακτικά.

10.2.1 Οργάνωση νημάτων

Στην CUDA τα νήματα οργανώνονται σε λογικές ομάδες που ονομάζονται *block νημάτων*. Ουσιαστικά, κάθε block νημάτων είναι μια σειρά νημάτων με έως και τρεις διαστάσεις. Κάθε συνάρτηση που επεξεργάζεται παράλληλα η GPU ονομάζεται *πυρήνας* (kernel). Κάθε πυρήνας εκτελείται από πολλαπλά block νημάτων, τα οποία σχηματίζουν το *πλέγμα πυρήνα*. Το πλέγμα του πυρήνα μπορεί να θεωρηθεί ως μια σειρά από block νημάτων με έως και τρεις



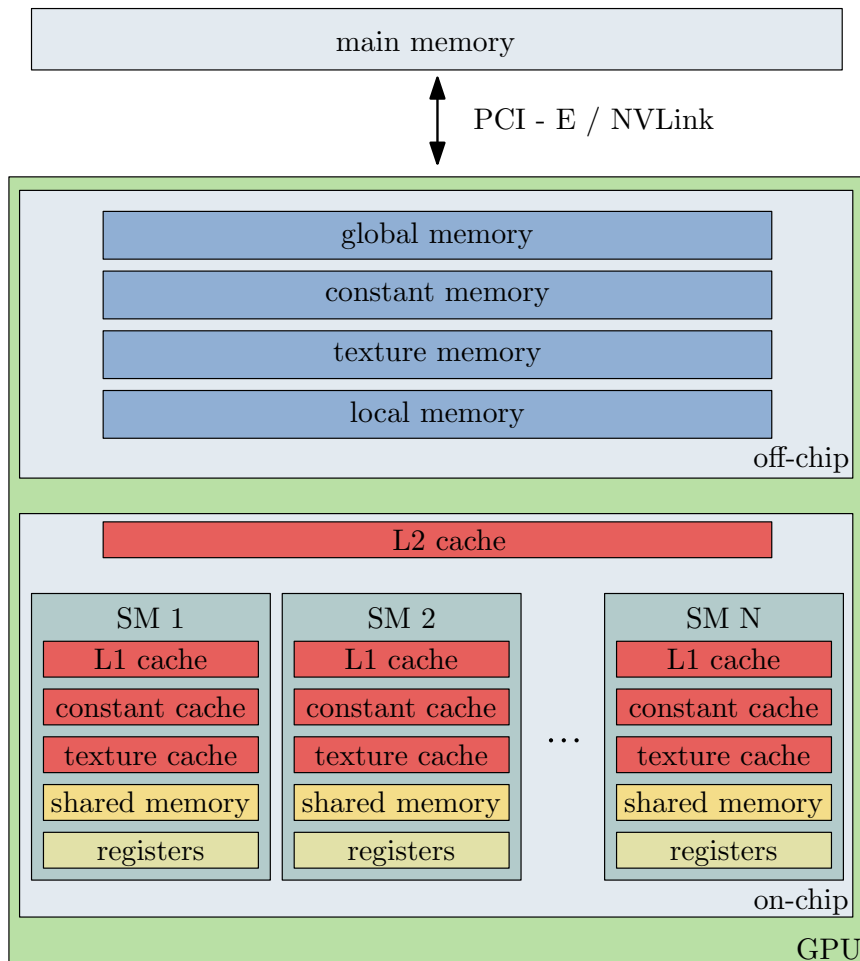
Σχήμα 10.8: Παράδειγμα αντιστοίχισης warp σε block νημάτων (από [Bel22]).

διαστάσεις (παλιότερα ήταν 2 οι επιτρεπόμενες διαστάσεις). Συνολικά, τα νήματα οργανώνονται σε μια ιεραρχία δύο επιπέδων: πλέγμα από block, όπου κάθε block περιέχει ένα πλέγμα από νήματα. Ένα παράδειγμα πλέγματος πυρήνα απεικονίζεται στο Σχήμα 10.7. Όλα τα νήματα στο ίδιο block εκτελούνται στο ίδιο SM και επιτρέπεται να επικοινωνούν μεταξύ τους. Η CUDA μπορεί να προγραμματίσει τα block να εκτελούνται ταυτόχρονα σε ένα SM σύμφωνα με τις δυνατότητες κάθε SM, από την άποψη των ταυτόχρονων block και ενεργών νημάτων που μπορεί να υποστηρίξει.

Τα χρονοδρομολογημένα block χωρίζονται περαιτέρω σε ομάδες των 32 νημάτων που ονομάζονται *warps*. Τα νήματα μέσα σε ένα warp ονομάζονται *λωρίδες* (lanes) και μοιράζονται τον ίδιο μετρητή εντολών, επομένως εκτελούνται ταυτόχρονα με έναν τρόπο του μοντέλου επεξεργασίας *μία εντολή - πολλαπλά δεδομένα* (Single Instruction - Multiple Data, SIMD). Για παράδειγμα, το SM στο Σχήμα 10.8 έχει 128 πυρήνες. Ως εκ τούτου, υποστηρίζει έως και 4 warps για ταυτόχρονη εκτέλεση. Ένα δισδιάστατο block νημάτων μεγέθους 16×5 που πρέπει να εκτελεστεί σε αυτό το SM απαιτεί συνολικά 80 νήματα. Επομένως, πρέπει να εκκινηθούν τρία warps και το τρίτο warp θα περιέχει μόνο 16 νήματα (*half-warp*). Ως αποτέλεσμα, 48 πυρήνες στον SM θα είναι αδρανείς κατά την εκτέλεση του συγκεκριμένου block νημάτων.

Γενικά, τα warps είναι η μικρότερη μονάδα εκτέλεσης. Ανά πάσα στιγμή, ένας SM επιλέγει τα warps που είναι έτοιμα για εκτέλεση, ώστε οι μεγάλοι χρόνοι αναμονής (δηλαδή, η αναμονή άλλων warps για την ανάκτηση των αναγκαίων δεδομένων για την εκτέλεσή τους) να επικαλύπτονται και, συνεπώς, να κρύβονται. Υπάρχουν δύο περιπτώσεις απόκλισης νημάτων οι οποίες υποβαθμίζουν την απόδοση, συγκεκριμένα, η *inter-warp*, όταν τα παράλληλα warps εκτελούνται ανομοιόμορφα, και η *intra-warp*, όταν οι λωρίδες των warps ακολουθούν διαφορετικές διαδρομές εκτέλεσης. Το τελευταίο αναφέρεται απλώς και ως *warp απόκλιση* (divergence) [NVI].

Τα μεγέθη πλέγματος πυρήνα και block νημάτων ορίζονται ως παράμετροι εκκίνησης του προγράμματος πυρήνα. Οι περιορισμοί χωρητικότητας μπορεί να διαδραματίσουν σημαντικό ρόλο στην εκτέλεση του προγράμματος πυρήνα. Υπάρχουν περιορισμοί υλικού α) στον αριθμό



Σχήμα 10.9: Ιεραρχία μνήμης για GPUs που υποστηρίζουν CUDA (εικόνα από [Bel22]).

Α των χρονοδρομολογημένων block ανά SM, β) στον αριθμό B των ενεργών warps ανά SM, γ) στον αριθμό C των χρονοδρομολογημένων νημάτων ανά SM, και δ) στον αριθμό D των νημάτων ανά block. Ως εκ τούτου, ο αριθμός των ενεργών νημάτων σε ένα πρόγραμμα πυρήνα δεν μπορεί να υπερβαίνει την ποσότητα $\min\{(A) \times (D), 32 \times (B), (C)\}$ πολλαπλασιασμένη με τον αριθμό των SM στην GPU.

10.2.2 Ιεραρχίες μνήμης

Η δομή της μνήμης σε μια GPU είναι ιεραρχική, όπως φαίνεται στο Σχήμα 10.9. Επιπλέον, οι μνήμες GPU διακρίνονται μεταξύ off-chip (εκτός τοιπ) και on-chip (πάνω στο τοιπ). Οι μνήμες εκτός τοιπ περιλαμβάνουν τις μνήμες: *καθολική* (global), *σταθερά* (constant), *υφής* (texture) και *τοπική* (local). Η καθολική μνήμη είναι η μεγαλύτερη, της τάξης των αρκετών GBs, αλλά πιο αργή μνήμη και είναι το μέρος όπου μεταφέρονται δεδομένα από την κύρια μνήμη του κεντρικού υπολογιστή μέσω μιας σύνδεσης PCI-E ή, πιο πρόσφατα, με ταχύτερες διασυνδέσεις, όπως το NVLink. Τα δεδομένα που βρίσκονται στην καθολική μνήμη είναι ορατά σε όλα τα νήματα. Τα περιεχόμενα της καθολικής μνήμης μπορούν επίσης να αποθηκευτούν προσωρινά στις κρυφές μνήμες L1 και L2 στο τοιπ. Η σταθερή μνήμη είναι πολύ μικρότερη (π.χ. 64 KBs), αλλά επιτρέπει σημαντικά μικρότερους χρόνους προσπέλασης. Χρησιμοποιείται για δεδομένα μόνο για ανάγνωση και αποθηκεύεται στην κρυφή μνήμη σε διαφορετικό

μέρος από τις κρυφές μνήμες L1 και L2. Ο βασικός λόγος για τους σύντομους χρόνους είναι ότι ένα μεμονωμένο νήμα μπορεί να διαβάσει μία φορά μια τιμή από τη σταθερή μνήμη και να μεταδώσει την τιμή που διαβάστηκε σε όλα τα άλλα νήματα στο ίδιο μισό warp. Η μνήμη υψής είναι επίσης μόνο για ανάγνωση και είναι βελτιστοποιημένη για μοτίβα πρόσβασης όπου οι επόμενες προσβάσεις είναι χωρικά κοντινές. Η τοπική μνήμη είναι μέρος της καθολικής μνήμης και χρησιμοποιείται όταν οι καταχωρητές που απαιτούνται για ένα νήμα είναι γεμάτοι ή δεν μπορούν να χωρέσουν τα απαιτούμενα δεδομένα κατά την ενεργοποίηση του λεγόμενου *register spilling*. Σε κάθε νήμα εκχωρείται το δικό του μέρος της τοπικής μνήμης.

Οι μονάδες μνήμης στο τσιπ αποτελούνται από την *κοινόχρηστη μνήμη* (shared), τις κρυφές μνήμες και τους καταχωρητές. Η κοινόχρηστη μνήμη είναι ο δεύτερος ταχύτερος τύπος μνήμης και η καθυστέρησή της είναι δύο τάξεις μεγέθους μικρότερη από αυτήν της καθολικής μνήμης. Κάθε SM έχει τη δική του κοινόχρηστη μνήμη. Επιπλέον, τα περιεχόμενα της κοινόχρηστης μνήμης είναι προσβάσιμα από όλα τα νήματα στο ίδιο block. Αυτή η δυνατότητα καθιστά την κοινόχρηστη μνήμη ιδιαίτερα χρήσιμη, καθώς επιτρέπει επίσης τη συνεργασία μεταξύ των νημάτων μέσα σε ένα block. Οι κρυφές μνήμες L1 και L2 εξυπηρετούν την καθολική μνήμη και έχουν δυνατότητα διάχυσης δεδομένων στην τοπική μνήμη. Υπάρχουν, επίσης, κρυφές μνήμες για τη σταθερά και τη μνήμη υψής. Τέλος, οι καταχωρητές είναι ο ταχύτερος τύπος μνήμης, όπως και στα τσιπ CPU. Πιο συγκεκριμένα, περιέχουν τις οδηγίες ενός μεμονωμένου νήματος και τις τοπικές μεταβλητές κατά τη διάρκεια ζωής αυτού του νήματος. Εάν ένα νήμα απαιτεί περισσότερους καταχωρητές από ένα όριο, αυτό οδηγεί σε μείωση των νημάτων που επιτρέπεται να τρέχουν ταυτόχρονα.

Η CUDA μπορεί να προγραμματίσει τα block να εκτελούνται ταυτόχρονα σε έναν SM, ανάλογα με την κοινόχρηστη μνήμη και τους καταχωρητές που χρησιμοποιούνται ανά block. Η αύξηση οποιουδήποτε από αυτούς τους παράγοντες μπορεί να οδηγήσει σε περιορισμένη ταυτόχρονη εκτέλεση block, η οποία οδηγεί σε χαμηλή *εκμετάλλευση* (occupancy). Η εκμετάλλευση ορίζεται ως η αναλογία των ενεργών warps σε έναν SM προς το μέγιστο επιτρεπόμενο πλήθος ενεργών warps ανά SM. Η μεγιστοποίηση της εκμετάλλευσης είναι μια καλή ευρετική προσέγγιση, αλλά δεν εγγυάται πάντα αύξηση της απόδοσης. Αντιθέτως, η διατήρηση υψηλής αποδοτικότητας εκτέλεσης σε ένα warp, το λεγόμενο *warp execution efficiency*, δηλαδή του μέσου ποσοστού ενεργών νημάτων σε κάθε εκτελούμενο warp, είναι μια πιο αποτελεσματική προσέγγιση για εργασίες διαχείρισης δεδομένων.

10.2.3 Περισσότερα για το SIMT μοντέλο

Το SIMT μπορεί να θεωρηθεί ως μια πιο ευέλικτη έκδοση του SIMD. Επιπλέον, τόσο το SIMT όσο και το SIMD προσεγγίζουν τον παραλληλισμό μέσω της ενεργοποίησης της ίδιας εντολής σε πολλαπλές μονάδες εκτέλεσης. Ωστόσο, διαφέρουν θεμελιωδώς ως προς τον τρόπο υλοποίησης αυτής της λειτουργικότητας. Σε μια αρχιτεκτονική SIMD, κάθε εντολή εφαρμόζει την ίδια πράξη παράλληλα σε πολλά στοιχεία δεδομένων. Το SIMD, συνήθως, υλοποιείται χρησιμοποιώντας επεξεργαστές με καταχωρητές και μονάδες εκτέλεσης διανυσμάτων, όπου ένα νήμα παράγει διανυσματικές εντολές που εκτελούνται με τρόπο SIMD. Σε μια αρχιτεκτονική SIMT, αντί ένα μεμονωμένο νήμα να παράγει εντολές που εφαρμόζονται σε διανύσματα δεδομένων, πολλαπλά νήματα εκδίδουν κοινές εντολές για αυθαίρετα δεδομένα. Για να υποστηρίξει το μοντέλο προγραμματισμού SIMT σε υψηλό επίπεδο, η CUDA εκτελεί νήματα ως

δέσμες των 32, δηλαδή warps. Κάθε νήμα μπορεί να έχει πρόσβαση στους δικούς του καταχωρητές, να φορτώνει και να αποθηκεύει δεδομένα από διαφορετικές διευθύνσεις και να ακολουθεί αποκλίνουσες διαδρομές ροής ελέγχου [NVI].

Όταν ένα warp επιλέγεται να εκτελεστεί, το υλικό SIMD εκτελεί την ίδια εντολή για όλα τα νήματα στο ίδιο warp. Εάν όλα τα νήματα σε ένα warp ακολουθούν την ίδια διαδρομή εκτέλεσης κατά την επεξεργασία των δεδομένων τους, η απόδοση εκτέλεσης είναι υψηλή. Όταν τα νήματα μέσα σε ένα warp ακολουθούν διαφορετικές διαδρομές ροής ελέγχου, το υλικό SIMD θα κάνει πολλαπλές διελεύσεις για κάθε μία από αυτές τις αποκλίνουσες διαδρομές. Για παράδειγμα, σε ένα σενάριο if-else ένα πέρασμα θα εκτελέσει εκείνα τα νήματα που ακολουθούν το τμήμα if και ένα άλλο πέρασμα θα εκτελέσει αυτά που ακολουθούν το else μέρος. Κατά τη διάρκεια κάθε περάσματος, τα νήματα που ακολουθούν την άλλη διαδρομή δεν είναι ενεργά. Αυτά τα δύο περάσματα για τα τμήματα if και else, αντίστοιχα, είναι διαδοχικά, επομένως ο χρόνος επεξεργασίας τους αθροίζεται. Συνεπώς, μειώνεται ο παραλληλισμός σε επίπεδο εντολών, που επίσης σημειώνεται ως *Instruction level parallelism* (ILP), το οποίο είναι μια χρήσιμη μετρική παραλληλισμού σε προγράμματα διαχείρισης δεδομένων σε GPU.

10.3 Παραδείγματα διαχείρισης δεδομένων σε GPUs

Η διαχείριση δεδομένων σε GPUs θέλει ιδιαίτερη προσοχή για μια σειρά από λόγους, όπως περιγράφεται σύντομα παρακάτω:

- Η μνήμη της κάρτας είναι πολύ μικρή για να χωρέσει μεγάλα δεδομένα, π.χ. της τάξης των 100 GBs.
- Το εύρος ζώνης για μεταφορά δεδομένων από τον κεντρικό υπολογιστή στη συσκευή είναι περιορισμένο.
- Ο τρόπος προσπέλασης της καθολικής μνήμης της κάρτας και η επίγνωση της ιεραρχίας μνήμης που αναλύθηκε προηγουμένως είναι κρίσιμοι παράγοντες για αποδοτικές υλοποιήσεις προγραμμάτων CUDA.
- Ισχυρά συνδεδεμένο με το παραπάνω είναι η εργασία που αναλαμβάνει το κάθε νήμα (thread), η έκταση της οποίας καθορίζει επίσης τις κατάλληλες διαστάσεις για το κάθε block και το πλέγμα πυρήνα.

Από τα παραπάνω γίνεται σαφές ότι ο προγραμματισμός σε CUDA είναι σε αρκετά χαμηλό επίπεδο, αρκετά μακριά από σύγχρονες δηλωτικές προσεγγίσεις, που απαλλάσσουν τον χρήστη από την ανάγκη καθορισμού λεπτομερειών εκτέλεσης, ειδικά όταν αυτές προϋποθέτουν μη επιδερμική κατανόηση του υλικού πάνω στο οποίο εκτελούνται τα προγράμματα. Όμως το κοινό με τα προγράμματα στο ευρύτερο προγραμματιστικό πλαίσιο του MapReduce είναι ότι το σημαντικό είναι να κατανοεί κάποιος τον όγκο της εργασίας που κάθε υπολογιστική μονάδα αναλαμβάνει και τη μεταφορά των δεδομένων ανάμεσα σε διαφορετικές οντότητες, είτε είναι αυτές εργασίες Map και Reduce είτε είναι διαφορετικού τύπου μνήμες σε κάρτες γραφικών.

Παράδειγμα 10.3. Έστω ένα πρόγραμμα πυρήνα στο οποίο κάθε νήμα αναλαμβάνει την επεξεργασία μίας ή ενός μικρού πλήθους θέσεων της καθολικής μνήμης, που έστω ότι καταλαμβάνουν 3-4 bytes. Κάτι τέτοιο μπορεί να συμβαίνει όταν θέλουμε να χρησιμοποιήσουμε μια GPU για πολλαπλασιασμό των στοιχείων ενός πίνακα με μια σταθερά ή όταν θέλουμε να μετατρέψουμε τα pixels μιας εικόνας από RGB σε διαφορετικούς τόνους του γκρι. Εάν θεωρήσουμε ότι, κατά μέσο όρο, το εύρος ζώνης ανάμεσα στην καθολική μνήμη και τους SM είναι 1 TB/sec (σημαντικά υψηλότερο από το αντίστοιχο των CPUs) και ότι γίνεται μια πράξη απλής ακρίβειας ανά byte, τότε διαπιστώνουμε ότι η κάρτα γραφικών υπο-απασχολείται παρόλο που τα νούμερα απόδοσης φαίνονται ικανοποιητικά. Αυτό συμβαίνει γιατί ο ρυθμός επεξεργασίας είναι 1TFLOP, περιοριζόμενος από το εύρος ζώνης ανάμεσα στην καθολική μνήμη και τους επεξεργαστές της GPU. Όμως, το 1 TFLOP είναι πολύ χαμηλότερο από τις δυνατότητες των σύγχρονων συσκευών. □

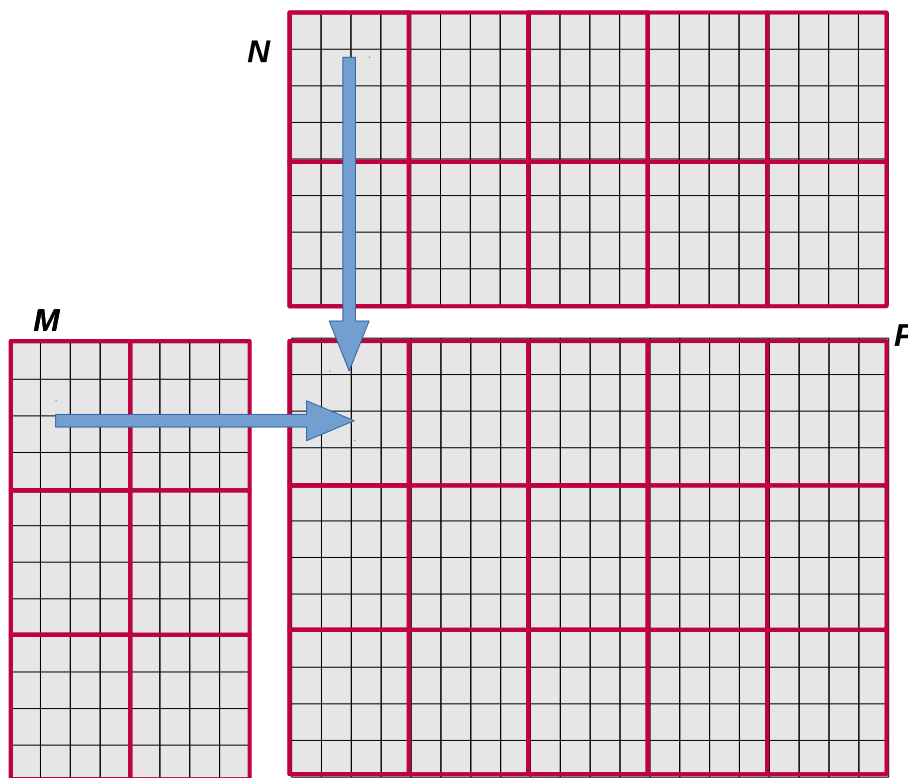
Το παραπάνω παράδειγμα είναι ενδεικτικό των παραγόντων που πρέπει ο προγραμματιστής CUDA να λαμβάνει υπόψη για αποδοτική εκμετάλλευση μιας κάρτας γραφικών.

10.3.1 Πολλαπλασιασμός πινάκων

Το πρώτο παράδειγμα που θα δούμε έχει να κάνει με τον πολλαπλασιασμό πινάκων, που είδαμε προηγουμένως και στο MapReduce. Αρχικά, θα θεωρήσουμε ότι κάθε νήμα είναι υπεύθυνο για τον υπολογισμό ενός κελιού του τελικού πίνακα $P = M \times N$. Εάν ο τελικός πίνακας έχει διαστάσεις 12×20 και χρησιμοποιήσουμε ένα πλέγμα block όπως στο Σχήμα 10.7, τότε ένα νήμα με συντεταγμένες (2,1) στο block με συντεταγμένες (1,1) υπολογίζει το κελί του πίνακα $P(5,6)$ αρχίζοντας από το 0, δηλαδή το πράσινο κελί στο σχήμα. Η διάσταση i του κελιού $P(i,j)$ προκύπτει από το μέγεθος του block στη διάσταση y (στο σχήμα αυτή η διάσταση είναι 4) επί τη συντεταγμένη του block στον άξονα y (που είναι 1 στο σχήμα) συν τη συντεταγμένη y του νήματος μέσα στο block: συνολικά $4 \cdot 1 + 1 = 5$. Αντίστοιχα, το j είναι το γινόμενο της διάστασης του block στον οριζόντιο άξονα επί τη x συντεταγμένη του block συν τη x συντεταγμένη του νήματος μέσα στο block: $4 \cdot 1 + 2 = 6$.

Σε μια τέτοια προσέγγιση, το κάθε νήμα αναλαμβάνει μόνο του εξ ολοκλήρου τον υπολογισμό του εσωτερικού γινομένου μίας γραμμής i του πίνακα M με τη στήλη j του πίνακα N . Για κάθε μία ανάγνωση ενός στοιχείου από τον M και ενός στοιχείου από τον N , πραγματοποιούμε 2 πράξεις κινητής υποδιαστολής: ένα γινόμενο και ένα άθροισμα σύμφωνα με τον τύπο του εσωτερικού γινομένου. Οπότε, αναμένουμε ότι θα ισχύει το σκεπτικό του Παραδείγματος 10.3 και συνολικά η κάρτα θα υποχρησιμοποιείται.

Επίσης, εάν θεωρήσουμε ότι οι πίνακες M και N γραμμικοποιούνται, δηλαδή εσωτερικά μετατρέπονται σε μονοδιάστατους, όπως και γίνεται συχνά, και αποθηκεύονται στην καθολική μνήμη ως μονοδιάστατοι, τότε η προσπέλαση των στοιχείων του M πραγματοποιείται σε συνεχόμενες θέσεις μνήμης, αλλά η αντίστοιχη προσπέλαση των στοιχείων N γίνεται σε θέσεις που απέχουν μεταξύ τους όσο και το πλήθος των στηλών του N . Αυτή η συμπεριφορά έχει επίσης, ενδεχομένως, ισχυρό αντίκτυπο στην απόδοση. Δεν είναι, όμως, το σημαντικό ένα νήμα να προσπελάζει συνεχόμενες θέσεις καθολικής μνήμης, αλλά όλα τα νήματα ενός block να προσπελάζουν στο ίδιο βήμα συνεχόμενες θέσεις. Εάν συμβαίνει η τελευταία περίπτωση, τότε το υλικό των GPUs είναι σε θέση να χειριστεί πιο αποδοτικά τις προσπελάσεις.



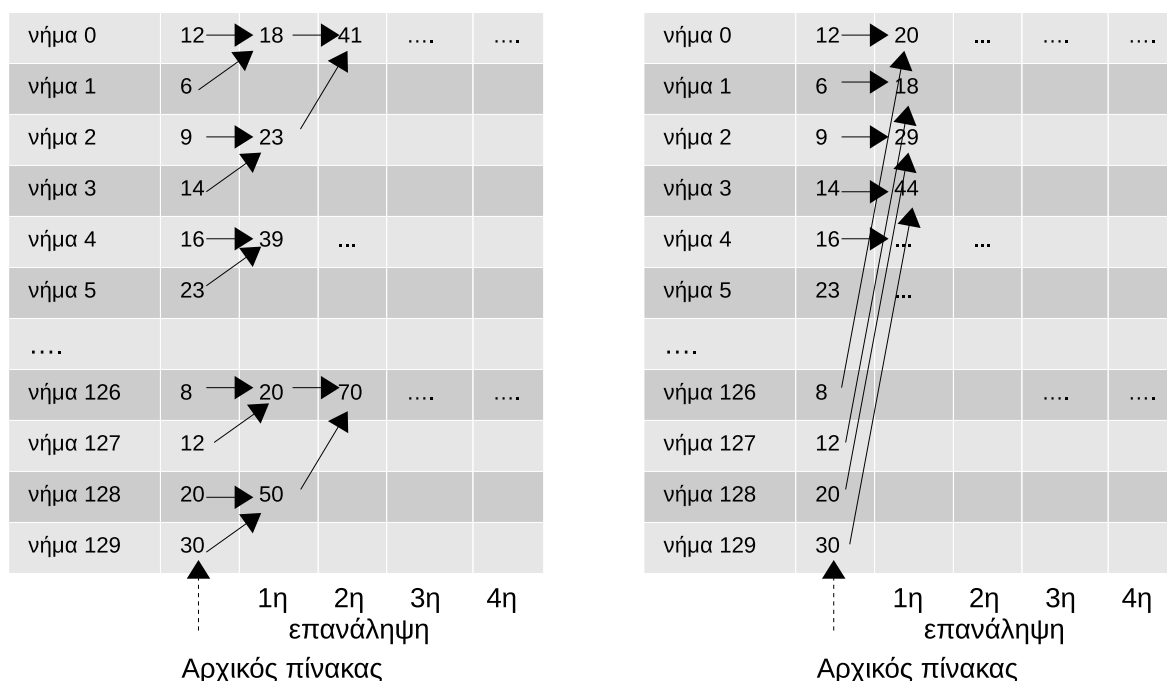
Σχήμα 10.10: Αποδοτικότερος υπολογισμός πολλαπλασιασμού πινάκων με τη μέθοδο tiling.

Με άλλα λόγια, στον πίνακα N οι προσπελάσεις γίνονται αποδοτικότερα, καθώς συγκεντρωτικά, για όλα τα νήματα ενός block σε ένα βήμα εκτέλεσης, αυτές οι προσπελάσεις είναι πιο ομαδοποιημένες.

Η βασική λύση σε αυτά τα προβλήματα είναι η υιοθέτηση της προσέγγισης του λεγόμενου *tiling*, όπου δεδομένα διαβάζονται μία φορά από την καθολική μνήμη σε ένα block νημάτων και αποθηκεύονται στην κοινόχρηστη μνήμη του SM. Μετά, αυτά τα δεδομένα μπορούν να προσπελαστούν από όλα τα νήματα ενός block με πολύ μικρότερες καθυστερήσεις και αυξημένο εύρος ζώνης κατά 2 περίπου τάξεις μεγέθους, καθώς τέτοια είναι η διαφορά ανάμεσα σε καθολική και κοινόχρηστη μνήμη.

Η εφαρμογή της συγκεκριμένης τεχνικής στο παράδειγμα πολλαπλασιασμού πινάκων απεικονίζεται στο Σχήμα 10.10, όπου έχουμε τον 12×8 πίνακα M και τον 8×20 πίνακα N . Έστω ότι τα μεγέθη της κοινόχρηστης μνήμης είναι τέτοια, που μας επιτρέπει να αποθηκεύουμε από τους 2 πίνακες 32 τιμές, 16 από τον πίνακα M και 16 από τον πίνακα N . Χωρίζουμε τότε τους δύο αυτούς πίνακες λογικά σε *tiles* 4×4 , που συμπτωματικά είναι οι ίδιες διαστάσεις με το block νημάτων.

Έστω στο τρέχον παράδειγμα ότι θέλουμε να υπολογίσουμε τις θέσεις του πίνακα P , που αναλαμβάνει το block με συντεταγμένες (0,0). Θα χρειαστεί να διαβαστούν τα *tiles* με συντεταγμένες (0,0) και (1,0) από τον M και τα *tiles* με συντεταγμένες (0,0) και (0,1) από τον N , όπως δείχνουν τα βέλη. Αυτό όμως θα γίνει συγχρονισμένα, δηλαδή σε 2 φάσεις. Πρώτα, διαβάζονται μία φορά τα *tiles* (0,0) των δύο πινάκων και όλα τα νήματα του block υπολογίζουν τα μερικά αθροίσματα γινομένων. Μετά, όταν όλα τα νήματα έχουν ολοκληρώσει την τρέχουσα εργασία τους, διαβάζονται τα άλλα 2 *tiles*. Εάν οι πίνακες ήταν διαστάσεων 12×12



Σχήμα 10.11: Απλή (αριστερά) και πιο αποδοτική (δεξιά) υλοποίηση υπολογισμού αθροίσματος στοιχείων πίνακα.

και 12×20 , αντίστοιχα, για το ίδιο μέγεθος tiles, θα είχαμε 3 φάσεις. Τι κερδίζουμε από αυτό; Οι ακριβές προσπελάσεις στην καθολική μνήμη μειώνονται κατά έναν παράγοντα ανάλογο της πλευράς των tiles. Οπότε, για tiles 16×16 , οι επιταχύνσεις αναμένεται να είναι τουλάχιστον 1 τάξης μεγέθους. Αυτό μπορεί να σημαίνει ότι παύει ο χρόνος μεταφοράς δεδομένων από την καθολική μνήμη προς τους επεξεργαστές της GPU να είναι ο βασικός χρόνος καθυστέρησης και, συνεπώς, ανοίγει ο δρόμος προς υψηλότερη εκμετάλλευση της διαθέσιμης υπολογιστικής ισχύος.

Προσοχή, όμως, απαιτείται στο ότι η κοινόχρηστη μνήμη διαμοιράζεται σε όλα τα τρέχοντα block νημάτων, οπότε είναι αναμενόμενο η διαθέσιμη μνήμη ανά block να είναι λίγα Kbytes μόνο. Ένα tile 16×16 καταλαμβάνει χώρο 1 KB, εάν κάθε θέση πίνακα χρειάζεται 4 bytes, και χρειαζόμαστε 2 tiles στη μνήμη για να δουλέψει το σκεπτικό. Οπότε, δεν πρέπει τα tiles να είναι πολύ μεγαλύτερα από αυτό το μέγεθος. Κλείνοντας, να διευκρινιστεί ότι υπάρχουν και άλλες ιδιαιτερότητες, όπως ότι τα μεγέθη των πινάκων μπορεί να είναι τέτοια που να μην ταιριάζουν ακριβώς τα block των νημάτων, οπότε θέλει προσοχή στον προγραμματισμό ώστε να ελέγχεται το εάν ένα νήμα διαβάζει χρήσιμα δεδομένα εντός ορίων ή όχι.

10.3.2 Επεξεργασία πινάκων και εξαγωγή στατιστικών

Ας αρχίσουμε από την απλή περίπτωση όπου έχουμε έναν μεγάλο μονοδιάστατο πίνακα αριθμητικών τιμών με n θέσεις και θέλουμε να υπολογίσουμε το άθροισμα αυτών των τιμών (ή τη μέγιστη ή την ελάχιστη τιμή). Προφανώς, κάθε αλγόριθμος που επιλύει αυτό το πρόβλημα έχει πολυπλοκότητα $\Omega(n)$.

Είναι λογικό, όταν ο πίνακας είναι πολύ μεγάλος, να θέλουμε ο υπολογισμός του αθροίσματος των στοιχείων να παραλληλοποιηθεί. Το Σχήμα 10.11 δείχνει δύο εναλλακτικές, που

φαινομενικά είναι ισοδύναμες, για έναν πίνακα με λιγότερα από 256 στοιχεία. Αρχικά, κάθε νήμα αναλαμβάνει να φέρει ένα στοιχείο του πίνακα από την κύρια μνήμη. Στην απλούστερη προσέγγιση που απεικονίζεται αριστερά στο σχήμα, στην 1η επανάληψη, ενεργοποιούνται μόνο τα ζυγά νήματα και το νήμα στη θέση i υπολογίζει το άθροισμα των στοιχείων στις θέσεις του πίνακα i και $i + 1$. Μετά, στη 2η επανάληψη, ενεργοποιούνται τα νήματα στις θέσεις που είναι πολλαπλάσια του 4, και κάθε ενεργοποιημένο νήμα στη θέση i υπολογίζει το άθροισμα των στοιχείων στις ήδη τροποποιημένες θέσεις του πίνακα i και $i + 2$. Αυτό συνεχίζεται για $\log n$ επαναλήψεις.

Δύο παρατηρήσεις είναι οι εξής. Πρώτον, πολλά νήματα μένουν ανενεργά κατά τη διάρκεια εκτέλεσης του πυρήνα. Δεύτερον, το φαινόμενο της απόκλισης είναι ιδιαίτερα έντονο ακόμη και από την πρώτη επανάληψη, όπου αναμένεται τα μισά νήματα ενός warp να ακολουθήσουν ένα μονοπάτι, αυτό της άθροισης δύο στοιχείων, και τα άλλα μισά άλλο μονοπάτι, αυτό που δεν θα εκτελέσει καμία εντολή.

Στο δεξί μέρος του σχήματος φαίνεται μια αποδοτικότερη εκδοχή. Η αυξημένη αποδοτικότητα οφείλεται στο ότι η ανάθεση εργασιών μόνο στα αρχικά νήματα αυξάνει κατά πολύ τις εκτελέσεις warp, στα οποία και τα 32 νήματα που μετέχουν ακολουθούν το ίδιο μονοπάτι, είτε αυτό του αθροίσματος είτε της μη εκτέλεσης κάποιας εντολής. Βλέπουμε, λοιπόν, ότι ακόμη και σε απλές περιπτώσεις, όπως το άθροισμα στοιχείων του πίνακα, η κατανόηση της λειτουργίας των καρτών γραφικών είναι σημαντική για αποδοτικές υλοποιήσεις.

Μία άλλη συχνή πράξη, γνωστή και ως prefix sum, μετατρέπει έναν μονοδιάστατο πίνακα ώστε κάθε στοιχείο του να είναι το αρχικό στοιχείο συν το προηγούμενο. Π.χ., στον πίνακα $A = [12, 6, 9, 14, 16, 23, \dots]$ το αποτέλεσμα θα ήταν $A' = [12, 18, 27, 41, 57, 80, \dots]$. Είναι, στην ουσία, εφαρμογή του τελεστή μείωσης (reduction) ενώ ονομάζεται και παράλληλη σάρωση (parallel scan). Ένας απλός αλγόριθμος για την επίτευξη αυτής της πράξης θυμίζει κάπως την προσέγγιση παράλληλων αθροισμάτων στο Σχήμα 10.11 (δεξιά) και λειτουργεί ως εξής:

- Αναθέτουμε ένα νήμα ανά θέση (στοιχείο) του πίνακα, το οποίο και ανακτά την αντίστοιχη θέση από την καθολική μνήμη.
- 1η επανάληψη: όλα τα νήματα, εκτός από το 1ο, υπολογίζουν το άθροισμα του στοιχείου στην αντίστοιχη θέση του πίνακα με την αμέσως προηγούμενη θέση. Π.χ., το στοιχείο στη θέση 2 στο μικρό παράδειγμα παραπάνω θα έχει πλέον τιμή $9 + 6 = 15$. Στο τέλος αυτής της επανάληψης, τα στοιχεία στις θέσεις 0 και 1 έχουν την τελική τους τιμή.
- 2η επανάληψη: όλα τα νήματα, εκτός από τα 2 πρώτα, υπολογίζουν το άθροισμα του στοιχείου στην αντίστοιχη θέση του πίνακα με την τιμή στο στοιχείο δύο θέσεις προηγουμένως. Σε αυτήν την επανάληψη i , όπου $i = 2$, τα πρώτα 2^i στοιχεία έχουν λάβει την τελική τους τιμή. Π.χ., το στοιχείο στη θέση 3 είχε τιμή 23 πριν από την έναρξη της επανάληψης, και αυτό στη θέση 1 είχε τιμή 18. Στο τέλος της επανάληψης, το στοιχείο στη θέση 3 παίρνει την τιμή $23 + 18 = 41$, που είναι και η τελική του.
- Επόμενες επαναλήψεις: με αντίστοιχο τρόπο, στην 3η επανάληψη, αθροίζοντας την τιμή του στοιχείου με αυτήν 4 θέσεις προηγουμένως, καθορίζονται μερικά αθροίσματα

που είναι τελικά μόνο για τις $2^3 = 8$ πρώτες θέσεις του πίνακα. Συνεχίζονται οι επαναλήψεις μέχρι να υπολογιστεί το τελικό αποτέλεσμα.

Αναφέρθηκε προηγουμένως ότι το παραπάνω σκεπτικό θυμίζει το αποδοτικό σκεπτικό του υπολογισμού αθροίσματος στοιχείων. Η ομοιότητα αυτή είναι κάπως επιφανειακή. Στην περίπτωση του αθροίσματος, οι πράξεις πρόσθεσης στον παράλληλο αλγόριθμο είναι ίσες με τις πράξεις πρόσθεσης σε έναν ακολουθιακό κώδικα, π.χ. σε Java ή Python. Δεν συμβαίνει, όμως, αυτό στην περίπτωση του prefix sum, όπου οι συνολικές προσθέσεις είναι περισσότερες κατά έναν παράγοντα $\log n$.

Υπάρχουν στη βιβλιογραφία λύσεις που σχεδιάζουν τα αθροίσματα στην πράξη prefix sum με τέτοιον τρόπο ώστε να είναι παρόμοια σε πλήθος με ακολουθιακούς αλγόριθμους, όμως η παρουσίαση τεχνικών λεπτομερειών δεν είναι στους σκοπούς αυτού του κεφαλαίου. Δίνουμε μόνο ένα περίγραμμα κάποιων πτυχών μιας τέτοιας λύσης ως εξής: να σπάσει ο μεγάλος πίνακας σε μικρά τμήματα, που να χωράνε στην κοινόχρηστη μνήμη. Έπειτα, χρησιμοποιώντας μονοδιάστατα block νημάτων, όλα τα νήματα να αναλαμβάνουν την ανάκτηση των τμημάτων αυτών από την καθολική μνήμη, ένα τμήμα ανά νήμα. Μετά, να υπολογίζεται το prefix sum τοπικά ανά τμήμα από ένα νήμα πάλι, πριν συνδυαστούν κατάλληλα τα αποτελέσματα. Συνδυάζοντας τα παραπάνω με μια ιεραρχική προσέγγιση, μπορεί να γίνει επεξεργασία πολύ μεγάλων πινάκων που δεν χωράνε στην κοινόχρηστη μνήμη. Σε αυτό το σκεπτικό, μειώνεται κατά πολύ το πλήθος των πράξεων αναθέτοντας περισσότερη εργασία σε κάθε νήμα, το οποίο πλέον δεν είναι υπεύθυνο για μόνο μία θέση του πίνακα, αλλά για τμήματα πολλαπλών (συνεχόμενων) θέσεων του πίνακα. Ο αναγνώστης μπορεί να βρει αντιστοιχίες με το σκεπτικό όπου αναθέταμε περισσότερη εργασία σε έναν reducer για να βελτιώσουμε ένα πρόγραμμα MapReduce.

Θα κλείσουμε αυτήν την ενότητα με τη συζήτηση της δημιουργίας ιστογραμμάτων, στα οποία η τεχνική ιδιαιτερότητα σε σύγκριση με τις προηγούμενες λειτουργίες είναι ότι πολλαπλά νήματα καλούνται να ενημερώσουν την ίδια θέση μνήμης χωρίς να μπορεί να λύσει το πρόβλημα που προκύπτει η δυνατότητα συγχρονισμού των νημάτων ανάμεσα σε επαναλήψεις. Ένα ιστόγραμμα στον πίνακα $A = [12, 6, 9, 14, 16, 23]$ μπορεί να υπολογίζει ότι υπάρχουν 2 τιμές στο διάστημα $[0, 10]$ και 4 στο διάστημα $(10, \infty)$.

Έστω στο συγκεκριμένο παράδειγμα ότι χρησιμοποιούμε 2 νήματα, το κάθε ένα εκ των οποίων αναλαμβάνει 4 θέσεις του πίνακα. Μπορεί κανείς να σκεφτεί ότι ο διαχωρισμός είναι οι 4 πρώτες θέσεις να ανατεθούν στο πρώτο νήμα και οι υπόλοιπες στο δεύτερο, αλλά καλό είναι να θυμόμαστε ότι οι βελτιστοποιημένες προσπελάσεις είναι αυτές όπου συνολικά τα νήματα ενός `map` προσπελάζουν συνεχόμενες θέσεις του πίνακα, οπότε αναμένουμε στη γενική περίπτωση κάθε νήμα να μην αναλαμβάνει συνεχόμενες θέσεις του πίνακα.

Συνεχίζοντας στο παράδειγμα, θεωρούμε ότι έχουμε δημιουργήσει στην κρυφή μνήμη μία δομή που έχει δύο μετρητές, έναν για το διάστημα $[0, 10]$ και έναν άλλον για το διάστημα $(10, \infty)$. Οι αρχικές τιμές είναι 0. Προφανώς, το δεύτερο νήμα εάν αναλάβει 2 στοιχεία του πίνακα έχει λιγότερη εργασία, αλλά αυτό, όπως είπαμε, συμβαίνει συχνά και αντιμετωπίζεται με προσθήκη κατάλληλης συνθήκης ελέγχου στο πρόγραμμα πυρήνα, χωρίς όμως να μπορεί να αποφευχθεί η απόκλιση εκτέλεσης νημάτων. Το πρόβλημα, όμως, είναι αλλού: έστω ότι και τα 2 νήματα θέλουν ταυτόχρονα να ενημερώσουν τον μετρητή για το διάστημα $(10, \infty)$. Η πράξη ενημέρωσης διασπάται στις βασικότερες πράξεις της ανάγνωσης τρέχουσας τιμής,

υπολογισμού νέας τιμής και εγγραφής της νέας τιμής. Εάν οι δύο αναγνώσεις γίνουν πριν κάποια από τις εγγραφές, το τελικό αποτέλεσμα του μετρητή θα είναι λανθασμένο κατά 1. Ευτυχώς, η CUDA δίνει τη δυνατότητα χρήσης των λεγόμενων *ατομικών* πράξεων, οι οποίες αναλαμβάνουν να σειριοποιήσουν τις ταυτόχρονες προσπάθειες ενημέρωσης και είναι απαραίτητο να χρησιμοποιούνται σε τέτοιες περιπτώσεις. Όπως περιμένουμε, η χρήση τους έχει αντίκτυπο στην απόδοση, γι' αυτό η καλύτερη πρακτική είναι να ελαττώνονται όσο γίνεται σε αριθμό. Π.χ., οι ενημερώσεις στους καθολικούς μετρητές να γίνονται αφού ολοκληρώσει την εργασία του ένα νήμα και όχι για κάθε στοιχείο που ελέγχει, εάν για να επιτευχθεί αυτό δεν υφίστανται περιορισμοί μνήμης αναφορικά με την αποθήκευση τοπικών μετρητών.

10.3.3 Συχνές λανθασμένες αντιλήψεις

Προσπαθώντας κάποιος να επιλύσει θέματα μεγάλων δεδομένων χρησιμοποιώντας κάρτες γραφικών, συχνά έρχεται αντιμέτωπος με κάποιες αντιλήψεις, οι οποίες θα πρέπει να διευκρινιστούν κατάλληλα. Παρακάτω, δίνονται τέτοιες διευκρινίσεις.

- Μία συχνά διατυπωμένη αντίληψη είναι ότι η GPU μπορεί να επιτύχει απευθείας, απλά και μόνο με τη χρήση της, επιτάχυνση κατά τάξεις μεγέθους. Αυτή η αντίληψη πηγάζει από τα κέρδη απόδοσης από τη χρήση GPU σε αλγόριθμους που μπορούν να παραλληλιστούν εύκολα. Το γεγονός ότι είναι σύνηθες για κάποιον να περιμένει ότι οι GPUs μπορούν να επιτύχουν απευθείας επιτάχυνση κατά τάξεις μεγέθους έχει καταρριφθεί από τα πρώτα χρόνια χρήσης των καρτών γραφικών για γενικούς υπολογισμούς. Είναι πιο ρεαλιστικό να αναμένουμε επιταχύνσεις της τάξης των λίγων παραγόντων ή ακόμη λιγότερο. Αυτό επαληθεύεται από τις πιο πρόσφατες εξελίξεις στην ανάλυση δεδομένων με δυνατότητα GPU, λαμβάνοντας υπόψη τα γενικά έξοδα φόρτωσης δεδομένων στην κύρια μνήμη της GPU.

Γιατί, λοιπόν, μια 100πλάσια αύξηση του αριθμού των πυρήνων δεν συνοδεύεται από αναλογική βελτίωση απόδοσης;

1. Είναι ευρέως γνωστό ότι οι βελτιώσεις των παράλληλων αλγορίθμων οριοθετούνται από τον νόμο του Amdahl, που αναφέραμε στην αρχή του κεφαλαίου, ο οποίος στην πραγματικότητα δίνει ένα πολύ χαλαρό άνω όριο σε πιθανές βελτιώσεις. Πολλαπλοί πρόσθετοι παράγοντες, όπως το κόστος μεταφοράς δεδομένων μεταξύ διαφορετικών τύπων μνήμης και ακριβώς μοτίβα πρόσβασης στη μνήμη, η απόκλιση νημάτων, η κρυφή σειριοποίηση των εντολών, που είναι εγγενή χαρακτηριστικά του τρόπου λειτουργίας των καρτών γραφικών, καθιστούν το πραγματικό όριο βελτίωσης πολύ χαμηλότερο στην πράξη.
2. Οι επιταχύνσεις αναφέρονται σε συγκρίσεις με βελτιστοποιημένες λύσεις CPU, που μπορούν να παραλληλιστούν αποτελεσματικά σε πολλά σενάρια. Το να εκτελείται ένα πρόγραμμα σε CPU δεν σημαίνει ούτε ότι δεν είναι ήδη παραλληλοποιημένο ούτε ότι δεν είναι βελτιστοποιημένο.
3. Δεν είναι κατάλληλοι όλοι οι αλγόριθμοι διαχείρισης δεδομένων για παραλληλισμό. Π.χ. μπορεί να απαιτούν ενημέρωση ενός ενιαίου συνόλου σημείων ή ενθυλάκωση πολλαπλών σημείων συγχρονισμού, ή, το πιο σημαντικό, οι τιμές δεδομένων

να καθορίζουν τη ροή εντολών (δηλαδή, ποια ακριβώς δεδομένα πρέπει να προσπελαστούν). Το τελευταίο συναντάται πολύ σε λειτουργίες διαχείρισης δεδομένων και εμποδίζει την ανάπτυξη παράλληλων λύσεων με blocks που λειτουργούν ανεξάρτητα το ένα από το άλλο.

- Μία αντίθετη αντίληψη με την προηγούμενη υπεραισιόδοξη είναι ότι η μεταφορά δεδομένων καθιστά την GPU πολύ ακριβή και, συνεπώς, ακατάλληλη για ανάλυση δεδομένων. Ο βασικός ισχυρισμός είναι ότι το υπολογιστικό πλεονέκτημα της GPU αντισταθμίζεται από την υπάρχουσα αργή διασύνδεση μεταξύ καρτών γραφικών και κεντρικού υπολογιστή. Όμως, θα πρέπει να λάβουμε υπόψη ότι το πρόβλημα με τέτοιες αργές διασυνδέσεις αναμένεται να ξεπεραστεί από τις προόδους υλικού στην τεχνολογία διασύνδεσης. Για παράδειγμα, το NVLink 2.0 αναφέρεται ότι κλιμακώνεται σε αυθαίρετους όγκους δεδομένων και διευκολύνει τη συνεπεξεργασία δεδομένων μεταξύ CPU και GPU. Όμως, ακόμη και με την τρέχουσα τεχνολογία των πιο αργών διασυνδέσεων CPU-GPU, το κόστος μεταφοράς δεδομένων δεν είναι απαραίτητα υψηλότερο από το κόστος επεξεργασίας ολόκληρου του συνόλου δεδομένων στη CPU αποκλειστικά. Ακόμη και αν το κόστος μεταφοράς είναι υψηλό, αυτό δεν σημαίνει ότι δεν πρέπει να χρησιμοποιούμε GPU για ανάλυση δεδομένων, έχοντας πάντα υπόψη ότι οι αναμενόμενες επιταχύνσεις μπορεί να μην είναι πολύ υψηλές ή ότι οι δύο τύποι επεξεργαστών μπορούν (και πρέπει συχνά) να δρουν συνεργατικά για να εκτελούν προγράμματα επεξεργασίας μεγάλων δεδομένων.
- Μία άλλη αντίληψη είναι ότι απαιτείται πλήρης εκμετάλλευση (occupancy) της υπολογιστικής ισχύος για αποτελεσματικούς αλγόριθμους GPU και ότι είναι πιο ωφέλιμο να εκκινεί ένα πρόγραμμα πυρήνα χιλιάδες νήματα για να κρύψει την καθυστέρηση εξαιτίας της μνήμης. Η πραγματική απάντηση είναι «εξαρτάται». Πιο συγκεκριμένα, το πρώτο βήμα προς τη βελτιστοποιημένη παραλληλοποίηση προγραμμάτων σε GPU είναι να κατανοήσουμε πού ξοδεύεται ο περισσότερος χρόνος εκτέλεσης. Ως εκ τούτου, ένα πρόβλημα μπορεί να περιορίζεται από τη μνήμη, δηλαδή ο περισσότερος χρόνος ξοδεύεται σε συναλλαγές μνήμης, ενώ ένα άλλο να περιορίζεται από την υπολογιστική ισχύ, δηλαδή ο περισσότερος χρόνος να αφιερώνεται σε υπολογιστικές πράξεις ή ένα μείγμα και των δύο. Για τα πρώτα προβλήματα, η πλήρης εκμετάλλευση της ιεραρχίας μνήμης και οι βελτιστοποιημένες ομαδοποιημένες προσπελάσεις στην καθολική μνήμη μπορούν να μετριάσουν τις δαπανηρές πράξεις ανάκτησης δεδομένων. Από την άλλη πλευρά, για τον δεύτερο τύπο προβλημάτων, απαιτούνται η μείωση της απόκλισης διακλάδωσης και η αποδοτικότητα των παράλληλων πράξεων σε ένα warp. Συνολικά, παρόλο που η πλήρης εκμετάλλευση είναι μια καλή ευρετική αρχή, το γεγονός ότι μπορεί να υπάρχουν προσπελάσεις στη μνήμη που καθορίζονται από απρόβλεπτες τιμές δεδομένων απαιτεί μια ισορροπία μεταξύ της πλήρους χρήσης των πυρήνων της GPU και της δυνατότητας λειτουργίας νημάτων σε χώρους μνήμης πάνω στο τσιπ. Το τελευταίο, στην πράξη, μειώνει τον αριθμό των νημάτων που εκτελούνται, καθώς η μνήμη όπου καταχωρίζονται τα δεδομένα προς επεξεργασία είναι πιο μικρή, αλλά αυξάνει την απόδοση εκτέλεσης.
- Θα πρέπει, τέλος, να θυμόμαστε πάντα ότι οι κάρτες γραφικών δεν είναι κατάλληλες για όλα τα προβλήματα μεγάλων δεδομένων. Πιο συγκεκριμένα, δεν είναι κατάλληλες

για εφαρμογές με υψηλή διακλάδωση ή όταν θέλουμε χαμηλό χρόνο απόκρισης για εφαρμογές πραγματικού χρόνου. Σε ένα τέτοιο περιβάλλον, μια εντατική ροή δεδομένων καλύτερα να υπόκειται σε επεξεργασία με εργαλεία όπως τα Apache Spark και Flink, που έχουν συζητηθεί νωρίτερα στο βιβλίο, καθώς η μεταφορά των δεδομένων στην κύρια μνήμη της GPU κοστίζει.

10.4 Περαιτέρω μελέτη

Σε αυτό το κεφάλαιο παρουσιάσαμε, αρχικά, τρόπους να ποσοτικοποιούμε την αποδοτικότητα παράλληλων προγραμμάτων στο ευρύτερο προγραμματιστικό πλαίσιο του MapReduce, δίνοντας και μία μεθοδολογία ανάλυσης. Η μεθοδολογία επικεντρώνεται σε δύο βασικά χαρακτηριστικά των παράλληλων αλγορίθμων που σχετίζονται άμεσα με την απόδοση, τον όγκο δεδομένων αναλογικά που μεταφέρονται στο δίκτυο και τον όγκο εργασίας ενός reducer. Στη συνέχεια, εξετάσαμε τη χρήση καρτών γραφικών και συζητήθηκαν τα βασικά στοιχεία αυτού του προγραμματιστικού πλαισίου, όπως η ιεραρχία μνήμης και η οργάνωση νημάτων, πριν προχωρήσουμε σε συγκεκριμένα παραδείγματα χρήσης των καρτών γραφικών.

Περισσότερες πληροφορίες για τα θέματα MapReduce που συζητήθηκαν, πέρα από το βιβλίο [LRU19], που αναφέρθηκε στην εισαγωγή, μπορούν να βρεθούν στις εργασίες [ADK⁺16, ASSU13, AU11, AU13, KNG14].

Ο τρόπος χρήσης των καρτών γραφικών για γενικού σκοπού επεξεργασία και πολλά από τα θέματα αυτού του κεφαλαίου είναι το αντικείμενο πολλών βιβλίων, όπως αυτών που αναφέρονται στην εισαγωγή. Πέρα, όμως, από συγκεκριμένα βιβλία, για την αξία των καρτών γραφικών για επεξεργασία δεδομένων μπορεί κανείς να ανατρέξει σε εργασίες, όπως στις [KDK⁺11, MV15, Gre16, LKC⁺10]. Θέματα αρχιτεκτονικής και το πώς η πρόοδος σε επίπεδο υλικού επηρεάζει θετικά το GPGPU συζητούνται σε εργασίες, όπως στην [LBZ⁺20]. Αναλυτικές τεχνικές λεπτομέρειες για διαχείριση δεδομένων σε GPU μπορεί κανείς να αναζητήσει σε εργασίες, όπως στις [HLY⁺09, HYF⁺08, Sør11, BS10, ZC12, WZY⁺14, ZGJ⁺18, HNL⁺19, SMY20, FXW⁺21, BG22, BG21]. Καλές πρακτικές συζητούνται επίσης σε εργασίες, όπως στις [MV15, SRS⁺12, TLO12], ενώ η εργασία [SR15] ασχολείται με τη σύγκριση διαφορετικών πλαισίων για παράλληλο υπολογισμό.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [ADK⁺16] Foto N. Afrati, Shlomi Dolev, Ephraim Korach, Shantanu Sharma, and Jeffrey D. Ullman. Assignment problems of different-sized inputs in mapreduce. *TKDD*, 11(2):18:1–18:35, 2016.
- [ASSU13] Foto N. Afrati, Anish Das Sarma, Semih Salihoglu, and Jeffrey D. Ullman. Upper and lower bounds on the cost of a map-reduce computation. *PVLDB*, 6(4):277–288, 2013.
- [AU11] Foto N. Afrati and Jeffrey D. Ullman. Optimizing multiway joins in a map-reduce environment. *IEEE Trans. Knowl. Data Eng.*, 23(9):1282–1298, 2011.

- [AU13] Foto N. Afrati and Jeffrey D. Ullman. Matching bounds for the all-pairs mapreduce problem. In *IDEAS*, pages 3–4, 2013.
- [Bel22] Christos Bellas. *Advanced Joins on GPUs*. PhD Thesis, 2022.
- [BG21] Christos Bellas and Anastasios Gounaris. Hyset: A hybrid framework for exact set similarity join using a GPU. *Parallel Comput.*, 104-105:102790, 2021.
- [BG22] Christos Bellas and Anastasios Gounaris. Exploiting gpus for fast intersection of large sets. *Inf. Syst.*, 108:101992, 2022.
- [BS10] Peter Bakkum and Kevin Skadron. Accelerating sql database operations on a gpu with cuda. In *GPGPU*, pages 94–103, 2010.
- [FXW⁺21] Sofoklis Floratos, Mengbai Xiao, Hao Wang, Chengxin Guo, Yuan Yuan, Rubao Lee, and Xiaodong Zhang. Nestgpu: Nested query processing on GPU. In *37th IEEE International Conference on Data Engineering, ICDE 2021*, pages 1008–1019. IEEE, 2021.
- [Gre16] Samuel Greengard. Gpus reshape computing. *Commun. ACM*, 59(9):14–16, 2016.
- [HLY⁺09] Bingsheng He, Mian Lu, Ke Yang, Rui Fang, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. Relational query coprocessing on graphics processors. *ACM Trans. Database Syst.*, 34(4), 2009.
- [HNL⁺19] Donghyoung Han, Yoon-Min Nam, Jihye Lee, Kyongseok Park, Hyunwoo Kim, and Min-Soo Kim. Distme: A fast and elastic distributed matrix computation engine using gpus. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD*, pages 759–774, 2019.
- [HYF⁺08] Bingsheng He, Ke Yang, Rui Fang, Mian Lu, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. Relational joins on graphics processors. In *SIGMOD Conference*, pages 511–524, 2008.
- [KDK⁺11] Stephen W. Keckler, William J. Dally, Brucek Khailany, Michael Garland, and David Glasco. Gpus and the future of parallel computing. *IEEE Micro*, 31(5):7–17, 2011.
- [KH17] David Blair Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors - A Hands-on Approach, 3rd Ed.* Morgan Kaufmann, 2017.
- [KNG14] Ioannis K. Koumarelas, Athanasios Naskos, and Anastasios Gounaris. Binary theta-joins using mapreduce: Efficiency analysis and improvements. In *Proceedings of the Workshops of the EDBT/ICDT 2014 Joint Conference (EDBT/ICDT 2014), Athens, Greece, March 28, 2014*, pages 6–9, 2014.
- [LBZ⁺20] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. Pump up the volume: Processing large data on gpus with fast interconnects. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference*, pages 1633–1649, 2020.

- [LKC⁺10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, et al. Debunking the 100x gpu vs. cpu myth: an evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th annual international symposium on Computer architecture*, pages 451–460, 2010.
- [LRU19] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. *Mining of Massive Datasets, 3rd Ed.* Cambridge University Press, 2019.
- [Mat17] Norm Matloff. *Programming on Parallel Machines.* 2017.
- [MV15] Sparsh Mittal and Jeffrey S. Vetter. A survey of CPU-GPU heterogeneous computing techniques. *ACM Comput. Surv.*, 47(4):69, 2015.
- [NVI] NVIDIA. Cuda toolkit documentation. <http://docs.nvidia.com/cuda>.
- [SMY20] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. A study of the fundamental performance characteristics of gpus and cpus for database analytics. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*, pages 1617–1632, 2020.
- [Sør11] Hans Henrik Brandenborg Sørensen. Auto-tuning dense vector and matrix-vector operations for fermi gpus. In *PPAM (1)*, pages 619–629, 2011.
- [SR15] Dilpreet Singh and Chandan K. Reddy. A survey on platforms for big data analytics. *Journal of Big Data*, 2(1):1–20, 2015.
- [SRS⁺12] John A. Stratton, Christopher I. Rodrigues, I-Jui Sung, Li-Wen Chang, Nasser Anssari, Geng (Daniel) Liu, Wen mei W. Hwu, and Nady Obeid. Algorithm and data optimization techniques for scaling to massively threaded systems. *IEEE Computer*, 45(8):26–32, 2012.
- [TLO12] Stanley Tzeng, Brandon Lloyd, and John D. Owens. A gpu task-parallel model with dependency resolution. *IEEE Computer*, 45(8):34–41, 2012.
- [WZY⁺14] Kaibo Wang, Kai Zhang, Yuan Yuan, Siyuan Ma, Rubao Lee, Xiaoning Ding, and Xiaodong Zhang. Concurrent analytical query processing with gpus. *PVLDB*, 7(11):1011–1022, 2014.
- [ZC12] Guoliang Zhou and Hong Chen. Parallel cube computation on modern cpus and gpus. *The Journal of Supercomputing*, 61(3):394–417, 2012.
- [ZGJ⁺18] Jingbo Zhou, Qi Guo, H. V. Jagadish, Lubos Krcál, Siyuan Liu, Wenhao Luan, Anthony K. H. Tung, Yueji Yang, and Yuxin Zheng. A generic inverted index framework for similarity search on the GPU. In *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*, pages 893–904, 2018.

ΑΛΓΟΡΙΘΜΙΚΑ ΘΕΜΑΤΑ ΑΝΑΛΥΣΗΣ ΜΕΓΑΛΩΝ ΔΕΔΟΜΕΝΩΝ

Περιεχόμενα

11.1	Εισαγωγή	265
11.2	Υπολογισμός ομοιότητας	265
11.2.1	Ομοιότητα εγγράφων - minhashing	266
11.2.2	Ομοιότητα εγγράφων - υπολογισμός	268
11.2.3	Μετρικές απόστασης	271
11.3	Επεξεργασία ροών	272
11.3.1	Αντιπροσωπευτικά δείγματα και φίλτρα Bloom	272
11.3.2	Μέτρηση διακριτών στοιχείων	274
11.4	Μηχανές αναζήτησης	275
11.4.1	Ο αλγόριθμος PageRank	276
11.4.2	Επεκτάσεις και θέματα υλοποίησης	279
11.5	Κανόνες συσχέτισης	281
11.5.1	Αλγόριθμος A-Priori	282
11.5.2	Αντιμετωπίζοντας το πρόβλημα της καταμέτρησης ζευγών και η επίλυσή του	285
11.5.3	Περιορισμός περασμάτων	286
11.6	Ομαδοποίηση	287
11.6.1	Ο αλγόριθμος των Bradley, Fayyad και Reina	289
11.6.2	Ο αλγόριθμος CURE	291
11.6.3	Ο αλγόριθμος DBSCAN	292

Καρακασίδης, Α., Κολωνiάρη, Γ., Γούναρης, Α., & Παπαδόπουλος, Α. (2025). Τεχνολογίες Επεξεργασίας και Ανάλυσης Μεγάλων Δεδομένων [Μεταπτυχιακό εγχειρίδιο]. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις. <http://dx.doi.org/10.57713/kallipos-1058>

11.7	Μηχανική μάθηση για μεγάλα δεδομένα	293
11.7.1	Αντίληπτρα (perceptrons)	294
11.7.2	Μηχανές διανυσμάτων υποστήριξης	295
11.7.3	Εκμάθηση από πλησιέστερους γείτονες	296
11.7.4	Δένδρα αποφάσεων	297
11.8	Ανάλυση μεγάλων γράφων κοινωνικών δικτύων	298
11.8.1	Ενδιαμεσότητα (betweenness)	299
11.8.2	SimRank	299
11.8.3	Καταμέτρηση τριγώνων	301
11.9	Συζήτηση και περαιτέρω μελέτη	302
	BIBΛΙΟΓΡΑΦΙΑ	303

Σύνοψη

Το παρόν κεφάλαιο εξετάζει κλιμακώσιμες προσεγγίσεις για προβλήματα μεγάλων δεδομένων στα παρακάτω θέματα:

- Υπολογισμός ομοιότητας, δίνοντας έμφαση σε ομοιότητα εγγράφων και τεχνικές *LSH*.
- Επεξεργασία ροών, συζητώντας δομές όπως τα φίλτρα *Bloom*.
- Τεχνικές για μηχανές αναζήτησης, επικεντρώνοντας την προσοχή στην τεχνική *PageRank*.
- Εύρεση συχνών στοιχειοσυνόλων με αλγορίθμους βασιζόμενους τόσο στον *A-priori* όσο και σε δομές όπως στο *FP-δένδρο*.
- Συσταδοποίηση με πολλαπλούς τρόπους, συμπεριλαμβανομένων του διαμερισμού περιοχής, ιεραρχικών αλγορίθμων και αλγορίθμων βάσει πυκνότητας.
- Επιβλεπόμενη μηχανική μάθηση, συζητώντας τεχνικές που χρησιμοποιούνται σε νευρωνικά δίκτυα και δένδρα απόφασης.
- Γράφοι κοινωνικών δικτύων, εξετάζοντας θέματα όπως την ενδιαμεσότητα, την τεχνική *SimRank* και την καταμέτρηση τριγώνων.

Προαπαιτούμενη γνώση

Δεν υπάρχει κάποια προαπαιτούμενη γνώση, όμως ο αναγνώστης, όπως και στο προηγούμενο κεφάλαιο, θα διευκολυνθεί εάν έχει βασικές γνώσεις διαχείρισης και εξόρυξης δεδομένων, μηχανικής μάθησης και ανάκτησης πληροφορίας.

11.1 Εισαγωγή

Τα προηγούμενα κεφάλαια επικεντρώθηκαν σε τεχνολογίες που επιτρέπουν την ανάλυση και επεξεργασία μεγάλων δεδομένων. Για λόγους πληρότητας, σε αυτό το κεφάλαιο θα εξετάσουμε περιληπτικά αλγορίθμους και τεχνικές που εκμεταλλεύονται και συμπληρώνουν αυτές τις τεχνολογίες. Ούτε οι τεχνολογίες από μόνες τους ούτε οι τεχνικές από μόνες τους μπορούν να οδηγήσουν σε αποδοτική αντιμετώπιση των προκλήσεων επεξεργασίας και ανάλυσης μεγάλων δεδομένων. Συνδυαστικά, όμως, αυτό μπορεί να επιτευχθεί.

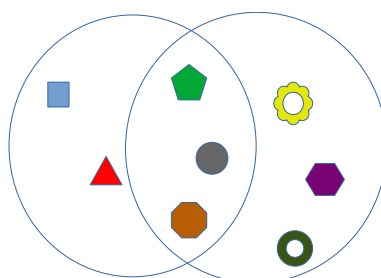
Πιο συγκεκριμένα, σε αυτό το κεφάλαιο θα δούμε θέματα α) υπολογισμού ομοιότητας που κρύβεται πίσω από ένα ιδιαίτερα ευρύ πεδίο τεχνικών εξόρυξης γνώσης από δεδομένα, β) επεξεργασίας ροών, και γ) υποστήριξης μηχανών αναζήτησης, όπως και τεχνικών για τις πιο κλασικές λειτουργίες σύγχρονης εξόρυξης δεδομένων. Η τελευταία κατηγορία περιλαμβάνει 1) κανόνες συσχέτισης, 2) ομαδοποίηση, 3) μηχανική μάθηση, και 4) ανάλυση πολύ μεγάλων γράφων κοινωνικών δικτύων.

Κάθε ένα από τα 7 παραπάνω θέματα θα αποτελεί μία ξεχωριστή ενότητα στο κεφάλαιο αυτό, ενώ η βάση του υλικού είναι, όπως και στην πρώτη ενότητα του προηγούμενου κεφαλαίου, το διεθνώς αναγνωρισμένο και ελεύθερο βιβλίο των Jure Leskovec, Anand Rajaraman και Jeffrey D. Ullman με τίτλο *Mining of Massive Datasets* [LRU19], στο οποίο παραπέμπεται ο αναγνώστης ως σημείο εκκίνησης περαιτέρω διερεύνησης.

11.2 Υπολογισμός ομοιότητας

Πολλές φορές, καλούμαστε να εντοπίσουμε ή να ανακτήσουμε όμοια ή σχεδόν όμοια αντικείμενα από μεγάλες συλλογές δεδομένων. Θα εστιάσουμε σε έναν συγκεκριμένο τύπο ομοιότητας, την ομοιότητα Jaccard, που αναφέρεται σε αντικείμενα που το κάθε ένα περιέχει ένα σύνολο εγγραφών και η ομοιότητα μεταξύ αυτών των συνόλων ορίζεται από το σχετικό μέγεθος των τομών τους. Αυτό το πρόβλημα εμφανίζεται σε μια ιδιαίτερα ευρεία ποικιλία εφαρμογών. Για παράδειγμα, επιτρέπει τον εντοπισμό ιστοσελίδων που μοιράζονται σε μεγάλο βαθμό κοινό κείμενο και τη δημιουργία προσωποποιημένων προσφορών σε πελάτες βάσει επιλογών άλλων πελατών που έχουν αγοράσει παρόμοια αντικείμενα.

Η ομοιότητα Jaccard δύο συνόλων S και T είναι ίση με $|S \cap T|/|S \cup T|$, δηλαδή είναι ίση με τον λόγο του πληθικού αριθμού της τομής των S και T προς τον πληθικό αριθμό της ένωσής τους. Εφεξής, θα αναπαριστούμε την ομοιότητα Jaccard μεταξύ δύο συνόλων S και T ως



Σχήμα 11.1: Δύο σύνολα με ομοιότητα Jaccard 3/8.

$\text{SIM}(S, T)$. Στο Σχήμα 11.1 δίνονται δύο σύνολα S και T . Υπάρχουν τρία στοιχεία στην τομή τους και συνολικά οκτώ στοιχεία στην ένωσή τους. Άρα: $\text{SIM}(S, T)=3/8$.

Προφανώς, μια άμεση προσέγγιση ωμής βίας, όπου εξετάζεται κάθε ζεύγος αντικειμένων, έχει τετραγωνική, δηλαδή απαγορευτικά υψηλή πολυπλοκότητα, δεδομένου ότι ακόμη και ένα μικρό ή έστω μικρομεσαίο σύνολο δεδομένων της τάξης του ενός εκατομμυρίου αντικειμένων εμπεριέχει μισό τρισεκατομμύριο ζεύγη προς εξέταση.

Ευτυχώς, έχουμε στη διάθεσή μας τεχνικές που κλιμακώνονται και είναι ιδιαίτερα πρακτικές. Μια τέτοια τεχνική, ή καλύτερα οικογένεια τεχνικών, είναι ο *κατακερματισμός με ευαισθησία στην τοπικότητα* (locality sensitive hashing, LSH), που μας επιτρέπει να ελέγχουμε πιο στοχευμένα πιθανώς όμοια ζεύγη, χωρίς να χρειάζεται να τα εξετάσουμε όλα ένα προς ένα. Έτσι, είναι δυνατό να αποφύγουμε την τετραγωνική αύξηση του χρόνου υπολογισμού που απαιτείται από τον αλγόριθμο ωμής βίας. Παρόλο που θα επικεντρωθούμε σε ομοιότητα Jaccard σε έγγραφα, οι τεχνικές που θα συζητήσουμε γενικεύονται και σε άλλες περιπτώσεις.

Η γενική ιδέα πίσω από το LSH είναι να κατακερματίζουμε αντικείμενα χρησιμοποιώντας πολλές συναρτήσεις κατακερματισμού, που είναι σχεδιασμένες με τρόπο ώστε παρόμοια ζεύγη να καταλήγουν σε κοινό κάδο. Αυτό επιτρέπει να ελεγχθούν μόνο ζεύγη αντικειμένων που είναι στον ίδιο κάδο, σύμφωνα με τουλάχιστον μία από τις εφαρμοσμένες συναρτήσεις κατακερματισμού. Συνήθως, υπάρχει ένα μειονέκτημα για τον κατακερματισμό με ευαισθησία στην τοπικότητα, εξαιτίας της παρουσίας ψευδών αρνητικών, δηλαδή ζευγών αντικειμένων που είναι παρόμοια αλλά δεν περιλαμβάνονται στο σύνολο των ζευγών που εξετάζουμε. Αυτό, συνήθως, το αντιμετωπίζουμε με μια ρύθμιση και έχει ως κόστος την αύξηση του πλήθους των υποψήφιων προς εξέταση ζευγών. Εάν, όμως, αυτή η ρύθμιση γίνει προσεκτικά, η σχετική αύξηση είναι περιορισμένη.

11.2.1 Ομοιότητα εγγράφων - minhashing

Ας πάρουμε ένα μόνο παράδειγμα, όπου θέλουμε να εντοπίσουμε παρόμοια έγγραφα. Αρχικά, θα πρέπει να βρούμε έναν αποτελεσματικό τρόπο αναπαράστασης ενός εγγράφου ως συνόλου. Μία βασική επιλογή είναι η εξαγωγή από το έγγραφο συνεχόμενων αλφαριθμητικών συγκεκριμένου μεγέθους k , τα οποία ονομάζονται *θρύμματα* (shingles). Το μέγεθος k ορίζεται εμπειρικά, π.χ. 9. Μετά, αυτά τα αλφαριθμητικά μπορούν να αναπαρασταθούν με τη βοήθεια μίας συνάρτησης κατακερματισμού σε ένα μικρότερο σύνολο τιμών, π.χ., έτσι ώστε το κάθε shingle να απαιτεί 4 bytes. Η παραπάνω διαδικασία έχει καταφέρει να αναπαραστήσει ένα έγγραφο ως σύνολο στοιχείων, αλλά οδηγεί σε σημαντική αύξηση του απαιτούμενου χώρου. Εάν κάθε shingle αναπαρασταθεί με 4 bytes, χονδρικά απλά τετραπλασιάζεται ο απαιτούμενος χώρος για κάθε έγγραφο. Είναι, λοιπόν, σαφές ότι ακόμη και μεσαίου μεγέθους συλλογές 1 εκατομμυρίου εγγράφων απαιτούν χώρο εκατοντάδων GBs και περισσότερο. Οπότε, πριν προσπαθήσουμε να εφαρμόσουμε την τεχνική LSH, θα αντικαταστήσουμε αυτά τα μεγάλα σύνολα στοιχείων ανά έγγραφο με μικρότερες αναπαραστάσεις που ονομάζονται *υπογραφές* (signatures).

Πριν, όμως, εξηγήσουμε την κατασκευή μικρών υπογραφών από μεγάλα σύνολα, θα βοηθήσει πολύ να αναπαραστήσουμε μία συλλογή συνόλων ως έναν *χαρακτηριστικό πίνακα*. Οι στήλες του πίνακα αντιστοιχούν στα σύνολα, ενώ οι γραμμές αντιστοιχούν σε όλα τα δυνατά

Στοιχείο	S_1	S_2	S_3	S_4
a	1	0	1	0
b	1	0	0	1
c	0	1	0	1
d	0	1	0	1
e	0	1	0	1
f	1	0	1	0
g	1	0	1	0

Σχήμα 11.2: Ένας πίνακας που αναπαριστά τέσσερα σύνολα.

στοιχεία (θρύμματα) που μπορεί να εμφανίζονται στα σύνολα. Υπάρχει η τιμή 1 στη γραμμή r και στήλη c , εάν το στοιχείο που αντιστοιχεί στη γραμμή r είναι μέλος του συνόλου όπου αντιστοιχεί η στήλη c . Διαφορετικά, η τιμή στη θέση (r, c) είναι 0. Στο Σχήμα 11.2 φαίνεται ένα παράδειγμα πίνακα που αναπαριστά σύνολα που περιέχουν τα στοιχεία $\{a, b, c, d, e, f, g\}$. Τα σύνολα είναι τα $S_1 = \{a, b, f, g\}$, $S_2 = \{c, d, e\}$, $S_3 = \{a, f, g\}$ και $S_4 = \{b, c, d, e\}$. Η πρώτη γραμμή και η πρώτη στήλη δεν είναι μέρος του πίνακα και χρησιμοποιούνται μόνο για υπενθύμιση σχετικά με το πού αντιστοιχούν οι γραμμές και οι στήλες.

Οι υπογραφές που θέλουμε να παραγάγουμε για τα σύνολα αποτελούνται από τα αποτελέσματα μεγάλου αριθμού υπολογισμών, κάθε ένας από τους οποίους είναι ένα minhash του χαρακτηριστικού πίνακα. Η εύρεση ενός minhash για ένα σύνολο που αναπαρίσταται από μία στήλη του χαρακτηριστικού πίνακα περιλαμβάνει δύο βήματα: α) την αναδιάταξη των γραμμών με τη βοήθεια μίας συνάρτησης κατακερματισμού, και β) τον εντοπισμό για κάθε στήλη του αριθμού της πρώτης γραμμής, μετά την αναδιάταξη, όπου η στήλη έχει τιμή 1. Αυτός ο αριθμός γραμμής είναι η minhash τιμή. Το σημαντικό είναι ότι η πιθανότητα η συνάρτηση minhash για μια τυχαία αναδιάταξη γραμμών να δώσει την ίδια τιμή για δύο σύνολα, δηλαδή για δύο στήλες του πίνακα, είναι ίση με την ομοιότητα Jaccard αυτών των δύο συνόλων. Ενώ ο χαρακτηριστικός πίνακας έχει μέγεθος ίσο με το πλήθος θρυμμάτων επί το πλήθος εγγράφων, προκύπτει ένας νέος πίνακας μεγέθους ίσου με το πλήθος αναδιατάξεων επί το πλήθος εγγράφων, κρατώντας παρόμοια ομοιότητα μεταξύ των στηλών στους δύο πίνακες.

Επιγραμματικά, ας υποθέσουμε ότι έχουμε σύνολο εγγράφων στη μορφή χαρακτηριστικού πίνακα, όπου το πλήθος των γραμμών είναι προφανώς ιδιαίτερα μεγάλο. Επιλέγουμε τυχαία έναν αριθμό n από αναδιατάξεις των γραμμών του πίνακα, όπου το n είναι στην τάξη των εκατοντάδων. Έστω ότι οι συναρτήσεις minhash που καθορίζονται από αυτές τις αναδιατάξεις είναι οι $h_1, h_2, \dots, h_n$. Για τη στήλη που αναπαριστά το σύνολο S , κατασκευάζουμε την υπογραφή minhash του S που είναι το διάνυσμα $[h_1(S), h_2(S), \dots, h_n(S)]^T$. Κατά αυτόν τον τρόπο, μπορούμε να παραγάγουμε από τον αρχικό χαρακτηριστικό πίνακα τον *πίνακα υπογραφών*, όπου οι στήλες του χαρακτηριστικού πίνακα έχουν αντικατασταθεί από την υπογραφή minhash για το κάθε έγγραφο.

Παράδειγμα 11.1. Στο παράδειγμα του Σχήματος 11.2, ας υποθέσουμε ότι παίρνουμε την αναδιατεταγμένη σειρά των γραμμών $eabgfdc$. Η αναδιάταξη ορίζει μία συνάρτηση minhash h που απεικονίζει τα σύνολα σε γραμμές. Ας υπολογίσουμε την τιμή minhash του συνόλου S_1 με βάση την h . Η πρώτη στήλη του χαρακτηριστικού πίνακα, που αντιστοιχεί στο σύνολο S_1 ,

	S_1	S_2	S_3	S_4	Αναδιατάξεις		
a	1	0	1	0	e	c	f
b	1	0	0	1	a	b	d
c	0	1	0	1	b	d	a
d	0	1	0	1	g	a	b
e	0	1	0	1	f	g	g
f	1	0	1	0	d	e	e
g	1	0	1	0	c	f	c

S_1	S_2	S_3	S_4
a	e	a	e
b	c	a	c
f	d	f	d

Σχήμα 11.3: Παράδειγμα μετατροπής του χαρακτηριστικού πίνακα σε πίνακα υπογραφών.

έχει 0 στη γραμμή e , οπότε προχωρούμε στη γραμμή a , όπου υπάρχει 1 και, άρα, $h(S_1) = a$. Για τη στήλη S_2 , στη γραμμή e υπάρχει 1, οπότε $h(S_2) = e$. Για τη στήλη S_3 , στη γραμμή e υπάρχει 0, οπότε προχωρούμε στην επόμενη γραμμή a με βάση την αναδιατάξη. Αφού εκεί υπάρχει 1, θέτουμε $h(S_3) = a$. Αντίστοιχα, βρίσκουμε ότι $h(S_4) = e$. Στο Σχήμα 11.3 δείχνουμε επιπλέον δύο αναδιατάξεις.

Εάν πάρουμε την ομοιότητα Jaccard ανάμεσα στις στήλες 1 και 2 στον χαρακτηριστικό πίνακα, αυτή είναι 0. Ομοίως, και η ομοιότητα στον πίνακα υπογραφών είναι 0. Οι αντίστοιχες ομοιότητες για S_1 και S_3 είναι 0,75 και 0,67. Αντίστοιχα, βλέπουμε ότι και για τα υπόλοιπα ζεύγη εγγράφων οι ομοιότητες είναι παρόμοιες, και αυτό θα συνέβαινε σε ακόμη μεγαλύτερο βαθμό εάν το πλήθος n των αναδιατάξεων ήταν μεγαλύτερο του 3. $\square$

Τέλος, σημειώνουμε ότι οι υπογραφές μπορεί να είναι βάσει της θέσης του πρώτου στοιχείου σε μία αναδιατάξη. Το αποτέλεσμα θα παραμείνει το ίδιο, με την έννοια ότι πάλι η ομοιότητα μεταξύ των στηλών θα είναι προσεγγιστικά ίση με την ομοιότητα των διανυσμάτων που προκύπτουν από τη θρυμματοποίηση. Στο παράδειγμα του Σχήματος 11.3, για το έγγραφο S_1 το αντίστοιχο διάνυσμα είναι το $[2\ 2\ 1]$ (αντί για $[a\ b\ f]$), καθώς στην 1η αναδιατάξη το δεύτερο στοιχείο αντιστοιχεί σε 1, στη 2η αναδιατάξη πάλι το δεύτερο στοιχείο αντιστοιχεί σε 1, ενώ στην 3η αναδιατάξη το πρώτο στοιχείο αντιστοιχεί σε 1. Ομοίως, το διάνυσμα για το S_2 γίνεται $[1\ 1\ 2]$, κ.ο.κ. Μια τέτοια προσέγγιση επιτρέπει και αποδοτικότερο υπολογισμό του πίνακα υπογραφών. Γενικότερα, και στην πρώτη προσέγγιση, αντί για τα στοιχεία, είναι αποδοτικότερο να καταχωρίζεται το id τους.

11.2.2 Ομοιότητα εγγράφων - υπολογισμός

Με την παραπάνω minhash διαδικασία, έχουμε καταφέρει να αναπαραστήσουμε έγγραφα ως σύνολα αντικειμένων χρησιμοποιώντας μικρές υπογραφές, η μέση ομοιότητα των οποίων αναμένεται να είναι ίση με την ομοιότητα μεταξύ των εγγράφων. Έχουμε αντιμετωπίσει το πρόβλημα του χώρου. Π.χ., χρησιμοποιώντας 200 αναδιατάξεις πλέον κάθε έγγραφο απαι-

οποιασδήποτε λωρίδας είναι $(1 - s^r)^b$. Άρα, η πιθανότητα ότι οι υπογραφές ταιριάζουν σε όλες τις γραμμές τουλάχιστον μίας λωρίδας και, επομένως, θα γίνει υποψήφιο ζεύγος, είναι $1 - (1 - s^r)^b$. Αυτά τα υποψήφια ζεύγη μπορούν μετά να ελεγχθούν για την απάλειψη ψευδώς θετικών αποτελεσμάτων.

Αναλόγως με το πόσο μεγάλη πιθανότητα θέλουμε να υπάρχουν ψευδώς αρνητικά αποτελέσματα, καθορίζονται τα b και r . Για παράδειγμα, έστω ότι επιθυμούμε να εντοπίσουμε έγγραφα με ομοιότητα Jaccard τουλάχιστον 0.8 και χρησιμοποιούμε υπογραφές μήκους 100. Έστω, επίσης, ότι διαιρούμε τον πίνακα υπογραφών σε 20 λωρίδες με 5 γραμμές ανά λωρίδα. Αν ελέγξουμε δύο έγγραφα με 80% ομοιότητα, τότε σε οποιαδήποτε λωρίδα έχουν πιθανότητα ίση με $0,8^5 = 0,328$ να συμφωνούν σε όλες τις γραμμές και άρα να γίνουν ένα υποψήφιο ζεύγος. Υπάρχουν, όμως, 20 λωρίδες και, άρα, 20 ευκαιρίες να γίνουν υποψήφιο ζεύγος. Η πιθανότητα να μην γίνουν ζεύγος σε μία λωρίδα είναι μεν $1 - 0,328 = 0,672$, αλλά η πιθανότητα να μην είναι ζεύγος σε καμία από τις 20 λωρίδες είναι 0,00035. Χονδρικά, μόνο ένα σε 3.000 ζεύγη εγγράφων με ομοιότητα 80% θα αποτύχει να γίνει υποψήφιο ζεύγος και, άρα, θα έχουμε μία ψευδή αρνητική απόκριση.

Είναι σημαντικό να διευκρινιστεί ότι οι τεχνικές shingling, minhashing και LSH είναι καλό να συνδυάζονται ως εξής, παρέχοντας μια ολοκληρωμένη λύση:

1. Shingling. Επιλέγουμε μία σταθερά k και κατασκευάζουμε για κάθε έγγραφο το σύνολο των k -θρυμμάτων, τα οποία μπορούμε να απεικονίσουμε σε μικρότερους αριθμούς που αντιστοιχούν σε κάδους. Βάσει κοινής ταξινόμησης των θρυμμάτων, κάθε έγγραφο απεικονίζεται με ένα διάνυσμα.
2. Minhashing. Επιλέγουμε το πλήθος αναδιατάξεων n που ισούται με το μήκος των υπογραφών minhash και μετατρέπουμε τα αρχικά διανύσματα σε μικρότερες υπογραφές.
3. LSH. Επιλέγουμε ένα κατώφλι t που καθορίζει το πόσο όμοια πρέπει να είναι τα έγγραφα, ώστε να επιλεγούν ως επιθυμητά όμοια ζεύγη. Επιλέγουμε το πλήθος των λωρίδων b και έναν αριθμό r για τις γραμμές, ώστε $b \cdot r = n$, ενώ το κατώφλι t θα προσεγγίζει το $(1/b)^{1/r}$. Εάν η αποφυγή ψευδών αρνητικών αποκρίσεων είναι σημαντική, μπορούμε να επιλέξουμε τα b και r ώστε να παραχθεί ένα κατώφλι χαμηλότερο από t . Εάν η ταχύτητα είναι σημαντική και επιθυμούμε να μειώσουμε τις ψευδώς θετικές αποκρίσεις, τότε επιλέγουμε b και r ώστε να παραγάγουν ένα υψηλότερο κατώφλι από το δοθέν t . Κατασκευάζουμε υποψήφια ζεύγη εφαρμόζοντας την τεχνική LSH.
4. Καθαρισμός αποτελεσμάτων από ψευδώς θετικά. Ελέγχουμε τις υπογραφές κάθε υποψήφιου ζεύγους για εύρεση της πραγματικής ομοιότητάς τους και το εάν αυτή ξεπερνά το κατώφλι t . Επιπρόσθετα, εάν θέλουμε, μπορούμε να ελέγξουμε και τα πραγματικά αντίστοιχα έγγραφα για ομοιότητα.

Σημειώνουμε, τέλος, ότι οι μέθοδοι βασισμένες σε LSH είναι περισσότερο αποτελεσματικές όταν ο βαθμός ομοιότητας είναι σχετικά μικρός. Όταν θέλουμε να βρούμε σύνολα που είναι σχεδόν ή εντελώς ίδια, μπορούμε να χρησιμοποιήσουμε και άλλες μεθόδους, χωρίς μάλιστα να έχουμε να αντιμετωπίσουμε το πρόβλημα των ψευδώς αρνητικών αποτελεσμάτων. Πιο συγκεκριμένα, τα διάφορα αντικείμενα - σύνολα στοιχείων, μπορούν να αναπαρασταθούν

ως αλφαριθμητικά βάσει των ταξινομημένων στοιχείων που περιέχουν. Τότε, σχεδόν ίδια σύνολα θα πρέπει να έχουν αλφαριθμητικά σχεδόν ίδιου μήκους, κάτι που επιτρέπει τη σύγκριση εντός αλφαριθμητικού μόνο με έναν περιορισμένο αριθμό άλλων αλφαριθμητικών. Επιπρόσθετα, μπορεί να γίνει δεικτοδότηση βάσει του προθέματός τους ή και της θέσης τους ή του επιθέματός τους.

11.2.3 Μετρικές απόστασης

Μέχρι τώρα, η συζήτηση βασιζόταν στην ομοιότητα Jaccard, που, όπως υποδηλώνει και η ονομασία, δεν είναι μια μετρική απόστασης. Όμως, η ποσότητα που είναι ίση με 1 μείον την ομοιότητα Jaccard είναι μια μετρική απόστασης και ονομάζεται *απόσταση Jaccard*. Δεν είναι, όμως, η μοναδική μετρική απόστασης. Αναλόγως με τη μετρική, διαφοροποιείται η τεχνική LSH, ενώ αυτές οι μετρικές έχουν μεγάλη σημασία και σε άλλες λειτουργίες της ανάλυσης δεδομένων, όπως στην ομαδοποίηση, που θα δούμε παρακάτω.

Πώς ορίζεται όμως μια μετρική απόστασης; Μια *μετρική απόστασης* σε έναν χώρο που αποτελείται από ένα σύνολο σημείων είναι μία συνάρτηση $d(x, y)$ που λαμβάνει 2 σημεία για είσοδο x και y και παράγει έναν πραγματικό αριθμό με τις ακόλουθες ιδιότητες: $d(x, y) \geq 0$, $d(x, y)=0$, αν και μόνο αν τα δύο σημεία ταυτίζονται, $d(x, y) = d(y, x)$ και $d(x, y) \leq d(x, z) + d(z, y)$. Η τελευταία ιδιότητα καλείται *τριγωνική ανισότητα* και είναι η πολυπλοκότερη συνθήκη.

Οι πλέον συνηθισμένες μετρικές απόστασης, πέρα από την απόσταση Jaccard, είναι οι εξής:

- Ευκλείδεια απόσταση. Ένας n -διάστατος ευκλείδειος χώρος είναι ένας χώρος, όπου τα σημεία αναπαρίστανται ως διανύσματα n πραγματικών αριθμών. Η τυπική μετρική απόστασης σε αυτόν τον χώρο, στην οποία πολλές φορές αναφερόμαστε και ως *νόρμα* L_2 , ορίζεται ως:

$$d([x_1, x_2, \dots, x_n], [y_1, y_2, \dots, y_n]) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Στον ευκλείδειο χώρο αυτή η απόσταση γενικεύεται. Πιο συγκεκριμένα, για οποιαδήποτε σταθερά r (και όχι μόνο για $r = 2$, όπως στην ευκλείδεια απόσταση) μπορούμε να ορίσουμε την απόσταση L_r να είναι η μετρική απόστασης d που ορίζεται ως εξής:

$$d([x_1, x_2, \dots, x_n], [y_1, y_2, \dots, y_n]) = \left(\sum_{i=1}^n |x_i - y_i|^r \right)^{1/r}$$

Όταν $r=1$, έχουμε την *απόσταση Manhattan*. Μία άλλη ενδιαφέρουσα μετρική απόστασης είναι η L_∞ , που είναι το όριο της L_r καθώς το r τείνει στο άπειρο, όπου κυριαρχεί η διάσταση με τη μέγιστη διαφορά.

- Απόσταση συνημίτονου. Υπολογίζουμε πρώτα το συνημίτονο της γωνίας μεταξύ δύο διανυσμάτων x και y διαιρώντας το εσωτερικό γινόμενο με τις L_2 -νόρμες των x και y και, έπειτα, μετατρέπουμε το αποτέλεσμα σε μία γωνία στο εύρος $[0, 180]$ μοίρες.

- Απόσταση διόρθωσης. Όταν τα σημεία είναι αλφαριθμητικά, η απόσταση μεταξύ τους είναι το ελάχιστο πλήθος εισαγωγών και διαγραφών απλών χαρακτήρων που μετατρέπουν το x σε y .
- Απόσταση Hamming. Ισούται με το πλήθος των συνιστωσών που διαφέρουν δύο διανύσματα x και y .

Διευκρινίζεται ότι δεν προαπαιτούνται αναγκαστικά ευκλείδειοι χώροι, π.χ. να ορίζεται ο μέσος όρος των σημείων στις παραπάνω μετρικές.

11.3 Επεξεργασία ροών

Είναι πολλά τα παραδείγματα πραγματικού κόσμου όπου τα δεδομένα παράγονται με τη μορφή ροής. Αισθητήρες, δορυφόροι, κάμερες παρακολούθησης, καταγραφείς κίνησης σε εξυπηρετητές δικτύου συνεχώς παράγουν δεδομένα με υψηλή συχνότητα και, επιπρόσθετα, δημιουργούν την ανάγκη της άμεσης επεξεργασίας αυτών των δεδομένων. Κατά την επεξεργασία ροών δεδομένων, υποθέτουμε ότι είναι αδύνατη η (έστω προσωρινή) αποθήκευση δεδομένων και εκ των υστέρων επεξεργασία τους. Για την επίτευξη της επεξεργασίας σε (σχεδόν) πραγματικό χρόνο, είναι καίριας σημασίας η δημιουργία περιλήψεων ή αλλιώς συνόψεων, ενώ είναι αποδεκτή μια προσεγγιστική απάντηση εάν έχει πολύ μικρότερες απαιτήσεις σε αποθηκευτικό χώρο και υπολογιστική ισχύ.

11.3.1 Αντιπροσωπευτικά δείγματα και φίλτρα Bloom

Ένα αρχικό ερώτημα που θα μας απασχολήσει είναι αυτό της δημιουργίας ενός αντιπροσωπευτικού και αξιόπιστου δείγματος. Έστω ότι σε μία ροή δεδομένων καταγραφής μιας μηχανής αναζήτησης θέλουμε να μελετήσουμε το ποσοστό των ερωτημάτων που έχουν επαναληφθεί από τους ίδιους χρήστες, κρατώντας το 10% μόνο των δεδομένων. Εάν ακολουθήσουμε μία απλοϊκή προσέγγιση, σύμφωνα με την οποία αποθηκεύουμε μόνο το 10% των ερωτημάτων κάθε χρήστη, τότε μόνο το 1% των ερωτημάτων που έχουν υποβληθεί δύο φορές θα εμφανίζονται δύο φορές και στο δείγμα, ενώ ένα ποσοστό των ερωτημάτων που έχουν υποβληθεί στην πραγματικότητα πολλαπλές φορές θα εμφανίζεται μόνο μία φορά στο δείγμα. Μία ορθότερη προσέγγιση είναι να κρατούνται όλα τα ερωτήματα του 10% των χρηστών στο δείγμα, κάτι το οποίο μπορεί να υποστηριχθεί εύκολα από μία συνάρτηση κατακερματισμού, χωρίς την ανάγκη να αποθηκεύεται λίστα με τους χρήστες που έχουν εμφανιστεί μέχρι τότε στη ροή ή για ειδική μεταχείριση των νέων χρηστών.

Στη γενική περίπτωση, όταν θέλουμε ένα δείγμα μεγέθους x/b , κατακερματίζουμε το κλειδί των δεδομένων, που στο παράδειγμά μας είναι ο χρήστης, σε b κάδους και κρατάμε στο δείγμα μόνο τις εγγραφές που κατακερματίζονται στους πρώτους x κάδους. Αυτή η προσέγγιση επιτρέπει να μειωθεί δυναμικά το μέγεθος του δείγματος. Π.χ., για εξοικονόμηση χώρου, να κρατηθούν δεδομένα που κατακερματίζονται στους πρώτους $x - 1$ ή ακόμη λιγότερους κάδους.

Οι συναρτήσεις κατακερματισμού μπορούν να αποδειχθούν χρήσιμα εργαλεία και για την

εφαρμογή τελεστών επιλογής πάνω σε δεδομένα ροής, όπου το κριτήριο είναι το εάν οι τιμές του γνωρίσματος στη συνθήκη επιλογής ανήκουν σε κάποιο ενδεχομένως πολύ μεγάλο σύνολο S . Στην τεχνική που είναι γνωστή ως *φίλτρο Bloom* η διαθέσιμη μνήμη χρησιμοποιείται ως πίνακας από δυαδικά ψηφία (bits). Εάν, π.χ., έχουμε διαθέσιμο 1GB κύριας μνήμης, μπορούμε να αποθηκεύσουμε οκτώ δισεκατομμύρια bits, αφού το κάθε byte αποτελείται από οκτώ bits. Έστω h μία συνάρτηση κατακερματισμού. Κάθε στοιχείο του συνόλου S κατακερματίζεται σε μία θέση του πίνακα με τα bits και το αντίστοιχο bit γίνεται 1, ενώ τα υπόλοιπα bits παραμένουν ως έχουν. Το σημαντικό είναι το πλήθος των θέσεων bit να είναι μικρότερο από το μέγεθος του S . Όταν έρχεται ένα νέο στοιχείο από τη ροή, εφαρμόζεται η συνάρτηση κατακερματισμού. Εάν το αντίστοιχο bit είναι 1, τότε επιτρέπουμε τη διέλευση του στοιχείου. Εάν, όμως, το bit είναι 0, τότε είμαστε βέβαιοι ότι το συγκεκριμένο στοιχείο πρέπει να φιλτραριστεί και να μην υποστεί περαιτέρω επεξεργασία.

Δυστυχώς, μερικά ανεπιθύμητα στοιχεία δεν θα απορριφθούν, δηλαδή δεν μπορούν να αποφευχθούν πλήρως τα ψευδώς θετικά αποτελέσματα. Αντιθέτως, εξ ορισμού, δεν υπάρχουν ψευδώς αρνητικά. Υπάρχουν και άλλες εναλλακτικές, όπως θα δούμε κατά τη μελέτη του φίλτρου Bloom. Για παράδειγμα, θα μπορούσαμε να χρησιμοποιήσουμε μία αλληλουχία από φίλτρα κάθε ένα εκ των οποίων θα απορρίπτει ένα ποσοστό των ανεπιθύμητων στοιχείων που δεν έχουν απορριφθεί προηγουμένως. Αναλυτικότερα, ένα φίλτρο Bloom αποτελείται από:

1. Έναν πίνακα με n bits, αρχικοποιημένα στο 0.
2. Μία συλλογή από k συναρτήσεις κατακερματισμού $h_1, h_2, \dots, h_k$. Κάθε συνάρτηση αντιστοιχίζει τις τιμές του κλειδιού σε έναν από τους n κάδους, όπου κάθε κάδος αντιστοιχεί σε ένα bit του πίνακα.
3. Ένα σύνολο S που περιέχει m κλειδιά. Κατακερματίζουμε ένα ένα όλα τα κλειδιά του συνόλου S χρησιμοποιώντας τις k συναρτήσεις κατακερματισμού. Έστω K η τιμή του κλειδιού. Το bit που αντιστοιχεί στη θέση $h_i(K)$ γίνεται 1.

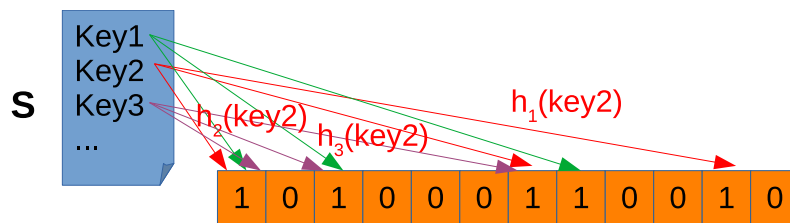
Σκοπός του φίλτρου Bloom είναι να επιτρέψει τη διέλευση σε όλα τα στοιχεία που είναι μέλη του S και να απορρίψει τα περισσότερα στοιχεία που δεν είναι μέλη του S . Για τον έλεγχο ενός νέου κλειδιού C , αρχικά, ελέγχουμε εάν όλες οι τιμές:

$$h_1(C), h_2(C), \dots, h_k(C)$$

αντιστοιχούν σε bits που είναι 1 στον πίνακα των bits. Εάν ισχύει αυτό, τότε επιτρέπεται η διέλευση του αντίστοιχου στοιχείου της ροής. Εάν μία από τις θέσεις αυτές περιέχει 0, τότε το κλειδί C δεν μπορεί να ανήκει στο σύνολο S , οπότε απορρίπτεται.

Στο Σχήμα 11.5 δίνεται ένα παράδειγμα με $k = 3$. Διαισθητικά, αντιλαμβάνεται κανείς εύκολα ότι η απόδοση του φίλτρου σχετίζεται άμεσα με το πλήθος των k των συναρτήσεων και το πόσα μηδενικά υπάρχουν.

Η ανάλυση της πιθανότητας ένα στοιχείο που δεν είναι μέλος του S να θεωρηθεί λανθασμένα ότι είναι μέλος έχει πολλές ομοιότητες με την ανάλυση του LSH. Έστω, αρχικά, ότι $k=1$. Η πιθανότητα ένα στοιχείο να είναι ψευδώς θετικό είναι ίση με το ποσοστό των 1 στο



Σχήμα 11.5: Παράδειγμα ενός Bloom φίλτρου με $k = 3$.

bitmap του Bloom φίλτρου. Έστω ότι έχουμε n θέσεις bit και το μέγεθος του S είναι m . Το κάθε μέλος του S έχει την ίδια πιθανότητα να κατακερματιστεί σε κάποιο από τα n bits.

- Η πιθανότητα ένα συγκεκριμένο μέλος του S να μην κατακερματιστεί σε ένα συγκεκριμένο bit είναι $(n - 1)/n$.
- Η πιθανότητα ότι σε ένα συγκεκριμένο bit δεν θα κατακερματιστεί κανένα μέλος του S από τα m συνολικά είναι $(\frac{n-1}{n})^m$. Η σχέση αυτή μπορεί να γραφεί και ως $(1 - \frac{1}{n})^{n(\frac{m}{n})}$.
- Χρησιμοποιώντας την προσέγγιση $(1 - \epsilon)^{1/\epsilon} = 1/e$ για μικρές τιμές του ϵ , καταλήγουμε στο ότι η πιθανότητα σε ένα συγκεκριμένο bit να μην κατακερματιστεί κανένα μέλος του S είναι $e^{-m/n}$.
- Συνεπώς, η πιθανότητα ένα bit να είναι 1 στο φίλτρο ισούται με $1 - e^{-m/n}$, όταν $k = 1$. Αυτή είναι και η πιθανότητα για ψευδώς θετικά.
- Στη γενική περίπτωση που $k \geq 1$, η πιθανότητα γίνεται $(1 - e^{-km/n})^k$.

Παράδειγμα 11.3. Έστω 1 σύνολο S με $m = 10^9$ στοιχεία και διαθέσιμος χώρος 1 GB για το Bloom φίλτρο. Άρα, $n = 8 \cdot 10^9$. Εάν κάθε στοιχείο κατακερματιστεί σε διαφορετική θέση, θα υπάρχουν 1 στο 12,5% των θέσεων bit χρησιμοποιώντας μία συνάρτηση κατακερματισμού. Το αναμενόμενο, όμως, είναι ότι τα bits με τιμή 1 να είναι το $1 - e^{-1/8} = 11,75\%$ του συνόλου, που όντως είναι πολύ κοντά στη χονδρική εκτίμηση του 12,5%. Εάν $k = 2$, η πιθανότητα για ψευδώς θετικά πέφτει στο $(1 - e^{-1/4})^2 = 4,93\%$. $\square$

Είναι σημαντικό να αναφέρουμε ότι η πιθανότητα για ψευδώς θετικά μειώνεται μέχρι κάποια (σχετικά μικρή) τιμή του k και από εκεί και πέρα αυξάνεται σημαντικά, καθώς με μεγάλο k όλο και λιγότερες θέσεις του πίνακα των bits είναι μηδενικές.

11.3.2 Μέτρηση διακριτών στοιχείων

Έστω ότι επιθυμούμε να γνωρίζουμε πόσα διαφορετικά στοιχεία έχουν εμφανισθεί στη ροή, μετρώντας από την αρχή της ροής ή από κάποια γνωστή χρονική στιγμή στο παρελθόν χρησιμοποιώντας πολύ λιγότερη μνήμη σε σχέση με το πλήθος των διαφορετικών στοιχείων της ροής. Μία λύση σε αυτό το πρόβλημα παρέχει ο αλγόριθμος Flajolet-Martin. Στον αλγόριθμο αυτόν, μπορούμε να εκτιμήσουμε τον αριθμό των μοναδικών στοιχείων κατακερματίζοντας τα στοιχεία σε έναν πίνακα από bits με αρκετά μεγάλο μέγεθος. Το μέγεθος του πίνακα πρέπει να είναι τέτοιο, ώστε οι τιμές που μπορεί να δώσει η συνάρτηση κατακερματισμού να

είναι περισσότερες από το πλήθος των στοιχείων του συνόλου. Για παράδειγμα, 64 bits είναι αρκετά για να κατακερματίσουμε URL τιμές.

Η βασική ιδέα πίσω από τον αλγόριθμο Flajolet-Martin είναι ότι όσο διαφορετικότερα είναι τα στοιχεία της ροής τόσο διαφορετικότερες θα είναι και οι τιμές της συνάρτησης κατακερματισμού. Όσο περισσότερες τιμές κατακερματισμού βλέπουμε τόσο πιθανότερο είναι ότι κάποια από αυτές τις τιμές θα είναι «ασυνήθιστη». Η συγκεκριμένη ασυνήθιστη ιδιότητα που θα εκμεταλλευτούμε είναι ότι η τιμή τελειώνει με πολλά μηδενικά, αν και υπάρχουν και διαφορετικές επιλογές. Όποτε εφαρμόζουμε μία συνάρτηση κατακερματισμού h σε ένα στοιχείο a της ροής, η τιμή $h(a)$ θα τελειώνει με κάποια μηδενικά, ίσως και κανένα. Ας ονομάσουμε το πλήθος των μηδενικών *μήκος ουράς* για το στοιχείο a και τη συνάρτηση h . Έστω R το μέγιστο μήκος ουράς για όλα τα στοιχεία που έχουμε δει. Τότε, το πλήθος των μοναδικών στοιχείων της ροής προσεγγίζεται με την ποσότητα 2^R .

Για να δώσουμε ένα αδρό σκεπτικό του αλγορίθμου, αρκεί να σκεφτούμε ότι περίπου τα μισά στοιχεία έχουν τιμή κατακερματισμού με μήκος ουράς 1, δηλαδή η τιμή κατακερματισμού είναι της μορφής $***0$. Αντίστοιχα, περίπου το 1/4 των στοιχείων έχουν τιμή κατακερματισμού της μορφής $**00$. Οπότε, εάν $R = 2$, δηλαδή το μέγιστο μήκος ουράς είναι 2, τότε έχουμε εντοπίσει προσεγγιστικά 4 διακριτά στοιχεία μέχρι τώρα.

Στον αλγόριθμο χρησιμοποιούμε πολλαπλές συναρτήσεις κατακερματισμού. Εάν υπολογίσουμε τον μέσο όρο των τιμών 2^R που παίρνουμε από κάθε συνάρτηση, τότε περιπτώσεις όπου θα υπάρχει υπερεκτίμηση του πλήθους των διακριτών στοιχείων θα αλλοιώσουν το τελικό αποτέλεσμα. Ένας άλλος τρόπος συνδυασμού εκτιμήσεων είναι να λάβουμε την ενδιάμεση τιμή (median) των εκτιμήσεων. Η ενδιάμεση τιμή δεν επηρεάζεται από πολύ μεγάλες τιμές ενός μεμονωμένου 2^R , όμως είναι πάντοτε δύναμη του δύο. Επομένως, όσες συναρτήσεις κατακερματισμού και αν χρησιμοποιήσουμε, εάν το πλήθος των διακριτών τιμών βρίσκεται ανάμεσα σε δύο δυνάμεις του 2 (π.χ. 400), τότε είναι αδύνατο να έχουμε μια καλή προσέγγιση. Ωστόσο, μπορούμε να συνδυάσουμε τις μεθόδους του μέσου όρου και της ενδιάμεσης τιμής. Πρώτα, ομαδοποιούμε τις συναρτήσεις κατακερματισμού σε μικρές ομάδες και υπολογίζουμε τον μέσο όρο για την κάθε ομάδα. Στη συνέχεια, υπολογίζουμε την ενδιάμεση τιμή των μέσων όρων.

Ολοκληρώνοντας την ενότητα αυτή, σημειώνουμε ότι, γενικότερα, υπάρχουν πολλοί εξειδικευμένοι αλγόριθμοι για ροές, όπως για τη μέτρηση ροπών (moments), όπου η ροπή k τάξης μίας ροής είναι το άθροισμα του αριθμού των εμφανίσεων κάθε στοιχείου υψωμένου στην k^1 , η εκτίμηση του πλήθους των μονάδων σε ένα παράθυρο, και για τη διατήρηση συχνότερων στοιχείων σε ένα εκθετικώς φθίνον παράθυρο.

11.4 Μηχανές αναζήτησης

Οι πρώτες μηχανές αναζήτησης αποτελούνταν από καταλόγους με κατηγορίες θεμάτων οι οποίοι επιβλέπονταν από ειδικούς, ή βασιζόνταν αποκλειστικά στο περιεχόμενο των ιστοσελίδων (βλ. Σχήμα 11.6). Η πρώτη προσέγγιση γρήγορα εγκαταλείφθηκε, καθώς δεν εξυπη-

¹Δηλαδή, η μηδενική ροπή είναι το πλήθος των διαφορετικών στοιχείων, ενώ η ροπή πρώτης τάξης είναι το μήκος της ροής.



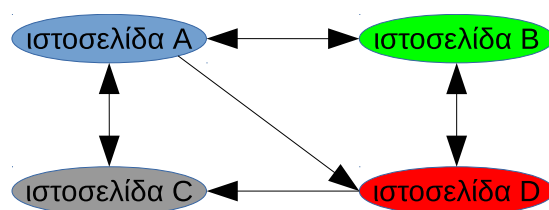
Σχήμα 11.6: Screenshot από μηχανή αναζήτησης πριν από την εισαγωγή των τεχνικών της Google.

ρετούσε ικανοποιητικά ερωτήματα αναζήτησης. Όμως, εξίσου προβληματική αποδείχθηκε και η δεύτερη προσέγγιση. Οι μηχανές αναζήτησης διάβαζαν όρους (λέξεις) μέσω της διαδικασίας crawling και ενημέρωναν κάποιον ανεστραμμένο κατάλογο. Αυτό οδηγούσε σε μια κατάσταση όπου εύκολα κάποιος ξεγελούσε τις μηχανές προσθέτοντας στη σελίδα πολλές φορές δημοφιλείς όρους, οι οποίοι δεν ήταν εμφανείς στον αναγνώστη της σελίδας, π.χ., χρησιμοποιώντας ίδιο χρώμα γραμματοσειράς με το χρώμα του φόντου.

Θα εξετάσουμε σε αυτήν την ενότητα τον αλγόριθμο PageRank, ο οποίος αναπτύχθηκε ώστε οι απαντήσεις στα ερωτήματα αναζήτησης, τα οποία μπορούν να θεωρηθούν ως λίστες με όρους, να μην εξαρτώνται αποκλειστικά από τα περιεχόμενα ανεστραμμένων καταλόγων, όπου η θέση και το πλήθος των εμφανίσεων των όρων στις ιστοσελίδες επηρεάζουν τη σχετικότητα των σελίδων για το συγκεκριμένο ερώτημα. Αντιθέτως, η τεχνική PageRank χρησιμοποιήθηκε για να προσομοιώσει την περιήγηση στον παγκόσμιο ιστό και να προσδιορίσει σε ποιες σελίδες οι περιηγητές έχουν την τάση να συγκεντρώνονται, ξεκινώντας από μία τυχαία ιστοσελίδα και ακολουθώντας τυχαία έναν σύνδεσμο από την τρέχουσα ιστοσελίδα. Οι ιστοσελίδες με πολλούς επισκέπτες χαρακτηρίζονται ως περισσότερο «σημαντικές» από ό,τι οι ιστοσελίδες με σπάνιες επισκέψεις. Επίσης, το περιεχόμενο μιας ιστοσελίδας δεν αξιολογείται με βάση την παρουσία ή όχι των όρων, αλλά και με βάση την εμφάνιση των όρων μέσα ή κοντά στους συνδέσμους που αναφέρονται στην ιστοσελίδα. Η προσέγγιση αυτή έχει αποδειχθεί, μετά από κάποιες επεκτάσεις, πολύ περισσότερο ανθεκτική σε κακόβουλους, οι οποίοι επιθυμούν να μπερδέψουν το σύστημα και να εμφανίζουν σε ερωτήματα αναζήτησης ιστοσελίδες που δεν είναι σχετικές στην πραγματικότητα.

11.4.1 Ο αλγόριθμος PageRank

Η μέθοδος PageRank ουσιαστικά υλοποιεί μία συνάρτηση που δίνει έναν πραγματικό αριθμό σε κάθε ιστοσελίδα του παγκόσμιου ιστού ή τουλάχιστον στις ιστοσελίδες που έχουν ανακαλυφθεί μέσω της διαδικασίας σάρωσης (crawling). Ο βασικός σκοπός είναι οι σημαντικότερες



Σχήμα 11.7: Υποθετική αναπαράσταση του Παγκόσμιου Ιστού με 4 μόνο ιστοσελίδες.

ιστοσελίδες να έχουν και υψηλότερο PageRank. Για τον υπολογισμό του PageRank δεν υπάρχει ένας μόνο αλγόριθμος, με αποτέλεσμα παραλλαγές της βασικής τεχνικής να μπορούν να αλλάξουν τη σχετική βαθμολογία δύο ιστοσελίδων.

Ας αναπαραστήσουμε τον παγκόσμιο ιστό ως γράφο όπου οι ιστοσελίδες είναι οι κόμβοι, ενώ υπάρχει ένα βέλος από τη σελίδα p_1 προς τη σελίδα p_2 , εάν υπάρχει τουλάχιστον ένας σύνδεσμος από την p_1 προς την p_2 . Στο Σχήμα 11.7 απεικονίζεται ένα παράδειγμα για ένα πολύ μικρό τμήμα του παγκόσμιου ιστού. Από τη σελίδα A υπάρχουν σύνδεσμοι προς όλες τις άλλες σελίδες, η σελίδα B έχει συνδέσμους προς τις σελίδες A και D , η σελίδα C έχει σύνδεσμο προς τη σελίδα A και, τέλος, η σελίδα D έχει συνδέσμους προς τις σελίδες B και C .

Το βασικό σκεπτικό της τεχνικής PageRank είναι να προσομοιώσει την περιήγηση στον παγκόσμιο ιστό όταν κάποιος χρήστης αρχίζει από μία τυχαία ιστοσελίδα και ακολουθεί τυχαία συνδέσμους αυτής της ιστοσελίδας προς άλλες σελίδες. Σε ένα τέτοιο σκεπτικό, οι χρήστες τείνουν να συγκεντρώνονται στις πιο σημαντικές σελίδες. Μία άλλη θεώρηση είναι οι σύνδεσμοι προς μία ιστοσελίδα να λειτουργούν έμμεσα ως ψήφοι σημαντικότητας αναφορικά με τη σελίδα στην οποία μεταβαίνουν. Προσοχή, όμως, στο γεγονός ότι η υπεραπλούστευση αυτής της θεώρησης, ώστε η σημαντικότητα μίας σελίδας να ποσοτικοποιείται βάσει του πλήθους των σελίδων οι οποίες έχουν σύνδεσμο προς αυτήν, είναι εύκολα χειραγωγίσιμη, αντίστοιχα με την προσέγγιση που λαμβάνει υπόψη μόνο τους όρους μέσα στο περιεχόμενο της σελίδας.

Η περιήγηση καθορίζεται από τον ίδιο τον γράφο που αναπαριστά τον παγκόσμιο ιστό. Αρχικά, χωρίς να θεωρούμε θέματα κλιμακωσιμότητας, αρκεί να ορίσουμε τον *πίνακα μετάβασης* για να περιγράψουμε και να προβλέψουμε τι συμβαίνει σε κάθε βήμα. Ο πίνακας αυτός, που τον συμβολίζουμε εδώ ως M , έχει n γραμμές και n στήλες που αναφέρονται στις n ιστοσελίδες. Υποθέτουμε ότι υπάρχει σύνδεσμος από την j -οστή προς την i -οστή σελίδα. Το στοιχείο m_{ij} , που βρίσκεται στην i -οστή γραμμή και την j -οστή στήλη, έχει τιμή $1/k$, όπου k είναι το πλήθος των σελίδων με εισερχόμενες συνδέσεις από την j -οστή σελίδα. Διαφορετικά, η τιμή του στοιχείου m_{ij} είναι μηδέν. Με αυτήν τη θεώρηση, κάθε στήλη του πίνακα είναι ένα διάνυσμα που δείχνει ποιες ιστοσελίδες μπορεί να επισκεφτεί ο περιηγητής εάν βρίσκεται στην ιστοσελίδα που αντιστοιχεί στη συγκεκριμένη στήλη.

Παράδειγμα 11.4. Έστω ότι ένας περιηγητής ξεκινά από τη σελίδα A του σχήματος. Υπάρχουν σύνδεσμοι προς τις σελίδες B , C και D , οπότε θα επισκεφθεί μία από αυτές τις σελίδες με πιθανότητα $1/3$, ενώ με μηδενική πιθανότητα θα παραμείνει στην A . Εάν ο περιηγητής βρεθεί στη σελίδα B , στο επόμενο βήμα θα βρεθεί στη σελίδα A με πιθανότητα $1/2$ και στη σελίδα D με πιθανότητα επίσης $1/2$, ενώ με μηδενική πιθανότητα θα βρεθεί στη σελίδα B ή

Γ. Ο πίνακας μεταβάσεων για το τμήμα του παγκόσμιου ιστού του Σχήματος 11.7 είναι:

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Στον πίνακα αυτόν, η σειρά των ιστοσελίδων είναι A, B, C και D . Επομένως, η πρώτη στήλη δείχνει ότι ένας περιηγητής που βρίσκεται στη σελίδα A στο επόμενο βήμα θα βρεθεί σε μία από τις σελίδες B, C και D με την ίδια πιθανότητα $1/3$. Η δεύτερη στήλη δείχνει ότι ένας περιηγητής που βρίσκεται στη σελίδα B θα βρεθεί στη σελίδα A με πιθανότητα $1/2$ ή στη σελίδα D με πιθανότητα $1/2$. Η τρίτη στήλη δείχνει ότι, εάν ο περιηγητής βρίσκεται στη σελίδα C , θα βρεθεί στη σελίδα A με πιθανότητα 1 . Τέλος, η τέταρτη στήλη αναφέρει ότι, εάν ο περιηγητής βρίσκεται στη σελίδα D , θα μεταβεί είτε στη σελίδα B είτε στη σελίδα C με πιθανότητα $1/2$. $\square$

Η (ιδανική) μέθοδος PageRank καλείται να υπολογίσει ένα διάνυσμα $\mathbf{v}$, όπου το στοιχείο στη θέση j είναι η πιθανότητα ο περιηγητής να βρεθεί στην j -οστή σελίδα. Η αρχική τιμή αυτού του διανύσματος, $\mathbf{v}_0$, έχει την τιμή $1/n$ σε όλες τις θέσεις, όπου n το πλήθος των σελίδων του παγκόσμιου ιστού.

Εάν M είναι ο πίνακας μεταβάσεων, τότε μετά από ένα βήμα η κατανομή της πιθανότητας γίνεται $M\mathbf{v}_0$, μετά από δύο βήματα η κατανομή θα είναι $M(M\mathbf{v}_0) = M^2\mathbf{v}_0$, κ.ο.κ. Γενικά, πολλαπλασιάζοντας το αρχικό διάνυσμα $\mathbf{v}_0$ με τον πίνακα M συνολικά i φορές, παίρνουμε την κατανομή πιθανότητας μετά από i βήματα. Καθώς το i αυξάνεται, βασιζόμενοι στη θεωρία των διεργασιών Markov, θεωρούμε ότι η τελική κατανομή πιθανότητας και το αποτέλεσμα της τεχνικής PageRank είναι το διάνυσμα $\mathbf{v}$ που ικανοποιεί τη συνθήκη $\mathbf{v} = M\mathbf{v}$, με την προϋπόθεση ότι ικανοποιούνται τα εξής:

1. ότι ο γράφος είναι ισχυρά συνεκτικός, που σημαίνει ότι για κάθε ζεύγος κόμβων u, v μπορούμε να μεταβούμε από τον u στον v ,
2. ότι δεν υπάρχουν κόμβοι που να μην έχουν εξερχόμενες ακμές, κάτι που δεν συμβαίνει στην πράξη όμως, οπότε χρειάζεται να αντιμετωπιστεί κατάλληλα.

Όταν ικανοποιούνται οι παραπάνω προϋποθέσεις, ο πίνακας M είναι στοχαστικός, δηλαδή το άθροισμα των τιμών της κάθε στήλης είναι 1 .

Σύμφωνα με τα παραπάνω, η τελική κατάσταση επιτυγχάνεται όταν επιπλέον πολλαπλασιασμός του διανύσματος $\mathbf{v}$ με τον πίνακα M δεν οδηγεί σε τροποποίηση του $\mathbf{v}$. Με άλλα λόγια, το διάνυσμα $\mathbf{v}$ που περιγράφει την κατανομή στην οριακή κατάσταση είναι ένα ιδιοδιάνυσμα του πίνακα M . Υπενθυμίζεται ότι ένα ιδιοδιάνυσμα $\mathbf{v}$ του M ικανοποιεί τη σχέση $\mathbf{v} = \lambda M\mathbf{v}$ για κάποια ιδιοτιμή λ . Επίσης, λόγω της στοχαστικότητας του M , γνωρίζουμε ότι η ιδιοτιμή που αντιστοιχεί στο κύριο ιδιοδιάνυσμα είναι 1 . Αυτό το κύριο ιδιοδιάνυσμα που προκύπτει από διαδοχικούς πολλαπλασιασμούς του αρχικού διανύσματος $\mathbf{v}_0$ με τον πίνακα M καταγράφει την πιθανότητα ένας τυχαίος περιηγητής να επισκεφτεί μία οποιαδήποτε ιστοσελίδα, και αυτή η πιθανότητα ποσοτικοποιεί σύμφωνα με την τεχνική PageRank τη σημαντικότητα της σελίδας. Το πλήθος των επαναλήψεων στην πράξη δεν ξεπερνά το 100 .

Παράδειγμα 11.5. Στο παράδειγμά μας, η ακολουθία διανυσμάτων που προκύπτει από διαδοχικούς πολλαπλασιασμούς έχει ως εξής:

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 9/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 15/48 \\ 11/48 \\ 11/48 \\ 11/48 \end{bmatrix}, \begin{bmatrix} 11/32 \\ 7/32 \\ 7/32 \\ 7/32 \end{bmatrix}, \dots, \begin{bmatrix} 3/9 \\ 2/9 \\ 2/9 \\ 2/9 \end{bmatrix}$$

□

Υπάρχουν δύο βασικά προβλήματα που πρέπει να αποφευχθούν για την εφαρμογή της παραπάνω μεθοδολογίας στην πράξη. Το πρώτο αφορά την ύπαρξη των αδιεξόδων, δηλαδή σελίδων που δεν έχουν εξερχόμενες συνδέσεις. Το δεύτερο πρόβλημα δημιουργείται από ομάδες σελίδων που, ενώ έχουν συνδέσμους προς άλλες σελίδες, οι σύνδεσμοι οδηγούν σε σελίδες μέσα στην ομάδα. Όταν συμβαίνουν τα παραπάνω, ειδικά στην πρώτη περίπτωση όπου ο πίνακας μετάβασης παύει να είναι στοχαστικός, τα διανύσματα που προκύπτουν δεν μπορούν να εξυπηρετήσουν τους σκοπούς μιας μηχανής αναζήτησης. Μπορεί να είναι είτε μηδενικά είτε να δείχνουν ότι μόνο συγκεκριμένες ιστοσελίδες έχουν πιθανότητα επίσκεψης.

Αυτά τα δύο προβλήματα επιλύονται με μια μέθοδο, σύμφωνα με την οποία ένας τυχαίος περιηγητής έχει πεπερασμένη πιθανότητα να εγκαταλείψει την περιήγηση σε οποιοδήποτε βήμα, ενώ νέοι περιηγητές ξεκινούν την περιήγησή τους. Αυτό είναι αντίστοιχο του να επιτρέπουμε σε έναν τυχαίο περιηγητή να μεταβεί σε μια τυχαία ιστοσελίδα με κάποια (μικρή) πιθανότητα, αντί να ακολουθήσει κάποιον εξερχόμενο σύνδεσμο. Πιο συγκεκριμένα, ο υπολογισμός του νέου διανύσματος $\mathbf{v}'$ με βάση το τρέχον διάνυσμα $\mathbf{v}$ και τον πίνακα μεταβάσεων γίνεται ως εξής:

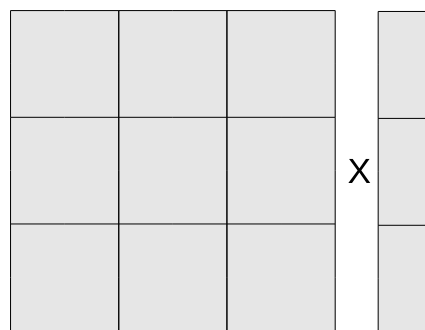
$$\mathbf{v}' = \beta M\mathbf{v} + (1-\beta)\mathbf{e}/n$$

όπου β είναι μια σταθερά, συνήθως μεταξύ των τιμών 0,8 και 0,9, ώστε $1 - \beta$ να είναι η πιθανότητα μετάβασης σε μια τυχαία σελίδα, $\mathbf{e}$ είναι ένα διάνυσμα με άσους, και n είναι το πλήθος των κόμβων στον γράφο του παγκόσμιου ιστού.

Διευκρινίζουμε ότι η βαθμολόγηση με τον παραπάνω τρόπο των ιστοσελίδων δεν σημαίνει ότι οι μηχανές αναζήτησης έπαψαν να χρησιμοποιούν ανεστραμμένο κατάλογο. Κάτι τέτοιο θα ήταν παράλογο, καθώς θα επέστρεφαν παρόμοια αποτελέσματα για κάθε ερώτημα. Οπότε, η βαθμολόγηση της σημαντικότητας λαμβάνεται υπόψη, αφού πρώτα έχουν εξαχθεί οι σελίδες που περιέχουν σχετικούς όρους. Επιπρόσθετα κριτήρια μπορεί να είναι το τμήμα της σελίδας που περιέχει ή από το οποίο απουσιάζει ένας όρος του ερωτήματος. Για την τελική διάταξη παίζει μεν σημαντικό ρόλο η τιμή PageRank, αλλά και πολλοί άλλοι παράγοντες, που μπορεί να φτάνουν σε πλήθος τις εκατοντάδες.

11.4.2 Επεκτάσεις και θέματα υλοποίησης

Ο υπολογισμός του κύριου ιδιοδιανύσματος μπορεί να γίνει με διαδοχικούς πολλαπλασιασμούς του διανύσματος $\mathbf{v}_0$ με τον πίνακα M για την ιδεατή περίπτωση, αλλά στη γενική περίπτωση η βασική πράξη είναι $\mathbf{v}_{new} = \beta M\mathbf{v}_{old}$.



Σχήμα 11.8: Διαχωρισμός πίνακα μετάβασης και διανύσματος για αποδοτικό υπολογισμό PageRank.

Αρχικά, θα πρέπει να σημειωθεί ότι ο πίνακας μεταβάσεων είναι αραιός, οπότε η αναπαράστασή του είναι ως λίστα γειτνίασης. Εάν θεωρήσουμε ότι κάθε στοιχείο του πίνακα και των διανυσμάτων αποθηκεύεται σε 4 bytes, τότε τα διανύσματα για 10^9 σελίδες απαιτούν 4 GBs το κάθε ένα, δηλαδή εύκολα αποθηκεύονται στην κύρια μνήμη. Όμως, το αντίστοιχο μέγεθος του πίνακα μετάβασης είναι απαγορευτικό, 4 exabytes. Αντιθέτως, όταν χρησιμοποιούνται λίστες γειτνίασης, το μέγεθος γίνεται ανάλογο των ακμών. Οπότε, εάν η μέση ιστοσελίδα έχει 20 εξερχόμενες ακμές, το μέγεθος αναμένεται να είναι περίπου $4 * (20 + 1) * 10^9$ bytes, δηλαδή λιγότερο από 100 GBs.

Επιπρόσθετα, η αποδοτική παράλληλη εκδοχή, λόγω των επαναλήψεων, δεν συνιστάται να ακολουθεί πιστά το υπολογιστικό πλαίσιο MapReduce. Γενικότερα, δεν απαιτείται εργασία reduce, ενώ κάθε εργασία map μπορεί να αναλαμβάνει την επεξεργασία ενός τμήματος του πίνακα M και μία λωρίδα από το διάνυσμα v . Επιπρόσθετα, το τμήμα του πίνακα μπορεί να μεταφερθεί στην κύρια μνήμη μόνο μία φορά στην αρχή. Πιο συγκεκριμένα, μπορεί κάποιος να ακολουθήσει μια προσέγγιση όπου ο πίνακας M (σε πραγματική μορφή λίστες γειτνίασης) χωρίζεται σε κάθετες λωρίδες και το διάνυσμα σε αντίστοιχες οριζόντιες.

Εάν αυτή η προσέγγιση οδηγεί στο φαινόμενο thrashing, λόγω π.χ. του αναγκαίου χώρου για αποθήκευση αποτελεσμάτων, μία πιο εξελιγμένη προσέγγιση έχει ως εξής. Έστω ότι έχουμε στη διάθεσή μας k^2 υπολογιστικούς κόμβους. Τότε, το διάνυσμα συνεχίζει να είναι διαχωρισμένο σε k οριζόντιες λωρίδες, αλλά ο πίνακας είναι διαχωρισμένος σε k^2 υποπίνακες. Το βασικό σκεπτικό αυτής της προσέγγισης απεικονίζεται στο Σχήμα 11.8 και είναι φανερό ότι υπονοεί ότι το διάνυσμα έχει βαθμό αντιγραφής k .

Πέρα από τα θέματα αποδοτικής υλοποίησης σε παράλληλο περιβάλλον, αξίζει να σημειωθεί ότι, όταν ο PageRank άρχισε να χρησιμοποιείται ως βασικό εργαλείο στις μηχανές αναζήτησης, οι κακόβουλοι χρήστες ανακάλυψαν τρόπους επηρεασμού της τιμής PageRank μιας ιστοσελίδας, το γνωστό φαινόμενο των *ανεπιθύμητων συνδέσμων* (link spam). Ως αντίδραση σε αυτό αναπτύχθηκε η τεχνική TrustRank, καθώς και άλλες τεχνικές εντοπισμού ανεπιθύμητων συνδέσμων, με στόχο να εμποδίσουν τους κακόβουλους χρήστες να αλλοιώσουν τις τιμές PageRank. Τέλος, υπάρχουν πολλές άλλες παραλλαγές του PageRank, όπως ο αλγόριθμος PageRank με βάση το θέμα (topic-sensitive PageRank) (που μπορεί να χρησιμοποιηθεί και για την καταπολέμηση ανεπιθύμητων συνδέσμων) και ο αλγόριθμος HITS που ανακαλύπτει «κέντρα» και «αυθεντίες» (“hubs” and “authorities”).

Ο αλγόριθμος HITS επεξεργάζεται και αυτός τις ακμές του γράφου του παγκόσμιου ιστού, όπως ο PageRank, αλλά λαμβάνει υπόψη και τις εισερχόμενες ακμές, ενώ βασίζεται σε δύο

Basket id	Items
1	A,B,C,D
2	B,C,D
3	A,D,E
4	B,C,D,F
5	D,F

Σχήμα 11.9: Παράδειγμα μοντέλου δεδομένων καλαθιού αγορών με 5 συναλλαγές.

τιμές, μία για την αξία της σελίδας ως κέντρου και μία ως αυθεντίας. Πολύ επιγραμματικά, ένα καλό κέντρο έχει εξερχόμενες ακμές προς καλές αυθεντίες, ενώ μια καλή αυθεντία έχει εισερχόμενες ακμές από καλά κέντρα.

11.5 Κανόνες συσχέτισης

Η εξαγωγή κανόνων συσχέτισης είναι μια βασική λειτουργία της εξόρυξης δεδομένων. Ένα βασικό της κομμάτι αποτελεί η εύρεση των συχνών στοιχειοσυνόλων. Οι κανόνες συσχέτισης εφαρμόζονται σε ένα μοντέλο δεδομένων που δεν είναι σχεσιακό και συχνά αναφέρεται ως μοντέλο δεδομένων καλαθιού αγορών (market-basket). Το μοντέλο αυτό χρησιμοποιείται για την περιγραφή της συσχέτισης δύο ειδών αντικειμένων. Από τη μία μεριά, έχουμε αντικείμενα και, από την άλλη, έχουμε καλάθια (baskets), που μερικές φορές καλούνται «συναλλαγές». Κάθε καλάθι αποτελείται από ένα σύνολο αντικειμένων που καλείται στοιχειοσύνολο (item-set). Αυτό το μοντέλο εύκολα προσομοιώνει αγορές σε σούπερ μάρκετ, αλλά έχει και πολλές άλλες χρήσεις. Το πλήθος των συναλλαγών είναι ενδεχομένως πολύ μεγάλο. Από την άλλη, το πλήθος των δυνατών αντικειμένων συχνά είναι πιο περιορισμένο. Π.χ., σε ένα σούπερ μάρκετ το πλήθος των διαφορετικών προϊόντων είναι της τάξης του $10^5 - 10^6$. Ακόμη μικρότερο είναι, συνήθως, το πλήθος των αντικειμένων σε μία συγκεκριμένη συναλλαγή. Π.χ., το μέσο καλάθι σε ένα σούπερ μάρκετ δεν ξεπερνά τις δεκάδες διαφορετικών προϊόντων.

Το Σχήμα 11.9 απεικονίζει ένα παράδειγμα του μοντέλου καλαθιού αγορών με 5 συναλλαγές. Είναι προφανές ότι το μοντέλο αυτό δεν είναι αποδοτικό να αποθηκεύεται απευθείας σε σχεσιακό σχήμα, καθώς έτσι θα χάνεται η πληροφορία για το ποια συγκεκριμένα αντικείμενα βρίσκονται «μέσα» σε κάθε καλάθι, λόγω των περιορισμών της 1ης κανονικής μορφής. Όμως, είναι εύκολο, με ένα group-by και απλή μετα-επεξεργασία αποτελεσμάτων, να παραχθεί η μορφή που απεικονίζεται στο σχήμα όταν τα αρχικά δεδομένα είναι αποθηκευμένα σε σχεσιακές βάσεις δεδομένων, όπως και συμβαίνει πιο συχνά στην πράξη.

Διαισθητικά, ένα σύνολο αντικειμένων που εμφανίζεται σε πολλά καλάθια λέγεται ότι είναι συχνό. Στο παράδειγμα του σχήματος, συχνό θα θεωρούνταν το αντικείμενο D. Πιο τυπικά, υποθέτουμε ότι υπάρχει ένας αριθμός s , που καλείται κατώφλι υποστήριξης (support threshold). Εάν I είναι το σύνολο των αντικειμένων ή αλλιώς στοιχειοσύνολο, η υποστήριξη (support) για το I είναι το πλήθος των καλαθιών, όπου το I είναι ένα υποσύνολο. Βολεύει συχνά η υποστήριξη και το κατώφλι της να εκφράζονται ως ποσοστό. Λέμε ότι το I είναι συχνό (frequent), εάν η υποστήριξή του είναι s ή μεγαλύτερη. Η υποστήριξη του I είναι, ουσιαστικά,

η πιθανότητα να βρεθεί το I σε μία τυχαία συναλλαγή.

Παράδειγμα 11.6. Έστω το παράδειγμα του Σχήματος 11.9 και ένα κατώφλι υποστήριξης $s = 0,6$. Τότε, το στοιχειοσύνολο $\{A, B\}$ έχει υποστήριξη 0,2 και δεν είναι συχνό, ενώ αντίθετα το στοιχειοσύνολο $\{B, C, D\}$ εμφανίζεται σε 3 από τις 5 συναλλαγές, άρα είναι συχνό. $\square$

Τα συχνά στοιχειοσύνολα και οι κανόνες συσχέτισης οι οποίοι βασίζονται σε τέτοια συχνά σύνολα έχουν ένα πλήθος εφαρμογών, από τοποθετήσεις προϊόντων μέχρι εύρεση λογοκλοπής και ανάπτυξη τεχνικών που βασίζονται σε επιλογή κατάλληλων βιοδεικτών. Π.χ., εάν τα καλάθια είναι προτάσεις και τα αντικείμενα κείμενα, τότε ζεύγη κειμένων που το ένα είναι (σχεδόν) αντιγραφή του άλλου θα έχουν υψηλό βαθμό υποστήριξης.

Τα συχνά στοιχειοσύνολα έχουν από μόνα τους αξία, αλλά είναι συνηθισμένο να χρησιμοποιούνται για την εξεύρεση κανόνων συσχέτισης. Έστω ένα στοιχειοσύνολο I που έχει υποστήριξη ίση με ή μεγαλύτερη από το κατώφλι και ένα αντικείμενο $j \in I$. Τότε, ένας κανόνας συσχέτισης είναι ο $I - \{j\} \rightarrow j$, ο οποίος παράγεται αν η εμπιστοσύνη του ξεπερνά ένα δεύτερο κατώφλι. Η εμπιστοσύνη είναι ο λόγος της υποστήριξης του I προς την υποστήριξη του $I - \{j\}$.

Παράδειγμα 11.7. Σε συνέχεια του παραπάνω παραδείγματος, έστω δύο κανόνες συσχέτισης που παράγονται από το συχνό στοιχειοσύνολο $\{B, C, D\}$: $B, C \rightarrow D$ και $D \rightarrow B$. Ο πρώτος κανόνας έχει εμπιστοσύνη 1, καθώς το $\{B, C, D\}$ εμφανίζεται και στις 3 συναλλαγές όπου εμφανίζεται το $\{B, C\}$. Αντιθέτως, στον δεύτερο κανόνα, η υποστήριξη είναι 0,6 καθώς το B εμφανίζεται στις 3 από τις 5 συναλλαγές όπου εμφανίζεται το D . $\square$

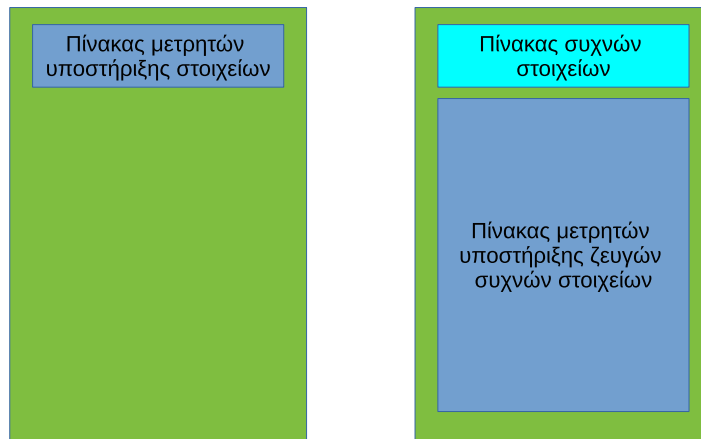
Στη συνέχεια της ενότητας αυτής, θα ασχοληθούμε αποκλειστικά με την αποδοτική εύρεση συχνών στοιχειοσυνόλων.

11.5.1 Αλγόριθμος A-Priori

Ένας από τους πιο γνωστούς αλγόριθμους εύρεσης συχνών στοιχειοσυνόλων είναι ο A-Priori. Ο αλγόριθμος αυτός τρέχει σε περάσματα. Στο πρώτο βρίσκει συχνά μεμονωμένα στοιχεία, στο δεύτερο συχνά ζεύγη, στο τρίτο συχνά στοιχειοσύνολα μεγέθους 3, κ.ο.κ. Η αποδοτικότητά του βασίζεται στην αρχή ότι, για να είναι συχνό ένα στοιχειοσύνολο μεγέθους k , πρέπει να είναι συχνά και τα k υποσύνολά του μεγέθους $k - 1$ το κάθε ένα.

Για αρχή, ας εστιάσουμε στην εύρεση συχνών ζευγών. Αν έχουμε αρκετή κύρια μνήμη για να μετρήσουμε όλα τα ζεύγη, τότε είναι απλή υπόθεση να διαβάσουμε σε ένα πέραςμα το αρχείο συναλλαγών με τα καλάθια. Για κάθε καλάθι χρησιμοποιούμε έναν διπλό βρόχο για να δημιουργήσουμε όλα τα ζεύγη. Κάθε φορά που δημιουργούμε ένα ζεύγος, προσθέτουμε 1 στον μετρητή του. Στο τέλος, εξετάζουμε όλα τα ζεύγη για να δούμε ποια ξεπερνούν (ή είναι ίσα με) το κατώφλι υποστήριξης s .

Όμως, αυτή η απλή προσέγγιση αποτυγχάνει εάν υπάρχουν πάρα πολλά ζεύγη αντικειμένων για να τα καταμετρήσουμε όλα στην κύρια μνήμη. Για να το αντιληφθούμε αυτό, θα πρέπει να καταλάβουμε ότι όσο μεγάλη και αν είναι η βάση δεδομένων συναλλαγών, αρκεί να έχουμε χώρο για να χωράει ακόμη και ένα μικρό πλήθος σελίδων της στην κύρια μνήμη, που



Σχήμα 11.10: Σχηματική αναπαράσταση της χρήσης της κύριας μνήμης κατά τη διάρκεια του πρώτου (αριστερά) και του δεύτερου (δεξιά) περάσματος του αλγορίθμου A-Priori.

θεωρείται ότι περιέχει μία ή περισσότερες πλήρεις συναλλαγές. Όμως, ο μετρητής της υποστήριξης για κάθε ζεύγος (συχνών) στοιχείων, για λόγους απόδοσης και αποφυγής thrashing, θα πρέπει να είναι εξ ολοκλήρου αποθηκευμένος στην κύρια μνήμη, καθώς οι εγγραφές του ενημερώνονται με αυθαίρετο τρόπο. Αυτός ο πίνακας των μετρητών μπορεί πολύ εύκολα να μεγαλώσει πολύ. Για παράδειγμα, αν έχουμε 10^6 συχνά στοιχεία, ο πίνακας των μετρητών απαιτεί χονδρικά $5 * 10^{11}$ θέσεις μνήμης.

Ο κλασικός αλγόριθμος A-Priori, που θα εξετάσουμε στη συνέχεια, είναι σχεδιασμένος ώστε να μειώνει τον αριθμό των ζευγών που πρέπει να καταμετρηθούν. Το κόστος για να επιτευχθεί αυτό είναι η εκτέλεση δύο περασμάτων στα δεδομένα αντί για ένα πέρασμα, θεωρώντας ότι αρκεί το γεγονός ότι στο πρώτο πέρασμα φιλτράρονται πολλά στοιχεία ως μη συχνά. Συνεπώς, ο αλγόριθμος αυτός δεν είναι πραγματικά ικανός να αποφύγει το πρόβλημα των πολλών υποψήφιων συχνών ζευγών όταν έχουμε όντως πάρα πολλά συχνά μεμονωμένα στοιχεία.

Το πρώτο πέρασμα του A-Priori. Στο πρώτο πέρασμα, οι απαιτήσεις στην κύρια μνήμη, ουσιαστικά, καθορίζονται από την ανάγκη αποθήκευσης ενός πίνακα μετρητών, όπου στην i -οστή θέση του πίνακα καταχωρείται η (τρέχουσα) υποστήριξη του i -οστού στοιχείου. Όταν ολοκληρωθεί η ανάγνωση όλων των εγγραφών συναλλαγών, είναι τετριμμένο να υπολογίζεται η υποστήριξη και να εξαχθούν τα συχνά στοιχεία. Προφανώς, αυτό μπορεί να γίνεται και κατά την ενημέρωση των μετρητών υποστήριξης.

Το δεύτερο πέρασμα του A-Priori. Για το δεύτερο πέρασμα του A-Priori, ασχολούμαστε μόνο με τα συχνά στοιχεία και πρακτικά βολεύει να ενημερώσουμε τα id τους σε μία εσωτερική δομή. Κατά τη διάρκεια του δεύτερου περάσματος, μετρούμε όλα τα ζεύγη που αποτελούνται από συχνά αντικείμενα, καθώς ένα ζεύγος δεν μπορεί να είναι συχνό, εκτός και εάν και τα δύο μέλη του είναι συχνά. Συνεπώς, δεν χάνουμε κάποιο συχνό ζεύγος. Ο χώρος που απαιτείται στο δεύτερο πέρασμα είναι $2m^2$ bytes, όπου m το πλήθος των συχνών στοιχείων.

Οι βασικές δομές δεδομένων που χρησιμοποιούνται στο πρώτο και το δεύτερο πέρασμα φαίνονται στο Σχήμα 11.10. Στο παράδειγμα παραπάνω, αναφέραμε τον τεράστιο όγκο μνήμης που απαιτείται, εάν έχουμε $m = 10^6$ συχνά στοιχεία. Εάν, όμως, τα συνολικά στοιχεία είναι 10^6 και μόνο το 10% από αυτά είναι συχνά, τότε το πρώτο πέρασμα καταφέρνει αποτελε-

σματικά να μειώσει τον χώρο που απαιτείται για την καταμέτρηση της υποστήριξης ζευγών. Γενικά, το όφελος είναι ανάλογο με τον βαθμό εξάλειψης των μη συχνών αντικειμένων. Εάν μόνο τα μισά αντικείμενα είναι συχνά, τότε χρειαζόμαστε το ένα τέταρτο του χώρου για την καταμέτρηση ζευγών.

Η ίδια τεχνική που χρησιμοποιείται για την εύρεση συχνών ζευγών χωρίς να καταμετρούμε όλα τα ζεύγη μας επιτρέπει να βρίσκουμε μεγαλύτερα συχνά στοιχειosύνολα χωρίς εξαντλητική καταμέτρηση όλων των συνόλων. Στον A-Priori κάνουμε ένα πέρασμα για κάθε σύνολο μεγέθους k βασιζόμενοι στα αποτελέσματα για μεγέθη $k - 1$. Εάν δεν βρεθεί κάποιο συχνό στοιχειosύνολο ενός συγκεκριμένου μεγέθους, μπορούμε να σταματήσουμε. Το βασικό σκεπτικό να πηγαίνουμε από ένα μέγεθος k στο επόμενο μέγεθος $k+1$ μπορεί να συνοψισθεί ως εξής. Για κάθε μέγεθος k , υπάρχουν δύο σύνολα στοιχειosυνόλων:

1. Το C_k είναι το σύνολο των υποψήφιων στοιχειosυνόλων μεγέθους k και περιλαμβάνει τα στοιχειosύνολα που πρέπει να μετρήσουμε για να καθορίσουμε το εάν είναι όντως συχνά.
2. Το L_k είναι το σύνολο των πραγματικά συχνών στοιχειosυνόλων μεγέθους k .

Αρχίζουμε με το C_1 , που περιέχει όλα τα στοιχειosύνολα ενός στοιχείου, δηλαδή όλα τα μεμονωμένα στοιχεία, καθώς δεν μπορούμε εκ των προτέρων να απορρίψουμε κάποιο στοιχείο ως μη συχνό. Το πρώτο βήμα φιλτραρίσματος είναι να καταμετρήσουμε όλα τα αντικείμενα. Όσα αντικείμενα έχουν μετρητή τουλάχιστον ίσο με το κατώφλι υποστήριξης s αποτελούν το σύνολο L_1 των συχνών αντικειμένων. Αυτό είναι το πρώτο πέρασμα που περιγράψαμε προηγουμένως.

Το σύνολο C_2 των υποψήφιων ζευγών είναι τα σύνολο των ζευγών που τα αντικείμενά τους είναι στο L_1 , δηλαδή τα συχνά αντικείμενα. Το δεύτερο πέρασμα του A-Priori μετρά όλα τα υποψήφια ζεύγη και καθορίζει ποια εμφανίζονται τουλάχιστον s φορές. Αυτά τα ζεύγη αποτελούν το L_2 , τα συχνά ζεύγη.

Ακολουθούμε αυτό το μοτίβο όσο θέλουμε. Το σύνολο C_3 των υποψήφιων τριπλετών δημιουργείται (έμμεσα) ως το σύνολο τριών στοιχείων, όπου το κάθε ζεύγος είναι στο L_2 . Για να είναι ένα στοιχειosύνολο μέλος του C_3 θα πρέπει όλα τα υποσύνολά του να είναι συχνά, αλλά αρκεί να ελέγξουμε εάν όλα τα ζεύγη που περιέχει είναι συχνά, μια και τα συχνά ζεύγη έχουν προέλθει αποκλειστικά από συχνά στοιχεία. Στην πράξη, ενώ αναμένουμε πολλά συχνά στοιχεία, και, συνεπώς, έναν ιδιαίτερα αυξημένο αριθμό υποψήφιων συχνών ζευγών, τα πραγματικά συχνά ζεύγη είναι περιορισμένα σε πλήθος, οπότε η καταμέτρηση της υποστήριξης των υποψήφιων συχνών τριπλετών δεν έχει ιδιαίτερα αυξημένες ανάγκες μνήμης για τον αντίστοιχο πίνακα μετρητών. Για την εύρεση του L_3 κάνουμε ένα τρίτο πέρασμα στο αρχείο των καλαθιών. Για κάθε καλάθι χρειάζεται να εξετάσουμε μόνο τα αντικείμενα που είναι στο L_1 και δημιουργούμε από αυτά τριπλέτες ελέγχοντας το εάν ανήκουν στο C_3 . Για κάθε τέτοια τριπλέτα που βρίσκεται αυξάνουμε κατά 1 τον μετρητή της.

Παρομοίως συνεχίζουμε για C_4, L_4, C_5, L_5 κ.ο.κ, μέχρις ότου δεν παράγεται σύνολο υποψήφιων συχνών στοιχειosυνόλων. Η παραγωγή των συνόλων C_k μπορεί να γίνει συνδυάζοντας 2 στοιχειosύνολα που ανήκουν στο L_{k-1} και έχουν τα πρώτα $k - 2$ στοιχεία συχνά. Όμως, μετά, πρέπει να ελεγχθεί εάν και τα υπόλοιπα $k - 2$ στοιχειosύνολα μεγέθους $k - 1$, που

περιέχονται στο παραγόμενο στοιχειοσύνολο, είναι όντως συχνά.

11.5.2 Αντιμετωπίζοντας το πρόβλημα της καταμέτρησης ζευγών και η επίλυσή του

Έχει γίνει φανερό από τη μέχρι τώρα συζήτηση ότι ο αλγόριθμος A-Priori είναι καλός όσο το βήμα με τις μεγαλύτερες απαιτήσεις κύριας μνήμης, που όπως επισημάνθηκε, αναμένεται να είναι η καταμέτρηση των υποψήφιων ζευγών, C_2 , έχει αρκετή μνήμη ώστε να μπορεί να επιτελεσθεί χωρίς συνεχείς προσπελάσεις στη δευτερεύουσα μνήμη (thrashing). Στη συνέχεια, θα δούμε μία από τις λύσεις που έχουν προταθεί για το συγκεκριμένο πρόβλημα.

Ο αλγόριθμος PCY (από τους συγγραφείς του Park, Chen και Yu) εκμεταλλεύεται την παρατήρηση ότι ο μη χρησιμοποιούμενος χώρος στην κύρια μνήμη κατά το πρώτο πέρασμα του A-Priori είναι μεγάλος, όπως φαίνεται και στο Σχήμα 11.10. Έστω μία διαθέσιμη μνήμη για μετρητές της τάξης των 4 GB και 10^6 στοιχεία. Τότε, ο πίνακας μετρητών μεμονωμένων στοιχείων αναμένεται να καταλαμβάνει περίπου το 1 χιλιοστό της διαθέσιμης μνήμης. Ο αλγόριθμος PCY χρησιμοποιεί αυτόν τον χώρο με βάση ένα σκεπτικό που έχει ομοιότητες με τα φίλτρα Bloom.

Πιο συγκεκριμένα, ο αλγόριθμος PCY δημιουργεί έναν πίνακα σαν έναν πίνακα κατακερματισμού που καταλαμβάνει όλον τον διαθέσιμο χώρο των μετρητών. Χρησιμοποιώντας μία συνάρτηση κατακερματισμού, κάθε ζεύγος στοιχείων κατακερματίζεται σε μία θέση (κάδο) του πίνακα. Το περιεχόμενο αυτού του πίνακα είναι ακέραιοι που δηλώνουν την υποστήριξη όλων των ζευγών που κατακερματίζονται στον συγκεκριμένο κάδο.

Ο πίνακας αυτός ενημερώνεται κατά το πρώτο πέρασμα του A-Priori. Ταυτόχρονα, δηλαδή, με την ενημέρωση του πίνακα μετρητών μεμονωμένων στοιχείων για κάθε ζεύγος στοιχείων που περιέχεται στη συναλλαγή, ο αντίστοιχος κάδος αυξάνεται κατά 1, ανεξάρτητα από το εάν τα στοιχεία του ζεύγους είναι συχνά ή όχι, κάτι που είναι άγνωστο ούτως ή άλλως κατά το πέρασμα. Με τον τρόπο αυτόν, στο τέλος του πρώτου περάσματος, πέρα από την πληροφορία των συχνών στοιχείων έχουμε και έναν πίνακα με κάδους, τους οποίους μπορούμε να τους κατατάξουμε σε συχνούς ή μη συχνούς. Το σκεπτικό είναι ότι, εάν ένα ζεύγος κατακερματίζεται σε μη συχνό κάδο, σίγουρα δεν είναι συχνό. Εάν, όμως, κατακερματίζεται σε συχνό κάδο, τότε ενδεχομένως είναι συχνό. Αυτή η παρατήρηση μας επιτρέπει να φιλτράρουμε σε μεγάλο βαθμό τα ζεύγη συχνών στοιχείων για τα οποία καταμετρούμε την υποστήριξη. Πλέον, αντί να μετράται η υποστήριξη ζευγών που αποτελούνται από 2 συχνά στοιχεία, μετράται η υποστήριξη ζευγών που τόσο αποτελούνται από 2 συχνά στοιχεία όσο και κατακερματίζονται σε συχνό κάδο.

Παράδειγμα 11.8. Έστω ένα σενάριο με 10^9 εγγραφές, κάθε μία εκ των οποίων έχει 20 στοιχεία. Έστω, επίσης, ότι το κατώφλι υποστήριξης ορίζεται στο 0,00001, δηλαδή στις 10^4 εγγραφές και ότι έχουμε κατά το πρώτο πέρασμα του A-Priori επιπλέον διαθέσιμη μνήμη 2 GBs. Τα οφέλη της χρήσης του PCY μπορούν να εκτιμηθούν ως εξής. Σε 2 GBs υπάρχει διαθέσιμος χώρος για $5 * 10^8$ κάδους ακεραίων, εάν θεωρήσουμε ότι ο κάθε ακέραιος θέλει 4 bytes. Το σύνολο των ζευγών που περιέχονται σε κάθε εγγραφή είναι 190. Άρα, συνολικά, παράγονται από τις εγγραφές $1,9 * 10^{11}$ ζεύγη. Αυτό ακριβώς το μέγεθος θα πρέπει να παραχθεί εάν αθροίσουμε τους μετρητές σε κάθε κάδο. Εάν θεωρήσουμε μία ακραία περίπτωση, όπου αυτό το πλήθος μετρητών προέρχεται μόνο από συχνούς κάδους οι οποίοι έχουν ακριβώς

την υποστήριξη που ζητάμε, τότε μπορούμε να βρούμε το μέγιστο πλήθος κάδων που είναι συχνόι: $\frac{1,9 \cdot 10^{11}}{10^4} = 1,9 \cdot 10^7$. Το $1,9 \cdot 10^7$ είναι το 3,8% του $5 \cdot 10^8$. Οπότε, είναι λογικό να αναμένουμε, θεωρώντας τυχαία τοποθέτηση στους κάδους, ότι για ένα μόνο μικρό ποσοστό των υποψήφιων συχνών ζευγών του κλασικού A-Priori, μικρότερο του 3,8%, τελικά, θα χρειαστεί να καταμετρηθεί η υποστήριξη. $\square$

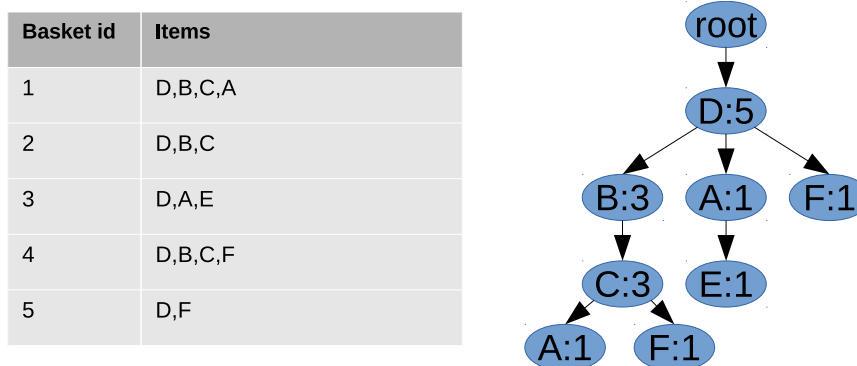
Το παραπάνω παράδειγμα δείχνει την αποτελεσματικότητα του PCY αλγορίθμου. Όμως, θεωρήσαμε ότι χρησιμοποιούμε σχεδόν όλη τη διαθέσιμη μνήμη για τον πίνακα κατακερματισμού, που σημαίνει ότι, εάν τον κρατήσουμε ως έχει στην κύρια μνήμη, δεν θα έχουμε πάλι αρκετή διαθέσιμη μνήμη για τη μέτρηση της υποστήριξης ζευγών. Αυτό αντιμετωπίζεται εύκολα, καθώς το μόνο που μας ενδιαφέρει κατά το δεύτερο πέρασμα είναι το εάν ένας κάδος είναι συχνός ή όχι, κάτι που μπορεί να συνοψιστεί σε ένα *bitmap*, με ένα bit για κάθε κάδο. Το bit είναι 1 εάν ο κάδος είναι συχνός, και 0 εάν δεν είναι. Έτσι, οι ακέραιοι των 32 bits αντικαθίστανται με μονά bits, ενώ το bitmap καταλαμβάνει μόνο το 1/32 του χώρου που διαφορετικά θα ήταν διαθέσιμος για την αποθήκευση μετρητών.

Σημειώνουμε, τέλος, ότι αυτό το σκεπτικό μπορεί να γενικευτεί με τη χρήση επιπλέον ενδιάμεσων περασμάτων ανάμεσα στην καταμέτρηση των συχνών στοιχείων και συχνών ζευγών ή, ακόμη καλύτερα, με τη χρήση περισσότερων από 1 πινάκων κατακερματισμού.

11.5.3 Περιορισμός περασμάτων

Ο αλγόριθμος για συχνά στοιχειοσύνολα που συζητήσαμε μέχρι τώρα, είτε στην κλασική του εκδοχή είτε με την ενσωμάτωση του αλγόριθμου PCY, χρησιμοποιεί ένα πέρασμα για κάθε μέγεθος στοιχειοσυνόλου που εξετάζεται. Η εύρεση όλων ή των περισσότερων συχνών στοιχειοσυνόλων χρησιμοποιώντας το πολύ δύο περάσματα της βάσης δεδομένων συναλλαγών από τη δευτερεύουσα μνήμη μπορεί να γίνει εύκολα.

Αρχικά, μπορούμε να εφαρμόσουμε δειγματοληψία και, αντί να χρησιμοποιήσουμε όλο το αρχείο με τα καλάθια, θα μπορούσαμε να πάρουμε ένα τυχαίο υποσύνολο των καλαθιών που μπορούμε να το επεξεργαστούμε εξ ολοκλήρου στην κύρια μνήμη. Σε μια τέτοια περίπτωση, πρέπει να διασφαλίσουμε ότι το δείγμα καλύπτει όλη τη βάση και δεν προέρχεται μόνο από ένα τμήμα της, π.χ., τις πλέον πρόσφατες εγγραφές. Το μέγεθος του δείγματος μπορεί να οριστεί ώστε να χωράει εξ ολοκλήρου στην κύρια μνήμη, αφήνοντας και όσον χώρο απαιτεί ο A-Priori για όλα του τα περάσματα. Ένα στοιχειοσύνολο που είναι συχνό στο σύνολο, αλλά όχι στο δείγμα, είναι *ψευδώς αρνητικό*, ενώ ένα στοιχειοσύνολο που είναι συχνό στο δείγμα, αλλά όχι στο σύνολο, είναι *ψευδώς θετικό*. Η προσέγγιση της δειγματοληψίας υποφέρει τόσο από ψευδώς θετικά όσο και από ψευδώς αρνητικά. Τα ψευδώς θετικά μπορούμε να τα απαλείψουμε κάνοντας ένα τελικό πέρασμα στα πλήρη δεδομένα και καταμετρώντας την πραγματική υποστήριξη των στοιχειοσυνόλων που βγήκαν ως συχνά στο δείγμα. Όμως, τα ψευδώς αρνητικά παραμένουν. Παρ' όλα αυτά, μπορούμε να μειώσουμε την εμφάνισή τους, χαμηλώνοντας τεχνητά το κατώφλι υποστήριξης s στο δείγμα, π.χ., εφαρμόζοντας τον αλγόριθμο με κατώφλι 0, 9s. Τότε, αναμένουμε πολύ λιγότερα ψευδώς αρνητικά, αλλά αυξάνονται τα ψευδώς θετικά. Η αύξηση των ψευδών θετικών είναι κάτι που πάλι πρέπει να μας απασχολήσει, παρόλο που με ένα πέρασμα μπορούμε να τα εντοπίσουμε και να τα διαγράψουμε, καθώς αυξάνουν τις απαιτήσεις στη μνήμη.



Σχήμα 11.11: Παράδειγμα FP-tree στα δεδομένα του Σχήματος 11.9.

Ένας άλλος τρόπος που είναι εγγενώς παραλληλοποιήσιμος και δίνει ακριβή αποτελέσματα, αλλά απαιτεί δύο πλήρη περάσματα, είναι να διαιρέσουμε το αρχείο εισόδου σε κομμάτια, κάθε ένα από τα οποία υφίσταται επεξεργασία εξ ολοκλήρου στην κύρια μνήμη, όπως και το δείγμα παραπάνω. Παίρνοντας την ένωση από όλα τα συχνά στοιχειοσύνολα από όλα τα κομμάτια δεδομένων, δεν υπάρχουν πλέον καθόλου ψευδώς αρνητικά, αλλά μόνο ψευδώς θετικά. Αυτό συμβαίνει επειδή δεν μπορεί ένα στοιχειοσύνολο να είναι συχνό σε όλη τη βάση συναλλαγών και να μην είναι συχνό σε τουλάχιστον ένα τμήμα της. Τα ψευδώς θετικά απαλείφονται με ένα πλήρες πέραςμα, όπως και στην περίπτωση της δειγματοληψίας. Αυτό το πέραςμα θα είναι το δεύτερο συνολικά. Πέρα από τον μειωμένο αριθμό περασμάτων, το πλεονέκτημα αυτής της τεχνικής είναι η εύκολη παραλληλοποίηση, καθώς, εάν έχουμε στη διάθεσή μας πολλαπλούς υπολογιστικούς κόμβους, κάθε τέτοιος κόμβος μπορεί να αναλάβει ένα ξεχωριστό τμήμα, ώστε να μπορεί να το επεξεργαστεί εξ ολοκλήρου στην κύρια μνήμη.

Ένας τελευταίος αλγόριθμος που θα αναφέρουμε βασίζεται σε προθεματικά επισημειωμένα (annotated) δένδρα, που ονομάζονται FP-trees (FP: frequent pattern). Στην αρχή του αλγόριθμου γίνεται ένα πέραςμα στη βάση δεδομένων των συναλλαγών και χτίζεται το δένδρο, που έχει όλη την πληροφορία του πίνακα. Το σημαντικό πλεονέκτημα αυτής της προσέγγισης είναι ότι αναμένουμε το δένδρο να είναι πολύ μικρότερο σε μέγεθος από τον αρχικό πίνακα εγγραφών και, συνεπώς, αυξάνεται πολύ η πιθανότητα να χωράει στην κύρια μνήμη. Για τη μεγαλύτερη μείωση του μεγέθους, συμφέρει τα στοιχεία σε κάθε εγγραφή να ταξινομηθούν βάσει της υποστήριξής τους. Ένα παράδειγμα φαίνεται στο Σχήμα 11.11.

Ο πλήρης αλγόριθμος που χρησιμοποιεί μια τέτοια δομή για την εύρεση συχνών στοιχειοσυνόλων ονομάζεται FP-Growth και ανήκει στους αλγορίθμους «διαίρει-και-βασίλευε», καθώς βρίσκει ξεχωριστά συχνά (ταξινομημένα) στοιχειοσύνολα που λήγουν σε κάθε διαφορετικό στοιχείο. Αυτό επιτρέπει και την εύκολη παραλληλοποίησή του.

11.6 Ομαδοποίηση

Ομαδοποίηση ή αλλιώς συσταδοποίηση είναι η κατάταξη ή ανάθεση των σημείων μιας συλλογής δεδομένων σε «ομάδες» σύμφωνα με ένα μέτρο απόστασης. Στην ενότητα αυτή, θα θεωρούμε ότι τα σημεία αναπαρίστανται από αριθμητικά διανύσματα με πραγματικούς αριθμούς, δηλαδή είναι πολυδιάστατα, αλλά υπάρχουν τεχνικές ομαδοποίησης για όλους τους

τύπους δεδομένων. Ο σκοπός είναι τα σημεία στην ίδια ομάδα να έχουν μικρή απόσταση μεταξύ τους, ενώ τα σημεία σε διαφορετικές ομάδες να είναι σε μεγάλη απόσταση το ένα από το άλλο. Υπάρχουν πολλοί γνωστοί αλγόριθμοι που κλιμακώνονται σε μεγάλα δεδομένα, όπως ο πολύ γνωστός k -means (ή, ισοδύναμα, k -μέσοι), ο οποίος μάλιστα παραλληλίζεται εύκολα.

Πριν πούμε δύο λόγια για τον αλγόριθμο των k -μέσων, είναι χρήσιμο να σχολιάσουμε τη λεγόμενη «κατάρτα της διαστασιμότητας». Αυτή η κατάρτα δεν πρέπει να μεταφράζεται ότι δεν υπάρχουν συστάδες σε χώρο πολλών διαστάσεων, αλλά υποδηλώνει το γεγονός ότι, όσο αυξάνονται οι διαστάσεις, ζεύγη τυχαίων σημείων τείνουν να ισαπέχουν μεταξύ τους και τυχαία διανύσματα είναι κάθετα μεταξύ τους, κάτι που δυσκολεύει την αποτελεσματική χρήση των μετρικών απόστασης. Αυτό συμβαίνει επειδή η αποτελεσματική χρήση των μετρικών απόστασης βασίζεται πολλές φορές στην εύρεση της μικρότερης απόστασης σε ένα σύνολο σημείων σε ένα περιβάλλον όπου οι αποστάσεις των ζευγών διαφοροποιούνται σημαντικά μεταξύ τους.

Ο αλγόριθμος των k -μέσων είναι ένας αλγόριθμος που, ενώ προτάθηκε εδώ και πολλές δεκαετίες, όχι μόνο παραμένει μία από τις βασικές επιλογές, αλλά είναι και κατάλληλος για χειρισμό μεγάλων δεδομένων. Συνοπτικά, ο αλγόριθμος αρχικοποιεί k κέντρα σε έναν ευκλείδειο χώρο με τρόπο ώστε κάθε κέντρο να βρίσκεται σε διαφορετική συστάδα. Αυτό δεν μπορεί να είναι εγγυημένο, αλλά έχουν προταθεί τεχνικές που πραγματοποιούν αρκούντως καλή αρχικοποίηση. Μετά, κάθε σημείο ανατίθεται στη συστάδα που το τρέχον κέντρο της είναι το κοντινότερο. Όταν έχουν ανατεθεί όλα τα σημεία, τα κέντρα επανα-υπολογίζονται, κάτι που μπορεί να οδηγήσει σε αλλαγή της ανάθεσης κάποιων σημείων. Τα βήματα της ανάθεσης και επανα-υπολογισμού των κέντρων επαναλαμβάνονται μέχρι να συγκλίνει ο αλγόριθμος ή να εκτελεστεί ένας προκαθορισμένος αριθμός επαναλήψεων. Ο αλγόριθμος είναι μη ντετερμινιστικός όταν, όπως και γίνεται συνήθως, η αρχικοποίηση των κέντρων χαρακτηρίζεται από κάποια τυχειότητα. Λόγω του μη ντετερμινισμού, ο αλγόριθμος μπορεί να επαναληφθεί πολλές φορές και, τελικά, να επιλεγεί το καλύτερο αποτέλεσμα βάσει μετρικών, όπως το άθροισμα των τετραγώνων των αποστάσεων από τα κέντρα των συστάδων όπου ανήκουν.

Δύο είναι τα χαρακτηριστικά του αλγορίθμου που τον κάνουν κατάλληλο για εφαρμογή σε μεγάλα δεδομένα. Πρώτον, η πολυπλοκότητα κάθε επανάληψης είναι $O(nkd)$, δηλαδή γραμμική, τόσο όσον αφορά το πλήθος των σημείων n όσο και το πλήθος των διαστάσεων d και των κέντρων k , μια και κάθε σημείο σε κάθε επανάληψη πρέπει να εντοπίσει το κοντινότερο κέντρο χρησιμοποιώντας, π.χ., ευκλείδεια απόσταση. Δεύτερον, το πλήθος των συγκρίσεων μπορεί να γίνει εύκολα με παράλληλο τρόπο. Ένα πολύ μεγάλο σύνολο δεδομένων μπορεί να διαμεριστεί σε πολλαπλούς υπολογιστικούς κόμβους μία φορά στην αρχή, χωρίς να χρειάζεται στις επόμενες επαναλήψεις να γίνει επαναδιαμερισμός. Κάθε υπολογιστικός κόμβος πρέπει να γνωρίζει όλα τα τρέχοντα κέντρα, αλλά αυτή η πληροφορία που αντιγράφεται σε όλους τους κόμβους είναι μικρή, της τάξης του $O(kd)$. Επίσης, κάθε κόμβος σε κάθε επανάληψη μπορεί να υπολογίσει απλά στατιστικά ανά τρέχον κέντρο, που τα συλλέγει ένας κόμβος, ώστε να κάνει τον επανα-υπολογισμό και να ενημερώσει όλους τους κόμβους για τα νέα κέντρα. Όμως, αυτή η πληροφορία, όπως είπαμε παραπάνω, είναι πολύ μικρή σε μέγεθος και το τμήμα του μη παραλληλοποιήσιμου υπολογισμού περιορισμένο.

Μία άλλη χρήσιμη οικογένεια τεχνικών είναι η ιεραρχική συσταδοποίηση. Η πιο συνηθισμένη της μορφή είναι η συσσωρευτική, που το βασικό της σκεπτικό έχει ως εξής. Αρχικά,

κάθε σημείο αποτελεί ξεχωριστή συστάδα και, μετά, οι δύο κοντινότερες συστάδες συγχωνεύονται μέχρις ότου ικανοποιηθούν κάποια κριτήρια. Υπάρχει ένα μεγάλο πλήθος επιλογών για τον ορισμό της κοντινότερης απόστασης μεταξύ συστάδων και κριτηρίων τερματισμού, καλύπτοντας πολλές διαφορετικές περιπτώσεις πραγματικού κόσμου. Όμως, η πολυπλοκότητα αυτών των τεχνικών, ακόμη και με χρήση ουρών προτεραιότητας, είναι $(n^2 \log n)$ και, άρα, δεν μπορούν να εφαρμοστούν απευθείας σε μεγάλα δεδομένα, παρά μόνο σε δείγματά τους.

Σε αυτήν την ενότητα θα εξετάσουμε άλλους τρεις αλγορίθμους που ενδείκνυνται όταν τα δεδομένα είναι πολλά. Οι δύο πρώτοι επεκτείνουν το σκεπτικό του k -means και των ιεραρχικών αλγορίθμων.

11.6.1 Ο αλγόριθμος των Bradley, Fayyad και Reina

Ο αλγόριθμος των k -μέσων έχει πολλά προτερήματα, αλλά η απόδοσή του είναι μικρή όταν τα δεδομένα δεν χωράνε στην κύρια μνήμη, παρόλη τη γραμμική του πολυπλοκότητα. Το πρόβλημα αυτό επιλύει ένας αλγόριθμος που θα ονομάζουμε *BFR* (από τους συγγραφείς του Bradley, Fayyad και Reina). Ο BFR μπορεί να θεωρηθεί ως μια παραλλαγή του k -means. Είναι και αυτή σχεδιασμένη για την ομαδοποίηση δεδομένων σε έναν πολυδιάστατο ευκλείδειο χώρο, αλλά μπορεί να εφαρμόζεται αποδοτικά σε πολύ μεγάλα δεδομένα χωρίς την ανάγκη παραλληλισμού, κάτι που δεν ισχύει για τον k -means, ο οποίος, όταν δεν επαρκεί η κύρια μνήμη, αναγκάζεται να κάνει πολλαπλά περάσματα στα δεδομένα από τη δευτερεύουσα μνήμη.

Ο BFR έχει και αυτός τους περιορισμούς του, αναφορικά με το ποιοτικό του αποτέλεσμα της συσταδοποίησης. Πιο συγκεκριμένα, κάνει την υπόθεση πως τα δεδομένα πρέπει να ακολουθούν την κανονική κατανομή γύρω από το κέντρο, αλλά η μέση τιμή και η τυπική απόκλιση για μία ομάδα μπορεί να είναι διαφορετικές σε κάθε διάσταση. Επιπλέον, οι διαστάσεις πρέπει να είναι ανεξάρτητες. Οπότε, στη γενική περίπτωση, σε χώρο δύο διαστάσεων οι συστάδες έχουν ελλειψοειδές σχήμα με άξονες παράλληλους με τους άξονες του χώρου.

Ο BFR ξεκινά επιλέγοντας k σημεία, χρησιμοποιώντας μία από τις μεθόδους που χρησιμοποιεί και ο k -means. Έπειτα, τα σημεία των δεδομένων διαβάζονται σε κομμάτια (chunks). Κάθε κομμάτι πρέπει να αποτελείται από ένα πλήθος σημείων που μπορούμε να επεξεργαστούμε στην κύρια μνήμη. Τα δεδομένα στην κύρια μνήμη, εκτός από τα κομμάτια της εισόδου, αποτελούνται από τρεις τύπους αντικειμένων:

1. *Απορριφθέν σύνολο* (discard set). Αντιστοιχεί σε πυκνές συστάδες για τις οποίες δεν είναι ανάγκη να κρατάμε τα πλήρη δεδομένα των σημείων που ανήκουν σε αυτές τις συστάδες, αλλά μόνο απλά στατιστικά των ομάδων. Η απόρριψη αναφέρεται ακριβώς στο γεγονός ότι τα αρχικά σημεία αυτού του συνόλου, που αναμένουμε να αποτελούν τη συντριπτική πλειονότητα όλων των εγγραφών μας, δεν παραμένουν στην κύρια μνήμη, καθώς τα στατιστικά που κρατάμε μπορούν να δώσουν μια καλή περιγραφή της συστάδας.
2. *Συμπιεσμένο σύνολο* (compressed set). Αυτές είναι περιλήψεις όμοιες με τις περιλήψεις των ομάδων στο απορριφθέν σύνολο, αλλά για μικρά υπο-σύνολα δεδομένων που έχουν

βρεθεί κοντά το ένα στο άλλο, αλλά όχι κοντά σε κάποια από τις ομάδες. Αποτελούν στην ουσία μικροσυστάδες. Τα σημεία που αντιπροσωπεύονται από συμπιεσμένο σύνολο έχουν επίσης απορριφθεί, με την έννοια ότι δεν εμφανίζονται ευθέως στην κύρια μνήμη.

3. *Διατηρημένο σύνολο (retained set)*. Κάποια σημεία δεν μπορούν ούτε να ανατεθούν σε μια ομάδα ούτε είναι επαρκώς κοντά σε άλλα σημεία, ώστε να αναπαρασταθούν με ένα συμπιεσμένο σύνολο. Αυτά τα σημεία κρατούνται στην κύρια μνήμη ακριβώς όπως εμφανίζονται στο αρχείο εισόδου και περιλαμβάνουν τις εξωκείμενες τιμές (outliers), που έχουν από μόνες τους μεγάλη σημασία σε πολλές εφαρμογές.

Τα στατιστικά που κρατάμε για τα δύο πρώτα σύνολα πρέπει να μπορούν να περιγράψουν επαρκώς τις συστάδες. Λόγω της υπόθεσης της ανεξάρτητης κανονικής κατανομής ανά διάσταση, τα απορριφθέντα και συμπιεσμένα σύνολα μπορούν να αναπαρίστανται με $2d + 1$ τιμές, όπου d το πλήθος διαστάσεων. Αυτές οι τιμές είναι α) το πλήθος των σημείων που αναπαρίστανται, β) το άθροισμα των συντεταγμένων όλων των στοιχείων σε κάθε διάσταση, και γ) το άθροισμα των τετραγώνων των συντεταγμένων όλων των στοιχείων σε κάθε διάσταση. Αυτές οι $2d + 1$ τιμές μπορούν να μας δώσουν τα κέντρα και τις τυπικές αποκλίσεις των συστάδων που περιγράφουν ανά διάσταση, που είναι τα μόνα στατιστικά για την περιγραφή των τιμών των κανονικών κατανομών.

Παράδειγμα 11.9. Έστω ότι εφαρμόζουμε τον αλγόριθμο των Bradley, Fayyad και Reina σε 1 τρισεκατομμύριο δεδομένα 3 διαστάσεων, που ανήκουν σε 8 πολύ πυκνές συστάδες κατά 99,9%, ενώ το 0,1% των δεδομένων δεν ανήκουν σε καμία συστάδα ή μικροσυστάδα. Θέλουμε να υπολογίσουμε πόση μνήμη σε GBs πρέπει να έχει ένα μηχάνημα, ώστε να τρέξει καλά ο αλγόριθμος. Εάν υποθέσουμε ότι η τιμή σε κάθε διάσταση χρειάζεται 4 bytes, τότε τα δεδομένα μας είναι 12 TBs. Για τα στατιστικά των 8 συστάδων χρειαζόμαστε αμελητέο όγκο μνήμης, ίσο με $8 * 4 * (2 * 3 + 1) = 224$ bytes. Όμως, το 0,1% των δεδομένων που είναι στο retained set απαιτεί χώρο ίσο με 12 GBs. □

Το παραπάνω παράδειγμα δείχνει ότι ο αλγόριθμος μειώνει πολύ τις απαιτήσεις στη μνήμη, αλλά μπορεί κάποιος να σκεφτεί ότι, πάλι, οι απαιτήσεις στη μνήμη μπορεί να είναι μεγαλύτερες από τη διαθεσιμότητα. Αυτό στην πραγματικότητα δεν ισχύει και θα το εξηγήσουμε, αφού περιγράψουμε συνοπτικά τι συμβαίνει όταν επεξεργαζόμαστε ένα κομμάτι από δεδομένα σημείων.

1. Όλα τα σημεία που είναι επαρκώς κοντά στο κεντροειδές μίας ομάδας προστίθενται στην ομάδα. Μετά, μπορούμε να διαγράψουμε το σημείο. Προσθέτουμε ένα σημείο p σε μία ομάδα εάν όχι μόνο το κεντροειδές της είναι το πλησιέστερο στο p , αλλά είναι και πολύ απίθανο να βρεθεί κάποιο κεντροειδές ομάδας που να είναι το πλησιέστερο στο p . Μπορούμε να χρησιμοποιήσουμε την απόσταση Mahalanobis με κανονικοποιημένες αποστάσεις και να μη δεχόμαστε να ανήκουν σε μία συστάδα σημεία που απέχουν περισσότερο από ένα πλήθος m τυπικών αποκλίσεων από το κέντρο της συστάδας. Παρόμοια ή και πιο εξελιγμένα σκεπτικά επίσης μπορούν να εφαρμοστούν.
2. Τα σημεία που δεν είναι επαρκώς κοντά σε κάποιο κεντροειδές τα ομαδοποιούμε μαζί με τα σημεία στο διατηρημένο σύνολο. Μπορούμε να χρησιμοποιήσουμε κάθε αλγό-

ριθμο κύριας μνήμης. Πάλι, επιλέγουμε κάποιο κριτήριο για το εάν πρέπει να συγχωνεύσουμε σημεία σε μία ομάδα. Επίσης, με αντίστοιχα κριτήρια βλέπουμε εάν κάποιες από τις μικροσυστάδες μπορούν να συγχωνευτούν είτε μεταξύ τους είτε με κάποια από τις κύριες συστάδες στο απορριφθέν σύνολο.

3. Τα σημεία που ανατίθενται σε μία ομάδα ή σε μία μικροσυστάδα, δηλαδή αυτά που δεν είναι στο διατηρημένο σύνολο, εγγράφονται στον δίσκο μαζί με την ανάθεσή τους.

Στο τέλος, προαιρετικά, μπορούμε να μετα-επεξεργαστούμε τα συμπιεσμένα και τα διατηρημένα σύνολα και να τα αναθέσουμε σε μία συστάδα, αλλά είναι επίσης χρήσιμο πολλές φορές να τα κρατήσουμε ως έχουν, υποδηλώνοντας ακραίες/ανώμαλες τιμές. Το πιο σημαντικό είναι ότι, εάν σε κάποια φάση του αλγορίθμου παρουσιαστεί πρόβλημα έλλειψης μνήμης, χαλαρώνοντας δυναμικά τα κριτήρια ανάθεσης ενός σημείου σε μία συστάδα και συγχώνευσης μικροσυστάδων, μπορεί να μειωθεί το μέγεθος της απαιτούμενης μνήμης ώστε να διασφαλιστεί πως ο αλγόριθμος μπορεί να εκτελεστεί χωρίς πρόβλημα απόδοσης.

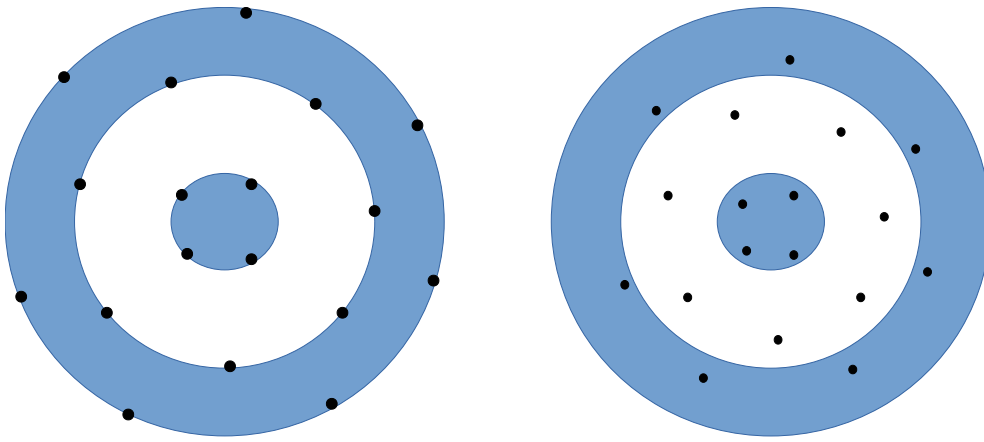
Σημειώνεται ότι το σκεπτικό της χρήσης των μικροσυστάδων συνδυαζόμενο με περιορισμένα περάσματα στα δεδομένα και αποθήκευσης στην κύρια μνήμη στατιστικών μπορεί να εκδηλωθεί με διαφορετικούς τρόπους, όπως στον αλγόριθμο BIRCH, ο οποίος μπορεί να θεωρηθεί περισσότερο επέκταση των ιεραρχικών αλγορίθμων.

11.6.2 Ο αλγόριθμος CURE

Ένας άλλος αλγόριθμος ομαδοποίησης, που μπορεί να χειριστεί μεγάλα δεδομένα και έχει συμπληρωματικά χαρακτηριστικά με τον BFR, είναι ο CURE (Clustering Using REpresentatives – ομαδοποίηση χρησιμοποιώντας αντιπροσώπους), που επίσης υποθέτει ευκλείδειο χώρο. Ο συγκεκριμένος αλγόριθμος στοχεύει στο να μη θέτει περιορισμούς αναφορικά με το σχήμα των ομάδων και την κατανομή των αποστάσεων των σημείων από τα κεντροειδή των συστάδων τους. Για να επιτευχθεί αυτό, έχει ληφθεί μια σειρά από αποφάσεις σχεδίασης, όπως η συστάδα να αναπαρίσταται από ένα σύνολο από *αντιπροσωπευτικά* σημεία, να χρησιμοποιείται δειγματοληψία και να εφαρμόζεται στο δείγμα κάποιος ιεραρχικός αλγόριθμος κατάλληλος για συσταδοποίηση με ομάδες αυθαίρετου σχήματος.

Ο αλγόριθμος CURE έχει πέντε φάσεις:

1. Λαμβάνεται ένα δείγμα αρκετά μικρό, με σκοπό όχι μόνο να χωράει στην κύρια μνήμη, αλλά και να μπορεί να υποστεί επεξεργασία αποδοτικά από έναν αλγόριθμο που μπορεί να χειριστεί συστάδες μη σφαιρικού ή ελλειψοειδούς σχήματος. Η βασική επιλογή είναι κάποια έκφανση ιεραρχικής συσταδοποίησης που επιλέγει συγχώνευση συστάδων, όταν έχουν ένα κοντινό ζεύγος σημείων.
2. Από τη συσταδοποίηση που προκύπτει επιλέγεται ένα υποσύνολο σημείων σε κάθε συστάδα, ώστε αυτά τα σημεία να απέχουν όσο γίνεται περισσότερο το ένα από το άλλο.
3. Τα αντιπροσωπευτικά σημεία μετακινούνται προς την κατεύθυνση του κεντροειδούς της συστάδας κατά ένα προκαθορισμένο ποσοστό, π.χ. 10% ή 20%. Αυτό το βήμα δεν θα μπορούσε να επιτευχθεί εάν ο χώρος δεν ήταν ευκλείδειος. Δεν υπάρχει θεωρητική



Σχήμα 11.12: Παράδειγμα επιλογής αντιπροσωπευτικών σημείων από κάθε μία από τις δύο συστάδες, που είναι όσο γίνεται μακρύτερα μεταξύ τους (αριστερά) και μετακίνησή τους προς το κεντροειδές (δεξιά).

απόδειξη ότι η μετακίνηση ωφελεί τη συσταδοποίηση, αλλά εμπειρικά φαίνεται να είναι ένα χρήσιμο βήμα (βλ. Σχήμα 11.12).

4. Εάν δύο συστάδες έχουν ζεύγη αντιπροσωπευτικών σημείων κοντά, τότε συγχωνεύονται.
5. Πραγματοποιείται η ανάθεση του συνόλου σημείων με βασικό κριτήριο την απόσταση από τα αντιπροσωπευτικά σημεία: κάθε σημείο ανατίθεται στη συστάδα όπου ανήκει το πλησιέστερο προς αυτό αντιπροσωπευτικό σημείο.

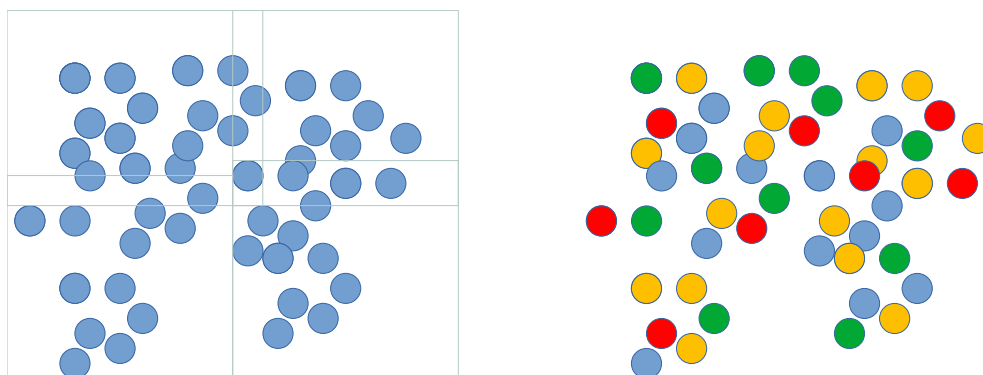
11.6.3 Ο αλγόριθμος DBSCAN

Κλείνουμε την ενότητα των αλγορίθμων ομαδοποίησης με τη συζήτηση του DBSCAN, που είναι μία από τις πλέον βασικές επιλογές αναφορικά με την ποιοτική συσταδοποίηση όταν τα δεδομένα δεν είναι πολύ μεγάλα. Το βασικό σκεπτικό του DBSCAN είναι να βασίζεται στον υπολογισμό της πυκνότητας των σημείων και να διαχωρίζει συστάδες όταν ανάμεσά τους υπάρχει περιοχή με αραιά δεδομένα. Ο αλγόριθμος, όμως, έχει πολυπλοκότητα $O(n^2)$, εκτός και αν τα δεδομένα είναι δισδιάστατα, οπότε και μόνο τότε η πολυπλοκότητα μπορεί να πέσει στο $O(n \log n)$.

Στην κλασική του εκδοχή, ο αλγόριθμος θεωρεί για ένα σημείο πόσα άλλα σημεία έχουν ευκλείδεια απόσταση από το συγκεκριμένο ίση το πολύ με ένα προκαθορισμένο όριο. Εάν αυτό το πλήθος ξεπερνά ένα δεύτερο όριο, τότε το σημείο που εξετάζεται μαζί με όλα τα γειτονικά του ανατίθενται στην ίδια ομάδα. Η ομάδα συνεχίζει να αναπτύσσεται με το ίδιο σκεπτικό μέχρις ότου δεν μπορούν να προστεθούν νέα σημεία.

Ο αλγόριθμος DBSCAN, τα τελευταία χρόνια, έχει επεκταθεί προς τρεις συμπληρωματικές κατευθύνσεις, ώστε να καταστεί εφαρμόσιμος σε μεγάλα δεδομένα, που είναι να:

1. τρέχει σε γραμμικό χρόνο προσεγγιστικά αυξάνοντας το όριο της γειτονιάς κατά μία μικρή σταθερά, σύμφωνα με την εργασία των Gan και Tao με τίτλο «DBSCAN Revisited: Mis-Claim, Un-Fixability, and Approximation».



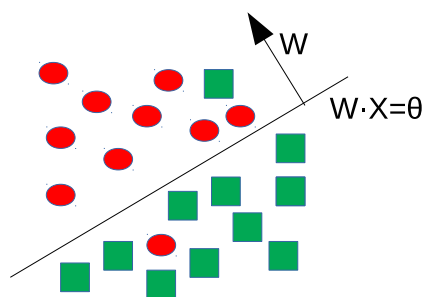
Σχήμα 11.13: Παράδειγμα διαμερισμού μερικώς επικαλυπτόμενων συνεχών περιοχών (αριστερά) και τυχαίου διαμερισμού (δεξιά).

2. μπορεί να εκτελείται (προσεγγιστικά) με έναν anytime τρόπο, ώστε να μπορεί να γίνει κατάλληλος για εφαρμογές πραγματικού χρόνου, όπως εξηγείται στην εργασία των Mai, Assent και Stogaard με τίτλο «AnyDBC: An Efficient Anytime Density-based Clustering Algorithm for Very Large Complex Datasets». Η ίδια εργασία είναι και ένα παράδειγμα πρότασης όπου δεν είναι αναγκαστική η χρήση ευκλείδειας απόστασης, αλλά κάθε μετρική μπορεί να χρησιμοποιηθεί ώστε τα αντικείμενα που συσταδοποιούνται να καλύπτουν εικόνες, γραφήματα, τροχιές, κ.λ.π.
3. εφαρμόζεται παραλληλισμός, με δεκάδες προτάσεις να έχουν εμφανιστεί στη βιβλιογραφία προς αυτήν την κατεύθυνση. Στις περισσότερες από αυτές, ο ευκλείδειος χώρος διαμοιράζεται σε πολλαπλούς υπολογιστικούς κόμβους με μερική επικάλυψη, δίνοντας έμφαση στη χωρική συνέχεια. Όμως, έχουν εμφανιστεί στη βιβλιογραφία και προτάσεις που είναι ιδιαίτερα αποδοτικές βασιζόμενες σε άλλους τρόπους διαμερισμού, όπως είναι ο τυχαίος διαμερισμός (π.χ., στην εργασία των Song και Lee με τίτλο «RP-DBSCAN: A Superfast Parallel DBSCAN Algorithm Based on Random Partitioning»). Παραδείγματα των δύο προσεγγίσεων φαίνονται στο Σχήμα 11.13.

11.7 Μηχανική μάθηση για μεγάλα δεδομένα

Στην προηγούμενη ενότητα εξετάστηκαν τρόποι κλιμάκωσης αλγορίθμων συσταδοποίησης, που πολλές φορές πλέον αναφέρονται ως τεχνικές μη-επιβλεπόμενης μάθησης, λόγω του ότι δεν υπάρχει ένα σύνολο δεδομένων εκπαίδευσης που υπαγορεύει το ποιες είναι οι ορθές συστάδες. Συμπληρωματικά, σε αυτήν την ενότητα θα εξετάσουμε ενδεικτικές τεχνικές επιβλεπόμενης μάθησης που στοχεύουν στη διαχείριση μεγάλων δεδομένων, ιδίως μέσω παραλληλισμού.

Σε όλες τις ενδεικτικές τεχνικές θεωρούμε ότι τηρούνται κάποιες βασικές αρχές σχεδιασμού, όπως ότι τα δεδομένα εκπαίδευσης αποτελούν ένα υποσύνολο των συνολικών διαθέσιμων δεδομένων για τα οποία είναι γνωστή η κλάση. Επίσης, σε μεγάλα δεδομένα μπορεί ο τρόπος μάθησης να είναι άμεσος, όπου το σύνολο εκπαίδευσης υφίσταται επεξεργασία ως ροή, δηλαδή τα αντίστοιχα δεδομένα προσπελάζονται μία φορά μόνο.



Σχήμα 11.14: Ένα αντίληπτρο διαιρεί τον χώρο με ένα υπερεπίπεδο σε δύο ημι-χώρους, χωρίς αναγκαστικά η ακρίβεια να (μπορεί να) είναι 100%.

11.7.1 Αντίληπτρα (perceptrons)

Το αντίληπτρο (perceptron) είναι ένας γραμμικός δυαδικός κατηγοριοποιητής. Η είσοδός του είναι ένα διάνυσμα $\mathbf{x} = [x_1, x_2, \dots, x_d]$ με στοιχεία d πραγματικούς αριθμούς. Το μοντέλο που δημιουργείται είναι ένα διάνυσμα από βάρη $\mathbf{w} = [w_1, w_2, \dots, w_d]$, τα οποία επίσης είναι πραγματικοί αριθμοί. Κάθε αντίληπτρο έχει ένα κατώφλι θ . Η έξοδος του αντίληπτρου είναι +1, εάν το εσωτερικό γινόμενο των $\mathbf{x}$ και $\mathbf{w}$ ξεπερνάει το κατώφλι, διαφορετικά είναι -1. Τα αντίληπτρα λειτουργούν όταν υπάρχει ένα υπερεπίπεδο που διαχωρίζει τα διανύσματα χαρακτηριστικών των θετικών παραδειγμάτων από αυτά των αρνητικών παραδειγμάτων (Σχήμα 11.14). Αυτός ο διαχωρισμός δεν είναι πάντοτε ούτε εφικτό να είναι απόλυτος, όπως ακριβώς φαίνεται στο σχήμα, ούτε μοναδικός.

Η συγκεκριμένη μέθοδος μηχανικής μάθησης, έτσι όπως την παρουσιάζουμε εδώ, προϋποθέτει ότι το σύνολο εκπαίδευσης έχει μόνο δύο ετικέτες κλάσης, θετικές και αρνητικές, αλλά εύκολα επεκτείνεται σε περισσότερες κλάσεις. Ο βασικός στόχος είναι η εύρεση του κατάλληλου υπερεπιπέδου, που ορίζεται από τη συνάρτηση $\mathbf{x} \cdot \mathbf{w} = \theta$. Συγκλίνουμε προς αυτό το υπερεπίπεδο προσαρμόζοντας τις εκτιμήσεις για το υπερεπίπεδο με βάση ένα κλάσμα, τον ρυθμό εκμάθησης, της κατεύθυνσης, που είναι ο μέσος όρος των τρεχόντων σημείων που έχουν κατηγοριοποιηθεί εσφαλμένα. Τα αντίληπτρα μπορούν να χειριστούν εκατομμύρια χαρακτηριστικά, αλλά έχουν νόημα μόνο εάν τα χαρακτηριστικά είναι αριθμητικά. Είναι αποτελεσματικά μόνο εάν υπάρχει ένας γραμμικός διαχωριστής ή τουλάχιστον ένα υπερεπίπεδο που να διαχωρίζει τις κλάσεις σε ικανοποιητικό βαθμό. Ωστόσο, μπορούμε να διαχωρίσουμε σημεία με ένα μη γραμμικό όριο, εάν μετατρέψουμε πρώτα τα σημεία. Το μοντέλο εκφράζεται, όπως ήδη αναφέρθηκε, από ένα διάνυσμα το οποίο έχει συχνά πολύ υψηλή διαστασιμότητα, κάτι που δυσχεραίνει την ερμηνεία του μοντέλου. Επίσης, συνδέοντας πολλά αντίληπτρα και σε διαφορετικά επίπεδα, παράγονται τα νευρωνικά δίκτυα που είναι πολύ αποτελεσματικά στην πράξη σε πολλές περιπτώσεις.

Η εφαρμογή των αντιλήπτρων σε μεγάλα δεδομένα είναι εφικτή κατά δύο έννοιες. Αρχικά, τα αντίληπτρα μπορούν επίσης να χρησιμοποιηθούν σε περιπτώσεις ροών, όπου τα δεδομένα εκπαίδευσης καταφθάνουν σε μορφή ακολουθίας και μπορούν να χρησιμοποιηθούν μία και μοναδική φορά. Σε αυτήν την περίπτωση, η κάθε τέτοια εγγραφή τροποποιεί το τρέχον διάνυσμα βαρών, συνήθως, σε πολύ μικρό βαθμό, ενώ ο διαχωρισμός είναι προσεγγιστικός χωρίς να είναι δυνατή η πραγματική σύγκλιση.

Επιπρόσθετα, η διαδικασία της εκπαίδευσης ενός αντιλήπτρου μπορεί να παραλληλο-

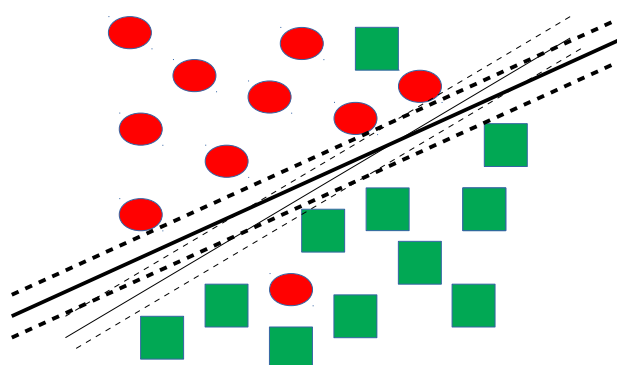
ποιηθεί, παρόλο που είναι εγγενώς ακολουθιακή, καθώς κάθε λανθασμένη κατηγοριοποίηση οδηγεί σε προσαρμογή του διανύσματος βαρών. Για τον σκοπό αυτόν, μεγάλα τμήματα των δεδομένων εκπαίδευσης ελέγχονται με το ίδιο (προσωρινό) διάνυσμα βαρών. Όταν μια εγγραφή, ένα διάνυσμα δηλαδή χαρακτηριστικών $\mathbf{x}$, κατηγοριοποιείται λανθασμένα, τότε, θεωρώντας το υπολογιστικό πλαίσιο MapReduce, παράγεται ένα ζεύγος κλειδιού-τιμής για κάθε μη μηδενικό στοιχείο x_i του $\mathbf{x}$. Το κλειδί είναι το i , $1 \leq i \leq d$ και η τιμή είναι η μεταβολή της αντίστοιχης θέσης στο διάνυσμα βαρών. Αντί, όμως, αυτή η μεταβολή να ενεργοποιείται μετά από κάθε λανθασμένη κατηγοριοποίηση, υπολογίζεται περιοδικά, αφού έχουν υποστεί επεξεργασία όλες οι εγγραφές του τμήματος των δεδομένων με το ίδιο προσωρινό διάνυσμα βαρών. Αυτός ο υπολογισμός εύκολα υλοποιείται με μία συνάρτηση Reduce. Το γενικότερο σκεπτικό είναι, αντί το $\mathbf{w}$ να προσαρμόζεται μετά τον έλεγχο κάθε εγγραφής, δεδομένου ότι οι αλλαγές που υπολογίζονται σε κάθε βήμα είναι συνήθως μικρές, η προσαρμογή του $\mathbf{w}$ να υπολογίζεται και να εφαρμόζεται περιοδικά, συναθροίζοντας τις υπολογισμένες αλλαγές σε κάθε βήμα.

Τέλος, όταν η διαστασιμότητα των δεδομένων είναι πολύ υψηλή, ο παραλληλισμός του υπολογισμού των εσωτερικών γινομένων έχει νόημα, ειδικά εάν τα διανύσματα δεν είναι αραιά.

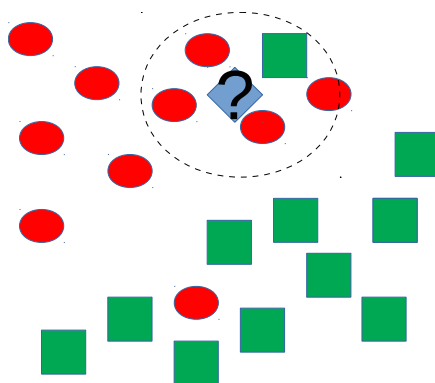
11.7.2 Μηχανές διανυσμάτων υποστήριξης

Μία *Μηχανή Διανυσμάτων Υποστήριξης* (support-vector machine ή SVM) αποτελεί μια βελτίωση του αντίληπτρου. Πιο συγκεκριμένα, ένα SVM στοχεύει στον υπολογισμό του διαχωριστικού υπερεπιπέδου, το οποίο μεγιστοποιεί το λεγόμενο *περιθώριο*, δηλαδή την απόσταση μεταξύ του υπερεπιπέδου και των πλησιέστερων σημείων του συνόλου εκπαίδευσης. Με αυτόν τον τρόπο, καλύπτεται μία βασική αδυναμία του αντίληπτρου, που δεν βελτιστοποιεί την επιλογή του υπερεπιπέδου με κριτήρια πέραν της τρέχουσας ακρίβειας κατηγοριοποίησης στα δεδομένα εκπαίδευσης. Ένα παράδειγμα φαίνεται στο Σχήμα 11.15, όπου το υπερεπίπεδο με πιο έντονο χρώμα έχει μεγαλύτερο περιθώριο από το αρχικό του Σχήματος 11.14.

Δεν είναι στους σκοπούς της ενότητας να δώσουμε λεπτομέρειες υλοποίησης, αλλά σε μεγάλα δεδομένα αρκεί να αναφέρουμε ότι, αντί η επίλυση των εξισώσεων εσωτερικά στο SVM να γίνεται με τη χρήση τετραγωνικού προγραμματισμού, συνιστάται η *επικλινής κάθοδος* (gradient descent). Επίσης, ο υπολογισμός ενός SVM μοντέλου μπορεί να παραλληλοποιηθεί



Σχήμα 11.15: Επιλογή υπερεπιπέδου μεγιστοποιώντας το περιθώριο.



Σχήμα 11.16: Παράδειγμα εκμάθησης από πλησιέστερους γείτονες.

με τρόπο ανάλογο με τα αντίληπτρα. Στο πλαίσιο MapReduce, κάθε εργασία Map αναλαμβάνει ένα τμήμα των δεδομένων και υπολογίζει τοπικά αλλαγές στο διάνυσμα βαρών, χωρίς όμως να τροποποιεί ακόμη το διάνυσμα βαρών ανάλογα. Αυτές οι αλλαγές συναθροίζονται και δημιουργούν ένα νέο διάνυσμα σε μια φάση Reduce. Προφανώς, απαιτούνται πολλαπλοί κύκλοι Map και Reduce μέχρι την τελική σύγκλιση, και σε κάθε κύκλο το τρέχον διάνυσμα βαρών έχει κοινοποιηθεί σε όλες τις εργασίες Map.

11.7.3 Εκμάθηση από πλησιέστερους γείτονες

Σε αυτόν τον τρόπο «εκμάθησης» δεν χτίζεται αναγκαστικά κάποιο μοντέλο. Αντιθέτως, ολόκληρο το σύνολο δεδομένων αποθηκεύεται έπειτα, ενδεχομένως, από κάποια προεπεξεργασία ή δεικτοδότηση/ευρετηρίαση. Στη συνέχεια, για κάθε εγγραφή για την οποία δεν γνωρίζουμε την κλάση, θεωρώντας την ως ένα σημείο στον χώρο μας, βρίσκουμε 1 ή περισσότερους πλησιέστερους γείτονες βάσει μίας συνάρτησης απόστασης (βλ. Σχήμα 11.16). Η εκτιμώμενη κλάση υπολογίζεται συνδυάζοντας τις ετικέτες από τους πλησιέστερους γείτονες. Αυτό μπορεί να πραγματοποιηθεί με διάφορους τρόπους, όπως ζυγισμένη ψηφοφορία, όμως σε κάθε περίπτωση είναι σχετικά εύκολα εξηγήσιμη.

Γενικά, η δισδιάστατη ή ολιγοδιάστατη περίπτωση είναι σχετικά εύκολη, ακόμη και αν τα αποθηκευμένα δεδομένα είναι πάρα πολλά. Αυτό οφείλεται στο ότι έχουν αναπτυχθεί πολλές κλιμακώσιμες δομές δεδομένων για την εύρεση πλησιέστερων γειτόνων, όταν αυξάνεται ο αριθμός των διαστάσεων και το σύνολο εκπαίδευσης είναι μεγάλο, όπως τα kd-δένδρα, τα R-δένδρα και τα τετραδικά δένδρα. Αυτές οι δομές, όμως, είναι αποτελεσματικές όταν ο αριθμός των διαστάσεων είναι σχετικά μικρός. Δυστυχώς, για δεδομένα υψηλών διαστάσεων, υπάρχουν λίγα που μπορούν να γίνουν ώστε να αποφευχθεί η αναζήτηση μεγάλου ή ακόμη και του μεγαλύτερου μέρους των δεδομένων, που σημαίνει ότι δεν έχει νόημα η χρήση ευρετηρίων. Αυτό το γεγονός είναι μία ακόμη ένδειξη της «κατάρας της διαστασιμότητας». Δύο τρόποι αντιμετώπισης της «κατάρας» είναι οι εξής:

1. *Αρχεία VA* (Vector-Approximation). Δεδομένου ότι πρέπει να εξετάσουμε ένα μεγάλο μέρος των δεδομένων ούτως ή άλλως για να βρούμε τους πλησιέστερους γείτονες ενός σημείου ερωτήματος q , αναπτύσσουμε μια μεθοδολογία δύο βημάτων. Πρώτον, δημιουργείται μια περίληψη του αρχείου δεδομένων χρησιμοποιώντας μόνο έναν μικρό αριθμό bits, που προσεγγίζουν τις τιμές του κάθε στοιχείου κάθε διανύσματος εκπαί-

δευσης. Για παράδειγμα, εάν χρησιμοποιούμε μόνο το $1/4$ των bits υψηλής τάξης των αριθμητικών στοιχείων, τότε μπορούμε να δημιουργήσουμε ένα αρχείο μεγέθους $1/4$ του μεγέθους του πλήρους συνόλου δεδομένων. Ωστόσο, σαρώνοντας αυτό το αρχείο μπορούμε να δημιουργήσουμε μια λίστα υποψήφιων που *μπορεί* να είναι μεταξύ των k πλησιέστερων γειτόνων του ερωτήματος q και αυτή η λίστα μπορεί να είναι ένα μικρό κλάσμα ολόκληρου του συνόλου δεδομένων. Στη συνέχεια, ψάχνουμε μόνο αυτούς τους υποψήφιους.

2. *Μείωση διαστάσεων.* Μπορούμε να χειριστούμε τα διανύσματα του συνόλου εκπαίδευσης ως πίνακα, όπου οι γραμμές είναι τα διανύσματα του παραδείγματος εκπαίδευσης, ενώ οι στήλες αντιστοιχούν στις διαστάσεις αυτών των διανυσμάτων. Εφαρμόζουμε μία από τις τεχνικές μείωσης διαστάσεων για τη συμπίεση των διανυσμάτων σε μικρό αριθμό διαστάσεων, που πρέπει να είναι αρκετά μικρός ώστε να μπορούν να χρησιμοποιηθούν οι πολυδιάστατοι κατάλογοι.

Τέλος, έχουμε επίσης την επιλογή να χρησιμοποιήσουμε την τεχνική LSH, που εξετάστηκε προηγουμένως. Υπενθυμίζεται ότι αυτή η τεχνική είναι προσεγγιστική και θα μπορούσε να παραγάγει ψευδώς αρνητικά αποτελέσματα, δηλαδή να υπάρχουν παραδείγματα εκπαίδευσης που είναι πλησιέστεροι γείτονες σε ένα ερώτημα, αλλά τελικά δεν εμφανίζονται σε μια αναζήτηση.

11.7.4 Δένδρα αποφάσεων

Ένα δένδρο αποφάσεων χρησιμοποιεί μια δενδρική δομή για να αναπαραστήσει το πώς οι τιμές των ιδιοτήτων ενός διανύσματος χαρακτηριστικών εισόδου, που δεν είναι αναγκαστικά αριθμητικές τιμές, χρησιμοποιούνται για να παραγάγουν την κλάση στην οποία ανήκει αυτή η είσοδος. Σε ένα δένδρο αποφάσεων, κάθε εσωτερικός κόμβος αντιπροσωπεύει έναν έλεγχο στην είσοδο. Τα παιδιά του είναι είτε περαιτέρω έλεγχοι ή φύλλα, που είναι η κλάση που δίνεται ως έξοδος. Τα παιδιά ενός κόμβου ελέγχου λαμβάνουν μία ετικέτα αναλόγως με το αποτέλεσμα του ελέγχου. Συνήθως, υπάρχουν μόνο δύο αποτελέσματα - αληθές ή ψευδές (ναι ή όχι) - αλλά θα μπορούσε να υπάρξει οποιοσδήποτε αριθμός αποτελεσμάτων ενός ελέγχου, π.χ. εάν το χρώμα ενός αυτοκινήτου είναι κόκκινο, άσπρο, σκούρο ή κάτι άλλο. Είναι συνηθισμένο να χρησιμοποιούμε *δάση αποφάσεων* με πολλά δένδρα που ψηφίζουν για την κλάση στην οποία ανήκει ένα δεδομένο διάνυσμα εισόδου.

Γενικά, η κατασκευή δένδρων αποφάσεων είναι υπολογιστικά πολύ ακριβή, καθώς θα πρέπει σε κάθε κόμβο του δένδρου να εξεταστούν όλα τα χαρακτηριστικά και όλοι οι σχετικοί έλεγχοι στις τιμές του συγκεκριμένου χαρακτηριστικού, ώστε μετά να επιλεγθεί το χαρακτηριστικό με τον αντίστοιχο έλεγχο που οδηγεί στον καλύτερο διαχωρισμό των δεδομένων εκπαίδευσης. Εάν το χαρακτηριστικό είναι αριθμητικό, τότε πρέπει να ταξινομήσουμε όλα τα παραδείγματα εκπαίδευσης που αντιστοιχούν στον κόμβο. Εάν το χαρακτηριστικό είναι κατηγορικό, πρώτα πρέπει να ομαδοποιήσουμε τα παραδείγματα εκπαίδευσης με βάση την τιμή του χαρακτηριστικού. Επιπλέον, εάν υπάρχουν πραγματικά περισσότερες από δύο κλάσεις, τότε, στα δυαδικά δένδρα πρέπει να εξετάσουμε όλους τους τρόπους χωρισμού των κλάσεων σε δύο ομάδες. Ευτυχώς, όλα τα παραπάνω μπορούν να ωφεληθούν εύκολα από παραλληλισμό.

- Σε έναν κόμβο μπορούμε να βρούμε παράλληλα τους καλύτερους διαχωρισμούς για όλα τα χαρακτηριστικά.
- Όλοι οι κόμβοι σε ένα επίπεδο μπορούν να εξεταστούν και να κατασκευαστούν παράλληλα. Κάθε εγγραφή εκπαίδευσης καταλήγει το πολύ σε έναν κόμβο σε οποιοδήποτε επίπεδο. Άρα, αναμένουμε η συνολική εργασία σε κάθε επίπεδο να είναι περίπου η ίδια, αντί να αυξάνεται με τον αριθμό των κόμβων.
- Η ομαδοποίηση των παραδειγμάτων εκπαίδευσης αναλόγως με την τιμή ενός χαρακτηριστικού μπορεί να γίνει αποτελεσματικά με παραλληλισμό. Το πλαίσιο MapReduce είναι εγγενώς πολύ αποτελεσματικό σε τέτοιες ομαδοποιήσεις.
- Ο παραλληλισμός μπορεί να επιταχύνει σημαντικά την ταξινόμηση, η οποία σε τεχνικές όπως οι SLIQ και SPRINT μπορεί να γίνει μία φορά για κάθε χαρακτηριστικό. Σημειώνουμε πως αυτές οι τεχνικές, επίσης, χτίζουν όλους τους κόμβους του δένδρου σε ένα επίπεδο ταυτόχρονα.

11.8 Ανάλυση μεγάλων γράφων κοινωνικών δικτύων

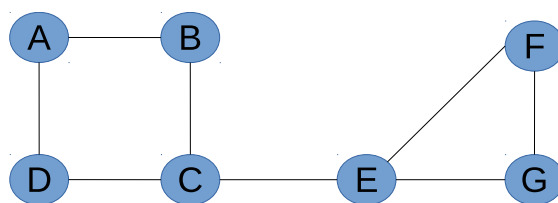
Τα κοινωνικά δίκτυα αναπαρίστανται ως γράφοι και η ανάλυσή τους, πέρα από από τη χρήση κλασικών αλγορίθμων για γράφους, απαιτεί την ανάπτυξη ιδιαίτερων τεχνικών. Π.χ., ένα σημαντικό πρόβλημα σε γράφους που αναπαριστούν κοινωνικά δίκτυα είναι η εύρεση κοινοτήτων, ένα πρόβλημα που είναι συγγενές με το πρόβλημα τμηματοποίησης ή ομαδοποίησης κορυφών, αλλά όχι ακριβώς το ίδιο, καθώς οι κοινότητες περιέχουν συνήθως μερικούς επικαλυπτόμενα σύνολα κορυφών.

Βασικά χαρακτηριστικά ενός κοινωνικού δικτύου είναι τα παρακάτω:

1. Υπάρχει μια συλλογή οντοτήτων που συμμετέχουν στο δίκτυο, π.χ. άνθρωποι ή λογαριασμοί.
2. Υπάρχει τουλάχιστον ένας τύπος σχέσης ανάμεσα στις οντότητες, π.χ. η φιλία.
3. Οι οντότητες τείνουν να ομαδοποιούνται και, εάν μια οντότητα A σχετίζεται και με την B και τη C , τότε υπάρχει μεγαλύτερη πιθανότητα από τον μέσο όρο να σχετίζεται επίσης η B με τη C .

Τα κοινωνικά δίκτυα καλύπτουν και άλλες περιπτώσεις, πέρα από ανθρώπους/λογαριασμούς και φιλίες. Μπορεί οι οντότητες να είναι αριθμοί τηλεφώνου και η σύνδεση να υποδηλώνει επικοινωνία μεταξύ τους. Ή μπορεί οι οντότητες να αντιστοιχούν σε επαγγελματίες και η σχέση να δείχνει συγκεκριμένο τρόπο συνεργασίας, όπως η συν-συγγραφή επιστημονικών άρθρων ανάμεσα σε ερευνητές.

Στη συγκεκριμένη ενότητα, θα συζητηθούν σύντομα κάποιοι αλγόριθμοι για την επεξεργασία μεγάλων γράφων κοινωνικών δικτύων.



Σχήμα 11.17: Παράδειγμα γράφου με 2 κοινότητες $\{A, B, C, D\}$ και $\{E, F, G\}$.

11.8.1 Ενδιαμεσότητα (betweenness)

Οι συνηθισμένες τεχνικές ομαδοποίησης, όπως οι ιεραρχικοί αλγόριθμοι (π.χ. ιεραρχικός συσσωρευτικός αλγόριθμος μονής σύνδεσης), αλγόριθμοι τμηματοποίησης (π.χ. k -means) ή βασιζόμενοι στην πυκνότητα (π.χ. DBSCAN), παρουσιάζουν προβλήματα όταν εφαρμόζονται σε κοινωνικά δίκτυα για την εύρεση κοινοτήτων. Γι' αυτό, έχουν αναπτυχθεί μερικές εξειδικευμένες τεχνικές ομαδοποίησης για να βρουν κοινότητες στα κοινωνικά δίκτυα. Μία από τις απλούστερες βασίζεται στην έννοια της *ενδιαμεσότητας* (betweenness) μίας ακμής (a, b) .

Η *ενδιαμεσότητα* της ακμής (a, b) ορίζεται ως το πλήθος των ζευγών κόμβων x και y ώστε η ακμή (a, b) να βρίσκεται στο συντομότερο μονοπάτι μεταξύ x και y . Ακριβέστερα, επειδή μπορεί να υπάρχουν αρκετά συντομότερα μονοπάτια μεταξύ του x και του y , η ακμή (a, b) πιστώνεται με το κλάσμα των συντομότερων μονοπατιών που περιλαμβάνουν την ακμή (a, b) .

Στο Σχήμα 11.17 περιμένουμε ότι η ακμή (C, E) έχει τη μεγαλύτερη τιμή ενδιαμεσότητας. Μια μεγάλη τιμή σε μια ακμή (a, b) είναι ισχυρή ένδειξη ότι οι κορυφές a και b ανήκουν σε διαφορετικές κοινότητες. Με άλλα λόγια, οι τιμές ενδιαμεσότητας για τις ακμές ενός γράφου συμπεριφέρονται σαν ένα μέτρο απόστασης για τους κόμβους του γράφου, αλλά χωρίς να ορίζεται για ζεύγη κόμβων που δεν είναι συνδεδεμένα και χωρίς να ικανοποιεί την τριγωνική ανισότητα. Κοινότητες που αποτελούνται από μη επικαλυπτόμενα σύνολα κορυφών μπορούν να παραχθούν αφαιρώντας τις ακμές με την υψηλότερη ενδιαμεσότητα, έως ότου ο γράφος σπάσει σε έναν κατάλληλο αριθμό συνεκτικών συνιστωσών. Στο παράδειγμα του σχήματος, με την αφαίρεση της ακμής (C, E) σχηματίζονται 2 συνεκτικές συνιστώσες που αντιστοιχούν σε 2 κοινότητες.

Σε μεγάλους γράφους το πιο ακριβό υπολογιστικά κομμάτι είναι ο υπολογισμός της ενδιαμεσότητας, που σε μη κατευθυνόμενους γράφους χωρίς βάρη γίνεται με επαναλήψεις κατά πλάτος διάτρεξης, αρχίζοντας από διαφορετική κορυφή (αλγόριθμος Girvan-Newman) κάθε φορά. Οπότε, σε έναν γράφο με n κορυφές και m ακμές η χρονική πολυπλοκότητα είναι της τάξης $O(nm)$, που είναι απαγορευτικά υψηλή για μεγάλα δεδομένα. Σε τέτοια γραφήματα, η ενδιαμεσότητα προσεγγίζεται επιλέγοντας τυχαία ένα υποσύνολο των κορυφών που αποτελούν ρίζα του δένδρου που αναπαριστά την BFS διάτρεξη.

11.8.2 SimRank

Η τεχνική SimRank ποσοτικοποιεί την ομοιότητα μεταξύ των κόμβων του ιδίου τύπου χρησιμοποιώντας τυχαίους περιπάτους. Επιπλέον, μπορεί να χρησιμοποιηθεί στην εύρεση κοινοτήτων. Στη βασική τεχνική, οι υπολογισμοί πραγματοποιούνται μία φορά για κάθε κόμβο

προέλευσης, οπότε υπάρχει ένα όριο στο μέγεθος των γράφων που μπορούν να αναλυθούν πλήρως με αυτόν τον τρόπο. Στην προσεγγιστική εκδοχή, αποφεύγονται οι επαναλαμβανόμενοι πολλαπλασιασμοί πινάκων-διανυσμάτων.

Εστιάζουμε σε έναν συγκεκριμένο κόμβο-πηγή S ενός κοινωνικού δικτύου και θέλουμε να δούμε πού θα καταλήξει ο τυχαίος περιπατητής με έναν σύντομο περίπατο από τον S . Για να μάθουμε πού τείνει ο περιπατητής να είναι, αλλάζουμε τον πίνακα των πιθανοτήτων μετάβασης για να έχουμε μικρή πιθανότητα μετάβασης πίσω στον S από οποιονδήποτε κόμβο. Δηλαδή, θα υπολογίσουμε τον ευαίσθητο θεματικά PageRank με σύνολο τηλεμεταφοράς, το οποίο αποτελείται μόνο από τον κόμβο S . Τυπικά, έστω M ο μεταβατικός πίνακας του γράφου G , όπως τον είδαμε όταν συζητούσαμε την τεχνική PageRank. Δηλαδή, το στοιχείο στη γραμμή i και στη στήλη j του M είναι $1/k$, εάν ο κόμβος j του G έχει βαθμό k και ένας από τους γειτονικούς κόμβους είναι ο i . Διαφορετικά, αυτό το στοιχείο είναι 0.

Ας χρησιμοποιήσουμε το β ως την πιθανότητα ότι ο περιπατητής θα συνεχίσει τυχαία, οπότε $1 - \beta$ είναι η πιθανότητα ο περιπατητής να τηλεμεταφερθεί στον κόμβο-πηγή S . Έστω e_S το διάνυσμα στήλης που έχει 1 στη γραμμή για τον κόμβο S και 0 για τους άλλους κόμβους. Τότε, εάν $\mathbf{v}$ είναι το διάνυσμα στήλης που αντικατοπτρίζει την πιθανότητα ο περιπατητής να βρίσκεται σε κάθε κόμβο σε έναν συγκεκριμένο γύρο, και $\mathbf{v}'$ είναι η πιθανότητα ο περιπατητής να βρίσκεται σε κάθε κόμβο στον επόμενο γύρο, τότε το $\mathbf{v}'$ σχετίζεται με το $\mathbf{v}$ ως εξής:

$$\mathbf{v}' = \beta M \mathbf{v} + (1 - \beta) e_S$$

Όταν συγκλίνει ο αλγόριθμος, βάσει των τιμών στο διάνυσμα $\mathbf{v}$ μπορούμε να εντοπίσουμε τους πιο όμοιους κόμβους με τον κόμβο-πηγή S .

Ο απλός υπολογισμός του SimRank περιλαμβάνει επαναλαμβανόμενες πράξεις πολλαπλασιασμού πινάκων-διανυσμάτων, όπως και σε κάθε υπολογισμό τύπου PageRank, με σχετικά αργή σύγκλιση. Ωστόσο, όταν υπάρχει ένας μοναδικός κόμβος S στο σύνολο τηλεμεταφοράς, οι κόμβοι που βρίσκονται μακριά από τον S είναι βέβαιο ότι θα πάρουν μια πολύ μικρή τιμή SimRank. Εάν είμαστε πρόθυμοι να δεχτούμε μια προσέγγιση προς τον πραγματικό SimRank, τότε μπορούμε να χρησιμοποιήσουμε μια άλλη μέθοδο, όπου δεν βλέπουμε καν τους κόμβους που είναι μακριά από τον S . Βάσει αυτής της ιδέας, αντί ο περιπατητής να μετακινείται σε έναν τυχαίο γείτονα (εάν δεν τηλεμεταφέρεται στον κόμβο-πηγή), θα μετακινείται ή θα παραμένει εκεί όπου είναι, με την ίδια πιθανότητα. Εάν αποφασίσει να μετακινηθεί, τότε θα μετακινηθεί σε οποιονδήποτε γείτονα με ίση πιθανότητα. Ωστόσο, υπάρχει μία άλλη πτυχή της απόφασης για το εάν θα μετακινηθεί ή όχι. Μπορούμε να θεωρήσουμε τον περιπατητή είτε ως μέρος του τελικού PageRank για τον κόμβο του, είτε ως μέρος του *υπολειπόμενου* PageRank, δηλαδή του PageRank που δεν έχει ακόμη ανατεθεί σε κάποιον κόμβο. Μόνο στην τελευταία περίπτωση εξετάζουμε τη μετακίνηση του περιπατητή. Πάνω σε αυτό το σκεπτικό έχει προταθεί ο προσεγγιστικός SimRank, χωρίς να είναι στους σκοπούς αυτής της ενότητας να δώσει περισσότερες λεπτομέρειες.

Είναι ενδιαφέρον ότι ο SimRank μπορεί να χρησιμοποιηθεί για τον εντοπισμό κοινοτήτων. Ας υποθέσουμε ότι ξεκινούμε με τον κόμβο-πηγή S και βρίσκουμε τον SimRank για όλους τους κόμβους για αυτήν την πηγή. Για τον σκοπό αυτόν, θα μπορούσαμε να χρησιμοποιήσουμε είτε τον επακριβή είτε τον προσεγγιστικό αλγόριθμο SimRank. Στη συνέχεια, θα διατάξουμε τους

κόμβους με βάση την τιμή SimRank τους και θα θεωρήσουμε την προσθήκη του καθενός σε μία κοινότητα που περιέχει τον S με βάση το υψηλότερο SimRank. Πρέπει να ξέρουμε πότε να σταματήσουμε να προσθέτουμε κόμβους, επομένως χρειαζόμαστε ένα κριτήριο τερματισμού. Μια απλή προσέγγιση είναι να προσθέτουμε κόμβους μέχρις ότου η *πυκνότητα* των ακμών - το κλάσμα των ακμών που συνδέουν κόμβους μέσα στην κοινότητα διαιρούμενο με το πλήθος των πιθανών ακμών που μπορούν να συνδέουν κόμβους στην κοινότητα (δηλαδή, $\binom{n}{2}$ για μία κοινότητα με n κόμβους) - πηγαίνει κάτω από ένα όριο. Προφανώς, υπάρχουν και πολλές άλλες επιλογές. Μετά, συνεχίζουμε για έναν νέο κόμβο-πηγή που δεν έχει ανατεθεί σε καμία ή σε πολύ λίγες κοινότητες.

11.8.3 Καταμέτρηση τριγώνων

Η καταμέτρηση τριγώνων είναι σημαντική σε έναν γράφο. Εάν ο γράφος είναι ένα κοινωνικό δίκτυο με n κορυφές και m ακμές, αναμένεται το πλήθος των τριγώνων να είναι πολύ μεγαλύτερο από ό,τι σε έναν τυχαίο γράφο, όπου μπορεί να αποδειχθεί ότι το αναμενόμενο πλήθος τριγώνων είναι $\frac{4}{3}(\frac{m}{n})^3$. Ο λόγος είναι ότι, εάν οι κόμβοι A και B είναι συνδεδεμένοι και ο A είναι επίσης συνδεδεμένος με τον C , τότε θα πρέπει να υπάρχει πολύ μεγαλύτερη πιθανότητα από τον μέσο όρο ότι ο B και ο C είναι επίσης συνδεδεμένοι. Ακόμη, έχει αποδειχθεί ότι η ηλικία μιας κοινότητας σχετίζεται με την πυκνότητα των τριγώνων. Δηλαδή, όταν μια ομάδα έχει μόλις σχηματισθεί, οι άνθρωποι τραβούν τους ομοϊδεάτες φίλους τους, αλλά το πλήθος των τριγώνων είναι σχετικά μικρό. Εάν ο A φέρει τους φίλους B και C , μπορεί να είναι πιθανό ότι οι B και C δεν γνωρίζουν ο ένας τον άλλον. Καθώς η κοινότητα ωριμάζει, οι B και C μπορεί να αλληλεπιδράσουν λόγω της συμμετοχής τους στην κοινότητα. Έτσι, υπάρχει μια καλή πιθανότητα κάποια στιγμή το τρίγωνο $\{A, B, C\}$ να ολοκληρωθεί.

Είναι γνωστοί αλγόριθμοι πολυπλοκότητας $O(m^{3/2})$ που είναι ασυμπτωτικά βέλτιστοι. Όμως, σε πολύ μεγάλους γράφους θα πρέπει να χρησιμοποιηθεί παραλληλισμός. Στο πλαίσιο MapReduce, συμφέρει να αντιμετωπιστεί το πρόβλημα καταμέτρησης ζευγών ως μία έκφραση σύνδεσης, που μάλιστα μπορεί να υπολογιστεί σε έναν κύκλο MapReduce ως σύνδεση πολλών δρόμων (multi-way join). Πιο συγκεκριμένα, έστω μία σχέση $E(A, B)$ που αναπαριστά όλες τις ακμές ανάμεσα σε δύο κορυφές, οι οποίες είναι αριθμημένες από το 1 μέχρι το n , ενώ σε κάθε πλειάδα η τιμή του χαρακτηριστικού A είναι μικρότερη του B , ώστε κάθε ακμή να υπάρχει στη σχέση μία μοναδική φορά. Χρησιμοποιώντας αυτήν τη σχέση, μπορούμε να εκφράσουμε το σύνολο των τριγώνων του γράφου του οποίου οι ακμές είναι η σχέση E από τη φυσική σύνδεση:

$$E(X, Y) \bowtie E(X, Z) \bowtie E(Y, Z) \quad (11.1)$$

Παρατηρούμε ότι κάθε τρίγωνο εμφανίζεται μία φορά σε αυτήν τη σύνδεση. Το τρίγωνο που αποτελείται από τους κόμβους v_1, v_2 και v_3 δημιουργείται όταν τα X, Y και Z είναι αυτοί οι τρεις κόμβοι σε αριθμητική διάταξη, δηλαδή, $X < Y < Z$. Για παράδειγμα, εάν η αριθμητική διάταξη των κόμβων είναι $v_1 < v_2 < v_3$, τότε το X μπορεί να είναι μόνο v_1 , το Y είναι v_2 και το Z είναι v_3 .

Καθώς στη σύνδεση τα τρία στιγμιότυπα της ίδιας σχέσης έχουν προφανώς το ίδιο μέγεθος, συμφέρει να θεωρήσουμε ότι οι διαθέσιμες εργασίες Reduce είναι b^3 και κάθε χαρακτηριστικό X, Y και Z κατακερματίζεται σε b κάδους. Κάθε εργασία Reduce σχετίζεται με μία ακολουθία

τριών αριθμών κάδων (x, y, z) , όπου το κάθε ένα από τα x, y και z βρίσκεται στην περιοχή από 1 έως b . Αυτό το σκεπτικό επεκτείνει τη συζήτηση για συνδέσεις πολλαπλών δρόμων που κάναμε στο προηγούμενο κεφάλαιο.

Πιο συγκεκριμένα, κάθε εργασία Map αναλαμβάνει ένα τμήμα της E . Ας υποθέσουμε ότι σε μια εργασία Map δίνεται η πλειάδα $E(u, v)$ για αποστολή σε συγκεκριμένες εργασίες Reduce. Πρώτον, θεωρούμε το (u, v) ως πλειάδα του όρου σύνδεσης $E(X, Y)$. Μπορούμε να κατακερματίσουμε τα u και v και να πάρουμε τους αριθμούς κάδων για τα X και Y , αλλά δεν γνωρίζουμε τον κάδο όπου κατακερματίζεται το Z . Έτσι, πρέπει να στείλουμε το $E(u, v)$ σε όλες τις b εργασίες Reduce που αντιστοιχούν σε μία ακολουθία τριών αριθμών κάδων $(h(u), h(v), z)$ για κάθε έναν από τους b πιθανούς κάδους z . Αλλά, η ίδια πλειάδα $E(u, v)$ πρέπει επίσης να αντιμετωπίζεται ως πλειάδα για τον όρο $E(X, Z)$ και για τον όρο $E(Y, Z)$. Έτσι, η συνολική απαιτούμενη επικοινωνία είναι $3b$ ζεύγη κλειδιού-τιμής για κάθε μία από τις m πλειάδες της σχέσης ακμών E . Δηλαδή, το ελάχιστο κόστος επικοινωνίας είναι $O(mb)$, εάν χρησιμοποιηθούν b^3 εργασίες Reduce.

Εφόσον ο συνολικός αριθμός ακμών που κατανέμονται στις b^3 εργασίες Reduce είναι $O(mb)$, έπεται ότι κάθε εργασία λαμβάνει $O(m/b^2)$ ακμές. Εάν χρησιμοποιήσουμε τοπικά σε κάθε εργασία Reduce έναν αλγόριθμο πολυπλοκότητας $O(m^{3/2})$, ο συνολικός υπολογισμός σε κάθε εργασία είναι $O((m/b^2)^{3/2})$ ή $(m^{3/2}/b^3)$. Δεδομένου ότι υπάρχουν b^3 εργασίες Reduce, το συνολικό κόστος υπολογισμού είναι ακριβώς $O(m^{3/2})$.

Η παραπάνω τεχνική μπορεί να βελτιωθεί περαιτέρω και να απαιτεί σημαντικά λιγότερες εργασίες Reduce, αλλά είναι ενδεικτική του πώς προσεγγίζεται αποδοτικά η παραλληλοποίηση.

11.9 Συζήτηση και περαιτέρω μελέτη

Σε αυτό το κεφάλαιο παρουσιάσαμε συνοπτικά αλγοριθμικές προσεγγίσεις για προβλήματα, όπως τον υπολογισμό ομοιότητας, την επεξεργασία ροών, τις μηχανές αναζήτησης, την ομαδοποίηση, την εύρεση συχνών στοιχειοσυνόλων, την επιβλεπόμενη μάθηση, την ανάλυση γραφών, όταν τα δεδομένα μας είναι μεγάλα.

Πιο αφαιρετικά, μπορούμε να διαπιστώσουμε ότι οι βασικοί μηχανισμοί που χρησιμοποιούνται για να μπορούν να εφαρμοστούν οι αλγόριθμοι σε μεγάλα δεδομένα είναι τέσσερις.

1. Παραλληλισμός. Είναι ένα ισχυρότατο εργαλείο για την πρακτική εφαρμογή αλγορίθμων σε μεγάλα δεδομένα, και το υπολογιστικό πλαίσιο MapReduce παρέχει ένα κατάλληλο υπόβαθρο για την αποδοτική χρήση του παραλληλισμού. Πολλές φορές, όπως στον υπολογισμό του PageRank, απαιτούνται πράξεις που παραλληλίζονται εύκολα, όπως ο πολλαπλασιασμός πινάκων με διανύσματα, αλλά και που επαναλαμβάνονται σε πολλές επαναλήψεις. Χρειάζεται, όμως, ιδιαίτερη προσοχή στο πώς ακριβώς γίνεται η υλοποίηση των παράλληλων προγραμμάτων, ώστε να αποφεύγονται αχρείαστες μετακινήσεις δεδομένων μέσω δικτύου ή από τη δευτερεύουσα μνήμη στην κύρια. Επίσης, πρέπει να εξετάζονται θέματα αντιγραφής των δεδομένων σε πολλαπλούς κόμβους αναφορικά με το εάν και σε ποιον βαθμό θα επιφέρουν μετέπειτα βελτιώσεις. Επί-

σης, όπως αναφέρθηκε και στα θέματα με τα αντίληπτρα, χρειάζεται προσοχή στο πού εφαρμόζεται ο παραλληλισμός. Η ανάλυση της πολυπλοκότητας των παράλληλων προγραμμάτων, όπως συζητήθηκε στην αρχή του προηγούμενου κεφαλαίου, συνεισφέρει σημαντικά στην ορθή κατανόηση και σχεδίαση αποδοτικών παράλληλων αλγορίθμων.

2. Κατάλληλες δομές δεδομένων, όπως ο πίνακας υπογραφών για την ομοιότητα εγγράφων, ή οι δυαδικοί πίνακες για την υλοποίηση των φίλτρων Bloom, ή οι δομές για την εύρεση πλησιέστερων γειτόνων, ή τα προθεματικά δένδρα για τη συμπίκνωση μιας βάσης δεδομένων συναλλαγών. Αντιστοίχως, είδαμε και τη χρήση πινάκων κατακερματισμού και αντίστοιχων bitmaps ώστε να καταστεί πρακτική η καταμέτρηση ζευγών που αποτελούνται από συχνά στοιχεία στον αλγόριθμο a-priori. Αναφορικά με τη συσταδοποίηση, συζητήθηκε η αναπαράσταση πολύ μεγάλου μέρους του συνόλου δεδομένων με λίγα στατιστικά.
3. Προσεγγιστικότητα, η οποία μπορεί να εμφανιστεί σε πολλές μορφές, όπως είναι η μείωση των διαστάσεων, οι λιγότερες διατρέξεις σε γραφήματα κοινωνικών δικτύων, LSH, ή η επεξεργασία δειγμάτων έναντι του συνόλου των δεδομένων. Το να επεξεργάζεται κανείς δείγματα δεν αποκλείει την εφαρμογή υπολογιστικά ακριβών τεχνικών, όπως ο αλγόριθμος ιεραρχικής συσταδοποίησης. Όμως, με την κατάλληλη μεταεπεξεργασία, τα αρχικά αποτελέσματα σε δείγματα δεν είναι κατ' ανάγκη προσεγγιστικά. Για παράδειγμα, υπάρχουν τεχνικές εύρεσης συχνών στοιχειοσυνόλων ενός περάσματος που αρχικά εφαρμόζονται σε δείγμα των δεδομένων και, κατόπιν, σε ολόκληρο το σύνολο δεδομένων.
4. Περιορισμός περασμάτων στα δεδομένα, που συμπληρωματικά με τα παραπάνω εφιστά την προσοχή όχι μόνο σε θέματα πολυπλοκότητας, αλλά και περιορισμού της άνεκτησης των δεδομένων από τη δευτερεύουσα μνήμη. Επισημάναμε το σκεπτικό ειδικά στα θέματα συσταδοποίησης και εύρεσης συχνών στοιχειοσυνόλων, αλλά η αξία της θεώρησης της συγκεκριμένης οπτικής είναι ευρύτερα χρήσιμη.

Πέρα από το βιβλίο [LRU19], το υλικό αυτού του κεφαλαίου βασίζεται σε πλήθος βιβλιογραφίας. Πιο συγκεκριμένα, η συζήτηση για τον υπολογισμό της ομοιότητας χρησιμοποιεί τις ιδέες από εργασίες, όπως τις [Man94, AI08, LOZ12, Bro97, BCFM98]. Το υλικό για τους αλγορίθμους σε ροές δεδομένων βασίζεται σε τεχνικές γνωστές από πολλές δεκαετίες, π.χ. [Blo70, AMS96]. Σύγχρονα θέματα αναφορικά με ροές μεγάλων δεδομένων συζητούνται σε εργασίες, όπως στις [HSS⁺13, FCKK20, RM19]. Αναφορικά με κανόνες συσχέτισης και συχνά στοιχειοσύνολα, μπορεί ο αναγνώστης να μελετήσει και βιβλία, όπως [Agg15, ZJ14], ενώ ειδικά για τα θέματα του DBSCAN η σχετική σύγχρονη βιβλιογραφία περιλαμβάνει εργασίες, όπως [SL18, GT15]. Ο PageRank παρουσιάστηκε στην εργασία [BP98], ενώ ο HITS στην [Kle99]. Η επεξεργασία γράφων κοινωνικών δικτύων που είδαμε βασίζεται σε εργασίες, όπως στις [GN02, JW02].

BIBΛΙΟΓΡΑΦΙΑ

[Agg15] Charu C. Aggarwal. *Data Mining - The Textbook*. Springer, 2015.

- [AI08] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM*, 51(1):117–122, 2008.
- [AMS96] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 20–29. ACM, 1996.
- [BCFM98] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. Min-wise independent permutations (extended abstract). In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 327–336. ACM, 1998.
- [Blo70] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM*, 13(7):422–426, 1970.
- [BP98] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Comput. Networks*, 30(1-7):107–117, 1998.
- [Bro97] Andrei Z. Broder. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, Proceedings*, pages 21–29. IEEE, 1997.
- [FCKK20] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. A survey on the evolution of stream processing systems. *CoRR*, abs/2008.00842, 2020.
- [GN02] Michelle Girvan and Mark EJ Newman. Community structure in social and biological networks. *Proceedings of the national academy of sciences*, 99(12):7821–7826, 2002.
- [GT15] Junhao Gan and Yufei Tao. DBSCAN revisited: Mis-claim, un-fixability, and approximation. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 519–530, 2015.
- [HSS⁺13] Martin Hirzel, Robert Soulé, Scott Schneider, Bugra Gedik, and Robert Grimm. A catalog of stream processing optimizations. *ACM Comput. Surv.*, 46(4):46:1–46:34, 2013.
- [JW02] Glen Jeh and Jennifer Widom. Simrank: a measure of structural-context similarity. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, July 23-26, 2002, Edmonton, Alberta, Canada*, pages 538–543. ACM, 2002.
- [Kle99] Jon M. Kleinberg. Authoritative sources in a hyperlinked environment. *J. ACM*, 46(5):604–632, 1999.
- [LOZ12] Ping Li, Art B. Owen, and Cun-Hui Zhang. One permutation hashing. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, pages 3122–3130, 2012.

- [LRU19] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. *Mining of Massive Datasets, 3rd Ed.* Cambridge University Press, 2019.
- [Man94] Udi Manber. Finding similar files in a large file system. In *USENIX Winter 1994 Technical Conference, San Francisco, California, USA, January 17-21, 1994, Conference Proceedings*, pages 1–10. USENIX Association, 1994.
- [RM19] Henriette Röger and Ruben Mayer. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Comput. Surv.*, 52(2):36:1–36:37, 2019.
- [SL18] Hwanjun Song and Jae-Gil Lee. RP-DBSCAN: A superfast parallel DBSCAN algorithm based on random partitioning. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD*, pages 1173–1187, 2018.
- [ZJ14] Mohammed J. Zaki and Wagner Meira Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2014.

ΕΓΚΑΤΑΣΤΑΣΗ ΚΑΙ ΧΡΗΣΗ ΤΟΥ ΑΡΑΧΕ ΗΑDΟΟΡ

Περιεχόμενα

A.1	Εγκατάσταση Hadoop αυτόνομα	308
A.2	Εκτέλεση σε αυτόνομη εγκατάσταση Hadoop	309
A.3	Κατανεμημένη εγκατάσταση Hadoop	309
A.4	Εκτέλεση σε κατανεμημένη εγκατάσταση Hadoop	316
A.5	Περαιτέρω μελέτη	316
	Βιβλιογραφία	316



Στο παράρτημα αυτό, παραθέτουμε στον αναγνώστη τη διαδικασία εγκατάστασης του Hadoop χρησιμοποιώντας λειτουργικό σύστημα Ubuntu Linux. Πριν ξεκινήσουμε, θα πρέπει να έχουμε στη διάθεσή μας το συνθηματικό χρήστη, καθώς θα πρέπει να εναλλάξουμε τον υφιστάμενο χρήστη με τον υπερχρήστη root.

Ξεκινώντας, θα πρέπει να εγκατασταθεί μια έκδοση του JDK της Java, καθώς το Hadoop για να λειτουργεί χρειάζεται το Java Runtime Environment (JRE):

```
sudo apt-get install default-jdk
```

Η συγκεκριμένη εγκατάσταση απαιτείται τόσο στην αυτόνομη όσο και στην κατανεμημένη εγκατάσταση.

Εάν δεν έχουμε επιχειρήσει πιο πριν να εκτελέσουμε άλλη ενέργεια ως root, θα μας ζητηθεί το συνθηματικό του τρέχοντος χρήστη, λόγω της χρήσης της sudo. Παρατηρήστε ότι στη συνέχεια αυτό δεν θα απαιτηθεί, εκτός εάν παρέλθει σημαντικό χρονικό διάστημα από την τελευταία επίκληση της sudo. Όταν ολοκληρωθεί η εγκατάσταση, μπορούμε να ελέγξουμε ότι αυτή έχει ολοκληρωθεί επιτυχώς με τη βοήθεια της παρακάτω εντολής, η οποία και θα εμφανίσει την έκδοση της Java και του JRE που έχουν εγκατασταθεί:

```
java -version
```

Σε αντίθετη περίπτωση, θα εμφανιστεί μήνυμα σφάλματος το οποίο θα ενημερώνει ότι η συγκεκριμένη εντολή δεν βρέθηκε. Στη συνέχεια, θα πρέπει να ενημερώσουμε τις μεταβλητές περιβάλλοντος ώστε να μπορεί το Hadoop να εντοπίσει πού βρίσκεται η ιδεατή μηχανή της Java (JVM-Java Virtual Machine). Αυτό γίνεται ως εξής:

```
echo export JAVA_HOME=/usr/lib/jvm/default-java >> .bashrc
```

με αποτέλεσμα να προστίθεται η γραμμή `JAVA_HOME=/usr/lib/jvm/default-java` στο τέλος του αρχείου `.bashrc`. Με την επόμενη σύνδεση στο τερματικό, η μεταβλητή θα είναι διαθέσιμη. Για να την κάνουμε διαθέσιμη και σε αυτήν τη συνεδρία, μπορούμε να εκτελέσουμε:

```
source .bashrc
```

και να επιβεβαιώσουμε ως: `echo $JAVA_HOME`, οπότε και θα προκύψει η παραπάνω διαδρομή.

A.1 Εγκατάσταση Hadoop αυτόνομα

Στο επόμενο βήμα μας θα μεταφορτώσουμε το Hadoop ώστε να το εγκαταστήσουμε. Αυτό μπορούμε να το κάνουμε πολύ απλά ως εξής, με τη χρήση της wget:

```
wget https://downloads.apache.org/hadoop/common/hadoop2.10.0/hadoop-2.10.0.tar.gz
```

Αφού ολοκληρωθεί η λήψη, στη συνέχεια, μπορούμε να αποσυμπιέσουμε το tarball ως εξής:

```
tar xvfz hadoop-2.10.0.tar.gz
```

Και με αυτό το βήμα, ολοκληρώθηκε η εγκατάσταση του Hadoop αυτόνομα.

A.2 Εκτέλεση σε αυτόνομη εγκατάσταση Hadoop

Για να εκτελέσουμε το Hadoop αυτόνομα, θα παρουσιάσουμε ένα παράδειγμα αντίστοιχο του οποίου βρίσκεται και στην ιστοσελίδα του Hadoop, από την οποία πραγματοποιήσαμε τη λήψη του συστήματος. Θα εκτελέσουμε από τα διαθέσιμα παραδείγματα του Hadoop τη λειτουργία `grep`, η οποία αναζητεί την ύπαρξη μίας συμβολοσειράς εντός ενός ή περισσότερων αρχείων.

Αρχικά, θα δημιουργήσουμε έναν κατάλογο που θα τον ονομάσουμε `input`:

```
mkdir input
```

Στη συνέχεια, θα αντιγράψουμε στον κατάλογο αυτόν κάποια αρχεία ρυθμίσεων τα οποία συνοδεύουν το Hadoop. Αυτά τα αρχεία δεν πρόκειται να αλλάξουν τη διαμόρφωση του Hadoop, αλλά θα χρησιμοποιηθούν ως είσοδος στην εργασία την οποία και θα εκτελέσουμε:

```
cp ~/hadoop-2.10.0/etc/hadoop/*.xml input
```

Έπειτα, θα εκτελέσουμε την `grep` ως εξής:

```
~/hadoop-2.10.0/bin/hadoop jar ~/hadoop2.10.0/share/hadoop/mapreduce/  
hadoopmapreduce-examples-*.jar grep myinput myoutput 'the'
```

Η παραπάνω εντολή μπορεί να μοιάζει σύνθετη, χωρίς όμως να είναι στην πραγματικότητα. Ας την αναλύσουμε:

- `~/hadoop-2.10.0/bin/hadoop`: το εκτελέσιμο του Hadoop μαζί με τη διαδρομή στην οποία βρίσκεται.
- `jar`: πρόκειται να εκτελεστεί μία κλάση η οποία βρίσκεται συμπιεσμένη ως `jar`.
- `~/hadoop-*/share/hadoop/mapreduce/hadoopmapreduce-examples-*.jar`: η κλάση που πρόκειται να εκτελεστεί ως `jar`, μαζί με τη διαδρομή της. Το `*` έχει την έννοια ότι μπορούμε να εκτελέσουμε την παραπάνω εντολή ανεξαρτήτως έκδοσης του Hadoop.
- `grep`: η εντολή την οποία θα εκτελέσουμε
- `myinput`: πρώτη παράμετρος της `grep` — το όνομα του καταλόγου στον οποίο θα γίνει αναζήτηση.
- `myoutput`: δεύτερη παράμετρος της `grep` — το όνομα του καταλόγου στον οποίο θα αποθηκευθούν τα αποτελέσματα της `grep`.
- `'the'`: η συμβολοσειρά την οποία αναζητούμε.

A.3 Κατανεμημένη εγκατάσταση Hadoop

Για λόγους απλότητας, θα υποθέσουμε ότι έχουμε δύο κόμβους στους οποίους θα πραγματοποιηθεί η εγκατάσταση, τους οποίους θα ονομάσουμε «master» και «worker01». Στον master

θα εκτελούνται ο NameNode και ο Secondary NameNode ενώ θα έχει και τον ρόλο του DataNode. Ο worker01 θα λειτουργεί απλώς ως DataNode. Με τον ίδιο τρόπο μπορεί να πραγματοποιηθεί η εγκατάσταση και για τρεις ή περισσότερους κόμβους, δηλαδή για δύο και περισσότερους workers, χωρίς να αλλάζει κάτι σημαντικά. Απλώς, όποιες ενέργειες αφορούν τον worker01 θα πρέπει να πραγματοποιηθούν και για κάθε έναν από τους workers, για την περίπτωση ύπαρξης περισσότερων του ενός. Για να προχωρήσουμε, θα πρέπει να έχουμε στη διάθεσή μας τις διευθύνσεις IP του master και του worker01. Θα αναφερόμαστε σε αυτές από εδώ και στο εξής ως <MASTER-IP> και <WORKER01-IP>.

Hint: Εάν δεν γνωρίζουμε τις διευθύνσεις αυτές, μπορούμε να τις βρούμε από το τερματικό μας εκτελώντας:

```
ifconfig -a
```

Η διεύθυνσή μας θα παρουσιάζεται ως εξής: inet addr: xx.xx.xx.xx

Στο πρώτο βήμα, θα πρέπει να ενημερώσουμε τις μηχανές μας για τα ονόματα που έχουμε αποφασίσει να τους δώσουμε. Αυτό το βήμα θα μπορούσε να παραλειφθεί και να χρησιμοποιηθούν τα πλήρη ονόματα (FQDN) που τους έχει αποδώσει ο διακομιστής DNS, αλλά είναι πιο εύκολο να αναφερόμαστε στην κάθε μηχανή με τη χρήση ενός σύντομου αναγνωριστικού. Οι τροποποιήσεις αυτές θα πρέπει να πραγματοποιηθούν και στη master και στη worker01 μηχανή. Στο σημείο αυτό, θα πρέπει να επεξεργαστούμε το αρχείο

```
/etc/hosts
```

Θα ήταν φρόνιμο να κρατήσουμε ένα αντίγραφο πριν προβούμε σε οποιεσδήποτε αλλαγές, καθώς πρόκειται για ένα σημαντικό αρχείο συστήματος:

```
sudo cp /etc/hosts /etc/hosts.mybk
```

Τώρα, μπορούμε να επεξεργαστούμε το εν λόγω αρχείο. Στο παράδειγμα που ακολουθεί, η επεξεργασία πραγματοποιείται με τη χρήση vim, αλλά μπορεί να χρησιμοποιηθεί οποιοσδήποτε επεξεργαστής κειμένου, είτε γραμμής εντολών, είτε οπτικός:

```
sudo vim /etc/hosts
```

Θα πρέπει να εισαχθούν οι παρακάτω γραμμές στο τέλος του αρχείου, κατά προτίμηση:

```
<MASTER-IP> master
<WORKER01-IP> worker01
```

Μία άλλη ενέργεια που θα πρέπει οπωσδήποτε να πραγματοποιηθεί, είναι να τοποθετηθεί σε σχόλιο η παρακάτω γραμμή:

```
127.0.1.1 fqdn...-
```

Εάν η γραμμή αυτή δεν τοποθετηθεί σε σχόλιο, το Hadoop δεν θα μπορέσει να λειτουργήσει σωστά. Έτσι η γραμμή θα πρέπει να έχει πλέον τη μορφή:

```
#127.0.1.1 fqdn...-
```

Παράδειγμα:

```
192.168.1.10 master
192.168.1.11 worker01
```

Αποθηκεύουμε και κλείνουμε το αρχείο.

Στα επόμενα βήματα, θα εγκαταστήσουμε μία σειρά από εργαλεία συστήματος που είναι απαραίτητα για τη λειτουργία του Hadoop σε κατανεμημένη διαμόρφωση. Αρχικά, θα εγκαταστήσουμε το πακέτο ssh, το οποίο είναι απαραίτητο για την απομακρυσμένη σύνδεση υπολογιστών μέσω ασφαλούς σύνδεσης. Επίσης, παρέχει τη δυνατότητα σύνδεσης χωρίς τη χρήση ονόματος χρήστη και συνθηματικού, χρησιμοποιώντας ζεύγη δημόσιου και ιδιωτικού κλειδιού.

```
sudo apt-get install ssh
```

Οι εγκαταστάσεις αυτές θα πρέπει να πραγματοποιηθούν και στους δύο υπολογιστές, master και worker01. Στο επόμενο βήμα, θα ρυθμίσουμε τον υπολογιστή master ώστε να μπορεί να συνδέεται στον υπολογιστή worker01 χωρίς τη χρήση συνθηματικού, ώστε να ξεκινά την εκτέλεση διεργασιών απομακρυσμένα. Για να γίνει αυτό, θα πρέπει να δημιουργήσουμε ένα ζεύγος δημόσιου-ιδιωτικού κλειδιού στον υπολογιστή master και να αντιγράψουμε το δημόσιο κλειδί στον υπολογιστή worker01. Αρχικά, θα μεταφερθούμε στον αρχικό μας κατάλογο:

```
cd
```

Στη συνέχεια, θα δημιουργήσουμε το ζεύγος δημόσιου-ιδιωτικού κλειδιού. Μετά την εκτέλεση της παρακάτω εντολής, θα πατήσουμε το πλήκτρο ENTER 3 φορές, χωρίς την εισαγωγή κανενός στοιχείου:

```
ssh-keygen -t rsa
```

Έπειτα, θα αντιγράψουμε το περιεχόμενο του δημόσιου κλειδιού στη λίστα με τα επιτρεπόμενα κλειδιά και θα τροποποιήσουμε τα δικαιώματα πρόσβασης:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys  
chmod og-wx ~/.ssh/authorized_keys
```

Μπορούμε να δοκιμάσουμε τώρα ότι συνδεόμαστε στον ίδιο υπολογιστή μέσω ssh χωρίς τη χρήση password:

```
ssh localhost
```

Τώρα, θα προσπαθήσουμε να συνδεθούμε απομακρυσμένα. Ας επανεκκινήσουμε την υπηρεσία ssh πιο πριν:

```
sudo service sshd restart
```

Και ας αντιγράψουμε το δημόσιο κλειδί μας από τον master στον worker01:

```
ssh-copy-id -i $HOME/.ssh/id_rsa.pub user@worker01
```

Θα ζητήσει το συνθηματικό του χρήστη user στον υπολογιστή worker01. Το εισάγουμε. Πλέον, θα πρέπει να μπορούμε να συνδεθούμε από τον master στον worker01 χωρίς τη χρήση συνθηματικού:

```
ssh worker01
```

Αποσυνδεόμαστε με exit ή ^D.

Όντας στον master, επιβεβαιώνουμε ότι μπορεί ο master να αναγνωρίσει τον εαυτό του ως master και μπορεί να πραγματοποιηθεί σύνδεση χωρίς τη χρήση συνθηματικού.

A.3.0.1 Επεξεργασία αρχείων ρυθμίσεων στον master και στον worker01

Για να μπορέσουμε να συνεχίσουμε, θα πρέπει να παραμετροποιήσουμε κάποια αρχεία ρυθμίσεων. Τα περισσότερα από αυτά είναι σε μορφή xml. Εδώ θα παρουσιαστεί μόνο ένα υποσύνολο των αρχείων αυτών, ώστε να επιτευχθεί μια στοιχειώδης παραμετροποίηση που θα επιτρέψει τη λειτουργία του Hadoop σε κατανεμημένη διαμόρφωση δύο κόμβων. Οι παράμετροι που εμφανίζονται εντός των αρχείων αυτών δεν είναι οι μόνες. Για περισσότερες επιλογές διαμόρφωσης, ο ενδιαφερόμενος αναγνώστης μπορεί να επισκεφθεί την online τεκμηρίωση του Apache Hadoop. Για την υφιστάμενη διαμόρφωση, θεωρούμε ότι ο φάκελος με τα αρχεία του Hadoop βρίσκεται στον αρχικό κατάλογο του χρήστη user και ότι έχει όνομα hadoop. Καθώς τα αρχεία αυτά βρίσκονται μέσα στον υποκατάλογο etc/hadoop, σε αυτήν την περίπτωση η πλήρης διαδρομή από τον αρχικό φάκελο του χρήστη user θα είναι ~/hadoop/etc/hadoop/.

Θα ξεκινήσουμε με το αρχείο core-site.xml το οποίο φαίνεται παρακάτω. Στην υφιστάμενη διαμόρφωση το αρχείο περιλαμβάνει τις πληροφορίες για τον NameNode. Συγκεκριμένα, για την ιδιότητα με όνομα fs.defaultFS έχει δοθεί τιμή hdfs://master:9000. Η τιμή αυτή υποδηλώνει ότι ο NameNode εκτελείται στον κόμβο με όνομα «master» στην πόρτα 9000:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://master:9000</value>
  </property>
</configuration>
```

Κώδικας A.1: core-site.xml

Στη συνέχεια, θα μελετήσουμε το αρχείο hdfs-site.xml. Σε αυτό βρίσκεται η παραμετροποίηση για το HDFS και πληροφορίες που αφορούν τον NameNode, τον Secondary NameNode καθώς και τους DataNodes. Στα περιεχόμενα του αρχείου που παρατίθεται παρακάτω, έχουν ρυθμιστεί τρεις παράμετροι. Ας τις δούμε μία προς μία με τη σειρά:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/home/user/hdfs/data</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/home/user/hdfs/name</value>
  </property>
</configuration>
```

Κώδικας A.2: hdfs-site.xml

Η πρώτη παράμετρος με όνομα dfs.replication ρυθμίζει το πλήθος αντιγράφων κάθε block του HDFS. Η προκαθορισμένη τιμή είναι 3, αλλά, καθώς εμείς έχουμε μόνο δύο κόμβους, αυτή η τιμή δεν μπορεί εκ των πραγμάτων να επιτευχθεί. Έτσι, ορίζουμε ρητά την τιμή αυτής της

παραμέτρου σε 2, και κάθε block του HDFS θα υπάρχει στο σύστημα αρχείων σε δύο θέσεις. Άρα, και ο master και ο worker01 θα περιέχουν ακριβώς τα ίδια δεδομένα.

Η επόμενη παράμετρος που ορίζουμε με ονομασία `dfs.datanode.data.dir` ορίζει σε ποιον κατάλογο του υπολογιστή θα αποθηκεύει το HDFS τα δεδομένα. Εάν δεν οριστεί η παράμετρος αυτή, το HDFS θα αποθηκεύσει τα δεδομένα του στον φάκελο `/tmp`. Όπως είναι γνωστό όμως, τα περιεχόμενα του φακέλου αυτού διαγράφονται κατά την επανεκκίνηση, οπότε αν μας ενδιαφέρει να διατηρήσουμε τα δεδομένα που αποθηκεύουμε θα πρέπει να ορίσουμε ρητά την τοποθεσία όπου θα γίνει αυτό, όπως στο παράδειγμά μας. Στη συγκεκριμένη περίπτωση, έχει οριστεί η διαδρομή `/home/user/hdfs/data` που, όπως βλέπουμε, βρίσκεται μέσα στον αρχικό κατάλογο του χρήστη `user`. Οι κατάλογοι αυτοί φτιάχνονται από το Hadoop. Στην περίπτωση που εξετάζουμε, η παράμετρος αυτή χρειάζεται και από τον master και από τον worker01, καθώς και στις δύο μηχανές αυτές εκτελούνται `DataNodes`.

Η τελευταία παράμετρος που ρυθμίζεται στο αρχείο αυτό είναι η `dfs.namenode.name.dir`, όπου και περιγράφεται με αντίστοιχο τρόπο με πριν η τοποθεσία όπου θα αποθηκευθούν τα δεδομένα του `NameNode`. Αυτή η περίπτωση περιγράφει μία ρύθμιση που αφορά τον master, επειδή σε αυτόν τρέχει ο `NameNode`.

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Κώδικας A.3: *mapred-site.xml*

Για την εκτέλεση εργασιών `map reduce`, θα πρέπει να ορίσουμε τις παραμέτρους εκτέλεσης. Αυτό ορίζεται εντός του αρχείου `mapred-site.xml`. Στη συγκεκριμένη διαμόρφωση, η μόνη ρύθμιση που υφίσταται αφορά τον διαχειριστή πόρων. Έτσι, στην παράμετρο `mapreduce.framework.name` έχουμε ορίσει το `yarn` ως τέτοιο. Άλλες πιθανές ρυθμίσεις που μπορούν να γίνουν εντός του αρχείου αυτού αφορούν το μέγεθος της μνήμης που θα είναι διαθέσιμη κατά το `map`, κατά το `reduce`, κατά τη φάση ταξινόμησης και ανάμειξης, κ.ά.

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Κώδικας A.4: *yarn-site.xml*

Το τελευταίο xml αρχείο που παρουσιάζουμε είναι το `yarn-site.xml`, στο οποίο και εισάγουμε παραμετροποιήσεις σχετικά με τη ρύθμιση του `yarn` και τις εφαρμογές που υποστηρίζει. Εδώ υποστηρίζεται μόνο το `mapreduce_shuffle`. Αντίστοιχα, μπορεί να προστεθεί και ρύθμιση για το `Apache Spark`.

Συνεχίζουμε με μη `.xml` αρχεία. Το επόμενο αρχείο είναι το `workers` όπου περιέχονται τα ονόματα των κόμβων όπου θα ξεκινήσουν ή θα τερματιστούν οι υπηρεσίες από τον master. Σε αυτήν την περίπτωση, καθώς και ο master και ο worker01 συμμετέχουν ως `DataNodes` και

NodeManagers, περιλαμβάνονται και οι δύο.

```
master
worker01
```

Κώδικας A.5: workers

A.3.0.2 Προετοιμασία συστήματος αρχείων HDFS

Όπως και πριν τη χρήση ενός προσωπικού υπολογιστή πραγματοποιείται διαμόρφωση του δίσκου, αντίστοιχα θα πρέπει να πραγματοποιηθεί διαμόρφωση του κατανεμημένου συστήματος αρχείων hdfs στον master κόμβο:

```
~/hadoop/bin/hdfs namenode -format
```

Θα εμφανιστεί μία σειρά από διαγνωστικά μηνύματα. Προσοχή! Η διαμόρφωση θα πρέπει να γίνει μόνο μία φορά στην αρχή χρήσης του συστήματος αρχείων, αλλιώς θα καταλήξουμε σε απώλεια δεδομένων.

Έπειτα, εκκινούμε το HDFS. Και αυτήν την εντολή την εκτελούμε μόνο στον master:

```
~/hadoop/sbin/start-dfs.sh
```

Με αυτήν την εντολή, εκκινούν οι απαραίτητες διεργασίες τόσο στον master όσο και στον worker01, καθώς μέσω ssh ο πρώτος έχει τη δυνατότητα να συνδεθεί στον δεύτερο και να εκκινήσει υπηρεσίες από αυτόν. Μετά την ολοκλήρωση αυτών, μπορούμε να ελέγξουμε ότι η διαδικασία εκκίνησης ολοκληρώθηκε σωστά. Με την εντολή:

```
jps
```

την οποία εκτελούμε τόσο στον master, όσο και στον worker01, βλέπουμε τις διεργασίες java που εκτελούνται. Στον master θα πρέπει η έξοδος να είναι παρόμοια με παρακάτω:

```
13936 NameNode
14322 SecondaryNameNode
14075 DataNode
14638 Jps
```

Ενώ στον worker01 θα πρέπει να εκτελούνται οι παρακάτω διεργασίες:

```
34275 DataNode
34378 Jps
```

Αφού έχουμε διαπιστώσει ότι λειτουργούν σωστά, τόσο ο master όσο και ο worker01, θα πρέπει να δημιουργήσουμε τους καταλόγους του HDFS. Προσοχή! Το παρακάτω βήμα θα πρέπει να πραγματοποιηθεί μόνο την πρώτη φορά χρήσης του HDFS. Εκτελούμε λοιπόν τις παρακάτω εντολές:

```
~/hadoop/bin/hadoop fs -mkdir /user
~/hadoop/bin/hadoop fs -mkdir /user/user
```


Παρατηρήστε ότι δεν πρόκειται για τυπογραφικό λάθος η παραπάνω επανάληψη. Η πρώτη εντολή δημιουργεί στον ριζικό κατάλογο του HDFS έναν φάκελο `user`, μέσα στον οποίο θα δημιουργηθούν φάκελοι για κάθε χρήστη του HDFS. Έπειτα, δημιουργούμε έναν φάκελο για τον τρέχοντα χρήστη ο οποίος θα χρησιμοποιήσει το `hdfs`. Καθώς το όνομα χρήστη είναι επίσης `user`, δημιουργούμε εντός του καταλόγου `user` τον κατάλογο του χρήστη `user`.

Πλέον, έχουμε ολοκληρώσει την εγκατάσταση του HDFS. Εάν θέλουμε, μπορούμε να το τερματίσουμε με την εντολή:

```
~/hadoop/sbin/stop-dfs.sh
```

Την επόμενη φορά που θα χρειαστεί να ξεκινήσουμε το HDFS, το ξεκινάμε αντιστοίχως ως:

```
~/hadoop/sbin/start-dfs.sh
```

Θα πρέπει να τονιστεί ξανά ότι κανένα από τα παραπάνω βήματα που παρουσιάστηκαν και αφορούσαν διαμόρφωση του HDFS και δημιουργία καταλόγων δεν πραγματοποιείται και πάλι.

A.3.0.3 Προετοιμασία για εκτέλεση

Έχοντας ξεκινήσει το HDFS, μπορούμε να δοκιμάσουμε να εκτελέσουμε το ίδιο παράδειγμα που εκτελέσαμε σε αυτόνομη διαμόρφωση.

Αντίστοιχα με την προηγούμενη ενότητα, δημιουργούμε έναν φάκελο `input`, αυτήν τη φορά εντός του HDFS:

```
~/hadoop/bin/hadoop fs -mkdir /user/user/myinput
```

Τοποθετούμε (αντιγράφουμε) κάποια αρχεία εντός του HDFS, αντίστοιχα με πριν:

```
~/hadoop/bin/hadoop fs -put ~/hadoop/etc/hadoop/*.xml myinput
```

Μπορούμε να δούμε τα αρχεία που αντιγράψαμε ως εξής:

```
~/hadoop/bin/hadoop fs -ls /user/user/myinput/
```

Για να εκτελέσουμε το παράδειγμά μας, θα εκκινήσουμε και το YARN από τον master:

```
~/hadoop/sbin/start-yarn.sh
```

Όπως και με το HDFS, από τον master εκκινούνται οι διεργασίες τόσο στον ίδιο τον master όσο και στον `worker01`. Με την εκκίνηση του YARN, θα εκκινηθούν στον κόμβο master ο `ResourceManager` και ο `NodeManager`, καθώς θεωρούμε ότι και ο master εκτελεί έργο επεξεργασίας δεδομένων, και στον `worker01` ο `NodeManager`. Το αποτέλεσμα αυτών των ενεργειών, αν εισαγάγουμε και πάλι την εντολή `jps`, θα είναι να δούμε τα εξής στον master:

```
13936 NameNode
14322 SecondaryNameNode
14075 DataNode
14888 ResourceManager
15002 NodeManager
15382 Jps
```


Ενώ στον worker01 θα πρέπει να εκτελούνται οι παρακάτω διεργασίες:

```
34275 DataNode
38996 NodeManager
39002 Jps
```

A.4 Εκτέλεση σε κατανεμημένη εγκατάσταση Hadoop

Για να εκτελέσουμε μια εργασία σε κατανεμημένη διαμόρφωση, εκτελούμε τις ίδιες στην ουσία εντολές με την περίπτωση της αυτόνομης διαμόρφωσης:

```
~/hadoop/bin/hadoop jar ~/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar grep myinput myoutput 'the'
```

Τα αποτελέσματα αποθηκεύονται και πάλι στον φάκελο myoutput. Για να τα μεταφορτώσουμε τοπικά, εκτελούμε τα παρακάτω:

```
~/hadoop/bin/hadoop fs -get myoutput ~/myoutput
```

Εάν θέλουμε να τρέξουμε και πάλι το ίδιο παράδειγμα, είτε θα πρέπει να διαγράψουμε τον φάκελο εξόδου στο HDFS ως εξής:

```
~/hadoop/bin/hadoop fs -rm -r /user/user/myoutput
```

είτε, εναλλακτικά, να επιλέξουμε έναν διαφορετικό κατάλογο προορισμού, π.χ. myoutput2.

Εάν έχουμε ολοκληρώσει τις εργασίες μας, μπορούμε να τερματίσουμε τόσο το yarn όσο και το HDFS:

```
~/hadoop/sbin/stop-yarn.sh
~/hadoop/sbin/stop-dfs.sh
```

A.5 Περαιτέρω μελέτη

Για περισσότερες λεπτομέρειες, ο ενδιαφερόμενος αναγνώστης μπορεί να ανατρέξει στις σχετικές δημοσιευμένες εργασίες [SKRC10, VMD⁺13, Had], αλλά και σε μία σειρά βιβλίων [Whi12, WSV14].

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [Had] Apache Hadoop. <http://hadoop.apache.org/>. Accessed: 2021-09-30.
- [SKRC10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [VMD⁺13] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al.

- Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing*, pages 1–16, 2013.
- [Whi12] Tom White. *Hadoop: The definitive guide*. ” O’Reilly Media, Inc.”, 2012.
- [WSV14] Sameer Wadkar, Madhu Siddalingaiah, and Jason Venner. *Pro Apache Hadoop*. Springer, 2014.

ΕΥΡΕΤΗΡΙΟ

3V

Όγκος, 2
Ποικιλία, 2
Ταχύτητα, 2

4V

Εγκυρότητα, 2

5V

Αξία, 2

ACID

Απομόνωση, 87
Ατομικότητα, 87
Διάρκεια, 87
Συνέπεια, 87

Agent, Mesos, 221

aggregation, 81

AllegroGraph, 207

Alluxio Client, 30

Alluxio Job Master, 29

Alluxio Job Worker, 30

Alluxio Leading Master, 29

Alluxio Master, 29

Alluxio Standby Masters, 29

Alluxio Worker, 30

Amazon AWS-IAM, 207

Amazon CloudWatch, 207

Amazon Neptune, 207

Amazon S3, 32, 207

Apache Cassandra, 196

Apache Giraph, 198

Apache HBase, 196

Apache Spark, 210

Apache Tinkerpop, 196

ApplicationManager, 212

ApplicationMaster, 211, 213

ArangoDB, 194

Auditor, 31

Availability, 87

barrier, 58

BASE Ιδιότητες, 88

BerkeleyDB, 196

Big Data, 122

BigTable, 8

block pool, 41

Block Storage, 33

Block storage, 24

Brick, 25

BSP, 198

bucket, 33

CAP Theorem, 87

Cayley, 207

Ceph, 27

- Ceph Manager, 28
- Ceph Metadata Server, 28
- Ceph Monitor, 27
- Ceph OSD Deamon, 27
- Checkpoint, 36
- Client Mode, 231
- Cloud Storage, 33
- Cloud-controller-manager, 227
- Cluster Mode, 230
- Combiner, 59
- Consistency, 87
- container image, 211
- Container runtime, 229
- Containers, 211
- DataNode, 35, 38
- Deployment Controller, 229
- Dgraph, 207
- Dispersed Volume, 26
- Distributed Dispersed Volume, 26
- Distributed Replicated Volume, 26
- Distributed Volume, 26
- DRF, 223
- Driver, Spark, 230
- ElasticSearch, 196
- embededness, 81
- Erasur Codes, 31
- Erasur Coding, 42
- etcd, 228
- Eventual Consistency, 88
- Executor, Mesos, 222
- Executors, Spark, 231
- FIFO, χρονοπρογραμματισμός, 215
- File Storage, 33
- File storage, 24
- Framework, Mesos, 221
- Front-End Layer, 33
- GlusterFS, 25
- Google Cloud, 33
- Google File System, GFS, 7
- GQL, Graph Query Language, 114
- GraphFrames, 200
- GraphX, 200
- Gremlin, 196
- Hadoop Distributed File System, 35
- Hadoop Streaming, 69, 199
- HDFS, 35
- HDFS Federation, 37
- HDFS High Availability, 36
- High Availability, 212
- Hive, 10
- HugeGraph, 207
- Hypervisor, 211
- image, 36, 211
- Jaccard, 265, 271
- JanusGraph, 196
- Java, 123
- Journal, 36
- kube-controller-manager, 228
- kube-proxy, 229
- kube-scheduler, 228
- kubelet, 229
- Kubernetes, 210, 224
- Local Mode, 231
- Location Service, 33
- LSH - locality sensitive hashing, 266
- Mahout, 10
- Map, 52
- Mapper, 58
- MapReduce, 8, 50
- Master Εφαρμογής, 213
- Master, Mesos, 220
- Master, Spark, 230
- Mesos, 210, 220
- NameNode, 35, 36
- Namespace Volume, 41
- NodeManager, 211, 212
- NoSQL, 194
- Object Storage, 24
- Oozie, 10
- OrientDB, 207
- PageRank, 7, 200

- Partition Layer, 33
- Partition Tolerance, 87
- Partitioner, 61
- Pig, Pig Latin, 10
- Pod, 225
- Python, 123

- R, 123
- Reconstructor, 31
- Reduce, 52
- Reducer, 58
- Region, 33
- Replicated Volume, 26
- Replication
 - Full Replication, 86
 - Partial Replication, 86
- Replication Factor, 40
- replication factor, 95
- Replicator, 31
- ResourceManager, 212
- Ring, 32

- Scala, 123
- scheduler, 212
- schemaless, 79
- ScyllaDB, 196
- Secondary Sort, 61
- Shuffle and Sort, 57
- Slave, Mesos, 221
- Spark Cluster Manager, 230
- Storage Pool, 25
- Storage Stamps, 33
- Stream Layer, 33
- Swift Account Server, 31
- Swift Container Server, 31
- Swift Object Server, 31
- Swift Proxy Server, 31

- TerminusDB, 207
- TigerGraph, 207
- tiling, 254
- Timelineserver, 218
- TitanDB, 196
- Translator, 25

- Volume, 25

- WAS, 33
- Windows Azure Storage, 33
- Worker, Spark, 231
- workload, 225

- YAML, 229
- YARN, 9, 210, 211

- ZooKeeper, 10
- Zookeeper, 182

- Ακατος, 225
- Όρος, 276
- Όψη, 162
- Αγωγοί δεδομένων, 174 Ακμή, 192
- Ακρίβεια, 168
- Αλγόριθμος
 - A-priori, 283
- Ανάλυση απαιτήσεων, 78
- Ανάλυση δικτύων, 192
- Ανάλυση συναισθήματος, 164
- Ανακάλυψη κοινοτήτων, 194
- Ανακατασκευαστής, 31
- Αναμένοντες Master, 29
- Αναπαραχθείς Τόμος, 26
- Αναφορική Ακεραιότητα, 79
- Αντιγραφέας, 31
- Αποθήκευση block, 24
- Αποθήκευση αντικειμένων, 24
- Αποθήκευση αρχείων, 24
- Απόσταση
 - L_2 , 271
 - L_r , 271
 - L_∞ , 271
 - Hamming, 272
 - Jaccard, 271
 - Manhattan, 271
 - Διόρθωσης, 272
 - συνημιτόνου, 271
- Αυτοπεριγραφικά δεδομένα, 80
- Βάσεις δεδομένων γραφημάτων, 194
- Βελτιστοποίηση ερωτήματος, 162
- Γράφημα ιδιοτήτων, 104
- Γράφημα κατευθυνόμενο, 194
- Δάσος αποφάσεων, 297

- Δίκαιος, χρονοπρογραμματισμός, 215
- Δαίμονας Ceph, 27
- Δακτύλιος, 32
- Δεδομένα
 - Αδόμητα, 4
 - Δομημένα, 4
 - Ημιδομημένα, 4
- Δεξαμενή block, 41
- Δεξαμενή Αποθήκευσης, 25
- Δευτερεύουσα Ταξινόμηση, 61
- Διάνυσμα, 279
- Διάσχιση γραφήματος, 104
- Διάχυτος Τόμος, 26
- Διαδίκτυο των Πραγμάτων, 3
- Διακομιστής Αντικειμένων, 31
- Διακομιστής Λογαριασμών, 31
- Διακομιστής Μεσολάβησης, 31
- Διακομιστής Περιεκτών, 31
- Διακομιστής ιστορικού, 145
- Διαμέριση, 125
- Διαμέριση Δεδομένων
 - directory-based sharding, 85
 - hash-based sharding, key-based sharding, 85
 - range-based sharding, 85
 - sharding, 85
 - Κάθετη Διαμέριση, 85
 - Οριζόντια Διαμέριση, 85
- Διαμεριστής, 61
- Διατήρηση δεδομένων, 150
- Διαχειριστής Ceph, 28
- Διαχειριστής Εφαρμογών, 212
- Διαχειριστής Κόμβου, 212
- Διεργασία Οδήγησης, 125
- Διεργασία εργάτη, 182
- Δομημένα δεδομένα, 158
- Εικόνα, 36
- Εκτελεστής, 125
- Ελεγκτής, 31
- Ενέργεια, 144
- Ενδιαμεσότητα, 299
- Εννοιολογικό Μοντέλο, 78
- Εξισορρόπηση Φόρτου, 84
- Επίπεδο Διαμέρισης, 33
- Επίπεδο Παρουσίασης, 33
- Επίπεδο Ροής, 33
- Επικεφαλής Master, 29
- Επικλινής κάθοδος, 295
- Επιστήμη δικτύων, 192
- Επόπτης, 182
- Επόπτης Ceph, 27
- Εργασία, 125
- Ερώτημα
 - αναζήτησης, 276
- Ερώτημα Σύζευξης, 81
- Ερώτημα προσβασιμότητας, 194
- Ευρεία συσχέτιση, 143
- Ημερολόγιο Καταγραφής, 36
- Θεωρία Γραφημάτων, 192
- Ιδιοδιάνυσμα, 278
- Ιδιότητα γειτνίασης χωρίς ευρετήριο, 104
- Kube-apiserver, 228
- Κάδος, 33
- Κέντρο δεδομένων, 7, 175
- Καλάθι
 - Αγορών, 281
- Κανονική Μορφή, 79
- Κανόνες
 - Συσχέτισης, 281
- Κατάλογος
 - ανεστραμμένος, 276
- Κατακερματισμός, 85
- Κατακόρυφη κλιμάκωση, 50
- Κατανεμημένος Αναπαραχθείς Τόμος, 26
- Κατανεμημένος Διάχυτος Τόμος, 26
- Κατανεμημένος Τόμος, 26
- Κορυφή γραφήματος, 192
- Κωδικοποίηση Διαγραφής, 42
- Κόμβοι Εργασίας, 125
- Κώδικες Διαγραφής, 31
- Λογικό Μοντέλο, Σχεσιακό Μοντέλο, 78
- Λωρίδα, 269
- Μέγεθος γραφήματος, 192
- Μείξη και ταξινόμηση, 57
- Μετάδοση δεδομένων, 150
- Μετασχηματισμός, 139
 - distinct(), 142
 - filter(), 140

- flatMap(), 141
- map(), 140
- sample(), 142
- Μεταφραστής, 25
- Μετρική απόστασης, 271
- Μηχανή Διανυσμάτων Υποστήριξης, 295
- Μοντέλο Οντοτήτων/Συσχετίσεων, 78
 - Οντότητα, 78
 - Συσχέτιση, 78
- Μπουλόνι, 181
- Νήμα, 248
- Ομαδοποίηση, 194
- Ομοσπονδία HDFS, 37
- Οριζόντια κλιμάκωση, 50
- Πίνακας
 - μεταβατικός, 300
- Παράγοντας Αντιγραφής, 40
- Περιέκτης, 211, 213
- Περιθώριο, 295
- Πλίνθος, 25
- Πλειάδα, 182
- Προώθηση ετικέτας, 205
- Πυκνότητα
 - ακμών, 301
- Πυρήνας, 248
- Ροή δεδομένων, 172
- Σημείο Ελέγχου, 36
- Στενή συσχέτιση, 143
- Στοιχειοσύνολο
 - υποψήφιο, 284
- Στόμιο, 181
- Συλλογή εγγράφων, 97
- Συνάθροιση, 160
- Συναρτησιακή γλώσσα, 183
- Συνδρομητικές Υπηρεσίες Αποθήκευσης
 - Νέφους, 32
- Συνδυαστής, 59
- Συσσωρευτής, 156
- Συσσώρευση δεδομένων, 150
- Συστάδα, 125
- Σχέση, Πίνακας, 78
- Σχήμα Βάσης Δεδομένων, 79
- Τάξη γραφήματος, 192
- Τεμπέλικη αξιολόγηση, 144
- Τριγωνική ανισότητα, 271
- Τόμος, 25
- Τόμος Χώρου Ονομάτων, 41
- Υπερεπόπτης, 211
- Υπολογιστικό Νέφος, 10
 - Λογισμικό ως υπηρεσία, 11
 - Πλατφόρμα ως Υπηρεσία, 10
 - Υποδομή ως Υπηρεσία, 10
- Φίλτρο
 - Bloom, 273
- Φόρτος Εργασίας, 225
- Χρονοπρογραμματιστής, 212
- Χωρητικότητα, χρονοπρογραμματισμός, 215
- επαγόμενο υπογράφημα, 111
- κανονικοποίηση, 79
- κλειδί, 78
- κλειστότητα, 155

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [AAKO22] Joseph Bamidele Awotunde, Emmanuel Abidemi Adeniyi, Paul Oluwatoba Kolawole, and Roseline Oluwaseun Ogundokun. 8 - application of big data in covid-19 epidemic. In Utku Kose, Deepak Gupta, Victor Hugo C. de Albuquerque, and Ashish Khanna, editors, *Data Science for COVID-19*, pages 141–165. Academic Press, 2022.
- [acc] Apache accumulo. <https://accumulo.apache.org/>.
- [ACL18] Tyler Akidau, Slava Chernyak, and Reuven Lax. *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media, Inc., 1st edition, 2018.
- [ADK⁺16] Foto N. Afrati, Shlomi Dolev, Ephraim Korach, Shantanu Sharma, and Jeffrey D. Ullman. Assignment problems of different-sized inputs in mapreduce. *TKDD*, 11(2):18:1–18:35, 2016.
- [AG08] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Comput. Surv.*, 40(1), 2008.
- [Agg15] Charu C. Aggarwal. *Data Mining - The Textbook*. Springer, 2015.
- [AHV20] Ibrahim Haleem Khan Abid Haleem, Mohd. Javaid and Raju Vaishya. Significant applications of big data in covid-19 pandemic. *Indian J Orthop*, 54(4):526–528, 2020.
- [AI08] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM*, 51(1):117–122, 2008.
- [all] Alluxio Arxhitecture. <https://docs.alluxio.io/os/user/stable/en/overview/Architecture.html>. Accessed: 2021-12-24.
- [AMS96] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the Twenty-Eighth Annual ACM*

- Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 20–29. ACM, 1996.
- [Ant22] Ravindranatha Anthapu. *Graph Data Processing with Cypher: A practical guide to building graph traversal queries using the Cypher syntax on Neo4j*. Packt Publishing, 2022.
- [Arn14] Joe Arnold. *Openstack swift: Using, administering, and developing for swift object storage*. ” O’Reilly Media, Inc.”, 2014.
- [ASSU13] Foto N. Afrati, Anish Das Sarma, Semih Salihoglu, and Jeffrey D. Ullman. Upper and lower bounds on the cost of a map-reduce computation. *PVLDB*, 6(4):277–288, 2013.
- [Ata20] Sevcan Atay. Integrated transcriptome meta-analysis of pancreatic ductal adenocarcinoma and matched adjacent pancreatic tissues. *PeerJ*, 8, 10 2020.
- [AU11] Foto N. Afrati and Jeffrey D. Ullman. Optimizing multiway joins in a map-reduce environment. *IEEE Trans. Knowl. Data Eng.*, 23(9):1282–1298, 2011.
- [AU13] Foto N. Afrati and Jeffrey D. Ullman. Matching bounds for the all-pairs mapreduce problem. In *IDEAS*, pages 3–4, 2013.
- [AW10] Charu C. Aggarwal and Haixun Wang. *Managing and Mining Graph Data*. Springer Publishing Company, Incorporated, 1st edition, 2010.
- [azu] Azure Storage. <https://docs.microsoft.com/en-us/azure/storage/common/storage-introduction>. Accessed: 2021-12-18.
- [Bad22] D.A. Bader. *Massive Graph Analytics*. Chapman & Hall/CRC Data Science Series. CRC Press, 2022.
- [BBC19] Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow. *MongoDB: the definitive guide: powerful and scalable data storage*. O’Reilly Media, 2019.
- [BBP12] Eric B Boyer, Matthew C Broomfield, and Terrell A Perrotti. Glusterfs one storage server to rule them all. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2012.
- [BBS22] Endrit Mëziu Blend Berisha and Isak Shabani. Big data analytics in cloud computing: an overview. *Journal of Cloud Computing*, 11(24), 2022.
- [BC20] Amitabha Bhattacharyya and Durgapada Chakravarty. (graph database: A survey). In *2020 International Conference on Computer, Electrical and Communication Engineering (ICCECE)*, pages 1–8, 2020.
- [BCD16] Rajkumar Buyya, R.N. Calheiros, and A.V. Dastjerdi. *Big Data: Principles and Paradigms*. 06 2016.
- [BCFM98] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. Min-wise independent permutations (extended abstract). In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 327–336. ACM, 1998.

- [Bel22] Christos Bellas. *Advanced Joins on GPUs*. PhD Thesis, 2022.
- [Ber] Oracle berkeley db. <https://www.oracle.com/in/database/technologies/related/berkeleydb.html>.
- [BG21] Christos Bellas and Anastasios Gounaris. Hyset: A hybrid framework for exact set similarity join using a GPU. *Parallel Comput.*, 104-105:102790, 2021.
- [BG22] Christos Bellas and Anastasios Gounaris. Exploiting gpus for fast intersection of large sets. *Inf. Syst.*, 108:101992, 2022.
- [BGHP18] Albert Bifet, Ricard Gavaldà, Geoffrey Holmes, and Bernhard Pfahringer. *Machine Learning for Data Streams with Practical Examples in MOA*. MIT Press, Cambridge, MA, 2018.
- [BHPG11] Albert Bifet, Geoff Holmes, Bernhard Pfahringer, and Ricard Gavaldà. Detecting sentiment change in twitter streaming data. In Tom Diethe, Jose Balcazar, John Shawe-Taylor, and Cristina Tirnauca, editors, *Proceedings of the Second Workshop on Applications of Pattern Analysis*, volume 17 of *Proceedings of Machine Learning Research*, pages 5–11, CIEM, Castro Urdiales, Spain, 19–21 Oct 2011. PMLR.
- [Big] Bigdawg polystore. <https://bigdawg.mit.edu/>.
- [Blo] Neo4j bloom. <https://neo4j.com/product/bloom/>.
- [Blo70] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM*, 13(7):422–426, 1970.
- [BP98] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Comput. Networks*, 30(1-7):107–117, 1998.
- [BP16] A.L. Barabási and M.Á. Pá3sfai. *Network Science*. Cambridge University Press, 2016.
- [BP20] D. Bechberger and J. Perryman. *Graph Databases in Action*. In Action. Manning Publications, 2020.
- [Bre12] Eric Brewer. Cap twelve years later: How the "rules" have changed. *Computer*, 45(2):23–29, 2012.
- [Bro97] Andrei Z. Broder. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, Proceedings*, pages 21–29. IEEE, 1997.
- [Bro15] Mat Brown. *Learning Apache Cassandra*. Packt Publishing Ltd, 2015.
- [BS10] Peter Bakkum and Kevin Skadron. Accelerating sql database operations on a gpu with cuda. In *GPGPU*, pages 94–103, 2010.
- [CAM⁺19] Prabhu Csr, Sreevallabh Aneesh, Aditya Mogadala, Rohit Ghosh, and Jenila Livingston. *Big Data Analytics: Systems, Algorithms, Applications*. 01 2019.

- [Car22] Carmen Carrión. Kubernetes scheduling: Taxonomy, ongoing issues and challenges. *ACM Computing Surveys*, 55(7):1–37, 2022.
- [CDG⁺06] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 205–218, 2006.
- [Cep] Ceph Architecture. <https://docs.ceph.com/en/pacific/architecture/>. Accessed: 2021-12-15.
- [Ces23] Eugenio Cesario. Big data analytics and smart cities: applications, challenges, and opportunities. *Frontiers in Big Data*, 6, 2023.
- [CH06] Diane J. Cook and Lawrence B. Holder. *Mining Graph Data*. John Wiley & Sons, 2006.
- [CH22] Jeff Carpenter and Eben Hewitt. *Cassandra: The Definitive Guide,(Revised)*. ” O’Reilly Media, Inc.”, 2022.
- [cin] OpenStack Cinder. <https://docs.openstack.org/cinder/latest/>. Accessed: 2021-12-18.
- [CM09] Ben Collingsworth and Ronaldo Menezes. *Identification of Social Tension in Organizational Networks*, pages 209–223. 04 2009.
- [Coma] Mongo compass. <https://www.mongodb.com/products/tools/compass>.
- [Comb] European Parliament Committes. Big data. europarl.europa.eu/committees/en/big-data/product-details/20161026CDT00981.
- [Cou13] National Research Council. *Frontiers in Massive Data Analysis*. The National Academies Press, Washington, DC, 2013.
- [CWO⁺11] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, et al. Windows azure storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 143–157, 2011.
- [Cyp] Neo4j cypher query language. <https://neo4j.com/docs/getting-started/cypher-intro/>.
- [CZ18] Bill Chambers and Matei Zaharia. *Spark: The definitive guide: Big data processing made simple*. O’Reilly Media, Inc., 2018.
- [DES⁺15] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. The bigdawg polystore system. *SIGMOD Rec.*, 44(2):11–16, aug 2015.

- [DG04a] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *OSDI'04: Sixth Symposium on Operating System Design and Implementation*, pages 137–150, San Francisco, CA, 2004.
- [DG04b] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *OSDI'04: Sixth Symposium on Operating System Design and Implementation*, pages 137–150, San Francisco, CA, 2004.
- [DO13] Alex Davies and Alessandro Orsaria. Scale out with glusterfs. *Linux Journal*, 2013(235):1, 2013.
- [DQR12] Jens Dittrich and Jorge-Arnulfo Quian  -Ruiz. Efficient big data processing in hadoop mapreduce. *Proceedings of the VLDB Endowment*, 5(12):2014–2015, 2012.
- [ela] Elasticsearch. <https://www.elastic.co/>.
- [ENEN15] R. Elmasri, S.B. Navathe, R. Elmasri, and S.B. Navathe. Fundamentals of database systems. In *Advances in Databases and Information Systems*, page 139. Springer, 2015.
- [FCKK20] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. A survey on the evolution of stream processing systems. *CoRR*, abs/2008.00842, 2020.
- [Fit04] Brad Fitzpatrick. Distributed caching with memcached. *Linux journal*, 2004(124):5, 2004.
- [FXW⁺21] Sofoklis Floratos, Mengbai Xiao, Hao Wang, Chengxin Guo, Yuan Yuan, Rubao Lee, and Xiaodong Zhang. Nestgpu: Nested query processing on GPU. In *37th IEEE International Conference on Data Engineering, ICDE 2021*, pages 1008–1019. IEEE, 2021.
- [Gar13] Nishant Garg. *Apache Kafka*. Packt Publishing, 2013.
- [GDS] Neo4j graph data science library. <https://neo4j.com/docs/graph-data-science/current/>.
- [Geo11] Lars George. *HBase: the definitive guide: random access to your planet-size data*. ” O’Reilly Media, Inc.”, 2011.
- [GGL03] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, pages 20–43, Bolton Landing, NY, 2003.
- [GGR18] Minos Garofalakis, Johannes Gehrke, and Rajeev Rastogi. *Data Stream Management: Processing High-Speed Data Streams*. Springer Publishing Company, Incorporated, 2018.
- [GK17] Purva Grover and Arpan Kumar Kar. Big Data Analytics: A Review on Theoretical Contributions and Tools Used in Literature. *Global Journal of Flexible Systems Management*, 18(3):203–229, September 2017.

- [GL02] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, 2002.
- [Glu] GlusterFS. <https://docs.gluster.org/en/latest/Administrator-Guide/>. Accessed: 2021-11-30.
- [GN02] Michelle Girvan and Mark EJ Newman. Community structure in social and biological networks. *Proceedings of the national academy of sciences*, 99(12):7821–7826, 2002.
- [Goe15] Ankur Goel. *Neo4j cookbook*. Packt Publishing Ltd, 2015.
- [goo] Google Cloud Storage. <https://cloud.google.com/products/storage>. Accessed: 2021-12-18.
- [GP14] Trey Grainger and Timothy Potter. *Solr in action*. Manning Publications Co., 2014.
- [GQL] Graph query language, gql. <https://www.gqlstandards.org/>.
- [Gre16] Samuel Greengard. Gpus reshape computing. *Commun. ACM*, 59(9):14–16, 2016.
- [GT15a] Junhao Gan and Yufei Tao. DBSCAN revisited: Mis-claim, un-fixability, and approximation. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 519–530, 2015.
- [GT15b] Clinton Gormley and Zachary Tong. *Elasticsearch: the definitive guide: a distributed real-time search and analytics engine*. ” O’Reilly Media, Inc.”, 2015.
- [GZ10] Lukasz Golab and M. Tamer Zsu. *Data Stream Management*. Morgan & Claypool Publishers, 2010.
- [GZH⁺11] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*, 2011.
- [Hada] Apache Hadoop. <http://hadoop.apache.org/>. Accessed: 2021-09-30.
- [hadb] Apache hadoop. <https://hadoop.apache.org/>. Accessed: 2023-09-30.
- [Har15] Guy Harrison. *Next Generation Databases: NoSQLand Big Data*. Apress, 2015.
- [HDFa] HDFS Commands. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html>. Accessed: 2021-11-30.
- [HDFb] HDFS High Availability. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSHighAvailabilityWithNFS.html>. Accessed: 2021-11-30.
- [hiv] Apache hive. <http://hive.apache.org/>.
- [HKJR10] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. {ZooKeeper}: Wait-free coordination for internet-scale systems. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*, 2010.

- [HKZ⁺11] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for {Fine-Grained} resource sharing in the data center. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, 2011.
- [HLY⁺09] Bingsheng He, Mian Lu, Ke Yang, Rui Fang, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. Relational query coprocessing on graphics processors. *ACM Trans. Database Syst.*, 34(4), 2009.
- [HNL⁺19] Donghyoung Han, Yoon-Min Nam, Jihye Lee, Kyongseok Park, Hyunwoo Kim, and Min-Soo Kim. Distme: A fast and elastic distributed matrix computation engine using gpus. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD*, pages 759–774, 2019.
- [HSS⁺13] Martin Hirzel, Robert Soulé, Scott Schneider, Bugra Gedik, and Robert Grimm. A catalog of stream processing optimizations. *ACM Comput. Surv.*, 46(4):46:1–46:34, 2013.
- [HYF⁺08] Bingsheng He, Ke Yang, Rui Fang, Mian Lu, Naga K. Govindaraju, Qiong Luo, and Pedro V. Sander. Relational joins on graphics processors. In *SIGMOD Conference*, pages 511–524, 2008.
- [Ign16] Roger Ignazio. *Mesos in action*. Simon and Schuster, 2016.
- [IHB⁺12] Mohammad Islam, Angelo K. Huang, Mohamed Battisha, Michelle Chiang, Santhosh Srinivasan, Craig Peters, Andreas Neumann, and Alejandro Abdelnur. Oozie: towards a scalable workflow management system for hadoop. In *Proceedings of the 1st ACM SIGMOD Workshop on Scalable Workflow Execution Engines and Technologies*, pages 1–10, 2012.
- [ILDS16] Anand Padmanabha Iyer, Li Erran Li, Tathagata Das, and Ion Stoica. Time-evolving graph processing at scale. In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems, GRADES '16*, New York, NY, USA, 2016. Association for Computing Machinery.
- [JHSS21] Mohd Javaid, Abid Haleem, Ravi Pratap Singh, and Rajiv Suman. Significant applications of big data in industry 4.0. *Journal of Industrial Integration and Management*, 6(04):429–447, 2021.
- [JW02] Glen Jeh and Jennifer Widom. Simrank: a measure of structural-context similarity. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, July 23-26, 2002, Edmonton, Alberta, Canada*, pages 538–543. ACM, 2002.
- [Kak15] Dharmesh Kakadia. *Apache Mesos Essentials*. Packt Publishing, 2015.
- [KDK⁺11] Stephen W. Keckler, William J. Dally, Brucek Khailany, Michael Garland, and David Glasco. Gpus and the future of parallel computing. *IEEE Micro*, 31(5):7–17, 2011.
- [KH17] David Blair Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors - A Hands-on Approach, 3rd Ed.* Morgan Kaufmann, 2017.

- [Kle99] Jon M. Kleinberg. Authoritative sources in a hyperlinked environment. *J. ACM*, 46(5):604–632, 1999.
- [KLY21] Eunsook Kim, Kyungwoon Lee, and Chuck Yoo. On the resource management of kubernetes. In *2021 International Conference on Information Networking (ICOIN)*, pages 154–158. IEEE, 2021.
- [KNG14] Ioannis K. Koumarelas, Athanasios Naskos, and Anastasios Gounaris. Binary theta-joins using mapreduce: Efficiency analysis and improvements. In *Proceedings of the Workshops of the EDBT/ICDT 2014 Joint Conference (EDBT/ICDT 2014), Athens, Greece, March 28, 2014*, pages 6–9, 2014.
- [KRG20] Pradyumna Kaushik, Srinidhi Raghavendra, Madhusudhan Govindaraju, and Devesh Tiwari. Exploring the potential of using power as a first class parameter for resource allocation in apache mesos managed clouds. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, pages 216–226. IEEE, 2020.
- [KTC14] Gang-Hoon Kim, Silvana Trimi, and Ji-Hyong Chung. Big-data applications in the government sector. *Communications of the ACM*, 57(3):78–85, 2014.
- [kub] Kubernetes concepts. <https://kubernetes.io/docs/concepts/>. Accessed: 2023-09-30.
- [Kum20] Abhishek Kumar. *Designing Event-Driven Systems with Apache Kafka*. O’Reilly Media, 2020.
- [KW17] Holden Karau and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. O’Reilly Media, Inc., 1st edition, 2017.
- [LBZ⁺20] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. Pump up the volume: Processing large data on gpus with fast interconnects. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference*, pages 1633–1649, 2020.
- [LD10] Jimmy Lin and Chris Dyer. Data-intensive text processing with mapreduce. *Synthesis Lectures on Human Language Technologies*, 3(1):1–177, 2010.
- [LGZ⁺14] Haoyuan Li, Ali Ghodsi, Matei Zaharia, Scott Shenker, and Ion Stoica. Tachyon: Reliable, memory speed storage for cluster computing frameworks. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 1–15, 2014.
- [Li18] Haoyuan Li. *Alluxio: A virtual distributed file system*. University of California, Berkeley, 2018.
- [LKC⁺10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, et al. Debunking the 100x gpu vs. cpu myth: an evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th annual international symposium on Computer architecture*, pages 451–460, 2010.
- [LM10] Avinash Lakshman and Prashant Malik. Cassandra: A decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, 44(2):35–40, 2010.

- [LOZ12] Ping Li, Art B. Owen, and Cun-Hui Zhang. One permutation hashing. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, pages 3122–3130, 2012.
- [LRU19] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. *Mining of Massive Datasets, 3rd Ed.* Cambridge University Press, 2019.
- [luc] Apache <https://lucene.apache.org/>.
- [man] OpenStack Manila. <https://wiki.openstack.org/wiki/Manila>. Accessed: 2021-12-18.
- [Man94] Udi Manber. Finding similar files in a large file system. In *USENIX Winter 1994 Technical Conference, San Francisco, California, USA, January 17-21, 1994, Conference Proceedings*, pages 1–10. USENIX Association, 1994.
- [Mar16] Andrey Markelov. *Certified OpenStack Administrator Study Guide*. Springer, 2016.
- [Mat17] Norm Matloff. *Programming on Parallel Machines*. 2017.
- [MD13] Travis B Murdoch and Allan S Detsky. The inevitable application of big data to health care. *Jama*, 309(13):1351–1352, 2013.
- [ME14] Arun C. Murthy and Doug Eadline. *Apache Hadoop YARN: moving beyond MapReduce and batch processing with Apache Hadoop 2*. Pearson Education, 2014.
- [mes] Apache mesos architecture. <https://mesos.apache.org/documentation/latest/architecture/>. Accessed: 2023-09-30.
- [Mey11] Mathias Meyer. Riak handbook. Technical report, Technical report, Basho, 2011.
- [MHGG10] Michael McCandless, Erik Hatcher, Otis Gospodnetić, and O Gospodnetić. *Lucene in action*, volume 2. Manning Greenwich, 2010.
- [MIBY20] Liyana Shuib Maria Ijaz Baig and Elaheh Yadegaridehkordi. Big data in education: a state of the art, limitations, and future research directions. *Int J Educ Technol High Educ*, 17, 2020.
- [MK19] Andreas Meier and Michael Kaufmann. *SQL & NoSQL databases*. Springer, 2019.
- [MMBSS20] Divya Muntimadugu, Bhavana Mohanraj, Laura Bailey, and Anjana Sriram Suparna. *Red Hat Gluster Storage 3.3 Administration Guide*. "Red Hat", 2020.
- [Mona] Mongo shell documentation. <https://www.mongodb.com/docs/mongodb-shell/>.
- [MOnb] MongoDB. <https://www.mongodb.com/>.
- [MPHH10] Peter Membrey, Eelco Plugge, DUPTim Hawkins, and Tim Hawkins. *The definitive guide to MongoDB: the noSQL database for cloud and desktop computing*. Springer, 2010.

- [Mut05] S. Muthukrishnan. Data streams: Algorithms and applications. *Foundations and Trends in Theoretical Computer Science*, 1(2):117–236, 2005.
- [MV15] Sparsh Mittal and Jeffrey S. Vetter. A survey of CPU-GPU heterogeneous computing techniques. *ACM Comput. Surv.*, 47(4):69, 2015.
- [n10] n10s, neosemantics. <https://neo4j.com/labs/neosemantics/>.
- [Nee15] Nishant Neeraj. *Mastering Apache Cassandra*. Packt Publishing Ltd, 2015.
- [NH19] Mark Needham and Amy E Hodler. *Graph algorithms: practical examples in Apache Spark and Neo4j*. O'Reilly Media, 2019.
- [NSP17] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: The Definitive Guide Real-Time Data and Stream Processing at Scale*. O'Reilly Media, Inc., 1st edition, 2017.
- [Nut] Apache nutch. <https://nutch.apache.org/>.
- [NVI] NVIDIA. Cuda toolkit documentation. <http://docs.nvidia.com/cuda>.
- [OFAD11] Sean Owen, B Ellen Friedman, Robin Anil, and Ted Dunning. *Mahout in action*. Simon and Schuster, 2011.
- [ope] OpenStack. <https://www.openstack.org/software/>. Accessed: 2021-12-18.
- [Ora19] Oracle. Multimodel database. Technical report, 2019.
- [ORS⁺08] Christopher Olston, Benjamin Reed, Utkarsh Srivastava, Ravi Kumar, and Andrew Tomkins. Pig latin: A not-so-foreign language for data processing. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, SIGMOD '08, page 1099–1110, New York, NY, USA, 2008. Association for Computing Machinery.
- [ÖV⁺99] M Tamer Özsu, Patrick Valduriez, et al. *Principles of distributed database systems*, volume 2. Springer, 1999.
- [PAHN20] Amit Phaltankar, Juned Ahsan, Michael Harrison, and Liviu Nedov. *MongoDB Fundamentals: A hands-on guide to using MongoDB and Atlas in the real world*. Packt Publishing Ltd, 2020.
- [PBMW98] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical report, Stanford Digital Library Technologies Project, 1998.
- [Pou23] Nigel Poulton. *The kubernetes book*. NIGEL POULTON LTD, 2023.
- [PS21] Vaibhav Pandey and Poonam Saini. A heuristic method towards deadline-aware energy-efficient mapreduce scheduling problem in hadoop yarn. *Cluster Computing*, 24:683–699, 2021.
- [Psa17] A. Psaltis. *Streaming Data: Understanding the real-time pipeline*. Manning, 2017.

- [R⁺09] Chhanda Ray et al. *Distributed database systems*. Pearson Education India, 2009.
- [RGG03] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.
- [RH10] Saeed K Rahimi and Frank S Haug. *Distributed database management systems: A Practical Approach*. John Wiley & Sons, 2010.
- [Ria] Riak. <https://riak.com/index.html>.
- [Riv17] Jonathan Rivers. Scidb: An array-native computational database for heterogeneous, multi-dimensional data sets. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 3206–3210, 2017.
- [RJD19] L. Rutkowski, M. Jaworski, and P. Duda. *Stream Data Mining: Algorithms and Their Probabilistic Properties*. Studies in Big Data. Springer International Publishing, 2019.
- [RLOW17] Sandy Ryza, Uri Laserson, Sean Owen, and Josh Wills. *Advanced Analytics with Spark: Patterns for Learning from Data at Scale*. O’Reilly, 2 edition, 2017.
- [RM19] Henriette Röger and Ruben Mayer. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Comput. Surv.*, 52(2):36:1–36:37, 2019.
- [RU11] Anand Rajaraman and Jeffrey David Ullman. *Mining of massive datasets*. Cambridge University Press, 2011.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. ” O’Reilly Media, Inc.”, 2015.
- [s3] Amazon S3. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html#CoreConcepts>. Accessed: 2021-12-18.
- [Sal17] Neelesh Srinivas Salian. *Spark Operations Cookbook: Solving the Practical Challenges of Spark Implementation*. O’Reilly Media, Inc., 1st edition, 2017.
- [SF13] Pramod J Sadalage and Martin Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.
- [sim] Amazon simpledb. <https://aws.amazon.com/simpledb/>.
- [Siv12] Swaminathan Sivasubramanian. Amazon dynamodb: a seamlessly scalable non-relational database service. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 729–730, 2012.
- [SKRC10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [SL18] Hwanjun Song and Jae-Gil Lee. RP-DBSCAN: A superfast parallel DBSCAN algorithm based on random partitioning. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD*, pages 1173–1187, 2018.

- [SMY20] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. A study of the fundamental performance characteristics of gpus and cpus for database analytics. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020*, pages 1617–1632, 2020.
- [sol] Apache solr. <https://solr.apache.org/>.
- [Sør11] Hans Henrik Brandenborg Sørensen. Auto-tuning dense vector and matrix-vector operations for fermi gpus. In *PPAM (1)*, pages 619–629, 2011.
- [spa] Spark standalone mode - spark documentation. <https://spark.apache.org/docs/latest/spark-standalone.html>. Accessed: 2023-09-30.
- [SR15] Dilpreet Singh and Chandan K. Reddy. A survey on platforms for big data analytics. *Journal of Big Data*, 2(1):1–20, 2015.
- [SRS⁺12] John A. Stratton, Christopher I. Rodrigues, I-Jui Sung, Li-Wen Chang, Nasser Anssari, Geng (Daniel) Liu, Wen mei W. Hwu, and Nady Obeid. Algorithm and data optimization techniques for scaling to massively threaded systems. *IEEE Computer*, 45(8):26–32, 2012.
- [Sta21] Statista. <https://www.statista.com/statistics/871513/worldwide-data-created>, 2021.
- [sub] Apache submarine. <https://submarine.apache.org/>.
- [sys] Neo4j Data Graph Platform. <https://neo4j.com/>. Accessed: 2022-02-20.
- [THA⁺15] Marcello Trovati, Richard Hill, Ashiq Anjum, Shao Ying Zhu, and Lu Liu, editors. *Big-Data Analytics and Cloud Computing: Theory, Algorithms and Applications*. Springer Cham, 2015.
- [TLO12] Stanley Tzeng, Brandon Lloyd, and John D. Owens. A gpu task-parallel model with dependency resolution. *IEEE Computer*, 45(8):34–41, 2012.
- [TZZ⁺20] Zhuo Tang, Ailing Zeng, Xuedong Zhang, Li Yang, and Kenli Li. Dynamic memory-aware scheduling in spark computing environment. *Journal of Parallel and Distributed Computing*, 141:10–22, 2020.
- [VB14] Rik Van Bruggen. *Learning Neo4j*. Packt Publishing Ltd, 2014.
- [VMD⁺13a] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing*, pages 1–16, 2013.
- [VMD⁺13b] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O’Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th Annual Symposium on Cloud Computing, SOCC ’13*, New York, NY, USA, 2013. Association for Computing Machinery.

- [WBM⁺06] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell DE Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Proceedings of the 7th symposium on Operating systems design and implementation*, pages 307–320, 2006.
- [WBMM06] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, and Carlos Maltzahn. Crush: Controlled, scalable, decentralized placement of replicated data. In *SC'06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, pages 31–31. IEEE, 2006.
- [Whi12] Tom White. *Hadoop: The definitive guide*. ” O'Reilly Media, Inc.”, 2012.
- [WLBM07] Sage A. Weil, Andrew W. Leung, Scott A. Brandt, and Carlos Maltzahn. Rados: a scalable, reliable storage service for petabyte-scale storage clusters. In *Proceedings of the 2nd international workshop on Petascale data storage: held in conjunction with Supercomputing'07*, pages 35–44, 2007.
- [WSV14] Sameer Wadkar, Madhu Siddalingaiah, and Jason Venner. *Pro Apache Hadoop*. Springer, 2014.
- [WZY⁺14] Kaibo Wang, Kai Zhang, Yuan Yuan, Siyuan Ma, Rubao Lee, Xiaoning Ding, and Xiaodong Zhang. Concurrent analytical query processing with gpus. *PVLDB*, 7(11):1011–1022, 2014.
- [yar] Apache hadoop yarn. <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>. Accessed: 2023-09-30.
- [ZC12] Guoliang Zhou and Hong Chen. Parallel cube computation on modern cpus and gpus. *The Journal of Supercomputing*, 61(3):394–417, 2012.
- [ZGJ⁺18] Jingbo Zhou, Qi Guo, H. V. Jagadish, Lubos Krcál, Siyuan Liu, Wenhao Luan, Anthony K. H. Tung, Yueji Yang, and Yuxin Zheng. A generic inverted index framework for similarity search on the GPU. In *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*, pages 893–904, 2018.
- [ZJ14] Mohammed J. Zaki and Wagner Meira Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2014.
- [ZXW⁺16] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, and et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

Το παρόν σύγγραμμα έρχεται να καλύψει ένα μεγάλο κενό που υπάρχει στην ελληνόγλωσση βιβλιογραφία σχετικά με την επεξεργασία και την ανάλυση μεγάλων δεδομένων. Πιο συγκεκριμένα, στοχεύει στην παρουσίαση και στη χρήση των πιο συχνά χρησιμοποιούμενων συστημάτων και τεχνικών για τη διαχείριση και εξόρυξη χρήσιμης γνώσης από δεδομένα που χαρακτηρίζονται από μεγάλο όγκο, ενδεχομένως γρήγορους ρυθμούς ανανέωσης και ποικιλομορφία ως προς τη δομή τους. Αναλυτικότερα, στο βιβλίο αυτό μελετάμε συγκεκριμένες μεθοδολογίες ανάπτυξης εφαρμογών, καθώς και συγκεκριμένα συστήματα. Για παράδειγμα, μελετάμε το μοντέλο MapReduce και πώς αυτό υλοποιείται στο Hadoop, στο Spark, καθώς και σε άλλα συστήματα. Επιπλέον, μελετάται αναλυτικά το οικοσύστημα του Hadoop, καθώς και η μεθοδολογία ανάπτυξης εφαρμογών σε Spark, ενώ δίνονται παραδείγματα σε γλώσσα Scala και Python. Επιπλέον, παρουσιάζονται τα συστήματα NOSQL και δίνεται έμφαση στις τέσσερις διαφορετικές κατηγορίες συστημάτων, ανάλογα με τη λειτουργικότητά τους. Στο βιβλίο μελετάμε, επίσης, τεχνικές για διαχείριση ροών δεδομένων και πώς αυτή πραγματοποιείται από τα συστήματα (π.χ., Spark, Flink, κλπ.), ενώ σε ξεχωριστό κεφάλαιο εξετάζουμε ζητήματα επεξεργασίας και ανάλυσης δεδομένων γραφημάτων. Στη συνέχεια, παρουσιάζονται τα βασικά στοιχεία της διαχείρισης των πόρων σε ένα καταναμημένο σύστημα, χρησιμοποιώντας διαφορετικά συστήματα διαχείρισης πόρων, και πώς εκτελούνται οι εφαρμογές. Επιπρόσθετα, σε ξεχωριστό κεφάλαιο μελετάμε τρόπους επεξεργασίας δεδομένων σε GPU συστήματα που επίσης χρησιμοποιούνται ευρέως για την ανάλυση δεδομένων με παράλληλο τρόπο. Στο βιβλίο δίνονται και λεπτομέρειες εγκατάστασης. Το σύγγραμμα απευθύνεται σε φοιτητές μεταπτυχιακού κύκλου σπουδών, κυρίως σε μαθήματα που σχετίζονται άμεσα με θέματα που αφορούν την επεξεργασία και την ανάλυση μεγάλων δεδομένων με χρήση μοντέρνων καταναμημένων και παράλληλων συστημάτων. Ωστόσο, τμήμα του βιβλίου θα μπορούσε να λειτουργήσει επικουρικά και σε προπτυχιακά μαθήματα με παρόμοιο αντικείμενο.

Το παρόν σύγγραμμα δημιουργήθηκε στο πλαίσιο του Έργου ΚΑΛΛΙΠΟΣ+	
Χρηματοδότης	Υπουργείο Παιδείας και Θρησκευμάτων, Προγράμματα ΠΑΕ, ΕΠΑ 2020-2025
Φορέας υλοποίησης	ΕΛΚΕ ΕΜΠ
Φορέας λειτουργίας	ΣΕΑΒ/Παράρτημα ΕΜΠ/Μονάδα Εκδόσεων
Διάρκεια 2ης Φάσης	2020-2023
Σκοπός	Η δημιουργία ακαδημαϊκών ψηφιακών συγγραμμάτων ανοικτής πρόσβασης (περισσότερων από 700) • Προπτυχιακών και μεταπτυχιακών εγχειριδίων • Μονογραφιών • Μεταφράσεων ανοικτών textbooks • Βιβλιογραφικών Οδηγών
Επιστημονικά Υπεύθυνος	Νικόλαος Μήτρου, Καθηγητής ΣΗΜΜΥ ΕΜΠ
ISBN: 978-618-228-309-7	DOI: http://dx.doi.org/10.57713/kallipos-1058

Το παρόν σύγγραμμα χρηματοδοτήθηκε από το Πρόγραμμα Δημοσίων Επενδύσεων του Υπουργείου Παιδείας